DEVOPS ENGINEERING MASTERY

ADVANCED DEVOPS ARCHITECTURE & ENTERPRISE PLATFORM ENGINEERING

Advanced Kubernetes, GitOps, Internal Developer Platforms, Platform Engineering, Multi-Cluster Operations & Production Engineering Capstone

VOLUME 16 • FINAL VOLUME • FIRST EDITION

```
nahid@technova:~$ kubectl get nodes
NAME              STATUS   ROLES         AGE   VERSION
prod-eu-01        Ready    control-plane  41d   v1.31.2
nahid@technova:~$ argocd app sync technova-platform
✓ Application technova-platform synced successfully
nahid@technova:~$ kubectl get pods -n technova-prod
```

Kubernetes • GitOps • Argo CD • Flux • Helm • Backstage • Terraform

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Advanced DevOps Architecture & Enterprise Platform Engineering

DEVOPS ENGINEERING MASTERY — Volume 16 (Final Volume)

Advanced Kubernetes, GitOps, Internal Developer Platforms, Platform Engineering, Multi-Cluster Operations ও Production Engineering Capstone

First Edition — Published 9 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS, Kubernetes 1.31+ (kind / Minikube / k3d for local labs), Argo CD 2.x, Flux 2.x, Helm 3, Backstage 1.x, Terraform 1.9+, with a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Cloud chapters name the resources that incur charges and always end with a cleanup step — never leave cloud resources running after a lab. Official documentation is the authority; no pricing or service limits are stated in this book.

How to use this book. This is the final volume of a 16-volume series. It assumes the foundations from earlier volumes — Linux, networking, Git, CI/CD, Docker, Kubernetes, Terraform, Ansible, observability, AWS, Azure, GCP, IBM Cloud, DevSecOps and SRE — and provides concise refreshers where a concept returns. Read the parts in order; each part builds on the previous one. Do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on production infrastructure: use local clusters and disposable cloud accounts, never paste a real secret into an example, and never commit a secret to Git.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional platform engineering — but the judgment itself comes from doing the labs, breaking a test environment, fixing it, and repeating. There is no shortcut for that practice. Advanced technology is not automatically better technology: this volume repeatedly asks whether a simpler architecture would serve the problem better, and teaches you to defend that answer.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **TROUBLESHOOTING** helps you fix a failure, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Infrastructure changes should first be practised in a disposable environment, never directly on production systems. The author and publisher accept no liability for data loss, service disruption or cloud charges caused by running commands without a backup, a plan review or an approval gate.

Who this book is for

A DevOps engineer who knows Linux, networking, Git, scripting, CI/CD, Docker, Kubernetes, Terraform, Ansible, observability and the major cloud platforms, has full-stack MERN development experience, and wants to design, build, operate and own enterprise-grade production platforms — advanced Kubernetes, GitOps, internal developer platforms, multi-cluster operations and enterprise architecture.

Learning outcomes

By the end of this book you will be able to: design enterprise DevOps and platform architecture and defend the trade-offs behind it; operate advanced Kubernetes platforms, including multi-cluster fleets; implement GitOps with Argo CD and Flux; build internal developer platforms with Backstage; automate cloud infrastructure as platform code; govern security and the software supply chain; measure reliability with SLIs, SLOs and error budgets; manage production incidents; control operational costs; and take ownership of production systems end to end.

সূচিপত্র — Contents

PART I — Advanced DevOps & Enterprise Architecture

Chapter 01 — Advanced DevOps Engineering and Enterprise Architecture 7

Chapter 02 — Production System Design 16

Chapter 03 — Microservices and Distributed Systems 24

Chapter 04 — API Gateway, Service Mesh and Traffic Management 32

Chapter 05 — Scalability, Resilience and Performance Architecture 41

PART II — Advanced Kubernetes & Cloud-Native Engineering

Chapter 06 — Advanced Kubernetes Internals 51

Chapter 07 — Multi-Cluster Kubernetes 66

Chapter 08 — Advanced Kubernetes Networking 77

Chapter 09 — Kubernetes Security and Governance 90

Chapter 10 — Kubernetes Storage and Stateful Workloads 103

PART III — GitOps & Platform Engineering

Chapter 11 — GitOps Fundamentals 116

Chapter 12 — Argo CD and Flux CD 126

Chapter 13 — Helm, Kustomize and Release Engineering 139

Chapter 14 — Internal Developer Platform Architecture 149

Chapter 15 — Developer Portals and Self-Service Infrastructure 158

PART IV — Advanced Automation & Enterprise Operations

Chapter 16 — Advanced CI/CD and Progressive Delivery 171

Chapter 17 — Platform as Code and Automation Architecture 180

Chapter 18 — Multi-Cloud Platform Operations 188

Chapter 19 — Advanced Observability and SRE 195

Chapter 20 — Enterprise Security and Supply Chain Governance 202

PART V — Production Architecture & Final Capstone

Chapter 21 — Enterprise Platform Architecture 210

Chapter 22 — Capacity, Cost, Reliability and DR Engineering 217

Chapter 23 — Engineering Leadership and Operational Ownership 224

Chapter 24 — Final Enterprise DevOps Platform Capstone 231

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Final Series Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (150 Terms) • GitOps & Platform References • Checklists • Templates

Guided labs (24), major projects (3) and the final series capstone are in the separate *Advanced DevOps Lab Workbook* . MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, GitOps and platform references, checklists and templates are in the separate *Enterprise Platform Quick Reference*.

PART I • DEVOPS ENGINEERING MASTERY

Advanced DevOps & Enterprise Architecture

Advanced DevOps-এর বিবর্তন, Enterprise Architecture, Production System Design, Microservices ও Distributed Systems, API Gateway ও Service Mesh, Scalability, Resilience ও Performance Architecture।

অধ্যায় ১ • PART I — Advanced DevOps & Enterprise Architecture

Chapter 1 — Advanced DevOps Engineering and Enterprise Architecture

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Advanced DevOps** বলতে ঠিক কী বোঝায় তা প্রাথমিক DevOps থেকে আলাদা করে ব্যাখ্যা করতে; **Enterprise Architecture** বা প্রতিষ্ঠান-স্তরের স্থাপত্যকে ব্যবসায়িক লক্ষ্যের সাথে যুক্ত করতে; **Platform Engineering**-এর ধারণা ও এর সীমানা নির্ধারণ করতে; **System Ownership** বা সিস্টেমের মালিকানার মডেলগুলো তুলনা করতে; স্থাপত্যের মৌলিক নীতিগুলো প্রয়োগ করে একটি বাস্তব সিদ্ধান্তের পক্ষে-বিপক্ষে যুক্তি দিতে; এবং প্রযুক্তিগত ট্রেড-অফ বা আপসের সিদ্ধান্ত লিখিতভাবে নথিভুক্ত করতে।

- DevOps-এর বিবর্তন — ম্যানুয়াল অপারেশন থেকে প্ল্যাটফর্ম ইঞ্জিনিয়ারিং পর্যন্ত — ধাপে ধাপে ব্যাখ্যা করা।
- এন্টারপ্রাইজ ইঞ্জিনিয়ারিংয়ের মডেলগুলো (কেন্দ্রীভূত, বিকেন্দ্রীভূত, ফেডারেটেড) তুলনা করা।
- Platform Engineering, SRE ও DevOps-এর ভূমিকার পার্থক্য নির্ণয় করা।
- Ownership মডেল — You Build It, You Run It থেকে শুরু করে প্ল্যাটফর্ম দলের দায়িত্ব পর্যন্ত — প্রয়োগ করা।
- একটি স্থাপত্য সিদ্ধান্তের ট্রেড-অফ বিশ্লেষণ করে লিখিত সুপারিশ দেওয়া।
- ইঞ্জিনিয়ারিং পরিপক্কতা (Maturity) পরিমাপের কাঠামো ব্যবহার করা।

১.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

একটি ছোট দলে DevOps মানে প্রায়শই একজন প্রকৌশলী যিনি কোড লেখেন, সার্ভার চালান, ডেপ্লয় করেন এবং রাতে অ্যালার্ট পেলে জেগে ওঠেন। সেই মডেল দশজন মানুষের দলে কাজ করে। কিন্তু TechNova-র মতো কোম্পানি যখন বাড়তে থাকে — পাঁচটি পণ্য দল, তিনটি পরিবেশ, শত শত মাইক্রোসার্ভিস, নিয়ন্ত্রক সংস্থার নিরীক্ষা, এবং গ্রাহক যারা এক মিনিটের ডাউনটাইমও মেনে নেয় না — তখন একই মডেল ভেঙে পড়ে। প্রতিটি দল নিজের মতো করে পাইপলাইন বানায়, নিরাপত্তা নিয়ম ভিন্ন ভিন্ন হয়, কেউ জানে না কোন সার্ভিসের মালিক কে, এবং একটি ছোট পরিবর্তন পুরো সিস্টেমকে থামিয়ে দেয়। এই পর্যায়ে DevOps আর শুধু টুলের বিষয় থাকে না; এটি স্থাপত্যের বিষয় হয়ে যায়। **Advanced DevOps Engineering** হলো সেই স্তর যেখানে আপনি একক সার্ভিস নয়, পুরো সিস্টেমের জীবনচক্র নকশা করেন — কীভাবে দলগুলো একসাথে কাজ করবে, কীভাবে সিদ্ধান্ত নেওয়া হবে, কীভাবে ব্যর্থতা সীমাবদ্ধ থাকবে, এবং কীভাবে গতি ও স্থিতিশীলতা একসাথে ধরে রাখা যাবে।

> **CONCEPT:** Enterprise Architecture মানে ভারী নথিপত্র বা অনুমোদনের স্তূপ নয়। এর মানে হলো — পুরো প্রতিষ্ঠানের প্রযুক্তিগত সিদ্ধান্তগুলো যেন পরস্পরবিরোধী না হয়, সেজন্য কিছু সচেতন, লিখিত ও পর্যালোচিত নিয়ম থাকা। ভালো এন্টারপ্রাইজ স্থাপত্য দলকে ধীর করে না; বরং দলকে বারবার একই ভুল না করে দ্রুত এগোতে সাহায্য করে।

REAL WORLD: TechNova-তে একসময় প্রতিটি পণ্য দল নিজের CI/CD পাইপলাইন বানিয়েছিল। ফলাফল ছিল সাত রকম পাইপলাইন, চার রকম সিক্রেট ব্যবস্থাপনা, এবং একটি নিরাপত্তা ঘটনা — একটি দলের পাইপলাইনে পুরোনো একটি লাইব্রেরি ব্যবহার হচ্ছিল যা অন্য দল মাসখানেক আগেই বাতিল করেছিল। কেউ দোষী ছিল না; কাঠামোটিই ছিল সমস্যা। প্ল্যাটফর্ম দল গঠনের পর একটিমাত্র পথ — একটি অনুমোদিত পাইপলাইন টেমপ্লেট — সব দলকে একই নিরাপত্তা ভিত্তিতে নিয়ে এলো।

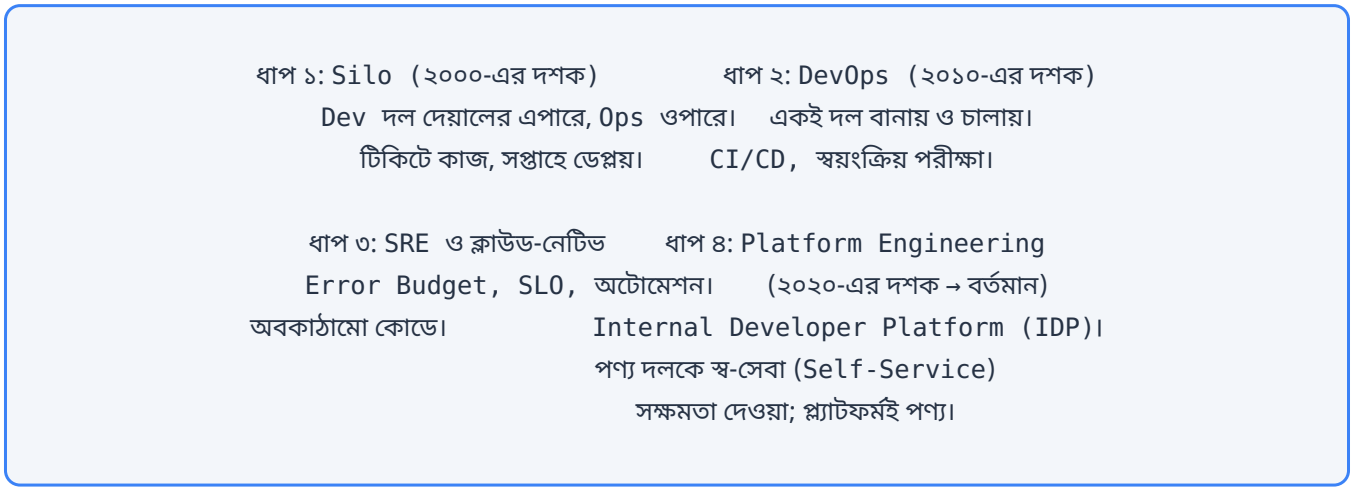
১.৩ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই ভলিউম শুরু করার আগে আপনার আগের ভলিউমগুলোর মূল ধারণাগুলো ঝালিয়ে নেওয়া দরকার। **Git**-এ ব্রাঞ্চ, মার্জ ও Pull Request; **CI/CD**-তে বিল্ড, পরীক্ষা ও ডেপ্লয় পাইপলাইন; **Docker**-এ কন্টেইনার ও ইমেজ; **Kubernetes**-এ Pod, Service ও Deployment; **Terraform**-এ ঘোষণামূলক অবকাঠামো; এবং **Observability**-তে লগ, মেট্রিক ও ট্রেস — এই ছয়টি স্তম্ভ না জানলে এই অধ্যায়ের আলোচনা ভাসা ভাসা মনে হবে। বিশেষ করে মনে রাখুন: **Idempotence** মানে একই কাজ যতবারই করা হোক, ফল একই থাকা; **Immutable Infrastructure** মানে চলমান জিনিস বদলানো নয়, নতুন করে বানিয়ে বদলে দেওয়া; এবং **GitOps** মানে Git-কে সত্যের একমাত্র উৎস ধরে সিস্টেম পরিচালনা করা। এই তিনটি নীতি এই ভলিউমের প্রতিটি অধ্যায়ে ফিরে ফিরে আসবে।

আপনি যদি Kubernetes-এ সার্ভিস কীভাবে একে অপরের সাথে কথা বলে, অথবা Terraform-এ State কীভাবে কাজ করে — এই দুটি বিষয়ে অনিশ্চিত থাকেন, তাহলে এগিয়ে যাওয়ার আগে সংশ্লিষ্ট ভলিউমের সারাংশটি একবার দেখে নিন। উন্নত স্থাপত্য শেখা মানে ভিত্তি ভুলে যাওয়া নয়; ভিত্তির উপর আরও জটিল সিদ্ধান্ত নেওয়া।

১.৪ DevOps-এর বিবর্তন: অপারেশন থেকে প্ল্যাটফর্ম

DevOps-এর ইতিহাসকে চারটি ধাপে ভাগ করা যায়। প্রতিটি ধাপ আগের ধাপের একটি নির্দিষ্ট যন্ত্রণার উত্তর।



১.৪.১ ধাপ ১ — Silo বা বিচ্ছিন্ন দল

প্রথম যুগে ডেভেলপাররা কোড লিখে "দেয়ালের ওপারে" ছুঁড়ে দিতেন, আর অপারেশন দল তা চালানোর চেষ্টা করত। ডেভেলপারদের সাফল্য মাপা হতো কত দ্রুত ফিচার দেওয়া যায় দিয়ে; অপারেশনদের সাফল্য মাপা হতো কত কম পরিবর্তন হয় দিয়ে। দুই দলের প্রণোদনা ছিল পরস্পরবিরোধী — এটাই ছিল মূল সমস্যা।

১.৪.২ ধাপ ২ — DevOps সংস্কৃতি

DevOps এসেছিল এই দেয়াল ভাঙতে: একই দল সফটওয়্যার বানাবে ও চালাবে, পরিবর্তন ছোট ছোট করে ঘন ঘন যাবে, স্বয়ংক্রিয় পরীক্ষা বিশ্বাস তৈরি করবে। এই ধাপের মূল আবিষ্কারটি প্রযুক্তিগত নয়, সাংগঠনিক: দায়িত্ব ভাগ করে নেওয়া। **You Build It, You Run It** — যে বানায়, সে চালায় — এই নীতিটি Amazon-এর কাছ থেকে জনপ্রিয় হয়।

১.৪.৩ ধাপ ৩ — SRE ও ক্লাউড-নেটিভ

Google-এর **Site Reliability Engineering (SRE)** DevOps-এ প্রকৌশলগত কঠোরতা যোগ করল: নির্ভরযোগ্যতাকে সংখ্যায় মাপা (**SLO — Service Level Objective**), ব্যর্থতার জন্য বাজেট রাখা (**Error Budget**), এবং হাতে করা কাজ (Toil) কমিয়ে অটোমেশন বাড়ানো। একই সময়ে কন্টেইনার, Kubernetes ও ক্লাউড অবকাঠামোকে কোডে লেখার সুযোগ দিল।

১.৪.৪ ধাপ ৪ — Platform Engineering

যখন প্রতিটি পণ্য দল নিজে নিজে Kubernetes, পাইপলাইন ও নিরাপত্তা শিখতে গিয়ে ধীর হয়ে গেল, তখন বোঝা গেল — প্রতিটি দলকে প্ল্যাটফর্ম বিশেষজ্ঞ হতে হবে না। একটি প্ল্যাটফর্ম দল সাধারণ সক্ষমতাগুলো একবার ভালোভাবে বানাবে, আর পণ্য দল সেগুলো স্ব-সেবায় ব্যবহার করবে। এটাই **Platform Engineering**। এর মূল পণ্য হলো **Internal Developer Platform (IDP)** — এমন একটি অভ্যন্তরীণ প্ল্যাটফর্ম যেখানে ডেভেলপার নিজে পরিবেশ বানাতে, ডেপ্লয় করতে ও পর্যবেক্ষণ করতে পারে, অপারেশন দলের টিকিট ছাড়াই।

> **WARNING:** Platform Engineering মানে আরেকটি কেন্দ্রীয় দল বানিয়ে সব সিদ্ধান্ত তাদের হাতে দেওয়া নয়। প্ল্যাটফর্ম দল যদি পণ্য দলের কথা না শুনে নিজের পছন্দের টুল চাপিয়ে দেয়, তাহলে সেটি পুরোনো Silo-রই নতুন রূপ — শুধু নাম বদলেছে। ভালো প্ল্যাটফর্ম দল তাদের "গ্রাহক" — অর্থাৎ পণ্য দলের ডেভেলপারদের — সাথে পণ্য ব্যবস্থাপকের মতো আচরণ করে: প্রয়োজন শোনে, ব্যবহার পরিমাপ করে, এবং প্ল্যাটফর্মকে ক্রমাগত উন্নত করে।

১.৫ এন্টারপ্রাইজ ইঞ্জিনিয়ারিং মডেল

বড় প্রতিষ্ঠানে প্রযুক্তিগত সিদ্ধান্ত কে নেবে — এই প্রশ্নের তিনটি প্রধান উত্তর আছে। কোনো মডেলই সর্বজনীনভাবে সেরা নয়; প্রতিটির খরচ ও সুবিধা আলাদা।

মডেল	সিদ্ধান্ত কে নেয়	সুবিধা	ঝুঁকি	কখন মানানসই
কেন্দ্রীভূত (Centralized)	একটি কেন্দ্রীয় স্থাপত্য দল	সামঞ্জস্য, নিরাপত্তা নিয়ন্ত্রণ সহজ	ধীর গতি, দলের ক্ষোভ	কঠোর নিয়ন্ত্রক পরিবেশ, ব্যাংক, স্বাস্থ্য
বিকেন্দ্রীভূত (Decentralized)	প্রতিটি পণ্য দল নিজে	দ্রুত গতি, স্থানীয় প্রেক্ষাপট	বিশৃঙ্খলা, পুনরাবৃত্তি, নিরাপত্তা ফাঁক	ছোট সংস্থা, স্বাধীন পণ্য
ফেডারেটেড (Federated)	কেন্দ্রীয় নীতি + দলীয় স্বাধীনতা	ভারসাম্য: নিরাপত্তা ও গতি দুটোই	সীমানা নিয়ে বিরোধ হতে পারে	বেশিরভাগ মাঝারি-বড় প্রতিষ্ঠান

TechNova-র মতো মাঝারি আকারের কোম্পানির জন্য সাধারণত **Federated** মডেল সবচেয়ে কার্যকর: কেন্দ্র ঠিক করে দেয় "কী করা যাবে না" — যেমন গোপন তথ্য কোডে রাখা যাবে না, সব সার্ভিসে mTLS বাধ্যতামূলক, ব্যক্তিগত তথ্য এনক্রিপ্টেড থাকতে হবে — আর দলগুলো নিজেরা ঠিক করে "কীভাবে করবে" — কোন ভাষা, কোন ডেটাবেস, কোন স্থাপত্য

ধাঁচ। কেন্দ্রীয় নীতিকে বলা হয় **Guardrail**, অর্থাৎ রাস্তার ধারের রেলিং: এটি গাড়িকে রাস্তায় রাখে, কিন্তু গাড়ি কত দ্রুত চলবে তা চালক ঠিক করে।

১.৫.১ Team Topologies: দলের আকৃতিই স্থাপত্য নির্ধারণ করে

Conway's Law বলে — কোনো সংস্থার সফটওয়্যার স্থাপত্য তার যোগাযোগ কাঠামোর প্রতিফলন হয়। চারটি দল যদি একে অপরের সাথে কথা না বলে, তাহলে সফটওয়্যারও চারটি বিচ্ছিন্ন টুকরো হবে। **Team Topologies** বইটি দলের চারটি ধরন প্রস্তাব করে: **Stream-Aligned Team** (একটি পণ্য-প্রবাহের জন্য দায়ী), **Platform Team** (সাধারণ সক্ষমতা দেয়), **Enabling Team** (অন্য দলকে শেখায় ও সাহায্য করে), এবং **Complicated-Subsystem Team** (বিশেষজ্ঞ জ্ঞানের জটিল অংশ সামলায়)। এই ধারণার মূল কথা: স্থাপত্যের নকশা শুধু কোডের নয়, দলেরও। আপনি যদি সার্ভিসের সীমানা বদলাতে চান, প্রথমে দেখুন দলের সীমানা কী বলে।

১.৬ সিস্টেমের মালিকানা (System Ownership)

মালিকানা ছাড়া স্থাপত্য কাগজেই থাকে। প্রতিটি সার্ভিস, ডেটাবেস, পাইপলাইন ও ড্যাশবোর্ডের একজন স্পষ্ট মালিক থাকতে হয় — এমন একজন বা একটি দল, যারা জানে সার্ভিসটি কী করে, কে এর উপর নির্ভর করে, ব্যর্থ হলে কী হবে, এবং পরিবর্তনের অনুমোদন কে দেবে। মালিকানা তিন স্তরে ভাবুন:

মালিকানার তিন স্তর (TechNova উদাহরণ)

স্তর ১ — সার্ভিস মালিক (Service Owner)
উদাহরণ: payments দলের প্রধান — payments-api সার্ভিসের মালিক।
দায়িত্ব: ফিচার, SLO, অন-কল, পরিবর্তনের ঝুঁকি।

স্তর ২ — প্ল্যাটফর্ম মালিক (Platform Owner)
উদাহরণ: প্ল্যাটফর্ম দল — Kubernetes ক্লাস্টার, CI/CD, পর্যবেক্ষণ।
দায়িত্ব: ভাগ করা সক্ষমতা, নিরাপত্তা ভিত্তি, আপগ্রেড।

স্তর ৩ — ডেটা ও নিরাপত্তা মালিক (Data / Security Owner)
উদাহরণ: নিরাপত্তা দল — গোপন তথ্য নীতি, নিরীক্ষা লগ।
দায়িত্ব: নিয়ম, নিরীক্ষা, ঘটনার তদন্ত।

মালিকানা নির্ধারণের ব্যবহারিক উপায় হলো **Service Catalog** — প্রতিটি সার্ভিসের নাম, মালিক দল, অন-কল যোগাযোগ, নির্ভরশীলতা ও SLO এক জায়গায় রাখা। Backstage-এর মতো টুল এই ক্যাটালগকে কোড থেকে তৈরি করে। ক্যাটালগ ছাড়া একটি ঘটনার সময় প্রথম দশ মিনিট শুধু “কার সার্ভিস এটা?” খুঁজতে চলে যায় — আর ঘটনার প্রথম দশ মিনিটই সবচেয়ে দামি।

```
# TechNova Service Catalog — catalog-info.yaml (Backstage ধাঁচ, সংক্ষিপ্ত)
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: payments-api
  description: TechNova পেমেন্ট API — অর্ডার থেকে চার্জ পর্যন্ত
```

```
  annotations:
    technova.example/on-call: payments-team
    technova.example/slo-availability: "99.9"
spec:
  type: service
  lifecycle: production
  owner: team-payments
  system: commerce-platform
  dependsOn:
    - component:orders-api
    - resource:postgres-payments
```

১.৭ স্থাপত্যের নীতিমালা (Architecture Principles)

নীতি হলো এমন নিয়ম যা হাজারো ছোট সিদ্ধান্তের সময় বারবার প্রয়োগ করা যায়। TechNova-র জন্য পাঁচটি নীতি যথেষ্ট:

- **১. সরলতা আগে (Simplicity First):** সবচেয়ে সহজ সমাধানটি বেছে নিন যা প্রয়োজন মেটায়। জটিলতা প্রমাণের ভার তার উপর যে জটিলতা প্রস্তাব করছে। একটি মনোলিথ যতদিন কাজ করছে, তাকে ভাঙার প্রয়োজন নেই।
- **২. ব্যর্থতা ধরে নিয়ে নকশা (Design for Failure):** সার্ভার মরবে, নেটওয়ার্ক ছিঁড়বে, নির্ভরশীল সার্ভিস ধীর হবে — এগুলো "যদি" নয়, "কখন"। প্রতিটি নকশায় প্রশ্ন করুন: এই অংশটি মারা গেলে ব্যবহারকারী কী দেখবে?
- **৩. স্বয়ংক্রিয়তাই ডিফল্ট (Automation by Default):** হাতে করা কাজ দ্বিতীয়বার করা হলে তা স্বয়ংক্রিয় করার প্রার্থী। মানুষের সময় সৃজনশীল কাজে যাবে, পুনরাবৃত্তিতে নয়।
- **৪. পর্যবেক্ষণযোগ্যতা নকশার অংশ (Observability by Design):** লগ, মেট্রিক ও ট্রেস পরে যোগ করা যায় না; এগুলো প্রথম দিন থেকে সার্ভিসের অংশ।
- **৫. নিরাপত্তা ও খরচ সমান নাগরিক (Security & Cost as First-Class):** নিরাপত্তা পর্যালোচনা ও খরচের হিসাব স্থাপত্য পর্যালোচনার আবশ্যিক অংশ, শেষ মুহূর্তের চেকলিস্ট নয়।

> **CONCEPT:** নীতি আর নিয়ম এক নয়। নিয়ম বলে "Kubernetes ব্যবহার করতে হবে"; নীতি বলে "পরিবর্তন নিরাপদে ও দ্রুত ফিরিয়ে নেওয়া যায় এমন হতে হবে" — যা Kubernetes দিয়েও হতে পারে, ভার্চুয়াল মেশিন দিয়েও হতে পারে। নীতি দীর্ঘস্থায়ী হয়, নির্দিষ্ট টুল বদলে যায়।

১.৮ প্রযুক্তিগত ট্রেড-অফ: সিদ্ধান্ত নেওয়ার কাঠামো

স্থাপত্যের প্রায় প্রতিটি সিদ্ধান্ত একটি আপস। গতি বাড়ালে জটিলতা বাড়ে; নির্ভরযোগ্যতা বাড়ালে খরচ বাড়ে; স্বাধীনতা বাড়ালে সামঞ্জস্য কমে। পেশাদার প্রকৌশলী ট্রেড-অফ লুকান না; তিনি তা স্পষ্ট করে লেখেন। একটি কার্যকর কাঠামো:

প্রশ্ন	কেন জিজ্ঞেস করবেন
এই সিদ্ধান্তটি কী সমস্যা সমাধান করছে?	সমস্যা ছাড়া সমাধান কেনা মানে জটিলতা কেনা।
সবচেয়ে সহজ বিকল্পটি কী?	সহজ বিকল্পটি ব্যর্থ না হওয়া পর্যন্ত জটিলতা নেবেন না।
এটি ব্যর্থ হলে কী হয়?	ব্যর্থতার পরিধি (Blast Radius) বোঝা যায়।

এর পরিচালন খরচ কে বহন করবে?	চালানোর খরচ কেনার খরচের চেয়ে বড় হতে পারে।
পরে কি ফিরে আসা যায়?	অপরিবর্তনীয় (Irreversible) সিদ্ধান্তে বেশি যত্ন লাগে।

উদাহরণ: TechNova-র পণ্য দল মাইক্রোসার্ভিসে যেতে চায়, কারণ "সবাই যাচ্ছে"। এই কাঠামোয় প্রশ্ন করলে দেখা যায় — আসল সমস্যা ডেপ্লয় ধীর হওয়া, যা মডুলার মনোলিথ দিয়েও সমাধান হতে পারে; মাইক্রোসার্ভিস আনলে নেটওয়ার্ক ব্যর্থতা, ডেটা সামঞ্জস্য ও পর্যবেক্ষণের জটিলতা বাড়বে; আর তা চালানোর জন্য প্ল্যাটফর্ম দলের সময় লাগবে। সিদ্ধান্তটি তখন আবেগের নয়, হিসাবের হয়। এই বিষয়টি অধ্যায় ৩-এ বিস্তারিত আসবে।

১.৯ সাংগঠনিক সীমানা ও যোগাযোগের খরচ

দুটি দলের মধ্যে প্রতিটি যোগাযোগের একটি খরচ আছে — মিটিং, টিকিট, অপেক্ষা, ভুল বোঝাবুঝি। স্থাপত্যের কাজের একটি বড় অংশ হলো এই খরচ কমানো: দলগুলো যেন নিজের কাজ নিজে করতে পারে, অন্য দলের অনুমোদনের জন্য না থামে। একে বলা হয় **Loose Coupling** বা ঢিলেঢালা সংযোগ — দলের স্তরেও, কোডের স্তরেও। একটি দল যখন অন্য দলের ডেটাবেসে সরাসরি হাত দেয়, তখন দুটি দল একে অপরের সাথে বাঁধা পড়ে; একটি API চুক্তি (Contract) দিয়ে সংযোগ করলে দুটি দল স্বাধীনভাবে এগোতে পারে। একই নীতি সংস্থার ভেতরে প্রযোজ্য: প্ল্যাটফর্ম দল যদি প্রতিটি ডেপ্লয়ের অনুমোদন দেয়, তাহলে প্ল্যাটফর্ম দলই বোতল-ঘাড় হয়ে যায়; প্ল্যাটফর্ম যদি স্ব-সেবা পথ দেয়, তাহলে দলগুলো নিজেরাই এগোয়।

> **TROUBLE:** একটি সাধারণ ব্যর্থতার ধাঁচ: "স্থাপত্য কমিটি" মাসে একবার বসে, প্রতিটি পরিবর্তন অনুমোদন করে, এবং তিন সপ্তাহের অপেক্ষা তৈরি করে। দলগুলো তখন কমিটি এড়াতে শুরু করে — ছোট পরিবর্তনকে "ছোট" বলে চালিয়ে দেয় — আর স্থাপত্যের নিয়ন্ত্রণ আরও কমে। সমাধান: অনুমোদনের বদলে Guardrail দিন — স্বয়ংক্রিয় পরীক্ষা ও নীতি যাচাই, যেখানে নিয়ম ভাঙলে পাইপলাইন নিজেই থামে, মানুষের অপেক্ষা নয়।

১.১০ ইঞ্জিনিয়ারিং পরিপক্কতা (Engineering Maturity)

পরিপক্কতা মানে "আমরা কত উন্নত টুল ব্যবহার করি" নয়; মানে "আমরা কত নির্ভরযোগ্যভাবে, কত দ্রুত, কত নিরাপদে ডেলিভার করতে পারি"। একটি সরল পরিমাপ কাঠামো:

স্তর	বৈশিষ্ট্য	TechNova-র উদাহরণ
১ — ম্যানুয়াল	হাতে ডেপ্লয়, অনথিভুক্ত জ্ঞান	শুরুর দিনগুলো: SSH করে ডেপ্লয়
২ — স্বয়ংক্রিয়	CI/CD, IaC, পরীক্ষা স্বয়ংক্রিয়	পাইপলাইন ও Terraform চালু
৩ — পরিমাপযোগ্য	SLO, Error Budget, DORA মেট্রিক	ডেপ্লয় ফ্রিকোয়েন্সি ও ব্যর্থতা মাপা হয়
৪ — প্ল্যাটফর্ম	IDP, স্ব-সেবা, Guardrail স্বয়ংক্রিয়	দল নিজে পরিবেশ বানাতে পারে
৫ — শেখা সংস্থা	Blameless Postmortem, ক্রমাগত উন্নতি	প্রতিটি ঘটনা থেকে স্থায়ী শিক্ষা

DORA মেট্রিক চারটি সংখ্যায় দলের স্বাস্থ্য মাপে: ডেপ্লয় কত ঘন ঘন হয় (Deployment Frequency), পরিবর্তন থেকে প্রোডাকশনে যেতে কত সময় লাগে (Lead Time for Changes), পরিবর্তনের কত শতাংশ ব্যর্থ হয় (Change Failure Rate), এবং ব্যর্থতা সারাতে কত সময় লাগে (Time to Restore Service)। এই সংখ্যাগুলো না মেপে "আমরা উন্নত হয়েছি" বলা শুধু অনুভূতির কথা। পরিপক্কতা বাড়ানোর সঠিক ক্রম হলো স্তর বাদ না দিয়ে এগোনো — স্তর ২ ছাড়া স্তর ৪-এর প্ল্যাটফর্ম বানানো মানে বালির উপর বাড়ি বানানো।

১.১১ TechNova দৃশ্যকল্প: ফেডারেটেড মডেলে যাওয়া

TechNova-র আজকের অবস্থা: তিনটি পণ্য দল (Commerce, Content, Analytics), একটি প্ল্যাটফর্ম দল (তিনজন), এবং একটি নিরাপত্তা দল (একজন)। সমস্যা: প্রতিটি দল নিজের মতো করে ডেপ্লয় করে; নিরাপত্তা দল প্রতিটি পরিবর্তন পর্যালোচনা করতে গিয়ে বোতল-ঘাড়; কেউ জানে না কোন সার্ভিসের SLO কী। পরিকল্পনাটি তিন ধাপে:

```
# ধাপ ১ — ক্যাটালগ ও মালিকানা (সপ্তাহ ১–২)
# প্রতিটি সার্ভিস catalog-info.yaml-এ নথিভুক্ত করুন।
# মালিক দল, অন-কল, SLO — কিছুই অনুমান নয়, লিখিত।

# ধাপ ২ — Guardrail স্বয়ংক্রিয় করা (সপ্তাহ ৩–৬)
# CI পাইপলাইনে যোগ করুন: গোপন তথ্য স্ক্যান, ইমেজ দুর্বলতা স্ক্যান,
# নীতি যাচাই (Policy as Code)। মানুষের অনুমোদন নয় — পাইপলাইন থামবে।

# ধাপ ৩ — স্ব-সেবা পথ (সপ্তাহ ৭–১২)
# একটি টেমপ্লেট থেকে নতুন সার্ভিস বানানো যায়: রিপো, পাইপলাইন,
# পর্যবেক্ষণ ড্যাশবোর্ড — সব এক কমান্ডে। প্ল্যাটফর্ম দল তখন
# টিকিট সামলায় না, প্ল্যাটফর্ম উন্নত করে।

$ technova-platform new-service --name search-api --team content --lang go
✓ Repository created: technova/search-api
✓ CI/CD pipeline from template platform-v2
✓ Dashboards and alerts provisioned
✓ SLO: 99.9% availability, 300ms p95 latency
```

লক্ষ্য করুন — এই পরিকল্পনায় কোনো নতুন টুল কেনার কথা নেই। প্রথমে মালিকানা, তারপর স্বয়ংক্রিয় নিয়ম, তারপর স্ব-সেবা। টুল শেষে আসে, কাঠামো আগে।

১.১২ নিরাপত্তা ও খরচ বিবেচনা

এন্টারপ্রাইজ স্তরে নিরাপত্তা মানে শুধু ফায়ারওয়াল নয়; মানে পরিচয় ও অনুমতি (**IAM**), গোপন তথ্যের জীবনচক্র, নিরীক্ষা লগ, এবং সরবরাহ শৃঙ্খলের নিরাপত্তা (**Supply Chain Security**) — কোন লাইব্রেরি, কোন বেস ইমেজ, কোন পাইপলাইন ধাপ থেকে কোড প্রোডাকশনে যাচ্ছে। প্রতিটি সার্ভিসে **Least Privilege** নীতি প্রয়োগ করুন: সার্ভিস শুধু সেই অনুমতিটুকু পাবে যা তার কাজের জন্য দরকার, তার বেশি নয়। গোপন তথ্য কখনোই কোডে নয়; ভল্ট বা ক্লাউড সিক্রেট ম্যানেজারে রাখুন, আর প্লেসহোল্ডার হিসেবে `CHANGE-ME-...` ব্যবহার করুন। খরচের দিক থেকে প্ল্যাটফর্ম দল নিজেই একটি খরচ — তিনজন প্রকৌশলী সারাবছর প্ল্যাটফর্ম বানাবেন, অথচ সরাসরি কোনো ফিচার দেবেন না। সেই খরচের যৌক্তিকতা প্রমাণ করতে হয় পণ্য দলের সময় বাঁচানোর হিসাব দিয়ে: প্রতিটি দল যদি সপ্তাহে পাঁচ ঘণ্টা প্ল্যাটফর্ম-কাজে বাঁচায়, তাহলে প্ল্যাটফর্ম দলের খরচ উঠে আসে। এই হিসাবটি লিখিতভাবে রাখুন; নইলে প্রথম বাজেট কাটায় প্ল্যাটফর্ম দলই প্রথম শিকার হবে।

> **WARNING:** “সবাই মাইক্রোসার্ভিস ব্যবহার করছে” বা “সার্ভিস মেশ আমাদের দরকার” — এই যুক্তিতে কখনো স্থাপত্য সিদ্ধান্ত নেবেন না। প্রতিটি উন্নত প্রযুক্তির একটি পরিচালন খরচ আছে: শেখা, চালানো, পর্যবেক্ষণ, আপগ্রেড। সেই খরচ যে সমস্যাটি সমাধান করছে তার চেয়ে বড় হলে, সহজ সমাধানটিই সঠিক। এই ভলিউমের মূল বার্তা এটাই: উন্নত হওয়া মানে জটিল হওয়া নয়; উন্নত হওয়া মানে সঠিক জটিলতা বেছে নেওয়া।

১.১৩ সাধারণ ভুল

- **টুল দিয়ে সংস্কৃতি কেনা:** Kubernetes কিনলেই DevOps হয় না; দল, মালিকানা ও প্রক্রিয়া না বদলালে শুধু জটিলতা বাড়ে।
- **স্থাপত্যকে অনুমোদনের স্তূপ বানানো:** প্রতিটি পরিবর্তনে কমিটির অনুমোদন চাইলে দল কমিটি এড়াতে শেখে।
- **মালিকবিহীন সার্ভিস:** “সবাই দেখে” মানে আসলে কেউ দেখে না। প্রতিটি সার্ভিসের একজন নাম-লেখা মালিক থাকতে হবে।
- **পরিপক্কতার স্তর লাফ দেওয়া:** CI/CD ঠিক না করে IDP বানানো, SLO না মেপে Error Budget ঘোষণা করা।
- **Conway's Law উপেক্ষা করা:** দলের সীমানা না বদলে সার্ভিসের সীমানা বদলানোর চেষ্টা — শেষে দুটোই ভাঙে।
- **প্ল্যাটফর্মকে “সবাই ব্যবহার করবে” ধরে নেওয়া:** পণ্য দলের প্রয়োজন না শুনে প্ল্যাটফর্ম বানালে কেউ তা ব্যবহার করবে না।

১.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
দলগুলো প্ল্যাটফর্ম ব্যবহার করছে না	প্ল্যাটফর্ম তাদের প্রয়োজন মেটায় না	পণ্য দলের সাথে বসুন; ব্যবহারের বাধা মাপুন; প্ল্যাটফর্মকে পণ্যের মতো চালান।
প্রতিটি ডেপ্লয়ে নিরাপত্তা দলের অনুমোদন লাগে	নিয়ম মানুষের উপর নির্ভরশীল	নিয়মকে পাইপলাইনে স্বয়ংক্রিয় যাচাইয়ে রূপান্তর করুন।
ঘটনার সময় মালিক খুঁজে পাওয়া যায় না	Service Catalog নেই	ক্যাটালগ বানান; প্রতিটি সার্ভিসে মালিক ও অন-কল বাধ্যতামূলক করুন।
“আমরা উন্নত হয়েছি” — কিন্তু প্রমাণ নেই	DORA মেট্রিক মাপা হয় না	চারটি DORA মেট্রিক মাসিক ড্যাশবোর্ডে তুলুন।
স্থাপত্য সিদ্ধান্ত নিয়ে বারবার বিতর্ক	সিদ্ধান্ত লিখিত নয়	ADR (অধ্যায় ২) চালু করুন; প্রতিটি বড় সিদ্ধান্ত লিখে রাখুন।

১.১৫ অধ্যায়ের সারাংশ

Advanced DevOps মানে আরও টুল নয়; মানে পুরো প্রতিষ্ঠানের স্কেলে সফটওয়্যার ডেলিভারির নকশা করা। DevOps বিবর্তিত হয়েছে Silo থেকে DevOps, SRE হয়ে Platform Engineering-এ — যেখানে প্ল্যাটফর্ম দল পণ্য দলকে স্ব-সেবা সক্ষমতা দেয়। এন্টারপ্রাইজ মডেলের মধ্যে Federated মডেল বেশিরভাগ মাঝারি-বড় প্রতিষ্ঠানে ভারসাম্য দেয়: কেন্দ্রীয় Guardrail, দলীয় স্বাধীনতা। মালিকানা, নীতি, ট্রেড-অফ বিশ্লেষণ ও পরিপক্কতা পরিমাপ — এই চারটি ছাড়া উন্নত স্থাপত্য

টেকে না। আর সবচেয়ে গুরুত্বপূর্ণ সতর্কবাণী: জটিলতাকে লক্ষ্য বানাবেন না; সঠিক জটিলতাটুকু বেছে নেওয়াই উন্নত প্রকৌশল।

১.১৬ জ্ঞান যাচাই

1. প্রাথমিক DevOps ও Advanced DevOps-এর পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন।
2. Platform Engineering, SRE ও DevOps — তিনটি ভূমিকার পার্থক্য কী?
3. Internal Developer Platform (IDP) কী? এটি ছাড়া কী সমস্যা হয়?
4. কেন্দ্রীভূত, বিকেন্দ্রীভূত ও ফেডারেটেড মডেলের মধ্যে কোনটি কখন মানানসই? উদাহরণ দিন।
5. Conway's Law কী? এটি স্থাপত্য নকশায় কীভাবে প্রভাব ফেলে?
6. Team Topologies-এর চারটি দলের ধরন ব্যাখ্যা করুন।
7. Service Catalog কেন ঘটনা ব্যবস্থাপনার জন্য গুরুত্বপূর্ণ?
8. Guardrail ও অনুমোদন-কমিটির পার্থক্য কী? কোনটি ভালো এবং কেন?
9. DORA-র চারটি মেট্রিক কী কী? এগুলো না মাপলে কী সমস্যা?
10. “You Build It, You Run It” নীতির সুবিধা ও সীমা কী?
11. পরিপক্কতার স্তর লাফ দিলে কী ক্ষতি হয়? উদাহরণ দিন।
12. একটি ছোট দলের জন্য প্ল্যাটফর্ম দল বানানো কেন ভুল হতে পারে? যুক্তি দিন।

১.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র তিনটি পণ্য দলের জন্য একটি Service Catalog তৈরি করুন: প্রতিটি সার্ভিসের নাম, মালিক দল, অন-কল, SLO ও নির্ভরশীলতা — একটি YAML বা Markdown ফাইলে লিখুন।
2. আপনার বর্তমান দল বা কাল্পনিক দলের পরিপক্কতা স্তর (১–৫) নির্ধারণ করুন এবং DORA-র চারটি মেট্রিকের বর্তমান মান অনুমান করে লিখুন; পরের স্তরে যেতে তিনটি সুনির্দিষ্ট পদক্ষেপ লিখুন।
3. একটি বাস্তব স্থাপত্য সিদ্ধান্ত বেছে নিন (যেমন: মনোলিথ বনাম মাইক্রোসার্ভিস) এবং ১.৮-এর পাঁচটি প্রশ্ন দিয়ে ট্রেড-অফ বিশ্লেষণ লিখুন — এক পৃষ্ঠার সুপারিশসহ।

অধ্যায় ২ • PART I — Advanced DevOps & Enterprise Architecture

Chapter 2 — Production System Design

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Requirements Engineering** বা প্রয়োজন বিশ্লেষণ থেকে শুরু করে একটি প্রোডাকশন সিস্টেমের সম্পূর্ণ নকশা তৈরি করতে; কার্যকরী (Functional) ও অকার্যকরী (Non-Functional) প্রয়োজন আলাদা করে লিখতে; **High-Level Design (HLD)** ও **Low-Level Design (LLD)**-এর মধ্যে পার্থক্য ও সংযোগ ব্যাখ্যা করতে; **Architecture Decision Record (ADR)** লিখে সিদ্ধান্তের ইতিহাস সংরক্ষণ করতে; গুণগত বৈশিষ্ট্য (Quality Attributes) ও ধারণক্ষমতার অনুমান (Capacity Estimation) দিয়ে নকশাকে সংখ্যায় যাচাই করতে; এবং ঝুঁকি বিশ্লেষণ ও স্থাপত্য পর্যালোচনার (Architecture Review) মাধ্যমে নকশার দুর্বলতা আগেই ধরতে।

- অস্পষ্ট ব্যবসায়িক চাহিদাকে পরিমাপযোগ্য প্রয়োজনে রূপান্তর করা।
- HLD থেকে LLD পর্যন্ত ধারাবাহিক নকশা নথি লেখা।
- ADR ফরম্যাটে সিদ্ধান্ত, বিকল্প ও পরিণতি লিপিবদ্ধ করা।
- লোড, স্টোরেজ ও নেটওয়ার্কের ধারণক্ষমতা অনুমান করা।
- ঝুঁকি নিবন্ধন (Risk Register) তৈরি করে প্রশমন পরিকল্পনা লেখা।
- স্থাপত্য পর্যালোচনা সভায় নকশা উপস্থাপন ও সমালোচনা গ্রহণ করা।

২.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

বেশিরভাগ প্রোডাকশন বিপর্যয় কোডের ভুলে হয় না; হয় নকশার ভুলে — এমন একটি অনুমান যা কেউ লেখেনি, এমন একটি প্রয়োজন যা কেউ জিজ্ঞেস করেনি, এমন একটি সীমা যা কেউ পরীক্ষা করেনি। TechNova-র অর্ডার সিস্টেম একবার একটি উৎসবের বিক্রিতে ভেঙে পড়েছিল। কারণ কোডে কোনো বাগ ছিল না; কারণ ছিল — কেউ জিজ্ঞেস করেনি একসাথে কতজন ব্যবহারকারী আসবে, ডেটাবেসের সংযোগ-সীমা কত, আর পেমেন্ট গেটওয়ে ধীর হলে কী হবে। নকশা মানে এই প্রশ্নগুলো আগেই করা, উত্তরগুলো লিখে রাখা, এবং উত্তরের ভিত্তিতে সিদ্ধান্ত নেওয়া। ভালো নকশা ধীরগতির বিলাসিতা নয়; এটি সবচেয়ে সস্তা ঝুঁকি-ব্যবস্থাপনা। একটি নকশার ভুল কাগজে ধরতে এক ঘণ্টা লাগে, প্রোডাকশনে ধরতে এক সপ্তাহ — আর গ্রাহক হারানোর খরচ তার চেয়েও বেশি।

CONCEPT: Production System Design মানে শুধু বাক্স আর তীর আঁকা নয়। একটি নকশা নথির আসল কাজ তিনটি: (১) সবাই একই বোঝাপড়ায় আসা — “সিস্টেমটি আসলে কী করবে?”; (২) কঠিন সিদ্ধান্তগুলো লিখিতভাবে নেওয়া — যাতে ছয় মাস পর কেউ জিজ্ঞেস করলে উত্তর থাকে; (৩) যা জানা যায়নি তা স্পষ্ট করা — অনুমানগুলো লিখে রাখা, যাতে ভুল প্রমাণিত হলে ধরা পড়ে।

REAL WORLD: TechNova-তে নতুন “ফ্ল্যাশ সেল” ফিচারের নকশা সভায় একজন জুনিয়র প্রকৌশলী জিজ্ঞেস করেছিলেন — “এক মিনিটে কতটি অর্ডার আসবে বলে ধরা হচ্ছে?” কেউ জানত না। ব্যবসায়িক দল বলল “অনেক”। নকশা থামিয়ে সংখ্যা বের করা হলো: আগের বিক্রির তথ্য বলল সর্বোচ্চ ৮০০ অর্ডার/মিনিট। সেই সংখ্যার ভিত্তিতে ডেটাবেস সংযোগ, ক্যাশ ও কিউয়ের আকার ঠিক হলো — আর ফ্ল্যাশ সেল নির্বিঘ্নে চলল। একটি প্রশ্ন পুরো বিপর্যয় ঠেকিয়েছিল।

২.৩ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার অধ্যায় ১-এর স্থাপত্য নীতি, মালিকানা ও ট্রেড-অফ কাঠামো জানা থাকতে হবে। আরও দরকার: HTTP অনুরোধ-উত্তরের জীবনচক্র, ডেটাবেসের মৌলিক ধারণা (টেবিল, ইনডেক্স, লেনদেন), ক্যাশ কীভাবে কাজ করে তার প্রাথমিক ধারণা, এবং লোড ব্যালান্সার কী করে তার ধারণা। গাণিতিক দক্ষতা হিসেবে শুধু গুণ-ভাগ ও শতকরা হিসাবই যথেষ্ট — ধারণক্ষমতা অনুমানের অঙ্কগুলো ইচ্ছাকৃতভাবে সরল রাখা হয়েছে। আপনি যদি **Latency** (বিলম্ব), **Throughput** (একক সময়ে কাজের পরিমাণ) ও **Availability** (চালু থাকার হার) — এই তিনটি শব্দের পার্থক্য জানেন, তাহলে এগোতে পারবেন।

২.৪ প্রয়োজন বিশ্লেষণ: অস্পষ্ট চাহিদা থেকে পরিমাপযোগ্য প্রয়োজন

ব্যবসায়িক দল বলে “সিস্টেমটি দ্রুত হতে হবে”। এটি কোনো প্রয়োজন নয়; এটি একটি ইচ্ছা। প্রকৌশলীর কাজ হলো ইচ্ছাকে পরিমাপযোগ্য প্রয়োজনে রূপান্তর করা। দুই ধরনের প্রয়োজন আছে।

২.৪.১ কার্যকরী প্রয়োজন (Functional Requirements)

Functional Requirements: সিস্টেমটি কী কী করবে — এর আচরণ। উদাহরণ: ব্যবহারকারী পণ্য খুঁজতে পারবে; কার্টে যোগ করতে পারবে; পেমেন্ট করতে পারবে; অর্ডারের অবস্থা দেখতে পারবে। এগুলো সাধারণত ব্যবহারকারীর গল্প (User Story) বা ব্যবহার-ক্ষেত্র (Use Case) আকারে লেখা হয়। প্রতিটি প্রয়োজনের সাথে একটি গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria) থাকতে হবে — “কী হলে বলব এটি হয়েছে?”

২.৪.২ অকার্যকরী প্রয়োজন (Non-Functional Requirements)

Non-Functional Requirements (NFR): সিস্টেমটি কত ভালোভাবে কাজ করবে — এর গুণগত বৈশিষ্ট্য। এগুলোই প্রোডাকশন নকশার আসল চালক, কারণ একই কার্যকরী প্রয়োজন দশজন ব্যবহারকারীর জন্য আর দশ লাখ ব্যবহারকারীর জন্য সম্পূর্ণ ভিন্ন নকশা চায়।

গুণগত বৈশিষ্ট্য	পরিমাপযোগ্য প্রয়োজনের উদাহরণ	নকশায় প্রভাব
Availability (প্রাপ্যতা)	মাসে ৯৯.৯% চালু (ডাউনটাইম সর্বোচ্চ ~৪৩ মিনিট/মাস)	একাধিক অঞ্চল, স্বয়ংক্রিয় পুনরুদ্ধার
Latency (বিলম্ব)	৯৫ শতাংশ অনুরোধ ৩০০ মিলিসেকেন্ডের মধ্যে	ক্যাশ, CDN, ডেটাবেস ইনডেক্স
Throughput (ক্ষমতা)	সর্বোচ্চ ১,০০০ অনুরোধ/সেকেন্ড	অনুভূমিক স্কেলিং, কিউ
Durability (স্থায়িত্ব)	অর্ডারের তথ্য কখনো হারানো যাবে না	ডেটাবেস প্রতিলিপি, ব্যাকআপ
Security (নিরাপত্তা)	ব্যক্তিগত তথ্য এনক্রিপ্টেড; অনুমোদিত প্রবেশাধিকার	mTLS, IAM, নিরীক্ষা লগ
Scalability (স্কেলযোগ্যতা)	আগামী ১২ মাসে ১০ গুণ লোড সামলানো	Stateless সার্ভিস, শার্ডিং পরিকল্পনা

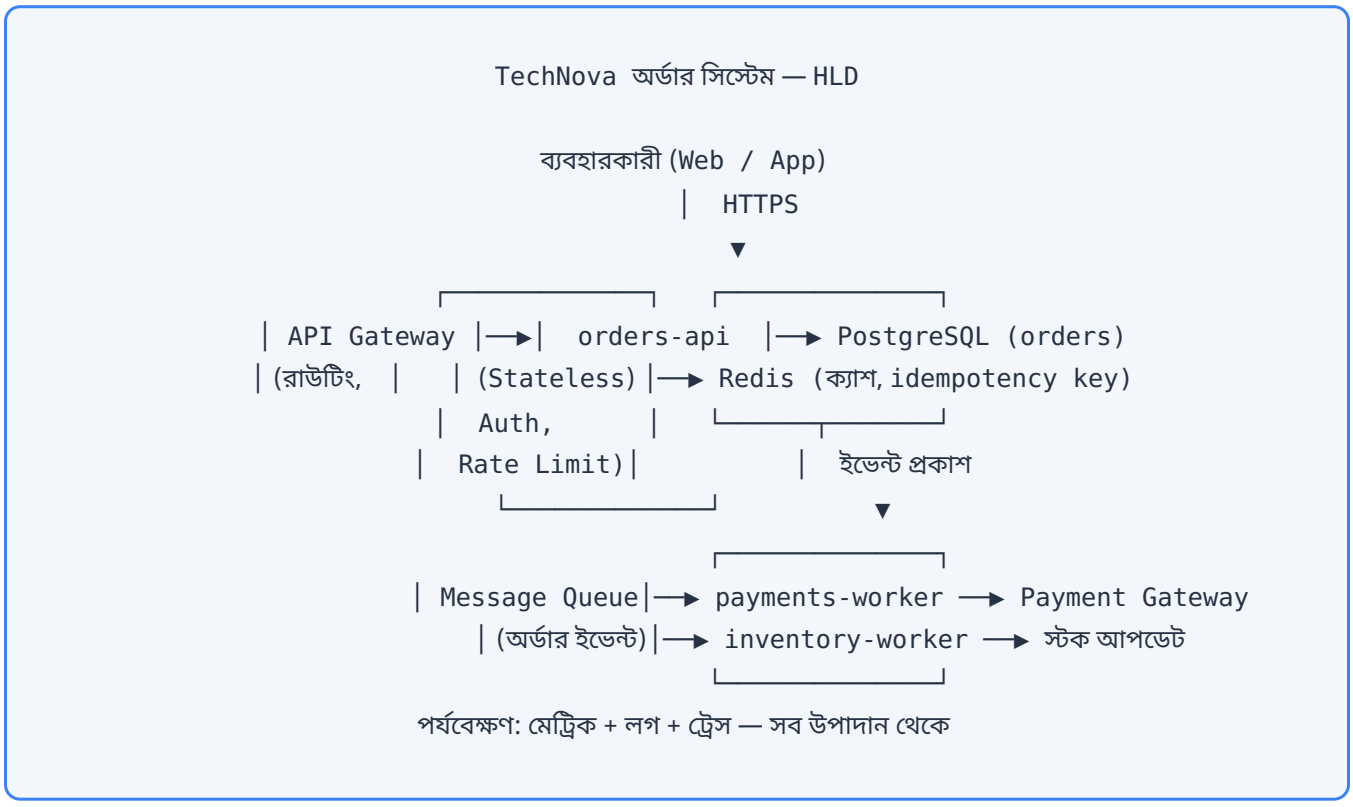
৯৯.৯% বনাম ৯৯.৯৯% — একটি অতিরিক্ত নয় (Nine) মানে দশ গুণ কম ডাউনটাইম, কিন্তু প্রায়শই দ্বিগুণ খরচ ও জটিলতা। তাই প্রতিটি NFR-এর সাথে প্রশ্ন করুন: “এই সংখ্যাটি ব্যবসার জন্য আসলে কতটা দামি?” একটি অভ্যন্তরীণ অ্যাডমিন প্যানেলের ৯৯.৯৯% প্রাপ্যতা দরকার নেই; পেমেন্ট API-র দরকার আছে।

```
# TechNova অর্ডার সিস্টেম — প্রয়োজন (সংক্ষিপ্ত, পরিমাপযোগ্য)
#
# কার্যকরী:
```

```
#    FR-1   ব্যবহারকারী অর্ডার দিতে পারবে (পণ্য, ঠিকানা, পেমেন্ট)
#    FR-2   অর্ডারের অবস্থা দেখা যাবে (placed → paid → shipped)
#    FR-3   ব্যর্থ পেমেন্টে অর্ডার বাতিল হয়ে স্টক ফেরত যাবে
#
# অকার্যকরী:
#    NFR-1  Availability: 99.9% (মাসে সর্বোচ্চ ~43 মিনিট ডাউনটাইম)
#    NFR-2  Latency: p95 < 300ms (অর্ডার তৈরির API)
#    NFR-3  Throughput: সর্বোচ্চ 1,000 অর্ডার/মিনিট
#    NFR-4  Durability: নিশ্চিত অর্ডার কখনো হারানো যাবে না
#    NFR-5  Consistency: পেমেন্ট নিশ্চিত না হলে অর্ডার নিশ্চিত নয়
```

২.৫ High-Level Design (HLD)

High-Level Design: সিস্টেমের বড় ছবি — কোন কোন প্রধান উপাদান (Component) থাকবে, তারা কীভাবে কথা বলবে, তথ্য কোথায় থাকবে, এবং বাইরের জগতের সাথে সংযোগ কোথায়। HLD-তে কোডের শ্রেণি বা ফাংশনের নাম থাকে না; থাকে সার্ভিস, ডেটাবেস, কিউ, ক্যাশ ও নেটওয়ার্ক সীমানা।



HLD লেখার সময় প্রতিটি তীরের পাশে প্রশ্ন করুন: এই সংযোগটি সিঙ্ক্রোনাস নাকি অ্যাসিঙ্ক্রোনাস? এটি ব্যর্থ হলে কী হয়? তথ্য কোন দিকে প্রবাহিত হয়? এই প্রশ্নগুলোর উত্তরই নকশার গুণ নির্ধারণ করে। উপরের ছবিতে পেমেন্ট ও স্টক আপডেট কিউয়ের মাধ্যমে অ্যাসিঙ্ক্রোনাস করা হয়েছে — যাতে পেমেন্ট গেটওয়ে ধীর হলেও অর্ডার নেওয়া বন্ধ না হয়। কিন্তু অর্ডার নিশ্চিত করার সিদ্ধান্ত সিঙ্ক্রোনাস ডেটাবেস লেনদেনে রাখা হয়েছে — কারণ টাকা ও অর্ডারের সামঞ্জস্য আপসের বিষয় নয়। এই পার্থক্যটিই HLD-র মূল সিদ্ধান্ত।

২.৬ Low-Level Design (LLD)

Low-Level Design: HLD-র একটি উপাদানকে ভেতর থেকে নকশা করা — ডেটা মডেল, API চুক্তি, ত্রুটি পরিচালনা, পুনরায় চেষ্টার নিয়ম, সিক্রেট ব্যবস্থাপনা। LLD ছাড়া HLD শুধু ছবি; LLD-ই বাস্তবায়নযোগ্য নকশা।

```
-- orders টেবিল (PostgreSQL) — LLD: ডেটা মডেল
CREATE TABLE orders (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL,
  status          TEXT NOT NULL CHECK (status IN
                    ('placed','paid','cancelled','shipped')),
  total_amount    NUMERIC(12,2) NOT NULL CHECK (total_amount > 0),
  idempotency_key TEXT UNIQUE NOT NULL,  -- একই অনুরোধ দুবার এলে একটিই অর্ডার
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_orders_user_created ON orders (user_id, created_at DESC);
CREATE INDEX idx_orders_status ON orders (status) WHERE status = 'placed';
```

Idempotency Key: একই অনুরোধ দুবার এলে দ্বিতীয়বার নতুন কাজ না করার ব্যবস্থা। ব্যবহারকারী পেমেন্ট বোতাম দুবার চাপলে বা নেটওয়ার্ক পুনরায় চেষ্টা করলে দুটি অর্ডার তৈরি হওয়া উচিত নয়। ক্লায়েন্ট প্রতিটি অনুরোধে একটি অনন্য কী পাঠায়; সার্ভার সেই কী দেখে বুঝতে পারে এটি নতুন নাকি পুনরাবৃত্তি। LLD-তে আরও থাকে: API-র প্রতিটি এন্ডপয়েন্টের অনুরোধ-উত্তরের আকৃতি, প্রতিটি ত্রুটি কোডের অর্থ, পুনরায় চেষ্টার সীমা (যেমন সর্বোচ্চ ৩ বার, Exponential Backoff-সহ), এবং কোন তথ্য লগে লেখা যাবে না (যেমন কার্ড নম্বর — কখনোই নয়)।

২.৭ Architecture Decision Records (ADR)

স্থাপত্যের সিদ্ধান্তগুলো ভুলে যাওয়া যায় না — কারণ ছয় মাস পর কেউ একজন এসে জিজ্ঞেস করবে “আমরা কেন PostgreSQL বেছে নিয়েছিলাম, MongoDB নয়?” উত্তর যদি কারও স্মৃতিতে থাকে, তাহলে সংস্থা সেই স্মৃতির উপর নির্ভরশীল। **ADR** হলো একটি ছোট, অপরিবর্তনীয় নথি যা প্রতিটি গুরুত্বপূর্ণ সিদ্ধান্তের প্রেক্ষাপট, বিবেচিত বিকল্প ও পরিণতি লিখে রাখে।

```
# ADR-0007 — অর্ডার ইভেন্টের জন্য Message Queue ব্যবহার
#
# অবস্থা: গৃহীত (Accepted) — 2026-10-18 — প্রস্তাবক: প্ল্যাটফর্ম দল
#
# প্রেক্ষাপট:
#   পেমেন্ট গেটওয়ে মাঝে মাঝে ধীর হয় (2–5 সেকেন্ড)। অর্ডার API যদি
#   গেটওয়ের উত্তরের জন্য সিঙ্ক্রোনাসভাবে অপেক্ষা করে, তাহলে সর্বোচ্চ
#   লোডে ব্যবহারকারী সময় শেষ (Timeout) দেখবে।
#
# সিদ্ধান্ত:
#   অর্ডার তৈরি ও পেমেন্ট প্রক্রিয়াকরণ আলাদা করা হবে। অর্ডার API
```

```
#   অর্ডার সংরক্ষণ করে একটি ইভেন্ট কিউতে প্রকাশ করবে; আলাদা Worker
#   সেই ইভেন্ট পড়ে পেমেন্ট প্রক্রিয়া করবে।
#
# বিবেচিত বিকল্প:
#   1. সিঙ্ক্রোনাস কল — সহজ, কিন্তু গেটওয়ের ধীরগতি সরাসরি
#      ব্যবহারকারীকে আঘাত করে। বাতিল।
#   2. ডেটাবেস পোলিং — Worker ডেটাবেসে নতুন অর্ডার খুঁজবে। সহজ,
#      কিন্তু দেরি হয় ও ডেটাবেসে চাপ পড়ে। বাতিল।
#
# পরিণতি:
#   ইতিবাচক: অর্ডার API দ্রুত থাকে; পেমেন্ট ব্যর্থ হলে পুনরায় চেষ্টা
#     করা যায়; Worker আলাদাভাবে স্কেল করা যায়।
#   নেতিবাচক: কিউ একটি নতুন ব্যর্থতার বিন্দু; ইভেন্টের ক্রম ও
#     দ্বৈত-প্রক্রিয়াকরণ সামলাতে হবে; পর্যবেক্ষণ জটিল হয়।
```

ADR-এর নিয়ম: সংক্ষিপ্ত রাখুন (এক পৃষ্ঠা), সংখ্যা দিন, কখনো মুছবেন না — সিদ্ধান্ত বদলালে নতুন ADR লিখুন যা পুরোনোটিকে “অতিক্রান্ত” (Superseded) ঘোষণা করে। এভাবে সিদ্ধান্তের ইতিহাস একটি শৃঙ্খলে থাকে।

২.৮ ধারণক্ষমতা অনুমান (Capacity Estimation)

নকশা সংখ্যায় যাচাই না করলে তা অনুমানই থেকে যায়। ধারণক্ষমতা অনুমানের জন্য পেছন থেকে হিসাব করুন — ব্যবসায়িক সংখ্যা থেকে প্রযুক্তিগত সংখ্যায়।

```
# TechNova — ধারণক্ষমতা অনুমান (ফ্ল্যাশ সেল, সর্বোচ্চ লোড)
#
# ব্যবসায়িক অনুমান:
#   দৈনিক সক্রিয় ব্যবহারকারী (DAU):       100,000
#   ফ্ল্যাশ সেলে একসাথে সক্রিয়:            10%  = 10,000
#   তাদের মধ্যে অর্ডার দেবে:                20%  =  2,000 অর্ডার / 10 মিনিট
#   সর্বোচ্চ হার:                          ~200 অর্ডার / মিনিট
#   নিরাপত্তা গুণক (x3):                   600 অর্ডার / মিনিট নকশা-লক্ষ্য
#
# ডেটাবেস:
#   প্রতি অর্ডারে লেখা: অর্ডার 1 + আইটেম গড়ে 3 = 4টি সারি
#   লেখার হার: 600 x 4 / 60 = 40 লেখা / সেকেন্ড — PostgreSQL-এর জন্য
#   স্বচ্ছন্দ পরিসর। কিন্তু সংযোগ-সীমা দেখুন: প্রতি API রেপ্লিকা
#   10টি সংযোগ ধরলে, 10টি রেপ্লিকা = 100 সংযোগ। PgBouncer ছাড়া
#   ডেটাবেসের সীমা ছাড়িয়ে যাবে।
#
# স্টোরেজ:
#   প্রতি অর্ডার ~2 KB (হেডার + আইটেম) → 600 x 2 KB = 1.2 MB/মিনিট
#   এক বছরে ~50 GB — একক ডিস্কেই চলে, কিন্তু ব্যাকআপ ও প্রতিলিপি
#   আলাদা করে হিসাব করতে হবে।
```

এই হিসাবের মূল্য সংখ্যায় নয়, পদ্ধতিতে: প্রতিটি অনুমান লিখুন, যাতে ব্যবসায়িক দল বলতে পারে “DAU আসলে ২০০,০০০ হবে” — আর আপনি দশ মিনিটে নতুন হিসাব করতে পারেন। যে অনুমান লেখা হয়নি, তা ভুল প্রমাণিত হলেও কেউ জানতে পারে না।

২.৯ ঝুঁকি বিশ্লেষণ ও প্রশমন

প্রতিটি নকশার সাথে একটি **Risk Register** রাখুন — কী ভুল হতে পারে, তার সম্ভাবনা কত, প্রভাব কত, আর কী করলে প্রভাব কমবে।

ঝুঁকি	সম্ভাবনা	প্রভাব	প্রশমন
পেমেন্ট গেটওয়ে বন্ধ	মাঝারি	বেশি — অর্ডার নেওয়া বন্ধ	কিউতে ইভেন্ট জমা রাখা; গেটওয়ে ফিরলে পুনরায় চেষ্টা; বিকল্প গেটওয়ে চুক্তি
ডেটাবেস প্রাইমারি বন্ধ	কম	বেশি — সব লেখা বন্ধ	স্বয়ংক্রিয় Failover; প্রতিলিপি; নিয়মিত Failover মহড়া
ফ্ল্যাশ সেলে ১০ গুণ লোড	মাঝারি	মাঝারি — ধীরগতি	আগে থেকে স্কেল; Rate Limit; ক্যাশ; লোড পরীক্ষা
গোপন তথ্য ফাঁস	কম	গুরুতর — আইনি ও সুনামের ক্ষতি	ভল্টে সিক্রেট; কোডে স্ক্যান; Least Privilege
একটি ভুল ডেপ্লয় সব নষ্ট করে	মাঝারি	বেশি	Canary ডেপ্লয়; স্বয়ংক্রিয় Rollback; Feature Flag

> **WARNING:** ঝুঁকি নিবন্ধন লিখে ড্রয়ারে রাখলে কোনো লাভ নেই। প্রতিটি ঝুঁকির একজন মালিক থাকতে হবে এবং একটি পরীক্ষার তারিখ থাকতে হবে — যেমন “Failover মহড়া প্রতি ত্রৈমাসিকে”। যে প্রশমন কখনো পরীক্ষা করা হয়নি, তা প্রশমন নয়, আশা মাত্র।

২.১০ স্থাপত্য পর্যালোচনা (Architecture Review)

নকশা একা লিখে প্রোডাকশনে পাঠানো উচিত নয়। একটি হালকা পর্যালোচনা প্রক্রিয়া রাখুন: নকশা নথি (HLD + ADR + ধারণক্ষমতা + ঝুঁকি) Pull Request-এর মতো করে পর্যালোচনার জন্য দিন; পর্যালোচকরা হবেন ভিন্ন দলের অন্তত দুজন প্রকৌশলী ও একজন নিরাপত্তা-সচেতন ব্যক্তি। পর্যালোচনার প্রশ্নগুলো নির্দিষ্ট রাখুন: “এই নকশায় একক ব্যর্থতার বিন্দু (Single Point of Failure) কোথায়?”, “কোন অনুমানটি ভুল হলে নকশা ভেঙে পড়বে?”, “Rollback পরিকল্পনা কী?”, “খরচের হিসাব কোথায়?” পর্যালোচনা মানে অনুমোদনের বাধা নয় — মানে অন্ধ-বিন্দু ধরা। পর্যালোচনা শেষে সিদ্ধান্ত ও আপত্তিগুলো নথিতে লিখে রাখুন; পরে কেউ যেন আবার একই বিতর্ক শুরু না করে।

২.১১ TechNova দৃশ্যকল্প: অর্ডার সিস্টেমের সম্পূর্ণ নকশা-নথি

এই অধ্যায়ের সব উপাদান একত্রে TechNova-র অর্ডার সিস্টেমের নকশা-নথির কাঠামোটি এই রকম:

```
docs/architecture/orders-system/
  01-requirements.md      # FR + NFR, পরিমাপযোগ্য সংখ্যাসহ
  02-hld.md               # উপাদান, সংযোগ, তথ্য-প্রবাহ (চিত্রসহ)
```

```
03-lld-orders-api.md    # API চুক্তি, ডেটা মডেল, ত্রুটি-কোড
04-capacity.md          # ধারণক্ষমতা অনুমান, অনুমানগুলো লিখিত
05-risks.md             # ঝুঁকি নিবন্ধন, মালিক ও পরীক্ষার তারিখসহ
adr/
  ADR-0007-queue-for-payment.md
  ADR-0008-postgres-for-orders.md
  ADR-0009-redis-idempotency.md
```

এই নথিটি জীবন্ত: প্রতিটি বড় পরিবর্তনে ADR যোগ হয়, প্রতিটি ত্রৈমাসিকে ধারণক্ষমতা পুনঃহিসাব হয়। নতুন প্রকৌশলী যোগ দিলে এই ফোল্ডারটিই তাঁর প্রথম পড়ার তালিকা — কারণ এখানেই লেখা আছে সিস্টেমটি কেন এমন।

২.১২ নিরাপত্তা ও খরচ বিবেচনা

নকশার পর্যায়ে নিরাপত্তা মানে **Threat Modeling** — আক্রমণকারী কীভাবে সিস্টেমে ঢুকতে পারে, তা আগেই ভাবা। প্রতিটি উপাদানের জন্য জিজ্ঞেস করুন: কে এতে প্রবেশ করতে পারে? তথ্য কোথায় সংরক্ষিত ও স্থানান্তরিত হয়? কোন তথ্য সবচেয়ে সংবেদনশীল? অর্ডার সিস্টেমে সবচেয়ে সংবেদনশীল তথ্য পেমেন্টের বিবরণ — তাই পেমেন্ট তথ্য TechNova-র ডেটাবেসে সংরক্ষণ না করে গেটওয়ের কাছে রাখাই নিরাপদ নকশা; TechNova রাখে শুধু একটি টোকেন। খরচের দিক থেকে: HLD-তে প্রতিটি উপাদানের পাশে মাসিক খরচের আনুমানিক হিসাব রাখুন — ডেটাবেস প্রতিলিপি, কিউ, ক্যাশ, পর্যবেক্ষণ — যাতে “৯৯.৯৯% চাই” বলার আগে তার দাম দেখা যায়। অনেক দল ৯৯.৯৯% চেয়ে বসে, অথচ তাদের ব্যবসা ৯৯.৯%-এই চলে; পার্থক্যটি শুধু খরচ।

২.১৩ সাধারণ ভুল

- **সংখ্যা ছাড়া NFR লেখা:** “সিস্টেম দ্রুত হবে” — এটি পরীক্ষা করা যায় না, তাই পূরণও হয় না।
- **HLD ছাড়া কোড লেখা:** উপাদানের সীমানা না ঠিক করে কোড লিখলে পরে সীমানা বদলাতে পুরো কোড বদলাতে হয়।
- **ADR না লেখা:** সিদ্ধান্তের কারণ ভুলে গেলে পরের দল সেই সিদ্ধান্ত উল্টে দেয় — একই ভুল আবার করে।
- **অনুমান না লেখা:** ধারণক্ষমতার অনুমান লিখিত না থাকলে ব্যবসা বদলালে নকশা নীরবে ভেঙে পড়ে।
- **Happy Path-এর নকশা:** শুধু সব ঠিক থাকলে কী হবে, তা ভাবা — ব্যর্থতার নকশা না করা।
- **পর্যালোচনাকে অনুমোদন-বাধা বানানো:** পর্যালোচনা ধীর হলে দল তা এড়াতে শেখে।

২.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
নকশা সভায় শুধু টুল নিয়ে বিতর্ক	প্রয়োজন লিখিত নয়	আগে FR/NFR লিখুন; টুল পরে আসবে।
ছয় মাস পর কেউ জানে না কেন এমন করা হয়েছিল	ADR নেই	এখন থেকেই প্রতিটি বড় সিদ্ধান্তে ADR লিখুন।
লোড বাড়লে সিস্টেম অপ্রত্যাশিতভাবে ভাঙে	ধারণক্ষমতা অনুমান নেই	ব্যবসায়িক সংখ্যা থেকে হিসাব করুন; লোড পরীক্ষা করুন।
একটি উপাদান বদলালে সব ভাঙে	সীমানা অস্পষ্ট	HLD-তে উপাদান ও চুক্তি স্পষ্ট করুন; LLD-তে চুক্তি পরীক্ষা লিখুন।

পর্যালোচনায় বারবার একই আপত্তি	আগের পর্যালোচনার সিদ্ধান্ত লেখা নেই	পর্যালোচনার সিদ্ধান্ত নথিতে লিপিবদ্ধ করুন।

২.১৫ অধ্যায়ের সারাংশ

প্রোডাকশন সিস্টেম ডিজাইন শুরু হয় পরিমাপযোগ্য প্রয়োজন দিয়ে — কার্যকরী প্রয়োজন বলে সিস্টেম কী করবে, অকার্যকরী প্রয়োজন বলে কত ভালোভাবে করবে। HLD বড় ছবি দেয়, LLD বাস্তবায়নযোগ্য বিস্তারিত দেয়, ADR সিদ্ধান্তের ইতিহাস সংরক্ষণ করে, ধারণক্ষমতা অনুমান নকশাকে সংখ্যায় যাচাই করে, আর ঝুঁকি নিবন্ধন ও স্থাপত্য পর্যালোচনা অন্ধ-বিন্দু ধরে। এই নথিগুলো একসাথে একটি সিস্টেমের "কেন" — যা কোড পড়ে জানা যায় না।

২.১৬ জ্ঞান যাচাই

1. Functional ও Non-Functional প্রয়োজনীয়তার পার্থক্য উদাহরণসহ ব্যাখ্যা করুন।
2. "সিস্টেমটি দ্রুত হবে" — কেন এটি ভালো NFR নয়? একে পরিমাপযোগ্য করুন।
3. HLD ও LLD-র পার্থক্য কী? কোনটি আগে লেখা হয় এবং কেন?
4. ADR কী? একটি ADR-তে কোন কোন অংশ থাকা উচিত?
5. Availability ৯৯.৯% মানে মাসে কত মিনিট ডাউনটাইম? হিসাব করে দেখান।
6. ধারণক্ষমতা অনুমানে "পেছন থেকে হিসাব" বলতে কী বোঝায়? উদাহরণ দিন।
7. Idempotency Key কেন প্রয়োজন? একটি বাস্তব দৃশ্য দিয়ে ব্যাখ্যা করুন।
8. ঝুঁকি নিবন্ধনে প্রতিটি ঝুঁকির মালিক থাকা কেন জরুরি?
9. স্থাপত্য পর্যালোচনায় কোন চারটি প্রশ্ন অবশ্যই করা উচিত?
10. Threat Modeling নকশার কোন পর্যায়ে করা উচিত এবং কেন?
11. একটি সিদ্ধান্ত বদলালে পুরোনো ADR মুছে ফেলা উচিত নয় কেন?
12. Happy Path-এর নকশা কেন বিপজ্জনক? ব্যর্থতার নকশা কীভাবে করবেন?

২.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র অর্ডার সিস্টেমের জন্য FR ও NFR-এর একটি সম্পূর্ণ তালিকা লিখুন — প্রতিটি NFR পরিমাপযোগ্য সংখ্যাসহ।
2. একটি পরিচিত সিস্টেমের (যেমন আপনার ইমেইল বা একটি খাবার ডেলিভারি অ্যাপ) HLD আঁকুন এবং ধারণক্ষমতা অনুমান করুন — DAU থেকে অনুরোধ/সেকেন্ড পর্যন্ত।
3. আপনার সাম্প্রতিক একটি প্রযুক্তিগত সিদ্ধান্তের জন্য একটি ADR লিখুন — প্রেক্ষাপট, বিকল্প, সিদ্ধান্ত ও পরিণতিসহ।

অধ্যায় ৩ • PART I — Advanced DevOps & Enterprise Architecture

Chapter 3 — Microservices and Distributed Systems

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Monolith**, **Modular Monolith** ও **Microservices**-এর মধ্যে সচেতনভাবে বেছে নিতে; বিতরণকৃত সিস্টেমের (Distributed System) মৌলিক সত্যগুলো ব্যাখ্যা করতে; সার্ভিসের সীমানা নির্ধারণ করতে; সিঙ্ক্রোনাস ও অ্যাসিঙ্ক্রোনাস যোগাযোগের ট্রেড-অফ বিশ্লেষণ করতে; **Event-Driven Architecture** ও মেসেজ কিউ নকশা করতে; ডেটা সামঞ্জস্য (Consistency) ও বিতরণকৃত লেনদেনের (Distributed Transaction) সীমা বুঝতে; এবং ব্যর্থতা, পুনরায় চেষ্টা ও পর্যবেক্ষণের মাধ্যমে বিতরণকৃত সিস্টেমকে নির্ভরযোগ্য করতে।

- তিনটি স্থাপত্য ধাঁচের খরচ ও সুবিধা তুলনা করা।
- Domain-Driven Design-এর Bounded Context দিয়ে সার্ভিস সীমানা আঁকা।
- REST, gRPC ও মেসেজ-ভিত্তিক যোগাযোগের মধ্যে বেছে নেওয়া।
- At-Least-Once ও At-Most-Once ডেলিভারির অর্থ ব্যাখ্যা করা।
- Saga প্যাটার্ন দিয়ে বিতরণকৃত লেনদেন নকশা করা।
- Idempotency, Outbox ও Dead Letter Queue প্রয়োগ করা।

৩.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

মাইক্রোসার্ভিস আজকের সবচেয়ে বেশি ভুল বোঝা ধারণাগুলোর একটি। অনেক দল মনে করে মাইক্রোসার্ভিস মানেই আধুনিকতা — আর তারপর আবিষ্কার করে যে দশটি ছোট সার্ভিস চালানো একটি বড় সার্ভিস চালানোর চেয়ে দশ গুণ কঠিন: নেটওয়ার্ক ব্যর্থ হয়, তথ্য অমিল হয়, একটি অনুরোধের পথ খুঁজে পাওয়া কঠিন হয়, এবং ডেপ্লয় সমন্বয় করা দুঃস্বপ্ন হয়। আবার, সঠিক প্রেক্ষাপটে মাইক্রোসার্ভিস সত্যিই মুক্তি দেয় — যখন দলগুলো স্বাধীনভাবে ডেপ্লয় করতে চায়, যখন একটি অংশের স্কেল অন্য অংশের চেয়ে আলাদা, যখন প্রযুক্তিগত বৈচিত্র্য দরকার। এই অধ্যায়ের লক্ষ্য আপনাকে “মাইক্রোসার্ভিস ভালো না খারাপ” — এই ভুল প্রশ্ন থেকে বের করে “কোন প্রেক্ষাপটে কোন ধাঁচ সঠিক” — এই সঠিক প্রশ্নে নিয়ে আসা।

CONCEPT: Distributed System মানে এমন একটি সিস্টেম যেখানে উপাদানগুলো আলাদা মেশিনে চলে ও নেটওয়ার্ক দিয়ে কথা বলে — আর নেটওয়ার্ক অবিশ্বস্ত। একটি ফাংশন কল ব্যর্থ হলে আপনি জানেন এটি ব্যর্থ হয়েছে; একটি নেটওয়ার্ক কল ব্যর্থ হলে আপনি অনেক সময় জানতেই পারেন না — অনুরোধটি পৌঁছেছে কি না, কাজটি হয়েছে কি না। এই অনিশ্চয়তাই বিতরণকৃত সিস্টেমের সমস্ত জটিলতার মূল।

REAL WORLD: TechNova-র প্রথম মাইক্রোসার্ভিস প্রচেষ্টায় অর্ডার সার্ভিস সরাসরি পেমেন্ট সার্ভিসকে সিঙ্ক্রোনাসভাবে ডাকত, পেমেন্ট সার্ভিস ডাকত ইনভেন্টরি সার্ভিসকে, ইনভেন্টরি ডাকত নোটিফিকেশন সার্ভিসকে। একদিন নোটিফিকেশন সার্ভিস ধীর হলো — আর পুরো অর্ডার প্রক্রিয়া থেমে গেল। একটি অপ্রয়োজনীয় নির্ভরশীলতা পুরো শৃঙ্খলকে ভেঙে দিয়েছিল। সমাধান ছিল সরল: নোটিফিকেশনকে কিউয়ের মাধ্যমে অ্যাসিঙ্ক্রোনাস করা — ব্যবহারকারীকে ইমেইল পেতে দুই মিনিট দেরি হলেও অর্ডার যেন না থামে।

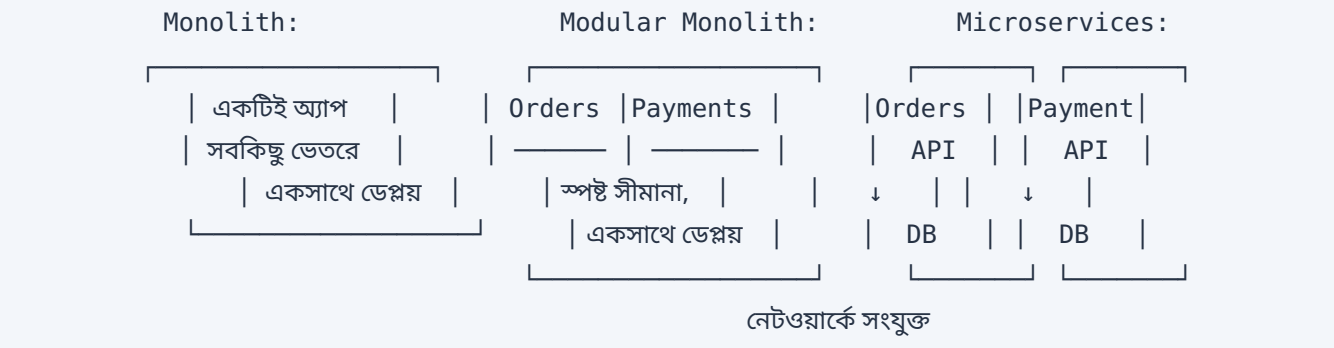
৩.৩ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার HTTP ও REST API-র ধারণা, ডেটাবেস লেনদেনের **ACID** বৈশিষ্ট্য (Atomicity, Consistency, Isolation, Durability), Docker কন্টেইনার ও Kubernetes-এ সার্ভিস কীভাবে চলে তার ধারণা, এবং অধ্যায় ২-এর HLD/LLD ও ADR জানা থাকতে হবে। মনে রাখুন: ACID লেনদেন একটি ডেটাবেসের ভেতরে শক্তিশালী নিশ্চয়তা দেয়; কিন্তু দুটি আলাদা ডেটাবেসের মধ্যে সেই একই নিশ্চয়তা পাওয়া যায় না — আর এখানেই বিতরণকৃত সিস্টেমের কঠিন সত্যগুলো শুরু হয়। আপনি যদি **CAP Theorem** শব্দটি আগে শুনে থাকেন, ভালো; না শুনলেও এই অধ্যায়ে তা প্রথম নীতি থেকে ব্যাখ্যা করা হবে।

৩.৪ Monolith বনাম Modular Monolith বনাম Microservices

তিনটি ধাঁচের মধ্যে বেছে নেওয়া মানে তিনটি ভিন্ন ধরনের জটিলতার মধ্যে বেছে নেওয়া।

ধাঁচ	কী	সুবিধা	অসুবিধা	কখন সঠিক
Monolith	একটি কোডবেস, একটি ডেপ্লয়যোগ্য একক	সহজ, দ্রুত শুরু, লেনদেন সহজ, ডিবাগ সহজ	বড় হলে ধীর ডেপ্লয়, একটি ভুলে সব ভাঙে	ছোট দল, অনিশ্চিত পণ্য, শুরুতে প্রায় সবসময়
Modular Monolith	একটি ডেপ্লয়, কিন্তু ভেতরে স্পষ্ট মডিউল ও সীমানা	মনোলিথের সহজতা + পরিষ্কার সীমানা; পরে ভাঙা সহজ	শৃঙ্খলা দরকার; মডিউল ভেঙে গেলে মনোলিথই থাকে	বেশিরভাগ মাঝারি দলের জন্য সেরা শুরু
Microservices	অনেক ছোট সার্ভিস, প্রতিটি নিজের ডেটাবেসসহ	স্বাধীন ডেপ্লয় ও স্কেল, দলীয় স্বাধীনতা	নেটওয়ার্ক জটিলতা, ডেটা সামঞ্জস্য, পরিচালন খরচ	বড় দল, স্বাধীন স্কেল প্রয়োজন, স্পষ্ট সীমানা



সবচেয়ে গুরুত্বপূর্ণ পরামর্শ: **শুরুতে Modular Monolith বেছে নিন**। ভেতরে মডিউলের সীমানা কঠোরভাবে মানুন — মডিউলগুলো যেন একে অপরের ডেটাবেসে সরাসরি হাত না দেয়, শুধু স্পষ্ট ইন্টারফেস দিয়ে কথা বলে। তাহলে পরে, যখন সত্যিই প্রয়োজন হবে, একটি মডিউলকে আলাদা সার্ভিসে বের করা সহজ হবে। আর যদি কখনো প্রয়োজন না হয়, তাহলে আপনি অযথা জটিলতা কেনেননি। Amazon-এর মতো বড় কোম্পানিও এই পথে হেঁটেছে — Prime Video দল একসময় মাইক্রোসার্ভিস থেকে মনোলিথে ফিরে গিয়ে খরচ কমিয়েছিল। সেটি পশ্চাৎপদতা নয়; সেটি পরিপক্ব সিদ্ধান্ত।

> **WARNING:** “মাইক্রোসার্ভিসে গেলে দল স্বাধীনভাবে ডেপ্লয় করতে পারবে” — এই প্রতিশ্রুতি তখনই সত্যি হয় যখন সার্ভিসের সীমানা সঠিক হয়। ভুল সীমানায় ভাঙলে প্রতিটি ডেপ্লয়ে অন্য সার্ভিসের সাথে সমন্বয় লাগে — তখন আপনি মনোলিথের অসুবিধা ও মাইক্রোসার্ভিসের অসুবিধা দুটোই পান, কোনো সুবিধা ছাড়াই। একে বলা হয় **Distributed Monolith** — সবচেয়ে খারাপ অবস্থা।

৩.৫ বিতরণকৃত সিস্টেমের মৌলিক সত্য

৩.৫.১ নেটওয়ার্ক অবিশ্বস্ত

নেটওয়ার্কে চারটি সম্ভাবনা থাকে: অনুরোধ পৌঁছায় ও উত্তর আসে; অনুরোধ পৌঁছায় না; অনুরোধ পৌঁছায় কিন্তু উত্তর হারায়; উত্তর আসে কিন্তু দেরিতে। তৃতীয় ক্ষেত্রটি সবচেয়ে বিপজ্জনক — আপনি জানেন না কাজটি হয়েছে কি না। তাই প্রতিটি গুরুত্বপূর্ণ অপারেশনকে **Idempotent** করতে হয়: একই অনুরোধ দুবার এলে দ্বিতীয়বার যেন কোনো নতুন প্রভাব না পড়ে।

৩.৫.২ CAP Theorem — তিনটির মধ্যে দুটি

CAP Theorem: একটি বিতরণকৃত ডেটা সিস্টেম একসাথে তিনটি নিশ্চয়তা দিতে পারে না — **Consistency** (সবাই একই তথ্য দেখবে), **Availability** (প্রতিটি অনুরোধে উত্তর আসবে), ও **Partition Tolerance** (নেটওয়ার্ক ভেঙে গেলেও সিস্টেম চলবে)। নেটওয়ার্ক বিভাজন (Partition) যখন ঘটে, তখন বেছে নিতে হয়: সামঞ্জস্য ধরে রাখতে সেবা বন্ধ করবেন (CP), নাকি সেবা চালু রেখে অস্থায়ী অমিল মেনে নেবেন (AP)? বাস্তবে বেশিরভাগ সিস্টেম AP বেছে নেয় ও পরে সামঞ্জস্য ফিরিয়ে আনে — একে বলে **Eventual Consistency** বা চূড়ান্ত সামঞ্জস্য। উদাহরণ: অর্ডার দেওয়ার পর স্টক সংখ্যা কয়েক সেকেন্ডের জন্য পুরোনো দেখাতে পারে; কিন্তু পেমেন্ট নিশ্চিত করার সময় স্টক আবার যাচাই করা হয়।

৩.৫.৩ সময়ের কোনো একক সত্য নেই

দুটি মেশিনের ঘড়ি কখনোই পুরোপুরি মিলবে না। তাই “আগে কী ঘটেছে” — এই প্রশ্নের উত্তর বিতরণকৃত সিস্টেমে জটিল। ইভেন্টের ক্রম নির্ধারণে **Logical Clock** বা সংস্করণ সংখ্যা (Version Number) ব্যবহার করুন, দেয়াল-ঘড়ির সময়ের উপর নির্ভর করবেন না।

৩.৬ সার্ভিস সীমানা: Domain-Driven Design

সার্ভিস কোথায় ভাঙবেন — এই প্রশ্নের উত্তর প্রযুক্তিতে নয়, ব্যবসায়িক ডোমেইনে। **Domain-Driven Design (DDD)**-এর মূল ধারণা **Bounded Context**: একটি সীমানা যার ভেতরে একটি শব্দের একটি নির্দিষ্ট অর্থ থাকে। “অর্ডার” শব্দটি Commerce প্রেক্ষাপটে মানে কেনাকাটার অর্ডার; Logistics প্রেক্ষাপটে মানে পাঠানোর নির্দেশ। দুটি ভিন্ন অর্থকে একটি সার্ভিসে জোর করে ঢোকালে সার্ভিসটি দুই দলের টানাটানিতে ছিঁড়ে যাবে। সীমানা নির্ধারণের ব্যবহারিক পরীক্ষা: দুটি ফিচার কি একই কারণে বদলায়? একই দল কি দুটোর মালিক? একটির ডেটা কি অন্যটি ছাড়া অর্থহীন? যদি উত্তর “না” হয়, তাহলে তারা আলাদা সীমানার প্রার্থী। আর একটি সতর্কতা: ডেটাবেস টেবিল দেখে সার্ভিস ভাঙবেন না; ব্যবসায়িক সক্ষমতা দেখে ভাঙুন।

৩.৭ যোগাযোগ: সিঙ্ক্রোনাস বনাম অ্যাসিঙ্ক্রোনাস

ধরন	উদাহরণ	কখন ব্যবহার করবেন	সতর্কতা
সিঙ্ক্রোনাস (অপেক্ষা করে)	REST, gRPC	ব্যবহারকারী এখনই উত্তর চায়; সামঞ্জস্য জরুরি	নির্ভরশীলতা শৃঙ্খল; ধীর সার্ভিস সবাইকে ধীর করে

অ্যাসিঙ্ক্রোনাস (কিউয়ে ফেলে)	Message Queue, ইভেন্ট	কাজটি পরে হলেও চলে; লোডের ঢেউ সামলাতে	চূড়ান্ত সামঞ্জস্য; ডেলিভারি নিশ্চয়তা বুঝতে হবে

নিয়মটি সহজ: ব্যবহারকারী যা দেখছে তার জন্য সিঙ্ক্রোনাস; ব্যবহারকারী যা দেখছে না তার জন্য অ্যাসিঙ্ক্রোনাস। অর্ডার নেওয়া — সিঙ্ক্রোনাস (ব্যবহারকারী জানতে চায় অর্ডার হয়েছে কি না)। ইমেইল পাঠানো, রিপোর্ট তৈরি, স্টক পুনঃগণনা — অ্যাসিঙ্ক্রোনাস। **gRPC** হলো সার্ভিস-টু-সার্ভিস যোগাযোগের জন্য দ্রুত ও টাইপ-নিরাপদ প্রোটোকল; **REST** বাইরের ক্লায়েন্টের জন্য সহজ ও সর্বজনীন। দুটোই বৈধ — প্রেক্ষাপটই ঠিক করে।

```
# TechNova — সিঙ্ক্রোনাস কল (gRPC ধাঁচ, ধারণাগত)
# orders-api → inventory-api: "এই পণ্য কি স্টকে আছে?"
# উত্তর এখনই দরকার, নইলে অর্ডার নেওয়া যাবে না।
# Timeout: 500ms; ব্যর্থ হলে ব্যবহারকারীকে স্পষ্ট বার্তা।

# TechNova — অ্যাসিঙ্ক্রোনাস ইভেন্ট (কিউ ধাঁচ)
# orders-api → queue: OrderPlaced { order_id, items, total }
# payments-worker, inventory-worker, notification-worker —
# প্রত্যেকে নিজের গতিতে ইভেন্টটি পড়ে। অর্ডার API অপেক্ষা করে না।
```

৩.৮ Event-Driven Architecture ও Message Queue

Event-Driven Architecture: সার্ভিসগুলো একে অপরকে সরাসরি ডাকে না; তারা ঘটনা (Event) প্রকাশ করে, আর আগ্রহী সার্ভিসগুলো সেই ঘটনা শুনে প্রতিক্রিয়া জানায়। এর সুবিধা: প্রকাশক জানে না কে শুনছে — তাই নতুন শ্রোতা যোগ করা সহজ; একটি শ্রোতা বন্ধ থাকলেও প্রকাশক থামে না। কিন্তু এর খরচও আছে: ঘটনার ক্রম, দ্বৈত-প্রক্রিয়াকরণ ও ত্রুটি পরিচালনা ভাবতে হয়।

৩.৮.১ ডেলিভারি নিশ্চয়তা

- **At-Most-Once:** ঘটনা সর্বোচ্চ একবার পৌঁছাবে — হারিয়ে যেতে পারে, কিন্তু দুবার যাবে না। লগ বা মেট্রিকের মতো হারিয়ে গেলে চলে, এমন কাজে ঠিক আছে।
- **At-Least-Once:** ঘটনা অন্তত একবার পৌঁছাবে — হারাবে না, কিন্তু দুবার আসতে পারে। বেশিরভাগ কিউ এই নিশ্চয়তা দেয়। তাই শ্রোতাকে **Idempotent** হতে হয়: একই ইভেন্ট দুবার এলে দ্বিতীয়বার কোনো প্রভাব নয়।
- **Exactly-Once:** বিতরণকৃত সিস্টেমে সাধারণ অর্থে অসম্ভব — কারণ নেটওয়ার্কের অনিশ্চয়তা দূর করা যায় না। যা করা যায় তা হলো “কার্যকরভাবে ঠিক একবার” (Effectively-Once): Idempotency Key দিয়ে দ্বৈত-প্রক্রিয়াকরণ আটকানো।

৩.৮.২ Outbox Pattern ও Dead Letter Queue

একটি ক্লাসিক সমস্যা: ডেটাবেসে অর্ডার সংরক্ষণ করলেন, কিন্তু কিউতে ইভেন্ট পাঠাতে গিয়ে ব্যর্থ হলেন — এখন ডেটাবেস ও কিউ অমিল। **Outbox Pattern**-এর সমাধান: ইভেন্টটি প্রথমে ডেটাবেসের একটি Outbox টেবিলে একই লেনদেনে লেখা হয়; একটি আলাদা Relay সেই টেবিল পড়ে কিউতে পাঠায়। এতে ডেটাবেস ও ইভেন্ট একই লেনদেনে সামঞ্জস্যপূর্ণ থাকে। আর যে ইভেন্ট বারবার ব্যর্থ হয়, তা **Dead Letter Queue (DLQ)**-তে যায় — যেখানে মানুষ পরে পরীক্ষা করে সিদ্ধান্ত নেয়, আর মূল কিউ আটকে থাকে না।

```
# Outbox Pattern — ধারণাগত প্রবাহ
# 1. BEGIN TRANSACTION
#      INSERT INTO orders (...) VALUES (...);
#      INSERT INTO outbox (event_type, payload)
#             VALUES ('OrderPlaced', '{...}');
#    COMMIT;    -- দুটি লেখা একসাথে হয়, নইলে কোনোটিই নয়
# 2. Outbox Relay: outbox টেবিল পড়ে → কিউতে প্রকাশ করে →
#  প্রকাশিত সারি চিহ্নিত করে।
# 3. Worker: কিউ থেকে ইভেন্ট পড়ে Idempotency Key যাচাই করে কাজ করে।
#  বারবার ব্যর্থ হলে → Dead Letter Queue-তে পাঠায়।
```

৩.৯ ডেটা সামঞ্জস্য ও বিতরণকৃত লেনদেন

একটি ডেটাবেসের ভেতরে ACID লেনদেন সহজ। দুটি সার্ভিসের মধ্যে ACID পাওয়া প্রায় অসম্ভব — **Two-Phase Commit (2PC)** তত্ত্বে আছে, বাস্তবে ধীর ও ভঙ্গুর বলে প্রায় ব্যবহার হয় না। বাস্তব সমাধান হলো **Saga Pattern**: একটি বড় লেনদেনকে ছোট ছোট স্থানীয় লেনদেনে ভাঙা, প্রতিটি ধাপের জন্য একটি ক্ষতিপূরণকারী ধাপ (Compensating Action) রাখা।

Saga — অর্ডার প্রক্রিয়া (TechNova)

ধাপ ১: অর্ডার তৈরি (orders) — ব্যর্থ হলে: কিছুই হয়নি
ধাপ ২: পেমেন্ট চার্জ (payments) — ব্যর্থ হলে: অর্ডার বাতিল
ধাপ ৩: স্টক কমানো (inventory) — ব্যর্থ হলে: পেমেন্ট ফেরত (Refund)
ধাপ ৪: শিপমেন্ট বুক (logistics) — ব্যর্থ হলে: স্টক ফেরত + পেমেন্ট ফেরত

প্রতিটি ধাপের ক্ষতিপূরণকারী ধাপ আগে থেকে লেখা থাকে।
Saga ব্যর্থ হলেও সিস্টেম সামঞ্জস্যপূর্ণ থাকে — তবে
মাঝখানে কিছু সময় “অস্থায়ী” অবস্থা দেখা যেতে পারে।

Saga-র দুটি রূপ আছে: **Choreography** (প্রতিটি সার্ভিস ইভেন্ট শুনে পরের ধাপ করে — বিকেন্দ্রীভূত, কিন্তু প্রবাহ বোঝা কঠিন) ও **Orchestration** (একটি Saga Orchestrator পুরো প্রবাহ চালায় — কেন্দ্রীভূত, কিন্তু বোঝা ও ডিবাগ সহজ)। জটিল প্রবাহে Orchestration বেছে নিন; সহজ প্রবাহে Choreography যথেষ্ট।

৩.১০ ব্যর্থতা পরিচালনা: পুনরায় চেষ্টা, Timeout ও Idempotency

বিতরণকৃত সিস্টেমে ব্যর্থতা স্বাভাবিক ঘটনা — তাই প্রতিটি কলে তিনটি নিয়ম মানুন: (১) **Timeout** বেঁধে দিন — কোনো কল যেন অনন্তকাল না ঝোলে; (২) পুনরায় চেষ্টা (**Retry**) করুন, কিন্তু **Exponential Backoff** ও **Jitter**-সহ — নইলে সব ক্লায়েন্ট একসাথে আবার চেষ্টা করে বিপদগ্রস্ত সার্ভিসকে আরও চাপ দেবে (একে বলে **Retry Storm**); (৩) শুধু নিরাপদ অপারেশনেই পুনরায় চেষ্টা করুন — পেমেন্ট চার্জের মতো অপারেশনে Idempotency Key ছাড়া কখনো পুনরায় চেষ্টা করবেন না।

```
# পুনরায় চেষ্টার নিয়ম (TechNova ক্লায়েন্ট লাইব্রেরি — ধারণাগত)
#   সর্বোচ্চ চেষ্টা: 3 বার
#   বিলম্ব: 100ms → 200ms → 400ms (Exponential Backoff)
#     Jitter: প্রতিটি বিলম্বে এলোমেলো ±50ms যোগ করুন,
#         যাতে সব ক্লায়েন্ট একসাথে আবার না আসে
#   শুধুমাত্র: Timeout ও 5xx ত্রুটিতে পুনরায় চেষ্টা
#   কখনো নয়: 4xx ত্রুটিতে (ক্লায়েন্টের ভুল, আবার চেষ্টা করে লাভ নেই)
#   পেমেন্টে: সবসময় Idempotency-Key হেডার পাঠান
```

৩.১১ পর্যবেক্ষণযোগ্যতা: বিতরণকৃত ট্রেসিং

একটি অনুরোধ যখন পাঁচটি সার্ভিস ঘুরে আসে, তখন একটি সার্ভিসের লগ দেখে কিছুই বোঝা যায় না। দরকার **Distributed Tracing**: প্রতিটি অনুরোধের একটি Trace ID থাকে, যা প্রতিটি সার্ভিসে ছড়িয়ে পড়ে; প্রতিটি ধাপ একটি Span হিসেবে রেকর্ড হয়। OpenTelemetry এই কাজের মানদণ্ড। সাথে তিনটি সোনালি সংকেত (Golden Signals) মাপুন: Latency, Traffic, Errors, Saturation। ট্রেসিং ছাড়া মাইক্রোসার্ভিস পরিচালনা করা মানে অন্ধকারে হাতড়ানো।

৩.১২ TechNova দৃশ্যকল্প: মডুলার মনোলিথ থেকে ধীরে ধীরে ভাঙা

TechNova-র সিদ্ধান্ত: আজই মাইক্রোসার্ভিসে যাওয়া নয়; বরং মডুলার মনোলিথকে শক্ত করা, এবং শুধু একটি সার্ভিস — নোটিফিকেশন — আলাদা করা, কারণ তার স্কেল ও ব্যর্থতার ধাঁচ বাকিদের থেকে আলাদা। পরিকল্পনা:

```
# ধাপ ১ — মডিউল সীমানা কঠোর করা
#   orders, payments, inventory, notification — প্রতিটি মডিউল
#   নিজের ডেটা নিজে রাখে; অন্য মডিউলের টেবিলে সরাসরি SQL নয়।
#   মডিউলের মধ্যে যোগাযোগ শুধু স্পষ্ট ইন্টারফেস দিয়ে।
#   লঙ্ঘন ধরতে CI-তে ArchUnit-ধাঁচের সীমানা পরীক্ষা।

# ধাপ ২ — নোটিফিকেশন আলাদা করা
#   নোটিফিকেশন মডিউল এখন OrderPlaced ইভেন্ট শোনে (কিউ থেকে)।
#   প্রথমে একই প্রক্রিয়ায় ইভেন্ট-শ্রোতা হিসেবে চালান,
#   তারপর আলাদা ডেপ্লয়যোগ্য সার্ভিসে বের করুন।
#   ইন্টারফেস বদলায় না — তাই বাকি সিস্টেম টের পায় না।

# ধাপ ৩ — পর্যবেক্ষণ প্রস্তুত করা
#   আলাদা করার আগেই Trace ID ও কেন্দ্রীয় লগ চালু করুন।
#   নইলে আলাদা করার পর প্রথম ঘটনাতেই অন্ধ হয়ে যাবেন।
```

```
$ kubectl get pods -n technova -l app=notification-api
NAME                                READY   STATUS    RESTARTS
notification-api-7d9f4b6c8-x2k4p    1/1     Running   0
notification-api-7d9f4b6c8-m9q7w    1/1     Running   0
```

৩.১৩ নিরাপত্তা ও খরচ বিবেচনা

প্রতিটি সার্ভিস একটি নতুন আক্রমণ-তল (Attack Surface): প্রতিটি এন্ডপয়েন্টে প্রমাণীকরণ ও অনুমোদন লাগে, সার্ভিস-টু-সার্ভিস যোগাযোগে mTLS লাগে, আর প্রতিটি ডেটাবেসে আলাদা পরিচয়পত্র লাগে। সার্ভিস বাড়লে গোপন তথ্যের সংখ্যাও বাড়ে — তাই সিক্রেট ব্যবস্থাপনা কেন্দ্রীয়ভাবে করুন। খরচের দিক থেকে: প্রতিটি সার্ভিসের ন্যূনতম সম্পদ (CPU/মেমরি), পর্যবেক্ষণ, লগ সংরক্ষণ ও পাইপলাইন — সব মিলিয়ে মাইক্রোসার্ভিসের "প্রতি-সার্ভিস কর" আছে। দশটি সার্ভিস মানে দশ গুণ কর। এই করের যৌক্তিকতা তখনই হয় যখন সার্ভিসগুলো সত্যিই স্বাধীনভাবে স্কেল বা ডেপ্লয় হয়।

> **TROUBLE:** "সার্ভিস A সার্ভিস B-কে ডাকছে, B সার্ভিস C-কে ডাকছে, C আবার A-কে ডাকছে" — চক্রাকার নির্ভরশীলতা (Circular Dependency) বিতরণকৃত সিস্টেমের একটি ক্লাসিক বিপদ। এটি সাধারণত ভুল সীমানার লক্ষণ। সমাধান: নির্ভরশীলতার দিক একমুখী করুন, অথবা ইভেন্ট দিয়ে চক্রটি ভাঙুন।

৩.১৪ সাধারণ ভুল

- **শুরুতেই মাইক্রোসার্ভিস:** সীমানা না জেনে ভাঙা — পরে সীমানা বদলাতে সার্ভিস বদলাতে হয়।
- **Distributed Monolith:** সার্ভিস আলাদা, কিন্তু ডেপ্লয় একসাথে করতে হয় — দুটো জগতের খারাপ দিক।
- **সিঙ্ক্রোনাস শৃঙ্খল:** পাঁচটি সার্ভিস একে অপরকে সিঙ্ক্রোনাসভাবে ডাকা — একটি ধীর হলেই সব ধীর।
- **Idempotency ছাড়া Retry:** পেমেন্ট দুবার চার্জ — গ্রাহকের সবচেয়ে খারাপ অভিজ্ঞতা।
- **Exactly-Once আশা করা:** কিউয়ের উপর অসম্ভব নিশ্চয়তা চাওয়া, বদলে Idempotent শ্রোতা না বানানো।
- **ট্রেসিং ছাড়া মাইক্রোসার্ভিস:** ঘটনার সময় কোন সার্ভিস দোষী, তা খুঁজে পাওয়া যায় না।
- **শেয়ার্ড ডেটাবেস:** দুটি সার্ভিস একই ডেটাবেস ভাগ করলে তারা আসলে আলাদা নয় — ডেটাবেসই তাদের বেঁধে রাখে।

৩.১৫ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একই অর্ডার দুবার তৈরি হচ্ছে	Retry-তে Idempotency নেই	Idempotency Key বাধ্যতামূলক করুন; ডেটাবেসে Unique সীমা দিন।
ইভেন্ট হারিয়ে যাচ্ছে	ডেটাবেস লেখা ও কিউ প্রকাশ আলাদা	Outbox Pattern চালু করুন।
কিউ আটকে গেছে, কিছুই এগোচ্ছে না	বিষাক্ত বার্তা বারবার ব্যর্থ হচ্ছে	DLQ-তে পাঠান; পুনরায় চেষ্টার সীমা বেঁধে দিন।
একটি সার্ভিস ধীর হলে সব ধীর	সিঙ্ক্রোনাস শৃঙ্খল	অপ্রয়োজনীয় কল অ্যাসিঙ্ক্রোনাস করুন; Timeout বেঁধে দিন।
ট্রেসে অনুরোধের পথ খুঁজে পাওয়া যায় না	Trace ID ছড়ানো হয়নি	OpenTelemetry দিয়ে সব সার্ভিসে প্রসঙ্গ ছড়ান।
ডেটা দুই সার্ভিসে অমিল	Eventual Consistency বোঝা হয়নি	কোন তথ্য কতক্ষণে সামঞ্জস্যপূর্ণ হবে, তা নথিভুক্ত করুন; গুরুত্বপূর্ণ পড়ায় পুনরায় যাচাই করুন।

৩.১৬ অধ্যায়ের সারাংশ

মাইক্রোসার্ভিস কোনো লক্ষ্য নয়; এটি একটি ট্রেড-অফ। Modular Monolith বেশিরভাগ দলের জন্য সেরা শুরু — সীমানা কঠোর রেখে, পরে প্রয়োজন হলে ভাঙা যায়। বিতরণকৃত সিস্টেমে নেটওয়ার্ক অবিশ্বস্ত, CAP-এর তিনটি নিশ্চয়তা একসাথে পাওয়া যায় না, এবং সময়ের একক সত্য নেই। যোগাযোগে সিঙ্ক্রোনাস ব্যবহার করুন যেখানে এখনই উত্তর দরকার, অ্যাসিঙ্ক্রোনাস যেখানে পরে হলেও চলে। At-Least-Once ডেলিভারি মেনে নিয়ে Idempotent শ্রোতা বানান, Outbox দিয়ে ডেটাবেস ও কিউয়ের সামঞ্জস্য রাখুন, Saga দিয়ে বিতরণকৃত লেনদেন সামলান, এবং ট্রেসিং ছাড়া কখনো মাইক্রোসার্ভিসে যাবেন না।

৩.১৭ জ্ঞান যাচাই

1. Monolith, Modular Monolith ও Microservices-এর পার্থক্য টেবিল আকারে ব্যাখ্যা করুন।
2. Distributed Monolith কী? কেন এটি সবচেয়ে খারাপ অবস্থা?
3. CAP Theorem ব্যাখ্যা করুন — একটি বাস্তব উদাহরণসহ।
4. Bounded Context কী? সার্ভিস সীমানা নির্ধারণে এটি কীভাবে সাহায্য করে?
5. কখন সিঙ্ক্রোনাস ও কখন অ্যাসিঙ্ক্রোনাস যোগাযোগ ব্যবহার করবেন?
6. At-Least-Once, At-Most-Once ও Exactly-Once-এর পার্থক্য কী? কোনটি বাস্তবসম্মত?
7. Outbox Pattern কোন সমস্যার সমাধান করে? ধাপগুলো লিখুন।
8. Saga Pattern কী? Choreography ও Orchestration-এর পার্থক্য কী?
9. Retry Storm কী? Exponential Backoff ও Jitter কীভাবে তা ঠেকায়?
10. Idempotency Key ছাড়া পেমেন্টে Retry করলে কী বিপদ? ব্যাখ্যা করুন।
11. Dead Letter Queue কেন প্রয়োজন? একটি উদাহরণ দিন।
12. Distributed Tracing ছাড়া মাইক্রোসার্ভিস পরিচালনা কেন কঠিন?

৩.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র অর্ডার, পেমেন্ট ও ইনভেন্টরি ডোমেইনের Bounded Context আঁকুন এবং প্রতিটি সীমানার যৌক্তিকতা এক বাক্যে লিখুন।
2. একটি Saga নকশা করুন: অর্ডার → পেমেন্ট → স্টক → শিপমেন্ট — প্রতিটি ধাপের ক্ষতিপূরণকারী ধাপসহ একটি চিত্র ও টেবিল বানান।
3. একটি ছোট সার্ভিসে Idempotency Key প্রয়োগ করুন: একই কী দুবার এলে দ্বিতীয়বার নতুন রেকর্ড তৈরি না হওয়ার পরীক্ষা লিখুন।

অধ্যায় ৪ • PART I — Advanced DevOps & Enterprise Architecture

Chapter 4 — API Gateway, Service Mesh and Traffic Management

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **API Gateway**, **Reverse Proxy**, **Load Balancer** ও **Service Mesh**-এর ভূমিকা আলাদা করে ব্যাখ্যা করতে; সার্ভিস ডিসকভারি কীভাবে কাজ করে তা বলতে; **Envoy**, **Istio** ও **Linkerd**-এর মধ্যে তুলনা করে বেছে নিতে; **Gateway API** দিয়ে ট্র্যাফিক রাউটিং কনফিগার করতে; **mTLS** দিয়ে সার্ভিস-টু-সার্ভিস যোগাযোগ সুরক্ষিত করতে; ট্র্যাফিক রাউটিং কৌশল (Canary, Blue-Green, Header-Based) প্রয়োগ করতে; এবং সার্ভিস মেশ কখন দরকার আর কখন অপ্রয়োজনীয় জটিলতা — তা যুক্তি দিয়ে বলতে।

- উত্তর-দক্ষিণ (North-South) ও পূর্ব-পশ্চিম (East-West) ট্র্যাফিকের পার্থক্য ব্যাখ্যা করা।
- API Gateway-তে প্রমাণীকরণ, Rate Limit ও রাউটিং কনফিগার করা।
- Sidecar Proxy-র কাজ ও খরচ বিশ্লেষণ করা।
- Istio VirtualService ও DestinationRule দিয়ে Canary করা।
- Gateway API-র মানদণ্ড দিয়ে Ingress বদলানো।
- সার্ভিস মেশের ট্রেড-অফ নথিভুক্ত করা।

৪.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

যখন সার্ভিসের সংখ্যা পাঁচ থেকে পঞ্চাশ হয়, তখন একটি নতুন প্রশ্ন ওঠে: বাইরের ব্যবহারকারী কোন দরজা দিয়ে ঢুকবে, আর ভেতরের সার্ভিসগুলো একে অপরকে কীভাবে খুঁজে পাবে ও বিশ্বাস করবে? প্রতিটি সার্ভিস যদি নিজে প্রমাণীকরণ, Rate Limit, TLS ও পুনরায় চেষ্টা সামলায়, তাহলে একই কোড পঞ্চাশ জায়গায় লেখা হয় — আর একটি ভুল পঞ্চাশ জায়গায় ছড়ায়। API Gateway ও Service Mesh এই সাধারণ কাজগুলোকে এক জায়গায় তুলে আনে: Gateway বাইরের ট্র্যাফিক সামলায়, Mesh ভেতরের ট্র্যাফিক সামলায়। কিন্তু এই দুটি প্রযুক্তিই জটিল ও ব্যয়বহুল হতে পারে — তাই কখন এগুলো দরকার, আর কখন একটি সাধারণ Ingress ও লাইব্রেরিই যথেষ্ট, তা জানা এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ শিক্ষা।

> **CONCEPT:** North-South Traffic মানে বাইরের জগত থেকে ক্লাস্টারের ভেতরে আসা ট্র্যাফিক — ব্যবহারকারী থেকে API Gateway হয়ে সার্ভিসে। East-West Traffic মানে ক্লাস্টারের ভেতরে সার্ভিস-টু-সার্ভিস ট্র্যাফিক — orders-api থেকে payments-api-তে। Gateway উত্তর-দক্ষিণের দরজা; Service Mesh পূর্ব-পশ্চিমের রাস্তা, সিগন্যাল ও পুলিশ। দুটোর কাজ আলাদা, তাই দুটোকে গুলিয়ে ফেলবেন না।

> **REAL WORLD:** TechNova-তে একবার একটি সার্ভিসের নতুন সংস্করণে প্রমাণীকরণের বাগ এলো — কিন্তু বাগটি শুধু সেই সার্ভিসে নয়, আরও তিনটি সার্ভিসে একই লাইব্রেরি কপি করা ছিল। চারটি সার্ভিস একসাথে আপডেট করতে হলো। API Gateway-তে প্রমাণীকরণ কেন্দ্রীভূত করার পর একই ধরনের বাগ এক জায়গায় ঠিক করা যায় — আর সার্ভিসগুলো নিজের ব্যবসায়িক কাজে মন দেয়।

৪.৩ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার HTTP-র মৌলিক ধারণা (পদ্ধতি, হেডার, স্ট্যাটাস কোড), TLS কীভাবে কাজ করে তার প্রাথমিক ধারণা, Kubernetes-এ Service ও Ingress কী করে, DNS কীভাবে নামকে ঠিকানায় রূপান্তর করে, এবং অধ্যায় ৩-এর সিঙ্ক্রোনাস বনাম অ্যাসিঙ্ক্রোনাস যোগাযোগ জানা থাকতে হবে। **Proxy** মানে মধ্যস্থতাকারী — ক্লায়েন্ট সরাসরি সার্ভারের সাথে কথা বলে না, মাঝখানে একজন দাঁড়িয়ে অনুরোধটি গ্রহণ করে, প্রয়োজনীয় কাজ করে, তারপর সার্ভারে পাঠায়। এই একটি ধারণা বুঝলেই Gateway ও Mesh দুটোই বোঝা সহজ হবে।

৪.৪ API Gateway: বাইরের দরজা

API Gateway: ক্লায়েন্ট ও সার্ভিসগুলোর মাঝখানে দাঁড়ানো একটিমাত্র প্রবেশপথ। ক্লায়েন্ট শুধু Gateway-কে চেনে; ভেতরে কয়টি সার্ভিস আছে, তারা কোথায় চলে — ক্লায়েন্টের জানার দরকার নেই। Gateway-র কাজগুলো:

- **Routing (রাউটিং):** পথ দেখে সঠিক সার্ভিসে পাঠানো — যেমন `/api/orders/*` → orders-api, `/api/payments/*` → payments-api।
- **Authentication ও Authorization:** টোকেন যাচাই করা; অনুমোদিত ব্যবহারকারীই শুধু ভেতরে যেতে পারে।
- **Rate Limiting:** একজন ক্লায়েন্ট কত দ্রুত অনুরোধ করতে পারে, তা বেঁধে দেওয়া — যাতে একজন অপব্যবহারকারী সিস্টেমকে ডুবিয়ে দিতে না পারে।
- **Request/Response Transformation:** হেডার যোগ করা, পুরোনো API-কে নতুন আকৃতিতে রূপান্তর করা।
- **Caching ও Compression:** বারবার চাওয়া উত্তর ক্যাশ থেকে দেওয়া।
- **Observability:** প্রতিটি অনুরোধের লগ ও মেট্রিক এক জায়গায়।

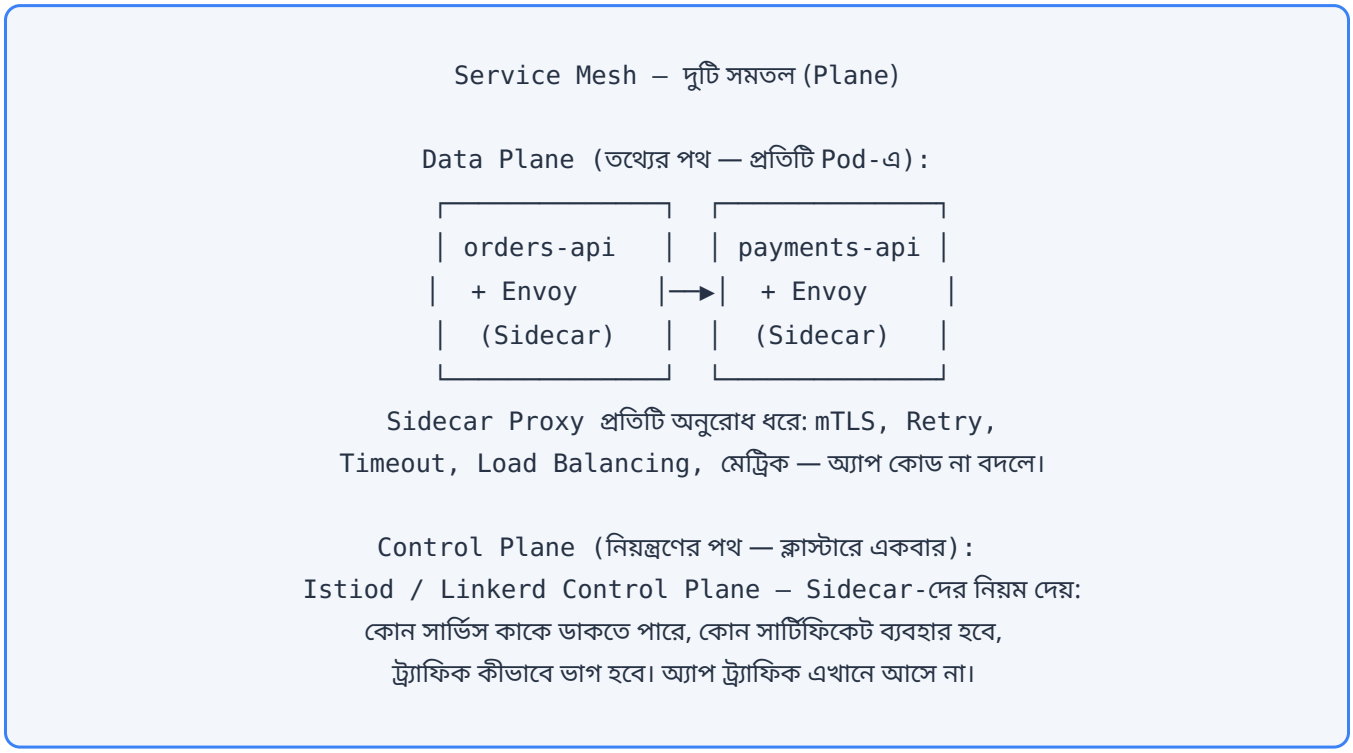
Gateway-কে **Reverse Proxy**-র সাথে গুলিয়ে ফেলবেন না। Reverse Proxy মূলত অনুরোধ পাঠায় ও লোড ভাগ করে; Gateway তার উপর আরও বুদ্ধিমত্তা যোগ করে — প্রমাণীকরণ, Rate Limit, API সংস্করণ ব্যবস্থাপনা। NGINX একটি শক্তিশালী Reverse Proxy; Kong, Traefik বা ক্লাউডের API Gateway পূর্ণাঙ্গ Gateway। ছোট সিস্টেমে NGINX Ingress-ই যথেষ্ট হতে পারে; Gateway তখনই দরকার যখন রাউটিংয়ের নিয়ম জটিল হয় বা অনেক দল অনেক API পরিচালনা করে।

৪.৫ Service Discovery: সার্ভিস কীভাবে একে অপরকে খুঁজে পায়

Kubernetes-এ Pod-এর IP ঠিকানা বদলায় — Pod মরলে নতুন Pod নতুন IP পায়। তাই সার্ভিস A যদি সার্ভিস B-র IP মনে রাখে, তাহলে B পুনরায় চালু হলেই যোগাযোগ ভেঙে যাবে। **Service Discovery** এই সমস্যার সমাধান: B-র একটি স্থির নাম থাকে (যেমন `payments-api.technova.svc.cluster.local`), আর একটি নিবন্ধন-ব্যবস্থা (Registry) নামকে বর্তমান ঠিকানায় রূপান্তর করে। Kubernetes-এ Service অবজেক্ট ও CoreDNS এই কাজটি করে — এটি ক্লায়েন্ট-পক্ষের ডিসকভারি। Consul বা etcd-ভিত্তিক সিস্টেমে সার্ভিস নিজে নিবন্ধন করে (Registration) ও স্বাস্থ্য জানায় (Health Check); অসুস্থ সার্ভিস তালিকা থেকে সরে যায়। Service Mesh এই ডিসকভারিকে আরও এগিয়ে নিয়ে যায়: Sidecar Proxy নিজেই জানে কোন সার্ভিস কোথায় আছে, তাই অ্যাপ্লিকেশন কোডকে কিছুই ভাবতে হয় না।

৪.৬ Service Mesh-এর ভেতরের কার্যপ্রণালী

Service Mesh: সার্ভিস-টু-সার্ভিস যোগাযোগের জন্য একটি নিবেদিত অবকাঠামো স্তর। এর মূল উপাদান দুটি:



Sidecar Proxy: প্রতিটি অ্যাপ্লিকেশন Pod-এর পাশে চলা একটি ছোট Proxy — সাধারণত **Envoy**। অ্যাপ্লিকেশন মনে করে সে সরাসরি অন্য সার্ভিসকে ডাকছে; আসলে তার অনুরোধ প্রথমে নিজের Sidecar-এ যায়, Sidecar সেটিকে সুরক্ষিত করে, গন্তব্যের Sidecar-এ পাঠায়, আর সেখান থেকে অ্যাপে পৌঁছায়। এই নকশার সুবিধা: যোগাযোগের নিয়ম অ্যাপ কোডের বাইরে থাকে — ভাষা বদলালেও নিয়ম একই থাকে। খরচ: প্রতিটি Pod-এ অতিরিক্ত একটি কন্টেইনার চলে — CPU, মেমরি ও সামান্য বিলম্ব (Latency) বাড়ে।

৪.৭ Envoy, Istio ও Linkerd: তুলনা

টুল	কী	শক্তি	দুর্বলতা	কখন বেছে নেবেন
Envoy	উচ্চ-কার্যক্ষমতাসম্পন্ন Proxy (Data Plane)	দ্রুত, নমনীয়, মানদণ্ড	একা ব্যবহারে নিয়ন্ত্রণ ব্যবস্থা নিজে বানাতে হয়	নিজের Gateway বা Mesh বানাতে চাইলে
Istio	পূর্ণাঙ্গ Service Mesh (Envoy + Control Plane)	সবচেয়ে বেশি সক্ষমতা: ট্র্যাফিক, নিরাপত্তা, পর্যবেক্ষণ	জটিল, সম্পদখেকো, শেখার বক্ররেখা খাড়া	বড় ক্লাস্টার, কঠোর নিরাপত্তা ও ট্র্যাফিক নীতি দরকার হলে
Linkerd	হালকা Service Mesh (নিজের Proxy: linkerd2-proxy)	সহজ, হালকা, দ্রুত চালু করা যায়	Istio-র তুলনায় কম সক্ষমতা	ছোট-মাঝারি দল, mTLS ও পর্যবেক্ষণই মূল চাহিদা হলে

একটি সাধারণ ভুল ধারণা: “Service Mesh মানেই Istio।” বাস্তবে Istio সবচেয়ে শক্তিশালী, কিন্তু সবচেয়ে ভারীও। TechNova-র মতো মাঝারি দলের জন্য Linkerd প্রায়শই ভালো শুরু — কারণ এটি একদিনে চালু করা যায়, আর mTLS ও সোনালি মেট্রিকের মতো মূল সুবিধাগুলো দেয়। Istio-তে যাওয়া উচিত তখনই, যখন Linkerd-এর সীমা সত্যিই আঘাত করে — যেমন জটিল ট্র্যাফিক নীতি বা বহু-ক্লাস্টার।

৪.৮ mTLS: সার্ভিস-টু-সার্ভিস বিশ্বাস

সাধারণ TLS-এ ক্লায়েন্ট সার্ভারকে যাচাই করে; সার্ভার ক্লায়েন্টকে যাচাই করে না। **mTLS (Mutual TLS)**-এ দুই পক্ষই একে অপরকে সার্টিফিকেট দিয়ে যাচাই করে। Service Mesh প্রতিটি সার্ভিসকে একটি পরিচয় দেয় (**Workload Identity**, যেমন SPIFFE ID) ও স্বয়ংক্রিয়ভাবে সার্টিফিকেট বিতরণ ও নবায়ন করে। ফলে কোনো আক্রমণকারী ক্লাস্টারের ভেতরে ঢুকলেও অন্য সার্ভিসের পরিচয়ে কথা বলতে পারে না। mTLS ছাড়া ক্লাস্টারের ভেতরের ট্র্যাফিক "বিশ্বস্ত নেটওয়ার্ক" ধরে নেওয়ার উপর চলে — আর আধুনিক নিরাপত্তা নীতি (**Zero Trust**) বলে কোনো নেটওয়ার্কই জন্মগতভাবে বিশ্বস্ত নয়। Istio-তে mTLS চালু করা একটি নীতিমাত্র:

```
# Istio — পুরো Namespace-এ mTLS বাধ্যতামূলক (PeerAuthentication)
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: technova-prod
spec:
  mtls:
    mode: STRICT        # শুধু mTLS গ্রহণ করা হবে; সাধারণ HTTP নয়
---
# Linkerd — mTLS ডিফল্টভাবে চালু থাকে; যাচাই:
$ linkerd check --proxy
# প্রতিটি meshed Pod-এ linkerd2-proxy চলছে কি না দেখুন।
$ linkerd viz tap deploy/orders-api -n technova-prod
# লাইভ ট্র্যাফিক: কোন সার্ভিস কাকে ডাকছে, সফলতার হার কত।
```

> **WARNING:** mTLS চালু করার আগে নিশ্চিত করুন ক্লাস্টারের বাইরে থেকে আসা স্বাস্থ্য-পরীক্ষা (Health Check) ও পর্যবেক্ষণ ব্যবস্থা যেন ভেঙে না যায়। STRICT মোডে হঠাৎ যাওয়া মানে mTLS-বিহীন সব ক্লায়েন্টকে একসাথে বন্ধ করে দেওয়া। সঠিক ক্রম: প্রথমে PERMISSIVE মোডে (mTLS ও সাধারণ দুটোই চলবে), পর্যবেক্ষণ করুন কোন ট্র্যাফিক এখনও mTLS ছাড়া আসছে, সেগুলো ঠিক করুন, তারপর STRICT-এ যান।

৪.৯ Gateway API: Ingress-এর উত্তরসূরি

Kubernetes-এর Ingress সহজ, কিন্তু সীমিত: ভিন্ন ভিন্ন Ingress Controller-এর অ্যানোটেশন ভিন্ন, আর একাধিক দলের জন্য ভূমিকা ভাগ করা কঠিন। **Gateway API** হলো Kubernetes-এর মানদণ্ড-ভিত্তিক উত্তরসূরি — এতে ভূমিকা আলাদা করা যায়: অবকাঠামো দল GatewayClass ও Gateway বানায়, অ্যাপ দল HTTPRoute লেখে।

```
# Gateway API — TechNova: /api/orders → orders-api
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: orders-route
```

```
  namespace: technova-prod
spec:
  parentRefs:
    - name: technova-gateway
      namespace: technova-prod
  hostnames:
    - api.technova.example
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api/orders
      backendRefs:
        - name: orders-api
          port: 8080
```

Gateway API-র সুবিধা: এটি Controller-নিরপেক্ষ — একই YAML Istio, NGINX বা অন্য Gateway-তে চলতে পারে (বাস্তবায়ন-ভেদে পার্থক্য থাকতে পারে)। নতুন প্রকল্পে Ingress-এর বদলে Gateway API দিয়ে শুরু করাই যুক্তিযুক্ত।

৪.১০ ট্র্যাফিক রাউটিং: Canary, Blue-Green ও Header-Based

নতুন সংস্করণ সরাসরি সব ব্যবহারকারীকে দেওয়া ঝুঁকিপূর্ণ। তিনটি নিরাপদ কৌশল:

- **Canary:** নতুন সংস্করণে প্রথমে ৫% ট্র্যাফিক পাঠান; মেট্রিক ভালো থাকলে ধীরে ধীরে ১০%, ২৫%, ৫০%, ১০০% করুন। সমস্যা দেখা গেলে এক কমান্ডে ফিরে যান।
- **Blue-Green:** নতুন সংস্করণ (Green) পুরোপুরি পাশে দাঁড় করান, পরীক্ষা করুন, তারপর একবারে সব ট্র্যাফিক ঘুরিয়ে দিন। ফিরে যাওয়া দ্রুত, কিন্তু দুটি পরিবেশের খরচ একসাথে বহন করতে হয়।
- **Header-Based Routing:** একটি হেডার (যেমন `X-Env: canary`) দেখে নির্দিষ্ট ব্যবহারকারীকে নতুন সংস্করণে পাঠান — অভ্যন্তরীণ পরীক্ষক বা নির্দিষ্ট গ্রাহকের জন্য আদর্শ।

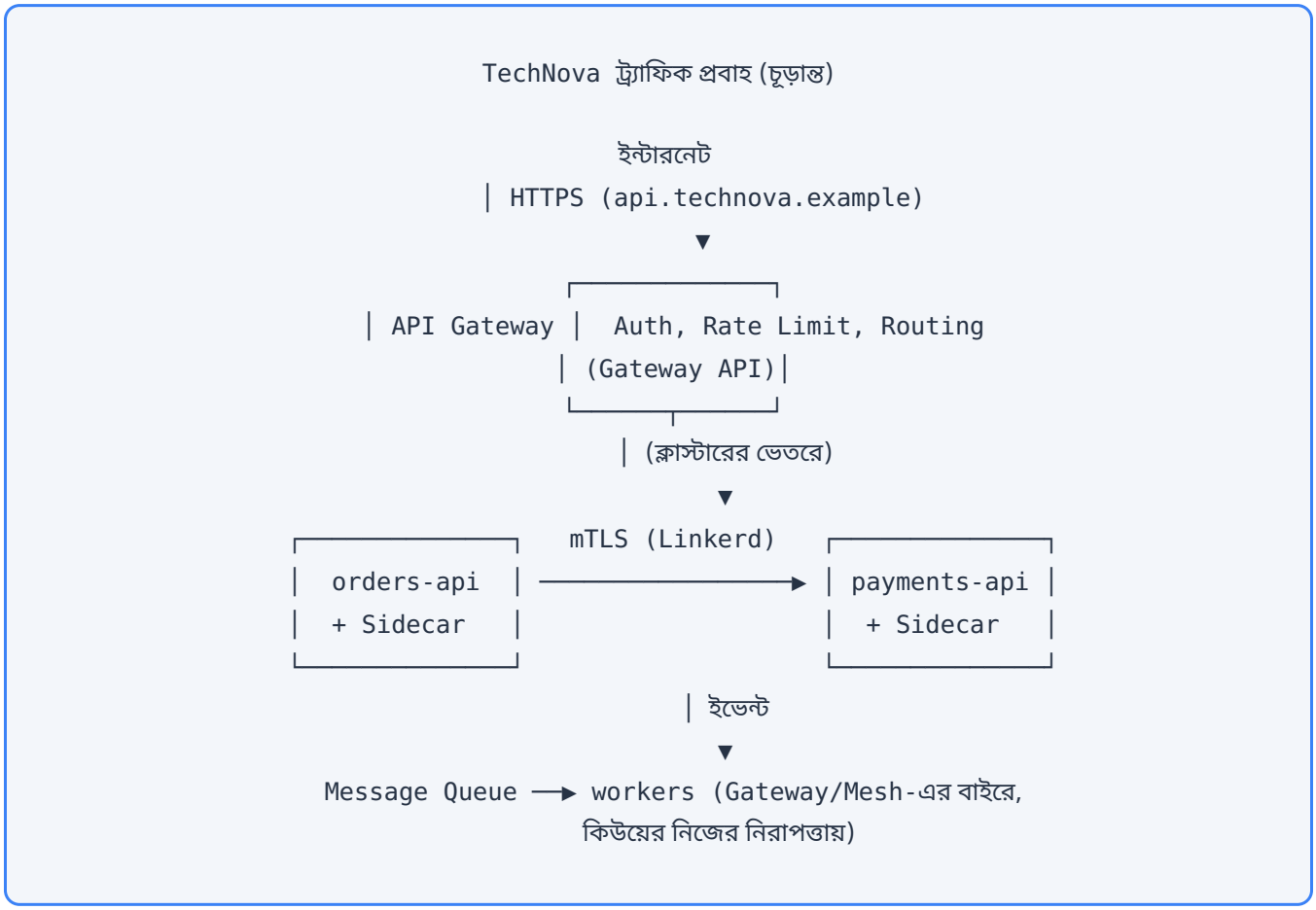
```
# Istio — Canary: 90% পুরোনো (v1), 10% নতুন (v2)
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: orders-api
  namespace: technova-prod
spec:
  hosts:
    - orders-api
  http:
    - route:
        - destination:
```

```
            host: orders-api
            subset: v1
          weight: 90
        - destination:
            host: orders-api
            subset: v2
          weight: 10
---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: orders-api
  namespace: technova-prod
spec:
  host: orders-api
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
```

Canary-র সাফল্য নির্ভর করে স্বয়ংক্রিয় বিশ্লেষণের উপর: শুধু “১০% ট্র্যাফিক পাঠালাম” যথেষ্ট নয়; ত্রুটির হার ও বিলম্বের মেট্রিক দেখে সিদ্ধান্ত নিতে হয় — মানুষের চোখে নয়, স্বয়ংক্রিয়ভাবে। Flagger বা Argo Rollouts এই বিশ্লেষণ ও ধাপে ধাপে এগোনো স্বয়ংক্রিয় করে।

৪.১১ TechNova দৃশ্যকল্প: Gateway + হালকা Mesh

TechNova-র সিদ্ধান্ত: বাইরে একটি API Gateway (Kong/NGINX Gateway), ভেতরে Linkerd — Istio নয়। যুক্তি: সার্ভিসের সংখ্যা এখনও মাঝারি, মূল চাহিদা mTLS ও পর্যবেক্ষণ, জটিল ট্র্যাফিক নীতি এখনও দরকার নেই। Gateway-তে প্রমাণীকরণ ও Rate Limit কেন্দ্রীভূত; Mesh-এ mTLS ও সোনালি মেট্রিক। ছয় মাস পর পুনর্মূল্যায়নের তারিখ ADR-তে লেখা আছে — প্রয়োজনে Istio-তে যাওয়ার পথ খোলা।




```
# যাচাই — Gateway ও Mesh ঠিকমতো চলছে কি না
$ kubectl get gateway -n technova-prod
NAME               CLASS     ADDRESS        READY
technova-gateway   nginx     203.0.113.10   True

$ kubectl get pods -n technova-prod -o wide | head
NAME                           READY   STATUS
orders-api-6d7f8b9c4-abc12   2/2     Running   # 2/2 = অ্যাপ + Sidecar
payments-api-5c6d7e8f9-def34 2/2     Running

$ curl -s https://api.technova.example/api/orders/health
{"status":"ok","version":"v1"}
```

এই নকশায় পর্যবেক্ষণযোগ্যতাও ভাবতে হয়: Gateway থেকে প্রতিটি অনুরোধের বিলম্ব ও ত্রুটির হার মাপা হয়, Mesh থেকে সার্ভিস-টু-সার্ভিস কলের সোনালি মেট্রিক আসে — আর দুটো মিলিয়ে একটি অনুরোধের পুরো পথ দেখা যায়। একটি সার্ভিস যদি ধীর হয়, তাহলে Gateway-র মেট্রিক বলবে বাইরের বিলম্ব বেড়েছে, আর Mesh-র ট্রেস বলবে ভেতরে কোন ধাপটি সময় খাচ্ছে। এই দুটি দৃষ্টিভঙ্গি ছাড়া সমস্যার উৎস খুঁজে পাওয়া কঠিন। আরেকটি ব্যবহারিক বিষয়: Gateway ও Mesh-এর নিয়মগুলোও কোড হিসেবে Git-এ রাখুন — VirtualService, DestinationRule, HTTPRoute সবই সংস্করণ-নিয়ন্ত্রিত হওয়া উচিত। কনসোলে হাতে বদলানো রাউটিং নিয়ম প্রথম ঘটনাতেই হারিয়ে যায়, কারণ কেউ জানে না কে, কখন, কেন বদলেছিল। GitOps-এর নীতি এখানেও প্রযোজ্য: Git-ই ট্র্যাফিক নিয়মের সত্যের উৎস।

CONCEPT: Gateway ও Mesh-এর মধ্যে দায়িত্ব ভাগ করে নেওয়ার সহজ নিয়ম: যা বাইরের ক্লায়েন্ট দেখে — যেমন API-র পথ, সংস্করণ, Rate Limit — তা Gateway-তে; যা শুধু ভেতরের সার্ভিসের বিষয় — যেমন mTLS, পুনরায় চেষ্টা, সার্ভিস-স্তরের ট্র্যাফিক ভাগ — তা Mesh-এ। এই সীমানা না মানলে একই নিয়ম দুই জায়গায় লেখা হয়, আর দুই জায়গার নিয়ম পরস্পরবিরোধী হলে ডিবাগ করা দুঃস্বপ্ন হয়।

৪.১২ নিরাপত্তা ও খরচ বিবেচনা

Gateway হলো সবচেয়ে সংবেদনশীল একক ব্যর্থতার বিন্দু — এটি বন্ধ হলে বাইরের সব ট্র্যাফিক বন্ধ। তাই Gateway-কে একাধিক রেপ্লিকা ও একাধিক অঞ্চলে চালান, এবং এর কনফিগারেশন Git-এ রাখুন। Mesh-এর নিরাপত্তা সুবিধা (mTLS, নীতি) তখনই কাজ করে যখন সব সার্ভিস Mesh-এ থাকে; অর্ধেক সার্ভিস Mesh-এর বাইরে থাকলে নিরাপত্তা ছিদ্রযুক্ত হয়। খরচ: Sidecar প্রতিটি Pod-এ CPU ও মেমরি খায় — হাজার Pod-এ তা উল্লেখযোগ্য। Istio-র Control Plane-ও সম্পদ চায়। এই খরচের হিসাব সার্ভিস মেশ বেছে নেওয়ার আগেই করুন, পরে নয়। আর মনে রাখুন: Gateway ও Mesh দুটোই আক্রমণ-তল বাড়ায় — তাই এগুলোর নিজের নিরাপত্তা আপডেট ও পর্যবেক্ষণও দরকার।

TROUBLE: “Gateway-তে Rate Limit আছে, তবু সার্ভিস ডুবে যাচ্ছে” — সাধারণ কারণ: Rate Limit শুধু প্রতি-IP হিসেবে বাঁধা, কিন্তু আক্রমণকারী অনেক IP ব্যবহার করছে; অথবা Rate Limit Gateway-তে আছে, কিন্তু অভ্যন্তরীণ সার্ভিস সরাসরি ডাকা যাচ্ছে (Gateway বাইপাস)। সমাধান: অভ্যন্তরীণ সার্ভিসে NetworkPolicy দিয়ে শুধু Gateway ও অনুমোদিত সার্ভিস থেকে ট্র্যাফিক আসতে দিন।

৪.১৩ সাধারণ ভুল

- **প্রতিটি সার্ভিসে প্রমাণীকরণ কপি করা:** একই নিয়ম পঞ্চাশ জায়গায় লেখা — একটি ভুল সব জায়গায় ছড়ায়।
- **শুরুতেই Istio:** পাঁচটি সার্ভিসের জন্য পূর্ণাঙ্গ Mesh — শেখা ও চালানোর খরচ সুবিধার চেয়ে বেশি।
- **Gateway-কে ব্যবসায়িক যুক্তির জায়গা বানানো:** Gateway-তে ব্যবসায়িক নিয়ম লিখলে তা পরীক্ষা করা কঠিন হয় ও Gateway ভারী হয়।
- **mTLS হঠাৎ STRICT করা:** PERMISSIVE ধাপ না পেরিয়ে সরাসরি STRICT — স্বাস্থ্য-পরীক্ষাসহ সব ভেঙে যায়।
- **Canary-তে শুধু ট্র্যাফিক ভাগ করা, মেট্রিক না দেখা:** ১০% ব্যবহারকারী খারাপ অভিজ্ঞতা পাচ্ছে, কিন্তু কেউ দেখছে না।
- **Ingress অ্যানোটেশনে ব্যবসা বাঁধা:** Controller-নির্দিষ্ট অ্যানোটেশনে আটকে যাওয়া — Gateway API-তে যাওয়া কঠিন হয়।

৪.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
বাইরে থেকে 502 Bad Gateway	ব্যাকএন্ড Pod অসুস্থ বা রাউট ভুল	Pod-এর অবস্থা ও Service-এর Endpoint দেখুন; Route-এর backendRef মেলান।
সার্ভিস-টু-সার্ভিস কলে TLS ত্রুটি	mTLS STRICT, কিন্তু কলার Mesh-এর বাইরে	কলারকে Mesh-এ আনুন অথবা সাময়িকভাবে PERMISSIVE করুন।

Sidecar যুক্ত Pod চালু হচ্ছে না	Injection ব্যর্থ বা সম্পদ সীমা	Namespace-এ Injection লেবেল দেখুন; Sidecar-এর লগ পড়ুন।
Canary-তে নতুন সংস্করণে ট্র্যাফিক যাচ্ছে না	Subset লেবেল বা Weight ভুল	DestinationRule-এর Subset ও Pod-এর লেবেল মেলান।
Gateway দিয়ে ধীর, সরাসরি দ্রুত	Gateway-তে অতিরিক্ত প্রক্রিয়াকরণ বা কম রেপ্লিকা	Gateway-র CPU/মেমরি ও রেপ্লিকা বাড়ান; অপ্রয়োজনীয় প্লাগইন সরান।

৪.১৫ অধ্যায়ের সারাংশ

API Gateway বাইরের ট্র্যাফিকের একক দরজা — রাউটিং, প্রমাণীকরণ, Rate Limit ও পর্যবেক্ষণ কেন্দ্রীভূত করে। Service Discovery নামকে ঠিকানায় রূপান্তর করে। Service Mesh Sidecar Proxy দিয়ে ভেতরের ট্র্যাফিকে mTLS, পুনরায় চেষ্টা ও পর্যবেক্ষণ যোগ করে — অ্যাপ কোড না বদলে। Envoy হলো Proxy, Istio পূর্ণাঙ্গ Mesh, Linkerd হালকা Mesh; ছোট-মাঝারি দলের জন্য Linkerd দিয়ে শুরু করা যুক্তিযুক্ত। Gateway API হলো Ingress-এর মানদণ্ড-ভিত্তিক উত্তরসূরি। Canary, Blue-Green ও Header-Based রাউটিং নতুন সংস্করণ নিরাপদে ছাড়তে সাহায্য করে। আর সবচেয়ে গুরুত্বপূর্ণ: Mesh কখনো লক্ষ্য নয় — mTLS ও পর্যবেক্ষণের মতো সুনির্দিষ্ট চাহিদা পূরণের উপায় মাত্র।

৪.১৬ জ্ঞান যাচাই

1. North-South ও East-West ট্র্যাফিকের পার্থক্য ব্যাখ্যা করুন।
2. API Gateway ও Reverse Proxy-র পার্থক্য কী?
3. Service Discovery কেন প্রয়োজন? Kubernetes-এ এটি কীভাবে কাজ করে?
4. Sidecar Proxy কী? এর সুবিধা ও খরচ কী কী?
5. Data Plane ও Control Plane-এর পার্থক্য ব্যাখ্যা করুন।
6. Istio ও Linkerd-এর মধ্যে কখন কোনটি বেছে নেবেন? যুক্তি দিন।
7. mTLS কী? সাধারণ TLS থেকে এর পার্থক্য কী?
8. PERMISSIVE থেকে STRICT-এ যাওয়ার সঠিক ক্রম কী এবং কেন?
9. Gateway API Ingress থেকে কীভাবে ভালো? তিনটি কারণ দিন।
10. Canary, Blue-Green ও Header-Based রাউটিংয়ের পার্থক্য ও ব্যবহার-ক্ষেত্র লিখুন।
11. Gateway বাইপাস করে সরাসরি সার্ভিস ডাকা কেন বিপজ্জনক? কীভাবে ঠেকাবেন?
12. পাঁচটি সার্ভিসের সিস্টেমে Service Mesh বসানো কেন ভুল হতে পারে?

৪.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র জন্য একটি Gateway API HTTPRoute লিখুন: `/api/orders` → orders-api, `/api/payments` → payments-api — হোস্ট `api.technova.example` সহ।
2. Istio VirtualService দিয়ে একটি Canary কনফিগার করুন: v2-তে ১০% ট্র্যাফিক, এবং মেট্রিক দেখে ৫০%-এ বাড়ানোর শর্ত লিখুন।
3. একটি ছোট Kubernetes Namespace-এ Linkerd ইনস্টল করে mTLS যাচাই করুন: `linkerd check` ও `linkerd viz tap`-এর আউটপুট সংরক্ষণ করুন।

অধ্যায় ৫ • PART I — Advanced DevOps & Enterprise Architecture

Chapter 5 — Scalability, Resilience and Performance Architecture

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Horizontal** ও **Vertical Scaling**-এর মধ্যে বেছে নিতে; **Stateless** স্থাপত্য নকশা করতে; ক্যাশিংয়ের স্তরগুলো (CDN, Application Cache, Database Cache) প্রয়োগ করতে; লোড ব্যালান্সিংয়ের কৌশল ব্যাখ্যা করতে; **Backpressure**, **Circuit Breaker**, **Bulkhead** ও **Rate Limiting** দিয়ে ব্যর্থতা সীমাবদ্ধ করতে; **Graceful Degradation** নকশা করতে; এবং পারফরম্যান্স ইঞ্জিনিয়ারিংয়ের মাধ্যমে একটি সিস্টেমের সীমা মেপে তা বাড়াতে।

- Stateless সার্ভিস নকশা করে অনুভূমিক স্কেলিং সক্ষম করা।
- Cache-Aside, Write-Through ও Cache Invalidation প্রয়োগ করা।
- লোড ব্যালান্সিং অ্যালগরিদমের মধ্যে বেছে নেওয়া।
- Circuit Breaker-এর তিনটি অবস্থা ব্যাখ্যা ও কনফিগার করা।
- Rate Limit-এর কৌশল (Token Bucket, Fixed Window) তুলনা করা।
- লোড পরীক্ষা দিয়ে সিস্টেমের ভাঙন-বিন্দু (Breaking Point) মাপা।

৫.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

স্কেল মানে শুধু “বড় সার্ভার কেনা” নয়। একটি সিস্টেম যখন দশ গুণ লোড পায়, তখন তার প্রতিটি দুর্বলতা দশ গুণ বড় হয়ে ধরা দেয়: ডেটাবেস সংযোগ ফুরিয়ে যায়, ক্যাশের অনুপস্থিতিতে প্রতিটি অনুরোধ ডেটাবেসে আঘাত করে, একটি ধীর নির্ভরশীলতা সব থ্রেড আটকে দেয়, আর একটি ভুল পুনরায়-চেষ্টা সিস্টেমকে নিজের হাতে ডুবিয়ে দেয়। Resilience বা সহনশীলতা মানে ব্যর্থতা কখনো ঘটবে না — তা নয়; মানে ব্যর্থতা ঘটলে তার পরিধি সীমাবদ্ধ থাকবে, ব্যবহারকারী হয়তো কিছুটা কম সুবিধা পাবে — কিন্তু সিস্টেম পুরোপুরি থামবে না। এই অধ্যায়ে আপনি শিখবেন কীভাবে স্কেল ও সহনশীলতাকে একসাথে নকশা করতে হয় — কারণ স্কেল ছাড়া সহনশীলতা অসম্পূর্ণ, আর সহনশীলতা ছাড়া স্কেল বিপজ্জনক।

> **CONCEPT:** Scalability মানে লোড বাড়লে সিস্টেমের ক্ষমতা বাড়ানো যায়। Resilience মানে উপাদান ব্যর্থ হলেও সিস্টেম সেবা দিতে পারে। Performance মানে প্রতিটি অনুরোধ কত দ্রুত ও দক্ষতার সাথে সম্পন্ন হয়। তিনটি আলাদা গুণ — কিন্তু তারা একে অপরের সাথে জড়িত: একটি ধীর সার্ভিস স্কেল করলে খরচ বাড়ে; একটি ভঙ্গুর সার্ভিস স্কেল করলে ব্যর্থতার পরিধি বাড়ে। তাই তিনটিকে একসাথে ভাবতে হয়।

> **REAL WORLD:** TechNova-র ফ্ল্যাশ সেলে একবার পেমেন্ট গেটওয়ে ধীর হয়ে গেল। অর্ডার API প্রতিটি অনুরোধে গেটওয়ের উত্তরের জন্য ৩০ সেকেন্ড অপেক্ষা করছিল — আর সব থ্রেড আটকে গেল। নতুন অনুরোধ আসছিল, কিন্তু কোনো থ্রেড খালি ছিল না। সিস্টেমটি আসলে “ব্যস্ত” ছিল না — সে “আটকে” ছিল। সমাধান ছিল তিনটি একসাথে: Timeout ২ সেকেন্ডে নামানো, Circuit Breaker বসানো, আর পেমেন্টকে কিউতে সরিয়ে নেওয়া। পরের ফ্ল্যাশ সেলে গেটওয়ে আবার ধীর হলো — কিন্তু অর্ডার নেওয়া চলতে থাকল।

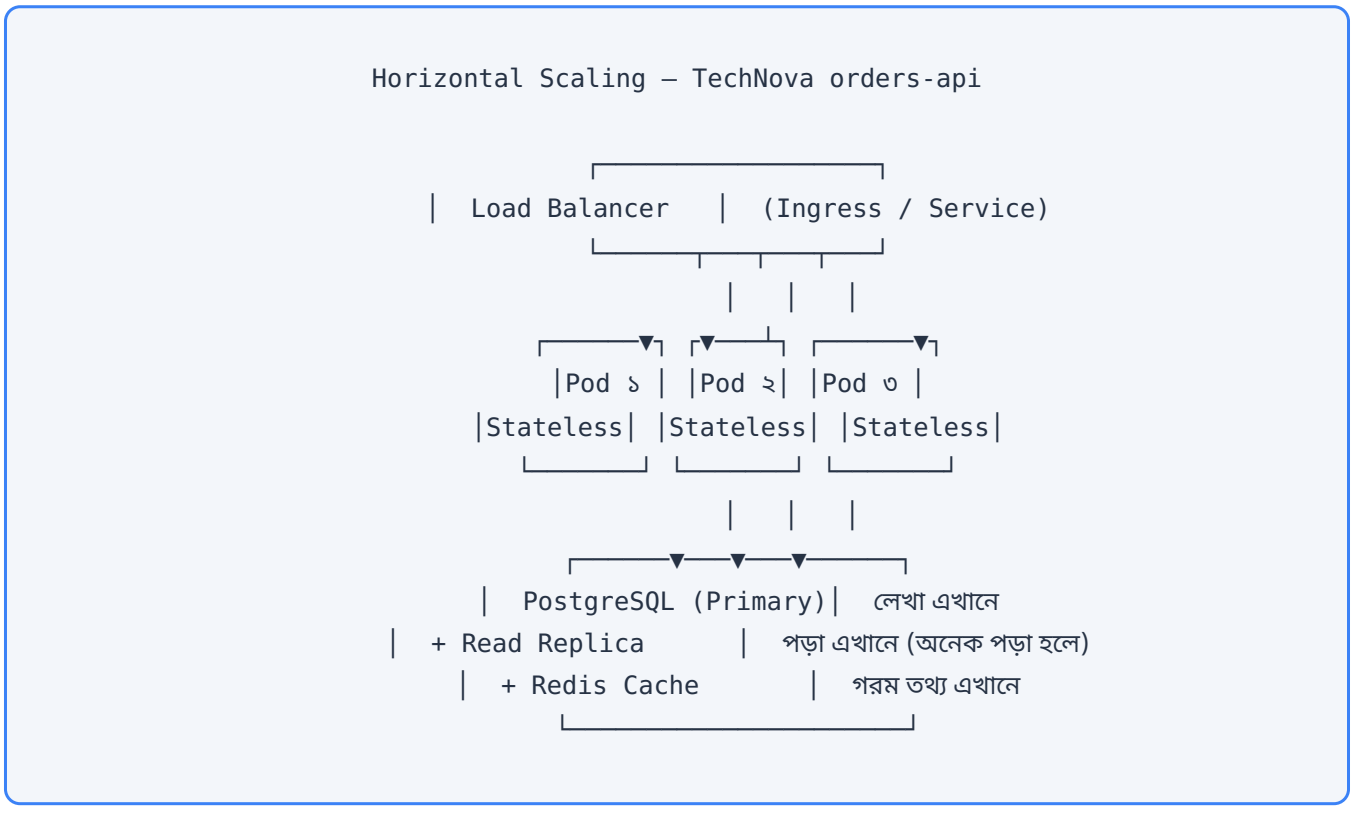
৫.৩ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার অধ্যায় ২-এর ধারণক্ষমতা অনুমান ও NFR, অধ্যায় ৩-এর Timeout, Retry ও Idempotency, অধ্যায় ৪-এর লোড ব্যালান্সিং ও Gateway-র Rate Limit, এবং Kubernetes-এ Deployment, ReplicaSet ও Horizontal Pod Autoscaler (HPA)-এর প্রাথমিক ধারণা জানা থাকতে হবে। মনে রাখুন: **Latency** হলো একটি অনুরোধের সময়; **Throughput** হলো একক সময়ে অনুরোধের সংখ্যা; **p95/p99** হলো শতকরা হিসাব — p95 ৩০০ মিলিসেকেন্ড মানে ৯৫% অনুরোধ ৩০০ মিলিসেকেন্ডের মধ্যে শেষ হয়, বাকি ৫% তার চেয়ে ধীর। গড় (Average) দিয়ে পারফরম্যান্স মাপবেন না — গড় ভালো দেখালেও সবচেয়ে ধীর ৫% ব্যবহারকারী ভয়াবহ অভিজ্ঞতা পেতে পারে।

৫.৪ স্কেলিং: অনুভূমিক বনাম উল্লম্ব

ধরন	কী	সুবিধা	সীমা
Vertical Scaling (Scale Up)	একই মেশিনকে বড় করা — বেশি CPU/মেমরি	সহজ; কোড বদলাতে হয় না	একটি সীমা আছে; বড় মেশিন ব্যয়বহুল; একক ব্যর্থতার বিন্দু থাকে
Horizontal Scaling (Scale Out)	একই জিনিসের আরও কপি যোগ করা	প্রায় অসীম; একটি কপি মরলেও সেবা চলে	Stateless নকশা দরকার; ডেটা স্তর জটিল হয়

নিয়ম: **অ্যাপ্লিকেশন স্তর অনুভূমিকভাবে স্কেল করুন, ডেটা স্তর সাবধানে**। Stateless API সার্ভারকে দশটি কপিতে চালানো সহজ; ডেটাবেসকে দশটি কপিতে চালানো কঠিন — কারণ তথ্যের সামঞ্জস্য রাখতে হয়। ডেটাবেস স্কেল করতে প্রথমে পড়া ও লেখা আলাদা করুন (Read Replica), তারপর ক্যাশ যোগ করুন, তারপর — শুধু যদি সত্যিই দরকার হয় — শার্ডিং (Sharding) ভাবুন। শার্ডিং হলো সবচেয়ে কঠিন স্কেলিং সিদ্ধান্ত; একবার শার্ড করলে ফিরে আসা কঠিন।




```
# Kubernetes HPA — CPU 70% ছাড়ালে রেপ্লিকা বাড়বে
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
metadata:
  name: orders-api-hpa
  namespace: technova-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: orders-api
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300    # হঠাৎ কমে গেলে ৫ মিনিট অপেক্ষা
    scaleUp:
      stabilizationWindowSeconds: 60
```

৫.৫ Stateless স্থাপত্য: স্কেলিংয়ের পূর্বশর্ত

Stateless: সার্ভারের নিজের স্মৃতিতে কোনো ব্যবহারকারীর অবস্থা (State) রাখা হয় না। প্রতিটি অনুরোধ নিজেই সম্পূর্ণ — প্রয়োজনীয় তথ্য হয় অনুরোধে থাকে (যেমন টোকেন), নয়তো ভাগ করা ভাণ্ডারে থাকে (ডেটাবেস, Redis)। Stateless সার্ভিসের যেকোনো কপি যেকোনো অনুরোধ সামলাতে পারে; তাই লোড ব্যালান্সার যেকোনো কপিতে পাঠাতে পারে, আর একটি কপি মরলে কোনো ব্যবহারকারীর কাজ হারায় না। **Stateful** সার্ভিস — যেমন সেশনে লগইন তথ্য সার্ভারের মেমরিতে রাখা — স্কেল করতে গেলে Sticky Session লাগে, যা ভঙ্গুর ও জটিল। নিয়ম: অবস্থা রাখুন ভাগ করা ভাণ্ডারে, সার্ভারে নয়। সেশন টোকেন (JWT) ক্লায়েন্টের কাছে, ক্যাশ Redis-এ, স্থায়ী তথ্য ডেটাবেসে — সার্ভার নিজে থাকবে হালকা ও প্রতিস্থাপনযোগ্য।

৫.৬ ক্যাশিং: সবচেয়ে সস্তা পারফরম্যান্স

ক্যাশ মানে একই প্রশ্নের উত্তর আবার হিসাব না করে মনে রাখা। তিনটি স্তর:

স্তর	কী ক্যাশ হয়	উদাহরণ
CDN (Edge Cache)	স্থির সম্পদ — ছবি, CSS, JS	ব্যবহারকারীর কাছাকাছি সার্ভার থেকে দেওয়া

Application Cache	গরম তথ্য — পণ্যের বিবরণ, মূল্য	Redis-এ, TTL-সহ (যেমন ৫ মিনিট)
Database Cache	প্রায়শই পড়া কুয়েরির ফল	ডেটাবেসের নিজের ক্যাশ, Query Result Cache

৫.৬.১ Cache-Aside ও Write-Through

Cache-Aside: অ্যাপ প্রথমে ক্যাশে খোঁজে; না পেলে ডেটাবেস থেকে পড়ে, তারপর ক্যাশে রাখে। সহজ, কিন্তু প্রথম পড়া ধীর হয়। **Write-Through:** লেখার সময় ডেটাবেস ও ক্যাশ দুটোতেই লেখা হয় — পড়া সবসময় দ্রুত, কিন্তু লেখা ধীর হয়। বেশিরভাগ সিস্টেমে Cache-Aside দিয়ে শুরু করুন। আর সবচেয়ে কঠিন প্রশ্নটি মনে রাখুন: **Cache Invalidation** — তথ্য বদলালে ক্যাশের পুরোনো কপি কীভাবে সরাবেন? কম্পিউটার বিজ্ঞানের বিখ্যাত রসিকতা: "কঠিন দুটি জিনিস — ক্যাশ ইনভ্যালিডেশন ও নামকরণ।" ব্যবহারিক নিয়ম: প্রতিটি ক্যাশ এন্ট্রিতে একটি মেয়াদ (TTL) দিন — যাতে ভুল থাকলেও তা নিজে নিজে মরে যায়।

```
# Redis — Cache-Aside ধাঁচ (ধারণাগত, Python-সদৃশ ছদ্ম-কোড)
#   def get_product(product_id):
#       cached = redis.get(f"product:{product_id}")
#       if cached:
#           return cached                    # Cache Hit — দ্রুত
#       product = db.query("SELECT ...")     # Cache Miss — ডেটাবেস
#       redis.setex(f"product:{product_id}", 300, product)  # TTL 5 মিনিট
#       return product
#
#   def update_product(product_id, data):
#       db.update(...)
#       redis.delete(f"product:{product_id}")  # পুরোনো ক্যাশ সরান
#    # দ্রষ্টব্য: মুছে ফেলা (Delete) নতুন করে লেখার চেয়ে নিরাপদ —
#    # পরের পড়া ডেটাবেস থেকে নতুন তথ্য এনে ক্যাশ করবে।
```

> **WARNING:** ক্যাশ ছাড়া ডেটাবেসে আঘাত করা আর ক্যাশের উপর পুরোপুরি নির্ভর করা — দুটোই বিপদ। **Cache Stampede** হলো এমন অবস্থা যখন একটি জনপ্রিয় ক্যাশ এন্ট্রি একসাথে মারা যায়, আর হাজারো অনুরোধ একসাথে ডেটাবেসে আছড়ে পড়ে। প্রতিকার: TTL-তে এলোমেলোতা (Jitter) যোগ করুন, অথবা একজন অনুরোধকে ডেটাবেসে যেতে দিন, বাকিদের পুরোনো কপি দিন (Stale-While-Revalidate)।

৫.৭ লোড ব্যালান্সিং: কাজ ভাগ করে দেওয়া

লোড ব্যালান্সারের কাজ: আসা অনুরোধগুলোকে সুস্থ কপিগুলোর মধ্যে ভাগ করে দেওয়া। সাধারণ কৌশলগুলো: **Round Robin** (একে একে সবাইকে), **Least Connections** (যার সংযোগ কম, তাকে), **Weighted** (শক্তিশালী মেশিনকে বেশি), ও **Consistent Hashing** (একই কী সবসময় একই কপিতে — ক্যাশের জন্য দরকারি)। সাথে **Health Check**: অসুস্থ কপিকে তালিকা থেকে সরিয়ে দেওয়া। Kubernetes-এ Service ও Ingress এই কাজটি করে; ক্লাউড লোড ব্যালান্সার আরও এগিয়ে যায় — একাধিক অঞ্চলে ট্র্যাফিক ভাগ করে।

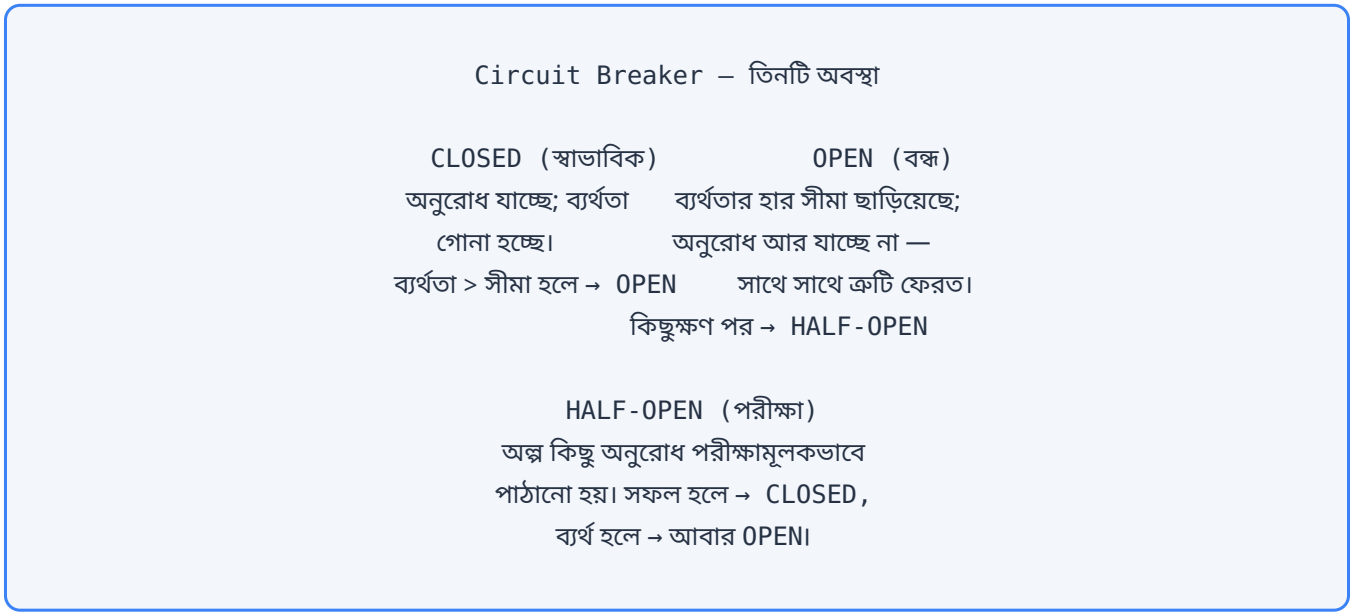
৫.৮ Backpressure: চাপের মুখে “না” বলা

Backpressure: যখন সিস্টেম আর কাজ নিতে পারছে না, তখন সে যেন স্পষ্টভাবে “আমি এখন নিতে পারছি না” বলে — নীরবে ভেঙে না পড়ে। কিউয়ের আকার বেঁধে দিন; কিউ ভরে গেলে নতুন কাজ গ্রহণ বন্ধ করুন (Load Shedding) ও ক্লায়েন্টকে `503 Service Unavailable` ও `Retry-After` হেডার দিয়ে ফেরত পাঠান। ক্লায়েন্ট তখন পরে আবার চেষ্টা করতে পারে। Backpressure ছাড়া একটি সিস্টেম যত কাজ আসে সব গিলতে চায় — মেমরি ফুরিয়ে যায়, থ্রেড আটকে যায়, আর শেষে পুরো সিস্টেম মারা যায়। সীমিতভাবে “না” বলা মানে বেশিরভাগ ব্যবহারকারীকে সেবা দেওয়া; সবাইকে “হ্যাঁ” বলে কাউকে সেবা না দেওয়া তার চেয়ে খারাপ।

৫.৯ Circuit Breaker, Bulkhead ও Rate Limiting

৫.৯.১ Circuit Breaker

Circuit Breaker: বৈদ্যুতিক ফিউজের মতো — নির্ভরশীল সার্ভিস বারবার ব্যর্থ হলে, তার সাথে যোগাযোগ সাময়িকভাবে বন্ধ করে দেওয়া, যাতে সিস্টেম বারবার ব্যর্থ চেষ্টায় সম্পদ নষ্ট না করে। তিনটি অবস্থা:



TechNova-র পেমেন্ট গেটওয়ে ধীর হলে Circuit Breaker খুলে যায়; অর্ডার API তখন সাথে সাথে “পেমেন্ট এখন সাময়িকভাবে অনুপলব্ধ, পরে আবার চেষ্টা করুন” জানায় — আর গেটওয়ে সুস্থ হলে নিজে নিজে বন্ধ হয়। ব্যবহারকারী অপেক্ষা না করে স্পষ্ট বার্তা পায়, আর সিস্টেমের থ্রেড আটকে থাকে না।

৫.৯.২ Bulkhead

Bulkhead: জাহাজের জলরোধী কক্ষের মতো — সিস্টেমকে আলাদা আলাদা ভাগে ভাগ করা, যাতে একটি ভাগের ব্যর্থতা অন্যটিকে ডুবাতে না পারে। ব্যবহারিক রূপ: পেমেন্ট কলের জন্য আলাদা থ্রেড পুল, নোটিফিকেশনের জন্য আলাদা; একটি পুল ভরে গেলে অন্যটি চলতে থাকে। আলাদা ডেটাবেস সংযোগ-পুল, আলাদা কিউ — প্রতিটি নির্ভরশীলতার জন্য আলাদা সীমা।

৫.৯.৩ Rate Limiting

Rate Limiting: কে, কত দ্রুত অনুরোধ করতে পারে, তা বেঁধে দেওয়া। দুটি জনপ্রিয় কৌশল: **Token Bucket** (একটি বালতিতে নিয়মিত টোকেন জমে; প্রতিটি অনুরোধ একটি টোকেন খায়; বালতি খালি হলে অনুরোধ থামে — হঠাৎ ঢেউ কিছুটা

সামলানো যায়) ও **Fixed Window** (প্রতি মিনিটে সর্বোচ্চ Nটি অনুরোধ — সহজ, কিন্তু উইন্ডোর সীমানায় দ্বিগুণ ঢেউ আসতে পারে)। Gateway-তে প্রতি-ব্যবহারকারী ও প্রতি-IP Rate Limit বসান; আর অভ্যন্তরীণ সার্ভিসে প্রতি-কলার Rate Limit — যাতে একটি সার্ভিসের বাগ অন্য সার্ভিসকে ডুবাতে না পারে।

```
# NGINX — Rate Limit (প্রতি IP, প্রতি সেকেন্ডে 10টি অনুরোধ)
#   limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
#   server {
#     location /api/ {
#       limit_req zone=api burst=20 nodelay;
#       proxy_pass http://orders-api:8080;
#  }
# }
# burst=20 মানে হঠাৎ ২০টি অতিরিক্ত অনুরোধ মেনে নেওয়া যাবে;
# তার বেশি হলে 503 ফেরত যাবে।
#
# Kubernetes — সম্পদ সীমা (প্রতিটি Pod-এর জন্য)
#   resources:
#     requests: { cpu: "250m", memory: "256Mi" }
#     limits:   { cpu: "1000m", memory: "512Mi" }
# সীমা না দিলে একটি Pod পুরো নোডের সম্পদ খেয়ে ফেলতে পারে।
```

৫.১০ Graceful Degradation: কম সুবিধা, কিন্তু সেবা চলবে

Graceful Degradation: কোনো অংশ ব্যর্থ হলে সিস্টেম যেন পুরোপুরি থামে না, বরং কম সুবিধা দিয়ে চলতে থাকে। উদাহরণ: সুপারিশ (Recommendation) সার্ভিস বন্ধ হলে পণ্যের পাতা যেন জনপ্রিয় পণ্যের তালিকা দেখায়, খালি পাতা নয়; রিয়েল-টাইম স্টক দেখা না গেলে "স্টক যাচাই করা যাচ্ছে না, অর্ডার করুন — আমরা নিশ্চিত করব" দেখায়; ছবি লোড না হলে টেক্সট দেখায়। প্রতিটি ফিচারের জন্য আগে থেকে ঠিক করুন: এটি ব্যর্থ হলে ব্যবহারকারী কী দেখবে? **Feature Flag** দিয়ে ভারী ফিচার বন্ধ করে সিস্টেমকে হালকা করা যায় — উৎসবের বিক্রিতে সুপারিশ বন্ধ করে অর্ডার-প্রক্রিয়াকে সম্পদ দেওয়া একটি সচেতন ট্রেড-অফ।

৫.১১ পারফরম্যান্স ইঞ্জিনিয়ারিং: মাপুন, তারপর ঠিক করুন

পারফরম্যান্স উন্নতির ক্রম: (১) মাপুন — কোন অংশটি ধীর? প্রোফাইলিং ছাড়া অনুমান করবেন না; (২) সবচেয়ে বড় বোতল-ঘাড়টি ঠিক করুন — একসাথে দশটি জিনিস নয়; (৩) আবার মাপুন — উন্নতি হয়েছে কি না, সংখ্যায় দেখুন। সাধারণ বোতল-ঘাড়: ডেটাবেসে ইনডেক্সের অভাব (N+1 Query), অতিরিক্ত নেটওয়ার্ক কল, বড় JSON উত্তর, সিঙ্ক্রোনাস শৃঙ্খল। লোড পরীক্ষা (**Load Testing**) দিয়ে ভাঙন-বিন্দু মাপুন: ধীরে ধীরে লোড বাড়ান, দেখুন কখন বিলম্ব বাড়তে শুরু করে, কখন ত্রুটি আসে। k6 বা Locust দিয়ে এই পরীক্ষা স্বয়ংক্রিয় করুন, আর প্রোডাকশনে যাওয়ার আগে Staging-এ চালান।

```
# k6 — লোড পরীক্ষা (ধারণাগত, সংক্ষিপ্ত)
#   import http from 'k6/http';
#   export const options = {
#     stages: [
#       { duration: '2m', target: 100 },   // ধীরে ধীরে ১০০ ব্যবহারকারী
```

```
#       { duration: '5m', target: 500 },   // তারপর ৫০০
#       { duration: '2m', target: 0 },     // ধীরে ধীরে কমান
#   ],
#     thresholds: {
#       http_req_duration: ['p(95)<300'],  // p95 ৩০০ মিলিসেকেন্ডের নিচে
#       http_req_failed: ['rate<0.01'],     // ব্যর্থতা ১%-এর নিচে
#   },
# };
#   export default function () {
#     http.get('https://api.technova.example/api/orders/health');
# }
$ k6 run load-test.js
# ফল দেখুন: কোন পর্যায়ে p95 বাড়তে শুরু করল? সেটাই সীমা।
```

৫.১২ TechNova দৃশ্যকল্প: ফ্ল্যাশ সেলের সহনশীলতা পরিকল্পনা

আগামী ফ্ল্যাশ সেলের জন্য TechNova-র পরিকল্পনা — এক পৃষ্ঠায়:

```
# ফ্ল্যাশ সেল — সহনশীলতা চেকলিস্ট
#   [ ] HPA: orders-api min 5, max 30; আগে থেকে 10-এ স্কেল করা
#   [ ] Redis ক্যাশ: পণ্যের তালিকা TTL 5 মিনিট; Cache Stampede প্রতিরোধ
#   [ ] Circuit Breaker: পেমেন্ট গেটওয়ে Timeout 2s, ব্যর্থতা 50% হলে OPEN
#   [ ] Bulkhead: পেমেন্টের থ্রেড পুল আলাদা (সর্বোচ্চ 50 সংযোগ)
#   [ ] Rate Limit: প্রতি IP 10 অনুরোধ/সেকেন্ড; প্রতি ব্যবহারকারী 2 অর্ডার/মিনিট
#   [ ] Graceful Degradation: সুপারিশ বন্ধ (Feature Flag); স্টক যাচাইয়ে
#   ব্যর্থ হলে "পরে নিশ্চিত করা হবে" বার্তা
# [ ] লোড পরীক্ষা: Staging-এ 2 গুণ লোডে k6 চালানো হয়েছে (তারিখ: ___)
#   [ ] Rollback: আগের সংস্করণে ফিরে যাওয়ার কমান্ড পরীক্ষা করা হয়েছে
# [ ] অন-কল: দুজন প্রকৌশলী, ড্যাশবোর্ড ও অ্যালার্ট যাচাই করা হয়েছে
```

৫.১৩ নিরাপত্তা ও খরচ বিবেচনা

স্কেলিংয়ের নিরাপত্তা দিক: Rate Limit শুধু পারফরম্যান্স নয়, নিরাপত্তাও — এটি DDoS ও অপব্যবহার ঠেকায়। কিন্তু Rate Limit ভুলভাবে বাঁধলে বৈধ ব্যবহারকারী আটকে যায় — বিশেষ করে একটি IP-র পেছনে অনেক ব্যবহারকারী থাকলে (যেমন অফিস বা বিশ্ববিদ্যালয়)। তাই Rate Limit ব্যবহারকারী-পরিচয় দিয়ে বাঁধুন, শুধু IP দিয়ে নয়। খরচের দিক: অনুভূমিক স্কেলিং মানে বেশি Pod — বেশি খরচ। HPA-র সর্বোচ্চ সীমা (maxReplicas) বেঁধে দিন, নইলে একটি আক্রমণ বা বাগ খরচকে আকাশে তুলতে পারে। ক্যাশও খরচ: Redis ক্লাস্টারের খরচ ডেটাবেসের চাপ কমানোর সাশ্রয়ের সাথে তুলনা করুন। আর সবচেয়ে বড় খরচের ফাঁদ: অপ্রয়োজনীয়ভাবে সবকিছু স্কেল করা — যে সার্ভিসে লোড নেই, তার ২০টি রেপ্লিকা চালানো শুধু অপচয়।

TROUBLE: “HPA রেপ্লিকা বাড়াচ্ছে, কিন্তু বিলম্ব কমছে না” — সাধারণ কারণ: বোতল-ঘাড় ডেটাবেসে, অ্যাপে নয়। Pod বাড়ালে ডেটাবেস সংযোগ আরও বাড়ে, ডেটাবেস আরও ধীর হয় — স্কেলিং সমস্যাকে আরও খারাপ করে। সমাধান: প্রথমে মাপুন কোথায় সময় যাচ্ছে (ট্রেসিং দেখুন); ডেটাবেসই সীমা হলে সংযোগ-পুল (PgBouncer) ও Read Replica ব্যবহার করুন, Pod বাড়াবেন না।

৫.১৪ সাধারণ ভুল

- **গড় দিয়ে পারফরম্যান্স মাপা:** p95/p99 না দেখে গড় দেখা — সবচেয়ে খারাপ অভিজ্ঞতা লুকিয়ে থাকে।
- **Timeout না বাঁধা:** একটি ধীর নির্ভরশীলতা সব থ্রেড আটকে দেয়।
- **Retry-তে Backoff না দেওয়া:** ব্যর্থ সার্ভিসকে একসাথে হাজারো পুনরায়-চেষ্টা দিয়ে ডুবিয়ে দেওয়া।
- **Stateful সার্ভিস স্কেল করা:** সেশনের তথ্য সার্ভারের মেমরিতে রেখে অনুভূমিক স্কেলিং আশা করা।
- **ক্যাশকে সত্যের উৎস বানানো:** ক্যাশ মুছে গেলে তথ্য হারানো — ক্যাশ সবসময় পুনর্গঠনযোগ্য হতে হবে।
- **HPA-র সর্বোচ্চ সীমা না বাঁধা:** খরচের বিস্ফোরণ।
- **লোড পরীক্ষা না করা:** প্রোডাকশনেই প্রথমবার সর্বোচ্চ লোড দেখা — সবচেয়ে ব্যয়বহুল পরীক্ষা।

৫.১৫ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
লোড বাড়লে বিলম্ব বাড়ে, Pod বাড়িয়েও কমে না	বোতল-ঘাড় ডেটাবেসে	ট্রেসিং দেখুন; ইনডেক্স, Read Replica, PgBouncer যাচাই করুন।
হঠাৎ সব অনুরোধ 503	Rate Limit বা Backpressure সীমা ছাড়িয়েছে	সীমা ও বর্তমান লোড দেখুন; বৈধ ট্র্যাফিক হলে সীমা পুনর্বিবেচনা করুন।
ক্যাশ থাকা সত্ত্বেও ডেটাবেসে চাপ	Cache Miss বেশি, TTL খুব কম, বা Stampede	Hit Ratio মাপুন; TTL-তে Jitter যোগ করুন।
একটি সার্ভিসের ব্যর্থতায় সব ভাঙে	Circuit Breaker বা Bulkhead নেই	নির্ভরশীলতায় Timeout + Circuit Breaker + আলাদা পুল বসান।
খরচ হঠাৎ বেড়ে গেছে	HPA সীমাহীন স্কেল করছে	maxReplicas বেঁধে দিন; কোন সার্ভিস কেন স্কেল করছে, তা দেখুন।
ফ্ল্যাশ সেলে অর্ডার নেওয়া বন্ধ	থ্রেড পুল আটকে গেছে	Timeout কমান; পেমেন্টকে কিউতে সরান; Bulkhead যাচাই করুন।

৫.১৬ অধ্যায়ের সারাংশ

স্কেলিং মানে অনুভূমিকভাবে বাড়া — আর তার পূর্বশর্ত Stateless নকশা। ক্যাশিং সবচেয়ে সস্তা পারফরম্যান্স-উন্নতি, কিন্তু Invalidation ও Stampede সামলাতে হয়। লোড ব্যালান্সিং কাজ ভাগ করে, Backpressure “না” বলতে শেখায়, Circuit Breaker ব্যর্থ নির্ভরশীলতা থেকে সিস্টেমকে রক্ষা করে, Bulkhead ব্যর্থতার পরিধি সীমাবদ্ধ করে, আর Rate Limiting অপব্যবহার ঠেকায়। Graceful Degradation মানে কম সুবিধা দিয়ে সেবা চালু রাখা। আর সবকিছুর আগে: মাপুন — p95/

p99 দেখুন, লোড পরীক্ষা করুন, তারপর ঠিক করুন। সবচেয়ে বড় ট্রেড-অফটি মনে রাখুন: বেশি নির্ভরযোগ্যতা মানে বেশি খরচ ও জটিলতা — তাই প্রতিটি NFR-এর দাম জিজ্ঞেস করুন, আর সঠিক জটিলতাটুকুই বেছে নিন।

৫.১৭ জ্ঞান যাচাই

1. Horizontal ও Vertical Scaling-এর পার্থক্য ও ব্যবহার-ক্ষেত্র ব্যাখ্যা করুন।
2. Stateless সার্ভিস কেন অনুভূমিক স্কেলিংয়ের পূর্বশর্ত? উদাহরণ দিন।
3. Cache-Aside ও Write-Through-এর পার্থক্য কী? কখন কোনটি?
4. Cache Stampede কী? কীভাবে ঠেকাবেন?
5. Backpressure কী? Load Shedding-এর সাথে এর সম্পর্ক কী?
6. Circuit Breaker-এর তিনটি অবস্থা ব্যাখ্যা করুন — একটি চিত্রসহ।
7. Bulkhead প্যাটার্নের নামটি কোথা থেকে এসেছে? এটি কীভাবে কাজ করে?
8. Token Bucket ও Fixed Window Rate Limit-এর পার্থক্য কী?
9. Graceful Degradation-এর তিনটি বাস্তব উদাহরণ দিন।
10. p95 ও গড় (Average)-এর মধ্যে কোনটি পারফরম্যান্স মাপতে ভালো এবং কেন?
11. HPA রেপ্লিকা বাড়ালেও বিলম্ব না কমলে কী করবেন? ধাপগুলো লিখুন।
12. ডেটাবেস স্কেল করতে কেন শার্ডিং শেষ উপায়? আগে কোন ধাপগুলো চেষ্টা করবেন?

৫.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র orders-api-র জন্য একটি HPA YAML লিখুন (min 3, max 20, CPU 70%) এবং Scale-Down Stabilization-এর যৌক্তিকতা এক বাক্যে লিখুন।
2. একটি Redis Cache-Aside উদাহরণ বাস্তবায়ন করুন: পণ্যের তথ্য পড়া, TTL দেওয়া, আপডেটে Invalidation করা — এবং Cache Hit Ratio মাপার উপায় লিখুন।
3. k6 দিয়ে একটি ছোট লোড পরীক্ষা চালান: ধীরে ধীরে লোড বাড়িয়ে p95 ও ব্যর্থতার হার নথিভুক্ত করুন এবং ভাঙন-বিন্দু চিহ্নিত করুন।

PART II • DEVOPS ENGINEERING MASTERY

Advanced Kubernetes & Cloud-Native Engineering

Advanced Kubernetes Internals, Multi-Cluster Kubernetes, Advanced Networking, Security & Governance, Storage & Stateful Workloads

অধ্যায় ৬ • PART II — Advanced Kubernetes & Cloud-Native Engineering

Chapter 6 — Advanced Kubernetes Internals

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Kubernetes Control Plane**-এর প্রতিটি উপাদান (API Server, etcd, Scheduler, Controller Manager) কী করে এবং তারা কীভাবে একে অপরের সাথে কথা বলে, তা ব্যাখ্যা করতে; **Reconciliation Loop** বা সমন্বয়-চক্রের ধারণা দিয়ে ব্যাখ্যা করতে কেন Kubernetes ঘোষণামূলক (Declarative) পদ্ধতিতে কাজ করে; **kubelet**, **Container Runtime** ও **CRI**-র ভূমিকা বলতে; **Custom Resource Definition (CRD)** ও **Operator** দিয়ে নিজের API বানানোর নীতি বুঝতে; এবং **Resource Request/Limit**, **QoS Class** ও ক্লাস্টার সমস্যা সমাধানের পদ্ধতিগত উপায় প্রয়োগ করতে।

- Control Plane ও Data Plane-এর উপাদানগুলোর দায়িত্ব আলাদা করে বলা।
- একটি Pod তৈরি হওয়ার শুরু থেকে চলা পর্যন্ত পুরো পথটি ধাপে ধাপে আঁকা।
- etcd-র ডেটা মডেল ও Watch ব্যবস্থা ব্যাখ্যা করা।
- Scheduler-এর সিদ্ধান্ত-প্রক্রিয়া (Filtering ও Scoring) বোঝা।
- CRD ও Operator প্যাটার্ন দিয়ে একটি কাস্টম কন্ট্রোলার ডিজাইন করা।
- Resource Management ও ক্লাস্টার Troubleshooting-এর জন্য `kubectl` কমান্ড ব্যবহার করা।

৬.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

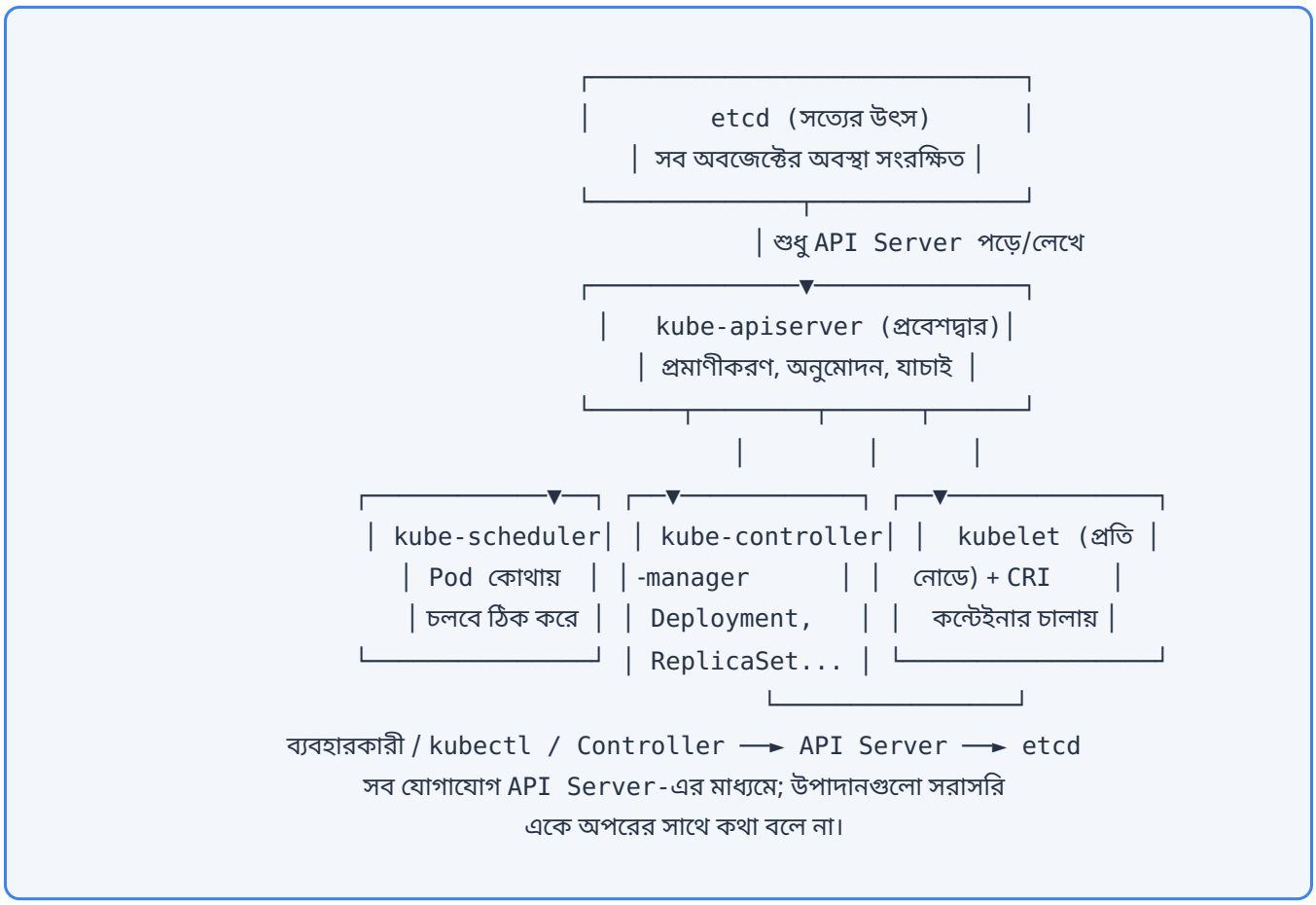
এই অধ্যায় পড়তে আপনার Pod, Deployment, Service ও Namespace-এর মৌলিক ধারণা, `kubectl apply` ও `kubectl get`-এর অভিজ্ঞতা, এবং YAML-এ `apiVersion`, `kind`, `metadata`, `spec` লেখার অভ্যাস থাকলেই চলবে। আগের ভলিউমগুলোতে আপনি দেখেছেন, Kubernetes-এ আপনি “কী চাই” তা ঘোষণা করেন, “কীভাবে হবে” তা Kubernetes ঠিক করে। এই অধ্যায়ে আমরা সেই “কীভাবে”-র ভেতরে ঢুকব — কোন উপাদান কী করে, কোথায় সিদ্ধান্ত হয়, কোথায় অবস্থা সংরক্ষিত হয়। মনে রাখুন: Kubernetes কোনো একক প্রোগ্রাম নয়; এটি অনেকগুলো ছোট ছোট প্রোগ্রামের একটি দল, যারা একটি ভাগ করা ডেটাবেস (etcd) ও একটি API-র মাধ্যমে সমন্বয় করে।

> **CONCEPT:** Kubernetes-এর মূল ধারণা **Desired State** বনাম **Actual State**। Desired State হলো আপনি যা চান — যেমন “ওয়েব অ্যাপের ৩টি রেপ্লিকা চলবে”। Actual State হলো বাস্তবে যা চলছে। Kubernetes-এর সব কন্ট্রোলার অবিরাম এই দুই অবস্থার পার্থক্য মাপে এবং পার্থক্য শূন্য করার জন্য কাজ করে। এই অবিরাম তুলনা ও সংশোধনের চক্রকেই বলে **Reconciliation Loop**।

৬.৩ প্রথম নীতি: কেন Kubernetes এভাবে বানানো হয়েছে

একটি বড় সিস্টেম চালাতে গেলে তিনটি প্রশ্নের উত্তর লাগে: (১) সত্যের একমাত্র উৎস কোথায় থাকবে? (২) কে সিদ্ধান্ত নেবে? (৩) সিদ্ধান্ত কীভাবে বাস্তবায়িত হবে? Kubernetes-এর উত্তর: সত্যের উৎস **etcd**, সিদ্ধান্তের কেন্দ্রবিন্দু **API Server**, আর বাস্তবায়নের দায়িত্ব ছড়িয়ে আছে **Controller** ও **kubelet**-এর মধ্যে। এই ভাগাভাগির কারণে কোনো একটি উপাদান থেমে গেলেও পুরো সিস্টেম একবারে ভেঙে পড়ে না; অন্য উপাদানগুলো নিজের কাজ চালিয়ে যায়। যেমন API Server কিছুক্ষণ বন্ধ থাকলেও চলমান Pod-গুলো চলতেই থাকে — নতুন সিদ্ধান্ত শুধু স্থগিত থাকে।

দ্বিতীয় নীতিটি হলো **Level-Triggered** আচরণ, **Edge-Triggered** নয়। Edge-Triggered সিস্টেম একটি ঘটনা (Event) ঘটলে একবার প্রতিক্রিয়া জানায়; ঘটনাটি হারিয়ে গেলে আর সুযোগ থাকে না। Level-Triggered সিস্টেম বারবার বর্তমান অবস্থা দেখে এবং কাঙ্ক্ষিত অবস্থার সাথে মেলায়। Kubernetes Level-Triggered — তাই কোনো কন্ট্রোলার একবার ব্যর্থ হলেও পরের চক্রে আবার চেষ্টা করে। এ কারণেই Pod মুছে গেলে Deployment আবার নতুন Pod বানায়, নোড মরে গেলে Pod অন্য নোডে সরে যায়।



৬.৪ API Server — ক্লাস্টারের একমাত্র প্রবেশদ্বার

kube-apiserver হলো Kubernetes-এর "সামনের দরজা"। `kubectl`, অন্যান্য কন্ট্রোলার, kubelet — সবাই শুধু API Server-এর সাথেই কথা বলে; কেউ সরাসরি etcd-তে হাত দেয় না। প্রতিটি অনুরোধ (Request) একটি নির্দিষ্ট ধাপের মধ্য দিয়ে যায়: প্রথমে **Authentication** (আপনি কে — সার্টিফিকেট, টোকেন বা OIDC), তারপর **Authorization** (আপনি এটা করতে পারেন কি না — RBAC, অধ্যায় ৯-এ বিস্তারিত), তারপর **Admission Control** (অনুরোধটি নীতির সাথে মেলে কি না — Mutating ও Validating Admission Webhook), তারপর **Validation** (YAML-এর ক্ষেত্রগুলো বৈধ কি না), এবং শেষে etcd-তে লেখা। কোনো ধাপে ব্যর্থ হলে অনুরোধটি বাতিল হয় এবং etcd অপরিবর্তিত থাকে।

API Server নিজে কোনো অবস্থা (State) ধরে রাখে না — এটি **Stateless**। তাই একাধিক API Server রেপ্লিকা একসাথে চলতে পারে, লোড ব্যালান্সারের পেছনে। বাস্তব অবস্থা থাকে শুধু etcd-তে। API Server আরও একটি গুরুত্বপূর্ণ কাজ করে: **Watch**। কোনো কন্ট্রোলার "Pod-এর পরিবর্তন দেখতে চাই" বললে API Server একটি দীর্ঘস্থায়ী সংযোগ খুলে রাখে এবং পরিবর্তন হলেই সঙ্গে সঙ্গে জানিয়ে দেয়। Polling-এর (বারবার জিজ্ঞেস করা) বদলে Watch ব্যবহার করায় ক্লাস্টার দ্রুত প্রতিক্রিয়া জানাতে পারে এবং অপ্রয়োজনীয় চাপ কমে।

```
# API Server-এর স্বাস্থ্য ও সংস্করণ দেখা
kubectl cluster-info
kubectl version
kubectl get --raw /healthz
kubectl get --raw /readyz
kubectl get --raw /livez

# কোন API Resource কোন সংস্করণে আছে
kubectl api-resources
kubectl api-versions | grep -E "apps/v1|networking.k8s.io/v1|gateway.networking.k8s.io/

# API Server-এর লগ (নিয়ন্ত্রণ-তলের নোডে, self-hosted ক্লাস্টারে)
kubectl -n kube-system logs -l component=kube-apiserver --tail=100
```

> **WARNING:** API Server-এর সামনে কখনোই সরাসরি ইন্টারনেট খোলা রাখবেন না। API Server-এর পোর্ট (সাধারণত 6443) শুধু বিশ্বস্ত নেটওয়ার্ক থেকে পৌঁছানো উচিত, আর প্রশাসনিক প্রবেশাধিকার (cluster-admin) খুব অল্প কয়েকজনের হাতে থাকা উচিত। একটি ফাঁস হওয়া kubeconfig ফাইল পুরো ক্লাস্টারের চাবির সমান।

৬.৫ etcd — বিতরণকৃত সত্যের ভান্ডার

etcd হলো একটি বিতরণকৃত কী-ভ্যালু স্টোর (Distributed Key-Value Store), যা **Raft** নামের ঐকমত্য প্রোটোকল (Consensus Protocol) ব্যবহার করে। সহজ ভাষায়: etcd-র একাধিক সদস্য (Member) একসাথে চলে, কিন্তু তারা সবাই একই তথ্য রাখে — একটি লেখা তখনই সফল ধরা হয়, যখন সংখ্যাগরিষ্ঠ (Quorum) সদস্য তাতে রাজি হয়। তিন সদস্যের ক্লাস্টারে একজন মারা গেলেও বাকি দুজন কাজ চালিয়ে যেতে পারে; পাঁচ সদস্যের ক্লাস্টারে দুজন পর্যন্ত হারানো যায়। তাই প্রোডাকশনে etcd সবসময় বিজোড় সংখ্যায় — সাধারণত ৩ বা ৫ — চালানো হয়।

etcd-তে কী থাকে? ক্লাস্টারের সবকিছু: প্রতিটি Pod, Deployment, Service, Secret, ConfigMap-এর সংজ্ঞা ও অবস্থা; প্রতিটি নোডের তথ্য; প্রতিটি Lease ও Event। etcd-তে হাতে কিছু লিখবেন না — সব লেখা API Server-এর মাধ্যমে হয়। তবে ব্যাকআপের জন্য etcd-র **Snapshot** নেওয়া অত্যন্ত গুরুত্বপূর্ণ: API Server-এর বাইরে থেকে `etcdctl snapshot save` দিয়ে পুরো ক্লাস্টারের অবস্থা সংরক্ষণ করা যায়, আর বিপর্যয়ে তা থেকে পুনরুদ্ধার করা যায়।

```
# etcd-র সদস্য ও স্বাস্থ্য (নিয়ন্ত্রণ-তলের নোডে)
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member list
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
```

```
  endpoint health

# Snapshot ব্যাকআপ (নিয়মিত CronJob বা স্ক্রিপ্টে রাখুন)
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /backup/etcd-snapshot-$(date +%F).db
ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-snapshot-2026-11-09.db
```

> **REAL WORLD:** TechNova-র একবার একটি ভুল স্ক্রিপ্টে শত শত Namespace মুছে গিয়েছিল। etcd Snapshot ছিল বলে পুরো ক্লাস্টার আগের অবস্থায় ফেরানো গিয়েছিল। Snapshot ছাড়া সেই ক্ষতি হতো অপরিবর্তনীয়। নিয়ম: etcd Snapshot নিয়মিত নিন, অন্য জায়গায় (Off-site) রাখুন, এবং পুনরুদ্ধার (Restore) করার মহড়া দিন — ব্যাকআপ যা কখনো পরীক্ষা করা হয়নি, তা ব্যাকআপ নয়।

etcd-র পারফরম্যান্স সরাসরি ক্লাস্টারের গতিকে প্রভাবিত করে। etcd ধীর হলে API Server ধীর হয়, আর সবকিছু ধীর হয়। তাই etcd-র ডিস্ক দ্রুত (SSD) হওয়া চাই, নেটওয়ার্ক বিলম্ব (Latency) কম হওয়া চাই, আর ডেটাবেসের আকারের দিকে নজর রাখা চাই — অতিরিক্ত Event ও পুরোনো Revision জমে গেলে **Compaction** ও **Defragmentation** প্রয়োজন হয়।

৬.৬ Scheduler — Pod কোথায় চলবে, সেই সিদ্ধান্ত

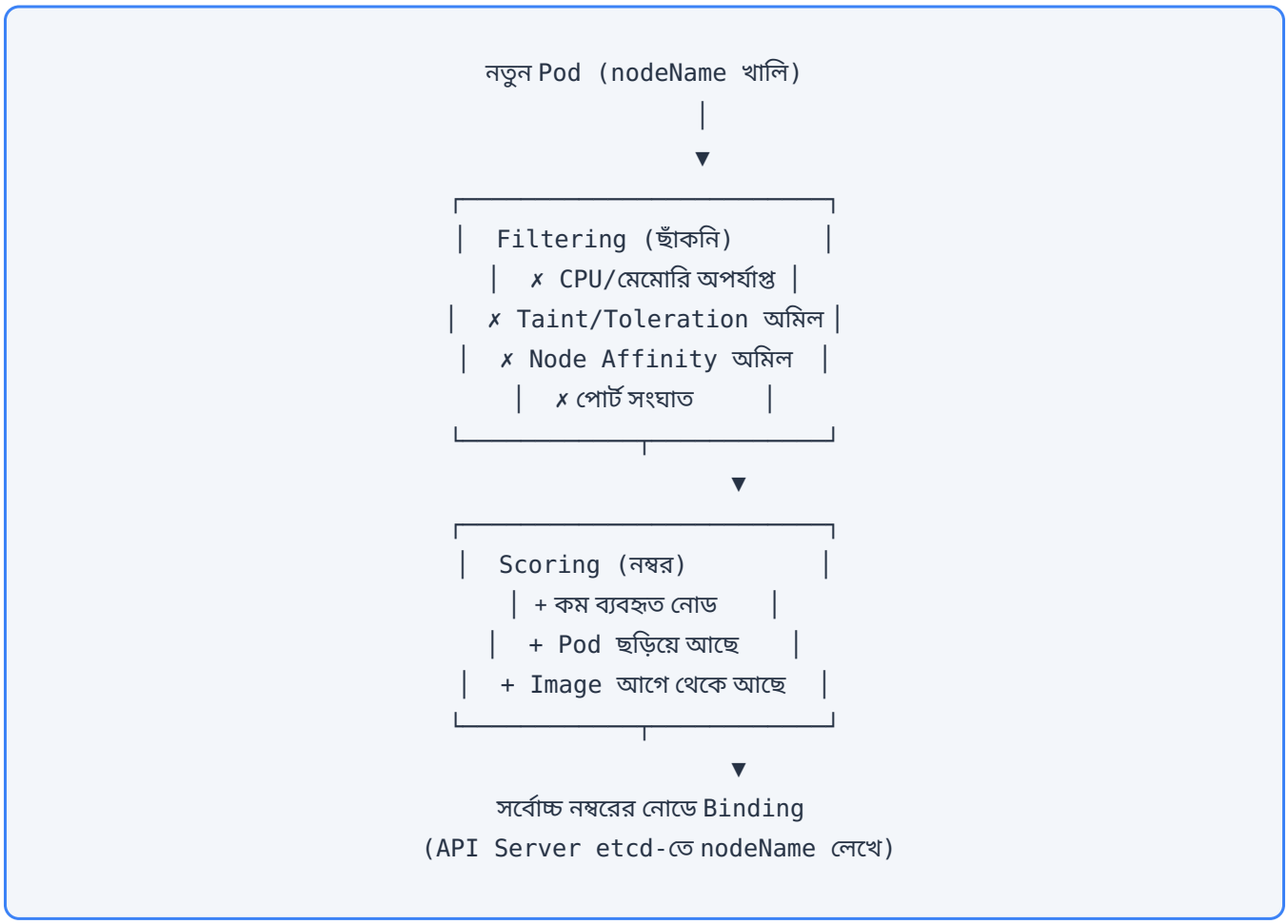
নতুন Pod তৈরি হলে তার `spec.nodeName` খালি থাকে — অর্থাৎ Pod-টি এখনো কোনো নোডে বাঁধা হয়নি (Unscheduled)। **kube-scheduler** Watch-এর মাধ্যমে এই Pod-টি দেখে এবং উপযুক্ত নোড খুঁজে বের করে। সিদ্ধান্তটি দুই ধাপে হয়: **Filtering** (ছাঁকনি) ও **Scoring** (নম্বর দেওয়া)। Filtering-এ যেসব নোড Pod-টি চালাতে পারবে না, সেগুলো বাদ যায় — যেমন যেখানে যথেষ্ট CPU/মেমোরি নেই, যেখানে প্রয়োজনীয় **Taint**-এর সাথে Pod-এর **Toleration** মেলে না, যেখানে **Node Selector** বা **Affinity** মেলে না, বা যেখানে প্রয়োজনীয় পোর্ট ইতিমধ্যে ব্যবহৃত হচ্ছে। তারপর Scoring-এ বাকি নোডগুলোকে নম্বর দেওয়া হয় — যেমন কম ব্যবহৃত নোডকে বেশি নম্বর, একই অ্যাপের Pod ছড়িয়ে থাকা নোডকে বেশি নম্বর (**Pod Anti-Affinity**), বা ছবি (Image) আগে থেকে থাকা নোডকে বেশি নম্বর। সবচেয়ে বেশি নম্বর পাওয়া নোডটি জেতে, আর Scheduler API Server-কে জানায় Pod-টিকে সেই নোডে বাঁধতে (**Binding**)।

Scheduler শুধু একবার সিদ্ধান্ত নেয় — Pod একবার নোডে বাঁধা হলে আর নড়ে না (যদি না Pod-টি মুছে নতুন Pod তৈরি হয়)। তাই **Descheduling** (চলমান Pod-কে সরিয়ে ভারসাম্য আনা) Kubernetes-এর অন্তর্নির্মিত কাজ নয়; প্রয়োজনে **Descheduler** নামের আলাদা টুল বা Pod Disruption Budget-সহ Rolling Update ব্যবহার করতে হয়।

```
# কোন Pod এখনো Schedule হয়নি, কেন — দেখা
kubectl get pods -A --field-selector=status.phase=Pending
kubectl describe pod web-7d9c4f6b5-abc12 | grep -A 15 Events

# নোডের ক্ষমতা ও বরাদ্দ দেখা
kubectl describe nodes
kubectl top nodes
kubectl get nodes -o custom-columns=NAME:.metadata.name,CPU:.status.capacity.cpu,MEM:.s
```

```
# Scheduler-এর লগ ও ইভেন্ট
kubectl -n kube-system logs -l component=kube-scheduler --tail=50
```



৬.৭ Controller Manager — কাঙ্ক্ষিত অবস্থার রক্ষক

kube-controller-manager আসলে অনেকগুলো ছোট কন্ট্রোলারের একটি দল, যারা একই প্রক্রিয়ায় চলে। প্রতিটি কন্ট্রোলারের একটি মাত্র কাজ: একটি নির্দিষ্ট ধরনের অবজেক্টের Desired State ও Actual State মেলানো। যেমন **Deployment Controller** দেখে Deployment-এ কয়টি রেপ্লিকা চাওয়া হয়েছে আর ReplicaSet-এ কয়টি আছে; কম থাকলে ReplicaSet-কে বাড়ায়, বেশি থাকলে কমায়। **ReplicaSet Controller** দেখে কয়টি Pod চলছে; কম থাকলে নতুন Pod বানায়। **Node Controller** দেখে কোনো নোড সাড়া দিচ্ছে কি না; একটি নোড নির্দিষ্ট সময় (সাধারণত ৪০ সেকেন্ড) সাড়া না দিলে তার Pod-গুলোকে অন্য নোডে সরানোর ব্যবস্থা করে। **Endpoints Controller** Service-এর সাথে মিলে যাওয়া Pod-গুলোর IP সংগ্রহ করে Endpoints অবজেক্টে লেখে। **Job Controller**, **Namespace Controller**, **ServiceAccount Controller** — প্রত্যেকে নিজের ছোট দায়িত্ব পালন করে।

প্রতিটি কন্ট্রোলার একটি **Work Queue** ও **Informer** ব্যবহার করে। Informer Watch-এর মাধ্যমে পরিবর্তন শোনে এবং একটি স্থানীয় ক্যাশে (Cache) রাখে; পরিবর্তন হলে সংশ্লিষ্ট অবজেক্টের চাবি (Key) Work Queue-তে পড়ে; কন্ট্রোলারের Worker-রা Queue থেকে চাবি নিয়ে Reconciliation চালায়। Reconciliation ব্যর্থ হলে চাবিটি আবার Queue-তে ফেরে — তবে এবার একটু দেরিতে (**Exponential Backoff**)। এই নকশার ফলে অস্থায়ী ত্রুটি (যেমন নেটওয়ার্ক ছিন্ন) নিজে থেকেই সেরে যায়।

> **CONCEPT:** Controller-দের মধ্যে **Leader Election** হয়। Controller Manager-এর একাধিক রেপ্লিকা চলতে পারে, কিন্তু এক সময়ে শুধু একজন Leader সক্রিয় থাকে; বাকিরা অপেক্ষায় থাকে। Leader মারা গেলে অন্যদের একজন নতুন Leader হয়। এই ব্যবস্থায় Control Plane উচ্চ-উপলব্ধ (Highly Available) থাকে, আবার দুজন একই কাজ একসাথে করে সংঘাতও বাধায় না।

৬.৮ kubelet, CRI ও Container Runtime — নোডের ভেতরে কী ঘটে

প্রতিটি Worker নোডে চলে **kubelet** — নোডের "তত্ত্বাবধায়ক"। kubelet API Server থেকে দেখে এই নোডে কোন Pod-গুলো চলার কথা, তারপর **Container Runtime Interface (CRI)** নামের একটি আদর্শ API দিয়ে কন্টেইনার রানটাইমকে (যেমন **containerd** বা CRI-O) নির্দেশ দেয়: ছবি টানুন, কন্টেইনার চালান, থামান, মুছুন। CRI-র কারণে Kubernetes নির্দিষ্ট কোনো রানটাইমের উপর নির্ভরশীল নয় — রানটাইম বদলালেও kubelet-এর কাজ একই থাকে।

kubelet আরও অনেক কিছু করে: Pod-এর **Probe** চালায় — **Liveness Probe** (কন্টেইনার বেঁচে আছে কি না; ব্যর্থ হলে পুনরায় চালু করে) ও **Readiness Probe** (কন্টেইনার ট্রাফিক নেওয়ার জন্য প্রস্তুত কি না; ব্যর্থ হলে Service থেকে সরিয়ে রাখে); **Volume** মাউন্ট করে; **Secret** ও **ConfigMap** ফাইল হিসেবে Pod-এ দেয়; নোডের অবস্থা (**Node Status**) নিয়মিত API Server-কে জানায়; এবং নোডের সম্পদ শেষ হয়ে গেলে (**Node Pressure** — ডিস্ক, মেমোরি, PID শেষ) কম গুরুত্বপূর্ণ Pod-গুলোকে সরিয়ে দেয় (**Eviction**)।

নোডে আরও দুটি উপাদান থাকে: **kube-proxy** (Service-এর ভার্চুয়াল IP থেকে Pod IP-তে ট্রাফিক পাঠানোর নিয়ম — iptables বা IPVS — বজায় রাখে; অধ্যায় ৮-এ বিস্তারিত) এবং **Container Network Interface (CNI)** প্লাগইন (Pod-কে IP দেওয়া ও নেটওয়ার্ক সংযোগ; অধ্যায় ৮)।

```
# kubelet-এর অবস্থা ও লগ দেখা (নোডে)
systemctl status kubelet
journalctl -u kubelet -n 100 --no-pager

# Pod-এর Probe ও অবস্থা
kubectl describe pod web-7d9c4f6b5-abc12 | grep -A 5 -i "probe\|state\|ready"

# Container Runtime দেখা
kubectl get nodes -o wide
crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps
crictl images | head -20
```

৬.৯ CRD ও Operator — নিজের API বানানো

Kubernetes-এর সবচেয়ে শক্তিশালী ধারণাগুলোর একটি হলো এর API সম্প্রসারণযোগ্য (Extensible)। **Custom Resource Definition (CRD)** দিয়ে আপনি Kubernetes-এ নিজের ধরনের অবজেক্ট যোগ করতে পারেন — যেমন TechNova-র জন্য `Database` নামের একটি অবজেক্ট, যাতে লেখা থাকবে ডেটাবেসের নাম, আকার ও ব্যাকআপ নীতি। CRD তৈরি করলেই API Server সেই নতুন ধরনের অবজেক্ট সংরক্ষণ, যাচাই ও Watch করার সুবিধা দেয় — ঠিক যেমন Pod বা Deployment-এর ক্ষেত্রে করে। কিন্তু CRD নিজে কিছু করে না; শুধু সংজ্ঞা ধরে রাখে।

আসল কাজ করে **Operator** — একটি কাস্টম কন্ট্রোলার, যা আপনার CRD-র অবজেক্ট Watch করে এবং সেই অনুযায়ী বাস্তব কাজ করে। যেমন `Database` অবজেক্ট দেখলে Operator একটি StatefulSet, একটি Service ও একটি Secret বানায়; ডেটাবেসের আকার বদলালে Operator সেই অনুযায়ী স্টোরেজ বাড়ায়। Operator প্যাটার্নের সারমর্ম: মানুষের পরিচালন-জ্ঞান (Operational Knowledge) — "ডেটাবেস বানাতে এই এই ধাপ লাগে, ব্যাকআপ এভাবে নিতে হয়" — কোডে লেখা হয়, আর তা বারবার, ভুল ছাড়া চলে। PostgreSQL, Redis, Kafka-র মতো জটিল সফটওয়্যারের জন্য প্রস্তুত Operator পাওয়া যায়; তবে Operator মানেই জটিলতা — Operator নিজেই একটি অ্যাপ্লিকেশন, যার বাগ, আপগ্রেড ও পর্যবেক্ষণ লাগে। সহজ কাজে Deployment ও Helm যথেষ্ট হলে Operator বানাবেন না।

```
# CRD উদাহরণ: TechNova-র নিজস্ব Database রিসোর্স
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.technova.example
spec:
  group: technova.example
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: ["engine", "size"]
              properties:
                engine:
                  type: string
                  enum: ["postgres", "mysql"]
                size:
                  type: string
                  enum: ["small", "medium", "large"]
                backupEnabled:
                  type: boolean
  scope: Namespaced
  names:
    plural: databases
    singular: database
    kind: Database
```

```
    shortNames: ["db"]
---
# CRD ব্যবহার করে একটি অবজেক্ট
apiVersion: technova.example/v1
kind: Database
metadata:
  name: technova-main-db
  namespace: production
spec:
  engine: postgres
  size: medium
  backupEnabled: true
```

> **TROUBLESHOOTING:** CRD তৈরি করার পর `kubectl get databases` কাজ না করলে প্রথমে দেখুন CRD-টি Established হয়েছে কি না: `kubectl get crd databases.technova.example -o jsonpath='{.status.conditions}'`। Schema-তে ভুল থাকলে CRD তৈরি হয় কিন্তু অবজেক্ট তৈরি ব্যর্থ হয় — ত্রুটি বার্তায় ঠিক কোন ক্ষেত্রটি অবৈধ, তা লেখা থাকে।

৬.১০ Resource Management — Request, Limit ও QoS

Pod-এর **Request** হলো “আমার অন্তত এতটুকু লাগবে” — Scheduler এই সংখ্যা দেখে নোড বাছে, আর নোডের মোট Request তার ক্ষমতার বেশি হলে নতুন Pod সেখানে যায় না। **Limit** হলো “আমার সর্বোচ্চ এতটুকু” — কন্টেইনার এর বেশি CPU বা মেমোরি ব্যবহার করতে পারবে না। CPU Limit ছাড়িয়ে গেলে কন্টেইনারকে থ্রটল (Throttling — গতি কমানো) করা হয়; মেমোরি Limit ছাড়িয়ে গেলে কন্টেইনারকে মেরে ফেলা হয় (**OOMKilled** — Out of Memory Killed)। এই পার্থক্য বোঝা অত্যন্ত জরুরি: CPU ছাড়ালে ধীর হয়, মেমোরি ছাড়ালে মরে যায়।

Request ও Limit-এর অনুপাত থেকে Pod-এর **Quality of Service (QoS) Class** নির্ধারিত হয়: **Guaranteed** (সব কন্টেইনারের Request = Limit — সর্বোচ্চ অগ্রাধিকার, নোড চাপে সবশেষে সরানো হয়), **Burstable** (Request আছে কিন্তু Limit বেশি বা নেই — বেশিরভাগ সাধারণ অ্যাপ), **BestEffort** (কোনো Request/Limit নেই — নোড চাপে প্রথমে সরানো হয়)। প্রোডাকশনের গুরুত্বপূর্ণ অ্যাপে সবসময় Request দিন; Limit দিন, তবে সাবধানে — খুব কম Limit দিলে অ্যাপ থ্রটল হয়ে ধীর হয়ে যাবে, খুব বেশি দিলে নোডের সম্পদ অপচয় হবে। সঠিক মান বের করার উপায়: কয়েক দিনের প্রকৃত ব্যবহার মাপুন (`kubectl top`, Metrics Server, Prometheus), তারপর Request = সাধারণ ব্যবহারের কাছাকাছি, Limit = সর্বোচ্চ ব্যবহারের কাছাকাছি রাখুন।

```
# Resource Request/Limit সহ Deployment (TechNova ওয়েব অ্যাপ)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: production
  labels:
```

```
    app: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: registry.technova.example/web:1.4.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          livenessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
---
# Namespace-এ সবার জন্য ন্যূনতম ও সর্বোচ্চ সীমা (LimitRange)
apiVersion: v1
kind: LimitRange
metadata:
```

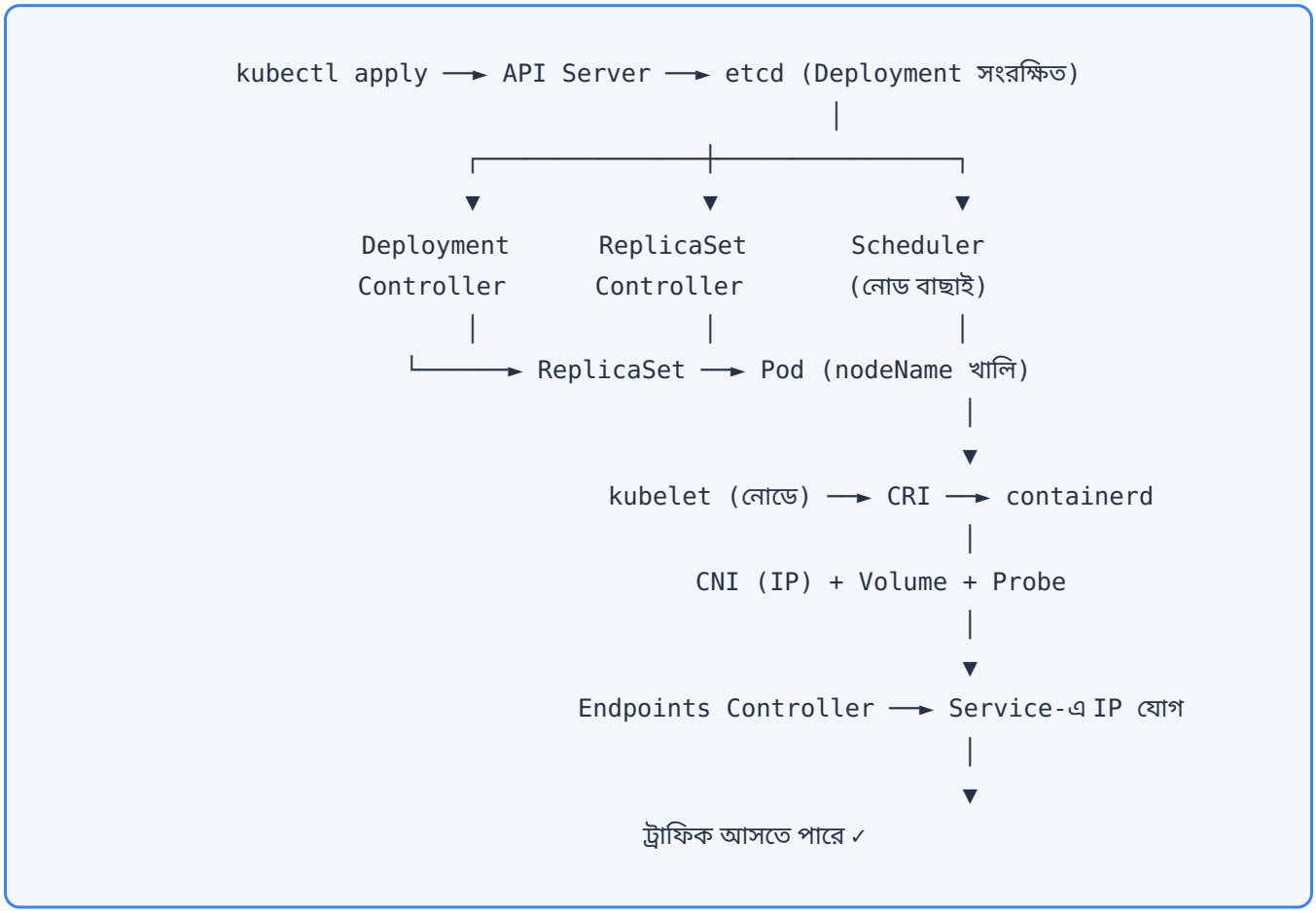
```
  name: default-limits
  namespace: production
spec:
  limits:
    - default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      max:
        cpu: "2"
        memory: "2Gi"
      type: Container
---
# Namespace-এর মোট কোটা (ResourceQuota)
apiVersion: v1
kind: ResourceQuota
metadata:
  name: prod-quota
  namespace: production
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
```

QoS Class	শর্ত	নোড চাপে আচরণ	উপযুক্ত ব্যবহার
Guaranteed	সব কন্টেইনারের Request = Limit	সবশেষে সরানো হয়	গুরুত্বপূর্ণ ডেটাবেস, পেমেন্ট সার্ভিস
Burstable	অন্তত একটি Request বা Limit আছে, সমান নয়	মাঝামাঝি	বেশিরভাগ ওয়েব অ্যাপ ও API
BestEffort	কোনো Request/Limit নেই	প্রথমে সরানো হয়	পরীক্ষামূলক কাজ, ব্যাচ জব

WARNING: মেমোরি Limit না দিলে একটি ত্রুটিপূর্ণ কন্টেইনার (যেমন মেমোরি লিক করা অ্যাপ) পুরো নোডের মেমোরি খেয়ে ফেলতে পারে, আর নোডের অন্য সব Pod মারা যেতে পারে। CPU Limit খুব কম দিলে অ্যাপ ধীর হয়ে Timeout খেতে পারে, অথচ লগে কোনো ত্রুটি দেখা যাবে না — শুধু ধীরগতি। তাই Limit-এর মান অনুমানে নয়, পরিমাপে ঠিক করুন।

৬.১১ একটি Pod-এর জীবনচক্র — শুরু থেকে শেষ

Pod তৈরি হওয়ার পুরো পথটি একবার দেখলে সব উপাদানের ভূমিকা পরিষ্কার হয়। (১) আপনি `kubectl apply -f deployment.yaml` চালান; kubectl YAML পড়ে API Server-কে অনুরোধ পাঠায়। (২) API Server প্রমাণীকরণ, অনুমোদন, Admission ও যাচাই শেষে Deployment-টি etcd-তে লেখে। (৩) Deployment Controller Watch-এর মাধ্যমে পরিবর্তন দেখে ReplicaSet বানায়। (৪) ReplicaSet Controller দেখে Pod কম আছে, তাই Pod অবজেক্ট বানায় — কিন্তু Pod-এর `nodeName` খালি। (৫) Scheduler Pod-টি দেখে উপযুক্ত নোড বাছে ও Binding লেখে। (৬) সেই নোডের kubelet Pod-টি দেখে CNI প্লাগইন দিয়ে Pod-কে IP দেয়, Volume মাউন্ট করে, CRI দিয়ে ছবি টেনে কন্টেইনার চালায়। (৭) Readiness Probe সফল হলে Endpoints Controller Pod-এর IP Service-এর Endpoints-এ যোগ করে — এখন ট্রাফিক আসতে পারে। (৮) Pod মুছে গেলে ReplicaSet Controller আবার নতুন Pod বানায় — চক্র চলতেই থাকে।



৬.১২ প্রোডাকশন দৃশ্যকল্প: TechNova-র নোড হঠাৎ মারা গেল

TechNova-র প্রোডাকশন ক্লাস্টারে তিনটি Worker নোড; একদিন রাত ২টায় একটি নোডের হার্ডওয়্যার ব্যর্থ হলো। কী ঘটল? Node Controller নোডটি থেকে Heartbeat পাওয়া বন্ধ দেখল; নির্দিষ্ট সময় পর নোডটিকে **NotReady** চিহ্নিত করল। সেই নোডের Pod-গুলোর জন্য **Taint** (`node.kubernetes.io/unreachable:NoExecute`) যোগ হলো, আর Pod-গুলোকে অন্য দুটি নোডে আবার Schedule করা হলো — তবে **PodDisruptionBudget** ও **Termination Grace Period** মেনে। ওয়েব অ্যাপের ৩টি রেপ্লিকার ১টি মারা গেলেও বাকি ২টি চলছিল, তাই ব্যবহারকারীরা কিছু টের পেলেন না। ডেটাবেসটি ছিল StatefulSet-এ, একটি Persistent Volume-সহ; তার Pod আবার তৈরি হতে একটু সময় লাগল, কারণ Volume আবার Attach করতে হলো। শিক্ষা: গুরুত্বপূর্ণ অ্যাপের রেপ্লিকা অন্তত ৩টি রাখুন, **Pod Topology Spread Constraints** দিয়ে Pod-গুলোকে বিভিন্ন নোডে (ও সম্ভব হলে বিভিন্ন Availability Zone-এ) ছড়িয়ে রাখুন, এবং Stateful কাজের জন্য Volume Attach-এর সময় হিসাব করুন।

```
# Pod-গুলোকে নোড ও জোন জুড়ে ছড়িয়ে দেওয়া
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: web
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: web
      containers:
        - name: web
          image: registry.technova.example/web:1.4.0
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
---
# PodDisruptionBudget: একসাথে কতগুলো Pod বন্ধ করা যাবে
apiVersion: policy/v1
kind: PodDisruptionBudget
```

```
metadata:
  name: web-pdb
  namespace: production
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: web
```

৬.১৩ সাধারণ ভুল

- **Request/Limit না দেওয়া:** BestEffort Pod বানানো — নোড চাপে প্রথমে এগুলোই মরে, আর Scheduler জানেই না Pod-টির আসলে কত সম্পদ লাগবে।
- **CPU ও মেমোরি Limit-কে একরকম ভাবা:** CPU ছাড়ালে থ্রটল, মেমোরি ছাড়ালে OOMKilled — দুটি সম্পূর্ণ ভিন্ন পরিণতি।
- **Liveness Probe-কে Readiness Probe-এর মতো লেখা:** Liveness Probe-এ ধীর বা বাইরের নির্ভরতা (ডেটাবেস) পরীক্ষা করলে সুস্থ Pod-ও বারবার Restart হতে থাকে — **CrashLoopBackOff**-এর একটি সাধারণ কারণ।
- **etcd ব্যাকআপ না নেওয়া:** "ক্লাউডে আছে, তাই নিরাপদ" ভাবা — Managed Kubernetes-এও নিজের অ্যাপের ডেটা ও কনফিগের ব্যাকআপ আপনার দায়িত্ব।
- **সবকিছুতে Operator ব্যবহার করা:** সহজ Deployment-এ Operator বসানো মানে অপ্রয়োজনীয় জটিলতা, আপগ্রেড-ঝুঁকি ও ডিবাগের কষ্ট।
- **Pod-কে চিরস্থায়ী ভাবা:** Pod ক্ষণস্থায়ী (Ephemeral) — তার IP, নাম, নোড যেকোনো সময় বদলাতে পারে। অবস্থা (State) রাখুন Volume-এ, পরিচয় (Identity) রাখুন Service বা StatefulSet-এ।
- **Latest ট্যাগ ব্যবহার করা:** `image: web:latest` দিয়ে কোন সংস্করণ চলছে, তা জানা যায় না; Rollback-ও অনিশ্চিত হয়। সবসময় নির্দিষ্ট ট্যাগ বা Digest ব্যবহার করুন।

৬.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Pod Pending-এ আটকে আছে	নোডে সম্পদ নেই, Taint/Toleration অমিল, Affinity অমিল	`kubectl describe pod` -এর Events দেখুন; `kubectl describe nodes` দিয়ে বরাদ্দ দেখুন
Pod CrashLoopBackOff	অ্যাপ শুরুতে ক্র্যাশ করছে, ভুল Probe, ভুল Config	`kubectl logs --previous` দেখুন; Probe-এর পথ ও পোর্ট যাচাই করুন
Pod OOMKilled	মেমোরি Limit কম, মেমোরি লিক	Limit বাড়ান বা লিক ঠিক করুন; `kubectl top pod` দিয়ে ব্যবহার মাপুন
ImagePullBackOff	ভুল ছবির নাম/ট্যাগ, Registry-তে প্রবেশাধিকার নেই	ছবির নাম যাচাই করুন; `imagePullSecrets` ঠিক আছে কি না দেখুন

নোড NotReady	kubelet বন্ধ, ডিস্ক পূর্ণ, নেটওয়ার্ক ছিন্ন	নোডে `systemctl status kubelet` ও `df -h` দেখুন
API Server ধীর	etcd ধীর, অতিরিক্ত Watch, বড় তালিকা অনুরোধ	etcd-র ডিস্ক ও Latency দেখুন; `kubectl get` -এ `--limit` ব্যবহার করুন
Deployment-এ নতুন Pod আসছে না	ReplicaSet-এর Selector অমিল, Quota শেষ	`kubectl describe deployment` ও `kubectl get rs` দেখুন

```
# ধাপে ধাপে ডিবাগের ক্রম
kubectl get pods -n production -o wide
kubectl describe pod <pod-name> -n production
kubectl logs <pod-name> -n production --previous
kubectl logs <pod-name> -n production -c web --tail=100
kubectl get events -n production --sort-by=.lastTimestamp
kubectl top pods -n production
kubectl get nodes
kubectl -n kube-system get pods  # Control Plane Pod-গুলো চলছে কি না
```

> **TROUBLESHOOTING:** `kubectl describe` -এর **Events** অংশটি সবসময় প্রথমে পড়ুন — Scheduler, kubelet ও কন্ট্রোলাররা সেখানেই লেখে কেন তারা কিছু করতে পারল না। “FailedScheduling”, “FailedMount”, “Unhealthy” — এই শব্দগুলো সরাসরি কারণের দিকে আঙুল তোলে। লগে ঝাঁপ দেওয়ার আগে Events পড়লে অর্ধেক সময় বাঁচে।

৬.১৫ প্রকৌশলগত আপস (Trade-off)

একটি বড় ক্লাস্টার বনাম অনেক ছোট ক্লাস্টার: বড় ক্লাস্টারে সম্পদ ভাগাভাগি ভালো হয়, কিন্তু একটি ব্যর্থতার প্রভাবও বড় হয়, আর Control Plane-এর চাপ বাড়ে। **Multi-cluster** নিয়ে বিস্তারিত অধ্যায় ৭-এ। Self-managed Control Plane বনাম Managed Kubernetes (যেমন GKE, EKS, AKS): নিজে চালালে নিয়ন্ত্রণ বেশি, কিন্তু etcd ব্যাকআপ, আপগ্রেড ও নিরাপত্তার দায়িত্ব আপনার; Managed সার্ভিসে সেই দায়িত্ব সরবরাহকারীর, বিনিময়ে কিছুটা নিয়ন্ত্রণ কম। ছোট দলের জন্য Managed-ই প্রায়শই সঠিক পছন্দ। একইভাবে, কাস্টম Operator বনাম Helm Chart: Operator জটিল পরিচালন-জ্ঞান স্বয়ংক্রিয় করে, কিন্তু তার নিজের রক্ষণাবেক্ষণ খরচ আছে; Helm সহজ, কিন্তু দিন-২-এর (Day-2) কাজ — ব্যাকআপ, আপগ্রেড, স্কেলিং — মানুষকে করতে হয়। সিদ্ধান্ত নিন কাজের জটিলতা ও দলের আকার দেখে, প্রযুক্তির আকর্ষণ দেখে নয়।

৬.১৬ অধ্যায়ের সারাংশ

Kubernetes একটি একক প্রোগ্রাম নয় — এটি API Server, etcd, Scheduler, Controller Manager, kubelet ও Container Runtime-এর সমন্বিত দল, যারা etcd-কে সত্যের উৎস করে Reconciliation Loop-এর মাধ্যমে কাঙ্ক্ষিত অবস্থা বজায় রাখে। API Server একমাত্র প্রবেশদ্বার ও Watch-এর কেন্দ্র; etcd Raft ঐকমত্যে অবস্থা সংরক্ষণ করে; Scheduler Filtering ও Scoring-এ Pod-এর নোড বাছে; Controller Manager ছোট ছোট কন্ট্রোলার দিয়ে প্রতিটি অবজেক্টের অবস্থা মেলায়; kubelet ও CRI নোডে কন্টেইনার চালায়। CRD ও Operator দিয়ে এই ব্যবস্থাকে সম্প্রসারিত করা যায়, আর Request/Limit ও QoS দিয়ে সম্পদ ভাগ করা যায়। ভেতরের কাজ বুঝলে সমস্যা সমাধান অনুমান নয় — পদ্ধতি হয়।

৬.১৭ জ্ঞান যাচাই

1. API Server ছাড়া কেউ সরাসরি etcd-তে লিখতে পারে না কেন? এই নকশার সুবিধা কী?
2. Desired State ও Actual State-এর পার্থক্য কী, আর Reconciliation Loop কীভাবে তা শূন্য করে?
3. Level-Triggered ও Edge-Triggered আচরণের পার্থক্য ব্যাখ্যা করুন — Kubernetes কেন Level-Triggered?
4. etcd-তে Quorum বলতে কী বোঝায়? ৩ সদস্যের ক্লাস্টারে কয়জন মারা গেলেও ক্লাস্টার চলবে?
5. Scheduler-এর Filtering ও Scoring ধাপ দুটি উদাহরণসহ ব্যাখ্যা করুন।
6. Deployment Controller, ReplicaSet Controller ও kubelet-এর দায়িত্ব আলাদা করে বলুন।
7. CPU Limit ও মেমোরি Limit ছাড়িয়ে গেলে কী কী ভিন্ন পরিণতি হয়?
8. Guaranteed, Burstable ও BestEffort QoS Class-এর পার্থক্য কী? কোনটি কখন ব্যবহার করবেন?
9. CRD ও Operator-এর সম্পর্ক কী? কখন Operator বানানো উচিত নয়?
10. একটি Pod Pending-এ আটকে থাকলে আপনি কোন ক্রমে ডিবাগ করবেন?

৬.১৮ ব্যবহারিক অনুশীলন

1. একটি Namespace-এ এই অধ্যায়ের Deployment, LimitRange ও ResourceQuota প্রয়োগ করুন। `kubectl top pods` ও `kubectl describe node` দিয়ে দেখুন Request কীভাবে Scheduler-এর সিদ্ধান্তকে প্রভাবিত করে; তারপর একটি Pod-এর Request ইচ্ছাকৃতভাবে নোডের ক্ষমতার বেশি করে Pending অবস্থা তৈরি করুন ও Events পড়ে কারণ লিখুন।
2. একটি Pod-এ ইচ্ছাকৃতভাবে ভুল Liveness Probe পথ (`/wrong-path`) দিন, CrashLoopBackOff পর্যবেক্ষণ করুন, তারপর Probe ঠিক করে Pod সুস্থ করুন। প্রতিটি ধাপে `kubectl describe pod` -এর Events লিখে রাখুন।
3. এই অধ্যায়ের CRD প্রয়োগ করে একটি `Database` অবজেক্ট তৈরি করুন। তারপর একটি সহজ Operator-এর পরিকল্পনা লিখুন: Database অবজেক্ট দেখলে Operator কোন কোন Kubernetes অবজেক্ট বানাবে, কোনো ক্ষেত্র বদলালে কী করবে, অবজেক্ট মুছলে কী করবে — কোড লেখার আগে নকশাটি একটি পৃষ্ঠায় লিখুন।
4. TechNova-র একটি নোড ব্যর্থতার মহড়া (Game Day) পরিকল্পনা করুন: একটি নোড Cordon ও Drain করলে কোন Pod কোথায় যাবে, PDB কীভাবে প্রভাব ফেলবে, StatefulSet-এর Volume কী হবে — মহড়ার আগে পূর্বাভাস লিখুন, মহড়ার পর প্রকৃত ফলাফলের সাথে মেলান।

অধ্যায় ৭ • PART II — Advanced Kubernetes & Cloud-Native Engineering

Chapter 7 — Multi-Cluster Kubernetes

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — কখন একটি ক্লাস্টার যথেষ্ট আর কখন একাধিক ক্লাস্টার প্রয়োজন, তা যুক্তি দিয়ে সিদ্ধান্ত নিতে; **Multi-Cluster Architecture**-র ধরনগুলো (Hub-and-Spoke, Federated, সম্পূর্ণ পৃথক) তুলনা করতে; **Cluster Lifecycle** — Provisioning, আপগ্রেড, পরিচয় (Identity) ও অবসর (Decommission) — পরিকল্পনা করতে; **Fleet Management**-এর ধারণা দিয়ে অনেক ক্লাস্টারে একই কনফিগ প্রয়োগ করতে; পরিবেশ পৃথকীকরণ (Environment Separation) ও **Failure Domain**-এর নকশা করতে; এবং **Workload Placement**-এর সিদ্ধান্তে খরচ, বিলম্ব (Latency) ও নিয়ন্ত্রক (Compliance) বিষয়গুলো বিবেচনা করতে।

- Single-cluster বনাম Multi-cluster-এর সুবিধা-অসুবিধা তুলনা করা।
- Hub-and-Spoke ও Federated মডেলের পার্থক্য ব্যাখ্যা করা।
- Cluster API ও GitOps দিয়ে ক্লাস্টার Provisioning-এর প্রবাহ আঁকা।
- Multi-region নকশায় Failure Domain ও ডেটা সার্বভৌমত্ব বিবেচনা করা।
- নিরাপদ ক্লাস্টার আপগ্রেডের ক্রম (Control Plane, তারপর নোড) পরিকল্পনা করা।
- ক্লাস্টার পরিচয়, নেটওয়ার্ক ও Workload Placement-এর সিদ্ধান্ত নেওয়া।

৭.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

অধ্যায় ৬-এর Control Plane উপাদানগুলো, Namespace দিয়ে পরিবেশ আলাদা করার ধারণা, `kubectl config use-context` দিয়ে একাধিক ক্লাস্টারে কাজ করার অভিজ্ঞতা, এবং GitOps-এর মূল ধারণা (Git-ই সত্যের উৎস) থাকলেই চলবে। মনে রাখুন: একটি ক্লাস্টারের ভেতরে Namespace দিয়ে পরিবেশ আলাদা করা যায়, কিন্তু Namespace কখনোই নিরাপত্তা বা ব্যর্থতার সীমানা (Boundary) নয় — একটি Namespace-এর ভুল অন্য Namespace-কে প্রভাবিত করতে পারে, আর একটি ক্লাস্টার মারা গেলে সব Namespace একসাথে মারা যায়। এই সীমাটি বুঝলেই Multi-Cluster-এর প্রয়োজন বোঝা যায়।

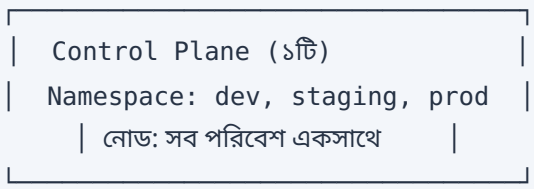
> **CONCEPT: Multi-Cluster** মানে একাধিক স্বাধীন Kubernetes ক্লাস্টার একসাথে পরিচালনা করা — প্রতিটির নিজস্ব API Server ও etcd থাকে। এটি **Multi-Node** বা **Multi-Namespace**-এর চেয়ে অনেক বড় সিদ্ধান্ত। একাধিক ক্লাস্টার মানে একাধিক Control Plane, একাধিক আপগ্রেড, একাধিক পর্যবেক্ষণ — অর্থাৎ পরিচালন-খরচ কয়েক গুণ। তাই Multi-Cluster শুরু করার আগে প্রশ্ন করুন: “একটি ক্লাস্টারে Namespace ও নীতি দিয়ে কি এই সমস্যার সমাধান হয় না?”

৭.৩ প্রথম নীতি: কেন একাধিক ক্লাস্টার

একাধিক ক্লাস্টারের বৈধ কারণগুলো সাধারণত পাঁচটি: (১) **ব্যর্থতার সীমানা (Blast Radius) ছোট করা** — একটি ক্লাস্টারের Control Plane বা নেটওয়ার্ক ব্যর্থ হলে অন্য ক্লাস্টার চলতে থাকে; (২) **ভৌগোলিক বিলম্ব কমানো** — ব্যবহারকারীর কাছাকাছি অঞ্চলে (Region) ক্লাস্টার রাখলে সাড়া দ্রুত হয়; (৩) **নিয়ন্ত্রক ও ডেটা সার্বভৌমত্ব** — কিছু দেশের আইনে ডেটা সেই দেশের ভেতরে রাখতে হয়; (৪) **পরিবেশের কঠোর পৃথকীকরণ** — Production ও

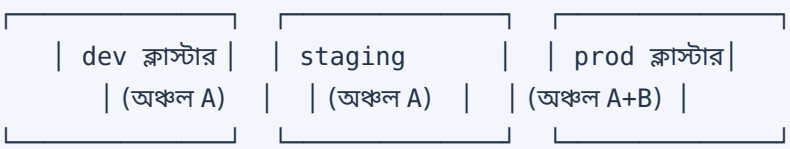
Development-এর মধ্যে কোনো ভাগ করা উপাদান না রাখা; (৫) **স্কেলের সীমা** — একটি ক্লাস্টারে নোড ও Pod-এর সংখ্যার ব্যবহারিক সীমা আছে (Kubernetes-এর নকশা-লক্ষ্য হাজার হাজার নোড হলেও, বাস্তবে Control Plane-এর চাপ, etcd-র আকার ও নেটওয়ার্ক জটিলতা সীমা তৈরি করে)। এর বিপরীতে, Multi-Cluster-এর খরচ: প্রতিটি ক্লাস্টারে Control Plane-এর খরচ, আপগ্রেডের কাজ, পর্যবেক্ষণ, নেটওয়ার্ক সংযোগ, পরিচয় ব্যবস্থাপনা — সব গুণ হয়।

একটি ক্লাস্টার (সহজ):



সুবিধা: সহজ, সস্তা। ঝুঁকি: একটিই ব্যর্থতার বিন্দু।

একাধিক ক্লাস্টার (জটিল):



সুবিধা: পৃথক ব্যর্থতার সীমানা। খরচ: পরিচালনা × N।

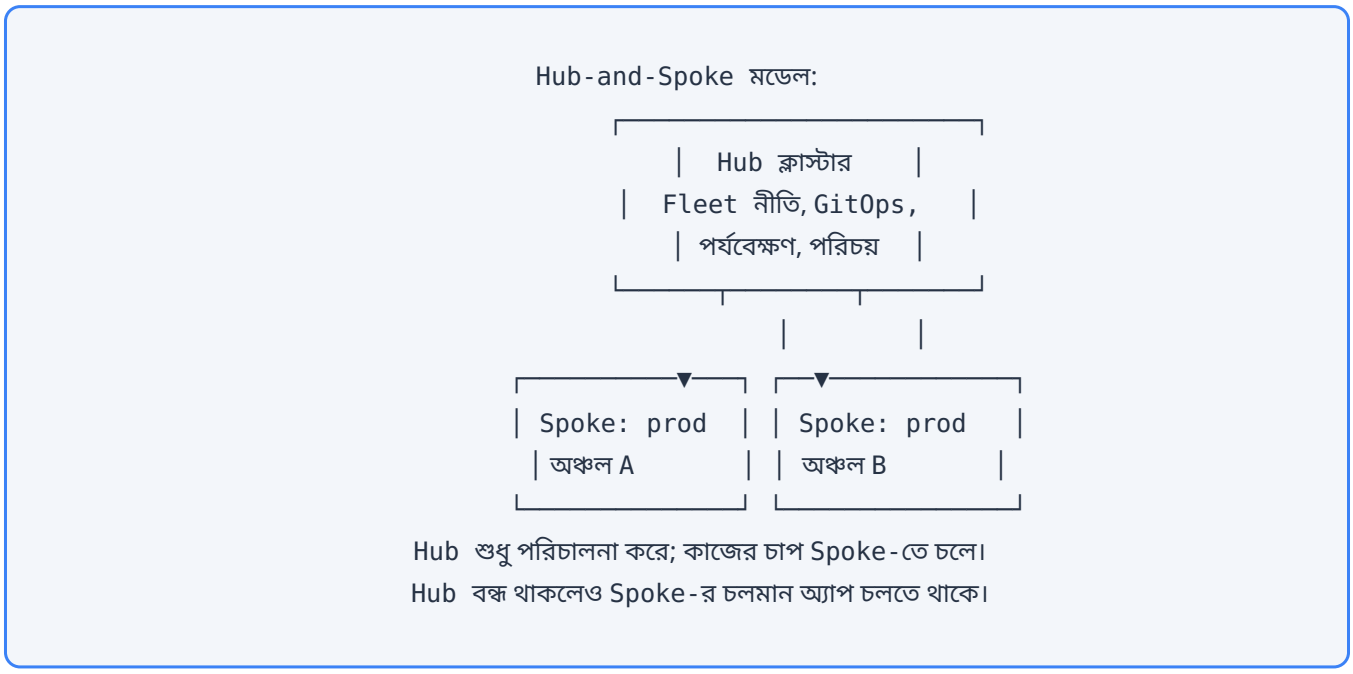
বিবেচনা	একটি ক্লাস্টার	একাধিক ক্লাস্টার
পরিচালন-খরচ	কম	ক্লাস্টার সংখ্যার সাথে বাড়ে
ব্যর্থতার সীমানা	বড় — সবকিছু একসাথে যায়	ছোট — একটি ক্লাস্টারের ব্যর্থতা অন্যকে ছোঁয় না
বিলম্ব (Latency)	এক অঞ্চলে সীমাবদ্ধ	ব্যবহারকারীর কাছাকাছি রাখা যায়
নিয়ন্ত্রক সম্মতি	সীমিত	দেশ/অঞ্চলভেদে ডেটা রাখা যায়
আপগ্রেডের ঝুঁকি	একবারে সবকিছু	একটি ক্লাস্টারে পরীক্ষা করে অন্যগুলোতে প্রয়োগ
দলের আকার	ছোট দলের জন্য উপযুক্ত	প্ল্যাটফর্ম দল প্রয়োজন

REAL WORLD: TechNova শুরুতে একটি ক্লাস্টারেই Dev, Staging ও Production চালাত — Namespace দিয়ে আলাদা করে। তখন দল ছোট ছিল, আর এটাই সঠিক সিদ্ধান্ত ছিল। কিন্তু যখন Production-এর গ্রাহক বাড়ল এবং একটি অঞ্চলের বিভ্রাটে (Outage) পুরো সেবা বন্ধ হয়ে গেল, তখন তারা Production-কে আলাদা ক্লাস্টারে নিল, আর পরে দ্বিতীয় অঞ্চলে আরেকটি Production ক্লাস্টার যোগ করল। শিক্ষা: Multi-Cluster-এ যাওয়ার সময়টি সমস্যা দেখা দেওয়ার পর, সমস্যা অনুমানের আগে নয়।

৭.৪ Multi-Cluster আর্কিটেকচারের ধরন

তিনটি প্রধান ধরন প্রচলিত। প্রথম, **সম্পূর্ণ পৃথক ক্লাস্টার (Independent Clusters)**: প্রতিটি ক্লাস্টার নিজে নিজে চলে, কোনো ভাগ করা নিয়ন্ত্রণ নেই; শুধু GitOps রিপোজিটরি ও পর্যবেক্ষণ ভাগ করা হয়। এটি সবচেয়ে সহজ ও সবচেয়ে মজবুত

— একটি ক্লাস্টারের ব্যর্থতা অন্যকে ছুঁতে পারে না — কিন্তু প্রতিটি ক্লাস্টারে আলাদা করে কনফিগ প্রয়োগ করতে হয়। দ্বিতীয়, **Hub-and-Spoke**: একটি কেন্দ্রীয় (Hub) ক্লাস্টার থেকে অন্যান্য (Spoke) ক্লাস্টারগুলো পরিচালিত হয় — Hub-এ থাকে Fleet-এর নীতি, অ্যাপ বিতরণের সিদ্ধান্ত ও পর্যবেক্ষণ; Spoke-গুলোতে শুধু কাজের চাপ (Workload) চলে। Hub মারা গেলে Spoke-গুলো চলতে থাকে, তবে নতুন বিতরণ বন্ধ থাকে — তাই Hub-এর উচ্চ-উপলব্ধতা গুরুত্বপূর্ণ। তৃতীয়, **Federated** (যেমন KubeFed-এর ধারণা): একাধিক ক্লাস্টারকে একটি একক যৌক্তিক ক্লাস্টারের মতো দেখানো হয়; একটি Federated Deployment লিখলে তা স্বয়ংক্রিয়ভাবে একাধিক ক্লাস্টারে ছড়িয়ে যায়। এটি আকর্ষণীয় শোনালেও বাস্তবে জটিল — ক্লাস্টারগুলোর মধ্যে নেটওয়ার্ক, পরিচয় ও অবস্থার সমন্বয় কঠিন, এবং Federation-এর অনেক টুল আর সক্রিয়ভাবে রক্ষণাবেক্ষণ হয় না। বর্তমান বাস্তব চর্চায় Hub-and-Spoke ও GitOps-ভিত্তিক স্বাধীন ক্লাস্টারই বেশি প্রচলিত।



৭.৫ Cluster Lifecycle — জন্ম থেকে অবসর

একটি ক্লাস্টারের জীবনচক্রের ধাপগুলো: **Provisioning** (তৈরি), **Bootstrap** (প্রাথমিক উপাদান বসানো — CNI, DNS, Ingress, পর্যবেক্ষণ), **Day-2 Operations** (আপগ্রেড, স্কেলিং, সার্টিফিকেট নবায়ন), ও **Decommission** (নিরাপদে অবসর — Workload সরানো, ডেটা মুছে ফেলা, সম্পদ মুক্ত করা)। হাতে ক্লাস্টার বানানো মানে প্রতিটি ধাপে মানুষের হস্তক্ষেপ — আর প্রতিটি ক্লাস্টার একটু ভিন্ন হয়ে যায়। সমাধান: **Cluster API** — Kubernetes দিয়ে Kubernetes ক্লাস্টার বানানোর প্রকল্প, যেখানে ক্লাস্টার নিজেই একটি Kubernetes অবজেক্ট (`Cluster`, `MachineDeployment`) হিসেবে ঘোষণা করা হয়, আর একটি Management ক্লাস্টার থেকে Workload ক্লাস্টার তৈরি ও পরিচালিত হয়। আরেকটি পথ: Terraform বা Pulumi দিয়ে অবকাঠামো বানিয়ে, তারপর GitOps (Argo CD, Flux) দিয়ে ক্লাস্টারের ভেতরের উপাদান বসানো।

```
# একাধিক ক্লাস্টারে কাজ: Context ব্যবস্থাপনা
kubectl config get-contexts
kubectl config use-context technova-prod-a
kubectl get nodes --context technova-prod-a
kubectl get nodes --context technova-prod-b

# সব ক্লাস্টারে একই পরীক্ষা (শেল লুপ — ছোট Fleet-এর জন্য)
```

```
for ctx in technova-dev technova-staging technova-prod-a technova-prod-b; do
  echo "=== $ctx ==="
  kubectl --context "$ctx" get nodes
  kubectl --context "$ctx" -n kube-system get pods --field-selector=status.phase!=Runni
done

# Cluster API ধারণা (Management ক্লাস্টারে ঘোষণা)
# apiVersion: cluster.x-k8s.io/v1beta1
# kind: Cluster
# metadata:
#   name: technova-prod-b
#   namespace: clusters
# spec:
#   clusterNetwork:
#     pods:
#       cidrBlocks: ["10.244.0.0/16"]
#   infrastructureRef:
#     apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
#     kind: AWSCluster   # বা অন্য প্রদানকারীর ধরন
#     name: technova-prod-b
```

> **CONCEPT: Cluster as Code** — ক্লাস্টারের সংজ্ঞা (সংস্করণ, নোডের আকার ও সংখ্যা, অঞ্চল, অ্যাড-অন) Git-এ লেখা থাকে। নতুন ক্লাস্টার মানে নতুন হাতে বানানো নয় — মানে Git-এ একটি ফাইল যোগ করা ও পাইপলাইন চালানো। এতে প্রতিটি ক্লাস্টার একই রকম হয়, আর কোনো ক্লাস্টার ধ্বংস হলে একই কোড থেকে আবার বানানো যায়।

৭.৬ Fleet Management — অনেক ক্লাস্টারে একই নিয়ম

Fleet Management হলো অনেক ক্লাস্টারকে একটি দল হিসেবে পরিচালনা করা। এর মূল প্রশ্ন: "এই পরিবর্তনটি কোন ক্লাস্টারগুলোতে যাবে?" — যেমন নতুন নিরাপত্তা নীতি সব Production ক্লাস্টারে যাবে, কিন্তু Development ক্লাস্টারে যাবে না; একটি নতুন অ্যাপ প্রথমে একটি অঞ্চলে (Canary) যাবে, তারপর বাকিগুলোতে। এই কাজে ব্যবহৃত হয় GitOps টুল (Argo CD-র **ApplicationSet**, Flux-এর **Kustomization**), যেখানে ক্লাস্টারগুলোকে **Label** দিয়ে চিহ্নিত করা হয় (`env=prod`, `region=ap-south-1`), আর অ্যাপের সংজ্ঞায় লেখা থাকে কোন Label-এর ক্লাস্টারে এটি যাবে। আরেকটি পদ্ধতি: **Policy Engine** (OPA Gatekeeper, Kyverno — অধ্যায় ৯) দিয়ে সব ক্লাস্টারে একই নীতি প্রয়োগ করা, যাতে কোনো ক্লাস্টারে নীতি-বিরোধী কিছু চলতে না পারে।

```
# Argo CD ApplicationSet ধারণা: Label দিয়ে ক্লাস্টার বাছাই
# apiVersion: argoproj.io/v1alpha1
# kind: ApplicationSet
# metadata:
#   name: technova-web
```

```
#   namespace: argocd
# spec:
#   generators:
#     - clusters:
#         selector:
#           matchLabels:
#             env: prod
#   template:
#     metadata:
#       name: "web-{{name}}"
#     spec:
#       project: default
#       source:
#         repoURL: https://git.technova.example/platform/apps.git
#         targetRevision: main
#         path: web
#       destination:
#         server: "{{server}}"
#         namespace: production
#       syncPolicy:
#         automated:
#           prune: true
#           selfHeal: true
```

৭.৭ পরিবেশ পৃথকীকরণ ও Cluster Identity

পরিবেশ পৃথকীকরণের স্তরগুলো: Namespace (এক ক্লাস্টারের ভেতরে), ক্লাস্টার (সম্পূর্ণ পৃথক Control Plane), অ্যাকাউন্ট/সাবস্ক্রিপশন (ক্লাউড স্তরে পৃথক), অঞ্চল (ভৌগোলিকভাবে পৃথক)। যত উপরে যাবেন, পৃথকীকরণ তত শক্ত হয়, খরচও তত বাড়ে। Production-এর জন্য সাধারণত অন্তত ক্লাস্টার-স্তরের পৃথকীকরণ সুপারিশ করা হয় — যাতে Development-এর একটি ভুল বা নিরাপত্তা-ত্রুটি Production-কে ছুঁতে না পারে।

Cluster Identity মানে প্রতিটি ক্লাস্টারের একটি স্পষ্ট পরিচয়: নাম (যেমন `technova-prod-a`), অঞ্চল, পরিবেশ, উদ্দেশ্য, মালিক দল, ও সংস্করণ। এই পরিচয় সর্বত্র একইভাবে লেখা থাকা উচিত — ক্লাস্টারের Label-এ, DNS নামে, পর্যবেক্ষণে, লগে, আর খরচের ট্যাগে। পরিচয় অস্পষ্ট হলে ভুল ক্লাস্টারে ভুল কমান্ড চালানোর ঝুঁকি বাড়ে — যেমন ভুল করে Production-এর বদলে Staging-এ ভেবে Production-এই `kubectl delete` চালানো। প্রতিরোধ: প্রতিটি Context-এ স্পষ্ট নাম, `kubectl`-এর জন্য আলাদা kubeconfig ফাইল, আর Production কমান্ডে অতিরিক্ত নিশ্চিতকরণ (যেমন `kubectx` ও `kube-ps1` দিয়ে শেল প্রম্পটে বর্তমান Context দেখানো)।

```
# নিরাপদ Context অভ্যাস
# শেল প্রম্পটে বর্তমান Context দেখান (kube-ps1 বা অনুরূপ)
# Production kubeconfig আলাদা ফাইলে রাখুন
export KUBECONFIG=~/.kube/technova-prod-a.yaml
```

```
kubectl config current-context

# বিপজ্জনক কমান্ডের আগে Context যাচাই করুন
kubectl config current-context && \
  kubectl get namespaces

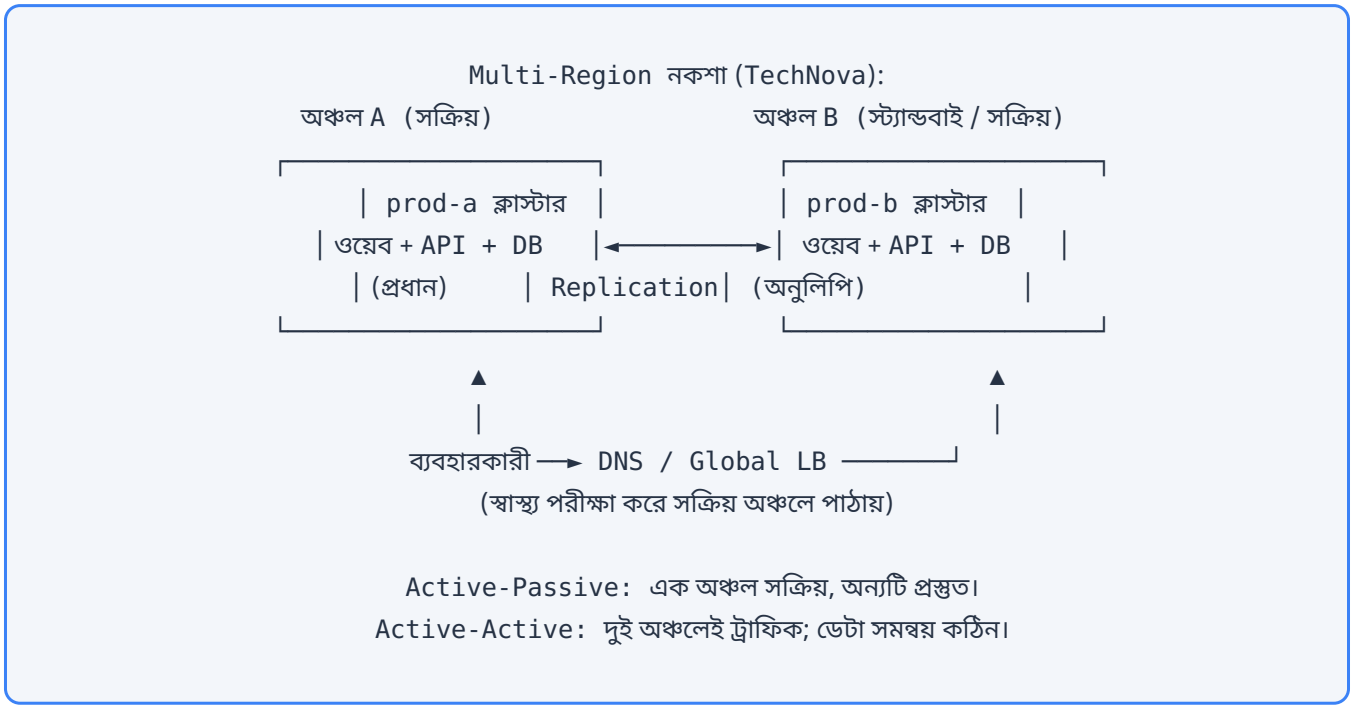
# একটি ক্লাস্টারের পরিচয় Label হিসেবে
kubectl label nodes --all cluster.technova.example/name=technova-prod-a --overwrite
kubectl get nodes -L cluster.technova.example/name
```

> **WARNING:** সবচেয়ে সাধারণ Multi-Cluster দুর্ঘটনা হলো ভুল ক্লাস্টারে কমান্ড চালানো। `kubectl delete namespace production` ভুল Context-এ চালালে ফলাফল বিপর্যয়কর। Production-এর kubeconfig-এ লেখার অনুমতি (Write Access) খুব অল্প কয়েকজনের হাতে রাখুন, আর গুরুত্বপূর্ণ মুছে ফেলার কাজ শুধু GitOps পাইপলাইনের মাধ্যমে করুন — সরাসরি `kubectl delete` নয়।

৭.৮ Multi-Region ক্লাস্টার ও Failure Domain

Failure Domain হলো ব্যর্থতার একটি একক — যেমন একটি Availability Zone, একটি অঞ্চল, একটি ডেটা সেন্টার। নকশার নীতি: গুরুত্বপূর্ণ কাজের চাপকে একাধিক Failure Domain-এ ছড়িয়ে রাখুন, যাতে একটি Domain মারা গেলেও সেবা চলে। Kubernetes-এ এর যন্ত্রগুলো: **Topology Spread Constraints** (অধ্যায় ৬), **Pod Anti-Affinity**, ও একাধিক অঞ্চলে ক্লাস্টার। তবে Multi-Region-এর কঠিন অংশটি হলো ডেটা: অ্যাপের Pod এক অঞ্চল থেকে অন্য অঞ্চলে সরে যেতে পারে, কিন্তু ডেটাবেসের ডেটা নিজে থেকে সরে যায় না। ডেটাবেসের **Replication** (এক অঞ্চল থেকে অন্য অঞ্চলে ডেটা কপি), **Failover** (মূল ডেটাবেস মারা গেলে অনুলিপিকে মূল বানানো), ও **Consistency** (দুই অঞ্চলের ডেটা কতটা মিলবে) — এই সিদ্ধান্তগুলো অ্যাপের স্তরে নিতে হয়, Kubernetes নিজে ডেটা Replication করে না। অধ্যায় ১০-এ স্টোরেজ ও ব্যাকআপে এই বিষয়টি আরও বিস্তারিত আসবে।

Multi-Region-এ আরও একটি বিষয়: অঞ্চলগুলোর মধ্যে নেটওয়ার্ক বিলম্ব ও খরচ। এক অঞ্চলের Pod অন্য অঞ্চলের ডেটাবেসে কথা বললে প্রতিটি অনুরোধে অতিরিক্ত বিলম্ব হয়; অঞ্চলের মধ্যে ডেটা স্থানান্তরের খরচও থাকে। তাই নিয়ম: অ্যাপ ও তার ডেটা একই অঞ্চলে রাখুন; অঞ্চলের মধ্যে শুধু প্রয়োজনীয় Replication ও ব্যাকআপ চলুক।



৭.৯ Workload Placement — কাজ কোথায় চলবে

Workload Placement-এর সিদ্ধান্তে চারটি বিষয় বিবেচনা করুন: (১) **বিলম্ব** — ব্যবহারকারীর কাছাকাছি অঞ্চলে রাখুন; (২) **খরচ** — কিছু অঞ্চলে কম্পিউট সস্তা, কিছুতে ব্যয়বহুল; অঞ্চলের মধ্যে ডেটা স্থানান্তরের খরচও হিসাব করুন; (৩) **সম্মতি** — ডেটা কোন দেশে থাকতে পারবে, আইনে বাঁধা আছে কি না; (৪) **ক্ষমতা** — কোনো অঞ্চলে প্রয়োজনীয় নোডের ধরন বা GPU পাওয়া যায় কি না। Placement-এর বাস্তবায়ন: ক্লাস্টার-স্তরে GitOps-এর Label Selector, ক্লাস্টারের ভেতরে Node Affinity ও Taint/Toleration, আর ক্রস-ক্লাস্টার Placement-এর জন্য **Karmada** বা **Argo CD ApplicationSet**-এর মতো টুল — যারা একটি অ্যাপের সংজ্ঞা নিয়ে সিদ্ধান্ত নেয় কোন ক্লাস্টারে কয়টি রেপ্লিকা যাবে। তবে মনে রাখুন: Placement-এর জটিলতা বাড়লে ডিবাগ করাও কঠিন হয় — "এই Pod কেন এই ক্লাস্টারে?" প্রশ্নের উত্তর যেন Git-এ লেখা থাকে।

৭.১০ Cluster আপগ্রেড — নিরাপদ ক্রম

ক্লাস্টার আপগ্রেডের সঠিক ক্রম: প্রথমে **Control Plane** (API Server, etcd, Controller Manager, Scheduler), তারপর **নোড** — একবারে একটি নোড, পুরোনো নোড Drain করে নতুন নোড যোগ করে। Kubernetes-এর নিয়ম: একবারে একটি **Minor Version** (যেমন 1.31 থেকে 1.32) আপগ্রেড করা যায়; দুটি সংস্করণ লাফ দেওয়া যায় না। আপগ্রেডের আগে যাচাই করুন: (১) আপনার API ব্যবহার (যেমন সরিয়ে দেওয়া — Deprecated — API সংস্করণ) নতুন সংস্করণে চলবে কি না (`kubectl`-এর `pluto` বা `kube-no-trouble` টুল দিয়ে); (২) CNI, CSI, Ingress ও অন্যান্য অ্যাড-অন নতুন সংস্করণের সাথে সামঞ্জস্যপূর্ণ কি না; (৩) etcd ব্যাকআপ নেওয়া হয়েছে কি না। আপগ্রেডের ক্রম: প্রথমে Development ক্লাস্টার, তারপর Staging, সবশেষে Production — আর Production-এর ক্ষেত্রে প্রথমে একটি অঞ্চল, তারপর অন্যটি। আপগ্রেডের সময় **PodDisruptionBudget** মেনে চলুন, যাতে একসাথে অনেক Pod বন্ধ না হয়।

```
# আপগ্রেডের আগে যাচাই
kubectl version
kubectl get nodes
# সরিয়ে দেওয়া API ব্যবহার হচ্ছে কি না (kube-no-trouble উদাহরণ)
kubectl krew install kubent 2>/dev/null || true
```

```
kubectl kubent

# নোড নিরাপদে Drain করা (আপগ্রেডের ধাপ)
kubectl cordon node-203-0-113-21
kubectl drain node-203-0-113-21 --ignore-daemonsets --delete-emptydir-data
# ... নোড আপগ্রেড / প্রতিস্থাপন ...
kubectl uncordon node-203-0-113-21
kubectl get nodes

# আপগ্রেডের পর যাচাই
kubectl -n kube-system get pods
kubectl get pods -A --field-selector=status.phase!=Running
kubectl get --raw /readyz
```

৭.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র অঞ্চল-বিভ্রাট

TechNova-র অঞ্চল A-তে ক্লাউড প্রদানকারীর একটি বড় বিভ্রাট হলো — prod-a ক্লাস্টারের নোডগুলো পৌঁছানো যাচ্ছে না। তাদের Multi-Region পরিকল্পনা কাজ করল কি না, তা পরীক্ষা হলো: DNS-এর স্বাস্থ্য পরীক্ষা (Health Check) prod-a-কে অসুস্থ দেখে ট্রাফিক prod-b-তে পাঠাতে শুরু করল; prod-b-তে ইতিমধ্যে ওয়েব ও API-র রেপ্লিকা চলছিল (Active-Passive নয়, তারা Warm Standby রেখেছিল — কম রেপ্লিকা, প্রয়োজনে স্কেল-আপ); ডেটাবেসের অনুলিপি (Replica) prod-b-তে প্রস্তুত ছিল, আর Failover-এর জন্য একটি লিখিত পদ্ধতি (Runbook) ছিল — কে সিদ্ধান্ত নেবে, কোন কমান্ড চালাবে, কীভাবে যাচাই করবে। পুনরুদ্ধারে লেগেছিল প্রায় ২৫ মিনিট — কারণ Runbook-টি আগে মহড়া দেওয়া হয়েছিল। শিক্ষা: Multi-Region শুধু ক্লাস্টার বানানো নয় — DNS, ডেটাবেস Failover, Runbook ও মহড়া ছাড়া দ্বিতীয় অঞ্চলটি শুধু খরচ, নিরাপত্তা নয়।

> **REAL WORLD:** অনেক দল দ্বিতীয় অঞ্চলে ক্লাস্টার বানায়, কিন্তু কখনো Failover-এর মহড়া দেয় না। বিপর্যয়ের দিন দেখা যায় DNS-এর TTL বেশি, ডেটাবেসের অনুলিপি পিছিয়ে আছে, বা Failover-এর সিদ্ধান্ত নেওয়ার কেউ নেই। নিয়ম: প্রতি তিন মাসে অন্তত একবার Failover মহড়া দিন, আর মহড়ার সময় মাপুন — পুনরুদ্ধারের সময় (RTO) ও ডেটা হারানোর পরিমাণ (RPO) আসলে কত।

৭.১২ সাধারণ ভুল

- **কারণ ছাড়া Multi-Cluster:** একটি ক্লাস্টারে Namespace দিয়ে সমাধান হয় এমন সমস্যায় একাধিক ক্লাস্টার বানানো — পরিচালন-খরচ বাড়ে, লাভ হয় না।
- **ভুল Context-এ কমান্ড:** Production ও Development-এর kubeconfig একসাথে মিশিয়ে রাখা; প্রম্পটে Context না দেখানো।
- **Hub-কে একক ব্যর্থতার বিন্দু বানানো:** Hub ক্লাস্টার মারা গেলে সব Spoke-র নতুন বিতরণ বন্ধ — Hub-এর উচ্চ-উপলব্ধতা নিশ্চিত করুন।
- **ডেটা ছাড়া Multi-Region:** Pod দুই অঞ্চলে ছড়িয়ে দেওয়া, কিন্তু ডেটাবেসের Replication ও Failover-এর পরিকল্পনা না থাকা।
- **আপগ্রেডে দুই সংস্করণ লাফ:** 1.31 থেকে সরাসরি 1.33-এ যাওয়া — সমর্থিত নয়, বিপজ্জনক।

- **ক্লাস্টারগুলোর মধ্যে ভিন্নতা:** প্রতিটি ক্লাস্টার হাতে বানানো হলে কোনো দুটি একরকম থাকে না — Cluster as Code ছাড়া Fleet পরিচালনা অসম্ভব।
- **Failover মহড়া না দেওয়া:** দ্বিতীয় অঞ্চল আছে, কিন্তু কখনো পরীক্ষা করা হয়নি — বিপর্যয়ের দিন তা কাজ করবে, এই অনুমান ভুল।

৭.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একটি ক্লাস্টারে অ্যাপ গেল না	Label Selector অমিল, GitOps Sync ব্যর্থ	ApplicationSet-এর Generator ও Sync Status দেখুন
দুই ক্লাস্টারে কনফিগ ভিন্ন	হাতে পরিবর্তন (Drift), Git-এর বাইরে সম্পাদনা	GitOps-এর Diff দেখুন; হাতে পরিবর্তন নিষিদ্ধ করুন
Failover-এ ট্রাফিক সরল না	DNS TTL বেশি, Health Check ভুল পথ দেখছে	DNS ও Health Check যাচাই করুন; TTL কমান
আপগ্রেডের পর অ্যাপ ভাঙল	সরিয়ে দেওয়া API, অ্যাড-অন অসামঞ্জস্য	আপগ্রেডের আগে kubent দিয়ে যাচাই করুন; Rollback পরিকল্পনা রাখুন
অঞ্চলের মধ্যে বিলম্ব বেশি	অ্যাপ ও ডেটাবেস ভিন্ন অঞ্চলে	অ্যাপকে ডেটার কাছাকাছি রাখুন; Caching বিবেচনা করুন
ক্লাস্টারগুলোর খরচ অপ্রত্যাশিত	প্রতিটি ক্লাস্টারে Control Plane ও অলস নোড	প্রতিটি ক্লাস্টারের ব্যবহার মাপুন; অপ্রয়োজনীয় ক্লাস্টার বন্ধ করুন

৭.১৪ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: প্রতিটি ক্লাস্টারের API Server-এর প্রবেশাধিকার আলাদা করে নিয়ন্ত্রণ করুন; একটি ক্লাস্টারের সার্ভিস অ্যাকাউন্ট যেন অন্য ক্লাস্টারে প্রবেশাধিকার না পায়; ক্লাস্টারগুলোর মধ্যে নেটওয়ার্ক ট্রাফিক এনক্রিপ্টেড (TLS) রাখুন; প্রতিটি ক্লাস্টারে একই নীতি (Policy as Code) প্রয়োগ করুন। খরচ: প্রতিটি ক্লাস্টারের Control Plane-এর খরচ, প্রতিটি অঞ্চলে ন্যূনতম নোডের খরচ, অঞ্চলের মধ্যে ডেটা স্থানান্তরের খরচ — এগুলো যোগ করে Multi-Cluster-এর মোট খরচ হিসাব করুন। একটি ক্লাস্টারে ৩টি নোড যথেষ্ট হলে, দুটি অঞ্চলে ২×৩ = ৬টি নোড লাগে — খরচ দ্বিগুণ। এই খরচ যুক্তিসঙ্গত কি না, তা ব্যবসায়িক ঝুঁকির সাথে মেলান: একটি অঞ্চলের বিভ্রাটে কতক্ষণ সেবা বন্ধ থাকলে কত ক্ষতি হবে? সেই ক্ষতি দ্বিগুণ খরচের চেয়ে বেশি হলে Multi-Region যুক্তিসঙ্গত।

৭.১৪.১ Fleet-এর খরচ নিয়ন্ত্রণ ও Drift শনাক্তকরণ

অনেক ক্লাস্টার চালানোর একটি লুকানো খরচ হলো **Drift** — Git-এ যা লেখা আছে, বাস্তব ক্লাস্টারে তা আর নেই। কেউ জরুরি প্রয়োজনে হাতে একটি Deployment বদলালেন, কেউ একটি ConfigMap সম্পাদনা করলেন — GitOps-এর Sync তখন বারবার সেই পরিবর্তন ফিরিয়ে আনার চেষ্টা করে, আর লগে সতর্কতা জমতে থাকে। এই Drift ধরার উপায়: GitOps টুলের Sync Status নিয়মিত দেখা, আর **Self-Heal** চালু রাখা — যাতে Git-ই সবসময় জেতে। তবে Self-Heal-এরও একটি ঝুঁকি আছে: কেউ যদি ভুল করে Git-এ ভুল কনফিগ লেখেন, তবে সেই ভুল স্বয়ংক্রিয়ভাবে সব ক্লাস্টারে ছড়িয়ে পড়বে। তাই Fleet-স্তরের পরিবর্তনে **Progressive Rollout** ব্যবহার করুন — প্রথমে একটি Development ক্লাস্টার, তারপর একটি Production ক্লাস্টার, তারপর বাকিগুলো — আর প্রতিটি ধাপে স্বাস্থ্য পরীক্ষা করুন। খরচের দিক থেকে আরও একটি

অভ্যাস: প্রতিটি ক্লাস্টারের নোডের ব্যবহার (**Utilization**) মাসে একবার পর্যালোচনা করুন। যে ক্লাস্টারে গড় CPU ব্যবহার ২০%-এর নিচে, সেখানে নোডের সংখ্যা কমানো যায় কি না, বা ক্লাস্টারটি অন্য ক্লাস্টারের সাথে একীভূত করা যায় কি না, তা ভাবুন। একটি অলস ক্লাস্টার শুধু টাকাই খায় না — তার আপগ্রেড, পর্যবেক্ষণ ও নিরাপত্তা-প্যাচের কাজও চলতেই থাকে।

> **REAL WORLD:** TechNova একবার দেখল তাদের Staging ক্লাস্টারটি সপ্তাহে মাত্র কয়েক ঘণ্টা ব্যবহৃত হয়, অথচ সারাক্ষণ চলছে। তারা Staging-কে একটি ছোট ক্লাস্টারে নামিয়ে আনল এবং রাতে ও সপ্তাহান্তে নোডের সংখ্যা শূন্যের কাছাকাছি করার (Scale-to-Zero) ব্যবস্থা করল। খরচ কমল, আর কেউ কিছু টেরও পেল না — কারণ Staging-এর কাজের চাপই ছিল কম।

৭.১৫ অধ্যায়ের সারাংশ

Multi-Cluster Kubernetes মানে একাধিক স্বাধীন ক্লাস্টার পরিচালনা করা — ব্যর্থতার সীমানা ছোট করা, বিলম্ব কমানো, নিয়ন্ত্রক সম্মতি ও কঠোর পৃথকীকরণের জন্য। Hub-and-Spoke ও GitOps-ভিত্তিক স্বাধীন ক্লাস্টারই বর্তমান বাস্তব চর্চা; Federation আকর্ষণীয় হলেও জটিল। Cluster as Code, Fleet Management, স্পষ্ট Cluster Identity, নিরাপদ আপগ্রেড-ক্রম ও নিয়মিত Failover মহড়া ছাড়া Multi-Cluster শুধু খরচ বাড়ায়। সিদ্ধান্তের মূল প্রশ্ন সবসময় একটাই: একটি ক্লাস্টারে Namespace ও নীতি দিয়ে কি কাজটি হয় না?

৭.১৬ জ্ঞান যাচাই

1. কোন পাঁচটি কারণে একাধিক ক্লাস্টার প্রয়োজন হতে পারে? প্রতিটির একটি উদাহরণ দিন।
2. Namespace দিয়ে পৃথকীকরণ ও ক্লাস্টার দিয়ে পৃথকীকরণের পার্থক্য কী?
3. Hub-and-Spoke ও Federated মডেলের পার্থক্য ব্যাখ্যা করুন — কোনটি কখন ভালো?
4. Cluster as Code বলতে কী বোঝায়? হাতে ক্লাস্টার বানানোর সমস্যা কী?
5. Fleet Management-এ Label Selector কীভাবে কাজ করে?
6. Failure Domain কী? একটি অ্যাপকে একাধিক Failure Domain-এ ছড়ানোর যন্ত্রগুলো কী কী?
7. ক্লাস্টার আপগ্রেডের সঠিক ক্রম কী? কেন একবারে দুই Minor Version লাফ দেওয়া যায় না?
8. Active-Passive ও Active-Active Multi-Region নকশার পার্থক্য কী? ডেটার দিক থেকে কোনটি কঠিন?
9. ভুল Context-এ কমান্ড চালানো রোধে কোন তিনটি অভ্যাস সাহায্য করে?
10. Multi-Cluster-এর খরচ হিসাব করতে কোন কোন উপাদান ধরবেন?

৭.১৭ ব্যবহারিক অনুশীলন

1. আপনার মেশিনে `kind` বা `minikube` দিয়ে দুটি ছোট ক্লাস্টার বানান (বা দুটি Context ব্যবহার করুন)। দুটি ক্লাস্টারে আলাদা নামে একই অ্যাপ Deploy করুন, তারপর একটি শেল লুপ লিখুন যা দুটি ক্লাস্টারের নোড ও Pod-এর অবস্থা একসাথে দেখায়। ভুল Context-এ কমান্ড চালানোর ঝুঁকি কমাতে আপনার শেল প্রম্পটে Context দেখানোর ব্যবস্থা করুন।
2. TechNova-র জন্য একটি Multi-Cluster সিদ্ধান্ত-নথি লিখুন: একটি ক্লাস্টার না একাধিক — আপনার সুপারিশ কী, কেন; কোন পরিবেশ কোন ক্লাস্টারে যাবে; প্রতিটি ক্লাস্টারের নাম, অঞ্চল ও উদ্দেশ্যের তালিকা; আর আপগ্রেডের ক্রম। সিদ্ধান্তের পেছনে খরচ ও ঝুঁকির যুক্তি লিখুন।

3. একটি Failover Runbook লিখুন: অঞ্চল A-র ক্লাস্টার অসুস্থ হলে কে সিদ্ধান্ত নেবে, DNS কীভাবে বদলাবে, ডেটাবেস Failover কীভাবে হবে, কীভাবে যাচাই করবেন যে অঞ্চল B সঠিকভাবে ট্রাফিক নিচ্ছে, আর অঞ্চল A ফিরে এলে কীভাবে ফিরবেন — প্রতিটি ধাপে দায়িত্বপ্রাপ্ত ভূমিকা ও যাচাই-কমান্ড লিখুন।

অধ্যায় ৮ • PART II — Advanced Kubernetes & Cloud-Native Engineering

Chapter 8 — Advanced Kubernetes Networking

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **CNI (Container Network Interface)** কীভাবে Pod-কে IP দেয় ও নেটওয়ার্ক সংযোগ করে, তা ব্যাখ্যা করতে; **Service** (ClusterIP, NodePort, LoadBalancer), **CoreDNS**, **Ingress** ও **Gateway API**-র ভূমিকা আলাদা করতে; **Network Policy** দিয়ে Pod-এর মধ্যে ট্রাফিক সীমাবদ্ধ করতে; **Service Mesh**-এর নেটওয়ার্কিং (Sidecar, mTLS, Traffic Splitting) বুঝতে; এবং নেটওয়ার্ক সমস্যা ডিবাগ করতে `kubectl`, `nslookup`, `curl` ও পর্যবেক্ষণ টুল ব্যবহার করতে।

- Pod নেটওয়ার্কিং মডেলের তিনটি নিয়ম ব্যাখ্যা করা।
- CNI প্লাগইনের (Calico, Cilium, Flannel) পার্থক্য তুলনা করা।
- Service-এর ধরনগুলো ও kube-proxy-র ভূমিকা বোঝা।
- CoreDNS-এর কাজ ও DNS সমস্যা ডিবাগ করা।
- Network Policy লেখা ও পরীক্ষা করা।
- Ingress বনাম Gateway API-র পার্থক্য ও স্থানান্তর পরিকল্পনা করা।
- Service Mesh-এর সুবিধা ও খরচের হিসাব করা।

৮.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

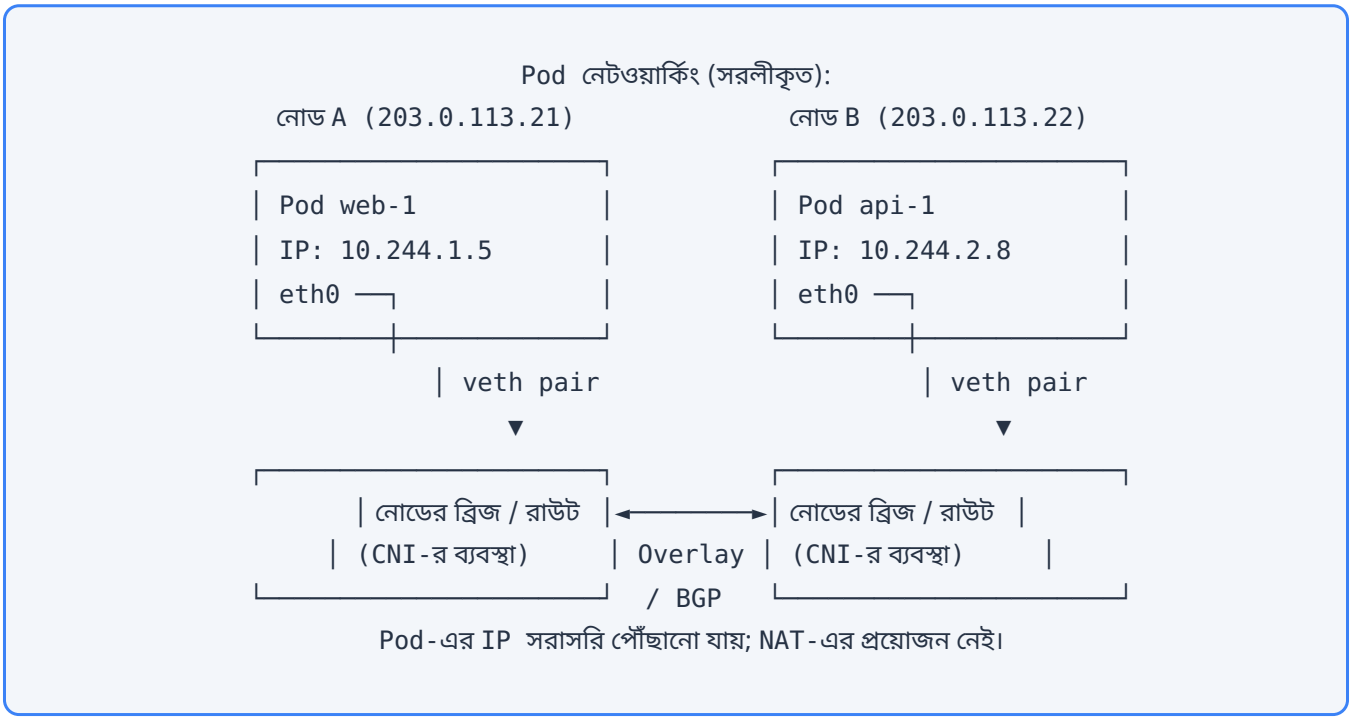
Pod-এর IP ক্ষণস্থায়ী, Service-এর স্থির নাম ও IP দিয়ে Pod-গুলোকে খুঁজে পাওয়া যায়, আর Ingress দিয়ে বাইরের ট্রাফিক ক্লাস্টারে ঢোকে — এই ধারণাগুলো থাকলেই চলবে। IP ঠিকানা, DNS, TCP পোর্ট ও HTTP পথ (Path)-এর প্রাথমিক ধারণাও প্রয়োজন। মনে রাখুন: Kubernetes নেটওয়ার্কিং-এর মূল প্রশ্নগুলো সবসময় একই — “এই Pod-এর IP কী?”, “কে কার সাথে কথা বলতে পারে?”, “বাইরের ট্রাফিক ভেতরে কীভাবে ঢুকবে?”, “নাম থেকে IP কীভাবে পাওয়া যায়?” এই অধ্যায়ে প্রতিটি প্রশ্নের উত্তরের ভেতরের যন্ত্রটি দেখব।

> **CONCEPT:** Kubernetes নেটওয়ার্ক মডেলের তিনটি মৌলিক নিয়ম: (১) প্রতিটি Pod-এর নিজস্ব IP থাকে, আর সব Pod NAT ছাড়াই একে অপরের সাথে কথা বলতে পারে; (২) নোডের Agent (kubelet) Pod-এর IP দিয়ে সরাসরি কথা বলতে পারে; (৩) Pod নিজেকে যে IP দিয়ে দেখে, অন্য Pod-ও তাকে সেই একই IP দিয়ে দেখে। এই নিয়মগুলোর কারণে Pod-কে একটি ছোট ভার্চুয়াল মেশিনের মতো ভাবা যায় — তার নিজের IP, নিজের পোর্ট, আর সরাসরি যোগাযোগ।

৮.৩ CNI আর্কিটেকচার — Pod কীভাবে IP পায়

CNI (Container Network Interface) হলো একটি আদর্শ — kubelet যখন Pod বানাতে বলে, Container Runtime CNI প্লাগইনকে ডাকে: “এই Pod-এর জন্য একটি নেটওয়ার্ক ইন্টারফেস বানাও, একটি IP দাও, আর রাউটিং ঠিক করো।” CNI প্লাগইন Pod-এর **Network Namespace**-এ একটি ভার্চুয়াল ইন্টারফেস (যেমন `eth0`) যোগ করে, IP বরাদ্দ করে, আর নোডের মধ্যে প্যাকেট পৌঁছানোর ব্যবস্থা করে। বিভিন্ন CNI-র বাস্তবায়ন ভিন্ন: **Flannel** সহজ — নোডগুলোর মধ্যে একটি Overlay নেটওয়ার্ক বানায় (VXLAN টানেল); **Calico** নেটওয়ার্ক নীতি (Network Policy) ও BGP রাউটিংয়ে

শক্তিশালী; **Cilium** eBPF ব্যবহার করে — কার্নেল-স্তরে দ্রুত প্যাকেট প্রক্রিয়া, গভীর পর্যবেক্ষণ (Observability) ও Layer-7 নীতি দেয়। কোনটি বেছে নেবেন? ছোট ক্লাস্টারে Flannel-এর সহজতা যথেষ্ট; Network Policy ও নিরাপত্তা গুরুত্বপূর্ণ হলে Calico; গভীর পর্যবেক্ষণ, Layer-7 নীতি ও Service Mesh-বিকল্প চাইলে Cilium। মনে রাখুন: CNI বদলানো কঠিন — ক্লাস্টার বানানোর সময় সিদ্ধান্ত নিন, পরে বদলাতে হলে ক্লাস্টার আবার বানাতে হতে পারে।




```
# Pod-এর IP ও নোড দেখা
kubectl get pods -o wide -n production
kubectl get pods -n production -o custom-columns=NAME:.metadata.name,IP:.status.podIP,N

# CNI Pod-গুলো চলছে কি না
kubectl -n kube-system get pods -l k8s-app=calico-node
kubectl -n kube-system get pods -l k8s-app=cilium
kubectl -n kube-system get pods -l app=flannel

# Pod-এর ভেতর থেকে নেটওয়ার্ক পরীক্ষা (অস্থায়ী ডিবাগ Pod)
kubectl run netdebug --rm -it --image=nicolaka/netshoot -- /bin/bash
# Pod-এর ভেতরে: ip addr, ip route, ping, curl, nslookup
```

৮.৪ Service নেটওয়ার্কিং ও kube-proxy

Pod-এর IP বদলায়, কিন্তু Service-এর IP ও DNS নাম স্থির থাকে। **ClusterIP Service** ক্লাস্টারের ভেতরে একটি স্থির ভার্চুয়াল IP দেয়; **NodePort** প্রতিটি নোডের একটি পোর্ট খুলে দেয়; **LoadBalancer** ক্লাউড প্রদানকারীর লোড ব্যালান্সার চায় (On-premises-এ MetalLB-র মতো সমাধান লাগে); **Headless Service** (`clusterIP: None`) কোনো ভার্চুয়াল IP দেয় না — DNS সরাসরি Pod-গুলোর IP ফেরত দেয়, StatefulSet-এর জন্য উপযোগী। Service-এর পেছনে আসল কাজটি করে **kube-proxy**: প্রতিটি নোডে চলে, Service ও Endpoints Watch করে, আর নোডের **iptables** বা **IPVS** নিয়ম লেখে — যাতে Service-এর IP-তে আসা প্যাকেট সঠিক Pod-এ যায়। kube-proxy নিজে ট্রাফিক বহন করে না

(iptables মোডে) — সে শুধু নিয়ম লেখে, প্যাকেট কার্নেল-স্তরে রাউট হয়। এই কারণে kube-proxy-র কর্মক্ষমতা সাধারণত সমস্যা হয় না; সমস্যা হয় যখন Endpoints-এর তালিকা ভুল হয় — যেমন Readiness Probe ব্যর্থ হওয়ায় Pod Endpoints থেকে বাদ পড়েছে।

```
# Service উদাহরণ (TechNova ওয়েব + API)
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: production
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: production
spec:
  type: ClusterIP
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 3000
---
# Service-এর Endpoints দেখা — Pod-গুলো যুক্ত আছে কি না
kubectl get endpoints web -n production
kubectl get endpointslices -n production
kubectl describe service web -n production

# Service-এর ভেতর থেকে পরীক্ষা
kubectl run curl-test --rm -it --image=curlimages/curl -- \
  curl -s -o /dev/null -w "%{http_code}\n" http://web.production.svc.cluster.local/
```

Service ধরন	কোথা থেকে পৌঁছানো যায়	উপযুক্ত ব্যবহার

ClusterIP	শুধু ক্লাস্টারের ভেতর	ভেতরের সার্ভিস-থেকে-সার্ভিস যোগাযোগ
NodePort	নোডের IP + পোর্ট দিয়ে	পরীক্ষা, On-premises-এ সীমিত ব্যবহার
LoadBalancer	ইন্টারনেট / বাইরের নেটওয়ার্ক	ক্লাউডে বাইরের ট্রাফিকের প্রবেশদ্বার
Headless	DNS সরাসরি Pod IP দেয়	StatefulSet, নিজে Load Balancing করা ক্লায়েন্ট
ExternalName	বাইরের DNS নামে পাঠায়	ক্লাস্টারের বাইরের সার্ভিসকে ভেতরের নাম দেওয়া

৮.৫ CoreDNS — নাম থেকে ঠিকানা

CoreDNS হলো ক্লাস্টারের DNS সার্ভার। Pod-এর ভেতরে `/etc/resolv.conf`-এ CoreDNS-এর ঠিকানা লেখা থাকে; Pod যখন `web.production.svc.cluster.local` নামটি জিজ্ঞেস করে, CoreDNS Service-এর IP ফেরত দেয়। Headless Service-এর ক্ষেত্রে CoreDNS Pod-গুলোর IP-র তালিকা দেয়। DNS ব্যর্থ হলে সার্ভিসগুলো একে অপরকে খুঁজে পায় না — তাই DNS সমস্যা প্রায়শই "অ্যাপ ভেঙে গেছে" বলে মনে হয়, অথচ আসল কারণ DNS। ডিবাগের উপায়: Pod-এর ভেতরে `nslookup web.production.svc.cluster.local` চালান; সাড়া না পেলে CoreDNS Pod-গুলো চলছে কি না দেখুন (`kubectl -n kube-system get pods -l k8s-app=kube-dns`), আর Pod-এর `resolv.conf` দেখুন। একটি সাধারণ ভুল: অ্যাপে Service-এর নামের বদলে Pod-এর IP হার্ডকোড করা — Pod মারা গেলে IP বদলে যায়, আর অ্যাপ ভেঙে যায়। সবসময় Service-এর DNS নাম ব্যবহার করুন।

```
# DNS পরীক্ষা
kubectl run dns-test --rm -it --image=busybox:1.36 -- nslookup web.production.svc.clust
kubectl run dns-test --rm -it --image=busybox:1.36 -- nslookup kubernetes.default.svc.c

# CoreDNS-এর অবস্থা ও লগ
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=50

# Pod-এর DNS কনফিগ দেখা
kubectl run dns-test --rm -it --image=busybox:1.36 -- cat /etc/resolv.conf
```

৮.৬ Network Policy — কে কার সাথে কথা বলতে পারে

ডিফল্টভাবে Kubernetes-এ সব Pod সব Pod-এর সাথে কথা বলতে পারে — এটি সুবিধাজনক, কিন্তু নিরাপদ নয়। একটি Pod আপসিত (Compromised) হলে আক্রমণকারী পুরো ক্লাস্টারে ঘুরে বেড়াতে পারে (**Lateral Movement**)। **Network Policy** এই স্বাধীনতা সীমাবদ্ধ করে: কোন Pod কোন Pod-এর সাথে, কোন পোর্টে, কোন দিকে (Ingress/Egress) কথা বলতে পারবে, তা ঘোষণা করা যায়। Network Policy শুধু ঘোষণা — তা কার্যকর করে CNI প্লাগইন (Calico, Cilium); Flannel-এর মতো সহজ CNI Network Policy কার্যকর করে না, তাই নীতি লেখার আগে CNI-র সমর্থন যাচাই করুন।

একটি গুরুত্বপূর্ণ নিয়ম: কোনো Namespace-এ একটি Network Policy তৈরি হলেই সেই Namespace-এর Pod-গুলো "বিচ্ছিন্ন" (Isolated) হয়ে যায় — শুধু নীতিতে অনুমোদিত ট্রাফিকই চলে। তাই প্রথম নীতি লেখার সময় সাবধান: ভুল নীতি বৈধ ট্রাফিকও বন্ধ করে দিতে পারে। নিরাপদ পদ্ধতি: প্রথমে **Default-Deny** নীতি দিয়ে সব বন্ধ করুন, তারপর প্রয়োজনীয় ট্রাফিক একে একে খুলুন — আর প্রতিটি ধাপে পরীক্ষা করুন।

```
# Default-Deny: Namespace-এর সব Ingress ও Egress বন্ধ
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: production
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
---
# ওয়েব Pod শুধু API Pod-এর সাথে কথা বলতে পারবে (Egress)
# আর Ingress Controller থেকে ট্রাফিক নেবে (Ingress)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: web-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              name: ingress-nginx
      ports:
        - port: 8080
    - from:
        - podSelector:
            matchLabels:
              app: api
      ports:
        - port: 8080
  egress:
```

```
    - to:
        - podSelector:
            matchLabels:
              app: api
      ports:
        - port: 3000
    - to:
        - namespaceSelector:
            matchLabels:
              name: kube-system
      ports:
        - port: 53
          protocol: UDP
        - port: 53
          protocol: TCP
---
# ডেটাবেস: শুধু API Pod থেকে, শুধু 5432 পোর্টে
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: postgres
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: api
      ports:
        - port: 5432
```

> **WARNING:** Network Policy-তে DNS (পোর্ট 53) ভুলে বন্ধ করে দিলে Pod কোনো নামই খুঁজে পাবে না — অ্যাপ "ডেটাবেস পাওয়া যাচ্ছে না" বলবে, অথচ আসল কারণ DNS বন্ধ। Egress নীতি লেখার সময় সবসময় kube-system Namespace-এর DNS (UDP ও TCP, পোর্ট 53) খোলা রাখুন, আর নীতি প্রয়োগের পরপরই অ্যাপ পরীক্ষা করুন।

৮.৭ Ingress ও Gateway API — বাইরের ট্রাফিকের প্রবেশদ্বার

বাইরের ট্রাফিক ক্লাস্টারে ঢোকার পথ: **Ingress** হলো পুরোনো, ব্যাপকভাবে ব্যবহৃত উপায় — একটি Ingress অবজেক্টে লেখা থাকে কোন হোস্টনাম ও পথ (Path) কোন Service-এ যাবে; আর **Ingress Controller** (যেমন NGINX Ingress Controller) সেই নিয়ম পড়ে প্রকৃত রাউটিং করে। Ingress-এর সীমা: এর API সহজ, কিন্তু উন্নত বৈশিষ্ট্য (Header-ভিত্তিক রাউটিং, Traffic Splitting, একাধিক দলের ভাগ করা Gateway) প্রতিটি Controller নিজের মতো করে Annotation দিয়ে করে — ফলে এক Controller থেকে অন্য Controller-এ যাওয়া কঠিন।

Gateway API (v1, স্থিতিশীল) হলো Ingress-এর উত্তরসূরি — Kubernetes-এর আদর্শ API, যা ভূমিকা আলাদা করে: **GatewayClass** (কোন ধরনের Gateway — যেমন NGINX, Envoy), **Gateway** (প্রবেশদ্বার নিজে — কোন পোর্ট, কোন হোস্টনাম, TLS সার্টিফিকেট), **HTTPRoute** (কোন পথ/হেডার কোন Service-এ যাবে), আর **GRPCRoute** / **TCPRoute**। সুবিধা: একাধিক দল একই Gateway ভাগ করে নিজেদের Route লিখতে পারে; Traffic Splitting (যেমন ৯০% পুরোনো সংস্করণে, ১০% নতুন সংস্করণে) আদর্শভাবে করা যায়; আর Controller বদলালেও Route-এর সংজ্ঞা একই থাকে। নতুন প্রকল্পে Gateway API ব্যবহার করাই সুপারিশ; বিদ্যমান Ingress থাকলে ধীরে ধীরে স্থানান্তর করুন — একসাথে সব বদলাবেন না।

```
# Gateway API উদাহরণ (TechNova)
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: technova-gateway
  namespace: production
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      port: 80
      protocol: HTTP
      hostname: app.technova.example
    - name: https
      port: 443
      protocol: HTTPS
      hostname: app.technova.example
      tls:
        mode: Terminate
        certificateRefs:
          - name: technova-tls
            kind: Secret
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
```

```
metadata:
  name: web-route
  namespace: production
spec:
  parentRefs:
    - name: technova-gateway
      namespace: production
  hostnames:
    - app.technova.example
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:
        - name: api
          port: 80
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: web
          port: 80
          weight: 90
        - name: web-canary
          port: 80
          weight: 10
---
# Ingress উদাহরণ (তুলনার জন্য — বিদ্যমান সেটআপে)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: technova-ingress
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
```

```
  - hosts:
      - app.technova.example
    secretName: technova-tls
rules:
  - host: app.technova.example
    http:
      paths:
        - path: /api
          pathType: Prefix
          backend:
            service:
              name: api
              port:
                number: 80
        - path: /
          pathType: Prefix
          backend:
            service:
              name: web
              port:
                number: 80
```

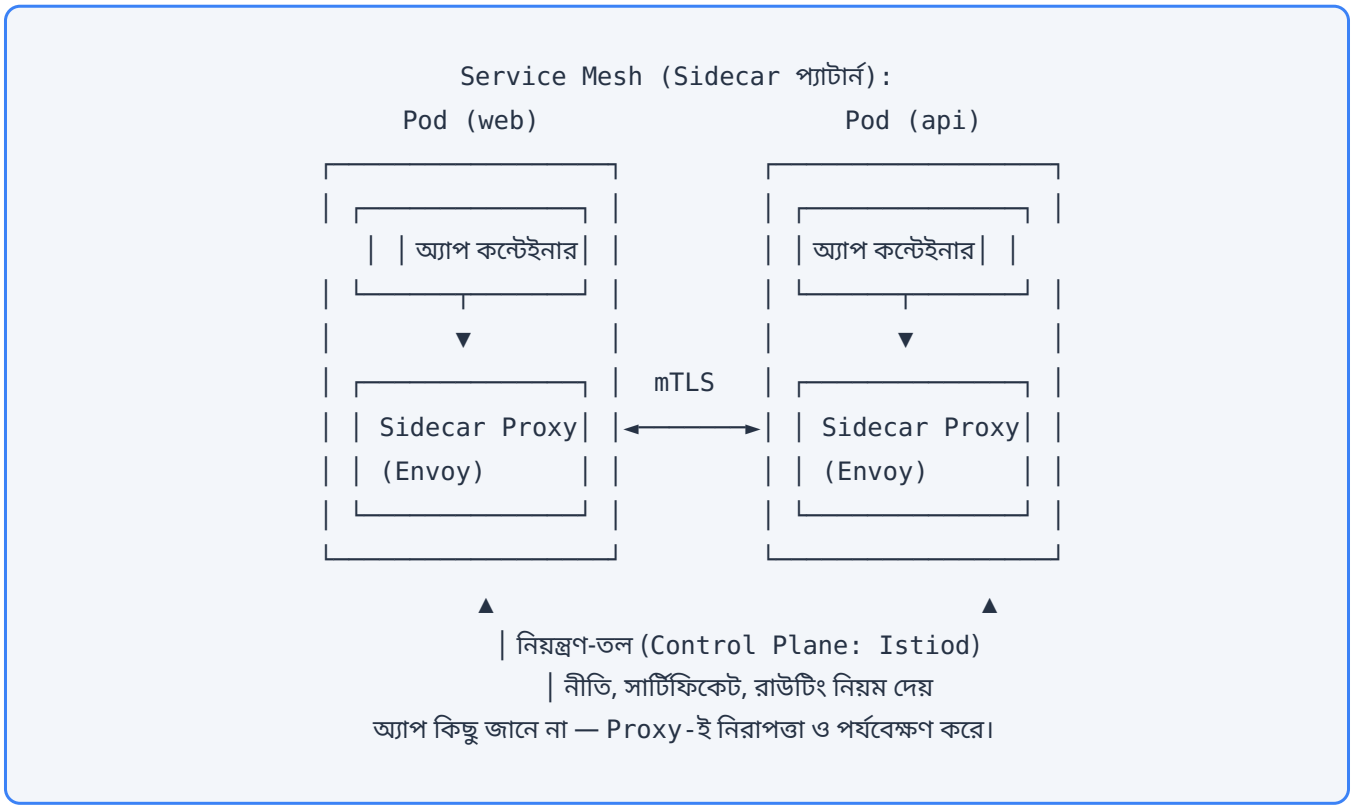
বিষয়	Ingress	Gateway API
API-র মর্যাদা	স্থিতিশীল, ব্যাপক ব্যবহৃত	v1 স্থিতিশীল (2023 থেকে)
ভূমিকা ভাগ করা	সীমিত — একটি অবজেক্টে সবকিছু	Gateway, Route, Policy আলাদা — দলভেদে ভাগ করা যায়
Traffic Splitting	Annotation-নির্ভর, Controller-ভেদে ভিন্ন	আদর্শভাবে Weight দিয়ে
Controller বদলানো	কঠিন — Annotation বদলাতে হয়	সহজ — Route একই থাকে
কখন ব্যবহার করবেন	বিদ্যমান সেটআপ, সহজ প্রয়োজন	নতুন প্রকল্প, একাধিক দল, উন্নত রাউটিং

৮.৮ Load Balancing — ট্রাফিক ভাগ করা

Load Balancing-এর স্তরগুলো: **Layer-4** (TCP/UDP — IP ও পোর্ট দেখে, দ্রুত, কিন্তু HTTP-র ভেতরে দেখে না) ও **Layer-7** (HTTP — হোস্টনাম, পথ, হেডার দেখে রাউট করে)। Kubernetes-এ LoadBalancer Service সাধারণত Layer-4 লোড ব্যালান্সার দেয়; Ingress/Gateway দেয় Layer-7। Pod-স্তরে Load Balancing করে kube-proxy (iptables/IPVS — Round Robin-এর মতো) বা CNI (Cilium-এর eBPF-ভিত্তিক)। একটি সাধারণ ভুল: LoadBalancer Service-এর **External Traffic Policy** না বোঝা — `Cluster` (ডিফল্ট) মানে ট্রাফিক যেকোনো নোডে এসে অন্য নোডের Pod-এ যেতে পারে (অতিরিক্ত Hop), আর `Local` মানে শুধু সেই নোডের Pod-এ যাবে যেখানে ট্রাফিক এসেছে (কম বিলম্ব, কিন্তু নোডে Pod না থাকলে ট্রাফিক ফেলে দেওয়া হয়)। উৎস IP (Source IP) সংরক্ষণ করতে চাইলে `Local` প্রয়োজন — তবে নিশ্চিত করুন প্রতিটি নোডে Pod আছে।

৮.৯ Service Mesh নেটওয়ার্কিং — Sidecar প্যাটার্ন

Service Mesh (যেমন Istio, Linkerd) প্রতিটি Pod-এর পাশে একটি **Sidecar Proxy** (সাধারণত Envoy) বসায় — Pod-এর সব নেটওয়ার্ক ট্রাফিক এই Proxy-র মধ্য দিয়ে যায়। এতে পাওয়া যায়: **mTLS** (সার্ভিসগুলোর মধ্যে এনক্রিপ্টেড ও পারস্পরিক প্রমাণীকৃত যোগাযোগ), **Traffic Management** (Canary, Circuit Breaking, Retry, Timeout), **Observability** (প্রতিটি অনুরোধের Trace, Metric), ও **Policy** (কে কাকে ডাকতে পারবে)। তবে খরচও আছে: প্রতিটি Pod-এ অতিরিক্ত কন্টেইনার (CPU/মেমোরি), অতিরিক্ত নেটওয়ার্ক Hop (বিলম্ব), আরেকটি Control Plane (Istiod) পরিচালনা, ও ডিবাগের জটিলতা ("সমস্যাটি অ্যাপে নাকি Proxy-তে?")। সিদ্ধান্ত: কয়েকটি সার্ভিসের ছোট ক্লাস্টারে Service Mesh অপ্রয়োজনীয় — Network Policy ও Gateway API-ই যথেষ্ট। অনেক সার্ভিস, কঠোর mTLS প্রয়োজন, বা বিস্তারিত Traffic Management চাইলে Mesh বিবেচনা করুন; আর তখন Linkerd-এর হালকা নকশা বনাম Istio-র পূর্ণ বৈশিষ্ট্য — দলের পরিচালন-ক্ষমতা দেখে বেছে নিন।



CONCEPT: Service Mesh-এর একটি বিকল্প হলো **Ambient Mesh** (Istio-র নতুন মোড) ও Cilium-এর eBPF-ভিত্তিক পদ্ধতি — যেখানে প্রতিটি Pod-এ Sidecar না বসিয়ে নোড-স্তরে Proxy চলে। এতে সম্পদের খরচ কমে, কিন্তু ধারণাটি নতুন — প্রোডাকশনে নেওয়ার আগে আপনার ব্যবহারের ক্ষেত্রে পরীক্ষা করুন।

৮.১০ নেটওয়ার্ক পর্যবেক্ষণ (Observability) ও সমস্যা সমাধান

নেটওয়ার্ক সমস্যা ডিবাগের পদ্ধতিগত ক্রম: (১) Pod চলছে কি না (`kubectl get pods`); (২) Service-এর Endpoints-এ Pod আছে কি না; (৩) DNS কাজ করছে কি না (`nslookup`); (৪) Pod-এর ভেতর থেকে সরাসরি Pod IP-তে `curl` করা যায় কি না (Pod-থেকে-Pod); (৫) Service IP-তে `curl` (Service স্তর); (৬) Ingress/Gateway দিয়ে বাইরে থেকে (প্রবেশদ্বার স্তর)। কোন স্তরে ভাঙে, তা জানলেই কারণের এলাকা ছোট হয়। পর্যবেক্ষণের জন্য: Cilium-এর **Hubble** (Pod-এর মধ্যে ট্রাফিকের প্রবাহ দেখায়), Prometheus-এর নেটওয়ার্ক মেট্রিক, আর Service Mesh-এর Trace। একটি সাধারণ

ভুল: শুধু অ্যাপের লগ দেখা — নেটওয়ার্ক সমস্যায় অ্যাপের লগে “Connection refused” বা “Timeout” ছাড়া আর কিছু থাকে না; স্তরে স্তরে পরীক্ষা করলেই আসল স্তরটি ধরা পড়ে।

```
# স্তরে স্তরে নেটওয়ার্ক ডিবাগ
kubectl get pods -n production -o wide
kubectl get endpoints web api -n production
kubectl run netdebug --rm -it --image=nicolaka/netshoot -- /bin/bash
# Pod-এর ভেতরে:
#   nslookup web.production.svc.cluster.local
#   curl -v http://10.244.1.5:8080/        # সরাসরি Pod IP
#   curl -v http://web.production:80/       # Service নাম
#   ip route
#   ss -tulpn
# বাইরে থেকে:
curl -v https://app.technova.example/api/health
kubectl -n ingress-nginx get pods
kubectl -n ingress-nginx logs -l app.kubernetes.io/component=controller --tail=50
```

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Pod একে অপরকে Ping করতে পারে না	CNI সমস্যা, নোডের নেটওয়ার্ক ছিন্ন	CNI Pod-এর লগ দেখুন; নোডের রাউট দেখুন
Service নামে সংযোগ হয় না	DNS ব্যর্থ, Endpoints খালি	`nslookup` ও `kubectl get endpoints` দেখুন
Service IP-তে সংযোগ হয় না, Pod IP-তে হয়	kube-proxy / iptables সমস্যা, Readiness Probe ব্যর্থ	Endpoints ও Probe দেখুন; kube-proxy লগ দেখুন
Ingress দিয়ে 404 / 502	ভুল পথ/হোস্ট, Backend Pod অসুস্থ	Ingress/Gateway Route ও Backend Service যাচাই করুন
Network Policy-র পর ট্রাফিক বন্ধ	নীতিতে প্রয়োজনীয় পোর্ট/DNS বাদ পড়েছে	নীতি সরিয়ে পরীক্ষা করুন; ধাপে ধাপে আবার যোগ করুন
ধীরগতি, Timeout	অঞ্চলের মধ্যে ট্রাফিক, MTU অমিল, DNS ধীর	অ্যাপ ও ডেটা একই অঞ্চলে রাখুন; MTU যাচাই করুন

৮.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র “হঠাৎ সব API বন্ধ”

একদিন TechNova-র ওয়েব অ্যাপ হঠাৎ API-কে ডাকতে পারল না। প্রথমে মনে হলো API মারা গেছে — কিন্তু API-র Pod-গুলো চলছিল। স্তরে স্তরে পরীক্ষায় দেখা গেল: ওয়েব Pod থেকে API-র Pod IP-তে `curl` হয়, কিন্তু Service নামে হয় না — DNS সমস্যা। CoreDNS-এর লগে দেখা গেল একটি নতুন Network Policy DNS ট্রাফিক (পোর্ট 53) বন্ধ করে দিয়েছে — নীতিটি লেখার সময় Egress-এ শুধু API-র পোর্ট খোলা হয়েছিল, DNS বাদ পড়েছিল। নীতিতে DNS যোগ করতেই সব ঠিক হলো। শিক্ষা: Network Policy পরিবর্তনের পরপরই DNS ও অ্যাপ পরীক্ষা করুন; আর নীতি পরিবর্তন Production-এ যাওয়ার আগে Staging-এ পরীক্ষা করুন।

৮.১২ সাধারণ ভুল

- **Pod IP হার্ডকোড করা:** Pod-এর IP বদলায় — সবসময় Service-এর DNS নাম ব্যবহার করুন।
- **Network Policy লিখে পরীক্ষা না করা:** নীতি প্রয়োগের পর DNS ও অ্যাপ পরীক্ষা না করলে বৈধ ট্রাফিক বন্ধ হয়ে যেতে পারে।
- **CNI না বুঝে বেছে নেওয়া:** Network Policy লাগবে, অথচ Flannel বসানো — নীতি কার্যকরই হবে না।
- **Ingress ও Gateway একসাথে গুলিয়ে ফেলা:** দুটি প্রবেশদ্বার একসাথে চালালে রাউটিং দ্বন্দ্ব হয় — একটি পথ বেছে নিন।
- **LoadBalancer-এর External Traffic Policy না বোঝা:** Source IP হারিয়ে গেলে লগ ও Rate Limit ভুল হয়।
- **Service Mesh সব সমস্যার সমাধান ভাবা:** Mesh ছাড়াই Network Policy, mTLS (অ্যাপ-স্তরে) ও Gateway API দিয়ে অনেক কিছু করা যায়।
- **TLS সার্টিফিকেটের মেয়াদ না দেখা:** সার্টিফিকেটের মেয়াদ শেষ হলে Gateway হঠাৎ ট্রাফিক ফিরিয়ে দেবে — cert-manager দিয়ে স্বয়ংক্রিয় নবায়ন করুন।

৮.১৩ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Default-Deny Network Policy দিয়ে শুরু করুন; Namespace-এর মধ্যে ও বাইরে ট্রাফিক সীমাবদ্ধ করুন; বাইরের প্রবেশদ্বারে TLS বাধ্যতামূলক করুন (HTTP-কে HTTPS-এ পাঠান); Ingress/Gateway-এ Rate Limit ও WAF (Web Application Firewall) বিবেচনা করুন; Service-থেকে-Service যোগাযোগে mTLS (Mesh বা অ্যাপ-স্তরে) ব্যবহার করুন। খরচ: প্রতিটি LoadBalancer Service ক্লাউডে একটি লোড ব্যালান্সারের খরচ তৈরি করে — অনেক Service-এ LoadBalancer না দিয়ে একটি Ingress/Gateway-এর পেছনে সব Route রাখুন; অঞ্চলের মধ্যে ট্রাফিকের খরচ হিসাব করুন; Service Mesh-এর Sidecar-গুলোর CPU/মেমোরি খরচ (প্রতিটি Pod-এ অতিরিক্ত) হিসাব করুন।

৮.১৩.১ কর্মক্ষমতার সূক্ষ্ম বিষয়: MTU, DNS Caching ও Connection Reuse

নেটওয়ার্কের কর্মক্ষমতা প্রায়শই বড় সিদ্ধান্তে নয়, ছোট ছোট বিষয়ে নির্ধারিত হয়। প্রথমটি **MTU (Maximum Transmission Unit)** — Overlay নেটওয়ার্ক (যেমন VXLAN) ব্যবহার করলে প্রতিটি প্যাকেটে অতিরিক্ত হেডার যোগ হয়; Pod-এর MTU যদি নোডের MTU-র চেয়ে বড় হয়, তবে প্যাকেট ভেঙে যায় (Fragmentation) বা ফেলে দেওয়া হয় — ফলে বড় ফাইল স্থানান্তর ধীর হয় বা ব্যর্থ হয়। CNI-র ডিফল্ট MTU সাধারণত ঠিক থাকে, তবে ক্লাউডের নেটওয়ার্কে MTU বদলালে CNI-র MTU-ও মেলাতে হয়। দ্বিতীয়টি DNS: প্রতিটি নতুন সংযোগে DNS জিজ্ঞাসা করলে বিলম্ব বাড়ে; **NodeLocal DNSCache** (প্রতিটি নোডে একটি স্থানীয় DNS ক্যাশ) DNS-এর বিলম্ব ও CoreDNS-এর চাপ দুটোই কমায় — বেশি ট্রাফিকের ক্লাস্টারে এটি একটি কার্যকর উন্নতি। তৃতীয়টি **Connection Reuse** — অ্যাপ যদি প্রতিটি অনুরোধে নতুন TCP সংযোগ খোলে, তবে Handshake-এর খরচ বাড়ে; Keep-Alive ও সংযোগ-পুল (Connection Pool) ব্যবহার করলে একই সংযোগে অনেক অনুরোধ যায়। এই তিনটি বিষয় পরীক্ষা করা সহজ: Pod-এর ভেতরে বড় ফাইল স্থানান্তর, DNS-এর সাড়ার সময় (`dig`-এর Query Time), আর সংযোগের সংখ্যা (`ss -s`) মাপুন — সংখ্যা না মেপে অনুমান করবেন না।

> **TROUBLESHOOTING:** ছোট অনুরোধ দ্রুত যায়, কিন্তু বড় ফাইল আপলোড/ডাউনলোড ব্যর্থ হয় বা অত্যন্ত ধীর — এটি MTU সমস্যার একটি চেনা লক্ষণ। Pod-এর ভেতরে বিভিন্ন আকারের প্যাকেট দিয়ে `ping -M do -s 1400` পরীক্ষা করুন; কোন আকারে প্যাকেট ফেলে দেওয়া হচ্ছে, তা দেখলে MTU-র সঠিক মান বেরিয়ে আসে।

৮.১৪ অধ্যায়ের সারাংশ

Kubernetes নেটওয়ার্কিং-এর ভিত্তি: CNI Pod-কে IP দেয়, Service স্থির নাম ও IP দেয়, kube-proxy ট্রাফিক রাউট করে, CoreDNS নাম খুঁজে দেয়, Network Policy ট্রাফিক সীমাবদ্ধ করে, আর Ingress/Gateway API বাইরের ট্রাফিক ভেতরে আনে। Service Mesh এই সবের উপরে mTLS, Traffic Management ও গভীর পর্যবেক্ষণ যোগ করে — তবে তার খরচ ও জটিলতা আছে। নেটওয়ার্ক সমস্যা ডিবাগের চাবি: স্তরে স্তরে পরীক্ষা করা — Pod, Service, DNS, Gateway — কোন স্তরে ভাঙে, তা খুঁজে বের করা।

৮.১৫ জ্ঞান যাচাই

1. Kubernetes নেটওয়ার্ক মডেলের তিনটি নিয়ম কী কী?
2. CNI কী? Flannel, Calico ও Cilium-এর পার্থক্য কী?
3. ClusterIP, NodePort, LoadBalancer ও Headless Service-এর পার্থক্য ব্যাখ্যা করুন।
4. kube-proxy আসলে কী করে? সে নিজে কি ট্রাফিক বহন করে?
5. CoreDNS ব্যর্থ হলে কী কী লক্ষণ দেখা যাবে?
6. Network Policy-তে Default-Deny কেন গুরুত্বপূর্ণ? DNS বাদ পড়লে কী হয়?
7. Ingress ও Gateway API-র পার্থক্য কী? নতুন প্রকল্পে কোনটি সুপারিশ করবেন?
8. HTTPRoute-এ Weight দিয়ে Traffic Splitting কীভাবে কাজ করে?
9. Service Mesh-এর সুবিধা ও খরচ কী কী? কখন Mesh নেওয়া উচিত নয়?
10. একটি অ্যাপ Service নামে পৌঁছাতে পারছে না, কিন্তু Pod IP-তে পারছে — আপনি কোন ক্রমে ডিবাগ করবেন?

৮.১৬ ব্যবহারিক অনুশীলন

1. একটি Namespace-এ ওয়েব, API ও ডেটাবেস Pod বানান। প্রথমে কোনো Network Policy ছাড়া সব Pod একে অপরকে `curl` করতে পারে কি না পরীক্ষা করুন। তারপর Default-Deny নীতি প্রয়োগ করে দেখুন সব বন্ধ হয়ে যায় কি না, তারপর এই অধ্যায়ের নীতিগুলো একে একে প্রয়োগ করে শুধু বৈধ ট্রাফিক খুলুন — প্রতিটি ধাপে পরীক্ষার ফলাফল লিখুন।
2. Gateway API দিয়ে একটি Gateway ও HTTPRoute বানান: `/api` পথ API Service-এ, `/` পথ ওয়েব Service-এ যাবে। তারপর Weight দিয়ে ৯০/১০ Traffic Splitting করুন ও একটি Canary সংস্করণ যোগ করুন। একই কাজ Ingress দিয়ে করলে কী কী Annotation লাগত, তা তুলনা করে লিখুন।
3. একটি DNS ব্যর্থতার মহড়া দিন: CoreDNS Pod-গুলো সাময়িকভাবে Scale-Down করে (পরীক্ষার ক্লাস্টারে) দেখুন অ্যাপের আচরণ কেমন হয়, তারপর আবার Scale-Up করে পুনরুদ্ধার মাপুন। এই অভিজ্ঞতা থেকে DNS-এর জন্য আপনার পর্যবেক্ষণ (Alert) পরিকল্পনা লিখুন।

অধ্যায় ৯ • PART II — Advanced Kubernetes & Cloud-Native Engineering

Chapter 9 — Kubernetes Security and Governance

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **RBAC** দিয়ে "কে কী করতে পারে" নিখুঁতভাবে নিয়ন্ত্রণ করতে; **Pod Security Standards** দিয়ে বিপজ্জনক Pod চালানো বন্ধ করতে; **Admission Control** (OPA Gatekeeper, Kyverno) দিয়ে ক্লাস্টার-ব্যাপী নীতি প্রয়োগ করতে; **Network Segmentation**, **Workload Identity**, **Secrets Management** ও **Image Provenance**-এর সিদ্ধান্ত নিতে; এবং **Audit Log** দিয়ে "কে, কখন, কী করল" — তা খুঁজে বের করতে।

- Role, ClusterRole, RoleBinding ও ClusterRoleBinding লেখা ও পরীক্ষা করা।
- Pod Security Standards-এর তিনটি স্তর (Privileged, Baseline, Restricted) ব্যাখ্যা করা।
- OPA Gatekeeper ও Kyverno-র পার্থক্য ও ব্যবহারের ক্ষেত্র বলা।
- ServiceAccount ও Workload Identity দিয়ে ক্লাউড সম্পদে নিরাপদ প্রবেশাধিকার দেওয়া।
- Secrets-এর সীমা বুঝে External Secrets / ভল্ট ব্যবহারের সিদ্ধান্ত নেওয়া।
- Image Signing ও Admission-এ যাচাইয়ের প্রবাহ আঁকা।
- Audit Log থেকে একটি সন্দেহজনক ঘটনা খুঁজে বের করা।

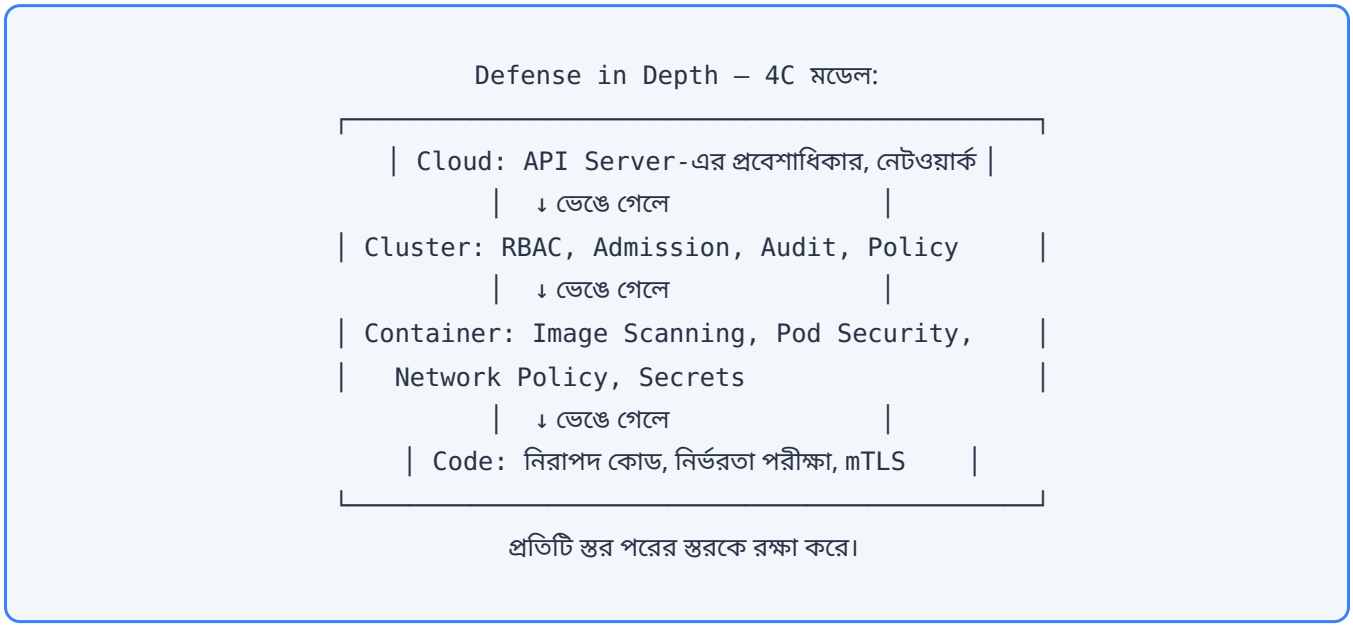
৯.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

অধ্যায় ৬-এর API Server-এর প্রমাণীকরণ ও অনুমোদনের ধাপ, অধ্যায় ৮-এর Network Policy, আর Namespace-এর ধারণা থাকলেই চলবে। মনে রাখুন: Kubernetes-এ নিরাপত্তা একটি একক বৈশিষ্ট্য নয় — এটি স্তরে স্তরে সাজানো (**Defense in Depth**): API Server-এর প্রবেশাধিকার, RBAC, Admission, Pod Security, Network Policy, Image যাচাই, Secrets, Audit — প্রতিটি স্তর একটি ভিন্ন আক্রমণের পথ বন্ধ করে। একটি স্তর ভেঙে গেলেও অন্য স্তরগুলো দাঁড়িয়ে থাকে। এই অধ্যায়ে প্রতিটি স্তর দেখব।

> **CONCEPT: Least Privilege** (সর্বনিম্ন অনুমতি) — প্রতিটি মানুষ, সার্ভিস অ্যাকাউন্ট ও Pod-কে শুধু ততটুকু অনুমতি দেওয়া, যতটুকু তার কাজের জন্য প্রয়োজন, আর একটুও বেশি নয়। `cluster-admin` দেওয়া সহজ, কিন্তু তা পুরো ক্লাস্টারের চাবি দিয়ে দেওয়ার সমান। একটি আপসিত (Compromised) Pod-এর cluster-admin থাকলে আক্রমণকারী পুরো ক্লাস্টার দখল করতে পারে।

৯.৩ প্রথম নীতি: 4C মডেল — নিরাপত্তার স্তর

ক্লাউড-নেটিভ নিরাপত্তার একটি পরিচিত কাঠামো হলো **4C**: **Cloud** (ক্লাউড প্রদানকারী ও নেটওয়ার্কের নিরাপত্তা), **Cluster** (API Server, etcd, Control Plane-এর নিরাপত্তা), **Container** (ছবি, রানটাইম, Pod-এর নিরাপত্তা), **Code** (অ্যাপ কোডের নিরাপত্তা)। প্রতিটি স্তরে ভিন্ন ভিন্ন নিয়ন্ত্রণ লাগে। যেমন Cloud স্তরে API Server-এর পাবলিক প্রবেশাধিকার বন্ধ করা; Cluster স্তরে RBAC ও Admission; Container স্তরে Image Scanning ও Pod Security; Code স্তরে নির্ভরতা (Dependency) পরীক্ষা। এই অধ্যায়ের মূল বিষয় Cluster ও Container স্তর; তবে মনে রাখুন, একটি স্তরে যত ভালো কাজই করুন, অন্য স্তর খোলা থাকলে আক্রমণের পথ থেকেই যায়।



৯.৪ RBAC — কে কী করতে পারে

RBAC (Role-Based Access Control) হলো Kubernetes-এর অনুমোদন ব্যবস্থা। এর চারটি উপাদান: **Role** (একটি Namespace-এর ভেতরে কী অনুমতি — যেমন Pod পড়া, Deployment বানানো), **ClusterRole** (ক্লাস্টার-ব্যাপী অনুমতি — যেমন নোড দেখা, বা সব Namespace-এ Pod পড়া), **RoleBinding** (Role-কে একজন ব্যবহারকারী/গোষ্ঠী/ServiceAccount-এর সাথে বাঁধা, একটি Namespace-এ), **ClusterRoleBinding** (ClusterRole-কে বাঁধা, ক্লাস্টার-ব্যাপী)। অনুমতি সবসময় যোগমূলক (Additive) — একটি RoleBinding দিলে অনুমতি বাড়ে, কখনো কমে না; "এই কাজটি করতে পারবে না" বলার কোনো উপায় RBAC-এ নেই। তাই অনুমতি দেওয়ার সময় শুধু প্রয়োজনীয় **Verb** (get, list, watch, create, update, patch, delete) ও **Resource** (pods, deployments, secrets) দিন।

একটি সাধারণ ও বিপজ্জনক ভুল: ServiceAccount-কে `cluster-admin` দেওয়া। অনেক Helm Chart ও টুল ডিফল্টভাবে cluster-admin চায় — তা দেবেন না। বরং টুলটির আসলে কী কী অনুমতি লাগে, তা দেখে একটি সীমিত ClusterRole বানান। আরেকটি নিয়ম: **Secrets পড়ার অনুমতি** অত্যন্ত সংবেদনশীল — যে Secrets পড়তে পারে, সে প্রায়শই অন্য অনুমতিও পেয়ে যায় (যেমন ServiceAccount-এর টোকেন পড়ে সেই অ্যাকাউন্টের পরিচয় নেওয়া)। তাই Secrets-এর অনুমতি শুধু তাদেরই দিন, যাদের আসলেই প্রয়োজন।

```
# Role: Production Namespace-এ Pod ও Deployment পড়া ও দেখা
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: app-reader
  namespace: production
rules:
  - apiGroups: ["", "apps"]
    resources: ["pods", "deployments", "replicasets"]
    verbs: ["get", "list", "watch"]
---
# RoleBinding: ডেভেলপার দলকে Role-টি দেওয়া
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-readers
  namespace: production
subjects:
  - kind: Group
    name: developers
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: app-reader
  apiGroup: rbac.authorization.k8s.io
---
# ServiceAccount: অ্যাপের পরিচয়
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api-sa
  namespace: production
---
# ClusterRole: শুধু পড়ার অনুমতি, ক্লাস্টার-ব্যাপী (সতর্কতার সাথে)
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-reader
rules:
  - apiGroups: ["", "apps", "networking.k8s.io"]
    resources: ["pods", "nodes", "deployments", "services", "ingresses"]
    verbs: ["get", "list", "watch"]

# অনুমতি পরীক্ষা করা — অনুমান নয়, যাচাই
kubectl auth can-i get pods -n production
kubectl auth can-i delete deployments -n production
kubectl auth can-i get secrets -n production --as=system:serviceaccount:production:api-
kubectl auth can-i create pods -n production --as=dev-user@example.com

# কার কী অনুমতি আছে, দেখা
kubectl get rolebindings,clusterrolebindings -A
kubectl describe clusterrolebinding cluster-admin
```

> **WARNING:** `kubectl auth can-i --list` দিয়ে দেখুন একটি অ্যাকাউন্ট আসলে কী কী করতে পারে — অনেক সময় একটি RoleBinding-এর মাধ্যমে অপ্রত্যাশিত অনুমতি চলে আসে। বিশেষ করে `system:masters` গোষ্ঠী, `cluster-admin` ClusterRoleBinding, ও Secrets পড়ার অনুমতি নিয়মিত পরীক্ষা করুন।

৯.৫ Pod Security Standards — বিপজ্জনক Pod বন্ধ করা

Pod Security Standards (PSS) হলো Kubernetes-এর অন্তর্নির্মিত নিরাপত্তা-মান, যা **Pod Security Admission (PSA)** দিয়ে কার্যকর হয়। তিনটি স্তর: **Privileged** (কোনো সীমা নেই — শুধু সিস্টেমের Pod-এর জন্য), **Baseline** (সাধারণ বিপদ বন্ধ — যেমন হোস্টের নেটওয়ার্ক/প্রসেস Namespace ব্যবহার, বিশেষ ক্ষমতা — Privileged কন্টেইনার), **Restricted** (সবচেয়ে কঠোর — Root নয় এমন ব্যবহারকারী, শুধু অনুমোদিত ক্ষমতা, Read-Only Root Filesystem)। প্রতিটি Namespace-এ Label দিয়ে স্তর ঠিক করা হয়: `pod-security.kubernetes.io/enforce: restricted`। **Enforce** মোডে নীতি-বিরোধী Pod তৈরি হতেই পারে না; **Audit** মোডে তৈরি হয়, কিন্তু লগে সতর্কতা লেখা হয়; **Warn** মোডে ব্যবহারকারীকে সতর্ক করা হয়। নিরাপদ স্থানান্তর: প্রথমে Audit/Warn দিয়ে দেখুন কোন Pod নীতি ভাঙছে, সেগুলো ঠিক করুন, তারপর Enforce করুন।

```
# Namespace-এ Pod Security Standards চালু করা
kubectl label namespace production \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted

# Restricted মান মেনে চলা Pod উদাহরণ
apiVersion: v1
kind: Pod
metadata:
  name: secure-web
  namespace: production
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: web
      image: registry.technova.example/web:1.4.0
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        runAsNonRoot: true
```

```
      capabilities:
        drop: ["ALL"]
    resources:
      requests:
        cpu: "250m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
```

বিষয়	Baseline	Restricted
Privileged কন্টেইনার	নিষিদ্ধ	নিষিদ্ধ
হোস্টের নেটওয়ার্ক/PID/IPC Namespace	নিষিদ্ধ	নিষিদ্ধ
Root ব্যবহারকারী	অনুমোদিত	নিষিদ্ধ (runAsNonRoot)
Privilege Escalation	অনুমোদিত	নিষিদ্ধ
Read-Only Root Filesystem	ঐচ্ছিক	আবশ্যিক
Capabilities	সীমিত	সব বাদ (drop ALL), শুধু প্রয়োজনে যোগ

> **CONCEPT: seccomp** (Secure Computing Mode) ও **AppArmor/SELinux** হলো কার্নেল-স্তরের নিরাপত্তা — কন্টেইনার কোন System Call করতে পারবে, তা সীমাবদ্ধ করে। PSS-এর Restricted স্তরে `RuntimeDefault` seccomp প্রোফাইল ব্যবহার করা হয়। এই প্রোফাইলগুলো বোঝা জটিল মনে হতে পারে, তবে নীতি সহজ: ডিফল্ট প্রোফাইল ব্যবহার করুন, আর শুধু প্রয়োজনে শিথিল করুন — কখনোই "সবকিছু খোলা" রাখবেন না।

৯.৬ Admission Control — OPA Gatekeeper ও Kyverno

RBAC বলে "কে" কী করতে পারে; **Admission Control** বলে "কী" অনুমোদিত — যেমন "সব Pod-এ Resource Limit থাকতে হবে", "`latest` ট্যাগের ছবি চলবে না", "Production Namespace-এ শুধু অনুমোদিত Registry-র ছবি চলবে"। Kubernetes-এর অন্তর্নির্মিত Admission (যেমন Pod Security Admission) ছাড়াও দুটি জনপ্রিয় Policy Engine আছে: **OPA Gatekeeper** ও **Kyverno**। Gatekeeper **Rego** ভাষায় নীতি লেখে — শক্তিশালী, কিন্তু শিখতে সময় লাগে; এটি **ConstraintTemplate** (নীতির ছাঁচ) ও **Constraint** (নীতির প্রয়োগ) দিয়ে কাজ করে, আর ক্লাস্টারের বর্তমান অবস্থা পরীক্ষা (Audit) করতে পারে। Kyverno নীতি YAML-এ লেখে — Kubernetes-এর সাথে পরিচিত যেকেউ পড়তে পারে; এটি শুধু বাধা দেয় না, **Mutate** (ক্ষেত্র যোগ করা — যেমন সব Pod-এ Label যোগ করা) ও **Generate** (একটি অবজেক্ট থেকে আরেকটি বানানো — যেমন Namespace তৈরি হলে Default NetworkPolicy বানানো) করতে পারে। কোনটি বেছে নেবেন? দলে Rego-র অভিজ্ঞতা থাকলে ও জটিল যুক্তি লাগলে Gatekeeper; YAML-এ দ্রুত শুরু করতে ও Mutate/Generate চাইলে Kyverno। অনেক প্রতিষ্ঠান শুধু একটি বেছে নেয় — দুটি একসাথে চালানো মানে দুটি নীতি-ব্যবস্থার রক্ষণাবেক্ষণ।

```
# Kyverno নীতি: সব Pod-এ Resource Limit বাধ্যতামূলক
apiVersion: kyverno.io/v1
```

```
kind: ClusterPolicy
metadata:
  name: require-resource-limits
spec:
  validationFailureAction: Enforce
  background: true
  rules:
    - name: check-limits
      match:
        any:
          - resources:
              kinds:
                - Pod
      validate:
        message: "প্রতিটি কন্টেইনারে CPU ও মেমোরি Limit থাকতে হবে।"
        pattern:
          spec:
            containers:
              - resources:
                  limits:
                    cpu: "?*"
                    memory: "?*"
---
# Kyverno নীতি: শুধু অনুমোদিত Registry-র ছবি
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: allowed-registries
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-registry
      match:
        any:
          - resources:
              kinds:
                - Pod
      validate:
        message: "শুধু registry.technova.example-এর ছবি অনুমোদিত।"
        pattern:
```

```
          spec:
            containers:
              - image: "registry.technova.example/*"
---
# OPA Gatekeeper ধারণা (ConstraintTemplate + Constraint)
# apiVersion: templates.gatekeeper.sh/v1
# kind: ConstraintTemplate
# metadata:
#   name: k8srequiredlabels
# spec:
#   crd:
#     spec:
#       names:
#         kind: K8sRequiredLabels
#   targets:
#     - target: admission.k8s.gatekeeper.sh
#       rego: |
#         package k8srequiredlabels
#         violation[{"msg": msg}] {
#           provided := {label | input.review.object.metadata.labels[label]}
#           required := {label | label := input.parameters.labels[_]}
#           missing := required - provided
#           count(missing) > 0
#           msg := sprintf("অনুপস্থিত Label: %v", [missing])
#    }
```

> **WARNING:** Admission নীতি প্রথমে **Audit** মোডে (Gatekeeper-এ `enforcementAction: dryrun`, Kyverno-তে `validationFailureAction: Audit`) চালান। সরাসরি Enforce করলে বিদ্যমান অ্যাপের নতুন Pod তৈরি বন্ধ হয়ে যেতে পারে — আর তা প্রোডাকশনে বিপর্যয় ডেকে আনতে পারে। কয়েক দিন Audit-এর ফলাফল দেখুন, ভাঙা Pod-গুলো ঠিক করুন, তারপর Enforce করুন।

৯.৭ Network Segmentation ও Workload Identity

Network Segmentation মানে নেটওয়ার্ককে ছোট ছোট অংশে ভাগ করা, যাতে একটি অংশ আপসিত হলে আক্রমণকারী অন্য অংশে যেতে না পারে। Kubernetes-এ এর যন্ত্র: Network Policy (অধ্যায় ৮), Namespace দিয়ে ভাগ করা, আর প্রয়োজনে Service Mesh-এর mTLS। একটি ভালো অভ্যাস: প্রতিটি অ্যাপের জন্য আলাদা Namespace, প্রতিটি Namespace-এ Default-Deny Network Policy, আর শুধু প্রয়োজনীয় সংযোগ খোলা।

Workload Identity মানে Pod-এর পরিচয় — Pod যখন ক্লাউডের কোনো সম্পদ (যেমন Object Storage, ডেটাবেস, Secret Manager) ব্যবহার করতে চায়, তখন সে কে, তা কীভাবে প্রমাণ করবে? পুরোনো পদ্ধতি: দীর্ঘস্থায়ী চাবি (Access Key) Secret-এ রেখে Pod-এ দেওয়া — চাবি ফাঁস হলে বিপদ, আর চাবি ঘোরানো (Rotation) কঠিন। আধুনিক পদ্ধতি:

Workload Identity Federation — Pod-এর ServiceAccount টোকেন (স্বল্পস্থায়ী, স্বয়ংক্রিয়ভাবে নবায়ন হয়) দিয়ে ক্লাউড প্রদানকারীর কাছে পরিচয় প্রমাণ করা, আর ক্লাউড প্রদানকারী সেই পরিচয়ের ভিত্তিতে স্বল্পস্থায়ী শংসাপত্র (Credential) দেয়। এতে কোনো দীর্ঘস্থায়ী চাবি Pod-এ থাকে না। প্রতিটি ক্লাউডের নিজস্ব নাম আছে (যেমন AWS-এ IRSA, GCP-তে Workload Identity Federation, Azure-এ Workload Identity), কিন্তু নীতিটি একই: Pod-এর পরিচয়ই তার শংসাপত্র, আর অনুমতি সর্বনিম্ন রাখুন।

৯.৮ Secrets — কী রাখা যায়, কী রাখা যায় না

Kubernetes **Secret** হলো সংবেদনশীল তথ্য (পাসওয়ার্ড, টোকেন, সার্টিফিকেট) রাখার অন্তর্নির্মিত উপায় — কিন্তু এর সীমা বুঝতে হবে। ডিফল্টভাবে Secret etcd-তে **Base64-এনকোড** করা থাকে — এটি এনক্রিপশন নয়, শুধু এনকোডিং; যে etcd পড়তে পারে, সে Secret পড়তে পারে। তাই প্রোডাকশনে **Encryption at Rest** চালু করুন (API Server-এর EncryptionConfiguration দিয়ে etcd-র Secret এনক্রিপ্ট করা), আর আরও ভালো: Secret Kubernetes-এ না রেখে একটি বাইরের **Secret Manager** (যেমন HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager) বা **External Secrets Operator** ব্যবহার করুন — Secret Manager-ই সত্যের উৎস, আর Operator শুধু প্রয়োজনে Kubernetes Secret-এ কপি করে (বা Pod সরাসরি Secret Manager থেকে পড়ে)। নিয়ম: Secret কখনোই Git-এ লিখবেন না; Secret-এর মান লগে ছাপাবেন না; Secret পড়ার RBAC অনুমতি সীমিত রাখুন; আর নিয়মিত Secret ঘোরান (Rotation)।

```
# Secret — শুধু উদাহরণ; মানগুলো প্লেসহোল্ডার
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: production
type: Opaque
stringData:
  username: technova_app
  password: CHANGE-ME-STRONG-PASSWORD
---
# Pod-এ Secret পরিবেশ চলক হিসেবে (সতর্কতার সাথে)
apiVersion: v1
kind: Pod
metadata:
  name: api
  namespace: production
spec:
  serviceAccountName: api-sa
  containers:
    - name: api
      image: registry.technova.example/api:1.4.0
      env:
```

```
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: password
---
# External Secrets Operator ধারণা (বাইরের Secret Manager থেকে)
# apiVersion: external-secrets.io/v1beta1
# kind: ExternalSecret
# metadata:
#   name: db-credentials
#   namespace: production
# spec:
#   refreshInterval: 1h
#   secretStoreRef:
#     name: technova-vault
#     kind: ClusterSecretStore
#   target:
#     name: db-credentials
#   data:
#     - secretKey: password
#       remoteRef:
#         key: production/db
#         property: password
```

৯.৯ Image Provenance — ছবিটি আসলেই যেখান থেকে এসেছে, সেখান থেকেই কি এসেছে?

Image Provenance মানে ছবির উৎস ও অখণ্ডতা যাচাই করা: এই ছবিটি কি আমাদের CI পাইপলাইনেই বানানো হয়েছে? কেউ কি মাঝপথে বদলে দিয়েছে? ছবিতে কি পরিচিত দুর্বলতা (Vulnerability) আছে? এর যন্ত্রগুলো: **Image Scanning** (Trivy, Grype দিয়ে ছবির দুর্বলতা খোঁজা — CI-তে ও Registry-তে), **Image Signing** (Sigstore/Cosign দিয়ে ছবিতে ডিজিটাল স্বাক্ষর করা — "এই ছবিটি আমাদের পাইপলাইন বানিয়েছে"), আর **Admission-এ যাচাই** (Kyverno/Gatekeeper দিয়ে স্বাক্ষরহীন ছবির Pod তৈরি বন্ধ করা)। একটি শক্তিশালী নিয়ম: Production Namespace-এ শুধু স্বাক্ষরিত ও স্ক্যান করা ছবি চলবে; `latest` ট্যাগ নিষিদ্ধ; আর ছবির ভিত্তি (Base Image) নিয়মিত আপডেট করা হবে। মনে রাখুন: ছবি স্ক্যানিং দুর্বলতা খুঁজে দেয়, কিন্তু তা ঠিক করে না — স্ক্যানের ফলাফল দেখে ভিত্তি ছবি আপডেট ও নির্ভরতা হালনাগাদ করার প্রক্রিয়া থাকতে হবে।

৯.১০ Audit — কে, কখন, কী করল

Kubernetes Audit Log হলো API Server-এর কাছে আসা প্রতিটি অনুরোধের রেকর্ড: কে (ব্যবহারকারী/ServiceAccount), কখন, কোন Resource-এ, কী কাজ (Verb) করল, সফল হলো কি না। Audit Log ছাড়া নিরাপত্তা-

ঘটনার তদন্ত অসম্ভব — “কে এই Secret-টি পড়ল?” প্রশ্নের উত্তর শুধু Audit Log-ই দিতে পারে। Audit নীতি (Audit Policy) দিয়ে ঠিক করুন কী লগ হবে: সবকিছু লগ করলে লগের পরিমাণ বিশাল হয় (আর খরচও); শুধু গুরুত্বপূর্ণ ঘটনা (যেমন Secrets পড়া, RoleBinding বানানো, Pod মুছে ফেলা) লগ করলে তদন্তের জন্য যথেষ্ট হয়। Audit Log-কে কেন্দ্রীয় লগ ব্যবস্থায় (SIEM) পাঠান, আর সেখানে সতর্কতা (Alert) বসান — যেমন “Production-এ cluster-admin ব্যবহার করে কেউ Pod মুছেছে”।

```
# Audit নীতির উদাহরণ (API Server-এ Audit Policy ফাইল হিসেবে)
# apiVersion: audit.k8s.io/v1
# kind: Policy
# rules:
#   # Secrets পড়া — বিস্তারিত লগ
#   - level: Metadata
#     resources:
#       - group: ""
#         resources: ["secrets"]
#   # RBAC পরিবর্তন — অনুরোধসহ লগ
#   - level: RequestResponse
#     resources:
#       - group: "rbac.authorization.k8s.io"
#         resources: ["roles", "rolebindings", "clusterroles", "clusterrolebindings"]
#  # বাকি সব — শুধু Metadata
#   - level: Metadata
#     omitStages:
#       - RequestReceived

# সন্দেহজনক ঘটনা খোঁজা (Audit Log-এ)
# kubectl -n kube-system logs -l component=kube-apiserver | grep -i "secrets"
# কেন্দ্রীয় লগে: "verb=delete resource=pods namespace=production" খুঁজুন
```

> **TROUBLESHOOTING:** “আমি কেন এই কাজটি করতে পারছি না?” — প্রথমে `kubectl auth can-i` দিয়ে অনুমতি যাচাই করুন; তারপর API Server-এর Audit Log-এ দেখুন অনুরোধটি কোন ধাপে বাতিল হয়েছে (RBAC নাকি Admission)। “Pod তৈরি হচ্ছে না” — Pod Security Admission বা Kyverno/Gatekeeper-এর নীতি বাধা দিচ্ছে কি না, তা `kubectl describe pod`-এর Events ও Policy-র লগে দেখুন।

৯.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র ফাঁস হওয়া টোকেন

TechNova-র একজন ডেভেলপার ভুল করে একটি ServiceAccount টোকেন একটি পাবলিক চ্যাটে পোস্ট করলেন। কী ঘটল? টোকেনটি শুধু Production Namespace-এ Pod পড়ার অনুমতি পেয়েছিল — তাই আক্রমণকারী Pod-গুলো দেখতে পেল, কিন্তু Secret পড়তে পারল না, Pod মুছতে পারল না, অন্য Namespace-এ যেতে পারল না। Audit Log-এ দেখা গেল অপরিচিত IP থেকে Pod তালিকা পড়ার চেষ্টা হয়েছে। দলটি টোকেনটি বাতিল করল (ServiceAccount-এর Secret মুছে),

RBAC পরীক্ষা করল, আর Audit-এ সতর্কতা বসাল। শিক্ষা: Least Privilege-এর কারণে ক্ষতি সীমিত ছিল। টোকেনটি cluster-admin হলে পুরো ক্লাস্টার দখল হয়ে যেত।

৯.১২ সাধারণ ভুল

- **সবাইকে cluster-admin দেওয়া:** সুবিধার জন্য পুরো ক্লাস্টারের চাবি দেওয়া — সবচেয়ে সাধারণ ও বিপজ্জনক ভুল।
- **Secrets-কে এনক্রিপশন ভাবা:** Base64 এনকোডিং এনক্রিপশন নয়; Encryption at Rest ও Secret Manager ব্যবহার করুন।
- **Secret Git-এ লেখা:** Git ইতিহাস থেকে Secret মোছা প্রায় অসম্ভব — একবার ফাঁস হলে Secret বদলাতে হবে।
- **Admission নীতি সরাসরি Enforce করা:** Audit ছাড়া Enforce করলে বৈধ Pod-ও বন্ধ হয়ে যেতে পারে।
- **Root হিসেবে কন্টেইনার চালানো:** ডিফল্টভাবে অনেক ছবি Root-এ চলে; runAsNonRoot ও Restricted মান ব্যবহার করুন।
- **Image-এ latest ট্যাগ:** কোন সংস্করণ চলছে, তা জানা যায় না; স্বাক্ষর ও স্ক্যান যাচাইও অনিশ্চিত হয়।
- **Audit Log না রাখা:** ঘটনার পর “কে করল?” প্রশ্নের উত্তর না থাকলে তদন্ত অসম্ভব।
- **Network Policy না থাকা:** সব Pod সব Pod-এর সাথে কথা বলতে পারলে একটি আপসিত Pod পুরো ক্লাস্টারে ঘুরতে পারে।

৯.১৩ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা একটি চলমান প্রক্রিয়া, এককালীন প্রকল্প নয়। নিয়মিত করুন: RBAC পরীক্ষা (কে cluster-admin, কে Secret পড়তে পারে), ছবি স্ক্যানিং ও ভিত্তি ছবি আপডেট, Audit Log পর্যালোচনা, নীতি-পরীক্ষা (Policy-র Audit ফলাফল), আর সার্টিফিকেটের মেয়াদ পরীক্ষা। খরচ: Policy Engine (Gatekeeper/Kyverno), Image Scanning, Secret Manager, SIEM — প্রতিটির পরিচালন ও লাইসেন্স খরচ আছে; তবে নিরাপত্তা-ঘটনার খরচের তুলনায় তা সামান্য। ছোট দলের জন্য শুরু করুন RBAC, Pod Security Standards ও Network Policy দিয়ে — এগুলো অন্তর্নির্মিত, অতিরিক্ত খরচ ছাড়াই পাওয়া যায় — তারপর প্রয়োজনে Gatekeeper/Kyverno ও Secret Manager যোগ করুন।

৯.১৩.১ সরবরাহ-শৃঙ্খল নিরাপত্তা: SBOM ও স্বাক্ষরিত প্রমাণ

আধুনিক নিরাপত্তার একটি গুরুত্বপূর্ণ অংশ হলো **Software Supply Chain** — আপনার ছবিটি বানাতে কোন ভিত্তি ছবি, কোন লাইব্রেরি, কোন টুল ব্যবহার হয়েছে, তার সম্পূর্ণ তালিকা। এই তালিকাকে বলে **SBOM (Software Bill of Materials)** — অনেকটা খাবারের উপাদান-তালিকার মতো। SBOM থাকলে একটি নতুন দুর্বলতা (যেমন একটি লাইব্রেরিতে) প্রকাশ পেলে আপনি সঙ্গে সঙ্গে জানতে পারেন আপনার কোন ছবিতে সেই লাইব্রেরি আছে। CI পাইপলাইনে SBOM তৈরি করুন (যেমন Syft দিয়ে), ছবির সাথে সংরক্ষণ করুন, আর **Provenance** (যেমন SLSA কাঠামো) দিয়ে প্রমাণ করুন ছবিটি কোন পাইপলাইনে, কোন কোড থেকে বানানো হয়েছে। Sigstore/Cosign দিয়ে ছবি ও SBOM-এ স্বাক্ষর করলে Admission-এ যাচাই করা যায়: “এই ছবির স্বাক্ষর আমাদের CI-র কি না, SBOM আছে কি না?” — না থাকলে Pod তৈরি হবে না। এই ব্যবস্থাটি বসাতে সময় লাগে, তবে একটি আপসিত ছবি Production-এ ঢোকার ঝুঁকি এর চেয়ে অনেক বেশি ব্যয়বহুল। শুরু করুন ছোট করে: প্রথমে CI-তে Image Scanning ও SBOM তৈরি, তারপর স্বাক্ষর, সবশেষে Admission-এ বাধ্যতামূলক যাচাই।

> **CONCEPT: SLSA (Supply-chain Levels for Software Artifacts)** হলো সরবরাহ-শৃঙ্খলের পরিপক্কতার একটি কাঠামো — কোন স্তরে আপনার বিল্ড প্রক্রিয়া কতটা যাচাইযোগ্য ও কারসাজি-প্রতিরোধী, তা মাপে। ছোট দলের জন্য SLSA-র সর্বোচ্চ স্তর লক্ষ্য নয়; লক্ষ্য হলো প্রতিটি ধাপে একটু করে যাচাই যোগ করা — সংস্করণ-নিয়ন্ত্রিত কোড, স্বাক্ষরিত ছবি, স্ক্যান করা নির্ভরতা।

৯.১৩.২ নিয়মিত নিরাপত্তা পর্যালোচনার ছন্দ

নিরাপত্তা একবার বসিয়ে ভুলে যাওয়ার জিনিস নয় — এটি একটি ছন্দ। প্রতি মাসে অন্তত একবার করুন: `cluster-admin` ও Secrets পড়ার অনুমতি কার কার আছে, তা পর্যালোচনা; ছবি স্ক্যানিং-এর ফলাফলে নতুন গুরুতর দুর্বলতা আছে কি না, তা দেখা; Audit Log-এ অস্বাভাবিক ঘটনা (যেমন রাত ৩টায় Production-এ Pod মুছে ফেলা) খোঁজা; আর সার্টিফিকেটের মেয়াদ পরীক্ষা করা। প্রতি তিন মাসে একবার করুন: নীতি-পরীক্ষার (Policy Audit) ফলাফল দেখে কোন Namespace-এ নীতি ভাঙা হচ্ছে, তা ঠিক করা; আর একটি ছোট **Tabletop Exercise** — "একটি Pod আপসিত হলে আমরা কী করব?" — দল বসে ধাপগুলো মুখে মুখে চালিয়ে দেখা। এই ছন্দটি লিখিতভাবে রাখুন — কে, কখন, কী পরীক্ষা করবে — যাতে নিরাপত্তা একজনের স্মৃতির উপর নির্ভর না করে।

৯.১৪ অধ্যায়ের সারাংশ

Kubernetes নিরাপত্তা হলো স্তরে স্তরে সাজানো প্রতিরক্ষা: Cloud, Cluster, Container ও Code — প্রতিটি স্তরে ভিন্ন নিয়ন্ত্রণ। RBAC "কে" নিয়ন্ত্রণ করে, Admission (Pod Security Standards, Gatekeeper, Kyverno) "কী" নিয়ন্ত্রণ করে, Network Policy "কার সাথে" নিয়ন্ত্রণ করে, আর Image Signing, Secrets Management ও Audit Log বাকি ফাঁকগুলো বন্ধ করে। Least Privilege-ই মূল নীতি — কম অনুমতি, কম ক্ষতি। নিরাপত্তা কখনো "শেষ" হয় না; এটি নিয়মিত পরীক্ষা, আপডেট ও মহড়ার একটি অভ্যাস।

৯.১৫ জ্ঞান যাচাই

1. Role, ClusterRole, RoleBinding ও ClusterRoleBinding-এর পার্থক্য ব্যাখ্যা করুন।
2. RBAC অনুমতি কেন যোগমূলক (Additive)? "এই কাজটি করতে পারবে না" বলা যায় না কেন?
3. Pod Security Standards-এর তিনটি স্তর কী কী? কোন Namespace-এ কোন স্তর ব্যবহার করবেন?
4. Enforce, Audit ও Warn মোডের পার্থক্য কী? নতুন নীতি চালুর সময় কোন ক্রম অনুসরণ করবেন?
5. OPA Gatekeeper ও Kyverno-র পার্থক্য কী? কোনটি কখন বেছে নেবেন?
6. Network Segmentation বলতে কী বোঝায়? Kubernetes-এ এর যন্ত্রগুলো কী কী?
7. Workload Identity Federation কেন দীর্ঘস্থায়ী চাবির চেয়ে ভালো?
8. Kubernetes Secret-এর সীমা কী? Encryption at Rest ও External Secrets Operator কী সমস্যা সমাধান করে?
9. Image Provenance-এর তিনটি উপাদান (Scanning, Signing, Admission যাচাই) কীভাবে একসাথে কাজ করে?
10. Audit Log-এ কী কী থাকে? কোন ঘটনাগুলোতে সতর্কতা (Alert) বসানো উচিত?

৯.১৬ ব্যবহারিক অনুশীলন

1. একটি Namespace-এ এই অধ্যায়ের Role ও RoleBinding প্রয়োগ করুন। একটি পরীক্ষামূলক ব্যবহারকারী/ServiceAccount দিয়ে `kubectl auth can-i` চালিয়ে যাচাই করুন: Pod পড়া যায়, Secret পড়া যায় না,

Deployment মুছা যায় না। তারপর অনুমতি বাড়িয়ে দেখুন কী বদলায় — আর কেন Secrets পড়ার অনুমতি বিপজ্জনক, তা লিখুন।

2. একটি Namespace-এ Pod Security Standards-এর Restricted স্তর প্রথমে Audit মোডে, তারপর Enforce মোডে চালু করুন। একটি Root-এ চলা Pod ও একটি Restricted-মানা Pod তৈরি করে দেখুন কোনটি Enforce-এ বাধা পায়, আর ত্রুটি বার্তাটি কী বলে।
3. Kyverno দিয়ে একটি নীতি লিখুন: সব Pod-এ `team` Label থাকতে হবে, আর ছবি শুধু আপনার Registry থেকে আসতে হবে। প্রথমে Audit মোডে চালিয়ে দেখুন কোন Pod নীতি ভাঙছে, তারপর Enforce করুন। নীতিটি ভাঙার চেষ্টা করে দেখুন ত্রুটি বার্তাটি কেমন হয়।
4. একটি Audit তদন্তের মহড়া দিন: একটি পরীক্ষামূলক ServiceAccount দিয়ে একটি Secret পড়ার চেষ্টা করুন (অনুমতি না থাকলে ব্যর্থ হবে), তারপর Audit Log-এ সেই চেষ্টাটি খুঁজে বের করুন — কে, কখন, কোন Resource, সফল না ব্যর্থ — লিখুন।

অধ্যায় ১০ • PART II — Advanced Kubernetes & Cloud-Native Engineering

Chapter 10 — Kubernetes Storage and Stateful Workloads

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **CSI (Container Storage Interface)**, **StorageClass**, **PersistentVolume (PV)** ও **PersistentVolumeClaim (PVC)**-এর সম্পর্ক ব্যাখ্যা করতে; **StatefulSet** দিয়ে ডেটাবেসের মতো Stateful কাজের চাপ চালাতে; স্টোরেজের কর্মক্ষমতা (IOPS, Throughput, Latency) ও খরচের সিদ্ধান্ত নিতে; **Snapshot, Backup ও Recovery**-এর পরিকল্পনা করতে; এবং ডেটা সামঞ্জস্য (Consistency), Failover ও পুনরুদ্ধারের মহড়া দিতে।

- Ephemeral ও Persistent স্টোরেজের পার্থক্য ব্যাখ্যা করা।
- Dynamic Provisioning-এর প্রবাহ (PVC → StorageClass → CSI → PV) আঁকা।
- StatefulSet-এর স্থির পরিচয় ও ক্রমানুসারে চালুর নীতি বোঝা।
- ডেটাবেস কাজের চাপে স্টোরেজের ধরন বেছে নেওয়া।
- VolumeSnapshot ও ব্যাকআপ দিয়ে পুনরুদ্ধার পরীক্ষা করা।
- ডেটা সামঞ্জস্য, Replication ও Failover-এর সিদ্ধান্ত নেওয়া।

১০.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

Pod ক্ষণস্থায়ী (Ephemeral) — Pod মারা গেলে তার ভেতরের ফাইলও হারিয়ে যায় — এই ধারণা, Volume-এর মৌলিক ধারণা (emptyDir), আর Deployment-এর ধারণা থাকলেই চলবে। মনে রাখুন: Stateless অ্যাপের (যেমন ওয়েব সার্ভার) কোনো স্থায়ী ডেটা থাকে না — যেকোনো Pod যেকোনো অনুরোধ সামলাতে পারে। Stateful অ্যাপের (যেমন ডেটাবেস) ডেটা তার পরিচয়ের অংশ — ডেটা হারালে সেবা হারায়। এই পার্থক্যই পুরো অধ্যায়ের কেন্দ্র।

> **CONCEPT:** Kubernetes-এ স্টোরেজের মূল ধারণা **ভাগ করা দায়িত্ব**: ব্যবহারকারী (ডেভেলপার) বলেন "আমার এতটুকু স্টোরেজ লাগবে, এই ধরনের" — এটি **PersistentVolumeClaim (PVC)**। ক্লাস্টার প্রশাসক বলেন "এই ধরনের স্টোরেজ কীভাবে বানানো হবে" — এটি **StorageClass**। আর স্টোরেজ প্রদানকারী (CSI Driver) আসলে ডিস্ক বানায় ও নোডে লাগায়। এই ভাগাভাগির কারণে ডেভেলপারকে জানতে হয় না ডিস্ক কোথা থেকে আসে, আর প্রশাসককে জানতে হয় না প্রতিটি অ্যাপের প্রয়োজন।

১০.৩ প্রথম নীতি: Stateless বনাম Stateful

একটি অ্যাপকে জিজ্ঞেস করুন: "এই Pod-টি মারা গেলে কী হারাবে?" উত্তর "কিছুই না" হলে অ্যাপটি Stateless — Deployment দিয়ে চালান, যেকোনো Pod যেকোনো সময় বদলানো যায়। উত্তর "ডেটা হারাবে" হলে অ্যাপটি Stateful — ডেটা Pod-এর বাইরে, একটি Persistent Volume-এ রাখতে হবে, আর Pod-এর পরিচয় স্থির রাখতে হবে। Stateful অ্যাপের আরও বৈশিষ্ট্য: Pod-গুলোর মধ্যে পার্থক্য থাকে (যেমন ডেটাবেসের Primary ও Replica — Primary-তে লেখা হয়, Replica থেকে পড়া হয়); Pod-গুলোকে একটি নির্দিষ্ট ক্রমে চালু করতে হয় (প্রথমে Primary, তারপর Replica); আর Pod-এর নাম ও নেটওয়ার্ক পরিচয় স্থির থাকতে হয় (যাতে Replica জানে Primary-কে কোথায় পাবে)। এই প্রয়োজনগুলো মেটায় **StatefulSet** — Deployment-এর Stateful সংস্করণ।

```
                Stateless (Deployment):
Pod web-1, web-2, web-3 — সব একরকম, যেকোনোটি বদলানো যায়
ডেটা: কোথাও নেই (বা বাইরের ডেটাবেসে)

                Stateful (StatefulSet):
Pod db-0 (Primary) ──► Volume db-data-0 (স্থির)
Pod db-1 (Replica) ──► Volume db-data-1 (স্থির)
Pod db-2 (Replica) ──► Volume db-data-2 (স্থির)
নাম স্থির, ক্রম স্থির, Volume স্থির — Pod বদলালেও
একই নামে নতুন Pod একই Volume পায়।
```

১০.৪ CSI, StorageClass, PV ও PVC — স্টোরেজের স্তর

CSI (Container Storage Interface) হলো একটি আদর্শ API — যেকোনো স্টোরেজ প্রদানকারী (ক্লাউডের Block Storage, NFS, Ceph, স্থানীয় ডিস্ক) একটি CSI Driver লিখে Kubernetes-এর সাথে যুক্ত হতে পারে। CSI Driver দুটি অংশে চলে: **Controller** (ডিস্ক বানানো, মুছে ফেলা, Snapshot নেওয়া — ক্লাস্টার-স্তরের কাজ) ও **Node** (ডিস্ক নোডে লাগানো — Mount — ও খোলা — প্রতিটি নোডে DaemonSet হিসেবে চলে)। **StorageClass** হলো স্টোরেজের "ধরন" — যেমন "দ্রুত SSD", "সস্তা HDD", "অঞ্চল-জুড়ে Replicated" — প্রতিটি StorageClass-এ লেখা থাকে কোন CSI Driver (Provisioner), কোন ধরনের ডিস্ক, আর কী কী বৈশিষ্ট্য (যেমন IOPS)। **PersistentVolume (PV)** হলো আসল ডিস্ক — ক্লাস্টারের একটি সম্পদ। **PersistentVolumeClaim (PVC)** হলো ডিস্কের জন্য অনুরোধ — "আমার ১০ গিগাবাইট, ReadWriteOnce, দ্রুত ধরনের ডিস্ক লাগবে"।

Dynamic Provisioning-এর প্রবাহ: আপনি PVC বানান; **Provisioner** (StorageClass-এর CSI Driver) দেখে PVC-র StorageClass, তারপর আসল ডিস্ক বানায় ও PV তৈরি করে; PVC সেই PV-র সাথে বাঁধা হয় (**Binding**); Pod PVC-কে Volume হিসেবে মাউন্ট করে ব্যবহার করে। Pod মুছে গেলেও PV ও তার ডেটা থেকে যায় (যদি **Reclaim Policy** `Retain` হয়); PVC মুছে গেলে PV-র ভাগ্য Reclaim Policy দিয়ে ঠিক হয়: `Delete` মানে PV-সহ ডিস্ক মুছে যাবে, `Retain` মানে ডিস্ক থেকে যাবে (হাতে মুছতে হবে)। প্রোডাকশনের গুরুত্বপূর্ণ ডেটায় ভুল করে PVC মুছে ডিস্ক হারানো রোধে অনেক দল StorageClass-এ `Retain` বা PVC-তে সুরক্ষা ব্যবহার করে।

```
# StorageClass উদাহরণ (দ্রুত SSD)
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
---
```

```
# StorageClass উদাহরণ (সস্তা, সাধারণ কাজের জন্য)
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Delete
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
---
# PersistentVolumeClaim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data
  namespace: production
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 20Gi
```

Access Mode	অর্থ	উপযুক্ত ব্যবহার
ReadWriteOnce (RWO)	একটি নোড থেকে পড়া ও লেখা যায়	ডেটাবেস, বেশিরভাগ Block Storage
ReadOnlyMany (ROX)	অনেক নোড থেকে শুধু পড়া যায়	স্থির ফাইল, কনফিগ, ছবি
ReadWriteMany (RWX)	অনেক নোড থেকে পড়া ও লেখা যায়	ভাগ করা ফাইল (NFS, EFS), একাধিক Pod-এর লেখা
ReadWriteOncePod (RWOP)	শুধু একটি Pod থেকে পড়া ও লেখা যায়	একটি Pod-এর একক ডিস্ক

> **CONCEPT: volumeBindingMode: WaitForFirstConsumer** মানে PV বানানো হবে তখনই, যখন প্রথম Pod PVC-টি ব্যবহার করতে চাইবে — আর তখন Pod-টি যে নোডে Schedule হবে, সেই নোডের অঞ্চলে (Zone) ডিস্ক বানানো হবে। এটি গুরুত্বপূর্ণ, কারণ Block Storage একটি অঞ্চলের ভেতরে থাকে — Pod এক অঞ্চলে আর ডিস্ক অন্য অঞ্চলে হলে Pod ডিস্ক ব্যবহার করতে পারবে না। ডিফল্ট `Immediate` মোডে PVC বানানোর সাথে সাথে ডিস্ক বানানো হয়, আর তা ভুল অঞ্চলে হতে পারে।

১০.৫ StatefulSet — স্থির পরিচয়, স্থির স্টোরেজ

StatefulSet Deployment-এর মতোই Pod পরিচালনা করে, কিন্তু তিনটি অতিরিক্ত নিশ্চয়তা দেয়: (১) **স্থির নাম** — Pod-গুলোর নাম হয় `db-0`, `db-1`, `db-2` — ক্রমিক ও অনুমানযোগ্য; (২) **স্থির নেটওয়ার্ক পরিচয়** — Headless Service-এর সাথে প্রতিটি Pod-এর নিজস্ব DNS নাম থাকে (`db-0.db.production.svc.cluster.local`); (৩) **স্থির স্টোরেজ** — প্রতিটি Pod-এর নিজস্ব PVC থাকে (`volumeClaimTemplates` দিয়ে বানানো), Pod বদলালেও একই নামের নতুন Pod একই PVC পায়। এছাড়া StatefulSet Pod-গুলোকে **ক্রমানুসারে** চালায় (প্রথমে db-0, তারপর db-1) ও মুছে ফেলে (উল্টো ক্রমে) — ডেটাবেস ক্লাস্টারের মতো, যেখানে Primary প্রথমে চালু হতে হয়। আপডেটের সময়ও ক্রমানুসারে হয়, আর **Pod Management Policy** দিয়ে সমান্তরাল চালু করা যায় — তবে ডেটাবেসের ক্ষেত্রে ক্রমানুসারেই নিরাপদ।

```
# PostgreSQL StatefulSet (TechNova-র মূল ডেটাবেস)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: production
spec:
  serviceName: postgres
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: registry.technova.example/postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
```

```
              value: technova_app
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: password
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          resources:
            requests:
              cpu: "500m"
              memory: 1Gi
            limits:
              cpu: "1"
              memory: 2Gi
          volumeMounts:
            - name: db-data
              mountPath: /var/lib/postgresql/data
          livenessProbe:
            exec:
              command: ["pg_isready", "-U", "technova_app"]
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "technova_app"]
            initialDelaySeconds: 5
            periodSeconds: 5
  volumeClaimTemplates:
    - metadata:
        name: db-data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: fast-ssd
        resources:
          requests:
            storage: 20Gi
---
# Headless Service — প্রতিটি Pod-এর নিজস্ব DNS নাম
apiVersion: v1
```

```
kind: Service
metadata:
  name: postgres
  namespace: production
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
```

> **WARNING:** StatefulSet-এ `volumeClaimTemplates` দিয়ে বানানো PVC স্বয়ংক্রিয়ভাবে মুছে যায় না — StatefulSet মুছে ফেললেও PVC ও PV থেকে যায়। এটি ইচ্ছাকৃত সুরক্ষা (ডেটা হারানো রোধে), কিন্তু ভুলে গেলে অপ্রয়োজনীয় খরচ হয়। StatefulSet মুছে ফেলার পর PVC-গুলো হাতে পরীক্ষা করে মুছুন — আর মুছে ফেলার আগে ব্যাকআপ নিশ্চিত করুন।

১০.৬ ডেটাবেস কাজের চাপ — Kubernetes-এ ডেটাবেস চালানো কি উচিত?

এই প্রশ্নের উত্তর "এটা নির্ভর করে"। Kubernetes-এ ডেটাবেস চালানোর সুবিধা: একই প্ল্যাটফর্মে সবকিছু, GitOps দিয়ে পরিচালনা, আর স্বয়ংক্রিয় পুনরুদ্ধার (Pod মারা গেলে নতুন Pod একই Volume নিয়ে ফিরে আসে)। অসুবিধা: ডেটাবেসের Replication, Failover, ব্যাকআপ, আপগ্রেড — এই "দিন-২"-এর কাজগুলো Kubernetes নিজে করে না; সেগুলোর জন্য **Operator** (যেমন CloudNativePG, CrunchyData, Zalando-র Postgres Operator) বা Managed ডেটাবেস সার্ভিস লাগে। একটি ছোট দলের জন্য প্রায়শই সঠিক সিদ্ধান্ত: গুরুত্বপূর্ণ Production ডেটাবেস Managed সার্ভিসে রাখা (যেখানে ব্যাকআপ, Replication ও Failover প্রদানকারী করে), আর Development/Staging-এ Kubernetes-এ ডেটাবেস চালানো। আর যদি Kubernetes-এই Production ডেটাবেস চালান, তবে একটি পরিণত Operator ব্যবহার করুন — হাতে লেখা StatefulSet দিয়ে Production ডেটাবেসের Replication ও Failover পরিচালনা করা অত্যন্ত ঝুঁকিপূর্ণ।

বিকল্প	সুবিধা	অসুবিধা	কখন
Managed ডেটাবেস (RDS, Cloud SQL)	ব্যাকআপ, Replication, Failover প্রদানকারীর দায়িত্ব	খরচ বেশি, ক্লাউডে বাঁধা	Production-এর গুরুত্বপূর্ণ ডেটা, ছোট দল
Operator দিয়ে Kubernetes-এ	একই প্ল্যাটফর্ম, স্বয়ংক্রিয় পরিচালনা	Operator-এর নিজের রক্ষণাবেক্ষণ	প্ল্যাটফর্ম দল আছে, একাধিক পরিবেশ
হাতে StatefulSet	সহজ, নিয়ন্ত্রণ বেশি	Replication/Failover নিজে করতে হয়	Development, শেখা, একক ডেটাবেস

১০.৭ স্টোরেজের কর্মক্ষমতা — IOPS, Throughput, Latency

স্টোরেজ বেছে নেওয়ার সময় তিনটি সংখ্যা দেখুন: **IOPS** (প্রতি সেকেন্ডে কতটি পড়া/লেখার কাজ — ছোট ছোট অনেক কাজের জন্য গুরুত্বপূর্ণ, যেমন ডেটাবেসের লেনদেন), **Throughput** (প্রতি সেকেন্ডে কত ডেটা — বড় ফাইল পড়া/লেখার জন্য গুরুত্বপূর্ণ, যেমন ব্যাকআপ), **Latency** (একটি কাজে কত সময় লাগে — ডেটাবেসের সাড়ার জন্য গুরুত্বপূর্ণ)।

ডেটাবেসের জন্য IOPS ও Latency সবচেয়ে গুরুত্বপূর্ণ — তাই দ্রুত SSD (যেমন gp3) বেছে নিন; বড় ফাইল সংরক্ষণের জন্য Throughput ও খরচ দেখুন। একটি সাধারণ ভুল: ডেটাবেসের জন্য সস্তা HDD বেছে নেওয়া — ডিস্ক সস্তা হলেও অ্যাপ ধীর হয়ে যায়, আর ব্যবহারকারীরা ক্ষতিগ্রস্ত হন। আরেকটি বিষয়: **Volume Expansion** — ডিস্ক পূর্ণ হয়ে গেলে PVC-র আকার বাড়ানো যায় কি না (`allowVolumeExpansion: true`) — এটি StorageClass-এ ঠিক করতে হয়, আর অনেক CSI Driver-এ ডিস্ক বাড়ানো যায়, কিন্তু ছোট করা যায় না। ডিস্কের ব্যবহার নিয়মিত পর্যবেক্ষণ করুন — ৮০% পূর্ণ হলে সতর্কতা (Alert) বসান, ৯০% হলে ব্যবস্থা নিন।

```
# PVC-র আকার বাড়ানো (StorageClass-এ allowVolumeExpansion: true থাকলে)
kubectl patch pvc db-data-postgres-0 -n production \
  -p '{"spec":{"resources":{"requests":{"storage":"40Gi"}}}}'
kubectl get pvc -n production

# ডিস্কের ব্যবহার দেখা
kubectl get pvc -n production
kubectl exec -n production postgres-0 -- df -h /var/lib/postgresql/data

# Volume-এর অবস্থা ও ইভেন্ট
kubectl describe pvc db-data-postgres-0 -n production
kubectl -n kube-system get pods -l app.kubernetes.io/name=ebs-csi-driver
```

১০.৮ Snapshot, Backup ও Recovery

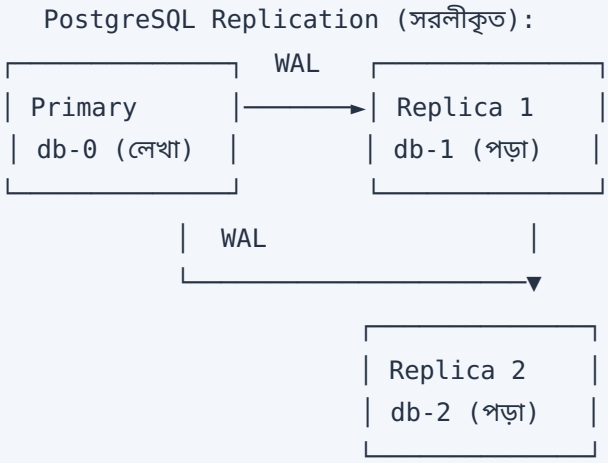
তিনটি ভিন্ন জিনিস, তিনটি ভিন্ন উদ্দেশ্য: **Snapshot** হলো একটি নির্দিষ্ট মুহূর্তে Volume-এর ছবি — দ্রুত নেওয়া যায়, দ্রুত ফেরানো যায়, কিন্তু সাধারণত একই অঞ্চলে/ক্লাউডে থাকে; **Backup** হলো ডেটার অনুলিপি অন্য জায়গায় (অন্য অঞ্চলে বা Object Storage-এ) — বিপর্যয়ে পুনরুদ্ধারের জন্য; **Recovery** হলো Snapshot বা Backup থেকে ডেটা ফিরিয়ে আনার পরীক্ষিত প্রক্রিয়া। Kubernetes-এ **VolumeSnapshot** (CSI Snapshot) দিয়ে Volume-এর Snapshot নেওয়া যায়, আর তা থেকে নতুন PVC বানানো যায়। তবে ডেটাবেসের ক্ষেত্রে শুধু Volume Snapshot যথেষ্ট নয় — ডেটাবেসের ভেতরের অবস্থা (যেমন চলমান লেনদেন) Snapshot-এ সামঞ্জস্যপূর্ণ নাও হতে পারে; তাই ডেটাবেসের নিজস্ব ব্যাকআপ (যেমন PostgreSQL-এর `pg_dump` বা WAL Archiving) ও Snapshot — দুটোই লাগে। আর সবচেয়ে গুরুত্বপূর্ণ নিয়ম: ব্যাকআপ যা কখনো পুনরুদ্ধার করে পরীক্ষা করা হয়নি, তা ব্যাকআপ নয়। নিয়মিত পুনরুদ্ধারের মহড়া দিন — একটি পরীক্ষার Namespace-এ Backup থেকে ডেটাবেস ফিরিয়ে আনুন, আর ডেটা ঠিক আছে কি না যাচাই করুন।

```
# VolumeSnapshotClass ও Snapshot
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: fast-ssd-snapshot
driver: ebs.csi.aws.com
deletionPolicy: Retain
---
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
```

```
metadata:
  name: db-snapshot-2026-11-09
  namespace: production
spec:
  volumeSnapshotClassName: fast-ssd-snapshot
  source:
    persistentVolumeClaimName: db-data-postgres-0
---
# Snapshot থেকে নতুন PVC বানানো (পুনরুদ্ধার)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data-restored
  namespace: staging
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 20Gi
  dataSource:
    name: db-snapshot-2026-11-09
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
---
# ডেটাবেসের নিজস্ব ব্যাকআপ (PostgreSQL উদাহরণ)
kubectl exec -n production postgres-0 -- \
  pg_dump -U technova_app technova > /backup/technova-2026-11-09.sql
# Velero-র মতো টুল দিয়ে ক্লাস্টার-ব্যাপী ব্যাকআপ (PV + অবজেক্ট) নেওয়া যায়
```

১০.৯ ডেটা সামঞ্জস্য, Replication ও Failover

ডেটাবেসের Replication-এ দুটি মডেল: **Synchronous** (Primary লেখার আগে Replica-কে নিশ্চিত করে — ডেটা হারায় না, কিন্তু লেখা ধীর হয়) ও **Asynchronous** (Primary প্রথমে লেখে, Replica পরে ধরে — লেখা দ্রুত, কিন্তু Primary মারা গেলে কিছু ডেটা হারাতে পারে)। কোনটি বেছে নেবেন, তা নির্ভর করে আপনার **RPO (Recovery Point Objective)**-এর উপর — কতটুকু ডেটা হারানো মেনে নেওয়া যায়। **Failover**-এর সময়: Primary মারা গেলে Replica-কে Primary বানানো হয় — কিন্তু কে সিদ্ধান্ত নেবে? হাতে করলে সময় লাগে, আর ভুলের ঝুঁকি থাকে; Operator বা Managed সার্ভিস স্বয়ংক্রিয়ভাবে করে। আরেকটি গুরুত্বপূর্ণ ধারণা: **Split-Brain** — নেটওয়ার্ক বিভক্ত হলে দুটি অংশই নিজেকে Primary ভেবে লিখতে শুরু করে — ফলে ডেটা দুই দিকে সরে যায়। এটি রোধে Quorum (সংখ্যাগরিষ্ঠ) ভিত্তিক সিদ্ধান্ত লাগে — যেমন etcd-র মতো। ডেটাবেসের ক্ষেত্রে এই সিদ্ধান্তগুলো অ্যাপ ও Operator-এর স্তরে হয়, Kubernetes নিজে করে না।



Primary মারা গেলে: Replica-কে Primary বানানো (Failover) — Operator বা Managed সার্ভিস স্বয়ংক্রিয়ভাবে করে।

১০.১০ প্রোডাকশন দৃশ্যকল্প: TechNova-র ডিস্ক পূর্ণ হয়ে গেল

TechNova-র ডেটাবেসের ডিস্ক এক রাতে ১০০% পূর্ণ হয়ে গেল — লগ ও অস্থায়ী ফাইল জমে। ডেটাবেস লেখা বন্ধ করে দিল, অ্যাপ ত্রুটি দিতে শুরু করল। কী করা হলো? প্রথমে অস্থায়ী ফাইল মুছে জায়গা করা হলো, তারপর PVC-র আকার ২০ গিগাবাইট থেকে ৪০ গিগাবাইটে বাড়ানো হলো (`allowVolumeExpansion: true` ছিল বলে তা সম্ভব হলো), আর ডেটাবেস আবার চালু হলো। তারপর স্থায়ী সমাধান: ডিস্কের ব্যবহারে সতর্কতা (৮০% ও ৯০%-এ Alert), লগ ঘোরানো (Rotation) ও পুরোনো ডেটা সংরক্ষণের (Archival) নীতি, আর নিয়মিত ক্ষমতা পর্যালোচনা। শিক্ষা: ডিস্ক পূর্ণ হওয়া একটি প্রতিরোধযোগ্য বিপর্যয় — পর্যবেক্ষণ ও সতর্কতা থাকলে তা কখনোই হঠাৎ ঘটে না।

১০.১১ সাধারণ ভুল

- **Pod-এর ভেতরে ডেটা রাখা:** emptyDir বা কন্টেইনারের ফাইল সিস্টেমে ডেটা রাখা — Pod মারা গেলেই ডেটা হারায়।
- **Deployment দিয়ে ডেটাবেস চালানো:** Deployment-এ Pod-এর নাম, ক্রম ও Volume স্থির নয় — ডেটাবেসের জন্য StatefulSet ব্যবহার করুন।
- **ভুল Access Mode:** RWO ডিস্ককে RWX ভেবে একাধিক Pod থেকে লেখা — বেশিরভাগ Block Storage তা সমর্থন করে না।
- **Reclaim Policy না বোঝা:** `Delete` নীতিতে PVC মুছলেই ডিস্ক মুছে যায় — গুরুত্বপূর্ণ ডেটায় `Retain` বিবেচনা করুন।
- **ব্যাকআপ পরীক্ষা না করা:** ব্যাকআপ নেওয়া হয়, কিন্তু কখনো ফিরিয়ে আনা হয়নি — বিপর্যয়ের দিন দেখা যায় ব্যাকআপটি অকেজো।
- **ডিস্কের ব্যবহার না দেখা:** ১০০% পূর্ণ না হওয়া পর্যন্ত কেউ জানে না — ৮০%-এ Alert বসান।
- **ডেটাবেসের নিজস্ব ব্যাকআপ না নেওয়া:** শুধু Volume Snapshot-এর উপর ভরসা করা — ডেটাবেসের ভেতরের সামঞ্জস্য নিশ্চিত হয় না।
- **ভুল অঞ্চলে ডিস্ক:** `volumeBindingMode: Immediate` দিয়ে ভুল অঞ্চলে ডিস্ক বানানো — Pod ডিস্ক ব্যবহার করতে পারে না।

১০.১২ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
PVC Pending-এ আটকে আছে	StorageClass নেই, Provisioner ব্যর্থ, অঞ্চলে ক্ষমতা নেই	`kubectl describe pvc` ও CSI Controller-এর লগ দেখুন
Pod-এ Volume Mount ব্যর্থ	ডিস্ক অন্য অঞ্চলে, ডিস্ক ইতিমধ্যে অন্য নোডে লাগানো	Pod ও PV-র অঞ্চল মেলান; `kubectl describe pod` -এর Events দেখুন
ডিস্ক পূর্ণ	লগ/অস্থায়ী ফাইল জমেছে, ডেটা বেড়েছে	অস্থায়ী ফাইল মুছুন; PVC বাড়ান; Rotation নীতি বসান
ডেটাবেস ধীর	ডিস্কের IOPS কম, Latency বেশি	দ্রুত StorageClass-এ যান; IOPS ও Latency মাপুন
Snapshot ব্যর্থ	CSI Driver Snapshot সমর্থন করে না, VolumeSnapshotClass ভুল	Driver-এর সমর্থন যাচাই করুন; Class-এর নাম মেলান
StatefulSet-এর Pod ক্রম ভেঙেছে	আগের Pod এখনো Terminating, PVC আটকে আছে	Pod-এর অবস্থা দেখুন; Volume Detach-এর সময় দিন
পুনরুদ্ধারের পর ডেটা পুরোনো	Snapshot-এর সময় ও ব্যাকআপের সময়ের পার্থক্য	Snapshot-এর সময় লিখে রাখুন; ডেটাবেস ব্যাকআপের সাথে মেলান

```
# স্টোরেজ ডিবাগের ক্রম
kubectl get pvc,pv -n production
kubectl describe pvc db-data-postgres-0 -n production
kubectl describe pod postgres-0 -n production | grep -A 10 Events
kubectl -n kube-system get pods -l app.kubernetes.io/name=ebs-csi-driver
kubectl -n kube-system logs -l app.kubernetes.io/name=ebs-csi-driver -c csi-provisioner
kubectl get volumesnapshot -n production
kubectl get storageclass
```

> **TROUBLESHOOTING:** “FailedMount” বা “FailedAttachVolume” ত্রুটি দেখলে প্রথমে `kubectl describe pod` -এর Events পড়ুন — সেখানে লেখা থাকে ডিস্ক কোন অঞ্চলে, কোন নোডে লাগানোর চেষ্টা হয়েছে, আর কেন ব্যর্থ হয়েছে। তারপর CSI Driver-এর লগ দেখুন। অনেক সময় কারণটি সহজ: Pod এক অঞ্চলে, ডিস্ক অন্য অঞ্চলে।

১০.১৩ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Volume-এর ডেটা এনক্রিপ্ট করুন (Encryption at Rest — CSI Driver বা ক্লাউডের সেটিংয়ে); Snapshot ও ব্যাকআপও এনক্রিপ্টেড রাখুন; PVC-র RBAC অনুমতি সীমিত রাখুন (যে কেউ PVC মুছতে পারলে ডেটা মুছে যেতে পারে); আর ব্যাকআপের প্রবেশাধিকার শুধু প্রয়োজনে দিন। খরচ: প্রতিটি PV-র খরচ (আকার × ধরন), Snapshot-এর খরচ (Snapshot-গুলো জমে গেলে খরচ বাড়ে — পুরোনো Snapshot মুছুন), অঞ্চলের মধ্যে ডেটা স্থানান্তরের খরচ, আর

অব্যবহৃত PVC/PV-র খরচ (StatefulSet মুছে ফেলার পর PVC থেকে গেলে খরচ চলতেই থাকে)। নিয়মিত `kubectl get pv` দিয়ে দেখুন কোন Volume আসলে ব্যবহৃত হচ্ছে, আর কোনটি অলস পড়ে আছে।

১০.১৪ প্রকৌশলগত আপস (Trade-off)

Kubernetes-এ ডেটাবেস বনাম Managed ডেটাবেস — এই সিদ্ধান্তটি খরচ, নিয়ন্ত্রণ ও পরিচালন-ক্ষমতার ভারসাম্য। Managed সার্ভিসে খরচ বেশি, কিন্তু পরিচালন-ভার কম; Kubernetes-এ খরচ কম, কিন্তু Replication, Failover ও ব্যাকআপের দায়িত্ব আপনার। একইভাবে, Block Storage (দ্রুত, একটি নোডে) বনাম Shared Filesystem (অনেক নোডে, ধীর) — ডেটাবেসের জন্য Block, ভাগ করা ফাইলের জন্য Shared। আর Snapshot বনাম ব্যাকআপ — Snapshot দ্রুত, কিন্তু একই জায়গায়; ব্যাকআপ ধীর, কিন্তু বিপর্যয়ে কাজে আসে। দুটোই লাগে, একটিকে অন্যটির বিকল্প ভাববেন না।

১০.১৪.১ স্থানীয় স্টোরেজ, Ephemeral Volume ও ক্যাশ

সব ডেটা Persistent Volume-এ রাখতে হয় না। তিনটি ক্ষেত্রে **Ephemeral** (ক্ষণস্থায়ী) স্টোরেজই সঠিক: (১) **ক্যাশ** — যেমন অ্যাপের অস্থায়ী ক্যাশ, যা হারালে আবার বানানো যায়; (২) **অস্থায়ী কাজের ফাইল** — যেমন ছবি প্রক্রিয়াকরণের মাঝামাঝি ফাইল; (৩) **খালি ডিরেক্টরি** (`emptyDir`) — Pod-এর জীবদ্দশায় থাকে, Pod মারা গেলে মুছে যায়। Ephemeral স্টোরেজের সুবিধা: দ্রুত, সস্তা, আর কোনো CSI Driver লাগে না — এটি নোডের স্থানীয় ডিস্ক ব্যবহার করে। তবে সীমাও আছে: নোডের ডিস্ক পূর্ণ হলে Pod Evict হতে পারে; আর নোডের স্থানীয় ডিস্ক সাধারণত Network Storage-এর চেয়ে কম নির্ভরযোগ্য। আরেকটি বিকল্প: **Local Persistent Volume** — নোডের স্থানীয় ডিস্ককে PV হিসেবে ব্যবহার করা; এটি খুব দ্রুত (নেটওয়ার্কের বিলম্ব নেই), কিন্তু Pod সেই নোডেই আটকে থাকে — নোড মারা গেলে ডেটাও যায়। তাই Local PV শুধু সেই কাজে ব্যবহার করুন, যেখানে গতি অত্যন্ত গুরুত্বপূর্ণ আর ডেটা অন্য কোথাও Replicate করা আছে (যেমন কিছু ডেটাবেসের নিজস্ব Replication)। নিয়মটি সহজ: ডেটা হারালে ক্ষতি হলে Persistent Volume; হারালে আবার বানানো গেলে Ephemeral।

> **REAL WORLD:** TechNova-র ছবি-প্রক্রিয়াকরণ সার্ভিসটি প্রথমে প্রতিটি কাজের জন্য একটি PVC বানাত — ধীর ও ব্যয়বহুল। পরে তারা বুঝল মাঝামাঝি ফাইলগুলো অস্থায়ী; emptyDir-এ সরিয়ে নেওয়ার পর কাজটি দ্রুত হলো, আর PVC-র সংখ্যা শূন্যে নেমে এলো। শিক্ষা: স্টোরেজ বেছে নেওয়ার আগে জিজ্ঞেস করুন — এই ডেটা কি Pod-এর চেয়ে বেশি দিন বাঁচতে হবে?

১০.১৫ অধ্যায়ের সারাংশ

Kubernetes-এ স্টোরেজের ভিত্তি: CSI Driver ডিস্ক বানায়, StorageClass ধরন ঠিক করে, PVC অনুরোধ করে, PV আসল ডিস্ক। StatefulSet স্থির নাম, স্থির নেটওয়ার্ক পরিচয় ও স্থির Volume দিয়ে ডেটাবেসের মতো কাজের চাপ চালায়। ডেটাবেস চালানোর সিদ্ধান্তে Operator বা Managed সার্ভিসের কথা ভাবুন — হাতে লেখা StatefulSet শুধু সহজ ক্ষেত্রে যথেষ্ট। Snapshot, ব্যাকআপ ও পুনরুদ্ধারের মহড়া ছাড়া ডেটা নিরাপদ নয়; আর ডিস্কের ব্যবহার, IOPS ও Latency পর্যবেক্ষণ ছাড়া কর্মক্ষমতা নিশ্চিত করা যায় না।

১০.১৬ জ্ঞান যাচাই

1. Stateless ও Stateful অ্যাপের পার্থক্য কী? প্রতিটির একটি উদাহরণ দিন।
2. CSI, StorageClass, PV ও PVC-র সম্পর্ক ব্যাখ্যা করুন — Dynamic Provisioning-এর প্রবাহ আঁকুন।
3. ReadWriteOnce, ReadOnlyMany ও ReadWriteMany-র পার্থক্য কী? ডেটাবেসের জন্য কোনটি?
4. volumeBindingMode-এ WaitForFirstConsumer কেন গুরুত্বপূর্ণ?

5. StatefulSet Deployment থেকে কী কী বিষয়ে ভিন্ন? প্রতিটি ভিন্নতার কারণ বলুন।
6. Reclaim Policy-তে Delete ও Retain-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
7. Snapshot, Backup ও Recovery-র পার্থক্য কী? কেন তিনটিই লাগে?
8. ডেটাবেসের জন্য শুধু Volume Snapshot যথেষ্ট নয় কেন?
9. Synchronous ও Asynchronous Replication-এর পার্থক্য কী? RPO-র সাথে এর সম্পর্ক কী?
10. Kubernetes-এ ডেটাবেস চালানো বনাম Managed ডেটাবেস — কখন কোনটি বেছে নেবেন?

১০.১৭ ব্যবহারিক অনুশীলন

1. একটি পরীক্ষার Namespace-এ এই অধ্যায়ের StorageClass, PVC ও PostgreSQL StatefulSet প্রয়োগ করুন। Pod-এ ডেটা লিখুন (একটি টেবিল বানিয়ে কয়েকটি সারি যোগ করুন), তারপর Pod মুছে ফেলুন ও দেখুন নতুন Pod একই নামে ফিরে এসে একই ডেটা পায় কি না। তারপর StatefulSet মুছে ফেলুন ও দেখুন PVC থেকে যায় কি না।
2. একটি VolumeSnapshot নিন, তা থেকে একটি নতুন PVC বানিয়ে Staging Namespace-এ ডেটাবেসের একটি অনুলিপি চালু করুন ও ডেটা মিলিয়ে দেখুন। তারপর `pg_dump` দিয়ে ডেটাবেসের নিজস্ব ব্যাকআপ নিন ও তা থেকে পুনরুদ্ধার করুন — দুটি পদ্ধতির পার্থক্য লিখুন।
3. TechNova-র জন্য একটি ব্যাকআপ ও পুনরুদ্ধার পরিকল্পনা লিখুন: কোন ডেটার কত ঘন ঘন ব্যাকআপ হবে, Snapshot কতদিন রাখা হবে, ব্যাকআপ কোথায় রাখা হবে, পুনরুদ্ধারের মহড়া কত ঘন ঘন হবে, RTO ও RPO কত — আর মহড়ার ধাপগুলো (কে কী করবে, কীভাবে যাচাই করবে) লিখুন।
4. একটি PVC-র আকার বাড়ানোর মহড়া দিন: ডিস্ক প্রায় পূর্ণ করে (পরীক্ষার ডেটা দিয়ে), তারপর PVC বাড়িয়ে দেখুন ডেটাবেসে কোনো ব্যাঘাত ঘটে কি না, আর বাড়ানোর পর `df -h` দিয়ে যাচাই করুন।

PART III • DEVOPS ENGINEERING MASTERY

GitOps & Platform Engineering

GitOps Fundamentals, Argo CD ↔ Flux CD, Helm ↔ Kustomize, Internal Developer Platform Architecture, Developer Portals ↔ Self-Service Infrastructure

অধ্যায় ১১ • PART III — GitOps & Platform Engineering

Chapter 11 — GitOps Fundamentals

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **GitOps** বলতে ঠিক কী বোঝায় তা প্রথম নীতি (First Principles) থেকে ব্যাখ্যা করতে; **Desired State** (কাঙ্ক্ষিত অবস্থা) ও **Actual State** (বাস্তব অবস্থা)-এর পার্থক্য এবং **Reconciliation** (সমন্বয়) কীভাবে সেই ব্যবধান বন্ধ করে, তা বলতে; Git-কে অবকাঠামো ও অ্যাপ্লিকেশন কনফিগারেশনের **Source of Truth** (সত্যের একমাত্র উৎস) হিসেবে ব্যবহার করতে; একটি সম্পূর্ণ GitOps রিপোজিটরি কাঠামো নকশা করতে; **Drift Detection** (বিচ্যুতি শনাক্তকরণ), **Rollback** ও পরিবেশ প্রমোশনের (Environment Promotion) কৌশল প্রয়োগ করতে; এবং কখন GitOps উপযুক্ত আর কখন প্রচলিত CI/CD-ই যথেষ্ট, সেই ট্রেড-অফের যুক্তিসংগত সিদ্ধান্ত নিতে।

- GitOps-এর চারটি মূল নীতি উদাহরণসহ ব্যাখ্যা করা।
- Push-ভিত্তিক ও Pull-ভিত্তিক ডেপ্লয়মেন্টের পার্থক্য তুলনা করা।
- TechNova-র জন্য মনোরেপো ও মাল্টি-রেপো কাঠামোর মধ্যে বেছে নেওয়া।
- Drift শনাক্ত হলে কী করণীয়, তা ধাপে ধাপে বলা।
- GitOps-এ গোপন তথ্য (Secret) নিরাপদে পরিচালনার পদ্ধতি বর্ণনা করা।

১১.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায় বুঝতে আপনার যা জানা থাকা দরকার, তা আগের ভলিউমগুলোতে এসেছে: **Git**-এর `commit`, `branch`, `merge` ও `pull request`; **Kubernetes**-এর **Deployment**, **Service**, **ConfigMap** ও **Namespace**; **Declarative Configuration** — অর্থাৎ YAML-এ "শেষ অবস্থাটি কেমন হবে" ঘোষণা করা; এবং CI/CD পাইপলাইনের মৌলিক ধারণা — কোড বদলালে পরীক্ষা চলে, ইমেজ তৈরি হয়, তারপর ডেপ্লয় হয়। **Helm** ও **Kustomize**-এর প্রাথমিক ধারণা থাকলে ভালো, তবে না থাকলেও চলবে — অধ্যায় ১৩-এ সেগুলো বিস্তারিত আসবে। যদি Kubernetes-এর **Control Loop** ধারণাটি মনে না থাকে, তবে এটুকু মনে রাখুন: Kubernetes সবসময় বাস্তব অবস্থাকে কাঙ্ক্ষিত অবস্থার দিকে টেনে নেয় — আর GitOps সেই একই ধারণাকে পুরো ক্লাস্টারের স্তরে, Git থেকে শুরু করে, প্রয়োগ করে।

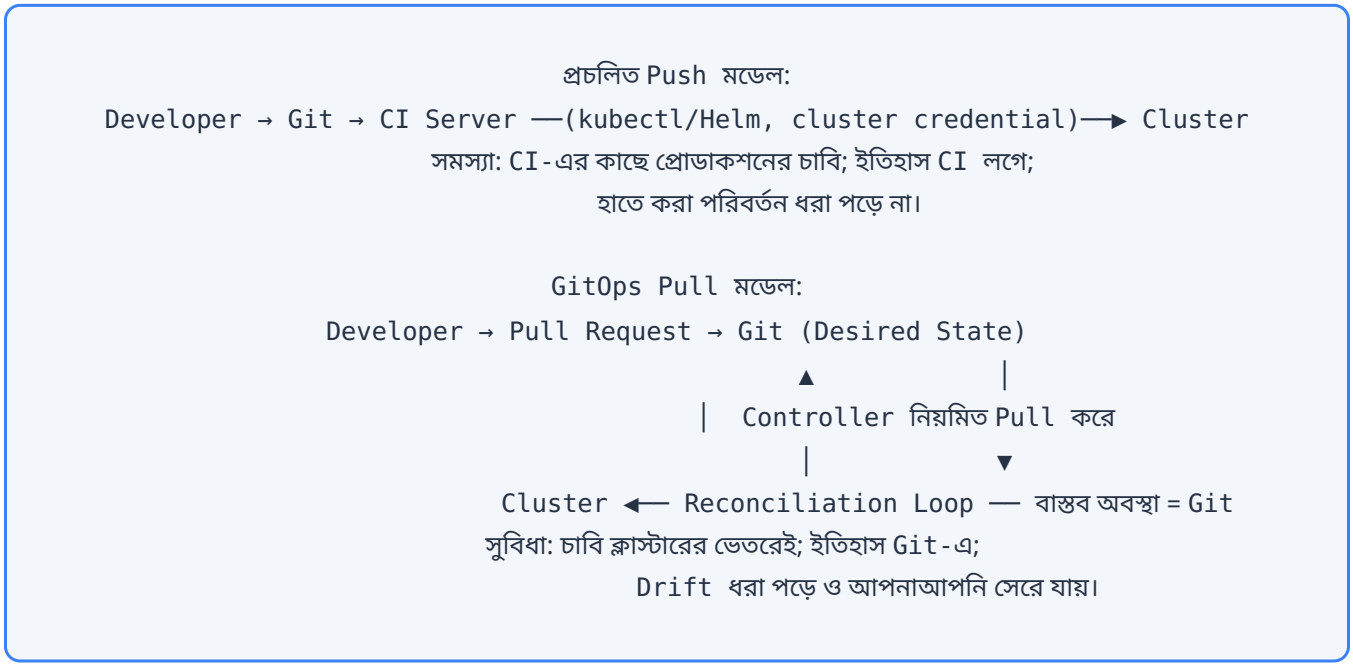
> **CONCEPT:** GitOps কোনো একটি টুলের নাম নয় — এটি একটি অপারেশনাল মডেল। Argo CD বা Flux হলো সেই মডেল বাস্তবায়নের টুল। GitOps-কে টুলের সাথে গুলিয়ে ফেললে আপনি টুল বদলালেই মডেলটি হারিয়ে ফেলবেন; নীতিগুলো আয়ত্ত করলে যেকোনো টুল দিয়েই তা বাস্তবায়ন করতে পারবেন।

১১.৩ প্রথম নীতি থেকে GitOps: সমস্যাটি কী ছিল

প্রচলিত CI/CD-তে ডেপ্লয়মেন্ট হয় **Push** পদ্ধতিতে: CI সার্ভার কোড পরীক্ষা করে, ইমেজ বানায়, তারপর ক্লাস্টারের **API Server**-এ সরাসরি `kubectl apply` চালায় বা Helm চালায়। এই মডেলের তিনটি গভীর সমস্যা আছে। প্রথমত, CI সার্ভারের কাছে ক্লাস্টারের **Credential** (প্রমাণপত্র) থাকে — সেই সার্ভার আক্রান্ত হলে আক্রমণকারী সরাসরি প্রোডাকশনে ঢুকতে পারে; এটিকে বলে **Blast Radius** (ক্ষতির পরিধি) বড় হওয়া। দ্বিতীয়ত, ডেপ্লয়মেন্টের ইতিহাস ছড়িয়ে থাকে — CI লগে, কারও টার্মিনালে, কারও মনে — Git-এ থাকে শুধু অ্যাপের কোড, ক্লাস্টারে আসলে কী চলছে তার পূর্ণ ছবি থাকে না। তৃতীয়ত, কেউ ক্লাস্টারে হাতে কিছু বদলে ফেললে (`kubectl edit` দিয়ে রেপ্লিকা বাড়ানো, হাতে একটি পোর্ট খোলা) তা

কেউ জানতে পারে না; পরের ডেপ্লয়মেন্টে বা ক্লাস্টার পুনর্নির্মাণে সেই পরিবর্তন নীরবে হারিয়ে যায়, অথবা তার উল্টো — হাতে করা পরিবর্তনই থেকে যায়, আর কোডের সাথে মেলে না।

GitOps এই সমস্যাগুলোকে উল্টে দেয়। এখানে Git রিপোজিটরিতে লেখা থাকে ক্লাস্টারের কাঙ্ক্ষিত অবস্থার সম্পূর্ণ ঘোষণা — কোন অ্যাপের কোন সংস্করণ, কয়টি রেপ্লিকা, কোন কনফিগারেশন। ক্লাস্টারের ভেতরে চলা একটি **Operator** বা **Controller** নিয়মিত Git-কে দেখে (Pull করে) এবং বাস্তব অবস্থাকে Git-এর লেখার সাথে মেলায়। CI-এর কাজ শুধু কোড পরীক্ষা করা ও ইমেজ বানানো; ক্লাস্টারে ঢোকার চাবি CI-এর কাছে থাকে না। পরিবর্তনের একমাত্র পথ হলো Git-এ Pull Request — ফলে প্রতিটি পরিবর্তনের লেখক, কারণ, পর্যালোচক ও সময়ের রেকর্ড থাকে।



১১.৪ GitOps-এর চারটি মূল নীতি

GitOps-এর সংজ্ঞা দাঁড়িয়ে আছে চারটি নীতির উপর — এগুলো Weaveworks-এর মূল প্রস্তাব থেকে এসেছে এবং আজ শিল্পের মানদণ্ড।

১১.৪.১ নীতি ১ — সম্পূর্ণ সিস্টেম ঘোষণামূলকভাবে বর্ণিত (Declarative)

সিস্টেমের কাঙ্ক্ষিত অবস্থা **Declarative Configuration**-এ লেখা থাকে — সাধারণত YAML। "সার্ভার বানাও, তারপর প্যাকেজ ইনস্টল করো" নয়; "তিনটি রেপ্লিকা, এই ইমেজ, এই পরিবেশ চলক" — এই ঘোষণাই যথেষ্ট। ঘোষণামূলক বর্ণনার সুবিধা: এটি পড়া যায়, পর্যালোচনা করা যায়, পরীক্ষা করা যায় এবং মেশিন তা বাস্তবায়ন করতে পারে।

১১.৪.২ নীতি ২ — কাঙ্ক্ষিত অবস্থা Git-এ সংস্করণ-নিয়ন্ত্রিত (Versioned and Immutable)

Git-ই **Source of Truth**। Git-এর প্রতিটি `commit` সিস্টেমের একটি পরিচিত অবস্থার স্ন্যাপশট। Git-এর **Immutability** — একবার লেখা ইতিহাস বদলানো যায় না (সাধারণভাবে) — এর অর্থ হলো কে, কখন, কেন কী বদলালেন, তার অপরিবর্তনীয় নিরীক্ষা-পথ (Audit Trail) থাকে। কোনো সমস্যা হলে আগের কমিটে ফেরা মানেই আগের পরিচিত ভালো অবস্থায় ফেরা।

১১.৪.৩ নীতি ৩ — পরিবর্তন স্বয়ংক্রিয়ভাবে প্রয়োগ হয় (Automatically Applied)

Git-এ মার্জ হলেই একজন সফটওয়্যার **Agent** — ক্লাস্টারের ভেতরে চলা Controller — সেই পরিবর্তন স্বয়ংক্রিয়ভাবে প্রয়োগ করে। মানুষকে হাতে `kubectl apply` চালাতে হয় না। এই স্বয়ংক্রিয়তাই GitOps-কে "Git-এ YAML রাখা" থেকে আলাদা করে; YAML Git-এ রাখা অর্ধেক কাজ, স্বয়ংক্রিয় প্রয়োগই বাকি অর্ধেক।

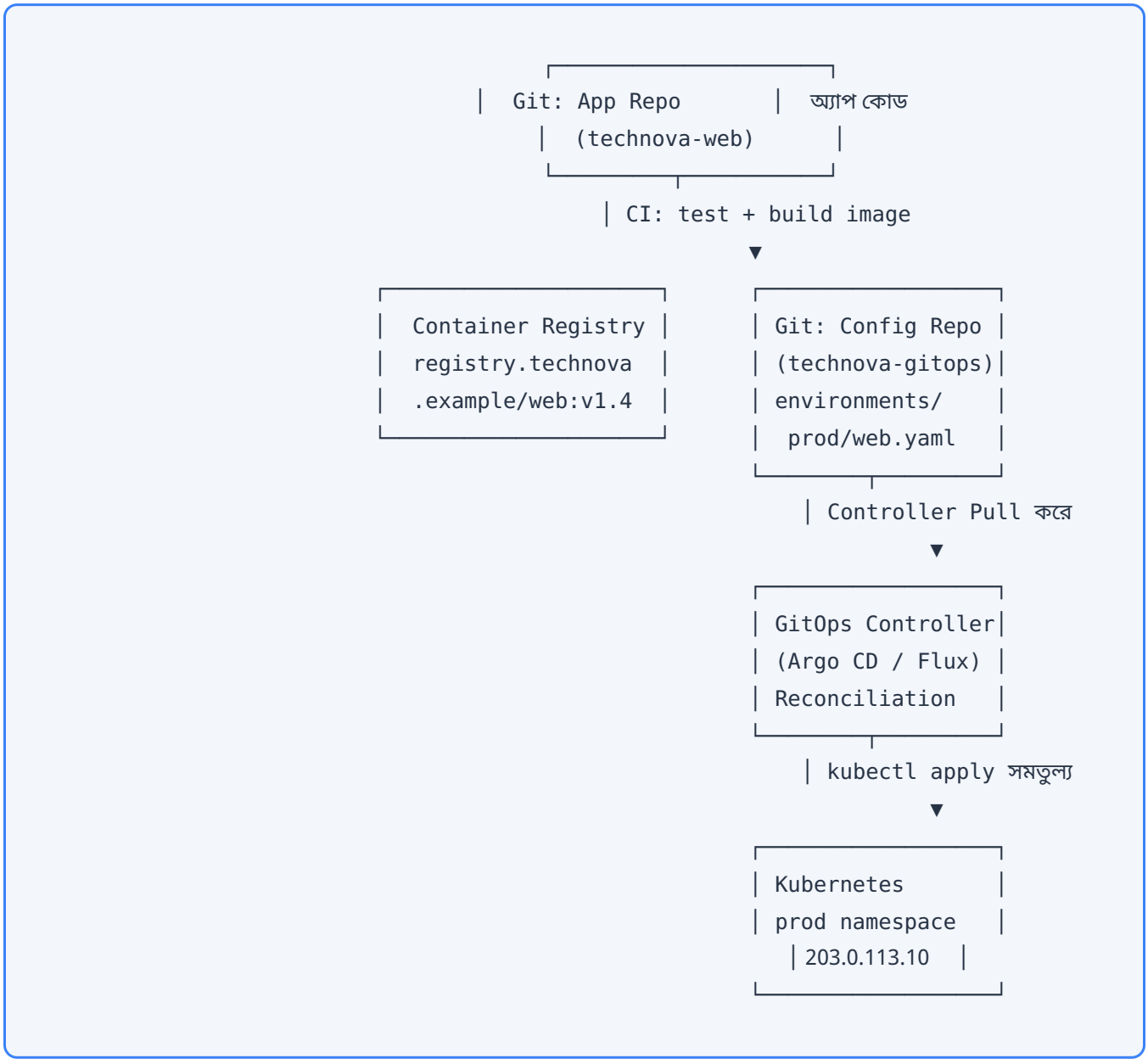
১১.৪.৪ নীতি ৪ — সফটওয়্যার Agent ক্রমাগত অবস্থা মেলায় (Continuously Reconciled)

Agent একবার প্রয়োগ করেই থেমে যায় না; এটি নিয়মিত ব্যবধানে — যেমন প্রতি তিন মিনিটে — Git-এর কাঙ্ক্ষিত অবস্থা ও ক্লাস্টারের বাস্তব অবস্থা তুলনা করে। পার্থক্য পেলেই তা সারিয়ে তোলে। একে বলে **Reconciliation Loop** বা সমন্বয় চক্র। এই চক্রই Drift-এর ওষুধ: হাতে করা পরিবর্তন টিকে থাকতে পারে না, কারণ পরের চক্রে তা মুছে গিয়ে Git-এর লেখা ফিরে আসে।

> **WARNING:** Reconciliation Loop দুধারী তলোয়ার। জরুরি অবস্থায় কেউ হাতে রেপ্লিকা বাড়িয়ে চাপ সামলালেন, আর GitOps Agent তিন মিনিট পর তা আগের সংখ্যায় ফিরিয়ে দিল — এমন ঘটনা বাস্তবে ঘটে। তাই জরুরি পরিবর্তনের নিয়ম আগেই ঠিক করুন: প্রথমে Git-এ পরিবর্তন করুন, অথবা সাময়িকভাবে Sync বন্ধ (Suspend) করার অনুমোদিত পদ্ধতি দলকে শেখান।

১১.৫ GitOps আর্কিটেকচার: উপাদানগুলো কীভাবে যুক্ত হয়

একটি সম্পূর্ণ GitOps ব্যবস্থায় সাধারণত এই উপাদানগুলো থাকে: **Application Source Repository** — অ্যাপের কোড; **Configuration / Environment Repository** — ক্লাস্টারের কাঙ্ক্ষিত অবস্থার YAML, Helm Chart বা Kustomize; **CI Pipeline** — পরীক্ষা, ইমেজ তৈরি, রেজিস্ট্রিতে পুশ, এবং (ঐচ্ছিকভাবে) কনফিগারেশন রিপোজিটরিতে ইমেজ ট্যাগ আপডেট করা; **GitOps Controller** — Argo CD বা Flux, ক্লাস্টারের ভেতরে চলে; **Kubernetes Cluster** — বাস্তব অবস্থা; এবং **Observability** — Sync অবস্থা, স্বাস্থ্য ও সতর্কতা দেখার ব্যবস্থা।



লক্ষ করুন: CI কখনো ক্লাস্টারে ঢোকে না। CI-এর শেষ কাজ হলো কনফিগারেশন রিপোজিটরিতে একটি কমিট করা — যেমন `image: web:v1.4.2` লেখা — অথবা ইমেজ ট্যাগ আপডেটের কাজটি **Image Updater** নামের একটি আলাদা উপাদানের হাতে ছেড়ে দেওয়া। ক্লাস্টারের ভেতরের Controller-ই একমাত্র উপাদান যার ক্লাস্টার পরিবর্তনের অনুমতি আছে। এই বিভাজনকে বলে **Separation of Concerns** — CI বানায়, GitOps ডেপ্লয় করে।

১১.৬ রিপোজিটরি কাঠামো: মনোরেপো বনাম মাল্টি-রেপো, অ্যাপ বনাম কনফিগারেশন

প্রথম ও সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত: অ্যাপের কোড আর ডেপ্লয়মেন্ট কনফিগারেশন কি একই রিপোজিটরিতে থাকবে, না আলাদা? বেশিরভাগ দলের জন্য সুপারিশ: **আলাদা রাখুন**। অ্যাপ রিপোজিটরি বদলায় ডেভেলপারদের হাতে, প্রতিদিন বহুবার; কনফিগারেশন রিপোজিটরি বদলায় ডেপ্লয়মেন্টের সময়, এবং তাতে প্রোডাকশনের সংবেদনশীল সেটিংস থাকে। আলাদা রাখলে অ্যাক্সেস নিয়ন্ত্রণ (কে প্রোডাকশন কনফিগারেশন বদলাতে পারে) সহজ হয়, ইতিহাস পরিষ্কার থাকে, এবং একাধিক অ্যাপ একই কনফিগারেশন কাঠামো ভাগ করতে পারে। ছোট দলে বা একটিমাত্র অ্যাপ থাকলে একই রিপোজিটরিতে `app/` ও `deploy/` ডিরেক্টরি আলাদা করেও শুরু করা যায় — পরে আলাদা করা কঠিন নয়।

```
# TechNova-র কনফিগারেশন রিপোজিটরি (technova-gitops) — সুপারিশকৃত কাঠামো
technova-gitops/
├── README.md                       # কোন পরিবেশে কীভাবে প্রমোশন হয়
├── apps/
│   ├── web/
│   │   ├── base/                   # পরিবেশ-নিরপেক্ষ সংজ্ঞা
│   │   │   ├── deployment.yaml
│   │   │   ├── service.yaml
│   │   │   └── kustomization.yaml
│   │   └── overlays/
│   │       ├── dev/
│   │       ├── staging/
│   │       └── prod/
│   └── api/
│       └── ...          # একই প্যাটার্ন
├── environments/
│   ├── dev/                        # Argo CD Application বা Flux Kustomization
│   ├── staging/
│   └── prod/
├── clusters/
│   ├── cluster-prod-01/            # ক্লাস্টার-স্তরের সংজ্ঞা (ingress, cert)
│   └── cluster-prod-02/
└── .github/workflows/              # কনফিগারেশন যাচাই (lint, kubeconform)
```

মনোরেপো বনাম মাল্টি-রেপো প্রশ্নটিও একই যুক্তিতে বিচার করুন। একটি কনফিগারেশন মনোরেপোতে সব অ্যাপ ও পরিবেশ থাকলে ক্রস-অ্যাপ পরিবর্তন একটিমাত্র Pull Request-এ করা যায়, আর Controller-এর কনফিগারেশন সহজ হয়। অসুবিধা: রিপোজিটরি বড় হয়, এবং একটি ভুল মার্জ সব অ্যাপকে প্রভাবিত করতে পারে — তাই **CODEOWNERS** ফাইল দিয়ে ডিরেক্টরিভিত্তিক পর্যালোচক বাধ্যতামূলক করুন। বহু রিপোজিটরি হলে প্রতিটি দল স্বাধীন থাকে, কিন্তু পরিবেশ-জুড়ে পরিবর্তন ছড়িয়ে যায় বহু PR-এ। TechNova-র মতো মাঝারি দলের জন্য একটি কনফিগারেশন মনোরেপো দিয়ে শুরু করা, আর অ্যাপ কোড আলাদা রাখা — এটাই ভারসাম্যপূর্ণ পছন্দ।

কাঠামো	সুবিধা	অসুবিধা	কাদের জন্য
অ্যাপ + কনফিগ একই রেপো	এক জায়গায় সব; শুরুতে সহজ	অ্যাক্সেস নিয়ন্ত্রণ কঠিন; ইতিহাস মিশে যায়	একক অ্যাপ, ১–৩ জনের দল
আলাদা কনফিগ রেপো (মনোরেপো)	পরিষ্কার বিভাজন; ক্রস-অ্যাপ PR সহজ	ইমেজ ট্যাগ আপডেটে অটোমেশন লাগে	TechNova-র মতো মাঝারি দল
প্রতি অ্যাপে আলাদা কনফিগ রেপো	দলভিত্তিক স্বাধীনতা সর্বোচ্চ	পরিবেশ-জুড়ে পরিবর্তন কঠিন	বড় সংস্থা, স্বাধীন দল

১১.৭ Desired State, Reconciliation ও Sync: ভেতরে কী ঘটে

Desired State হলো Git-এ লেখা ঘোষণা; **Actual State** হলো ক্লাস্টারে যা চলছে; **Live State** কখনও কখনও **Cluster State** নামেও পরিচিত। **Sync** হলো Desired-কে Actual-এ পরিণত করার কাজ। **Reconciliation** হলো সেই চক্র যা বারবার পার্থক্য (Diff) বের করে ও সারায়। ভেতরে ধাপগুলো এইরকম: (১) Controller Git রিপোজিটরি থেকে নির্দিষ্ট **Branch** ও **Path** পড়ে; (২) YAML রেন্ডার করে — Helm Template চালায় বা Kustomize Build করে, যদি থাকে; (৩) রেন্ডার করা ম্যানিফেস্টের সাথে ক্লাস্টারের বর্তমান অবজেক্টগুলোর তুলনা করে — একে বলে **Diff**; (৪) পার্থক্য থাকলে **Apply** করে, অর্থাৎ Kubernetes API-তে তৈরি/আপডেট/মুছে ফেলার অনুরোধ পাঠায়; (৫) প্রতিটি রিসোর্সের **Health** পরীক্ষা করে — Pod চলছে কি না, Deployment-এর রেপ্লিকা প্রস্তুত কি না; (৬) ফলাফল রেকর্ড করে ও পরের চক্রের জন্য অপেক্ষা করে।

দুটি গুরুত্বপূর্ণ ধারণা: **Sync Policy** — পরিবর্তন কি স্বয়ংক্রিয়ভাবে প্রয়োগ হবে (**Automated Sync**), না কি মানুষের অনুমোদন লাগবে (**Manual Sync**)? প্রোডাকশনে অনেক দল শুরুতে Manual রাখে, আস্থা বাড়লে Automated-এ যায়। আর **Prune** — Git থেকে মুছে ফেলা রিসোর্স ক্লাস্টার থেকেও মুছে ফেলবে কি না? Prune চালু না থাকলে Git থেকে মুছে ফেলা Deployment ক্লাস্টারে অনাথ হয়ে থেকে যায়; চালু থাকলে ভুলে মুছে ফেলা ফাইল প্রোডাকশন রিসোর্স মুছে দিতে পারে — তাই Prune-এর সাথে **Sync Window** (কখন Sync করা যাবে না, যেমন ব্যবসার ব্যস্ত সময়) ও সুরক্ষা নিয়ম ব্যবহার করুন।

```
# Reconciliation চক্রটি টার্মিনালে কেমন দেখায় (ধারণাগত)
$ kubectl -n argocd get applications
NAME          SYNC      HEALTH    AGE
web-prod      Synced    Healthy   42d
api-prod      OutOfSync Degraded 42d    # Drift বা ব্যর্থ Sync

# Drift-এর একটি সাধারণ দৃশ্য: কেউ হাতে রেপ্লিকা বাড়িয়েছিলেন
$ kubectl -n prod get deployment web -o jsonpath='{.spec.replicas}'
5   # Git-এ লেখা আছে 3; পরের Reconciliation চক্রে 3-এ ফিরবে
```

১১.৮ পরিবেশ প্রমোশন: Dev থেকে Staging হয়ে Production

একটি রিলিজ কীভাবে পরিবেশ থেকে পরিবেশে যায় — এটাই **Environment Promotion**। GitOps-এ তিনটি প্রধান পদ্ধতি আছে। **Branch-based Promotion:** প্রতিটি পরিবেশের জন্য আলাদা Branch (`dev`, `staging`, `main`); প্রমোশন মানে একটি Branch থেকে আরেকটিতে মার্জ। সহজ, তবে Branch-গুলোর মধ্যে পার্থক্য বাড়তে পারে। **Folder-based Promotion:** একই Branch-এ `overlays/dev`, `overlays/staging`, `overlays/prod`; প্রমোশন মানে এক ফোল্ডারের মান আরেক ফোল্ডারে কপি করা — সাধারণত একটি স্ক্রিপ্ট বা Image Updater দিয়ে ইমেজ ট্যাগ বদলানো। TechNova-র জন্য এটাই সুপারিশকৃত পদ্ধতি, কারণ একটি Branch-এই সব পরিবেশের ইতিহাস দেখা যায়। **Tag/Release-based Promotion:** প্রতিটি রিলিজ একটি Git Tag; পরিবেশ সেই Tag-কে অনুসরণ করে। নিরীক্ষার জন্য চমৎকার, তবে Tag পরিচালনা বাড়তি শৃঙ্খলা দাবি করে।

যে পদ্ধতিই নিন, একটি নিয়ম অপরিবর্তনীয়: **প্রমোশন সবসময় Git-এর মধ্য দিয়ে যাবে, ক্লাস্টারে হাতে কিছু করে নয়।** প্রোডাকশনে যাওয়ার আগে Staging-এ একই আর্টিফ্যাক্ট (একই ইমেজ ডাইজেস্ট) পরীক্ষা করুন — “Staging-এ এক ইমেজ, Production-এ আরেক ইমেজ” মানে আপনি আসলে পরীক্ষা না করা জিনিসই প্রোডাকশনে পাঠাচ্ছেন। ইমেজ ট্যাগ

হিসেবে `latest` ব্যবহার করবেন না; **Immutable Tag** — যেমন Git Commit SHA বা Semantic Version — ব্যবহার করুন, যাতে একই ট্যাগ কখনো ভিন্ন বিষয়বস্তু না বোঝায়।

> **REAL WORLD:** TechNova-তে একবার Staging-এ `web:v1.4.0` পরীক্ষা করে সবুজ সংকেত পাওয়ার পর প্রোডাকশনে পাঠানো হলো `web:latest` — যা এর মধ্যে আবার তৈরি হয়ে গিয়েছিল অন্য একটি কমিট থেকে। প্রোডাকশনে এলো পরীক্ষা না করা কোড, আর রাত দুটোয় একটি বাগ ধরা পড়ল। তারপর থেকে নিয়ম: ইমেজ সবসময় Digest-সহ পিন করা — `web:v1.4.0@sha256:CHANGE-ME-...`।

১১.৯ Drift Detection ও Rollback

Drift হলো Actual State-এর Desired State থেকে সরে যাওয়া। এর কারণ তিন রকম: মানুষের হাতে করা পরিবর্তন (`kubectl edit`, ড্যাশবোর্ড থেকে স্কেল করা); ক্লাস্টারের নিজস্ব স্বয়ংক্রিয়তা (যেমন **Horizontal Pod Autoscaler** রেপ্লিকা বদলায় — এটি বৈধ Drift, Git-এর সাথে যার মিল থাকার কথা নয়); এবং বাহ্যিক সিস্টেম (ক্লাউড কন্ট্রোলার, অন্য কোনো Operator)। ভালো GitOps সেটআপ এই তিনটিকে আলাদা করতে পারে: HPA-পরিচালিত ক্ষেত্রগুলোকে Diff থেকে বাদ দেওয়ার নিয়ম (**Ignore Differences**) লিখে রাখুন, যেমন `spec.replicas` যখন HPA নিয়ন্ত্রণ করে।

Drift ধরা পড়লে Controller-এর আচরণ Sync Policy অনুযায়ী হয়: **Self-Heal** চালু থাকলে তা আপনাআপনি সারিয়ে তোলে; বন্ধ থাকলে শুধু **OutOfSync** দেখায়, মানুষের সিদ্ধান্তের অপেক্ষায়। প্রোডাকশনে Self-Heal চালু রাখা সাধারণত সঠিক, তবে প্রথম কয়েক সপ্তাহ বন্ধ রেখে Drift-এর কারণগুলো বোঝা বুদ্ধিমানের কাজ — নইলে আপনি জানতেই পারবেন না কে, কেন হাতে পরিবর্তন করছে।

Rollback GitOps-এ বিস্ময়করভাবে সহজ: খারাপ রিলিজের পরের কাজ হলো Git-এ `git revert` করা। Revert কমিটটি মার্জ হলেই Controller আগের অবস্থায় ফিরিয়ে নেয় — আলাদা কোনো "রোলব্যাক বোতাম" লাগে না। তবে দুটি সতর্কতা: ডেটাবেস **Migration** ফিরিয়ে নেওয়া যায় না শুধু Git Revert দিয়ে — Migration-এর Rollback পরিকল্পনা আলাদাভাবে লিখতে হয়; এবং ইমেজ ট্যাগ Revert করার সময় নিশ্চিত করুন যে পুরোনো ইমেজটি রেজিস্ট্রিতে এখনও আছে (রেজিস্ট্রির **Retention Policy** পুরোনো ইমেজ মুছে দিলে Rollback অসম্ভব হয়ে যায়)।

```
# Rollback — GitOps পদ্ধতিতে (কোনো kubectl rollback নয়)
$ cd technova-gitops
$ git log --oneline -3
a1b2c3d web: v1.5.0-এ আপডেট (খারাপ রিলিজ)
e4f5g6h web: v1.4.2-এ আপডেট (ভালো ছিল)
$ git revert a1b2c3d        # v1.4.2-এ ফেরার কমিট তৈরি হয়
$ git push origin main
# Argo CD / Flux পরের চক্রে v1.4.2 ডেপ্লয় করবে; Sync অবস্থা দেখুন:
$ kubectl -n argocd get application web-prod -w
```

১১.১০ নিরাপত্তা বিবেচনা

GitOps নিরাপত্তার জন্য শক্তিশালী, তবে শুধু সঠিকভাবে ব্যবহার করলে। প্রথমত, Git রিপোজিটরি এখন প্রোডাকশনের চাবি — তাই **Branch Protection** (সরাসরি পুশ নিষিদ্ধ, অন্তত একজন পর্যালোচক বাধ্যতামূলক, Status Check পাস করা বাধ্যতামূলক) চালু করুন, **Signed Commit** (GPG বা SSH স্বাক্ষর) বিবেচনা করুন, এবং কনফিগারেশন রিপোজিটরিতে

অ্যাক্সেস সীমিত রাখুন। দ্বিতীয়ত, গোপন তথ্য কখনোই সাধারণ Git-এ লিখবেন না: **Sealed Secrets**, **External Secrets Operator** বা **SOPS** (Secrets OPerationS — Age/GPG দিয়ে এনক্রিপ্ট করা ফাইল) ব্যবহার করুন — অধ্যায় ১২-এ এগুলো বিস্তারিত আসবে। তৃতীয়ত, Controller-এর **Service Account**-কে **Least Privilege** দিন: শুধু যে Namespace-গুলোতে তার কাজ, সেখানেই অনুমতি; ক্লাস্টার-ব্যাপী অ্যাডমিন দেবেন না। চতুর্থত, **Admission Control** — যেমন **OPA Gatekeeper** বা **Kyverno** — দিয়ে নীতি প্রয়োগ করুন: যেমন "প্রোডাকশনে কোনো Pod-এ `latest` ট্যাগ চলবে না" বা "সব Deployment-এ Resource Limit থাকতেই হবে"। পঞ্চমত, Controller-এর নিজের অ্যাক্সেস সুরক্ষিত করুন: Argo CD-র UI-তে **SSO** ও **RBAC**, Git রিপোজিটরিতে শুধু Read-Only **Deploy Key** বা সীমিত **Token**।

> **WARNING:** Git ইতিহাস থেকে গোপন তথ্য মুছে ফেলা অত্যন্ত কঠিন — একটি কমিটে একবার পাসওয়ার্ড ঢুকলে তা সব ক্লোনে ছড়িয়ে যায়। ভুল করে Secret কমিট করলে শুধু `git rm` যথেষ্ট নয়; Secret-টি বাতিল (Revoke) করে নতুনটি বানান, তারপর ইতিহাস পরিষ্কার করার সরঞ্জাম (যেমন `git filter-repo`) ব্যবহার করুন। প্রতিরোধই আসল সমাধান: Pre-commit Hook-এ **gitleaks** বা **trufflehog** চালান।

১১.১১ খরচ ও পরিচালনা বিবেচনা

GitOps-এর সরাসরি খরচ কম: Controller-গুলো ছোট, সাধারণত কয়েকটি Pod; Git হোস্টিং ও CI-র খরচ আগেই আছে। আসল খরচ হলো ***শেখা ও রক্ষণাবেক্ষণ*** — দলকে নতুন মানসিক মডেল শেখানো, রিপোজিটরি কাঠামো ঠিক রাখা, Controller আপগ্রেড করা। ছোট দলে (২–৩ জন, একটি ক্লাস্টার, কয়েকটি অ্যাপ) GitOps-এর সুফল খরচকে ছাড়িয়ে যায়, কারণ সেটআপ একবার করলেই ডেপ্লয়মেন্টের হাতে-কলমে কাজ প্রায় শূন্যে নেমে আসে। খুব ছোট পরীক্ষামূলক প্রকল্পে — যেখানে প্রতিদিন পরিবেশ ভাঙা-গড়া হয় — সরল `kubectl apply` বা **Skaffold** যথেষ্ট হতে পারে; সেখানে পূর্ণ GitOps অপ্রয়োজনীয় ভার। খরচের আরেকটি দিক: Reconciliation-এর ঘনত্ব — খুব ঘন ঘন চক্র (প্রতি ৩০ সেকেন্ড) Git API ও ক্লাস্টার API-তে চাপ বাড়ায়; বহু অ্যাপ হলে ব্যবধান বাড়ান (যেমন ৩–৫ মিনিট) অথবা **Webhook** দিয়ে Git পুশের সাথে সাথে Sync ট্রিগার করুন।

১১.১২ TechNova দৃশ্যকল্প: প্রথম GitOps পাইপলাইন

TechNova-র ওয়েব অ্যাপ (`web.technova.example`, 203.0.113.10) এখন Push পদ্ধতিতে ডেপ্লয় হয়: CI থেকে সরাসরি `kubectl apply`। পরিকল্পনা: (১) `technova-gitops` রিপোজিটরি তৈরি করে `apps/web/base`-এ বর্তমান ম্যানিফেস্টগুলো তোলা; (২) `overlays/dev`, `overlays/staging`, `overlays/prod` বানানো — Prod-এ ৩ রেপ্লিকা, Dev-এ ১; (৩) Argo CD ইনস্টল করে (অধ্যায় ১২) প্রথমে `dev`-এ Manual Sync দিয়ে পরীক্ষা; (৪) CI-তে ইমেজ বিল্ডের পর কনফিগারেশন রিপোজিটরিতে ট্যাগ আপডেটের ধাপ যোগ করা — CI ক্লাস্টারে ঢুকবে না, শুধু Git-এ কমিট করবে; (৫) দুই সপ্তাহ Staging-এ সফলভাবে চালিয়ে প্রোডাকশনে Automated Sync চালু করা। মাইগ্রেশনের সময় পুরোনো Push পাইপলাইন একবারে বন্ধ করবেন না: প্রথমে GitOps-কে **Read-Only** (Manual Sync, Prune বন্ধ) মোডে চালিয়ে দেখুন যে Diff শূন্য — অর্থাৎ Git-এর লেখা ও ক্লাস্টারের বাস্তবতা মেলে — তারপর ধীরে ধীরে নিয়ন্ত্রণ হস্তান্তর করুন।

১১.১৩ সাধারণ ভুল

- **Git-এ Secret লেখা:** সবচেয়ে বিপজ্জনক ভুল; Sealed Secrets, External Secrets বা SOPS ব্যবহার করুন।
- **`latest` ট্যাগ ব্যবহার:** কোন সংস্করণ চলছে, তা জানা যায় না; Rollback অনিশ্চিত হয়।
- **জরুরি পরিবর্তন Git-এ না লেখা:** হাতে ঠিক করা সমস্যা পরের Sync-এ মুছে যায় — অথবা Git-এর পুরোনো লেখা আবার ফিরে এসে সমস্যা ফিরিয়ে আনে।

- **সবকিছু একটিমাত্র বড় Application-এ রাখা:** একটি ভুল Sync সব অ্যাপকে প্রভাবিত করে; অ্যাপভিত্তিক Application আলাদা করুন।
- **Health Check ছাড়া Automated Sync:** Sync সফল মানেই অ্যাপ সুস্থ নয়; Health Assessment ও Smoke Test যুক্ত করুন।
- **Prune না বুঝে চালু করা:** ভুল ফাইল মুছে গেলে প্রোডাকশন রিসোর্স মুছে যেতে পারে; প্রথমে Prune বন্ধ রেখে শিখুন।

১১.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Application ক্রমাগত OutOfSync	হাতে করা পরিবর্তন; HPA রেপ্লিকা বদলাচ্ছে	Diff দেখুন; বৈধ ক্ষেত্রে Ignore Differences লিখুন
Sync হয়, কিন্তু Pod CrashLoopBackOff	ভুল ইমেজ ট্যাগ বা পরিবেশ চলক	লগ দেখুন; Git Revert দিয়ে আগের সংস্করণে ফিরুন
Controller Git-এ পৌঁছাতে পারছে না	Deploy Key মেয়াদোত্তীর্ণ; নেটওয়ার্ক নীতি	Credential পরীক্ষা করুন; Controller লগ দেখুন
পরিবর্তন মার্জ হলেও Sync হচ্ছে না	ভুল Branch/Path; Sync Window বন্ধ	Application সংজ্ঞা পরীক্ষা করুন; Window দেখুন
Rollback-এর পরও পুরোনো বাগ ফিরে এলো	ডেটাবেস Migration ফেরানো হয়নি	Migration-এর আলাদা Rollback পরিকল্পনা চালান

১১.১৫ প্রকৌশল ট্রেড-অফ: GitOps বনাম প্রচলিত CI/CD

GitOps সব জায়গায় সেরা নয়। এর শক্তি: নিরীক্ষাযোগ্যতা, Drift নিয়ন্ত্রণ, নিরাপত্তা (CI-এর কাছে ক্লাস্টারের চাবি নেই), এবং Rollback-এর সহজতা। এর দুর্বলতা: শেখার বক্ররেখা, রিপোজিটরি শৃঙ্খলার দাবি, এবং **Secret** পরিচালনার বাড়তি জটিলতা। যে পরিস্থিতিতে প্রচলিত Push CI/CD যথেষ্ট: একটিমাত্র পরিবেশ, দ্রুত পরিবর্তনশীল পরীক্ষামূলক কাজ, অথবা এমন প্ল্যাটফর্ম যেখানে Kubernetes নেই (যেমন সরল VM-ভিত্তিক ডেপ্লয়মেন্ট)। যে পরিস্থিতিতে GitOps প্রায় বাধ্যতামূলক: একাধিক পরিবেশ, একাধিক ক্লাস্টার, নিয়ন্ত্রক নিরীক্ষার প্রয়োজন (**Compliance**), অথবা বড় দল যেখানে কে কী ডেপ্লয় করল, তা প্রমাণ করতে হয়। TechNova-র জন্য — একাধিক পরিবেশ, ভবিষ্যতে একাধিক ক্লাস্টার — GitOps সঠিক বিনিয়োগ। তবে শুরুটা ছোট করুন: একটি অ্যাপ, একটি পরিবেশ, Manual Sync — তারপর বিস্তার করুন।

১১.১৬ অধ্যায়ের সারাংশ

GitOps হলো Git-কে সত্যের উৎস করে, ঘোষণামূলক কনফিগারেশন দিয়ে, ক্লাস্টারের ভেতরের Agent দিয়ে ক্রমাগত সমন্বয় করার অপারেশনাল মডেল। এর চার নীতি — ঘোষণামূলক বর্ণনা, Git-এ সংস্করণ-নিয়ন্ত্রণ, স্বয়ংক্রিয় প্রয়োগ ও ক্রমাগত Reconciliation — একসাথে Push মডেলের নিরাপত্তা-ঝুঁকি, ইতিহাসের অভাব ও Drift-এর সমস্যা দূর করে। আলাদা কনফিগারেশন রিপোজিটরি, পরিবেশভিত্তিক Overlay, Immutable ইমেজ ট্যাগ ও Git Revert-ভিত্তিক Rollback — এই চর্চাগুলোই GitOps-কে প্রোডাকশন-উপযোগী করে। পরের অধ্যায়ে আমরা এই মডেলের দুটি প্রধান বাস্তবায়ন — Argo CD ও Flux — হাতে-কলমে শিখব।

১১.১৭ জ্ঞান যাচাই

1. GitOps-এর চারটি মূল নীতি লিখুন এবং প্রতিটির একটি করে বাস্তব সুবিধা ব্যাখ্যা করুন।
2. Push ও Pull ডেপ্লয়মেন্ট মডেলের পার্থক্য কী? Pull মডেলে CI-এর কাছে ক্লাস্টারের Credential না থাকার নিরাপত্তা-সুবিধা কী?
3. Desired State ও Actual State-এর সংজ্ঞা দিন; Reconciliation Loop কীভাবে এদের ব্যবধান বন্ধ করে?
4. Drift কী? HPA-র কারণে হওয়া Drift কেন বৈধ, এবং তা Diff থেকে কীভাবে বাদ দেবেন?
5. অ্যাপ কোড ও কনফিগারেশন আলাদা রিপোজিটরিতে রাখার তিনটি যুক্তি দিন।
6. পরিবেশ প্রমোশনের তিনটি পদ্ধতি তুলনা করুন; TechNova-র জন্য কোনটি সুপারিশ করা হয়েছে ও কেন?
7. GitOps-এ Rollback কীভাবে কাজ করে? ডেটাবেস Migration-এর ক্ষেত্রে এর সীমা কী?
8. Git-এ Secret লেখা কেন বিপজ্জনক? Sealed Secrets, External Secrets ও SOPS-এর মধ্যে পার্থক্য সংক্ষেপে বলুন।
9. Prune চালু করার আগে কী কী ঝুঁকি বিবেচনা করবেন?
10. কোন পরিস্থিতিতে GitOps না করে প্রচলিত Push CI/CD ব্যবহার করাই যুক্তিসংগত? দুটি উদাহরণ দিন।

১১.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র জন্য একটি কনফিগারেশন রিপোজিটরির কাঠামো আঁকুন: `apps/web`-এর base ও তিনটি পরিবেশের Overlay, `environments/` ও `clusters/` ডিরেক্টরি-সহ। প্রতিটি ডিরেক্টরির উদ্দেশ্য এক লাইনে লিখুন।
2. একটি কাল্পনিক Drift দৃশ্য লিখুন: কেউ হাতে প্রোডাকশনে রেপ্লিকা ৩ থেকে ৫ করলেন। Self-Heal চালু ও বন্ধ — দুই অবস্থায় পরের এক ঘণ্টায় কী ঘটবে, তা ধাপে ধাপে বর্ণনা করুন।
3. একটি রিলিজের প্রমোশন পরিকল্পনা লিখুন: `web:v1.5.0` Dev থেকে Staging হয়ে Production-এ যাবে — প্রতিটি ধাপে Git-এ কী কমিট হবে, কে পর্যালোচনা করবে, এবং Rollback-এর শর্ত কী হবে, তা লিখুন।

অধ্যায় ১২ • PART III — GitOps & Platform Engineering

Chapter 12 — Argo CD and Flux CD

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Argo CD** ও **Flux CD**-এর আর্কিটেকচার তুলনা করে TechNova-র জন্য একটি বেছে নিতে; Argo CD-তে **Application**, **AppProject** ও **ApplicationSet** লিখতে; **Sync Policy**, **Health Assessment** ও **Automated Synchronization** কনফিগার করতে; Flux-এ **Source**, **Kustomization** ও **HelmRelease** লিখতে; Helm-কে GitOps-এর সাথে যুক্ত করতে; মাল্টি-ক্লাস্টার GitOps নকশা করতে; এবং দুটি টুলের সাধারণ ব্যর্থতা নির্ণয় ও সারাতে।

- Argo CD-র উপাদানগুলোর (API Server, Repo Server, Controller, Redis) ভূমিকা ব্যাখ্যা করা।
- Flux-এর উপাদানগুলোর (Source Controller, Kustomize Controller, Helm Controller) ভূমিকা ব্যাখ্যা করা।
- একটি সম্পূর্ণ Argo CD Application YAML ও Flux Kustomization YAML হাতে লেখা।
- Sync ব্যর্থ হলে লগ ও অবস্থা পড়ে কারণ নির্ণয় করা।
- মাল্টি-ক্লাস্টার সেটআপে কোন ক্লাস্টারে কী Sync হবে, তা নকশা করা।

১২.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

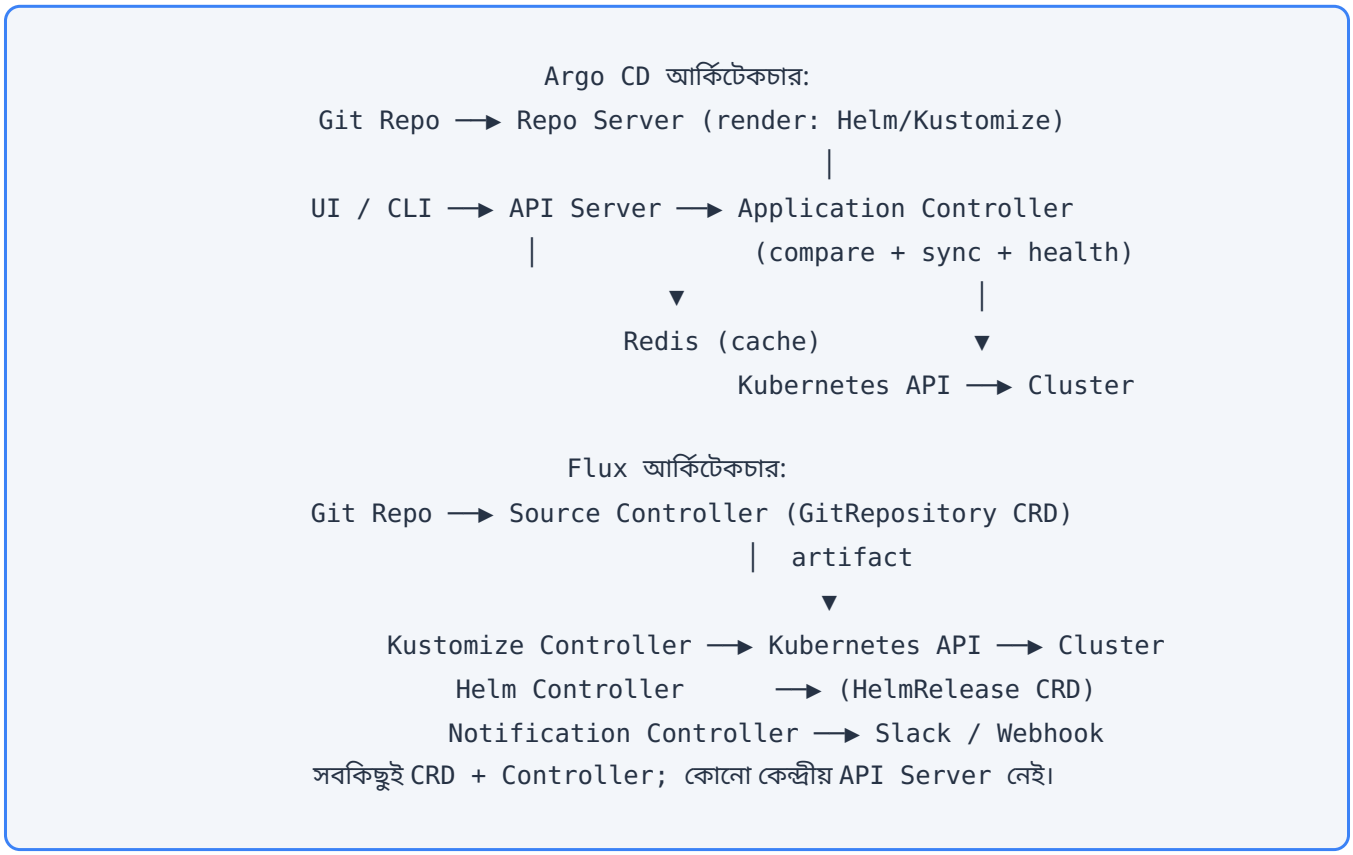
অধ্যায় ১১-এর GitOps নীতিগুলো — Desired State, Reconciliation, Drift, Rollback — এই অধ্যায়ের ভিত্তি। আরও দরকার Kubernetes **RBAC** (Role, RoleBinding, ServiceAccount), **Custom Resource Definition (CRD)**-এর ধারণা — অর্থাৎ Kubernetes-এ নতুন ধরনের অবজেক্ট সংজ্ঞায়িত করা যায় — এবং Helm Chart-এর প্রাথমিক ধারণা (অধ্যায় ১৩-এ বিস্তারিত আসবে, তবে এখানে `values.yaml` ও `helm install`-এর ধারণাটুকু জানলেই চলবে)। মনে রাখুন: Argo CD ও Flux দুটোই Kubernetes-এর ভেতরে চলে এবং Kubernetes API ব্যবহার করেই রিসোর্স বানায় — তারা Kubernetes-এর বাইরের কোনো জাদু নয়, বরং Kubernetes-এরই প্রসারণ (Extension)।

> **CONCEPT:** Argo CD ও Flux একই সমস্যার দুটি ভিন্ন দর্শন। Argo CD বলে: “একটি কেন্দ্রীয় নিয়ন্ত্রণ-তল (Control Plane) ও সমৃদ্ধ UI থাকুক, যেখানে সব Application-এর অবস্থা এক জায়গায় দেখা যায়।” Flux বলে: “সবকিছু Kubernetes-এর নিজস্ব ধাঁচে হোক — প্রতিটি জিনিস একটি CRD, প্রতিটি কাজ একটি Controller, আর কনফিগারেশন `kubectl` দিয়েই পরিচালিত হোক।” কোনোটিই সর্বত্র সেরা নয়; দলের অভ্যাসই সিদ্ধান্ত নেয়।

১২.৩ প্রথম নীতি: দুটি টুল, একটি চক্র — ভেতরে কী ঘটে

দুটি টুলেরই হৃদয়ে একই চক্র: Git দেখো → রেন্ডার করো → তুলনা করো → প্রয়োগ করো → স্বাস্থ্য পরীক্ষা করো। পার্থক্য বাস্তবায়নে। Argo CD-র **Application Controller** একটি কেন্দ্রীয় প্রক্রিয়া যা সব Application-এর Reconciliation চালায়; **Repo Server** Git থেকে ম্যানিফেস্ট এনে রেন্ডার করে (Helm, Kustomize); **API Server** UI ও CLI-কে সেবা দেয়; **Redis** ক্যাশ রাখে। Flux-এর ক্ষেত্রে কাজটি ভাগ করা: **Source Controller** Git বা Helm রিপোজিটরি থেকে আর্টিফ্যাক্ট আনে; **Kustomize Controller** সেই আর্টিফ্যাক্ট প্রয়োগ করে; **Helm Controller** Helm Release পরিচালনা করে; **Notification Controller** ঘটনার খবর পাঠায়। Flux-এর প্রতিটি Controller স্বাধীনভাবে স্কেল ও পুনরারম্ভ করা যায়

— এটি **Microservices** দর্শন; Argo CD-র মনোলিথিক নিয়ন্ত্রণ-তল পরিচালনায় সহজ — এটি **Monolith** দর্শন, তবে ভালো অর্থে।



১২.৪ Argo CD ইনস্টল ও প্রথম Application

Argo CD ইনস্টল করা হয় একটি Namespace-এ — সাধারণত `argocd` — এবং প্রথমেই অ্যাডমিন পাসওয়ার্ড বদলানো ও **Ingress** বা **Port-Forward** দিয়ে UI-তে ঢোকা হয়। প্রোডাকশনে UI-কে ইন্টারনেটে খোলা রাখবেন না; **SSO** (যেমন GitHub বা OIDC) চালু করুন এবং **RBAC** দিয়ে কে কী দেখতে ও করতে পারবে, তা সীমিত করুন। নিচের কমান্ডগুলো TechNova-র প্রোডাকশন ক্লাস্টারে (203.0.113.10) ইনস্টলের ধারণা দেয়।

```
# Argo CD ইনস্টল (Argo CD 2.x; অফিসিয়াল ম্যানিফেস্ট ব্যবহার করুন)
$ kubectl create namespace argocd
$ kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/

# ইনস্টল হয়েছে কি না, দেখুন
$ kubectl -n argocd get pods
NAME                                  READY   STATUS    AGE
argocd-application-controller-0       1/1     Running   2m
argocd-applicationset-controller-0    1/1     Running   2m
argocd-repo-server-0                  1/1     Running   2m
argocd-server-0                       1/1     Running   2m
argocd-redis-0                        1/1     Running   2m
```

```
# প্রথম লগইন (শুধু প্রাথমিক সেটআপের জন্য; তারপর পাসওয়ার্ড বদলান)
$ kubectl -n argocd get secret argocd-initial-admin-secret \
    -o jsonpath="{.data.password}" | base64 -d
$ kubectl -n argocd port-forward svc/argocd-server 8080:443
```

১২.৫ Argo CD Application: মূল একক

Argo CD-তে সবকিছুর মূল একক হলো **Application** — একটি CRD যা বলে: "এই Git রিপোজিটরির এই Path থেকে, এই ক্লাস্টারের এই Namespace-এ, এইভাবে Sync করো।" একটি Application একটি অ্যাপ বা একটি অ্যাপের একটি পরিবেশের সাথে মেলে — TechNova-তে `web-prod`, `web-staging`, `api-prod` — প্রতিটি আলাদা Application। নিচের YAML-টি একটি সম্পূর্ণ, বৈধ Application — লক্ষ করুন `syncPolicy`-তে **Automated**, **Prune** ও **Self-Heal** কীভাবে লেখা হয়।

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web-prod
  namespace: argocd
  labels:
    team: platform
    environment: prod
spec:
  project: technova-prod
  source:
    repoURL: https://github.com/technova/technova-gitops.git
    targetRevision: main
    path: apps/web/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - PruneLast=true
    retry:
      limit: 5
      backoff:
        duration: 5s
```

```
      factor: 2
      maxDuration: 3m
```

`targetRevision`-এ Branch-এর নাম (`main`) লেখা সাধারণ, তবে প্রোডাকশনে একটি নির্দিষ্ট Tag বা Commit SHA পিন করলে আরও নিরীক্ষাযোগ্য হয়। `PruneLast=true` মানে মুছে ফেলার কাজটি সবশেষে হবে — নতুন রিসোর্স আগে তৈরি হয়ে তারপর পুরোনো মুছবে, যাতে মাঝখানে সেবা বন্ধ না হয়। `retry` ব্লকটি বলে Sync ব্যর্থ হলে কতবার, কত ব্যবধানে আবার চেষ্টা করবে — **Exponential Backoff** (ব্যবধান দ্বিগুণ হওয়া) দিয়ে API-তে চাপ কমানো হয়।

১২.৬ AppProject: নিরাপত্তার সীমানা

AppProject হলো Application-গুলোর উপর নিরাপত্তা ও সাংগঠনিক সীমানা। প্রতিটি Project বলে: কোন Git রিপোজিটরি থেকে Source নেওয়া যাবে (`sourceRepos`), কোন ক্লাস্টার ও Namespace-এ Deploy করা যাবে (`destinations`), এবং কোন ধরনের রিসোর্স তৈরি করা যাবে (`clusterResourceWhitelist` / `namespaceResourceWhitelist`)। এটি **Multi-Tenancy** (একাধিক দল এক ক্লাস্টারে) পরিস্থিতিতে অপরিহার্য: ওয়েব দল শুধু `prod` Namespace-এ Deploy করতে পারবে, ক্লাস্টার-স্তরের রিসোর্স (যেমন `ClusterRole`) বানাতে পারবে না।

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: technova-prod
  namespace: argocd
spec:
  description: Production applications for TechNova
  sourceRepos:
    - https://github.com/technova/technova-gitops.git
  destinations:
    - server: https://kubernetes.default.svc
      namespace: prod
    - server: https://kubernetes.default.svc
      namespace: prod-*
  clusterResourceWhitelist:
    - group: ""
      kind: Namespace
  namespaceResourceBlacklist:
    - group: ""
      kind: ResourceQuota
  roles:
    - name: web-team
      description: Web team can sync web-prod
      policies:
        - p, role:web-team, applications, sync, technova-prod/web-prod, allow
```

```
      - p, role:web-team, applications, get, technova-prod/web-prod, allow
    groups:
      - technova-web-team
```

> **WARNING:** ডিফল্ট Project (`default`) সব রিপোজিটরি ও সব Destination-এ অনুমতি দেয়। প্রোডাকশনে কখনোই `default` Project ব্যবহার করবেন না — প্রতিটি পরিবেশের জন্য আলাদা Project বানান, এবং `default` Project-কে সীমিত করে রাখুন। একটি ভুল Application ভুল ক্লাস্টারে Sync হয়ে যাওয়া — এটি বাস্তবে ঘটে যাওয়া বিপর্যয়।

১২.৭ ApplicationSet: একই প্যাটার্ন, বহু পরিবেশ ও ক্লাস্টার

TechNova-র তিনটি পরিবেশ ও দুটি ক্লাস্টারের জন্য প্রতিটি Application হাতে লেখা ক্লান্তিকর ও ভুলপ্রবণ। **ApplicationSet** একটি টেমপ্লেট থেকে বহু Application তৈরি করে — **Generator** (যেমন Git ডিরেক্টরি, ক্লাস্টারের তালিকা, বা মানের তালিকা) থেকে মান নিয়ে। নিচের উদাহরণে Git Generator `apps/*/overlays/*` ডিরেক্টরিগুলো দেখে প্রতিটি অ্যাপ ও পরিবেশের জন্য Application বানায়।

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: technova-apps
  namespace: argocd
spec:
  generators:
    - git:
        repoURL: https://github.com/technova/technova-gitops.git
        revision: main
        directories:
          - path: apps/*/overlays/*
  template:
    metadata:
      name: "{{path.segments[1]}}-{{path.segments[3]}}"
    spec:
      project: technova-prod
      source:
        repoURL: https://github.com/technova/technova-gitops.git
        targetRevision: main
        path: "{{path}}"
      destination:
        server: https://kubernetes.default.svc
        namespace: "{{path.segments[3]}}"
```

```
    syncPolicy:
      automated:
        prune: true
        selfHeal: true
```

ApplicationSet-এর শক্তি হলো: নতুন অ্যাপ বা পরিবেশ যোগ করা মানে শুধু Git-এ একটি ডিরেক্টরি যোগ করা — Argo CD নিজেই Application বানিয়ে নেয়। দুর্বলতা: টেমপ্লেটে ভুল থাকলে সেই ভুল সব Application-এ ছড়িয়ে যায়; তাই টেমপ্লেট পরিবর্তন প্রথমে একটি পরিবেশে পরীক্ষা করুন।

১২.৮ Sync Policy, Health Assessment ও Automated Synchronization

Health Assessment হলো Argo CD-র প্রতিটি রিসোর্সের স্বাস্থ্য বোঝার ব্যবস্থা: Deployment-এর জন্য "সব রেপ্লিকা প্রস্তুত কি না", Service-এর জন্য "অস্তিত্ব আছে কি না", Ingress-এর জন্য "তৈরি হয়েছে কি না" — এবং কাস্টম রিসোর্সের জন্য **Lua** স্ক্রিপ্ট দিয়ে নিজের স্বাস্থ্য-নিয়ম লেখা যায় (`resource.customizations.health`)। Sync সফল হলেও Health **Degraded** থাকলে Application-এর সামগ্রিক অবস্থা সুস্থ নয় — এই পার্থক্যটি বোঝা জরুরি: Sync বলে "Git-এর লেখা প্রয়োগ হয়েছে", Health বলে "অ্যাপ আসলে চলছে কি না"।

Sync Wave দিয়ে রিসোর্স তৈরির ক্রম নিয়ন্ত্রণ করা যায়: `argocd.argoproj.io/sync-wave: "-1"` অ্যানোটেশনযুক্ত রিসোর্স (যেমন Namespace, Secret) আগে তৈরি হয়, তারপর Deployment, তারপর Job। ডেটাবেস Migration-এর মতো কাজে এটি অপরিহার্য: Migration Job-কে Sync Wave 0-তে, অ্যাপকে Wave 1-এ রাখুন। **Sync Hook** দিয়ে Sync-এর আগে বা পরে বিশেষ কাজ (যেমন PreSync Job) চালানো যায়। আর **Sync Window** দিয়ে সময় বেঁধে দিন: ব্যবসার ব্যস্ত সময়ে (যেমন TechNova-র সকাল ৯টা–১২টা) কোনো Automated Sync হবে না — জরুরি ক্ষেত্রে Manual Override-এর অনুমতি কাদের থাকবে, তা আগেই ঠিক করুন।

```
# Sync Wave ও Hook-এর উদাহরণ: Migration আগে, অ্যাপ পরে
apiVersion: batch/v1
kind: Job
metadata:
  name: web-migrate
  namespace: prod
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
    argocd.argoproj.io/sync-wave: "0"
spec:
  template:
    spec:
      containers:
        - name: migrate
          image: registry.technova.example/web:v1.5.0
          command: ["./migrate.sh"]
          env:
```

```
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: web-db-secret
                  key: url
      restartPolicy: Never
```

১২.৯ Flux আর্কিটেকচার: Source, Kustomization ও HelmRelease

Flux-এ সবকিছু শুরু হয় **GitRepository** নামের একটি CRD দিয়ে — এটি Source Controller-কে বলে কোন Git রিপোজিটরি, কোন Branch, কত ব্যবধানে দেখতে হবে। তারপর **Kustomization** CRD বলে সেই Source-এর কোন Path থেকে, কোন Namespace-এ, কীভাবে প্রয়োগ করতে হবে। Helm-এর জন্য আছে **HelmRepository** ও **HelmRelease**। প্রতিটি CRD-এর নিজস্ব Controller আছে, প্রতিটি নিজের Reconciliation চক্র চালায় — একটি ব্যর্থ হলে অন্যটি চলতে থাকে।

```
# Flux 2.x — Git Source
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: technova-gitops
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/technova/technova-gitops.git
  ref:
    branch: main
---
# Flux 2.x — Kustomization: Source থেকে Path প্রয়োগ করা
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: web-prod
  namespace: flux-system
spec:
  interval: 5m
  path: ./apps/web/overlays/prod
  prune: true
  sourceRef:
    kind: GitRepository
    name: technova-gitops
  targetNamespace: prod
```

```
  timeout: 2m
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: web
      namespace: prod
```

`interval: 5m` মানে প্রতি পাঁচ মিনিটে Reconciliation; `prune: true` মানে Git থেকে মুছে ফেলা রিসোর্স ক্লাস্টার থেকেও মুছবে; `healthChecks` মানে Sync-এর পর এই Deployment-এর স্বাস্থ্য পরীক্ষা করে তবে সফল ঘোষণা করা হবে। `timeout` ছাড়িয়ে গেলে Kustomization ব্যর্থ বলে চিহ্নিত হয় — দীর্ঘ Migration-এর জন্য এটি বাড়াতে হয়।

১২.১০ Flux-এ Helm ইন্টিগ্রেশন

Flux-এ Helm Chart ডেপ্লয় করা হয় **HelmRelease** দিয়ে — Chart-টি কোথা থেকে আসবে (**HelmRepository** বা Git), কোন Values দিয়ে রেন্ডার হবে, কত ব্যবধানে আপডেট চেক করা হবে, তা এই CRD-তেই লেখা থাকে। Helm Controller Chart ডাউনলোড করে, Values মার্জ করে, `helm upgrade`-এর সমতুল্য কাজ করে এবং Release-এর ইতিহাস রাখে — ফলে Helm-এর নিজস্ব Rollback-ও (`helm rollback`-এর সমতুল্য) Flux-এর মাধ্যমে করা যায়।

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: bitnami
  namespace: flux-system
spec:
  interval: 1h
  url: https://charts.bitnami.com/bitnami
---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: postgresql
  namespace: prod
spec:
  interval: 30m
  chartRef:
    kind: OCIRepository
    name: postgres-oci
  values:
    auth:
      username: technova_app
      password: CHANGE-ME-DB-PASSWORD
```

```
    primary:
      persistence:
        size: 20Gi
  install:
    remediation:
      retries: 3
  upgrade:
    remediation:
      retries: 3
      remediateLastFailure: true
```

> **WARNING:** উপরের উদাহরণে পাসওয়ার্ড সরাসরি Values-এ লেখা হয়েছে শুধু কাঠামো দেখানোর জন্য — বাস্তবে কখনোই তা করবেন না। Flux-এ Values-এর সংবেদনশীল অংশ আলাদা Secret থেকে নিন (`valuesFrom` দিয়ে Secret রেফারেন্স), অথবা SOPS দিয়ে এনক্রিপ্ট করুন। এই বইয়ের কোনো উদাহরণেই আসল পাসওয়ার্ড লিখবেন না — সবসময় `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করুন।

১২.১১ Argo CD বনাম Flux: কোনটি কখন

দিক	Argo CD	Flux CD
UI	সমৃদ্ধ Web UI, অ্যাপের গাছ-আকৃতির দৃশ্য, Diff দেখা যায়	অফিসিয়াল UI নেই; `kubectl` ও CLI-তেই কাজ
কনফিগারেশন ধাঁচ	নিজস্ব CRD (Application) + UI/CLI	সম্পূর্ণ Kubernetes-নেটিভ CRD
মাল্টি-ক্লাস্টার	কেন্দ্রীয় Argo CD থেকে বহু ক্লাস্টার পরিচালনা সহজ	প্রতি ক্লাস্টারে Flux ইনস্টল; কেন্দ্রীয় দৃশ্য কম
Helm/ Kustomize	দুটোই সমর্থন করে; Helm-এর জন্য অতিরিক্ত সেটআপ কম	HelmRelease CRD দিয়ে গভীর Helm ইন্টিগ্রেশন
শেখার বক্ররেখা	UI থাকায় নতুনদের জন্য সহজ	Kubernetes CRD-তে অভ্যস্তদের জন্য স্বাভাবিক
Secret পরিচালনা	প্লাগইন (Argo CD Vault Plugin) বা External Secrets	SOPS-এর সাথে গভীর ইন্টিগ্রেশন

TechNova-র জন্য সুপারিশ: দলে Kubernetes-এ নতুন সদস্য আছেন এবং একটি কেন্দ্রীয় দৃশ্য দরকার — তাই **Argo CD দিয়ে শুরু করুন**। পরে, যে দলগুলো সম্পূর্ণ Git-ভিত্তিক (GitOps-এর সাথে সাথে "UI-তে ক্লিক নয়" শৃঙ্খলা) কাজ করতে চায়, তাদের জন্য Flux-এর পাইলট চালান। দুটি টুল একই ক্লাস্টারে একসাথে চালানো যায়, তবে একই Application দুটি টুল দিয়ে পরিচালনা করবেন না — Reconciliation চক্র দুটি পরস্পরের সাথে লড়াই করবে।

১২.১২ মাল্টি-ক্লাস্টার GitOps

TechNova-র ভবিষ্যৎ: প্রোডাকশন ক্লাস্টার (203.0.113.10) ও একটি দ্বিতীয় ক্লাস্টার (203.0.113.11) — হয়তো আলাদা অঞ্চলে, বা Staging-এর জন্য। মাল্টি-ক্লাস্টার GitOps-এর দুটি প্যাটার্ন আছে। **Hub-and-Spoke (কেন্দ্রীয়):** একটি Management Cluster-এ Argo CD চলে, সেখান থেকে সব Spoke ক্লাস্টারে Deploy হয়; সুবিধা — এক জায়গায় সব

দেখা যায়; অসুবিধা — Management Cluster নষ্ট হলে সব Sync থেমে যায় (তবে চলমান অ্যাপ চলতেই থাকে, কারণ Sync থামা মানে অ্যাপ থামা নয়)। **Distributed (বিতরণ):** প্রতিটি ক্লাস্টারে নিজস্ব Argo CD বা Flux চলে, প্রতিটি নিজের Git Path দেখে; সুবিধা — একটি ক্লাস্টারের সমস্যা অন্যটিকে ছোঁয় না; অসুবিধা — কেন্দ্রীয় দৃশ্য থাকে না। TechNova-র জন্য শুরুতে Distributed প্যাটার্ন — প্রতিটি ক্লাস্টারে নিজস্ব Argo CD, একই Git রিপোজিটরির ভিন্ন Path — সহজ ও নিরাপদ। বড় হলে Hub-and-Spoke-এ যাওয়া যায়, ApplicationSet-এর **Cluster Generator** দিয়ে।

```
# মাল্টি-ক্লাস্টার: Argo CD-তে দ্বিতীয় ক্লাস্টার যোগ করা
$ argocd cluster add cluster-prod-02 \
    --name cluster-prod-02 \
    --server https://203.0.113.11:6443

$ argocd cluster list
SERVER                          NAME              VERSION  STATUS
https://kubernetes.default.svc  in-cluster        1.30+    Successful
https://203.0.113.11:6443       cluster-prod-02   1.30+    Successful

# Application-এর destination-এ সেই ক্লাস্টারের নাম ব্যবহার করুন:
#   destination:
#     server: https://203.0.113.11:6443
#     namespace: prod
```

১২.১৩ Secret পরিচালনা: Sealed Secrets, External Secrets ও SOPS

GitOps-এর সবচেয়ে কঠিন সমস্যা Secret। তিনটি প্রতিষ্ঠিত সমাধান: **Sealed Secrets** — Secret-কে ক্লাস্টারের Public Key দিয়ে এনক্রিপ্ট করে Git-এ রাখা যায়; ক্লাস্টারের ভেতরে Controller তা Decrypt করে আসল Secret বানায়। সুবিধা: Git-এই সবকিছু থাকে; অসুবিধা: ক্লাস্টার বদলালে আবার Sealed করতে হয়। **External Secrets Operator** — Git-এ শুধু রেফারেন্স থাকে (“এই Secret-টি Vault থেকে নাও”), Controller বাহ্যিক Vault থেকে এনে Kubernetes Secret বানায়। সুবিধা: Secret কখনো Git-এ যায় না; অসুবিধা: Vault-এর উপর নির্ভরতা। **SOPS** — পুরো YAML ফাইল Age বা GPG দিয়ে এনক্রিপ্ট করে Git-এ রাখা; Flux-এর সাথে গভীরভাবে যুক্ত। TechNova-র জন্য সুপারিশ: বাহ্যিক Vault না থাকলে Sealed Secrets দিয়ে শুরু করুন, Vault এলে External Secrets-এ যান।

```
# Sealed Secrets — ব্যবহারের ধারণা (কমান্ডগুলো ধারণাগত)
$ kubectl create secret generic web-db-secret \
    --from-literal=password=CHANGE-ME-DB-PASSWORD \
    --dry-run=client -o yaml | kubeseal -o yaml > sealed-web-db.yaml

# sealed-web-db.yaml-এ এখন শুধু এনক্রিপ্ট করা মান আছে — এটি Git-এ রাখা নিরাপদ
# External Secrets Operator — Git-এ শুধু রেফারেন্স
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: web-db-secret
```

```
  namespace: prod
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: technova-vault
    kind: ClusterSecretStore
  target:
    name: web-db-secret
  data:
    - secretKey: password
      remoteRef:
        key: prod/web/db
        property: password
```

১২.১৪ TechNova দৃশ্যকল্প: Argo CD দিয়ে প্রথম প্রোডাকশন Sync

TechNova-র ওয়েব অ্যাপ প্রোডাকশনে যাওয়ার দিন: সকাল ১০টায় `web-prod` Application তৈরি করা হলো Manual Sync-এ, Prune বন্ধ রেখে। প্রথম Sync-এ Diff দেখা গেল — ক্লাস্টারে একটি পুরোনো ConfigMap ছিল যা Git-এ নেই; তদন্তে দেখা গেল সেটি হাতে বানানো হয়েছিল মাসখানেক আগে। সিদ্ধান্ত: ConfigMap-টি Git-এ তুলে তারপর Sync করা হলো — হাতে করা জিনিস মুছে ফেলার বদলে Git-এ ধারণ করা হলো। দুপুরে Health সবুজ হলো, বিকেলে Automated Sync চালু করা হলো Self-Heal-সহ। সন্ধ্যায় ডেভেলপার একটি Pull Request মার্জ করলেন — তিন মিনিট পর Argo CD নিজেই নতুন সংস্করণ ডেপ্লয় করল, কোনো মানুষের হস্তক্ষেপ ছাড়াই। সেই রাতে দলের Slack-এ প্রথমবারের মতো ডেপ্লয়মেন্টের খবর এলো বটের মাধ্যমে, মানুষের মাধ্যমে নয়।

১২.১৫ সাধারণ ভুল

- **একই Application দুটি টুল দিয়ে পরিচালনা:** Argo CD ও Flux পরস্পরের পরিবর্তন মুছে ফেলার চেষ্টা করবে।
- **`default` Project ব্যবহার:** কোনো সীমানা থাকে না; ভুল ক্লাস্টারে Deploy হওয়ার ঝুঁকি।
- **Health না দেখে শুধু Sync দেখা:** Sync সফল মানেই অ্যাপ সুস্থ নয়।
- **Secret সরাসরি Values-এ লেখা:** Helm Values Git-এ যায়; সংবেদনশীল মান আলাদা রাখুন।
- **Sync Window না বোঝা:** ব্যস্ত সময়ে Automated Sync চালু রেখে দুর্ঘটনা ঘটানো।
- **ApplicationSet টেমপ্লেট পরীক্ষা না করে প্রয়োগ:** একটি ভুল সব Application-এ ছড়িয়ে যায়।

১২.১৬ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Argo CD: “ComparisonError”	Git-এ পৌঁছানো যাচ্ছে না; ভুল Repo URL	Repo Server লগ দেখুন; Credential পরীক্ষা করুন
Flux: Kustomization “BuildFailed”	YAML সিনট্যাক্স ভুল; Kustomize পথ ভুল	`flux build kustomization` দিয়ে লোকালে রেন্ডার করুন

Sync হয়, কিন্তু Pod পুরোনো ইমেজেই আছে	ইমেজ ট্যাগ `latest`; ImagePullPolicy ভুল	Immutable ট্যাগ ব্যবহার করুন; Digest পিন করুন
HelmRelease “UpgradeFailed”	Values ভুল; Chart সংস্করণ অসামঞ্জস্য	`helm template` দিয়ে রেন্ডার পরীক্ষা করুন
মাল্টি-ক্লাস্টারে একটি ক্লাস্টারে Sync হচ্ছে না	ক্লাস্টার Credential মেয়াদোত্তীর্ণ	`argocd cluster list` দিয়ে অবস্থা দেখুন; আবার যোগ করুন
Sealed Secret Decrypt হচ্ছে না	অন্য ক্লাস্টারের Key দিয়ে Sealed করা	সঠিক ক্লাস্টারের Public Key দিয়ে আবার Sealed করুন

```
# Flux সমস্যা নির্ণয়ের প্রধান কমান্ডগুলো
$ flux check
$ flux get kustomizations --watch
$ flux logs --kind=Kustomization --name=web-prod -n flux-system
$ flux diff kustomization web-prod   # কী বদলাবে, প্রয়োগের আগেই দেখুন
$ flux reconcile kustomization web-prod --with-source  # এখনই Sync ট্রিগার
```

১২.১৭ প্রকৌশল ট্রেড-অফ

Argo CD বনাম Flux-এর সিদ্ধান্তটি আসলে দলের সংস্কৃতির সিদ্ধান্ত। UI-নির্ভর দল, যেখানে অপারেটররা Diff দেখে অনুমোদন দিতে চান, তাদের জন্য Argo CD। সম্পূর্ণ Git-ভিত্তিক, CLI-নির্ভর দল, যেখানে সবকিছু CRD দিয়ে পরিচালিত হয়, তাদের জন্য Flux। দুটি টুলই CNCF-এর **Graduated** প্রকল্প — দীর্ঘমেয়াদে কোনোটিই হারিয়ে যাওয়ার ঝুঁকিতে নেই। আরেকটি ট্রেড-অফ: Automated Sync বনাম Manual Sync। Automated দ্রুত, কিন্তু ভুল দ্রুত ছড়িয়ে পড়ে; Manual নিরাপদ, কিন্তু মানুষের অপেক্ষায় গতি কমে। TechNova-র নিয়ম: Dev ও Staging-এ Automated, Production-এ শুরুতে Manual, আস্থা বাড়লে Automated + Sync Window + Health Check-এর সমন্বয়।

১২.১৮ অধ্যায়ের সারাংশ

Argo CD একটি কেন্দ্রীয় নিয়ন্ত্রণ-তল ও সমৃদ্ধ UI দিয়ে GitOps বাস্তবায়ন করে — Application, AppProject ও ApplicationSet তার মূল একক; Flux সম্পূর্ণ Kubernetes-নেটিভ CRD দিয়ে একই কাজ করে — GitRepository, Kustomization ও HelmRelease তার মূল একক। দুটোরই হৃদয়ে একই Reconciliation চক্র, একই Sync Policy, Health Assessment ও Prune ধারণা। Secret-এর জন্য Sealed Secrets, External Secrets বা SOPS — যেকোনো একটি বেছে নিন, কিন্তু কখনোই সরাসরি Git-এ Secret লিখবেন না। মাল্টি-ক্লাস্টারে শুরুতে Distributed প্যাটার্ন নিন, আর টুল বেছে নিন দলের অভ্যাস অনুযায়ী — TechNova-র জন্য Argo CD দিয়ে শুরু। পরের অধ্যায়ে আমরা Helm ও Kustomize দিয়ে রিলিজ প্রকৌশল শিখব — GitOps-এর ভেতরে প্যাকেজিংয়ের স্তরটি।

১২.১৯ জ্ঞান যাচাই

1. Argo CD-র চারটি প্রধান উপাদানের নাম ও ভূমিকা লিখুন।
2. Flux-এর Source Controller, Kustomize Controller ও Helm Controller-এর ভূমিকা ব্যাখ্যা করুন।
3. Application YAML-এ `syncPolicy.automated.prune` ও `selfHeal`-এর অর্থ কী? কখন এগুলো বন্ধ রাখা উচিত?

4. AppProject কেন প্রয়োজন? `default` Project ব্যবহারের ঝুঁকি কী?
5. ApplicationSet-এর Git Generator কীভাবে কাজ করে? এর একটি ঝুঁকি লিখুন।
6. Sync সফল কিন্তু Health Degraded — এই অবস্থার অর্থ কী? উদাহরণ দিন।
7. Sync Wave ও Sync Hook-এর পার্থক্য কী? Migration Job-এর জন্য কোনটি ব্যবহার করবেন ও কেন?
8. Flux Kustomization YAML-এ `prune`, `healthChecks` ও `timeout`-এর ভূমিকা লিখুন।
9. Sealed Secrets, External Secrets Operator ও SOPS-এর মধ্যে পার্থক্য ও ব্যবহারের পরিস্থিতি তুলনা করুন।
10. মাল্টি-ক্লাস্টার GitOps-এ Hub-and-Spoke বনাম Distributed প্যাটার্নের ট্রেড-অফ ব্যাখ্যা করুন।

১২.২০ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র `api-prod`-এর জন্য একটি সম্পূর্ণ Argo CD Application YAML লিখুন: Project `technova-prod`, Path `apps/api/overlays/prod`, Automated Sync, Prune ও Self-Heal চালু, Retry-সহ। তারপর একই অ্যাপের জন্য Flux GitRepository + Kustomization YAML লিখুন।
2. একটি AppProject YAML লিখুন যেখানে ওয়েব দল শুধু `prod` ও `staging` Namespace-এ Deploy করতে পারবে, কিন্তু কোনো ClusterRole বানাতে পারবে না।
3. একটি Sync ব্যর্থতার দৃশ্য বানান: HelmRelease “UpgradeFailed” দেখাচ্ছে। `flux logs`, `helm template` ও `flux diff` দিয়ে ধাপে ধাপে কীভাবে কারণ নির্ণয় করবেন, তা লিখুন।

অধ্যায় ১৩ • PART III — GitOps & Platform Engineering

Chapter 13 — Helm, Kustomize and Release Engineering

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Helm Chart** তৈরি করতে (Template, Values, Dependency); **Kustomize Overlay** দিয়ে পরিবেশভিত্তিক কনফিগারেশন পরিচালনা করতে; Helm ও Kustomize-এর মধ্যে সঠিকভাবে বেছে নিতে; **Release Versioning** ও **Rollback**-এর কৌশল প্রয়োগ করতে; Chart পরীক্ষা (**Chart Testing**) ও **Release Governance** (অনুমোদন, নীতি, নিরীক্ষা) বাস্তবায়ন করতে; এবং TechNova-র রিলিজ প্রক্রিয়াকে পুনরাবৃত্তিযোগ্য ও নিরীক্ষাযোগ্য করতে।

- একটি সম্পূর্ণ Helm Chart হাতে লেখা ও ইনস্টল করা।
- Kustomize-এ Base, Overlay, Patch ও Transformer ব্যবহার করা।
- Helm Values-এর স্তরবিন্যাস (Layering) ব্যাখ্যা করা।
- Semantic Versioning অনুযায়ী Chart সংস্করণ নির্ধারণ করা।
- Chart-এর জন্য স্বয়ংক্রিয় পরীক্ষা (Lint, Template, Unit) লেখা।

১৩.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

আপনার Kubernetes ম্যানিফেস্ট (Deployment, Service, Ingress, ConfigMap) লেখার অভিজ্ঞতা, YAML-এর মৌলিক ধারণা, এবং অধ্যায় ১১–১২-এর GitOps ধারণা — Desired State, Reconciliation, Argo CD Application, Flux Kustomization — জানা থাকা দরকার। **Go Template**-এর ধারণা (যেমন `{{ .Values.replicas }}`) না জানলেও চলবে; এই অধ্যায়েই তা শেখানো হবে। মনে রাখুন: Helm ও Kustomize দুটোই একই সমস্যার সমাধান — “একই অ্যাপ, ভিন্ন পরিবেশে ভিন্ন কনফিগারেশন” — কিন্তু ভিন্ন পদ্ধতিতে।

> **CONCEPT:** Helm হলো প্যাকেজ ম্যানেজার — যেমন Linux-এ apt বা Node.js-এ npm; এটি অ্যাপকে একটি প্যাকেজ (Chart) হিসেবে বানায়, সংস্করণ দেয়, নির্ভরতা (Dependency) সামলায় ও বিতরণ করে। Kustomize হলো কনফিগারেশন কাস্টমাইজার — এটি কোনো টেমপ্লেট ভাষা ছাড়াই, শুধু YAML-এর উপর Patch ও Overlay লাগিয়ে, একই Base থেকে ভিন্ন পরিবেশের সংস্করণ বানায়। Helm “বানায়”, Kustomize “বদলায়” — এই পার্থক্যটি মনে রাখুন।

১৩.৩ প্রথম নীতি: কেন টেমপ্লেটিং দরকার

TechNova-র ওয়েব অ্যাপের Deployment YAML-এ রেপ্লিকা সংখ্যা ৩, ইমেজ `web:v1.5.0`, মেমরি সীমা ৫১২ Mi। Dev-এ চাই রেপ্লিকা ১, মেমরি ২৫৬ Mi; Staging-এ রেপ্লিকা ২; Production-এ রেপ্লিকা ৩। একই YAML তিনবার কপি করে প্রতিটিতে হাতে বদলানো যায় — কিন্তু তখন তিনটি কপি ধীরে ধীরে আলাদা হয়ে যায়: কেউ Production-এ একটি পরিবেশ চলক যোগ করলেন, Dev-এ যোগ করতে ভুলে গেলেন। এই সমস্যার নাম **Configuration Drift** — অধ্যায় ১১-এর Drift-এরই একটি রূপ, তবে Git-এর ভেতরে। সমাধান: একটি **Single Source of Truth** (একটি Base সংজ্ঞা) + পরিবেশভিত্তিক

পার্থক্য (Values বা Overlay) — পার্থক্য ছোট, স্পষ্ট ও পর্যালোচনাযোগ্য থাকবে। Helm সেই পার্থক্যকে Values ফাইলে রাখে; Kustomize Overlay ডিরেক্টরিতে রাখে।

১৩.৪ Helm: Chart-এর শারীরস্থান

একটি Helm Chart হলো একটি ডিরেক্টরি, যার ভেতরে `Chart.yaml` (Chart-এর নাম, সংস্করণ, বর্ণনা), `values.yaml` (ডিফল্ট মান), `templates/` (Go Template-সহ Kubernetes ম্যানিফেস্ট), এবং ঐচ্ছিকভাবে `charts/` (নির্ভরশীল Chart) থাকে। **Helm 3**-এ আর **Tiller** নামের সার্ভার-উপাদান নেই — সবকিছু ক্লায়েন্ট (Helm CLI) থেকে Kubernetes API-তে সরাসরি যায়; এটি Helm 2-এর নিরাপত্তা-সমস্যার সমাধান। Chart ইনস্টল করলে Helm Template রেন্ডার করে, Kubernetes-এ প্রয়োগ করে এবং **Release** নামে একটি রেকর্ড রাখে (Secret হিসেবে, ক্লাস্টারের ভেতরে) — যাতে পরে `helm upgrade` ও `helm rollback` করা যায়।

```
# TechNova web Chart — কাঠামো
charts/web/
├── Chart.yaml
├── values.yaml
├── values-prod.yaml
├── values-dev.yaml
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    └── _helpers.tpl

# Chart.yaml
apiVersion: v2
type: application
name: web
description: TechNova web application
version: 1.5.0
appVersion: "1.5.0"

# values.yaml (ডিফল্ট মান)
replicaCount: 2
image:
  repository: registry.technova.example/web
  tag: "1.5.0"
  pullPolicy: IfNotPresent
resources:
  limits:
    cpu: 500m
```

```
    memory: 512Mi
  requests:
    cpu: 100m
    memory: 128Mi
ingress:
  enabled: true
  host: web.technova.example
```

১৩.৫ Helm Template: Go Template দিয়ে ম্যানিফেস্ট বানানো

Template ফাইলে Go Template সিনট্যাক্স ব্যবহার করা হয়: `{{ .Values.replicaCount }}` মানে Values ফাইলের `replicaCount` মানটি এখানে বসাও; `{{ .Release.Name }}` মানে Release-এর নাম; `{{- if .Values.ingress.enabled }}` মানে শর্তসাপেক্ষে একটি অংশ রেন্ডার করো। **Helper Template** (`_helpers.tpl`) দিয়ে পুনরাবৃত্ত অংশ — যেমন লেবেলের সেট — এক জায়গায় লেখা যায়। Template লেখার সময় সবচেয়ে বড় ভুল হলো Template-কে প্রোগ্রামিং ভাষা বানিয়ে ফেলা: জটিল যুক্তি Template-এ নয়, Values-এ রাখুন; Template শুধু মান বসানোর কাজ করুক।

```
# templates/deployment.yaml — Helm Template
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-web
  labels:
    app: web
    version: {{ .Chart.AppVersion | quote }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 3000
```

```
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
          env:
            - name: NODE_ENV
              value: {{ .Values.environment | default "production" | quote }}
```

```
# রেন্ডার পরীক্ষা (ক্লাস্টারে না পাঠিয়ে)
$ helm template web-prod ./charts/web -f ./charts/web/values-prod.yaml
$ helm lint ./charts/web
==> Linting ./charts/web
[INFO] Chart.yaml: icon is recommended
1 chart(s) linted, 0 chart(s) failed
```

`helm template` কমান্ডটি অমূল্য: এটি ক্লাস্টারে কিছু না পাঠিয়ে দেখিয়ে দেয় শেষ YAML কেমন হবে — GitOps-এর Diff দেখার মতোই, তবে Helm স্তরে। প্রতিটি Pull Request-এ `helm template` ও `helm lint` চালান; ভুল Template প্রোডাকশনে যাওয়ার আগেই ধরা পড়বে।

১৩.৬ Helm Values-এর স্তরবিন্যাস ও Dependency

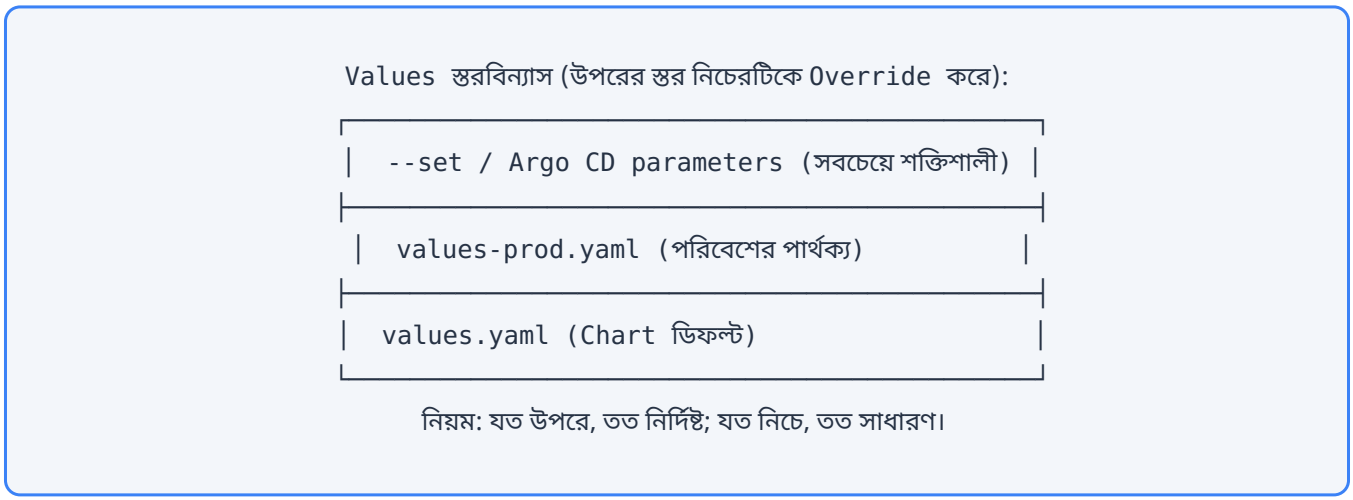
Values আসে স্তরে স্তরে, পরের স্তর আগেরটিকে **Override** করে: (১) Chart-এর `values.yaml` (ডিফল্ট); (২) পরিবেশের Values ফাইল (`values-prod.yaml`, `-f` দিয়ে দেওয়া); (৩) কমান্ড লাইনের `--set` মান; (৪) Argo CD বা Flux-এর Application-এ লেখা Values। নিয়ম: ডিফল্ট মান `values.yaml`-এ, পরিবেশের পার্থক্য পরিবেশের ফাইলে, সংবেদনশীল মান Secret-এ — কখনোই `--set password=...` দিয়ে কমান্ড লাইনে পাসওয়ার্ড দেবেন না (তা শেল ইতিহাসে থেকে যায়)। **Dependency** হলো Chart-এর ভেতরে আরেকটি Chart ব্যবহার করা — যেমন TechNova-র web Chart PostgreSQL Chart-এর উপর নির্ভর করে; `Chart.yaml`-এর `dependencies` অংশে তা ঘোষণা করা হয়, `helm dependency update` দিয়ে ডাউনলোড করা হয়।

```
# values-prod.yaml — শুধু পার্থক্য লিখুন, সবকিছু নয়
replicaCount: 3
image:
  tag: "1.5.0"
resources:
  limits:
    cpu: 1000m
    memory: 1Gi
environment: production

# Chart.yaml-এ Dependency ঘোষণা
dependencies:
  - name: postgresql
    version: "15.x"
    repository: "oci://registry-1.docker.io/bitnamicharts"
```

```
    condition: postgresql.enabled
    tags:
      - database

$ helm dependency update ./charts/web
$ helm install web-prod ./charts/web -f ./charts/web/values-prod.yaml \
    --namespace prod --create-namespace
$ helm list -n prod
NAME       NAMESPACE  REVISION  STATUS    CHART      APP VERSION
web-prod   prod       1         deployed  web-1.5.0  1.5.0
```



১৩.৭ Kustomize: টেমপ্লেট ছাড়াই কাস্টমাইজেশন

Kustomize Kubernetes-এরই অংশ (`kubectl apply -k` দিয়ে চলে) এবং কোনো টেমপ্লেট ভাষা ব্যবহার করে না। এর মূল ধারণা: একটি **Base** — সম্পূর্ণ, বৈধ Kubernetes ম্যানিফেস্ট — আর তার উপর **Overlay** — যা Base-কে বদলায়। Overlay-তে থাকে **Patch** (একটি ক্ষেত্রের মান বদলানো), **ConfigMap Generator** (ConfigMap বানানো), **Image Transformer** (ইমেজ ট্যাগ বদলানো), **Replica Transformer**, **Namespace Transformer** ও **Common Labels/Annotations**। Kustomize-এর সুবিধা: Base-টি সবসময় বৈধ YAML — `kubectl apply` দিয়েই পরীক্ষা করা যায়; শেখার বক্ররেখা Helm-এর চেয়ে কম; Flux ও Argo CD দুটোতেই নেটিভ সমর্থন আছে। অসুবিধা: জটিল শর্তসাপেক্ষ যুক্তি (if/else) লেখা কঠিন; Chart-এর মতো প্যাকেজিং ও সংস্করণ-ব্যবস্থা নেই।

```
# Kustomize — Base ও Overlay কাঠামো
apps/web/
├── base/
│   ├── deployment.yaml      # replicas: 2, image: web:1.5.0
│   ├── service.yaml
│   └── kustomization.yaml
└── overlays/
    └── prod/
        ├── kustomization.yaml
```

```
        └── replica-patch.yaml

# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
commonLabels:
  app: web
  team: platform

# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: prod
resources:
  - ../../base
images:
  - name: registry.technova.example/web
    newTag: "1.5.0"
replicas:
  - name: web
    count: 3
configMapGenerator:
  - name: web-config
    literals:
      - NODE_ENV=production
      - API_URL=https://api.technova.example

$ kubectl kustomize apps/web/overlays/prod   # রেন্ডার দেখুন
$ kubectl apply -k apps/web/overlays/prod     # প্রয়োগ করুন
```

১৩.৮ Helm বনাম Kustomize: কোনটি কখন

দিক	Helm	Kustomize
মূল কাজ	প্যাকেজিং, সংস্করণ, বিতরণ	পরিবেশভিত্তিক কাস্টমাইজেশন
টেমপ্লেট ভাষা	Go Template (শিখতে হয়)	নেই — শুধু YAML
তৃতীয় পক্ষের অ্যাপ	Chart বিতরণের মানদণ্ড; সহজে ইনস্টল	Base হিসেবে নিয়ে Patch করা যায়

সংস্করণ ও Rollback	Release ইতিহাস; `helm rollback`	Git ইতিহাসই Rollback; নিজস্ব Release নেই
জটিল যুক্তি	শর্ত, লুপ, ফাংশন সম্ভব	সীমিত; Patch দিয়ে সামলাতে হয়
GitOps সমর্থন	Argo CD ও Flux দুটোতেই আছে	দুটোতেই নেটিভ সমর্থন

বাস্তবে দুটো একসাথে ব্যবহৃত হয়: Helm Chart দিয়ে তৃতীয় পক্ষের অ্যাপ (যেমন PostgreSQL, Ingress Controller) ইনস্টল করা, আর নিজের অ্যাপের পরিবেশভিত্তিক কনফিগারেশনে Kustomize ব্যবহার করা — অথবা Helm Chart-এর উপর Kustomize Overlay লাগানো (Argo CD-তে এটি সমর্থিত: Helm Chart রেন্ডার করে তারপর Kustomize Patch প্রয়োগ করা যায়)। TechNova-র নিয়ম: নিজের অ্যাপে Kustomize, তৃতীয় পক্ষের অ্যাপে Helm — এবং যে অ্যাপ অন্য দলের সাথে ভাগ করতে হয়, তার জন্য Helm Chart বানানো।

> **REAL WORLD:** TechNova-র API দল প্রথমে সবকিছু Helm Chart-এ লিখেছিল — প্রতিটি পরিবেশের জন্য আলাদা Values ফাইল। কিছুদিন পর দেখা গেল Values ফাইলগুলো এত বড় হয়ে গেছে যে আসল পার্থক্যটি খুঁজে পাওয়া যায় না। তারা Base ম্যানিফেস্ট Kustomize-এ নিয়ে এলো, আর শুধু তৃতীয় পক্ষের নির্ভরতাগুলো Helm-এ রাখল। ফল: পরিবেশের পার্থক্য এখন এক পর্দায় দেখা যায় — তিনটি Overlay ফাইল, প্রতিটি দশ লাইনের কম।

১৩.৯ Release Versioning: Semantic Versioning ও Chart সংস্করণ

Semantic Versioning (SemVer) — `MAJOR.MINOR.PATCH` (যেমন ১.৫.০) — Chart ও অ্যাপ দুটোরই সংস্করণের ভাষা। নিয়ম: **MAJOR** বাড়ে যখন পেছন-অসামঞ্জস্যপূর্ণ (Breaking) পরিবর্তন হয়; **MINOR** বাড়ে যখন নতুন ফিচার যোগ হয় কিন্তু পুরোনো কাজ ভাঙে না; **PATCH** বাড়ে যখন শুধু বাগ ঠিক হয়। Chart-এর `version` (Chart-এর নিজের সংস্করণ) ও `appVersion` (অ্যাপের সংস্করণ) আলাদা রাখুন: Chart-এর Template বদলালে `version` বাড়ে, অ্যাপের ইমেজ বদলালে `appVersion` বাড়ে — দুটো একসাথে গুলিয়ে ফেললে কোন পরিবর্তনটি কী বদলেছে, তা বোঝা যায় না। Chart প্রকাশ করুন একটি **Chart Repository**-তে (যেমন OCI রেজিস্ট্রি — একই রেজিস্ট্রি যেখানে ইমেজ থাকে) যাতে Argo CD বা Flux সেখান থেকে Chart টানতে পারে।

```
# Chart প্যাকেজ ও প্রকাশ (OCI রেজিস্ট্রি)
$ helm package ./charts/web
Successfully packaged chart and saved it to: ./web-1.5.0.tgz

$ helm push web-1.5.0.tgz oci://registry.technova.example/charts
Pushed: registry.technova.example/charts/web:1.5.0

# সংস্করণের নিয়ম — উদাহরণ
# 1.4.0 → 1.4.1 : শুধু বাগ ঠিক (PATCH)
# 1.4.1 → 1.5.0 : নতুন ফিচার, পুরোনো কাজ ভাঙে না (MINOR)
# 1.5.0 → 2.0.0 : Breaking পরিবর্তন — যেমন values.yaml-এর ক্ষেত্রের নাম বদল
```

১৩.১০ Rollback: Helm, Kustomize ও GitOps-এ ফেরা

Helm-এ Rollback সহজ: `helm rollback web-prod 1` — আগের Revision-এ ফেরা যায়, কারণ Helm প্রতিটি Revision-এর সম্পূর্ণ অবস্থা Secret হিসেবে সংরক্ষণ করে। তবে সতর্কতা: Helm Rollback শুধু Kubernetes রিসোর্স

ফেরায় — ডেটাবেস Migration ফেরায় না, এবং বাহ্যিক রিসোর্স (যেমন ক্লাউড লোড ব্যালান্সার) ফেরায় না। Kustomize-এ নিজস্ব Rollback নেই — Rollback মানে Git-এ Revert (অধ্যায় ১১)। GitOps পরিবেশে সুপারিশ: **Helm Rollback নয়, Git Revert ব্যবহার করুন** — কারণ Git Revert-ই সত্যের উৎসকে ফেরায়, আর Controller তা প্রয়োগ করে; Helm দিয়ে হাতে Rollback করলে Git ও ক্লাস্টার আবার অমিল হয়ে যায় (Drift)। ব্যতিক্রম: GitOps Controller নিজেই নষ্ট হলে, জরুরি অবস্থায় `helm rollback` ব্যবহার করা যায় — তবে তারপর Git-কেও একই অবস্থায় ফিরিয়ে আনতে হবে।

```
# Helm Rollback — শুধু জরুরি অবস্থায়, GitOps-এর বাইরে
$ helm history web-prod -n prod
REVISION  UPDATED                   STATUS      CHART      APP VERSION
1         2026-11-08 10:00:00       superseded  web-1.4.2  1.4.2
2         2026-11-09 09:30:00       deployed    web-1.5.0  1.5.0

$ helm rollback web-prod 1 -n prod
Rollback was a success! Happy Helming!

# GitOps-এ সঠিক পদ্ধতি (সুপারিশকৃত):
#   git revert <খারাপ কমিট> && git push
#   Argo CD / Flux নিজেই আগের অবস্থায় ফিরিয়ে নেবে।
```

১৩.১১ Chart Testing ও Release Governance

Chart Testing মানে Chart-কে প্রোডাকশনে যাওয়ার আগে স্বয়ংক্রিয়ভাবে পরীক্ষা করা। স্তরগুলো: (১) **Lint** — `helm lint` দিয়ে সিনট্যাক্স ও সেরা চর্চা পরীক্ষা; (২) **Template Test** — `helm template` দিয়ে রেন্ডার করা YAML-কে `kubeconform` বা `kubeval` দিয়ে Kubernetes Schema-র বিরুদ্ধে যাচাই; (৩) **Unit Test** — `helm-unittest` প্লাগইন দিয়ে Template-এর প্রত্যাশিত আউটপুট পরীক্ষা (যেমন "Prod-এ রেপ্লিকা ৩ হবে"); (৪) **Integration Test** — **ct (chart-testing)** টুল দিয়ে Chart-কে একটি অস্থায়ী ক্লাস্টারে (**kind** বা **k3d**) ইনস্টল করে স্বাস্থ্য পরীক্ষা। **Release Governance** হলো এর উপরের স্তর: Chart প্রকাশের আগে অনুমোদন (Pull Request + CODEOWNERS), **Policy as Code** (যেমন "কোনো Chart-এ `latest` ট্যাগ থাকবে না" — **OPA Conftest** দিয়ে পরীক্ষা), এবং নিরীক্ষা — কে কোন Chart সংস্করণ কখন প্রকাশ করল, তার রেকর্ড।

```
# Chart পরীক্ষার পাইপলাইন (CI-তে চালানোর ধারণা)
$ helm lint ./charts/web
$ helm template web-prod ./charts/web -f ./charts/web/values-prod.yaml \
    | kubeconform -strict
$ helm unittest ./charts/web
# ct দিয়ে পরিবর্তিত Chart-গুলো পরীক্ষা
$ ct lint --config ct.yaml
$ ct install --config ct.yaml

# Policy পরীক্ষা — Conftest দিয়ে (উদাহরণ নীতি: latest ট্যাগ নিষিদ্ধ)
$ helm template web-prod ./charts/web | conftest test -p policy/ -
FAIL - Deployment web must not use image tag 'latest'
```

১৩.১২ TechNova দৃশ্যকল্প: একটি রিলিজের সম্পূর্ণ যাত্রা

TechNova-র `web v1.5.0` রিলিজের যাত্রা: ডেভেলপার ফিচার শেষ করে Pull Request খুললেন — CI-তে `helm lint`, `helm template` + `kubeconform` ও `helm unittest` চলল। মার্জের পর CI ইমেজ বানিয়ে `web:1.5.0` ট্যাগ করল, Chart-এর `appVersion` আপডেট করে একটি Release Pull Request খুলল কনফিগারেশন রিপোজিটরিতে। পর্যালোচক সেই PR-এ শুধু দুটি লাইন দেখলেন — সংস্করণ বদল — এবং অনুমোদন দিলেন। মার্জের পর Argo CD Dev-এ Sync করল, স্বয়ংক্রিয় Smoke Test সবুজ হলো; Staging-এ একইভাবে; তারপর প্রোডাকশনে Manual Sync-এর অনুমোদন দেওয়া হলো। পুরো যাত্রায় মানুষের হাতে কোনো `kubectl` চালানো হয়নি — প্রতিটি ধাপ Git-এ রেকর্ড আছে। রিলিজের পর Chart-টি OCI রেজিস্ট্রিতে প্রকাশ করা হলো, যাতে ভবিষ্যতে অন্য দল একই Chart ব্যবহার করতে পারে।

১৩.১৩ সাধারণ ভুল

- **Template-এ জটিল যুক্তি লেখা:** Template পড়া কঠিন হয়; যুক্তি Values-এ রাখুন।
- **সব মান `--set` দিয়ে দেওয়া:** কমান্ড লাইনে Secret ফাঁস হয়; Values ফাইল ও Secret ব্যবহার করুন।
- **Chart ও App সংস্করণ গুলিয়ে ফেলা:** `version` ও `appVersion` আলাদা রাখুন।
- **`latest` ট্যাগ Chart-এ লেখা:** কোন সংস্করণ চলছে, তা জানা যায় না।
- **Kustomize Base-তে পরিবেশ-নির্দিষ্ট মান লেখা:** Base সবসময় পরিবেশ-নিরপেক্ষ রাখুন; পার্থক্য Overlay-তে।
- **Chart পরীক্ষা ছাড়া প্রকাশ:** `helm lint` ও Template Test ছাড়া Chart প্রোডাকশনে পাঠাবেন না।

১৩.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`helm template`-এ ভুল মান আসছে	Values স্তরবিন্যাসে Override ভুল	`helm template --debug` ও `helm get values` দিয়ে দেখুন
Kustomize Patch কাজ করছে না	Patch-এর Target নাম/ধরন মিলছে না	`kubectl kustomize` দিয়ে রেন্ডার দেখুন; নাম যাচাই করুন
Helm Upgrade ব্যর্থ: “already exists”	Helm-এর বাইরে হাতে রিসোর্স বানানো হয়েছে	Helm-এর `adopt` অ্যানোটেশন বা হাতে মুছে আবার চেষ্টা
Chart ইনস্টলে Dependency পাওয়া যাচ্ছে না	`helm dependency update` চালানো হয়নি	Dependency আপডেট করুন; OCI URL যাচাই করুন
Rollback-এর পরও বাগ আছে	ডেটাবেস Migration ফেরানো হয়নি	Migration Rollback পরিকল্পনা চালান

১৩.১৫ প্রকৌশল ট্রেড-অফ

Helm বনাম Kustomize-এর সিদ্ধান্তটি আসলে “প্যাকেজিং বনাম সরলতা”-র সিদ্ধান্ত। Helm শক্তিশালী, তবে শেখার খরচ বেশি এবং Template-এর ভুল ডিবাগ করা কঠিন। Kustomize সরল, তবে প্যাকেজিং ও সংস্করণের জন্য Git-এর উপর নির্ভর করতে হয়। আরেকটি ট্রেড-অফ: **Chart-এর আকার** — সবকিছু একটি বড় Chart-এ রাখলে পরিচালনা সহজ, কিন্তু একটি ছোট পরিবর্তনেও পুরো Chart-এর সংস্করণ বাড়াতে হয়; ছোট ছোট Chart-এ ভাগ করলে নমনীয়তা বাড়ে, তবে Dependency পরিচালনা জটিল হয়। TechNova-র নিয়ম: একটি অ্যাপ = একটি Chart (বা একটি Kustomize Base); ভাগ করুন শুধু যখন দুটি অ্যাপ সত্যিই স্বাধীনভাবে রিলিজ হয়।

১৩.১৬ অধ্যায়ের সারাংশ

Helm অ্যাপকে প্যাকেজ করে — Chart, Values, Template ও Dependency দিয়ে — আর Kustomize একই Base থেকে Overlay দিয়ে পরিবেশভিত্তিক সংস্করণ বানায়। দুটো একসাথে ব্যবহৃত হয়: তৃতীয় পক্ষের অ্যাপে Helm, নিজের অ্যাপের কনফিগারেশনে Kustomize। Semantic Versioning, Immutable ট্যাগ, Chart Testing (Lint, Template, Unit, Integration) ও Release Governance (অনুমোদন, Policy as Code, নিরীক্ষা) — এই চর্চাগুলোই রিলিজকে পুনরাবৃত্তিযোগ্য ও নিরীক্ষাযোগ্য করে। GitOps পরিবেশে Rollback মানে Git Revert — Helm Rollback শুধু জরুরি অবস্থার হাতিয়ার। পরের অধ্যায়ে আমরা এই সবকিছুর উপরে দাঁড়ানো Internal Developer Platform-এর আর্কিটেকচার শিখব।

১৩.১৭ জ্ঞান যাচাই

1. Helm Chart-এর মূল ফাইলগুলোর নাম ও ভূমিকা লিখুন; Helm 3-এ Tiller না থাকার সুবিধা কী?
2. Go Template-এ `{{ .Values.replicaCount }}` ও `{{ .Release.Name }}`-এর অর্থ কী?
3. Helm Values-এর স্তরবিন্যাস ব্যাখ্যা করুন; Secret কেন `--set` দিয়ে দেবেন না?
4. Kustomize-এ Base ও Overlay-এর পার্থক্য কী? Patch, Image Transformer ও Replica Transformer-এর কাজ লিখুন।
5. Helm বনাম Kustomize — তৃতীয় পক্ষের অ্যাপের জন্য কোনটি ভালো ও কেন?
6. Chart-এর `version` ও `appVersion`-এর পার্থক্য কী? SemVer-এর তিনটি অংশের নিয়ম লিখুন।
7. GitOps পরিবেশে Helm Rollback-এর বদলে Git Revert ব্যবহারের যুক্তি দিন।
8. Chart Testing-এর চারটি স্তর (Lint, Template, Unit, Integration) ব্যাখ্যা করুন।
9. Release Governance বলতে কী বোঝায়? Policy as Code-এর একটি উদাহরণ দিন।
10. একটি বড় Chart-কে ছোট ছোট Chart-এ ভাগ করার ট্রেড-অফ ব্যাখ্যা করুন।

১৩.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র API অ্যাপের জন্য একটি Helm Chart বানান: `Chart.yaml`, `values.yaml`, `values-prod.yaml` ও Deployment Template — Prod-এ রেপ্লিকা ৩, Dev-এ ১। `helm template` দিয়ে দুটি পরিবেশের আউটপুট তুলনা করুন।
2. একই API অ্যাপের জন্য Kustomize Base + Dev/Prod Overlay লিখুন; Prod Overlay-তে ইমেজ ট্যাগ, রেপ্লিকা ও ConfigMap Generator ব্যবহার করুন। `kubectl kustomize` দিয়ে রেন্ডার যাচাই করুন।
3. একটি Chart-এর জন্য CI পরীক্ষার ধাপ লিখুন: `helm lint`, `helm template` + `kubeconform`, `helm unittest` ও Conftest নীতি — প্রতিটি ধাপে কী ধরা পড়বে, তা লিখুন।

অধ্যায় ১৪ • PART III — GitOps & Platform Engineering

Chapter 14 — Internal Developer Platform Architecture

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Platform Engineering** ও **Internal Developer Platform (IDP)** বলতে ঠিক কী বোঝায় তা ব্যাখ্যা করতে; **Developer Experience (DevEx)**-কে পরিমাপযোগ্য লক্ষ্যে রূপান্তর করতে; **Golden Path** (সোনালি পথ) নকশা করতে; **Platform API** ও **Self-Service Infrastructure**-এর আর্কিটেকচার আঁকতে; প্ল্যাটফর্মকে পণ্য হিসেবে পরিচালনা করতে (**Platform Product Management**); ডেভেলপার ওয়ার্কফ্লো বিশ্লেষণ করে বোতল-ঘাড় (Bottleneck) চিহ্নিত করতে; প্ল্যাটফর্ম গ্রহণ (Adoption) বাড়ানোর কৌশল প্রয়োগ করতে; এবং প্ল্যাটফর্মের মূল্য পরিমাপ করতে — আর কখন প্ল্যাটফর্ম না বানানোই সঠিক সিদ্ধান্ত, তা যুক্তি দিয়ে বলতে।

- DevOps, SRE ও Platform Engineering-এর পার্থক্য ব্যাখ্যা করা।
- একটি Golden Path-এর ধাপগুলো (Scaffolding থেকে Production পর্যন্ত) আঁকা।
- Platform API-এর একটি সংজ্ঞা (যেমন Environment Provisioning API) লেখা।
- Thinnest Viable Platform (TVP) দিয়ে ছোট করে শুরু করার পরিকল্পনা করা।
- DORA মেট্রিক দিয়ে প্ল্যাটফর্মের মূল্য পরিমাপ করা।

১৪.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

আপনার Kubernetes, GitOps (অধ্যায় ১১–১২), Helm/Kustomize (অধ্যায় ১৩), CI/CD পাইপলাইন ও **API**-এর মৌলিক ধারণা জানা থাকা দরকার। **Service Mesh**, **Observability** (লগ, মেট্রিক, ট্রেস) ও **Infrastructure as Code**-এর ধারণাগুলো আগের ভলিউমগুলোতে এসেছে; এখানে সেগুলোকে প্ল্যাটফর্মের উপাদান হিসেবে সাজানো হবে। মনে রাখুন: এই অধ্যায়ে কোনো নির্দিষ্ট টুল শেখানো হচ্ছে না — শেখানো হচ্ছে আর্কিটেকচারের সিদ্ধান্ত; টুলগুলো (যেমন Backstage) আসবে অধ্যায় ১৫-এ।

> **CONCEPT:** Internal Developer Platform মানে ডেভেলপারদের জন্য একটি পণ্য — যার "গ্রাহক" হলো আপনার নিজের কোম্পানির ডেভেলপাররা। প্ল্যাটফর্ম দল যদি ডেভেলপারদের গ্রাহক হিসেবে না দেখে, তবে প্ল্যাটফর্মটি হয়ে যায় আরেকটি বাধ্যতামূলক প্রক্রিয়া — যা ডেভেলপাররা এড়িয়ে চলবে। ভালো প্ল্যাটফর্মের পরীক্ষা সহজ: ডেভেলপাররা কি স্বেচ্ছায় এটি ব্যবহার করে, কারণ এটি তাদের কাজ সহজ করে?

১৪.৩ প্রথম নীতি: ডেভেলপারের সময়ই সবচেয়ে দামি সম্পদ

একজন ডেভেলপারের দিন কেমন কাটে? কোড লেখা — আসল মূল্য সৃষ্টির কাজ — আর তার চারপাশে অনেক "অন্য কাজ": পরিবেশ বানানো, CI পাইপলাইন ঠিক করা, Kubernetes YAML লেখা, Secret খুঁজে বের করা, ডেপ্লয়মেন্ট ব্যর্থ হলে লগ খুঁজে বের করা, কার কাছে অনুমতি চাইতে হবে তা জানা। শিল্পের গবেষণায় দেখা যায়, ডেভেলপাররা তাঁদের সময়ের একটি বড় অংশ এই "অন্য কাজে" ব্যয় করেন — একে বলে **Cognitive Load** (মানসিক ভার): একজন মানুষ একসাথে কতটুকু জটিলতা মাথায় রাখতে পারে, তার সীমা। Platform Engineering-এর প্রথম নীতি: **ডেভেলপারের Cognitive**

Load কমানোই প্ল্যাটফর্মের কাজ — Kubernetes-এর জটিলতা ডেভেলপারের কাছ থেকে লুকিয়ে, তাঁর হাতে একটি সহজ, নিরাপদ ও পুনরাবৃত্তিযোগ্য পথ তুলে দেওয়া।

এই নীতি থেকে তিনটি সিদ্ধান্ত আসে। প্রথম: প্ল্যাটফর্ম **Opinionated** (মতামত-সম্পন্ন) হবে — "এইভাবে করো" বলবে, "যেকোনোভাবে করো" বলবে না; কারণ প্রতিটি পছন্দই ডেভেলপারের সিদ্ধান্তের ভার বাড়ায়। দ্বিতীয়: প্ল্যাটফর্ম **Self-Service** হবে — ডেভেলপার টিকিট না কেটে, কারও অপেক্ষা না করে নিজেই পরিবেশ বানাতে পারবেন। তৃতীয়: প্ল্যাটফর্ম **Paved Road** (পাকা রাস্তা) হবে — ডিফল্ট পথটি এত সহজ ও নিরাপদ হবে যে ডেভেলপার স্বেচ্ছায় সেটিই নেবেন; তবে যাঁদের বিশেষ প্রয়োজন, তাঁরা পাকা রাস্তা ছেড়ে নিজের পথে যেতে পারবেন — একে বলে **Golden Path with Escape Hatch**।

প্ল্যাটফর্ম ছাড়া ডেভেলপারের দিন:
কোড লেখা (৩০%) + `YAML` লেখা + পাইপলাইন ঠিক করা
+ পরিবেশের জন্য টিকিট + লগ খোঁজা + অনুমতির অপেক্ষা (৭০%)

প্ল্যাটফর্ম-সহ ডেভেলপারের দিন:
কোড লেখা (৭০%) + `Golden Path`-এ ডেপ্লয় (২০%)
+ শেখা ও পরীক্ষা (১০%)
প্ল্যাটফর্ম দল সামলায়: অবকাঠামো, পাইপলাইন, নিরাপত্তা, পর্যবেক্ষণ।

১৪.৪ Platform Engineering বনাম DevOps বনাম SRE

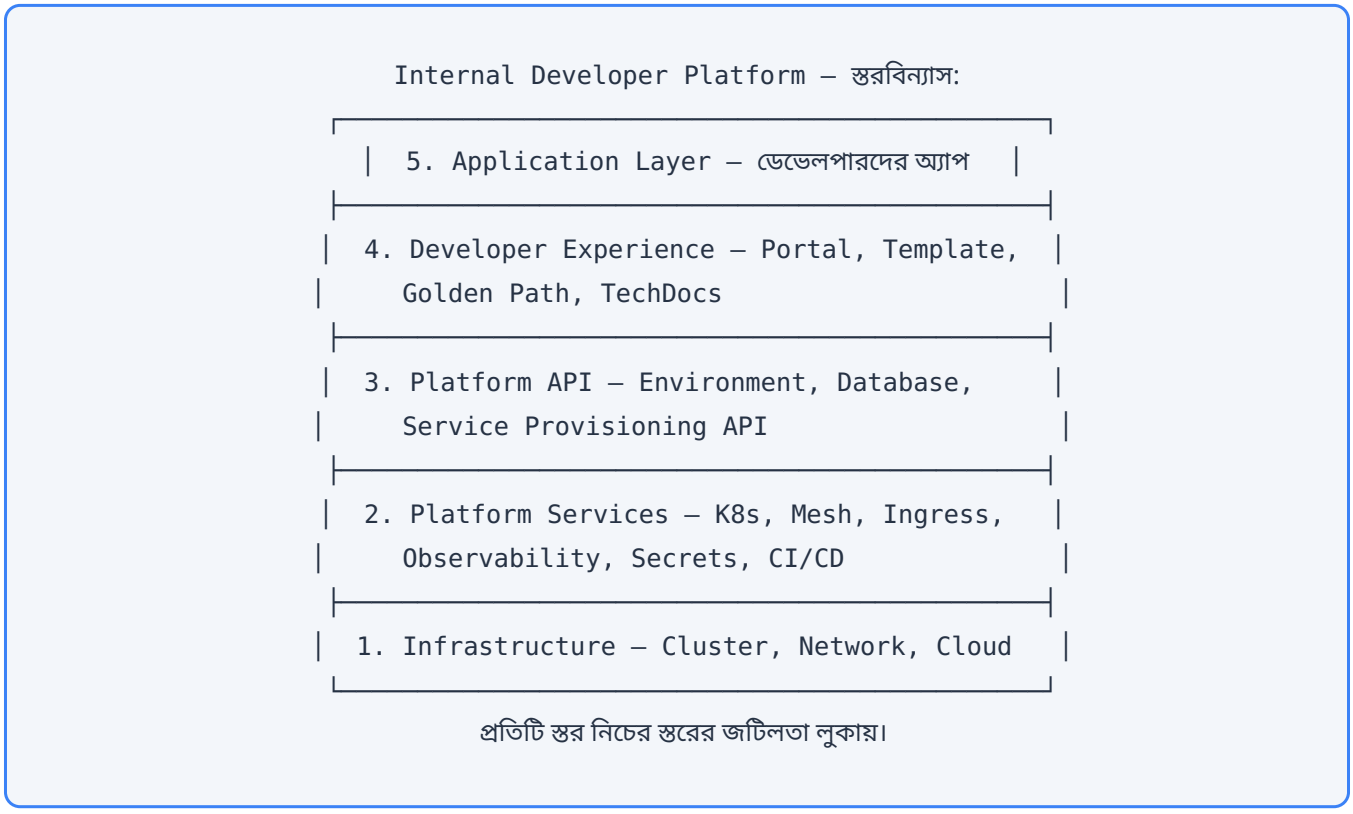
এই তিনটি ভূমিকা প্রায়ই গুলিয়ে ফেলা হয়। **DevOps** হলো একটি সংস্কৃতি ও চর্চার সমষ্টি — ডেভেলপার ও অপারেটরদের মধ্যে সহযোগিতা, স্বয়ংক্রিয়তা, প্রতিক্রিয়া-চক্র। **SRE (Site Reliability Engineering)** হলো নির্ভরযোগ্যতার প্রকৌশল — **SLO** (Service Level Objective), **Error Budget**, ঘটনা-প্রতিক্রিয়া — সিস্টেমকে নির্ভরযোগ্য রাখার শৃঙ্খলা। **Platform Engineering** হলো সেই পণ্যটি বানানো যা DevOps ও SRE-র চর্চাগুলোকে ডেভেলপারদের হাতে সহজ করে তুলে দেয় — IDP-ই সেই পণ্য। সংক্ষেপে: DevOps বলে "কীভাবে কাজ করব", SRE বলে "কতটা নির্ভরযোগ্য থাকব", Platform Engineering বলে "কোন টুল ও পথ দিয়ে সেই কাজটি সহজ করব"। TechNova-তে একজন ব্যক্তি একাধিক ভূমিকা পালন করতে পারেন — তবে ভূমিকাগুলোর পার্থক্য বোঝা জরুরি, নইলে প্ল্যাটফর্ম দল SRE-র কাজে বা DevOps-এর প্রক্রিয়ায় হারিয়ে যায়।

ভূমিকা	মূল প্রশ্ন	সাফল্যের মাপকাঠি
DevOps	ডেলিভারি কীভাবে দ্রুত ও নিরাপদ করব?	Deployment Frequency, Lead Time
SRE	সিস্টেম কতটা নির্ভরযোগ্য থাকবে?	SLO পূরণ, Error Budget
Platform Engineering	ডেভেলপারদের কাজ কীভাবে সহজ করব?	DevEx, Time-to-First-Deploy, Adoption

১৪.৫ IDP আর্কিটেকচার: স্তরে স্তরে সাজানো

একটি IDP-কে পাঁচটি স্তরে ভাবুন — নিচ থেকে উপরে: **(১) Infrastructure Layer** — ক্লাস্টার, নেটওয়ার্ক, স্টোরেজ, ক্লাউড সম্পদ (Terraform দিয়ে পরিচালিত); **(২) Platform Services Layer** — Kubernetes, Service Mesh, Ingress, Observability, Secret পরিচালনা, CI/CD রানার; **(৩) Platform API Layer** — ডেভেলপারদের জন্য একটি

সুসংগত API — “একটি পরিবেশ দাও”, “একটি ডেটাবেস দাও”, “একটি সার্ভিস ডেপ্লয় করো” — এর ভেতরে জটিলতা লুকানো থাকে; **(৪) Developer Experience Layer** — Developer Portal (Backstage), Software Template, Golden Path, ডকুমেন্টেশন; **(৫) Application Layer** — ডেভেলপারদের অ্যাপ। এই স্তরবিন্যাসের মূল কথা: উপরের স্তর নিচের স্তরের জটিলতা জানে না — ডেভেলপার Platform API-কে ডাকেন, API-টি জানে কীভাবে Kubernetes ও Terraform-কে ডাকতে হয়।



১৪.৬ Golden Path: সোনালি পথ নকশা করা

Golden Path হলো একটি নির্দিষ্ট ধরনের কাজের জন্য প্ল্যাটফর্মের সুপারিশকৃত, সম্পূর্ণ পথ — শুরু থেকে শেষ পর্যন্ত। TechNova-র সবচেয়ে গুরুত্বপূর্ণ Golden Path: “নতুন মাইক্রোসার্ভিস তৈরি করে প্রোডাকশনে ডেপ্লয় করা”। এর ধাপগুলো: (১) Developer Portal-এ Software Template বেছে নেওয়া (যেমন “Node.js API Service”); (২) নাম, মালিক দল ও পরিবেশ দিয়ে ফর্ম পূরণ করা; (৩) Template রিপোজিটরি তৈরি করে, CI পাইপলাইন, GitOps Application ও পর্যবেক্ষণ ড্যাশবোর্ড সেট করে দেয়; (৪) ডেভেলপার শুধু ব্যবসায়িক কোড লেখেন; (৫) Pull Request মার্জ হলেই Staging-এ ডেপ্লয়; (৬) অনুমোদন পেলে Production-এ ডেপ্লয়। একটি ভালো Golden Path-এর বৈশিষ্ট্য: এটি **সম্পূর্ণ** (শেষ পর্যন্ত যায়, মাঝপথে থেমে যায় না), **পরীক্ষিত** (প্ল্যাটফর্ম দল নিজে এটি ব্যবহার করে), **নথিভুক্ত** (প্রতিটি ধাপে কী হয়, তা লেখা আছে) এবং **ঐচ্ছিক নয়, ডিফল্ট** (নতুন সার্ভিস মানেই এই পথ)।

```
# TechNova Golden Path — "নতুন Node.js API সার্ভিস"
# ধাপ ১: Portal-এ Template বেছে নিন (Backstage Software Template)
# ধাপ ২: ফর্ম পূরণ করুন
#   serviceName: payments-api
#   owner: team-payments
#   language: node
# ধাপ ৩: Template স্বয়ংক্রিয়ভাবে তৈরি করে:
```

```
#    - GitHub রিপোজিটরি: technova/payments-api
#    - CI পাইপলাইন: test → build → scan → push image
#    - GitOps Application: apps/payments-api/overlays/{dev,staging,prod}
#    - Argo CD Application: payments-api-dev / -staging / -prod
#    - Observability: Dashboard + Alert (error rate > 1%)
#    - TechDocs: সার্ভিসের ডকুমেন্টেশন পাতা
# ধাপ ৪: ডেভেলপার কোড লেখেন; প্রথম ডেপ্লয় Staging-এ ৩০ মিনিটের মধ্যে।
```

Golden Path বানানোর সবচেয়ে বড় ভুল হলো একসাথে অনেকগুলো বানানো। শুরুতে একটি — সবচেয়ে বেশি ব্যবহৃত পথটি — বানান, তা নিখুঁত করুন, ডেভেলপারদের প্রতিক্রিয়া নিন, তারপর দ্বিতীয়টি। একটি ভালো পথের চেয়ে পাঁচটি অসম্পূর্ণ পথ অনেক খারাপ।

১৪.৭ Platform API ও Self-Service Infrastructure

Platform API হলো IDP-র মেরুদণ্ড: ডেভেলপার ও টুলগুলো যে ইন্টারফেস দিয়ে প্ল্যাটফর্মের সেবা চায়। এটি সাধারণত **REST API** বা Kubernetes-এর নিজস্ব API-এর উপরে বানানো **Custom Resource** (যেমন **Crossplane**-এর **Composite Resource**) হয়। একটি ভালো Platform API-এর বৈশিষ্ট্য: **Declarative** ("আমার একটি PostgreSQL ডেটাবেস চাই, আকার মাঝারি" — কীভাবে বানাতে হবে, তা বলতে হয় না); **Self-Service** (অনুমোদনের জন্য টিকিট লাগে না — নীতি API-তেই বাঁধা); **Discoverable** (Portal-এ নথিভুক্ত ও উদাহরণসহ); এবং **Versioned** (API বদলালে পুরোনো সংস্করণ কিছুদিন চলে)। **Self-Service Infrastructure** মানে ডেভেলপার নিজেই — Portal বা API দিয়ে — পরিবেশ, ডেটাবেস, ক্যাশ, কিউ বানাতে পারেন, তবে শুধু অনুমোদিত সীমার মধ্যে (যেমন "Dev-এ ছোট ডেটাবেস, Prod-এ বড় ডেটাবেসের জন্য অনুমোদন লাগবে")।

```
# Platform API — ধারণাগত উদাহরণ (Crossplane Composite Resource)
apiVersion: platform.technova.example/v1alpha1
kind: Database
metadata:
  name: payments-db
  namespace: prod
spec:
  engine: postgresql
  version: "16"
  size: medium        # small | medium | large — ডেভেলপার শুধু আকার বলেন
  backup: true
# প্ল্যাটফর্ম দল এই ঘোষণাকে বাস্তব সম্পদে রূপান্তর করে:
#   medium = 4 vCPU, 16 Gi RAM, 100 Gi স্টোরেজ, দৈনিক ব্যাকআপ
# ডেভেলপার জানেন না এটি কোন ক্লাউডে, কীভাবে বানানো হয় — জানার দরকারও নেই।
```

> **WARNING:** Platform API-তে "সবকিছুই সম্ভব" বানাবেন না। প্রতিটি নতুন ক্ষেত্র, প্রতিটি নতুন বিকল্প — ডেভেলপারের সিদ্ধান্তের ভার বাড়ায়। API-তে শুধু সেই বিকল্পগুলো রাখুন যা বাস্তবে ব্যবহৃত হয়; বাকিগুলো প্ল্যাটফর্ম দলের সিদ্ধান্ত হিসেবে ডিফল্টে লুকিয়ে রাখুন। একটি ছোট, সুসংগত API একটি বড়, বিভ্রান্তিকর API-র চেয়ে ভালো।

১৪.৮ Platform Product Management: প্ল্যাটফর্মকে পণ্য হিসেবে চালানো

প্ল্যাটফর্ম দল একটি পণ্য দল — তার **Product Manager** (বা Product Owner) থাকা উচিত, তার **Roadmap** থাকা উচিত, এবং তার গ্রাহকদের (ডেভেলপারদের) সাথে নিয়মিত কথা বলা উচিত। চর্চাগুলো: **User Research** — ডেভেলপারদের সাক্ষাৎকার নিন: "আপনার দিনের সবচেয়ে বিরক্তিকর কাজটি কী?"; **Backlog** — অনুরোধগুলোকে অগ্রাধিকার দিন প্রভাব অনুযায়ী (কতজন ডেভেলপার, কত সময় বাঁচবে); **Dogfooding** — প্ল্যাটফর্ম দল নিজে নিজের Golden Path ব্যবহার করুক; **Release Notes** — প্ল্যাটফর্মের প্রতিটি পরিবর্তন ডেভেলপারদের জানান; এবং **Deprecation Policy** — পুরোনো পথ বন্ধ করার আগে পর্যাপ্ত সময় ও মাইগ্রেশন নির্দেশনা দিন। প্ল্যাটফর্মের সাফল্য মাপা হয় ডেভেলপারদের সন্তুষ্টি ও সময়-সাশ্রয় দিয়ে — প্ল্যাটফর্ম দলের নিজের সুবিধা দিয়ে নয়।

১৪.৯ ডেভেলপার ওয়ার্কফ্লো ও বোতল-ঘাড় বিশ্লেষণ

প্ল্যাটফর্ম বানানোর আগে বর্তমান ওয়ার্কফ্লোটি মাপুন। TechNova-র একটি নতুন সার্ভিসের যাত্রা মাপা হলো: ধারণা থেকে প্রথম Staging ডেপ্লয় পর্যন্ত সময় লাগে গড়ে ৯ দিন — যার মধ্যে কোড লেখা ২ দিন, বাকি ৭ দিন পরিবেশ, পাইপলাইন, অনুমতি ও ডকুমেন্টেশনের অপেক্ষা। এই ৭ দিনই প্ল্যাটফর্মের লক্ষ্য। **Value Stream Mapping** (মূল্য-প্রবাহ মানচিত্র) দিয়ে প্রতিটি ধাপে অপেক্ষার সময় ও কাজের সময় আলাদা করে লিখুন; যে ধাপে অপেক্ষা সবচেয়ে বেশি, সেখানেই প্রথম আঘাত করুন। সাধারণ বোতল-ঘাড়গুলো: পরিবেশের জন্য টিকিট, Secret-এর জন্য হাতে-কলমে অনুরোধ, CI পাইপলাইন কপি-পেস্ট করা, ডেপ্লয়মেন্টের অনুমোদনের জন্য নির্দিষ্ট ব্যক্তির অপেক্ষা, এবং "কোথায় কী আছে" জানার জন্য Slack-এ জিজ্ঞেস করা।

ধাপ	কাজের সময়	অপেক্ষার সময়	প্ল্যাটফর্ম সমাধান
রিপোজিটরি তৈরি	৩০ মিনিট	১ দিন	Software Template
CI পাইপলাইন সেটআপ	২ ঘণ্টা	১ দিন	Template-এ অন্তর্ভুক্ত পাইপলাইন
পরিবেশ বানানো	১ ঘণ্টা	২ দিন	Self-Service Provisioning
Secret পাওয়া	১৫ মিনিট	১ দিন	External Secrets + Template
প্রথম ডেপ্লয়	৩০ মিনিট	২ দিন	GitOps Application Template

১৪.১০ Thinnest Viable Platform: ছোট করে শুরু করা

সবচেয়ে বড় ভুল হলো একবারে সম্পূর্ণ প্ল্যাটফর্ম বানাতে যাওয়া — ছয় মাস ধরে কিছু না দিয়ে, তারপর একটি বিশাল সিস্টেম হাজির করা যা কেউ চায়নি। এর বদলে **Thinnest Viable Platform (TVP)** — সবচেয়ে পাতলা, তবু ব্যবহারযোগ্য প্ল্যাটফর্ম — দিয়ে শুরু করুন। TechNova-র TVP: (১) একটি Software Template (Node.js API); (২) একটি Golden Path (Template থেকে Staging ডেপ্লয়); (৩) একটি Developer Portal পাতা (সার্ভিসের তালিকা ও মালিক); (৪) একটি Platform API (পরিবেশ বানানো)। এই চারটি দিয়ে ডেভেলপাররা আসল মূল্য পান — তারপর তাঁদের প্রতিক্রিয়া অনুযায়ী বাড়ান। প্রতিটি নতুন ক্ষমতা যোগ করার আগে জিজ্ঞেস করুন: "এটি ছাড়া ডেভেলপাররা কি আটকে আছেন?" — উত্তর "না" হলে, সেটি পরে করুন।

> **REAL WORLD:** TechNova-র প্ল্যাটফর্ম দল প্রথমে একটি পূর্ণাঙ্গ Portal বানাতে ছয় মাস সময় নিয়েছিল — ড্যাশবোর্ড, মেট্রিক, প্লাগইন, সবকিছু। চালু হওয়ার পর দেখা গেল ডেভেলপাররা শুধু একটি জিনিস চেয়েছিলেন: "নতুন সার্ভিস বানানোর বোতাম"। Portal-এর ৮০% বৈশিষ্ট্য কেউ ব্যবহার করেনি। দ্বিতীয়বার তারা TVP পদ্ধতি নিল — প্রথমে শুধু Template ও Golden Path — এবং তিন সপ্তাহে প্রথম ডেভেলপাররা সেটি ব্যবহার করতে শুরু করলেন।

১৪.১১ প্ল্যাটফর্ম গ্রহণ (Adoption) ও পরিবর্তন পরিচালনা

সেরা প্ল্যাটফর্মও ব্যর্থ হয় যদি কেউ তা ব্যবহার না করে। Adoption-এর কৌশল: **Champions** — প্রতিটি দলে একজন করে উৎসাহী ডেভেলপার খুঁজে বের করুন, তাঁদের আগে শেখান, তাঁরাই দলে ছড়িয়ে দেবেন; **Migration Support** — পুরোনো পথ থেকে নতুন পথে যেতে সাহায্য করুন, একা ছেড়ে দেবেন না; **Make the Right Thing Easy** — সঠিক পথটি সবচেয়ে সহজ পথ হোক; ভুল পথটি কঠিন হোক (তবে নিষিদ্ধ নয় — Escape Hatch রাখুন); **Measure and Share** — Adoption-এর সংখ্যা সবাইকে দেখান ("এই মাসে ১২টি সার্ভিস Golden Path-এ তৈরি হয়েছে"); এবং **Feedback Loop** — প্রতি মাসে ডেভেলপারদের কাছ থেকে প্রতিক্রিয়া নিন ও Roadmap-এ দেখান। জোর করে Adoption করাবেন না — জোর করলে ডেভেলপাররা প্ল্যাটফর্মের চারপাশ দিয়ে যাওয়ার পথ খুঁজে নেবে (**Shadow IT**)।

১৪.১২ প্ল্যাটফর্মের মূল্য পরিমাপ করা

প্ল্যাটফর্মের মূল্য পরিমাপ করুন ডেভেলপারদের সময় ও ডেলিভারির গতি দিয়ে। প্রধান মেট্রিকগুলো: **DORA Metrics** — Deployment Frequency (কত ঘন ঘন ডেপ্লয় হয়), Lead Time for Changes (কমিট থেকে প্রোডাকশন পর্যন্ত সময়), Change Failure Rate (কত ডেপ্লয় ব্যর্থ হয়), Time to Restore Service (ব্যর্থতা থেকে পুনরুদ্ধারের সময়); **Time to First Deploy** — নতুন ডেভেলপার বা নতুন সার্ভিসের প্রথম ডেপ্লয় পর্যন্ত সময়; **Developer Satisfaction** — নিয়মিত জরিপ (যেমন "প্ল্যাটফর্ম আপনার কাজ কতটা সহজ করেছে, ১–৫ স্কেলে?"); এবং **Adoption Rate** — কত শতাংশ সার্ভিস Golden Path-এ আছে। এই মেট্রিকগুলো প্ল্যাটফর্ম দলের Roadmap-এর যুক্তি হয়: "আমরা Lead Time ৯ দিন থেকে ২ দিনে নামিয়েছি" — এটাই প্ল্যাটফর্মের ব্যবসায়িক মূল্য।

```
# TechNova প্ল্যাটফর্ম মেট্রিক — উদাহরণ (ধারণাগত, মাসিক)
#   Deployment Frequency:     সপ্তাহে ৪ বার → সপ্তাহে ১৮ বার
#   Lead Time for Changes:    ৯ দিন → ২ দিন
#   Change Failure Rate:      ২৫% → ৮%
#   Time to Restore:          ৪ ঘণ্টা → ৪৫ মিনিট
#   Time to First Deploy:     ৯ দিন → ৪ ঘণ্টা (নতুন সার্ভিস)
#   Adoption Rate:            ১৫% → ৭০% সার্ভিস Golden Path-এ
#   Developer Satisfaction:   ২.৮/৫ → ৪.২/৫
# মেট্রিকগুলো প্ল্যাটফর্ম দলের সাফল্য প্রমাণ করে — অনুমান নয়, সংখ্যা।
```

১৪.১৩ নিরাপত্তা ও খরচ বিবেচনা

প্ল্যাটফর্ম নিরাপত্তার জন্য শক্তিশালী হতে পারে — যদি Golden Path-এ নিরাপত্তা বাঁধা থাকে: প্রতিটি Template-এ **Image Scanning**, **Secret Scanning**, **Resource Limit** ও **Network Policy** ডিফল্টভাবে থাকে — ডেভেলপারকে আলাদা করে ভাবতে হয় না। তবে ঝুঁকিও আছে: প্ল্যাটফর্ম নিজেই একটি **Single Point of Failure** ও আক্রমণের লক্ষ্য হতে পারে — Template-এ একটি ভুল সব সার্ভিসে ছড়িয়ে যায়; Platform API-তে একটি দুর্বলতা সব পরিবেশকে খুলে দেয়। তাই

প্ল্যাটফর্মের নিজের নিরাপত্তা (API-তে প্রমাণীকরণ, RBAC, নিরীক্ষা-লগ) সর্বোচ্চ অগ্রাধিকার পাবে। খরচের দিক: প্ল্যাটফর্ম দল একটি স্থায়ী বিনিয়োগ — সাধারণত ২–৪ জন প্রকৌশলী — এবং প্ল্যাটফর্মের অবকাঠামো (Portal, CI রানার, পর্যবেক্ষণ) চলমান খরচ। এই খরচ তখনই যুক্তিসংগত, যখন ডেভেলপারদের সাশ্রয় করা সময় তার চেয়ে বেশি মূল্যবান হয় — ছোট দলে (৫ জনের কম ডেভেলপার) হিসাবটি প্রায়ই মেলে না।

১৪.১৪ কখন প্ল্যাটফর্ম বানাবেন না

এটি এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ অংশ। প্ল্যাটফর্ম বানাবেন না যখন: ডেভেলপার সংখ্যা কম (৫ জনের কম) — তখন সরাসরি সহযোগিতা ও সহজ স্ক্রিপ্টই যথেষ্ট; দলের কাজের ধরন এখনও স্থির হয়নি — প্ল্যাটফর্ম বানানো মানে অস্থির প্রক্রিয়াকে পাথরে খোদাই করা; কোনো স্পষ্ট সমস্যা নেই — “সবাই প্ল্যাটফর্ম বানাচ্ছে” কোনো যুক্তি নয়; প্ল্যাটফর্ম দলকে পূর্ণকালীন রক্ষণাবেক্ষণ দেওয়ার সামর্থ্য নেই — অরক্ষিত প্ল্যাটফর্ম দ্রুত পুরোনো হয়ে যায় ও ডেভেলপাররা তা ছেড়ে দেয়; অথবা যখন একটি ভালো **SaaS** বা পরিচালিত সেবা (Managed Service) একই সমস্যা সমাধান করে — নিজে বানানোর আগে কিনে নেওয়ার বিকল্পটি বিবেচনা করুন। TechNova-র ক্ষেত্রে — ১৫+ ডেভেলপার, একাধিক দল, বারবার একই সমস্যা — প্ল্যাটফর্ম যুক্তিসংগত। তবে শুরুটা TVP দিয়ে, পূর্ণ প্ল্যাটফর্ম দিয়ে নয়।

> **WARNING:** “Platform Engineering” নামে অনেক দল আসলে শুধু টিকিট-ভিত্তিক অপারেশনকেই নতুন নাম দিয়েছে — ডেভেলপার এখনও টিকিট কাটেন, শুধু দলটির নাম বদলেছে। পরীক্ষা: ডেভেলপার কি টিকিট ছাড়া নিজে পরিবেশ বানাতে পারেন? উত্তর “না” হলে, আপনার কোনো প্ল্যাটফর্ম নেই — আপনার আছে একটি পুনঃনামকৃত অপারেশন দল।

১৪.১৫ সাধারণ ভুল

- **একবারে সম্পূর্ণ প্ল্যাটফর্ম বানানো:** ছয় মাস কিছু না দিয়ে, তারপর কেউ চায়নি এমন সিস্টেম হাজির করা।
- **ডেভেলপারদের সাথে কথা না বলা:** প্ল্যাটফর্ম দলের অনুমানে বানানো পথ কেউ ব্যবহার করে না।
- **Golden Path-কে বাধ্যতামূলক করা:** Escape Hatch ছাড়া বাধ্যতামূলক পথ Shadow IT তৈরি করে।
- **প্ল্যাটফর্মকে শুধু টুলের সংগ্রহ ভাবা:** টুল আছে, কিন্তু Golden Path নেই — ডেভেলপাররা এখনও জানেন না কীভাবে শুরু করবেন।
- **মেট্রিক ছাড়া কাজ করা:** প্ল্যাটফর্মের মূল্য প্রমাণ করা যায় না, বাজেট বন্ধ হয়ে যায়।
- **পুরোনো পথ রক্ষণাবেক্ষণ না করা:** নতুন পথ বানিয়ে পুরোনো পথ ফেলে রাখা — ডেভেলপাররা বিভ্রান্ত হন।

১৪.১৬ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
কেউ Golden Path ব্যবহার করছে না	পথটি অসম্পূর্ণ বা ডেভেলপারদের সমস্যার সাথে মেলে না	সাক্ষাৎকার নিন; সবচেয়ে বিরক্তিকর ধাপটি ঠিক করুন
Platform API-তে অনুরোধ জমে আছে	API-এর সীমা খুব কঠোর; অনুমোদন প্রক্রিয়া ধীর	সীমা পর্যালোচনা করুন; নীতি API-তে স্বয়ংক্রিয় করুন
Template থেকে বানানো সার্ভিস ব্যর্থ হচ্ছে	Template পুরোনো হয়ে গেছে	Template-এর নিয়মিত পরীক্ষা (Template-কে নিজে ব্যবহার করুন)

প্ল্যাটফর্ম দল টিকিটে ডুবে আছে	Self-Service কাজ করছে না; ডেভেলপাররা আটকে যাচ্ছেন	সবচেয়ে বেশি আসা টিকিটগুলোকে Self-Service-এ রূপান্তর করুন
বাজেট প্রশ্নের মুখে	মূল্য পরিমাপ করা হয়নি	DORA মেট্রিক ও Time-to-First-Deploy প্রকাশ করুন

১৪.১৭ প্রকৌশল ট্রেড-অফ: বিনিয়োগ বনাম পরিচালনা-ভার

প্ল্যাটফর্ম একটি বিনিয়োগ — এর ফেরত আসে ধীরে, কিন্তু চক্রবৃদ্ধি হারে। প্রথম মাসে প্ল্যাটফর্ম দল শুধু খরচ — ডেভেলপাররা এখনও পুরোনো পথে কাজ করছেন। তৃতীয় মাসে প্রথম Golden Path ব্যবহার শুরু হয়। ষষ্ঠ মাসে সময়-সাশ্রয় দৃশ্যমান হয়। এই বিলম্বের কারণে অনেক সংস্থা মাঝপথে হাল ছেড়ে দেয় — আর তখন তারা দুটি খরচই বহন করে: অসম্পূর্ণ প্ল্যাটফর্মের রক্ষণাবেক্ষণ ও পুরোনো পথের অদক্ষতা। সিদ্ধান্ত নেওয়ার আগে হিসাব করুন: ডেভেলপার সংখ্যা × প্রতিদিনের অপচয় করা সময় × সময়ের মূল্য — বনাম প্ল্যাটফর্ম দলের খরচ। TechNova-র হিসাবে: ১৫ জন ডেভেলপার, প্রতিদিন গড়ে ১ ঘণ্টা অপচয় — প্ল্যাটফর্ম দলের ২ জন প্রকৌশলীর খরচ তার চেয়ে কম। ছোট দলে হিসাবটি উল্টো হতে পারে — তখন প্ল্যাটফর্ম নয়, সহজ স্ক্রিপ্ট ও ভালো ডকুমেন্টেশনই সঠিক উত্তর।

১৪.১৮ অধ্যায়ের সারাংশ

Internal Developer Platform হলো ডেভেলপারদের জন্য একটি পণ্য — যার লক্ষ্য Cognitive Load কমিয়ে, Golden Path দিয়ে, Self-Service Infrastructure ও Platform API দিয়ে ডেলিভারির গতি বাড়ানো। এর আর্কিটেকচার পাঁচ স্তরে সাজানো — অবকাঠামো থেকে Developer Experience পর্যন্ত — এবং এর পরিচালনা পণ্য-পরিচালনার মতো: ব্যবহারকারী গবেষণা, Roadmap, Dogfooding ও পরিমাপ। শুরু করুন Thinnest Viable Platform দিয়ে — একটি Template, একটি Golden Path — আর মূল্য প্রমাণ করুন DORA মেট্রিক দিয়ে। আর সবচেয়ে গুরুত্বপূর্ণ: ছোট দলে, অস্থির প্রক্রিয়ায় বা স্পষ্ট সমস্যা ছাড়া প্ল্যাটফর্ম বানাবেন না। পরের অধ্যায়ে আমরা এই প্ল্যাটফর্মের মুখ — Backstage Developer Portal — হাতে-কলমে বানাব।

১৪.১৯ জ্ঞান যাচাই

1. Cognitive Load কী? প্ল্যাটফর্ম কীভাবে তা কমায়? একটি উদাহরণ দিন।
2. DevOps, SRE ও Platform Engineering-এর পার্থক্য ব্যাখ্যা করুন।
3. IDP-র পাঁচটি স্তরের নাম লিখুন এবং প্রতিটি স্তরের একটি করে উদাহরণ দিন।
4. Golden Path কী? একটি ভালো Golden Path-এর চারটি বৈশিষ্ট্য লিখুন।
5. Escape Hatch কী এবং কেন এটি প্রয়োজন? বাধ্যতামূলক পথের ঝুঁকি কী?
6. Platform API-এর চারটি বৈশিষ্ট্য (Declarative, Self-Service, Discoverable, Versioned) ব্যাখ্যা করুন।
7. Thinnest Viable Platform (TVP) কী? TechNova-র TVP-তে কোন চারটি জিনিস আছে?
8. DORA মেট্রিকের চারটি উপাদান লিখুন; প্ল্যাটফর্মের মূল্য প্রমাণে এগুলো কীভাবে ব্যবহৃত হয়?
9. প্ল্যাটফর্ম Adoption বাড়ানোর পাঁচটি কৌশল লিখুন।
10. কোন তিনটি পরিস্থিতিতে প্ল্যাটফর্ম বানানো উচিত নয়? যুক্তি দিন।

১৪.২০ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র একটি ডেভেলপার ওয়ার্কফ্লো ম্যাপ করুন: নতুন সার্ভিসের ধারণা থেকে প্রথম Production ডেপ্লয় পর্যন্ত প্রতিটি ধাপে কাজের সময় ও অপেক্ষার সময় লিখুন; সবচেয়ে বড় বোতল-ঘাড়টি চিহ্নিত করুন।
2. একটি Golden Path নকশা করুন: “নতুন Python Worker সার্ভিস” — Template থেকে Production পর্যন্ত প্রতিটি ধাপ, প্রতিটি ধাপে কে কী করে, এবং Escape Hatch কোথায়, তা লিখুন।
3. TechNova-র জন্য একটি TVP পরিকল্পনা লিখুন: প্রথম মাসে কোন চারটি জিনিস থাকবে, কীভাবে মূল্য পরিমাপ করবেন, এবং দ্বিতীয় মাসে কী যোগ করবেন — সময়সীমাসহ।

অধ্যায় ১৫ • PART III — GitOps & Platform Engineering

Chapter 15 — Developer Portals and Self-Service Infrastructure

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Backstage** Developer Portal-এর আর্কিটেকচার ব্যাখ্যা করতে; **Software Catalog**-এ সার্ভিস, কম্পোনেন্ট ও মালিকানা নিবন্ধন করতে; **Software Template** দিয়ে প্রজেক্ট Scaffolding স্বয়ংক্রিয় করতে; **TechDocs** দিয়ে ডকুমেন্টেশনকে কোডের পাশে রাখতে; **Plugin Architecture** ব্যবহার করে Portal-কে প্রসারিত করতে; ডেভেলপার অনবোর্ডিং-এর পথ নকশা করতে; CI/CD ও Kubernetes ইন্টিগ্রেশন যুক্ত করতে; **Platform API**-কে Portal-এর সাথে সংযুক্ত করতে; এবং নিরাপত্তা ও অ্যাক্সেস নিয়ন্ত্রণ (Authentication, Authorization, RBAC) বাস্তবায়ন করতে।

- একটি সম্পূর্ণ `app-config.yaml` লিখতে।
- একটি `catalog-info.yaml` দিয়ে সার্ভিস নিবন্ধন করতে।
- একটি Software Template YAML লিখতে (প্যারামিটার, ধাপ, আউটপুট)।
- TechDocs-এর জন্য `mkdocs.yml` সাজাতে।
- Portal-এর নিরাপত্তা মডেল (SSO, RBAC, Audit) ব্যাখ্যা করতে।

১৫.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

অধ্যায় ১৪-এর IDP ধারণা — Golden Path, Platform API, Self-Service, TVP — এই অধ্যায়ের ভিত্তি। আরও দরকার: **Node.js** ও **TypeScript**-এর প্রাথমিক ধারণা (Backstage এগুলো দিয়ে বানানো), **YAML** লেখার অভিজ্ঞতা, GitHub/GitLab-এর মৌলিক ধারণা (রিপোজিটরি, Pull Request), এবং Kubernetes-এর **Service** ও **Deployment** ধারণা। Backstage-এর ভেতরের কোড পড়তে হবে না — আমরা ব্যবহারকারী ও প্ল্যাটফর্ম পরিচালকের দৃষ্টিকোণ থেকে শিখব: কীভাবে কনফিগার করবেন, কীভাবে Template লিখবেন, কীভাবে প্লাগইন যুক্ত করবেন।

> **CONCEPT:** Developer Portal হলো IDP-র "সদর দরজা" — ডেভেলপাররা যেখানে এসে জানতে পারেন "কী কী সার্ভিস আছে, কার মালিকানায়, কীভাবে নতুন সার্ভিস বানাব, কোথায় ডকুমেন্টেশন আছে"। Portal ছাড়া IDP হলো দরজা ছাড়া বাড়ি — ভেতরে সবকিছু আছে, কিন্তু কেউ ঢুকতে পারে না। Backstage হলো এই Portal বানানোর সবচেয়ে প্রচলিত ওপেন-সোর্স কাঠামো — Spotify-তে জন্ম নেওয়া, CNCF-এর অধীনে।

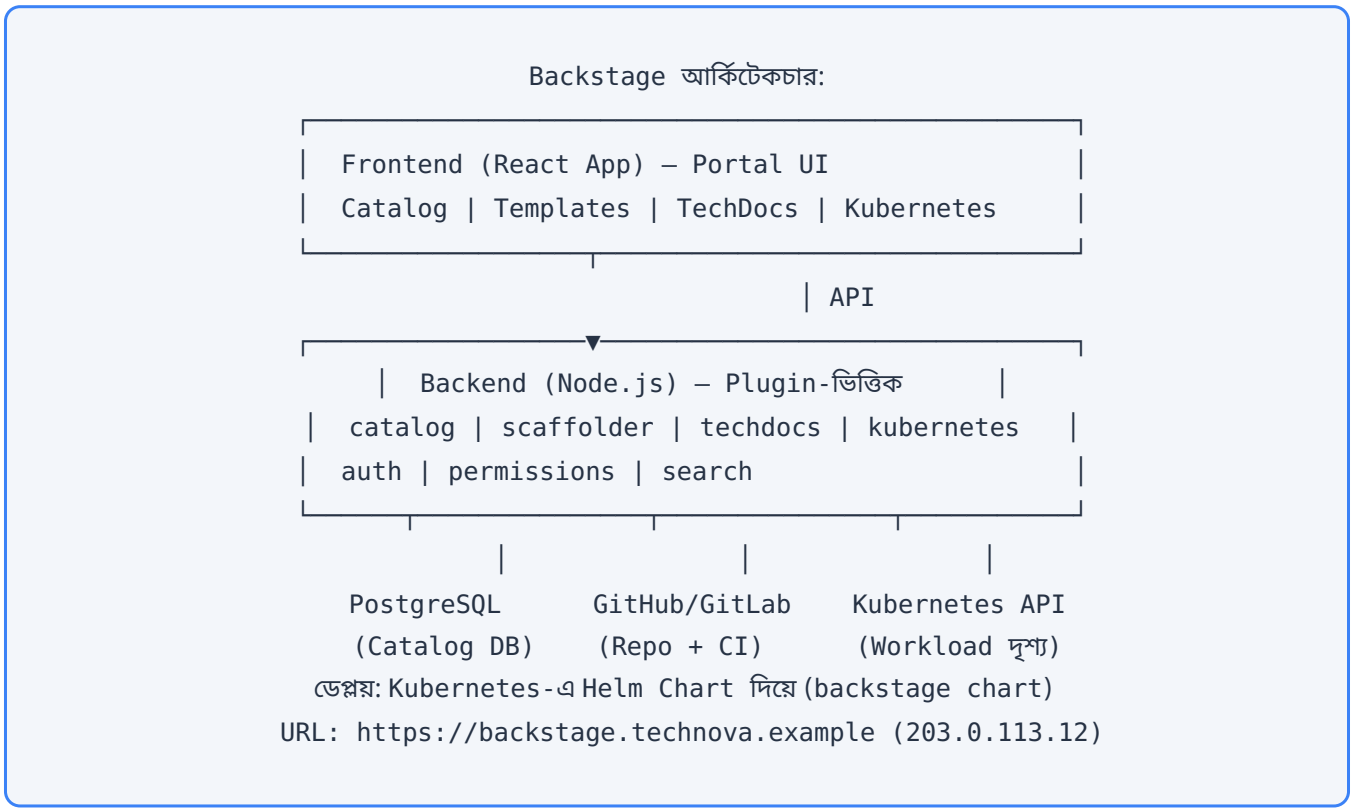
১৫.৩ প্রথম নীতি: আবিষ্কারযোগ্যতাই প্ল্যাটফর্মের প্রাণ

একটি বড় সংস্থায় সবচেয়ে ব্যয়বহুল প্রশ্নগুলো হলো: "এই সার্ভিসটি কার?", "এর API ডকুমেন্টেশন কোথায়?", "এর মালিক কে?", "এটি কি এখনও ব্যবহৃত হয়, নাকি মৃত?", "আমি কি নতুন করে বানাচ্ছি, নাকি এটি আগেই আছে?" — প্রতিটি প্রশ্নের উত্তরে Slack-এ জিজ্ঞেস করা, Wiki খোঁজা, কোড ঘাঁটা — ঘণ্টার পর ঘণ্টা যায়। এই সমস্যার নাম **Discoverability** (আবিষ্কারযোগ্যতা)-র অভাব। Developer Portal-এর প্রথম কাজ হলো সংস্থার সব সফটওয়্যার সম্পদকে একটি **Catalog**-এ (তালিকায়) আনা — প্রতিটি সার্ভিসের নাম, মালিক, অবস্থা, নির্ভরতা ও ডকুমেন্টেশন এক

জায়গায়। Catalog ছাড়া Golden Path-ও অসম্পূর্ণ — ডেভেলপার জানবেনই না কোন Templateটি তাঁর কাজের জন্য সঠিক।

১৫.৪ Backstage আর্কিটেকচার: ভেতরে কী আছে

Backstage 1.x একটি **Node.js** অ্যাপ্লিকেশন — Frontend-এ **React**, Backend-এ **Express**-ভিত্তিক প্লাগইন সিস্টেম। মূল উপাদানগুলো: **Frontend (App)** — ডেভেলপাররা যা দেখেন; **Backend** — API সার্ভার, যেখানে প্রতিটি ক্ষমতা একটি **Plugin**; **Software Catalog** — সব Entity-র (সার্ভিস, কম্পোনেন্ট, API, ব্যবহারকারী, দল) ডেটাবেস; **Database** — সাধারণত PostgreSQL; এবং **Integrations** — GitHub, GitLab, Kubernetes, CI সিস্টেমের সাথে সংযোগ। Backstage-এর দর্শন: **Core ছোট, Plugin দিয়ে বড়** — মূল কাঠামোতে শুধু Catalog ও Template থাকে; বাকি সব — Kubernetes দৃশ্য, CI অবস্থা, TechDocs — Plugin হিসেবে যোগ করা হয়।



১৫.৫ Software Catalog: সংস্থার সফটওয়্যারের মানচিত্র

Software Catalog হলো Backstage-এর হৃদয়। প্রতিটি জিনিস একটি **Entity** — **Component** (একটি সার্ভিস বা লাইব্রেরি), **API** (একটি API-র সংজ্ঞা), **System** (একটি ব্যবসায়িক সিস্টেম — যেমন “Payments”), **Domain** (একটি ব্যবসায়িক এলাকা), **User** ও **Group** (মানুষ ও দল), **Resource** (যেমন ডেটাবেস)। প্রতিটি Entity একটি `catalog-info.yaml` ফাইলে ঘোষিত হয় — সাধারণত সার্ভিসের নিজের রিপোজিটরিতে — আর Backstage তা নিয়মিত পড়ে Catalog-এ তোলে। **Ownership** (মালিকানা) হলো Catalog-এর সবচেয়ে গুরুত্বপূর্ণ ক্ষেত্র: প্রতিটি Component-এর একজন মালিক (একটি Group বা User) থাকতেই হবে — মালিক ছাড়া সার্ভিস মানে “কারও দায়িত্বে নেই” সার্ভিস।

```
# catalog-info.yaml — TechNova web সার্ভিসের নিবন্ধন
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
```

```
  name: web
  description: TechNova web application (Next.js frontend)
  annotations:
    github.com/project-slug: technova/web
    backstage.io/techdocs-ref: dir:.
spec:
  type: service
  lifecycle: production
  owner: team-platform
  system: customer-portal
  providesApis:
    - web-api
  consumesApis:
    - payments-api
  dependsOn:
    - resource:prod-postgres
---
apiVersion: backstage.io/v1alpha1
kind: API
metadata:
  name: web-api
  description: Public web API
spec:
  type: openapi
  lifecycle: production
  owner: team-platform
  definition:
    $text: https://github.com/technova/web/blob/main/openapi.yaml
```

Catalog-এর শক্তি হলো **Dependency Graph** (নির্ভরতা-চিত্র): কোন সার্ভিস কোন API ব্যবহার করে, কোন ডেটাবেসের উপর নির্ভর করে — তা একটি চিত্রে দেখা যায়। একটি সার্ভিস বন্ধ করার আগে দেখা যায় কে কে তার উপর নির্ভরশীল; একটি API বদলানোর আগে দেখা যায় কারা ক্ষতিগ্রস্ত হবে। এই দৃশ্যটিই বড় সংস্থায় Portal-এর সবচেয়ে মূল্যবান বৈশিষ্ট্য।

১৫.৬ Software Template: প্রজেক্ট Scaffolding

Software Template (Scaffolder) হলো Golden Path-এর বাস্তবায়ন: একটি ফর্ম পূরণ করলেই নতুন রিপোজিটরি, CI পাইপলাইন, GitOps Application ও ডকুমেন্টেশন তৈরি হয়ে যায়। Template লেখা হয় YAML-এ: `parameters` (ফর্মের ক্ষেত্রগুলো — নাম, মালিক, ভাষা), `steps` (কী কী কাজ হবে — রিপোজিটরি তৈরি, Template ফাইল কপি, Pull Request খোলা), এবং `output` (ফলাফল — রিপোজিটরি URL, Catalog লিংক)। Template-এর ভেতরে **Cookiecutter**-ধাঁচের Template ফাইল থাকে — যেখানে `{{ values.name }}` লিখলে ব্যবহারকারীর দেওয়া নাম বসে যায়।

```
# Software Template — Node.js API সার্ভিস (সরলীকৃত, বৈধ YAML কাঠামো)
apiVersion: scaffolder.backstage.io/v1beta1
kind: Template
metadata:
  name: nodejs-api-service
  title: Node.js API Service
  description: Create a new Node.js API service with CI and GitOps
spec:
  owner: team-platform
  type: service
  parameters:
    - title: Service Information
      required:
        - name
        - owner
      properties:
        name:
          title: Service Name
          type: string
          description: Unique service name (lowercase, no spaces)
          pattern: "^[a-z][a-z0-9-]*$"
        owner:
          title: Owner Team
          type: string
          ui:field: OwnerPicker
        description:
          title: Description
          type: string
  steps:
    - id: fetch-template
      name: Fetch Template Files
      action: fetch:template
      input:
        url: ./template
        values:
          name: ${{ parameters.name }}
          owner: ${{ parameters.owner }}
          description: ${{ parameters.description }}
    - id: publish-repo
      name: Publish Repository
```

```
      action: publish:github
      input:
        allowedHosts:
          - github.com
        description: ${{ parameters.description }}
        repoUrl: github.com?owner=technova&repo=${{ parameters.name }}
        defaultBranch: main
    - id: register-catalog
      name: Register in Catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps['publish-repo'].output.repoContentsUrl }}
        catalogInfoPath: /catalog-info.yaml
  output:
    links:
      - title: Repository
        url: ${{ steps['publish-repo'].output.remoteUrl }}
      - title: Catalog Entry
        url: /catalog/default/component/${{ parameters.name }}
```

> **WARNING:** Template-এ `publish:github` ধাপটি GitHub-এ রিপোজিটরি তৈরি করে — এর জন্য Backstage-কে একটি GitHub Token দিতে হয়। সেই Token-এর অনুমতি সীমিত রাখুন (শুধু রিপোজিটরি তৈরি ও লেখা), Token-টি কখনোই Template YAML-এ লিখবেন না — `app-config.yaml`-এর Integrations অংশে পরিবেশ চলক থেকে নিন।

১৫.৭ TechDocs: ডকুমেন্টেশন কোডের পাশে

TechDocs হলো Backstage-এর ডকুমেন্টেশন ব্যবস্থা — **Docs-as-Code** (ডকুমেন্টেশন কোড হিসেবে) ধারণার বাস্তবায়ন। প্রতিটি সার্ভিসের রিপোজিটরিতে `docs/` ডিরেক্টরিতে Markdown ফাইল ও `mkdocs.yml` থাকে; Backstage তা পড়ে সুন্দরভাবে Portal-এ দেখায়। ডকুমেন্টেশন কোডের পাশে থাকার সুবিধা: কোড বদলালে ডকুমেন্টেশনও একই Pull Request-এ বদলায় — Wiki-র মতো আলাদা জায়গায় পুরোনো ডকুমেন্টেশন পড়ে থাকে না। **TechDocs-এর নিয়ম:** প্রতিটি সার্ভিসের অন্তত তিনটি পাতা থাকবে — “এই সার্ভিসটি কী করে”, “কীভাবে চালাব”, “সমস্যা হলে কী করব” — এবং প্রতিটি পাতায় মালিক দলের নাম থাকবে।

```
# mkdocs.yml — TechNova web সার্ভিসের TechDocs কনফিগারেশন
site_name: TechNova Web
site_description: Documentation for the TechNova web application
nav:
  - Home: index.md
  - Runbook: runbook.md
```

```
  - Architecture: architecture.md
  - Troubleshooting: troubleshooting.md
plugins:
  - techdocs-core

# docs/index.md — প্রথম পাতার উদাহরণ
# TechNova Web
# এই সার্ভিসটি TechNova-র গ্রাহক-মুখী ওয়েব অ্যাপ্লিকেশন (Next.js)।
# মালিক: team-platform | সিস্টেম: customer-portal
# চালাতে: `npm run dev` — বিস্তারিত Runbook পাতায় দেখুন।
```

১৫.৮ Plugin Architecture: Portal-কে প্রসারিত করা

Backstage-এর শক্তি তার **Plugin** ব্যবস্থা। অফিসিয়াল ও কমিউনিটি প্লাগইনগুলোর মধ্যে উল্লেখযোগ্য: **Kubernetes Plugin** — সার্ভিসের Pod, Deployment ও লগ Portal-এ দেখা যায়; **CI/CD Plugin** (GitHub Actions, Jenkins) — পাইপলাইনের অবস্থা; **TechDocs** — ডকুমেন্টেশন; **Search** — Catalog, ডকুমেন্টেশন ও Template-এ খোঁজা; **Cost Insights** — ক্লাউড খরচ; **Scorecard** — সার্ভিসের পরিপক্কতা (যেমন “ডকুমেন্টেশন আছে কি না, মালিক আছে কি না”)। প্লাগইন যোগ করার নিয়ম: প্রথমে অফিসিয়াল প্লাগইন দিয়ে শুরু করুন; কাস্টম প্লাগইন বানান শুধু যখন কোনো অফিসিয়াল সমাধান নেই — কারণ প্রতিটি কাস্টম প্লাগইন রক্ষণাবেক্ষণের ভার। TechNova-র শুরুর সেট: Catalog, Scaffolder, TechDocs, Kubernetes, GitHub Actions ও Search — এই ছয়টি দিয়েই ৮০% মূল্য পাওয়া যায়।

১৫.৯ app-config.yaml: Backstage কনফিগারেশন

Backstage-এর মূল কনফিগারেশন ফাইল হলো `app-config.yaml` — এখানে Portal-এর শিরোনাম, সংস্থার নাম, ডেটাবেস, প্রমাণীকরণ (Auth), GitHub/GitLab ইন্টিগ্রেশন, Catalog-এর উৎস (Location) ও Kubernetes ক্লাস্টার সংজ্ঞায়িত হয়। সংবেদনশীল মান (Token, Secret) কখনোই এই ফাইলে সরাসরি লিখবেন না — পরিবেশ চলক (`${GITHUB_TOKEN}`) থেকে নিন। নিচের উদাহরণটি একটি সম্পূর্ণ, বৈধ কাঠামো — TechNova-র জন্য সাজানো।

```
# app-config.yaml — TechNova Backstage (সংবেদনশীল মান পরিবেশ চলকে)
app:
  title: TechNova Developer Portal
  baseUrl: https://backstage.technova.example

organization:
  name: TechNova

backend:
  baseUrl: https://backstage.technova.example
  listen:
    port: 7007
  database:
    client: pg
```

```
    connection:
      host: ${POSTGRES_HOST}
      port: ${POSTGRES_PORT}
      user: ${POSTGRES_USER}
      password: ${POSTGRES_PASSWORD}
  cache:
    store: memory
  cors:
    origin: https://backstage.technova.example

auth:
  environment: production
  providers:
    github:
      development:
        clientId: ${AUTH_GITHUB_CLIENT_ID}
        clientSecret: ${AUTH_GITHUB_CLIENT_SECRET}

integrations:
  github:
    - host: github.com
      token: ${GITHUB_TOKEN}
  kubernetes:
    - name: cluster-prod-01
      url: https://203.0.113.10:6443
      serviceAccountToken: ${K8S_PROD_TOKEN}

catalog:
  locations:
    - type: url
      target: https://github.com/technova/technova-gitops/blob/main/catalog/all.yaml
  rules:
    - allow: [Component, System, API, Resource, Location]

techdocs:
  builder: local
  generator:
    runIn: local
  publisher:
    type: local
```

```
permission:
  enabled: true
```

১৫.১০ ডেভেলপার অনবোর্ডিং: প্রথম দিন থেকে প্রথম ডেপ্লয়

নতুন ডেভেলপার যোগ দিলে তাঁর যাত্রা কেমন হওয়া উচিত? **Day 1:** Portal-এ SSO দিয়ে লগইন, নিজের দলের System ও Component-গুলো দেখা, TechDocs পড়া; **Day 2:** একটি বিদ্যমান সার্ভিসের লোকাল সেটআপ — Template-এর README অনুসরণ করে; **Day 3:** প্রথম ছোট পরিবর্তন — Pull Request, CI, Staging ডেপ্লয়; **Week 1:** একটি নতুন সার্ভিস Template থেকে তৈরি করে প্রথম ডেপ্লয়। অনবোর্ডিং-এর মাপকাঠি: **Time to First Commit** ও **Time to First Production Deploy** — TechNova-র লক্ষ্য: প্রথম কমিট প্রথম দিনে, প্রথম প্রোডাকশন ডেপ্লয় প্রথম সপ্তাহে। Portal-এ একটি "নতুন ডেভেলপার নির্দেশিকা" (Onboarding Template) রাখুন — ধাপে ধাপে চেকলিস্ট, প্রতিটি ধাপে লিংক — যাতে কেউ Slack-এ জিজ্ঞেস না করে নিজেই এগোতে পারেন।

> **REAL WORLD:** TechNova-তে আগে নতুন ডেভেলপারের অনবোর্ডিং নির্ভর করত একজন "বন্ধু" (Buddy)-র উপর — তিনি ছুটিতে গেলে অনবোর্ডিং থেমে যেত। Portal-এ অনবোর্ডিং নির্দেশিকা ও Template আসার পর নতুন ডেভেলপাররা নিজেই প্রথম দিনে পরিবেশ তৈরি করেন, দ্বিতীয় দিনে প্রথম Pull Request খোলেন — Buddy-র ভূমিকা এখন শুধু প্রশ্নের উত্তর দেওয়া, হাতে ধরে সবকিছু করানো নয়।

১৫.১১ CI/CD ও Kubernetes ইন্টিগ্রেশন

Portal-এ CI/CD ও Kubernetes ইন্টিগ্রেশন মানে ডেভেলপার Portal ছাড়াই তাঁর সার্ভিসের সম্পূর্ণ অবস্থা দেখতে পান: শেষ পাইপলাইন রান সফল হয়েছে কি না, কোন কমিট ডেপ্লয় হয়েছে, Pod-গুলো চলছে কি না, লগে কী লেখা আছে। **Kubernetes Plugin**-এর জন্য প্রতিটি Component-এর `catalog-info.yaml`-এ একটি অ্যানোটেশন যোগ করতে হয় — `backstage.io/kubernetes-id` — যা বলে কোন লেবেলের Pod-গুলো এই সার্ভিসের। **CI Plugin**-এর জন্য GitHub Actions বা Jenkins-এর সাথে ইন্টিগ্রেশন কনফিগার করতে হয়। গুরুত্বপূর্ণ নীতি: Portal শুধু **দেখায়**, **পরিবর্তন করে না** — ডেপ্লয়মেন্টের কাজ GitOps Controller-এর (অধ্যায় ১২); Portal-এ "ডেপ্লয়" বোতাম থাকলেও তা আসলে Git-এ একটি কমিট বা Argo CD-তে একটি Sync ট্রিগার করে — সরাসরি `kubectl apply` নয়।

```
# Kubernetes Plugin — catalog-info.yaml-এ অ্যানোটেশন
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: web
  annotations:
    backstage.io/kubernetes-id: web
    backstage.io/kubernetes-namespace: prod
spec:
  type: service
  lifecycle: production
  owner: team-platform
```

```
# এই অ্যানোটেশন দেখে Kubernetes Plugin prod Namespace-এ
# app=web লেবেলের Pod, Deployment ও Service খুঁজে Portal-এ দেখায়।

# CI অবস্থা দেখার ধারণা (GitHub Actions Plugin):
#   Component পাতায় "CI/CD" ট্যাবে শেষ রানগুলো দেখা যায় —
#  কোন কমিট, কখন, সফল না ব্যর্থ, লগের লিংক-সহ।
```

১৫.১২ Platform API ও Portal-এর সংযোগ

Portal শুধু দেখার জায়গা নয় — এটি Platform API-র মুখ। “একটি ডেটাবেস চাই” বোতামে ক্লিক করলে Portal Platform API-কে ডাকে (অধ্যায় ১৪.৭-এর Database CRD), API সম্পদ বানায়, আর ফলাফল Portal-এ দেখায়। এই সংযোগের জন্য **Custom Plugin** বা **Scaffolder Action** ব্যবহার করা হয় — Scaffolder-এর একটি ধাপ Platform API-কে ডেকে একটি **Database** বা **Environment** তৈরি করতে পারে, তারপর সেই তথ্য Catalog-এ নিবন্ধন করে। নিয়ম: Portal-এর প্রতিটি “বানাও” বোতামের পেছনে একটি API কল থাকবে — কোনো বোতাম সরাসরি ক্লাস্টার বা ক্লাউড API-তে হাত দেবে না। এতে নিরাপত্তা নীতি (কে কী বানাতে পারবে, কত বড়) এক জায়গায় — Platform API-তে — থাকে, Portal-এ ছড়িয়ে থাকে না।

১৫.১৩ নিরাপত্তা ও অ্যাক্সেস নিয়ন্ত্রণ

Portal-এর নিরাপত্তা তিন স্তরে: **Authentication (প্রমাণীকরণ)** — কে আপনি? GitHub OAuth বা OIDC (যেমন Google, Okta) দিয়ে SSO; **Authorization (অনুমোদন)** — আপনি কী করতে পারবেন? Backstage-এর **Permission Framework** দিয়ে — যেমন “শুধু team-platform-এর সদস্যরা Template প্রকাশ করতে পারবেন”, “সবাই Catalog দেখতে পারবেন, কিন্তু Secret দেখতে পারবেন না”; এবং **Audit (নিরীক্ষা)** — কে কী করল, তার লগ। **RBAC** নীতি লিখুন Catalog-এর মালিকানার সাথে মিলিয়ে: একটি দল নিজের সার্ভিসের Template বদলাতে পারবে, অন্য দলের সার্ভিসের নয়। আরেকটি গুরুত্বপূর্ণ নিয়ম: Portal-এ কখনোই **Secret-এর মান** দেখাবেন না — শুধু Secret-এর অস্তিত্ব ও মেয়াদ দেখান; মান দেখার জন্য Vault-এর নিজস্ব অনুমোদন লাগবে।

```
# Permission নীতি — ধারণাগত উদাহরণ (Backstage Permission Framework)
#   - catalog.entity.read: সবাই (সব Entity দেখতে পারবে)
#   - scaffolder.template.execute: সবাই (Template চালাতে পারবে)
#   - scaffolder.template.publish: শুধু team-platform
#   - catalog.entity.delete: শুধু Entity-র মালিক দল
#   - kubernetes.pod.logs.read: শুধু সার্ভিসের মালিক দল
# নীতি লেখার সময় ন্যূনতম অনুমতির নীতি (Least Privilege) অনুসরণ করুন:
#  প্রথমে সবকিছু বন্ধ, তারপর প্রয়োজন অনুযায়ী খুলুন — উল্টোটি নয়।
```

> **WARNING:** Backstage-এর ডিফল্ট সেটআপে Guest ব্যবহারকারী সবকিছু দেখতে পারে — প্রোডাকশনে যাওয়ার আগে Guest অ্যাক্সেস বন্ধ করুন এবং SSO বাধ্যতামূলক করুন। Portal-এ Kubernetes ServiceAccount Token থাকে — সেই Token-এর অনুমতি শুধু পড়ার (Read-Only) রাখুন; Portal থেকে কেউ যেন Pod মুছতে বা Deployment বদলাতে না পারে।

১৫.১৪ খরচ ও পরিচালনা বিবেচনা

Backstage চালানোর খরচ: একটি ছোট PostgreSQL ডেটাবেস, কয়েকটি Pod, এবং প্ল্যাটফর্ম দলের রক্ষণাবেক্ষণ সময়। আসল খরচ হলো **Catalog-কে হালনাগাদ রাখা** — প্রতিটি নতুন সার্ভিসে `catalog-info.yaml` লেখা, মালিকানা ঠিক রাখা, পুরোনো সার্ভিস মুছে ফেলা। এই শৃঙ্খলা না থাকলে Catalog দ্রুত পুরোনো হয়ে যায়, আর ডেভেলপাররা Portal-কে বিশ্বাস করা বন্ধ করে দেন — তখন Portal-এর মূল্যই শেষ। সমাধান: Catalog-এর স্বাস্থ্য পরিমাপ করুন (কত শতাংশ Component-এর মালিক আছে, ডকুমেন্টেশন আছে, শেষ আপডেট কবে) এবং **Scorecard Plugin** দিয়ে তা সবাইকে দেখান। আরেকটি খরচ: Backstage আপগ্রেড — Plugin-গুলোর সাথে সংস্করণের সামঞ্জস্য রাখতে হয়; বছরে কয়েকবার আপগ্রেডের সময় বরাদ্দ রাখুন।

১৫.১৫ TechNova দৃশ্যকল্প: Portal চালু হওয়ার প্রথম মাস

TechNova-র Portal চালু হলো TVP দিয়ে — Catalog, একটি Template (Node.js API), TechDocs ও Kubernetes Plugin। প্রথম সপ্তাহে প্ল্যাটফর্ম দল নিজেরাই ৫টি বিদ্যমান সার্ভিস Catalog-এ নিবন্ধন করলেন — প্রতিটিতে মালিক, System ও ডকুমেন্টেশন লিংক। দ্বিতীয় সপ্তাহে Payments দল Template দিয়ে একটি নতুন সার্ভিস বানালেন — রিপোজিটরি থেকে Staging ডেপ্লয় পর্যন্ত সময় লাগল ৪৫ মিনিট, আগের ৯ দিনের বদলে। তৃতীয় সপ্তাহে একজন ডেভেলপার Portal-এ দেখলেন Payments API-র একটি নতুন সংস্করণ আছে — আগে তিনি জানতেনই না সেটি আছে, আর নিজে একই API আবার বানাতে যাচ্ছিলেন। মাস শেষে Catalog-এ ১৮টি Component, Adoption ৭০% — আর প্ল্যাটফর্ম দলের কাছে পরের মাসের Roadmap-এর জন্য ডেভেলপারদের অনুরোধের তালিকা।

১৫.১৬ সাধারণ ভুল

- **Guest অ্যাক্সেস খোলা রাখা:** প্রোডাকশন Portal-এ SSO ছাড়া চালানো।
- **Catalog-এ মালিক না লেখা:** মালিক ছাড়া Component মানে দায়িত্ব ছাড়া সার্ভিস।
- **Template-এ Secret লেখা:** Token ও পাসওয়ার্ড পরিবেশ চলকে রাখুন।
- **সব প্লাগইন একসাথে যোগ করা:** শুরুতে ছয়টি মূল প্লাগইনই যথেষ্ট; বাকিগুলো চাহিদা অনুযায়ী।
- **TechDocs কোড থেকে আলাদা রাখা:** ডকুমেন্টেশন Wiki-তে রাখলে তা পুরোনো হয়ে যায়।
- **Portal থেকে সরাসরি ক্লাস্টার পরিবর্তন:** Portal শুধু দেখাবে ও API ডাকবে; পরিবর্তন GitOps-এর পথে যাবে।

১৫.১৭ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Catalog-এ Component দেখা যাচ্ছে না	`catalog-info.yaml` -এর সিনট্যাক্স ভুল; Location যোগ করা হয়নি	YAML যাচাই করুন; `app-config.yaml` -এর Location দেখুন
Template চালালে রিপোজিটরি তৈরি হচ্ছে না	GitHub Token-এর অনুমতি কম; Repo URL ভুল	Token-এর Scope পরীক্ষা করুন; Scaffolder লগ দেখুন
TechDocs পাতা খালি দেখাচ্ছে	`mkdocs.yml` ভুল; `techdocs-ref` অ্যানোটেশন নেই	অ্যানোটেশন যাচাই করুন; TechDocs জেনারেটর লগ দেখুন
Kubernetes Plugin-এ Pod দেখা যাচ্ছে না	অ্যানোটেশন ভুল; ServiceAccount Token-এর অনুমতি নেই	লেবেল ও Namespace যাচাই করুন; RBAC পরীক্ষা করুন

SSO লগইন ব্যর্থ হচ্ছে	Client ID/Secret ভুল; Callback URL মিলছে না	OAuth অ্যাপের Callback URL যাচাই করুন

১৫.১৮ প্রকৌশল ট্রেড-অফ: Portal বনাম সরল ডকুমেন্টেশন

Backstage শক্তিশালী, তবে ভারী — ইনস্টল, কনফিগার, রক্ষণাবেক্ষণ — সবই সময় নেয়। ছোট দলে (৫ জনের কম) একটি ভালো README, একটি Wiki ও কয়েকটি Template স্ক্রিপ্টই যথেষ্ট হতে পারে; সেখানে Backstage অপ্রয়োজনীয় ভার। Backstage যুক্তিসংগত হয় যখন: সার্ভিসের সংখ্যা ১০-এর বেশি, দল একাধিক, "এই সার্ভিসটি কার?" প্রশ্নটি নিয়মিত আসে, এবং প্ল্যাটফর্ম দল রক্ষণাবেক্ষণের দায়িত্ব নিতে প্রস্তুত। TechNova-র ক্ষেত্রে — ১৮টি সার্ভিস, তিনটি দল — Portal যুক্তিসংগত। তবে শুরুটা TVP দিয়ে: Catalog + একটি Template + TechDocs — বাকি প্লাগইনগুলো চাহিদা অনুযায়ী। Portal-কে কখনোই "সব সমস্যার সমাধান" ভাববেন না — এটি IDP-র মুখ, IDP নিজে নয়; Golden Path ও Platform API ছাড়া Portal শুধু একটি সুন্দর তালিকা।

১৫.১৯ অধ্যায়ের সারাংশ

Developer Portal — Backstage দিয়ে বানানো — IDP-র সদর দরজা: Software Catalog সংস্থার সব সার্ভিস, API ও মালিকানার মানচিত্র দেয়; Software Template Golden Path-কে বোতামে রূপান্তর করে; TechDocs ডকুমেন্টেশনকে কোডের পাশে রাখে; Plugin Architecture Portal-কে Kubernetes, CI/CD ও খরচের সাথে যুক্ত করে। `app-config.yaml`, `catalog-info.yaml` ও Template YAML — এই তিনটি ফাইলই Portal-এর মূল কনফিগারেশন। নিরাপত্তায় SSO, Least Privilege RBAC ও Read-Only Kubernetes Token বাধ্যতামূলক। আর মনে রাখুন: Portal ছাড়া IDP অসম্পূর্ণ, কিন্তু Portal-ই IDP নয় — Golden Path ও Platform API-ই আসল প্ল্যাটফর্ম। এই অধ্যায় দিয়ে PART III শেষ হলো — GitOps-এর নীতি থেকে Argo CD ও Flux, Helm ও Kustomize, IDP আর্কিটেকচার হয়ে Developer Portal পর্যন্ত — TechNova-র প্ল্যাটফর্মের সম্পূর্ণ ছবি এখন আপনার হাতে।

১৫.২০ জ্ঞান যাচাই

1. Developer Portal কী এবং কেন এটি IDP-র "সদর দরজা"? Discoverability-র অভাবের একটি উদাহরণ দিন।
2. Backstage-এর মূল উপাদানগুলোর (Frontend, Backend, Catalog, Database, Integrations) ভূমিকা লিখুন।
3. Software Catalog-এ Entity কাকে বলে? Component, API, System ও Domain-এর পার্থক্য ব্যাখ্যা করুন।
4. `catalog-info.yaml`-এ Ownership কেন সবচেয়ে গুরুত্বপূর্ণ ক্ষেত্র? মালিক ছাড়া সার্ভিসের ঝুঁকি কী?
5. Software Template-এর তিনটি অংশ (Parameters, Steps, Output) ব্যাখ্যা করুন।
6. TechDocs ও Docs-as-Code বলতে কী বোঝায়? Wiki থেকে এর সুবিধা কী?
7. Plugin Architecture-এর সুবিধা কী? TechNova-র শুরুর ছয়টি প্লাগইনের নাম লিখুন।
8. `app-config.yaml`-এ সংবেদনশীল মান কীভাবে রাখা উচিত? সরাসরি লেখার ঝুঁকি কী?
9. Portal থেকে সরাসরি ক্লাস্টার পরিবর্তন করা কেন উচিত নয়? সঠিক পথটি কী?
10. ছোট দলে Backstage না বানিয়ে কী করা যেতে পারে? Portal বানানোর সিদ্ধান্তের তিনটি শর্ত লিখুন।

১৫.২১ ব্যবহারিক অ্যাসাইনমেন্ট

1. TechNova-র `payments-api` সার্ভিসের জন্য একটি সম্পূর্ণ `catalog-info.yaml` লিখুন: Component, মালিক `team-payments`, System, একটি API (providesApis) ও একটি ডেটাবেস নির্ভরতা (dependsOn)-সহ।

2. একটি Software Template লিখুন: “Python Worker Service” — প্যারামিটার (নাম, মালিক), ধাপ (Template Fetch, GitHub-এ প্রকাশ, Catalog-এ নিবন্ধন) ও আউটপুট (রিপোজিটরি ও Catalog লিংক)-সহ।
3. TechNova-র জন্য একটি অনবোর্ডিং নির্দেশিকা লিখুন: নতুন ডেভেলপারের প্রথম সপ্তাহের দিনভিত্তিক চেকলিস্ট — Portal, Template, TechDocs ও প্রথম Production ডেপ্লয়-সহ — এবং Time to First Commit ও Time to First Deploy-এর লক্ষ্য লিখুন।

PART IV • DEVOPS ENGINEERING MASTERY

Advanced Automation & Enterprise Operations

Advanced CI/CD ↬ Progressive Delivery, Platform as Code, Multi-Cloud Platform Operations, Advanced Observability ↬ SRE, Enterprise Security ↬ Supply Chain Governance

অধ্যায় ১৬ • PART IV — Advanced Automation & Enterprise Operations

Chapter 16 — Advanced CI/CD and Progressive Delivery

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Continuous Integration (CI)** ও **Continuous Delivery / Deployment (CD)**-এর পার্থক্য প্রথম নীতি থেকে ব্যাখ্যা করতে; পুনঃব্যবহারযোগ্য পাইপলাইন আর্কিটেকচার নকশা করতে; **Blue-Green Deployment**, **Canary Release** ও **Rolling Deployment**-এর মধ্যে সঠিক কৌশলটি বেছে নিতে; **Feature Flag** দিয়ে ডেপ্লয় ও রিলিজ আলাদা করতে; **Progressive Delivery**-র ধারণা, **Argo Rollouts** ও **Flagger**-এর মতো টুলের ভূমিকা, স্বয়ংক্রিয় রোলব্যাক ও রিলিজ যাচাইয়ের প্রক্রিয়া TechNova-র প্রেক্ষাপটে প্রয়োগ করতে।

- উন্নত পাইপলাইন আর্কিটেকচার: টেমপ্লেট, পুনঃব্যবহারযোগ্য ধাপ ও পরিবেশভিত্তিক প্রচার (Promotion) নকশা করা।
- তিনটি মূল ডেপ্লয়মেন্ট কৌশলের কার্যপ্রণালী, ঝুঁকি ও খরচ তুলনা করা।
- Feature Flag, Canary বিশ্লেষণ ও স্বয়ংক্রিয় রোলব্যাক দিয়ে নিরাপদ রিলিজ বানানো।
- একটি প্রোডাকশন রিলিজের সম্পূর্ণ যাচাই-তালিকা তৈরি করা।

১৬.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায় পড়ার আগে আপনার Git-এর শাখা (Branch) ও Pull Request, Docker ইমেজ বানানো, Kubernetes-এ Deployment ও Service-এর ধারণা, এবং আগের ভলিউমগুলোতে শেখা Terraform দিয়ে অবকাঠামো বানানোর প্রাথমিক ধারণা থাকা দরকার। **CI** মানে কোড জমা দেওয়ার সাথে সাথে স্বয়ংক্রিয়ভাবে বিল্ড ও পরীক্ষা করা; **CD**-র দুটি অর্থ আছে — **Continuous Delivery** মানে যেকোনো সময় ডেপ্লয় করার মতো প্রস্তুত অবস্থায় রাখা, আর **Continuous Deployment** মানে প্রতিটি সফল পরিবর্তন স্বয়ংক্রিয়ভাবে প্রোডাকশনে পৌঁছে যাওয়া। এই অধ্যায়ে আমরা ধরে নিচ্ছি TechNova-র অ্যাপ্লিকেশনটি কনটেইনারে চলে, Kubernetes-এ ডেপ্লয় হয়, এবং Git-ই সত্যের উৎস (Source of Truth)।

> **CONCEPT:** ডেপ্লয় (Deploy) আর রিলিজ (Release) এক নয়। ডেপ্লয় মানে কোড প্রোডাকশন পরিবেশে পৌঁছানো; রিলিজ মানে ব্যবহারকারীর কাছে সেই কোডের আচরণ উন্মুক্ত করা। Feature Flag থাকলে কোড প্রোডাকশনে ডেপ্লয় করেও রিলিজ বন্ধ রাখা যায় — আর এটাই Progressive Delivery-র ভিত্তি।

১৬.৩ প্রথম নীতি থেকে: পাইপলাইন আসলে কী সমস্যা সমাধান করে

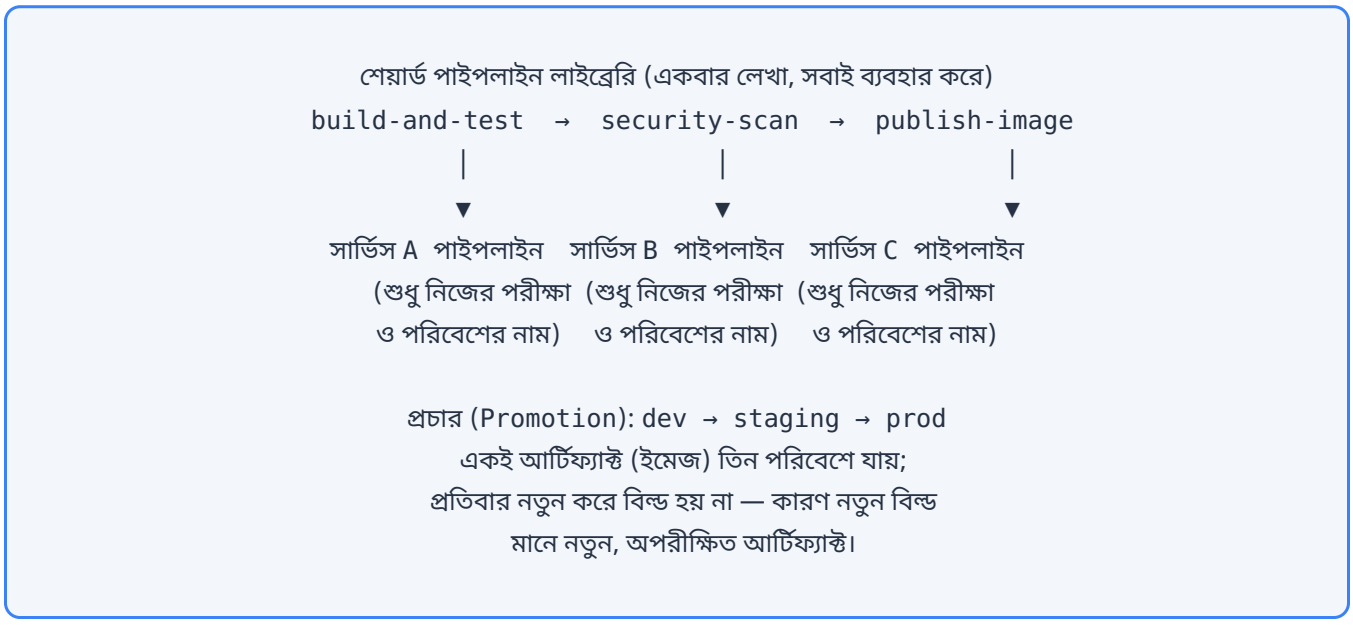
সফটওয়্যার ডেলিভারির মূল সমস্যাটি গাণিতিক নয়, মানবিক: পরিবর্তন যত বড় ও যত দেরিতে একত্রিত হয়, তত বেশি সংঘাত (Conflict), তত বেশি অজানা ঝুঁকি, তত বেশি ভয়। ছোট ছোট পরিবর্তন ঘন ঘন একত্রিত করলে প্রতিটি পরিবর্তনের প্রভাব বোঝা যায়, ভুল হলে দায়ী পরিবর্তনটি খুঁজে পাওয়া যায়, আর ফিরে যাওয়া সহজ হয়। CI/CD পাইপলাইন এই নীতিটিকে যন্ত্রে রূপ দেয়: প্রতিটি কমিট একটি নির্দিষ্ট ধাপের সারি পেরোয় — বিল্ড, পরীক্ষা, নিরাপত্তা স্ক্যান, আর্টিফ্যাক্ট সংরক্ষণ, পরিবেশে প্রচার — এবং প্রতিটি ধাপের ফলাফল লিখিত থাকে। পাইপলাইন যত স্বয়ংক্রিয়, ডেলিভারি তত অনুমানযোগ্য; যত হাতে-চালানো ধাপ, তত বেশি অনিশ্চয়তা।

কিন্তু স্বয়ংক্রিয়তা নিজেই লক্ষ্য নয়। একটি পাইপলাইন ভালো হয় তখনই, যখন তা তিনটি প্রশ্নের উত্তর দ্রুত ও সত্যভাবে দেয়: এই পরিবর্তনটি কি সঠিকভাবে বানানো হয়েছে? এটি কি নিরাপদ? এটি কি আগের অবস্থার চেয়ে খারাপ আচরণ করবে না?

এই তিন প্রশ্নের উত্তর দেওয়ার জন্য পরীক্ষা, স্ক্যান ও পর্যবেক্ষণ — তিনটি স্তরই দরকার। কোনো একটি স্তর বাদ দিলে পাইপলাইন দ্রুত হয়, কিন্তু অন্ধ হয়।

১৬.৪ উন্নত পাইপলাইন আর্কিটেকচার

ছোট দলে একটি মাত্র পাইপলাইন ফাইলেই সবকিছু লেখা যায়। কিন্তু TechNova-র মতো জায়গায় যখন দশটি সার্ভিস, তিনটি পরিবেশ ও একাধিক দল থাকে, তখন প্রতিটি রিপোজিটরিতে পাইপলাইন কপি-পেস্ট করলে যা হয়, তার নাম **Pipeline Sprawl** — একই ভুল দশ জায়গায়, একই উন্নতি দশ জায়গায় করতে হয়। সমাধান হলো পাইপলাইনকে সফটওয়্যারের মতো গঠন করা: সাধারণ ধাপগুলোকে পুনঃব্যবহারযোগ্য টেমপ্লেট বা শেয়ার্ড লাইব্রেরিতে তোলা, আর প্রতিটি সার্ভিসের পাইপলাইনে শুধু তার নিজের বিশেষত্ব রাখা।



Artifact Promotion-এর এই নীতিটি মনে রাখা জরুরি: পরিবেশ বদলালে আর্টিফ্যাক্ট বদলায় না, বদলায় শুধু কনফিগারেশন — পরিবেশ চলক, সিক্রেটের উৎস, রেপ্লিকার সংখ্যা। যে দল প্রতিটি পরিবেশে নতুন করে ইমেজ বানায়, তারা আসলে প্রতিটি পরিবেশে ভিন্ন জিনিস পরীক্ষা করে, আর প্রোডাকশনে কী চলছে তা নিশ্চিতভাবে জানে না। আর্টিফ্যাক্টকে অপরিবর্তনীয় (Immutable) ধরা, তাকে একটি রেজিস্ট্রিতে ট্যাগসহ সংরক্ষণ করা, আর সেই একই ট্যাগকে পরিবেশে পরিবেশে এগিয়ে নেওয়া — এটাই উন্নত পাইপলাইনের প্রথম চিহ্ন।

১৬.৪.১ পাইপলাইনের স্তরবিন্যাস

একটি পরিপক্ক পাইপলাইনে সাধারণত চারটি স্তর থাকে। প্রথম স্তর **Commit Stage**: দ্রুত একক পরীক্ষা (Unit Test), লিন্ট ও বিল্ড — কয়েক মিনিটের মধ্যে ফল। দ্বিতীয় স্তর **Acceptance Stage**: সংযোজন পরীক্ষা (Integration Test) ও API পরীক্ষা, একটি ক্ষণস্থায়ী পরিবেশে। তৃতীয় স্তর **Security & Compliance Stage** : নির্ভরতার দুর্বলতা স্ক্যান, কনটেইনার ইমেজ স্ক্যান, নীতি পরীক্ষা। চতুর্থ স্তর **Deploy Stage**: পরিবেশে পরিবেশে প্রচার, প্রতিটি ধাপে অনুমোদন বা স্বয়ংক্রিয় গেট। স্তরগুলো যত দেরিতে চলে, তত বেশি খরচ — তাই দ্রুত ব্যর্থ হওয়া (Fail Fast) নীতিতে সস্তা ও দ্রুত পরীক্ষা আগে, ধীর ও ব্যয়বহুল পরীক্ষা পরে রাখা হয়।

১৬.৫ ডেপ্লয়মেন্ট অর্কেস্ট্রেশন: পরিবর্তন কীভাবে প্রোডাকশনে পৌঁছায়

Deployment Orchestration মানে শুধু নতুন কোড চালানো নয় — পুরোনো সংস্করণ থেকে নতুন সংস্করণে যাওয়ার পুরো নৃত্য পরিচালনা করা: নতুন Pod বানানো, স্বাস্থ্য পরীক্ষা করা, ট্রাফিক সরানো, পুরোনো Pod সরানো, আর কিছু ভুল হলে

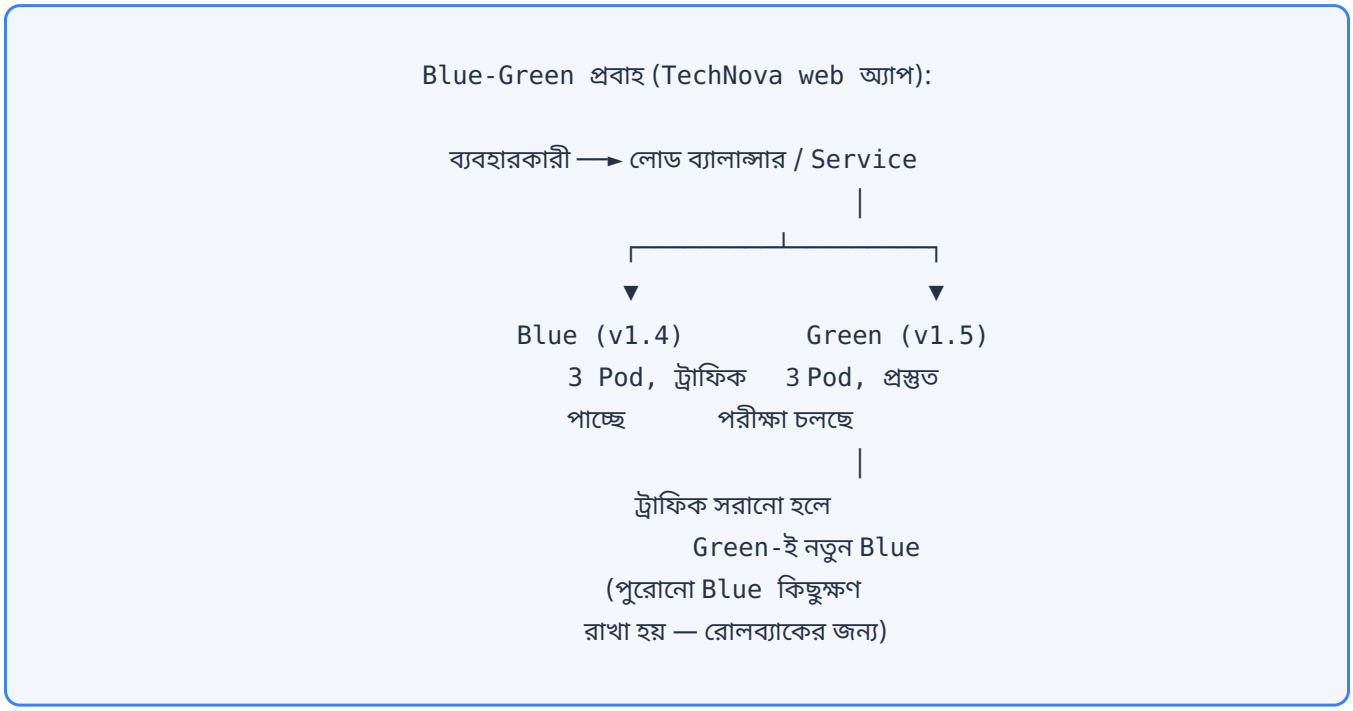
আগের অবস্থায় ফেরা। Kubernetes-এ Deployment রিসোর্স এই অর্কেস্ট্রেশনের একটি অংশ করে, কিন্তু কৌশলটি আপনাকেই বেছে নিতে হয়।

কৌশল	কীভাবে কাজ করে	সুবিধা	ঝুঁকি / খরচ	কখন ব্যবহার করবেন
Rolling Deployment	একটি করে বা কয়েকটি করে Pod বদলে নতুন সংস্করণ আনা হয়	অতিরিক্ত সম্পদ কম লাগে; Kubernetes-এ পূর্বনির্ধারিত	কিছুক্ষণ দুটি সংস্করণ একসাথে চলে; ভুল ধরা ধীর	সাধারণ দৈনন্দিন রিলিজ, ছোট পরিবর্তন
Blue-Green Deployment	নতুন সংস্করণ (Green) সম্পূর্ণ আলাদাভাবে বানিয়ে, পরীক্ষা করে, তারপর ট্রাফিক একবারে সরানো হয়	তাৎক্ষণিক রোলব্যাক; প্রোডাকশন-সম পরিবেশে পরীক্ষা	দ্বিগুণ সম্পদ লাগে; ডেটাবেস স্কিমা বদল জটিল	বড় রিলিজ, ঝুঁকিপূর্ণ পরিবর্তন, নিশ্চিত রোলব্যাক দরকার হলে
Canary Release	নতুন সংস্করণ প্রথমে ১–৫% ট্রাফিক পায়; মেট্রিক ভালো থাকলে ধাপে ধাপে বাড়ে	বাস্তব ব্যবহারকারীর উপর ছোট আকারে পরীক্ষা; ক্ষতি সীমিত	ট্রাফিক ভাগ, মেট্রিক বিশ্লেষণ ও অর্কেস্ট্রেশন জটিল	ঝুঁকিপূর্ণ ফিচার, পারফরম্যান্স-সংবেদনশীল পরিবর্তন
Recreate	পুরোনো সব Pod থামিয়ে তারপর নতুন চালানো হয়	সহজ; দুটি সংস্করণ কখনো একসাথে চলে না	ডাউনটাইম হয়	শুধু ডেভেলপমেন্ট বা সহনীয় ডাউনটাইমের জায়গায়

মনে রাখবেন, কোনো কৌশলই সর্বোত্তম নয় — প্রশ্নটি হলো কোন ঝুঁকিটি আপনি নিতে চান। Rolling সস্তা কিন্তু অন্ধ; Blue-Green নিরাপদ কিন্তু ব্যয়বহুল; Canary বুদ্ধিমান কিন্তু জটিল। TechNova-র মতো ছোট দলের জন্য দৈনন্দিন কাজে Rolling, বড় রিলিজে Blue-Green, আর ঝুঁকিপূর্ণ ফিচারে Canary — এই মিশ্রণটিই বাস্তবসম্মত।

১৬.৬ Blue-Green Deployment — বিস্তারিত কার্যপ্রণালী

Blue-Green-এ প্রোডাকশনে দুটি একরকম পরিবেশ থাকে: **Blue** (বর্তমানে ট্রাফিক পাচ্ছে) ও **Green** (নতুন সংস্করণ, প্রস্তুত কিন্তু ট্রাফিক পাচ্ছে না)। নতুন সংস্করণ Green-এ ডেপ্লয় হয়, সেখানে স্বাস্থ্য পরীক্ষা, Smoke Test ও কিছু বাস্তব অনুরোধ (Synthetic Request) চালানো হয়। সব ঠিক থাকলে লোড ব্যালান্সার বা Kubernetes Service-এর Selector বদলে ট্রাফিক Green-এ সরানো হয়। কোনো সমস্যা দেখা দিলে একইভাবে ট্রাফিক আবার Blue-তে ফিরিয়ে নেওয়া হয় — কোড পুনরায় ডেপ্লয় করতে হয় না, শুধু রাউটিং বদলাতে হয়, তাই রোলব্যাক হয় সেকেন্ডে।



Blue-Green-এর কঠিন অংশটি কোড নয়, ডেটা। অ্যাপ্লিকেশন কোড দুটি সংস্করণে ভাগ করা গেলেও ডেটাবেস একটি। নতুন সংস্করণের জন্য ডেটাবেস স্কিমা বদলাতে হলে তা এমনভাবে বদলাতে হয় যাতে পুরোনো সংস্করণও সেই স্কিমায় চলতে পারে — একে বলে **Expand-and-Contract** প্যাটার্ন: প্রথমে নতুন কলাম যোগ করা (Expand), পুরোনো কোড সরিয়ে নতুন কোড চালানো, তারপর পুরোনো কলাম সরানো (Contract)। স্কিমা বদল না ভেবে Blue-Green করলে রোলব্যাকের সময় ডেটা অসামঞ্জস্য হতে পারে — এটাই সবচেয়ে সাধারণ বিপদ।

> **WARNING:** Blue-Green-এ পুরোনো পরিবেশটি (Blue) ট্রাফিক সরানোর সাথে সাথে মুছে ফেলবেন না। অন্তত একটি রিলিজ-চক্র বা যথেষ্ট পর্যবেক্ষণ সময় ধরে রাখুন। মুছে ফেললে রোলব্যাকের পথটিও মুছে যায় — আর ঠিক তখনই রোলব্যাক দরকার হয়।

১৬.৭ Canary Release ও Progressive Delivery

Canary Release-এর নাম এসেছে কয়লা খনির ক্যানারি পাখি থেকে — বিপদ আগে ছোট পরিসরে ধরা। নতুন সংস্করণ প্রথমে অল্প ট্রাফিক পায় — ধরুন ৫% — আর পর্যবেক্ষণ ব্যবস্থা দুটি সংস্করণের মেট্রিক তুলনা করে: ত্রুটির হার (Error Rate), প্রতিক্রিয়ার সময় (Latency), সফল অনুরোধের হার। মেট্রিক ভালো থাকলে ট্রাফিক ২৫%, ৫০%, তারপর ১০০%-এ বাড়ে। খারাপ হলে স্বয়ংক্রিয়ভাবে ট্রাফিক ফিরিয়ে নেওয়া হয়। এই ধাপে ধাপে, মেট্রিক-চালিত রিলিজই **Progressive Delivery**।

Progressive Delivery শুধু Canary নয় — এর আওতায় পড়ে Feature Flag দিয়ে নির্দিষ্ট ব্যবহারকারী গোষ্ঠীকে ফিচার দেখানো, A/B পরীক্ষা, এবং ভৌগোলিক ধাপে ধাপে রিলিজ। মূল নীতিটি এক: রিলিজ একটি একক ঘটনা নয়, একটি নিয়ন্ত্রিত প্রক্রিয়া। প্রতিটি ধাপে প্রমাণ দেখে পরের ধাপে যাওয়া হয়।

১৬.৭.১ Argo Rollouts ও Flagger

সাধারণ Kubernetes Deployment-এ Canary-র ধাপগুলো হাতে পরিচালনা করতে হয়। **Argo Rollouts** ও **Flagger** এই কাজটি স্বয়ংক্রিয় করে। Argo Rollouts একটি কাস্টম রিসোর্স দেয় — Rollout — যেখানে আপনি ধাপ লেখেন (৫% ট্রাফিক, ১০ মিনিট অপেক্ষা, মেট্রিক বিশ্লেষণ, ৫০% ট্রাফিক...), আর কন্ট্রোলার সেই ধাপগুলো চালায়, মেট্রিক দেখে,

প্রয়োজনে থামে বা ফিরিয়ে নেয়। Flagger একই কাজ করে GitOps ধারায়: Git-এ লেখা Canary সংজ্ঞা পড়ে সে স্বয়ংক্রিয়ভাবে নতুন সংস্করণ পরীক্ষা করে, ট্রাফিক বাড়ায়, আর মেট্রিক খারাপ হলে রোলব্যাক করে। দুটির মধ্যে বেছে নেওয়ার প্রশ্নটি মূলত আপনার পরিবেশের: Argo পরিবেশে থাকলে Argo Rollouts স্বাভাবিক; Flux/Flagger পরিবেশে থাকলে Flagger স্বাভাবিক। টুল বদলানোর চেয়ে যে টুলটি আপনার দল চালাতে পারে, সেটি বেছে নেওয়াই বুদ্ধিমানের কাজ।

```
# Argo Rollouts — ধাপে ধাপে Canary (সংক্ষিপ্ত ধারণা, YAML)
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: web
  namespace: technova-prod
spec:
  replicas: 4
  strategy:
    canary:
      steps:
        - setWeight: 5
        - pause: { duration: 10m }
        - setWeight: 25
        - pause: { duration: 10m }
        - setWeight: 50
        - pause: { duration: 10m }
      analysis:
        templates:
          - templateName: success-rate
        args:
          - name: service-name
            value: web
  template:
    spec:
      containers:
        - name: web
          image: registry.technova.example/web:1.5.0
```

উপরের ধাপগুলো পড়ে বোঝা যায়: ৫% ট্রাফিক দিয়ে শুরু, দশ মিনিট পর্যবেক্ষণ, তারপর ২৫%, আবার পর্যবেক্ষণ, তারপর ৫০%। `analysis` অংশটি বলে কোন মেট্রিক দেখে সিদ্ধান্ত নেওয়া হবে — এখানে সফল অনুরোধের হার। কোনো ধাপে বিশ্লেষণ ব্যর্থ হলে Rollout থেমে যায় এবং পুরোনো সংস্করণে ফিরে যায়।

১৬.৮ Feature Flag — ডেপ্লয় ও রিলিজ আলাদা করা

Feature Flag (Feature Toggle): কোডে একটি শর্ত — একটি ফিচার চালু হবে কি না, তা কোডের সংস্করণ দিয়ে নয়, একটি কনফিগারেশন মান দিয়ে নির্ধারিত হয়। ফ্ল্যাগ বন্ধ থাকলে নতুন কোড প্রোডাকশনে থাকলেও অদৃশ্য; ফ্ল্যাগ চালু করলে

তা ব্যবহারকারীর কাছে আসে — কোনো ডেপ্লয় ছাড়াই, সেকেন্ডে। এতে তিনটি বড় সুবিধা হয়: অসম্পূর্ণ ফিচার main শাখায় মার্জ করা যায় (Trunk-Based Development সহজ হয়), রিলিজ ধাপে ধাপে করা যায় (প্রথমে অভ্যন্তরীণ ব্যবহারকারী, তারপর ১০%, তারপর সবাই), আর কোনো সমস্যা হলে ফ্ল্যাগ বন্ধ করেই রোলব্যাক — ডেপ্লয় নয়।

```
// Feature Flag — ধারণা (সরল উদাহরণ)
if (flags.newCheckoutEnabled(user)) {
  renderNewCheckout();
} else {
  renderOldCheckout();
}

// ফ্ল্যাগের নিয়ম বাইরে থেকে বদলানো যায়:
//  - সব ব্যবহারকারীর জন্য বন্ধ
//  - শুধু technova.example কর্মীদের জন্য চালু
//  - 10% ব্যবহারকারীর জন্য চালু (ধাপে ধাপে রিলিজ)
```

তবে Feature Flag-এরও দায় আছে। ফ্ল্যাগ জমতে থাকলে কোডে শর্তের জঙ্গল তৈরি হয় — একে বলে **Flag Debt**। প্রতিটি ফ্ল্যাগের একটি মেয়াদ থাকা উচিত: রিলিজ সম্পূর্ণ হলে পুরোনো পথের কোড ও ফ্ল্যাগ সরিয়ে ফেলা হয়। TechNova-তে নিয়মটি এমন হতে পারে — কোনো ফ্ল্যাগ ত্রিশ দিনের বেশি জীবিত থাকবে না, যদি না তার মালিক লিখিতভাবে মেয়াদ বাড়ান। আর ফ্ল্যাগের মান বদলানোর অনুমতিও নিয়ন্ত্রিত হওয়া উচিত: যে কেউ যেকোনো সময় প্রোডাকশন ফ্ল্যাগ বদলাতে পারলে, সেটি একটি অপরীক্ষিত ডেপ্লয়ের সমান।

> **REAL WORLD:** TechNova একবার বড় চেকআউট পরিবর্তন একবারে সব ব্যবহারকারীর জন্য চালু করেছিল — ভুল ধরা পড়েছিল এক ঘণ্টা পর, ততক্ষণে অনেক অর্ডার আটকে গেছে। পরের বার একই পরিবর্তন Feature Flag দিয়ে প্রথমে কর্মীদের, তারপর ৫% ব্যবহারকারীর জন্য চালু করা হয়েছিল — ভুল ধরা পড়েছিল পাঁচ মিনিটে, ক্ষতি হয়েছিল নগণ্য, আর ফ্ল্যাগ বন্ধ করেই সমস্যা মেটানো গিয়েছিল।

১৬.৯ স্বয়ংক্রিয় রোলব্যাক ও রিলিজ যাচাই (Release Verification)

রোলব্যাক মানে “আগের সংস্করণে ফেরা” — কিন্তু প্রোডাকশনে ফেরার পথটি আগে থেকে পরীক্ষা করা না থাকলে সেটি নিজেই একটি ঝুঁকিপূর্ণ ডেপ্লয়। স্বয়ংক্রিয় রোলব্যাকের জন্য তিনটি জিনিস লাগে: কী দেখে ফিরতে হবে, তার সীমা (Threshold); কে সিদ্ধান্ত নেবে — মানুষ না যন্ত্র; এবং ফেরার প্রক্রিয়াটি পরীক্ষিত কি না। সাধারণ সীমাগুলো হলো ত্রুটির হার একটি নির্দিষ্ট মাত্রা ছাড়িয়ে যাওয়া, প্রতিক্রিয়ার সময় অস্বাভাবিক বেড়ে যাওয়া, বা স্বাস্থ্য পরীক্ষা (Health Check) বারবার ব্যর্থ হওয়া। Argo Rollouts ও Flagger-এ এই সীমা বিশ্লেষণ টেমপ্লেটে লেখা থাকে; সীমা ছাড়ালে তারা নিজেরাই ট্রাফিক ফিরিয়ে নেয়।

Release Verification হলো রিলিজের পর একটি আনুষ্ঠানিক যাচাই: Smoke Test (মূল পথগুলো চলে কি না), মেট্রিক তুলনা (আগের সংস্করণের সাথে), লগে নতুন ত্রুটির ধরন আছে কি না, এবং ব্যবসায়িক মেট্রিক (যেমন সফল অর্ডারের হার) স্বাভাবিক কি না। এই যাচাই পাইপলাইনের অংশ হওয়া উচিত — মানুষের স্মৃতিতে নয়। একটি ভালো অভ্যাস হলো প্রতিটি রিলিজের জন্য একটি স্বয়ংক্রিয় যাচাই-তালিকা চালানো এবং তার ফলাফল রিলিজ নোটে সংরক্ষণ করা, যাতে পরে কেউ প্রশ্ন করলে প্রমাণ থাকে।

```
# রিলিজের পর দ্রুত যাচাই — TechNova web (উদাহরণ কমান্ড)
kubectl -n technova-prod rollout status deployment/web
kubectl -n technova-prod get pods -l app=web
curl -s -o /dev/null -w "%{http_code}\n" https://app.technova.example/healthz
# মেট্রিক দেখুন: ত্রুটির হার, p95 লেটেন্সি — আগের সংস্করণের সাথে তুলনা করুন
# সব ঠিক থাকলে রিলিজ নোটে লিখুন: সংস্করণ, সময়, যাচাইয়ের ফলাফল
```

১৬.১০ TechNova প্রোডাকশন দৃশ্যকল্প

TechNova-র web অ্যাপে একটি নতুন পেমেন্ট ফ্লো আসছে। ঝুঁকি বেশি — পেমেন্ট ভুল হলে টাকা ও বিশ্বাস দুটোই যায়। পরিকল্পনাটি এমন: (১) ফিচারটি Feature Flag-এর আড়ালে main শাখায় মার্জ করা হয়, ফ্ল্যাগ বন্ধ অবস্থায়; (২) CI পাইপলাইন পরীক্ষা ও নিরাপত্তা স্ক্যান চালায়, ইমেজ প্রকাশ করে; (৩) Staging-এ ফ্ল্যাগ চালু করে পূর্ণ পরীক্ষা হয়; (৪) প্রোডাকশনে Blue-Green-এ ডেপ্লয় হয়, ফ্ল্যাগ তখনও বন্ধ — অর্থাৎ নতুন কোড প্রোডাকশনে আছে কিন্তু অদৃশ্য; (৫) পরের দিন সকালে ফ্ল্যাগ শুধু অভ্যন্তরীণ ব্যবহারকারীদের জন্য চালু হয়; (৬) মেট্রিক ভালো থাকলে ৫%, তারপর ২৫%, তারপর সবার জন্য। প্রতিটি ধাপে স্বয়ংক্রিয় যাচাই চলে; কোনো ধাপে ত্রুটির হার বাড়লে ফ্ল্যাগ বন্ধ হয় — ডেপ্লয় নয়। এই পরিকল্পনায় ডেপ্লয়ের ঝুঁকি ও রিলিজের ঝুঁকি আলাদা করা হয়েছে, আর প্রতিটি ধাপ ফিরে যাওয়ার পথসহ নকশা করা হয়েছে।

১৬.১১ নিরাপত্তা ও খরচ বিবেচনা

পাইপলাইন নিজেই একটি নিরাপত্তা-সংবেদনশীল ব্যবস্থা: এটি প্রোডাকশনে কোড পৌঁছে দেয়, তাই এতে যে সিক্রেট (ক্লাউড শংসাপত্র, রেজিস্ট্রি পাসওয়ার্ড) থাকে, তা সুরক্ষিত রাখতে হয় — কখনোই পাইপলাইন ফাইলে বা লগে লিখে নয়। প্রতিটি পরিবেশের অনুমতি আলাদা হওয়া উচিত: Staging-এ ডেপ্লয় করার অনুমতি থাকলেই প্রোডাকশনে ডেপ্লয় করার অনুমতি থাকবে, তা নয়। প্রোডাকশন ডেপ্লয়ে অনুমোদন (Approval) গেট রাখা উচিত, আর কে অনুমোদন দিলেন তার রেকর্ড থাকা উচিত। খরচের দিক থেকে Blue-Green দ্বিগুণ সম্পদ চায় — ছোট দলের জন্য তা ভারী হতে পারে; Canary-তে ট্রাফিক ভাগ ও পর্যবেক্ষণের জটিলতা বাড়ে, কিন্তু সম্পদ দ্বিগুণ লাগে না। Feature Flag সার্ভিসের জন্য আলাদা ব্যবস্থা লাগতে পারে — নিজে বানানো না কিনে নেওয়া, এই সিদ্ধান্তটি দলের আকার ও প্রয়োজনে নির্ভর করে।

১৬.১২ সাধারণ ভুল

- **প্রতিটি পরিবেশে নতুন করে বিল্ড করা:** একই আর্টিফ্যাক্ট প্রচার না করে প্রতিবার নতুন বিল্ড করলে প্রোডাকশনে কী চলছে তা নিশ্চিত থাকে না।
- **পরীক্ষা ছাড়া Canary:** মেট্রিক ও সীমা ছাড়া ট্রাফিক ভাগ করলে Canary শুধু ধীর Rolling-এ পরিণত হয়।
- **ফ্ল্যাগ জমিয়ে রাখা:** মেয়াদোত্তীর্ণ ফ্ল্যাগ কোডকে জটিল করে; রিলিজ শেষে ফ্ল্যাগ সরান।
- **রোলব্যাক কখনো পরীক্ষা না করা:** রোলব্যাকের পথ পরীক্ষা না থাকলে জরুরি মুহূর্তে তা কাজ নাও করতে পারে।
- **ডেটাবেস বদল ভুলে যাওয়া:** কোডের কৌশল যতই ভালো হোক, স্কিমা বদল সামঞ্জস্যপূর্ণ না হলে রিলিজ ভাঙবে।
- **পাইপলাইনে সিক্রেট লেখা:** সিক্রেট পাইপলাইন ফাইলে বা লগে এলে তা ঘুরে বেড়ায়; সিক্রেট ম্যানেজার ব্যবহার করুন।

১৬.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়

পাইপলাইন ধীর, ডেভেলপাররা অপেক্ষা করে	ধীর পরীক্ষা আগে চলছে; ক্যাশ নেই	দ্রুত পরীক্ষা আগে রাখুন; নির্ভরতা ক্যাশ করুন; পরীক্ষা সমান্তরালে চালান
Canary-তে ত্রুটি বাড়লেও ট্রাফিক বাড়ছে	বিশ্লেষণ টেমপ্লেট বা সীমা বসানো হয়নি	মেট্রিক-ভিত্তিক বিশ্লেষণ ও স্বয়ংক্রিয় থামার নিয়ম যোগ করুন
রোলব্যাকের পরও পুরোনো ভুল আচরণ	ডেটাবেস স্কিমা ফেরানো হয়নি	Expand-and-Contract অনুসরণ করুন; স্কিমা রোলব্যাক পরিকল্পনা আগে লিখুন
একই কোড Staging-এ চলে, প্রোডাকশনে ভাঙে	পরিবেশভেদে ভিন্ন আর্টিফ্যাক্ট বা কনফিগারেশন	একই আর্টিফ্যাক্ট প্রচার করুন; কনফিগারেশনের পার্থক্য তালিকাভুক্ত করুন
ফ্ল্যাগ চালু করেও ফিচার দেখা যায় না	ফ্ল্যাগের নিয়ম বা ক্যাশ পুরোনো	ফ্ল্যাগের লক্ষ্য নিয়ম ও ক্যাশের মেয়াদ পরীক্ষা করুন

১৬.১৪ প্রকৌশল ট্রেড-অফ: GitOps বনাম প্রচলিত CI/CD

GitOps মানে Git-ই প্রোডাকশনের কাঙ্ক্ষিত অবস্থার উৎস, আর ক্লাস্টারের ভেতরের একটি এজেন্ট (যেমন Argo CD) ক্রমাগত Git-এর সাথে বাস্তব অবস্থা মিলিয়ে চলে। প্রচলিত CI/CD-তে পাইপলাইন নিজে ক্লাস্টারে `kubectl apply` করে — অর্থাৎ পাইপলাইনের হাতে ক্লাস্টারের চাবি থাকে। GitOps-এর সুবিধা: পরিবর্তনের ইতিহাস Git-এ, রোলব্যাক মানে Git-এ ফিরে যাওয়া, আর পাইপলাইনের হাতে ক্লাস্টারের সরাসরি অনুমতি থাকে না। অসুবিধা: নতুন ধারণা শিখতে হয়, আর Git-এর বাইরে কেউ হাতে কিছু বদলালে তা মুছে যাবে — যা কখনো কখনো জরুরি হস্তক্ষেপকে কঠিন করে। ছোট দল বা একক অ্যাপের জন্য প্রচলিত CI/CD যথেষ্ট ও সহজ; একাধিক দল, একাধিক ক্লাস্টার ও কঠোর অডিটের প্রয়োজনে GitOps-এর শৃঙ্খলা মূল্য দেয়। সহজ পথটি যখন কাজ করে, তখন জটিল পথে যাওয়া প্রকৌশল নয়, বিলাসিতা।

> **TROUBLESHOOTING:** রিলিজের সময় সবচেয়ে দরকারি প্রশ্নটি হলো — “আগের সংস্করণে ফিরতে কতক্ষণ লাগবে, আর সেই পথটি কি গত সপ্তাহে পরীক্ষা করা হয়েছে?” উত্তরটি যদি “জানি না” হয়, তবে রিলিজের আগে রোলব্যাক পরীক্ষা করুন — প্রোডাকশনে নয়, Staging-এ।

১৬.১৫ অধ্যায়ের সারাংশ

উন্নত CI/CD মানে শুধু দ্রুত পাইপলাইন নয় — মানে পুনঃব্যবহারযোগ্য পাইপলাইন, একই আর্টিফ্যাক্টের প্রচার, ডেপ্লয় ও রিলিজের পৃথকীকরণ, এবং মেট্রিক-চালিত ধাপে ধাপে রিলিজ। Rolling দৈনন্দিন কাজে, Blue-Green বড় ও ঝুঁকিপূর্ণ রিলিজে, Canary বাস্তব ট্রাফিকে ছোট আকারে পরীক্ষায় — প্রতিটি কৌশলের নিজের জায়গা আছে। Feature Flag ডেপ্লয় থেকে রিলিজকে আলাদা করে, Argo Rollouts ও Flagger Canary-কে স্বয়ংক্রিয় করে, আর স্বয়ংক্রিয় রোলব্যাক ও রিলিজ যাচাই রিলিজকে অনুমানযোগ্য করে। GitOps বনাম প্রচলিত CI/CD-র সিদ্ধান্তটি দলের আকার, অডিটের প্রয়োজন ও জটিলতা সহ্য করার ক্ষমতার উপর নির্ভর করে।

১৬.১৬ জ্ঞান যাচাই

1. Continuous Delivery ও Continuous Deployment-এর পার্থক্য ব্যাখ্যা করুন।
2. Artifact Promotion কেন গুরুত্বপূর্ণ? প্রতিটি পরিবেশে নতুন করে বিল্ড করলে কী সমস্যা হয়?
3. Rolling, Blue-Green ও Canary — তিনটি কৌশলের মধ্যে কোনটি কখন বেছে নেবেন? উদাহরণসহ লিখুন।
4. Blue-Green-এ ডেটাবেস স্কিমা বদল কেন জটিল? Expand-and-Contract কীভাবে সাহায্য করে?

5. Feature Flag কীভাবে ডেপ্লয় ও রিলিজকে আলাদা করে? Flag Debt কী?
6. Progressive Delivery বলতে কী বোঝায়? Canary কি এর একমাত্র রূপ?
7. Argo Rollouts ও Flagger কী সমস্যা সমাধান করে? সাধারণ Deployment দিয়ে Canary করতে কী অসুবিধা?
8. স্বয়ংক্রিয় রোলব্যাকের জন্য কোন তিনটি জিনিস আগে থেকে ঠিক থাকা দরকার?
9. GitOps বনাম প্রচলিত CI/CD — কখন কোনটি ভালো? যুক্তি দিন।
10. একটি রিলিজ যাচাই-তালিকায় কী কী থাকা উচিত?

১৬.১৭ ব্যবহারিক অনুশীলন

1. TechNova-র web অ্যাপের জন্য একটি পাইপলাইন নকশা আঁকুন: Commit, Acceptance, Security ও Deploy স্তর, প্রতিটি স্তরে কী পরীক্ষা চলবে, এবং কোন স্তরে অনুমোদন গেট থাকবে — তা লিখুন। তারপর ব্যাখ্যা করুন কেন এই ক্রমটি বেছে নিলেন।
2. একটি কাল্পনিক Canary পরিকল্পনা লিখুন: ৫% → ২৫% → ৫০% → ১০০% ধাপ, প্রতিটি ধাপে কোন মেট্রিক দেখবেন, কোন সীমা ছাড়ালে থামবেন, আর থামলে কী করবেন।
3. একটি Feature Flag-এর জীবনচক্রের পরিকল্পনা করুন: ফ্ল্যাগটি কখন চালু হবে, কাদের জন্য, কখন সরানো হবে, এবং কে তার মালিক থাকবেন — একটি ছোট নথি লিখুন।
4. Staging পরিবেশে একটি Rolling ডেপ্লয় করে তারপর রোলব্যাক পরীক্ষা করুন (বা পরীক্ষার ধাপগুলো লিখুন): রোলব্যাকে কত সময় লাগল, কী কী ধাপ লাগল, আর কী শিখলেন — তা নথিভুক্ত করুন।

অধ্যায় ১৭ • PART IV — Advanced Automation & Enterprise Operations

Chapter 17 — Platform as Code and Automation Architecture

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Platform as Code** বলতে কী বোঝায় তা ব্যাখ্যা করতে; Terraform মডিউল ও প্রোভাইডার অ্যাবস্ট্রাকশন দিয়ে পুনঃব্যবহারযোগ্য অবকাঠামো নকশা করতে; Ansible অটোমেশন, **Crossplane** ও Kubernetes অপারেটরের ধারণা ও ব্যবহারের জায়গা চিহ্নিত করতে; প্ল্যাটফর্ম API, স্টেট ব্যবস্থাপনা, **Drift Detection**, অটোমেশন পরীক্ষা ও অটোমেশন গভর্ন্যান্স TechNova-র প্রেক্ষাপটে প্রয়োগ করতে।

- Terraform মডিউলের গঠন, সংস্করণ ও প্রকাশের নিয়ম ব্যাখ্যা করা।
- Ansible, Crossplane ও অপারেটর — তিনটি অটোমেশন ধারার মধ্যে সঠিকটি বেছে নেওয়া।
- স্টেট, Drift ও গভর্ন্যান্সের জন্য বাস্তবসম্মত নীতি লেখা।
- একটি প্ল্যাটফর্ম API-র ন্যূনতম নকশা তৈরি করা।

১৭.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

আগের ভলিউমগুলো থেকে আপনার Terraform-এর Resource, Variable, Output ও State-এর ধারণা, Git-এ কোড রাখা ও Pull Request-এ পর্যালোচনা, এবং Kubernetes-এর মৌলিক ধারণা — Pod, Deployment, Service — জানা থাকা দরকার। **Module** মানে Terraform কোডের একটি পুনঃব্যবহারযোগ্য প্যাকেজ; **Provider** মানে Terraform যার সাথে কথা বলে — যেমন একটি ক্লাউড বা Kubernetes। **Operator** মানে Kubernetes-এর ভেতরে চলা একটি নিয়ন্ত্রক (Controller), যা একটি কাস্টম রিসোর্স দেখে বাস্তব অবস্থাকে কাঙ্ক্ষিত অবস্থার দিকে নিয়ে যায়। এই অধ্যায়ে আমরা ধরে নিচ্ছি TechNova ইতিমধ্যে Terraform দিয়ে অবকাঠামো বানায়, কিন্তু প্রতিটি দল আলাদা আলাদাভাবে একই জিনিস লিখছে।

> **CONCEPT:** Platform as Code মানে শুধু অবকাঠামো কোডে লেখা নয় — মানে পুরো প্ল্যাটফর্মটিকেই কোড হিসেবে দেখা: নেটওয়ার্ক, ক্লাস্টার, ডেটাবেস, পর্যবেক্ষণ, নিরাপত্তা নীতি, এমনকি দলের জন্য পরিবেশ পাওয়ার পথটিও। লক্ষ্য একটাই — প্ল্যাটফর্মের যেকোনো অংশ যেন কোড থেকে পুনর্নির্মাণ করা যায়, পরীক্ষা করা যায় ও পর্যালোচনা করা যায়।

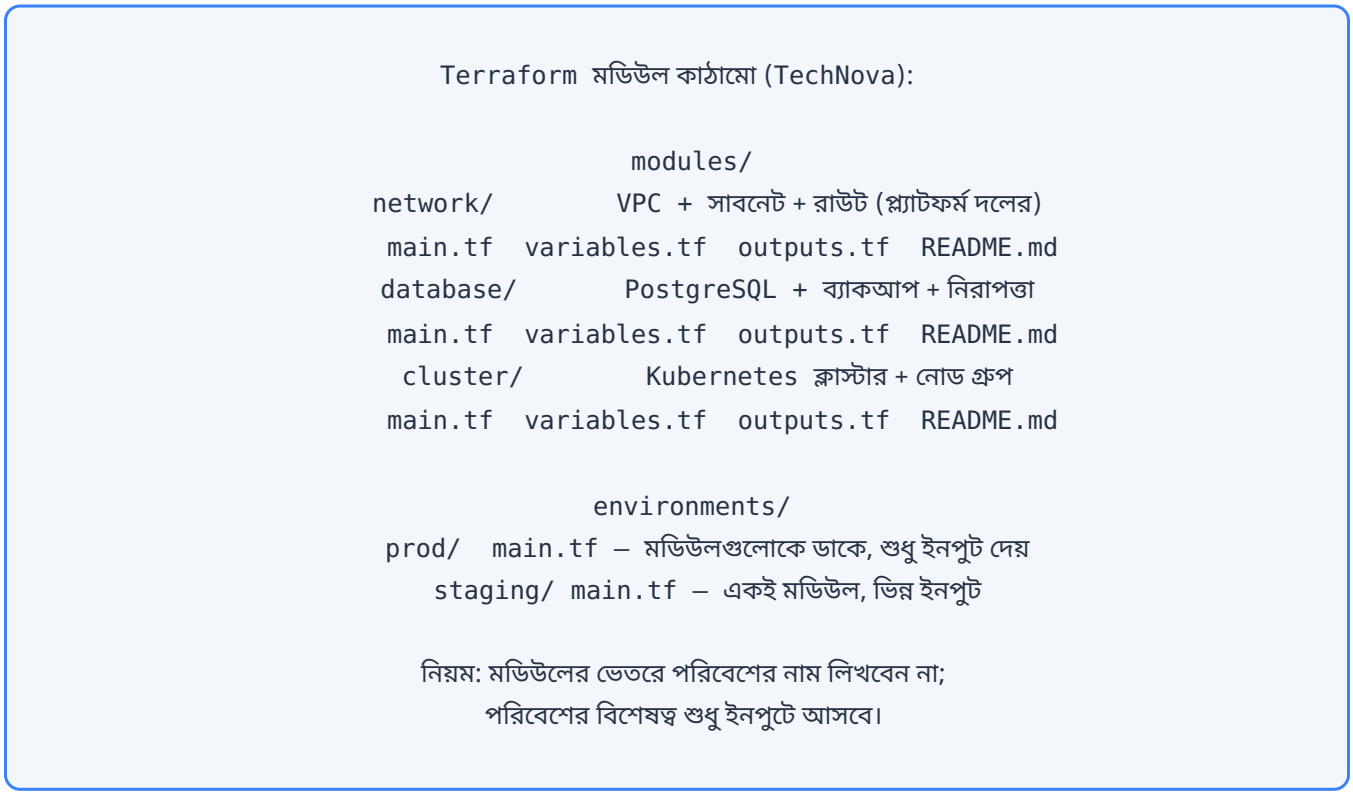
১৭.৩ প্রথম নীতি থেকে: অটোমেশন কেন কাঠামো চায়

অটোমেশন শুরু হয় একটি স্ক্রিপ্ট দিয়ে — একটি কাজ, একবার, একজন মানুষের জন্য। সমস্যা শুরু হয় যখন স্ক্রিপ্টটি দশজন ব্যবহার করে, দশ জায়গায় কপি হয়, আর প্রতিটি কপি সামান্য আলাদা হয়ে যায়। তখন অটোমেশন আর সময় বাঁচায় না; বরং নতুন ধরনের বিশৃঙ্খলা তৈরি করে — কেউ জানে না কোন কপিটি সঠিক, কোনটি পুরোনো। এই সমস্যার উত্তর হলো কাঠামো: সাধারণ জিনিস এক জায়গায়, বিশেষ জিনিস আলাদা জায়গায়, আর দুটোর মধ্যে একটি পরিষ্কার চুক্তি (Interface)। Terraform মডিউল, Ansible Role, Kubernetes Operator — তিনটিই এই একই নীতির ভিন্ন ভিন্ন রূপ: পুনঃব্যবহারযোগ্য একক, পরিষ্কার ইনপুট-আউটপুট, আর সংস্করণ।

দ্বিতীয় নীতিটি হলো — অটোমেশন যত শক্তিশালী, তার ভুল তত দ্রুত ছড়ায়। হাতে একটি সার্ভার ভুলভাবে বানালে ক্ষতি একটি সার্ভারে সীমাবদ্ধ; অটোমেশনে একটি ভুল মডিউল দিয়ে একশো সার্ভার বানালে ক্ষতি একশো সার্ভারে। তাই অটোমেশন আর্কিটেকচারে পরীক্ষা ও গভর্ন্যান্স কোনো বিলাসিতা নয় — এগুলো অটোমেশনেরই অংশ।

১৭.৪ Terraform মডিউল: পুনঃব্যবহারের মূল একক

Terraform মডিউল হলো একগুচ্ছ Resource-এর একটি প্যাকেজ, যার নিজের ইনপুট (Variable) ও আউটপুট (Output) আছে। TechNova-তে প্রতিটি দল নিজের মতো VPC বানানোর বদলে, প্ল্যাটফর্ম দল একটি `network` মডিউল বানায় — ভেতরে VPC, সাবনেট, রাউট টেবিল, নিরাপত্তা নিয়ম — আর দলগুলো শুধু ইনপুট দেয়: পরিবেশের নাম, ঠিকানা পরিসর, অঞ্চল। মডিউলটি ভালো হলে দলগুলো ভেতরের জটিলতা না জেনেই নিরাপদ নেটওয়ার্ক পায়; খারাপ হলে মডিউলটি একটি কালো বাক্স হয়ে যায়, যার ভেতরে কী আছে কেউ বোঝে না।



১৭.৪.১ মডিউল নকশার নিয়ম

ভালো মডিউলের চারটি গুণ থাকে। প্রথমত, **একক দায়িত্ব** — একটি মডিউল একটি জিনিস করে: নেটওয়ার্ক মডিউল ডেটাবেস বানায় না। দ্বিতীয়ত, **পরিষ্কার ইনপুট** — প্রতিটি Variable-এর বর্ণনা, ধরন ও পূর্বনির্ধারিত মান (Default) থাকে; সংবেদনশীল ইনপুট চিহ্নিত থাকে। তৃতীয়ত, **সংস্করণ** — মডিউলের প্রতিটি প্রকাশের একটি সংস্করণ থাকে, আর পরিবেশগুলো একটি নির্দিষ্ট সংস্করণে আটকে (Pin) থাকে; `latest` ব্যবহার করা মানে অপরীক্ষিত পরিবর্তন গ্রহণ করা। চতুর্থত, **নথি ও উদাহরণ** — README-তে কী ইনপুট দিতে হয়, কী আউটপুট পাওয়া যায়, আর একটি সম্পূর্ণ ব্যবহারের উদাহরণ থাকে। নথি ছাড়া মডিউল ব্যবহার হয় না; ব্যবহার না হলে মডিউল লেখার কোনো মূল্য নেই।

```
# মডিউল ব্যবহার — পরিবেশের কোড (সংক্ষিপ্ত ধারণা)
module "network" {
  source  = "git::https://git.technova.example/platform/modules.git//network?ref=v2.3.0
  env     = "prod"
  cidr    = "203.0.113.0/24"
  region  = "ap-south-1"
}

module "database" {
```

```
  source            = "git::https://git.technova.example/platform/modules.git//database
  env               = "prod"
  admin_password    = var.db_password    # সিক্রেট: কখনো কোডে লিখবেন না
  backup_retention  = 7
}

# ref=v2.3.0 — সংস্করণে আটকে রাখা; নতুন সংস্করণ
# পরীক্ষা করে, আলাদা Pull Request-এ নেওয়া হয়।
```

১৭.৫ প্রোভাইডার অ্যাবস্ট্রাকশন: ক্লাউডের উপরে একটি স্তর

Provider Abstraction মানে দলের কোড যাতে সরাসরি কোনো নির্দিষ্ট ক্লাউডের বিশেষত্বের উপর নির্ভর না করে, তার জন্য মাঝখানে একটি স্তর রাখা। উদাহরণ: দল চায় "একটি ডেটাবেস" — সে যেন AWS-এর RDS, Azure-র Database না GCP-র Cloud SQL — কোনটির নাম না জেনেই চাইতে পারে। মডিউল সেই অনুরোধটি নির্দিষ্ট ক্লাউডের Resource-এ অনুবাদ করে। এতে ভবিষ্যতে ক্লাউড বদলানো সহজ হয় — অন্তত তত্ত্বে। বাস্তবে সম্পূর্ণ বদলানো কঠিন, কারণ প্রতিটি ক্লাউডের নিজস্ব সেবা, সীমা ও দামের কাঠামো আলাদা। অ্যাবস্ট্রাকশনের আসল মূল্য তাই বদলানোর স্বাধীনতায় নয়, ধারাবাহিকতায়: সব দল একইভাবে ডেটাবেস চায়, একই ব্যাকআপ নীতি পায়, একই ট্যাগিং পায় — ক্লাউড যেটিই হোক। তবে মনে রাখবেন, অ্যাবস্ট্রাকশন যত পুরু, তত বেশি ক্লাউডের নিজস্ব সুবিধা লুকিয়ে যায়। যে দলের ক্লাউডের বিশেষ কোনো সুবিধা দরকার, তার জন্য অ্যাবস্ট্রাকশন ভেদ করে নিচে যাওয়ার একটি পথ (Escape Hatch) রাখা উচিত — নইলে দলগুলো অ্যাবস্ট্রাকশন এড়িয়ে হাতে কাজ করতে শুরু করবে।

১৭.৬ Ansible অটোমেশন: সার্ভারের ভেতরের অবস্থা

Terraform সম্পদ তৈরি করে; কিন্তু সার্ভারের ভেতরে কী থাকবে — কোন প্যাকেজ, কোন ফাইল, কোন সার্ভিস চলবে — তা **Configuration Management**-এর কাজ, আর Ansible তার সবচেয়ে পরিচিত টুল। Ansible-এ কাজ লেখা হয় **Playbook**-এ (YAML), ধাপগুলোকে বলে **Task**, আর পুনঃব্যবহারযোগ্য এককে বলে **Role**। Ansible-এর বড় সুবিধা: লক্ষ্য সার্ভারে কোনো এজেন্ট লাগে না — SSH দিয়ে কাজ করে। TechNova-তে Ansible ব্যবহার হতে পারে এমন জায়গায় যেখানে Kubernetes নেই — যেমন একটি পুরোনো VM-এ লগ সংগ্রহের এজেন্ট বসানো, বা একটি ডেটাবেস সার্ভারের অপারেটিং সিস্টেম শক্ত করা (Hardening)।

```
# Ansible Playbook — ধারণা (TechNova লগ এজেন্ট)
- name: Install log agent on app servers
  hosts: app_servers
  become: true
  roles:
    - log-agent
# Role-এর ভেতরে: প্যাকেজ ইনস্টল, কনফিগ ফাইল, সার্ভিস চালু —
# প্রতিটি ধাপ Idempotent: দ্বিতীয়বার চালালে কিছু বদলায় না।
# ansible-playbook -i inventory/prod site.yml --check
#   --check দিয়ে আগে দেখুন কী বদলাবে (Dry Run)।
```

Ansible-এর সীমাও জানা দরকার: এটি Terraform-এর মতো State রাখে না — বাস্তব অবস্থা কী, তা সে প্রতিবার পরীক্ষা করে বোঝে। বড় সংখ্যক সার্ভারে এটি ধীর হতে পারে, আর জটিল অর্কেস্ট্রেশনে Playbook জটিল হয়ে যায়। তাই নিয়মটি

হলো — সম্পদ তৈরিতে Terraform, সার্ভারের ভেতরের কনফিগারেশনে Ansible, আর কনটেইনার ওয়ার্কলোডে Kubernetes — প্রতিটি টুল তার নিজের জায়গায়।

১৭.৭ Crossplane ও Kubernetes অপারেটর: Kubernetes-ই নিয়ন্ত্রণ কেন্দ্র

Crossplane হলো Kubernetes-এর একটি এক্সটেনশন, যা ক্লাউড সম্পদকেও Kubernetes রিসোর্সের মতো করে দেখায়। আপনি Kubernetes-এ একটি `PostgreSQLInstance` নামের রিসোর্স লিখলে Crossplane বাস্তব ক্লাউডে ডেটাবেস বানিয়ে দেয়, আর তার অবস্থা Kubernetes-এ দেখায়। এতে ডেভেলপারদের আলাদা Terraform শিখতে হয় না — তাঁরা Kubernetes API দিয়েই অবকাঠামো চাইতে পারেন। **Composition** নামের ব্যবস্থায় প্ল্যাটফর্ম দল একটি উচ্চ-স্তরের রিসোর্স (যেমন `AppDatabase`) সংজ্ঞায়িত করে, যার ভেতরে ক্লাউড-নির্দিষ্ট বিবরণ লুকানো থাকে।

Kubernetes Operator একই ধারণার আরেকটি রূপ: একটি কাস্টম রিসোর্স (Custom Resource) ও তাকে পরিচালনা করা একটি কন্ট্রোলার। উদাহরণ: একটি ডেটাবেস অপারেটর দেখে একটি `Database` রিসোর্স তৈরি হয়েছে, তারপর সে ডেটাবেস বানায়, ব্যাকআপ সময়সূচি সেট করে, আর ডেটাবেস মুছে গেলে ব্যাকআপ নীতিও সামলায়। Operator-এর শক্তি হলো **Reconciliation Loop** — কন্ট্রোলার বারবার পরীক্ষা করে বাস্তব অবস্থা কাঙ্ক্ষিত অবস্থার সাথে মেলে কি না, না মিললে ঠিক করে। Terraform-এর `apply` একবার চালাতে হয়; Operator ক্রমাগত নজর রাখে। তবে Operator বানানো ও চালানো কঠিন — এটি একটি সফটওয়্যার তৈরি করা, শুধু কনফিগারেশন লেখা নয়। ছোট দলের জন্য প্রথমে Terraform মডিউল, তারপর প্রয়োজন হলে Crossplane বা Operator — এই ক্রমটি বাস্তবসম্মত।

তিনটি অটোমেশন ধারার তুলনা:

	Terraform মডিউল	Crossplane	Operator
কোড →	`apply` (একবার)	K8s API →	ক্রমাগত কাস্টম রিসোর্স
State	ফাইলে হিসাব	সমন্বয় (Reconcile) →	ক্রমাগত সমন্বয়
	ক্লাউড-নিরপেক্ষ	K8s-কেন্দ্রিক	অ্যাপ-নির্দিষ্ট যুক্তি
	শেখা সহজ	K8s জানা থাকলে সহজ	বানানো কঠিন

TechNova-র ক্রম: মডিউল প্রথমে, Crossplane পরে, Operator শুধু যেখানে জটিল জীবনচক্রের যুক্তি দরকার।

১৭.৮ প্ল্যাটফর্ম API: দলের জন্য একটি দরজা

প্ল্যাটফর্ম দলের আসল পণ্যটি অবকাঠামো নয় — পণ্যটি হলো দলগুলোর জন্য একটি সহজ পথ: “আমার একটি পরিবেশ চাই”, “আমার একটি ডেটাবেস চাই”, “আমার অ্যাপটি ডেপ্লয় করো”। **Platform API** সেই পথটিকে একটি পরিষ্কার চুক্তিতে বাঁধে — সেটি একটি REST API হতে পারে, একটি CLI হতে পারে, বা Git-এ একটি ফর্ম (যেমন একটি YAML ফাইল) হতে পারে। গুরুত্বপূর্ণ বিষয়টি মাধ্যম নয়, চুক্তিটি: কী চাওয়া যায়, কী ইনপুট দিতে হয়, কী পাওয়া যায়, কতক্ষণ লাগে, আর ভুল হলে কী বার্তা আসে। একটি ভালো প্ল্যাটফর্ম API-র পেছনে Terraform, Crossplane বা Operator যেকোনো কিছু থাকতে পারে — দলটি তা জানবে না, জানার দরকারও নেই। TechNova-তে একটি ন্যূনতম প্ল্যাটফর্ম API এমন হতে পারে: দল একটি YAML ফাইলে লেখে `service: web, env: staging, database: postgres, replicas: 2`, Pull Request মার্জ হলে প্ল্যাটফর্ম সেই পরিবেশ বানিয়ে দেয় ও ঠিকানা ফেরত দেয়। এই **Internal Developer Platform (IDP)**-এর ধারণাটি ২৪তম অধ্যায়ের ক্যাপস্টোনে পূর্ণ রূপ পাবে।

> **REAL WORLD:** TechNova-তে একসময় নতুন পরিবেশ পেতে টিকিট কাটতে হতো আর অপেক্ষা করতে হতো তিন দিন। প্ল্যাটফর্ম দল একটি ছোট YAML ফর্ম ও তার পেছনে Terraform মডিউল বানানোর পর একই কাজ হয় Pull Request-এ, এক ঘণ্টায় — আর প্রতিটি পরিবেশ একই নীতি মেনে বানানো হয়, কারণ নীতিটি মডিউলে লেখা, মানুষের স্মৃতিতে নয়।

১৭.৯ স্টেট ব্যবস্থাপনা ও Drift Detection

Terraform **State** হলো বাস্তব অবকাঠামোর হিসাব — কোন Resource কোথায়, কী অবস্থায়। State হারালে Terraform জানে না কী আছে, আর ভুল করে নতুন করে বানাতে বা মুছতে পারে। তাই State-এর জন্য তিনটি নিয়ম: State দূরবর্তী (Remote) জায়গায় রাখা — যেমন একটি ক্লাউড স্টোরেজে, সংস্করণসহ; State-এ লক (Lock) রাখা যাতে দুজন একসাথে Apply না করতে পারে; আর State ফাইলে সিক্রেট থাকতে পারে বলে তাকে সুরক্ষিত রাখা ও এনক্রিপ্ট করা। **Drift** মানে বাস্তব অবস্থা কোডের লেখা অবস্থা থেকে সরে যাওয়া — কেউ কনসোলে হাতে কিছু বদলালে Drift হয়। **Drift Detection** মানে নিয়মিত `terraform plan` বা সমতুল্য পরীক্ষা চালিয়ে এই সরে যাওয়া ধরা। Drift ধরা পড়লে দুটি পথ: হয় কোডে সেই পরিবর্তন লিখে নেওয়া (যদি পরিবর্তনটি সঠিক হয়), নয়তো Apply করে বাস্তবতাকে কোডের দিকে ফেরানো। Drift-কে উপেক্ষা করলে একসময় কোড আর বাস্তবতা দুটি আলাদা জগৎ হয়ে যায় — আর তখন IaC-র মূল্য থাকে না।

```
# Drift পরীক্ষা — নিয়মিত চালানোর ধারণা
terraform plan -detailed-exitcode
# ফলাফল: 0 = কোনো পরিবর্তন নেই; 1 = ত্রুটি; 2 = পরিবর্তন আছে (Drift)
# এই পরীক্ষাটি CI-তে সময়সূচি (Schedule) দিয়ে চালান;
# Drift পেলে দলকে জানান — নীরবে Apply করবেন না।
```

১৭.১০ অটোমেশন পরীক্ষা ও গভর্ন্যান্স

অটোমেশন কোডও কোড — তাই তার পরীক্ষাও দরকার। Terraform মডিউলের পরীক্ষা হতে পারে তিন স্তরে: **Static** — ফরম্যাট ও বৈধতা পরীক্ষা (`terraform fmt`, `validate`); **Policy** — নীতি পরীক্ষা, যেমন কোনো সম্পদে এনক্রিপশন ছাড়া তৈরি হবে না, বা কোনো নিরাপত্তা নিয়ম সবার জন্য খোলা থাকবে না (Policy as Code — ২০তম অধ্যায়ে বিস্তারিত); **Integration** — আসলেই একটি অস্থায়ী পরিবেশ বানিয়ে দেখা মডিউলটি কাজ করে কি না, তারপর মুছে ফেলা। Ansible Role-এর পরীক্ষাও একই ধারায়: Dry Run (`--check`) দিয়ে দেখা কী বদলাবে, আর একটি পরীক্ষামূলক সার্ভারে চালিয়ে দেখা।

Automation Governance মানে অটোমেশন কে, কী, কখন চালাতে পারে তার নিয়ম। এর মধ্যে পড়ে: মডিউলের সংস্করণ নীতি (কোন সংস্করণ ব্যবহার করা যাবে), অনুমোদন (প্রোডাকশন Apply-এ মানুষের অনুমোদন), অডিট (কে কী Apply করল, তার রেকর্ড), আর নামকরণ ও ট্যাগিংয়ের মান (প্রতিটি সম্পদে দল, পরিবেশ ও মালিকের ট্যাগ থাকা)। গভর্ন্যান্স যত কঠোর, গতি তত কমে — তাই নিয়মগুলো ঝুঁকি অনুযায়ী স্তরভিত্তিক হওয়া উচিত: Development-এ হালকা, Production-এ কঠোর।

> **WARNING:** State ফাইল কখনো Git-এ রাখবেন না, আর কখনো হাতে সম্পাদনা করবেন না। State-এ সিক্রেট থাকতে পারে, আর হাতে বদলালে Terraform-এর হিসাব ভেঙে যায়। State-এ সমস্যা হলে ব্যাকআপ থেকে ফেরান, হাতে ঠিক করতে যাবেন না।

১৭.১১ TechNova প্রোডাকশন দৃশ্যকল্প

TechNova-র তিনটি দল প্রত্যেকে নিজের মতো ডেটাবেস বানাচ্ছিল — কারো ব্যাকআপ ছিল, কারো ছিল না; কারো এনক্রিপশন ছিল, কারো নয়। প্ল্যাটফর্ম দল একটি `database` মডিউল বানাল — ভেতরে এনক্রিপশন, সাত দিনের ব্যাকআপ, নির্দিষ্ট নেটওয়ার্ক নিয়ম — আর ঘোষণা করল: এখন থেকে ডেটাবেস শুধু এই মডিউল দিয়ে। দলগুলো প্রথমে আপত্তি করেছিল — "আমাদের বিশেষ প্রয়োজন আছে" — কিন্তু মডিউলে একটি Escape Hatch রাখা হয়েছিল: বিশেষ প্রয়োজন Pull Request-এ যুক্তি দিয়ে চাওয়া যায়, আর তা মডিউলে সবার জন্য যোগ হয়। ছয় মাস পর কোনো দলই আর নিজের মতো ডেটাবেস বানায় না — কারণ মডিউলটি সহজ, নিরাপদ, আর তাদের প্রয়োজন মডিউলেই আছে। এটাই Platform as Code-র সাফল্যের চিহ্ন: দলগুলো নিয়ম মানছে কারণ নিয়মটি সহজ পথ, কঠিন পথ নয়।

১৭.১২ নিরাপত্তা ও খরচ বিবেচনা

অটোমেশন কোডে নিরাপত্তা নীতি লেখা যায় — এটাই এর বড় সুবিধা: কোনো ডেটাবেস এনক্রিপশন ছাড়া তৈরি হবে না, কোনো স্টোরেজ সবার জন্য খোলা থাকবে না — এই নিয়ম মডিউলে লেখা থাকলে প্রতিটি ব্যবহারে তা মানা হয়। কিন্তু মডিউলের ভেতরে সিক্রেট বা শংসাপত্র লেখা যাবে না; সিক্রেট আসবে ভল্ট বা পরিবেশ চলক থেকে। খরচের দিক থেকে মডিউল একটি দ্বৈত প্রভাব ফেলে: ভালো মডিউল খরচ কমায় (সঠিক আকার, স্বয়ংক্রিয় বন্ধের নিয়ম), খারাপ মডিউল খরচ বাড়ায় (অতিরিক্ত বড় সম্পদ সবাই ব্যবহার করছে)। তাই মডিউলে খরচ-সংবেদনশীল পূর্বনির্ধারিত মান রাখুন, আর বড় সম্পদের জন্য অনুমোদন গেট রাখুন।

১৭.১৩ সাধারণ ভুল

- **অতিরিক্ত বড় মডিউল:** একটি মডিউলে সবকিছু ঢোকালে তা পুনঃব্যবহার করা যায় না; একক দায়িত্ব মেনে ছোট রাখুন।
- **সংস্করণে না আটকানো:** `latest` ব্যবহার করলে অপরীক্ষিত পরিবর্তন প্রোডাকশনে ঢুকে যায়।
- **নথি ছাড়া মডিউল:** নথি ও উদাহরণ ছাড়া কেউ মডিউল ব্যবহার করবে না।
- **State Git-এ রাখা বা হাতে সম্পাদনা:** State-এর নিয়ম ভাঙলে হিসাব ভেঙে যায়।
- **Drift উপেক্ষা করা:** Drift জমতে দিলে কোড আর বাস্তবতা আলাদা হয়ে যায়।
- **সবকিছুতে Operator বানানো:** Operator বানানো কঠিন; প্রথমে মডিউল দিয়ে সমস্যা মেটে কি না দেখুন।

১৭.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
মডিউল ব্যবহার করতে কেউ চায় না	মডিউল কঠিন বা নথি নেই	নথি ও উদাহরণ যোগ করুন; দলের প্রয়োজন শুনে মডিউল ঠিক করুন
Apply-তে অপ্রত্যাশিত পরিবর্তন দেখায়	Drift বা State সমস্যা	Plan ভালো করে পড়ুন; Drift-এর উৎস খুঁজুন; State ব্যাকআপ পরীক্ষা করুন
দুজন একসাথে Apply করে সংঘাত	State Lock নেই	Remote State-এ Lock চালু করুন; Apply-এ অনুমোদন গেট রাখুন
Ansible দ্বিতীয়বার চালালে বদলায়	Task Idempotent নয়	Task-গুলো Idempotent করে লিখুন; `--check` দিয়ে যাচাই করুন

Crossplane/Operator কাজ করছে না	অনুমতি বা কন্ট্রোলার ত্রুটি	কন্ট্রোলারের লগ ও ইভেন্ট দেখুন; শংসাপত্র ও অনুমতি পরীক্ষা করুন

১৭.১৫ প্রকৌশল ট্রেড-অফ: কতটা অ্যাবস্ট্রাকশন

অ্যাবস্ট্রাকশন একটি ভারসাম্যের খেলা। খুব কম অ্যাবস্ট্রাকশনে প্রতিটি দল নিজের মতো কাজ করে — দ্রুত শুরু, কিন্তু বিশৃঙ্খলা। খুব বেশি অ্যাবস্ট্রাকশনে সবকিছু একই ছাঁচে ঢালা হয় — শৃঙ্খলা, কিন্তু দলগুলোর বিশেষ প্রয়োজন মেটে না, আর প্ল্যাটফর্ম দল বোতল-ঘাড় হয়ে যায়। সঠিক জায়গাটি হলো: ৮০% সাধারণ প্রয়োজন মডিউলে, ২০% বিশেষ প্রয়োজন Escape Hatch-এ। আর একটি সতর্কতা: অ্যাবস্ট্রাকশন বানানোর আগে অন্তত তিনটি বাস্তব ব্যবহার দেখুন। একটি ব্যবহার দেখে বানানো অ্যাবস্ট্রাকশন প্রায়ই ভুল হয় — কারণ তখন আপনি একটি বিশেষ ক্ষেত্রকে সাধারণ নিয়ম ভেবে বসেন।

> **TROUBLESHOOTING:** মডিউল বদলানোর আগে জিজ্ঞেস করুন — “এই মডিউলটি কারা ব্যবহার করছে, আর এই পরিবর্তন তাদের কী প্রভাব ফেলবে?” উত্তর না জেনে মডিউল বদলানো মানে অনেক দলের প্রোডাকশন একসাথে বদলে দেওয়া।

১৭.১৬ অধ্যায়ের সারাংশ

Platform as Code মানে পুরো প্ল্যাটফর্মকে কোড হিসেবে দেখা: Terraform মডিউল পুনঃব্যবহারের মূল একক, প্রোভাইডার অ্যাবস্ট্রাকশন ধারাবাহিকতা দেয়, Ansible সার্ভারের ভেতরের অবস্থা সামলায়, Crossplane ও Operator Kubernetes-কেন্দ্রিক অটোমেশন দেয়। প্ল্যাটফর্ম API দলগুলোর জন্য একটি পরিষ্কার দরজা তৈরি করে। State ব্যবস্থাপনা, Drift Detection, অটোমেশন পরীক্ষা ও গভর্ন্যান্স ছাড়া অটোমেশন টেকসই হয় না। সফল প্ল্যাটফর্মের চিহ্ন হলো দলগুলো নিয়ম মানে কারণ নিয়মটি সহজ পথ।

১৭.১৭ জ্ঞান যাচাই

1. Platform as Code ও Infrastructure as Code-এর পার্থক্য ব্যাখ্যা করুন।
2. ভালো Terraform মডিউলের চারটি গুণ লিখুন ও প্রতিটির উদাহরণ দিন।
3. মডিউল সংস্করণে আটকে (Pin) রাখা কেন জরুরি?
4. প্রোভাইডার অ্যাবস্ট্রাকশনের সুবিধা ও সীমা ব্যাখ্যা করুন। Escape Hatch কেন দরকার?
5. Terraform ও Ansible-র কাজের ভাগটি কোথায়? কখন কোনটি ব্যবহার করবেন?
6. Crossplane কী? এটি Terraform-এর বিকল্প নাকি ভিন্ন ধারা?
7. Kubernetes Operator-এর Reconciliation Loop কীভাবে কাজ করে?
8. Drift কী? Drift Detection না করলে কী হয়?
9. অটোমেশন পরীক্ষার তিনটি স্তর ব্যাখ্যা করুন।
10. অতিরিক্ত অ্যাবস্ট্রাকশন কেন ক্ষতিকর হতে পারে?

১৭.১৮ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি ছোট Terraform মডিউলের নকশা লিখুন (যেমন স্টোরেজ বাকেট): কী ইনপুট নেবে, কী আউটপুট দেবে, কোন নিরাপত্তা নীতি ভেতরে থাকবে, আর README-তে কী লিখবেন — তা নথিভুক্ত করুন।

2. একটি Ansible Role-এর পরিকল্পনা করুন: একটি সার্ভারে একটি সার্ভিস ইনস্টল ও চালু করার ধাপগুলো Idempotent করে লিখুন, আর `--check` দিয়ে কী যাচাই করবেন তা লিখুন।
3. TechNova-র একটি প্ল্যাটফর্ম API-র ন্যূনতম চুক্তি লিখুন: দল কীভাবে ডেটাবেস চাইবে, কী ইনপুট দেবে, কী ফেরত পাবে, আর ভুল হলে কী বার্তা আসবে।
4. একটি Drift Detection পরিকল্পনা বানান: কত ঘন ঘন পরীক্ষা চালাবেন, Drift পেলে কাকে জানাবেন, আর কখন কোডে লিখবেন বনাম কখন Apply করে ফেরাবেন — তা লিখুন।

অধ্যায় ১৮ • PART IV — Advanced Automation & Enterprise Operations

Chapter 18 — Multi-Cloud Platform Operations

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Multi-Cloud** ও **Hybrid Cloud**-এর পার্থক্য ব্যাখ্যা করতে; AWS, Azure, GCP ও IBM Cloud-এর ভূমিকা ও বৈশিষ্ট্য তুলনা করতে; পরিচয় ফেডারেশন (Identity Federation), নেটওয়ার্ক সংযোগ, ওয়ার্কলোড স্থাপন (Workload Placement), Kubernetes বহনযোগ্যতা (Portability), ক্লাউড গভর্ন্যান্স, ডেটা স্থানান্তর ও **Egress Cost** (বাইরে ডেটা পাঠানোর খরচ)-এর প্রভাব TechNova-র প্রেক্ষাপটে মূল্যায়ন করতে; এবং কখন একক ক্লাউড ভালো আর কখন মাল্টি-ক্লাউড যুক্তিযুক্ত, তা যুক্তি দিয়ে বলতে।

- চারটি প্রধান ক্লাউডের সেবা-ধরন ও শক্তির জায়গা তুলনা করা।
- মাল্টি-ক্লাউড পরিচয়, নেটওয়ার্ক ও ডেটা-প্রবাহের নকশা আঁকা।
- Egress খরচ ও লক-ইন (Lock-in)-এর ঝুঁকি হিসাবের মধ্যে আনা।
- একটি ওয়ার্কলোড কোথায় চালাবেন, তার সিদ্ধান্ত-কাঠামো লেখা।

১৮.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার ক্লাউডের মৌলিক ধারণা — ভার্চুয়াল মেশিন, অবজেক্ট স্টোরেজ, পরিচালিত ডেটাবেস, ভার্চুয়াল নেটওয়ার্ক (VPC/VNet) — এবং Kubernetes-এ অ্যাপ চালানোর অভিজ্ঞতা থাকা দরকার। **Hybrid Cloud** মানে নিজের ডেটা সেন্টার (On-Premises) ও একটি ক্লাউডের মিশ্রণ; **Multi-Cloud** মানে একাধিক ক্লাউড ব্যবহার করা। **Egress** মানে ক্লাউড থেকে বাইরে ডেটা পাঠানো — সাধারণত এর জন্য আলাদা খরচ হয়, আর সেই খরচ অনেক সময় অপ্রত্যাশিতভাবে বড় হয়। **Vendor Lock-in** মানে কোনো ক্লাউডের এমন বিশেষ সেবায় আটকে যাওয়া, যা থেকে বের হওয়া কঠিন ও ব্যয়বহুল।

> **CONCEPT:** মাল্টি-ক্লাউড মানে “সব ক্লাউডে সবকিছু” নয়। এর অর্থ হলো — প্রতিটি ওয়ার্কলোডের জন্য সচেতনভাবে জায়গা বেছে নেওয়া, আর সেই পছন্দের খরচ, জটিলতা ও ঝুঁকি হিসাব করে নেওয়া। বেছে নেওয়া না হলে মাল্টি-ক্লাউড হয়ে যায় দুর্ঘটনাবশত মাল্টি-ক্লাউড — যা সবচেয়ে ব্যয়বহুল রূপ।

১৮.৩ প্রথম নীতি থেকে: কেন একাধিক ক্লাউড

একটি ক্লাউডে সবকিছু রাখা সহজ: একটি বিল, একটি পরিচয় ব্যবস্থা, একটি নেটওয়ার্ক ধারণা, একটি দক্ষতা-সেট। মাল্টি-ক্লাউডে যাওয়ার কারণগুলো তাই যথেষ্ট শক্তিশালী হতে হয়। সাধারণ কারণগুলো হলো: (১) **ঝুঁকি ছড়ানো** — একটি ক্লাউডের বড় বিভ্রাটে পুরো ব্যবসা থেমে না যাওয়া; (২) **সেরা সেবা বেছে নেওয়া** — একটি ক্লাউডের ডেটা বিশ্লেষণ সেবা ভালো, আরেকটির মেশিন লার্নিং সেবা ভালো; (৩) **ভৌগোলিক ও নিয়ন্ত্রক প্রয়োজন** — কোনো অঞ্চলে একটি ক্লাউডের উপস্থিতি ভালো, বা কোনো নিয়মে ডেটা নির্দিষ্ট জায়গায় রাখতে হয়; (৪) **দর-কষাকষি** — একাধিক সরবরাহকারী থাকলে দামের আলোচনায় সুবিধা; (৫) **অধিগ্রহণ বা ইতিহাস** — দুটি কোম্পানি এক হলে তাদের ক্লাউডও দুটি হয়। কিন্তু প্রতিটি কারণেরই একটি মূল্য আছে: জটিলতা, দক্ষতার ছড়িয়ে পড়া, আর ক্লাউডগুলোর মধ্যে ডেটা পাঠানোর খরচ।

সবচেয়ে গুরুত্বপূর্ণ প্রশ্নটি তাই “আমরা কি মাল্টি-ক্লাউডে যাব?” নয় — প্রশ্নটি হলো “কোন ওয়ার্কলোডটি কোন ক্লাউডে থাকবে, আর কেন?” যে উত্তর দিতে পারে না, তার মাল্টি-ক্লাউড কৌশলটি আসলে কোনো কৌশল নয়, একটি ঘটনা।

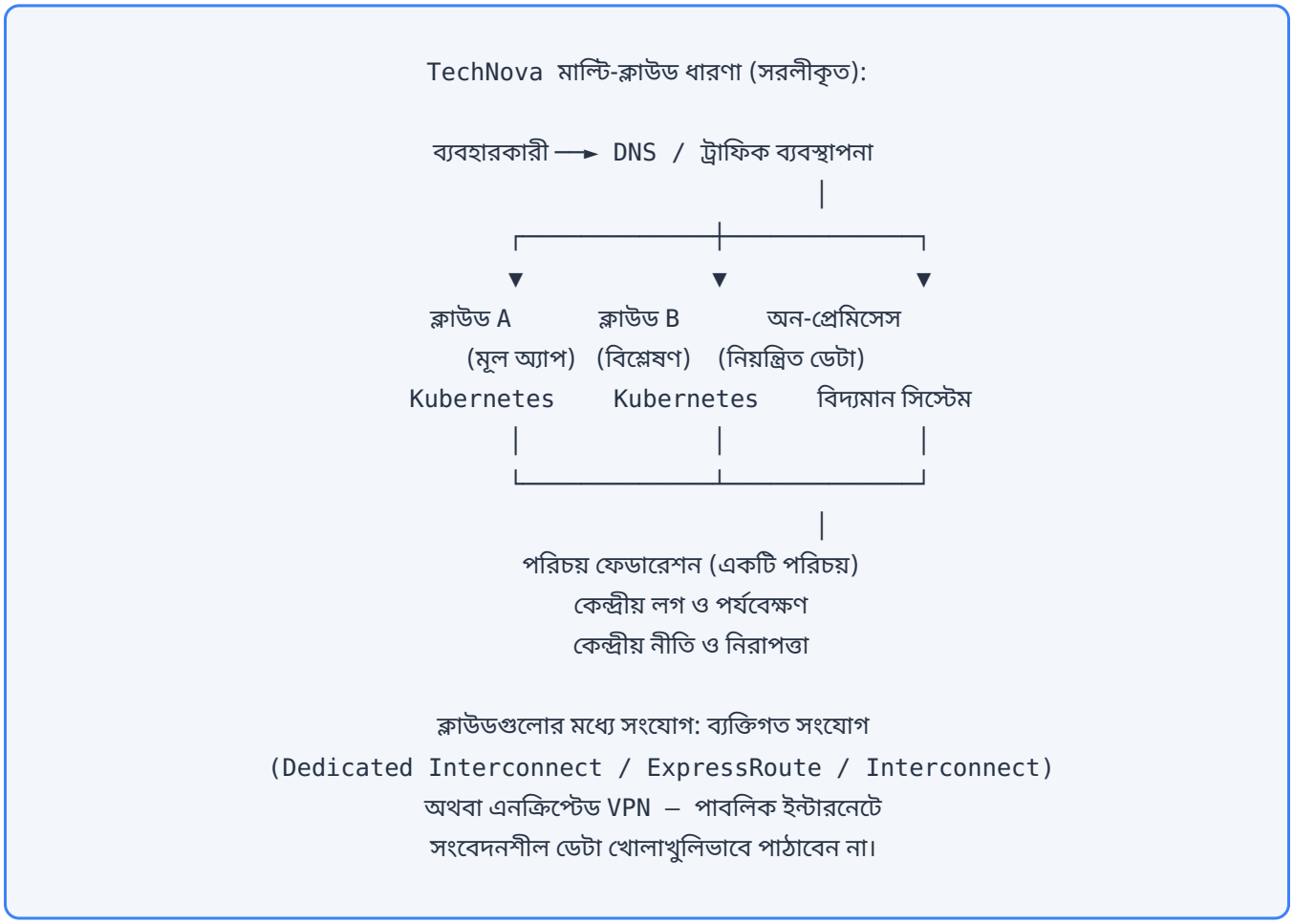
১৮.৪ চারটি ক্লাউড: AWS, Azure, GCP ও IBM Cloud

চারটি ক্লাউডেরই মূল সেবাগুলো একই ধরনের — কম্পিউট, স্টোরেজ, নেটওয়ার্ক, পরিচালিত ডেটাবেস, Kubernetes — কিন্তু তাদের শক্তির জায়গা ও ইতিহাস আলাদা। **AWS (Amazon Web Services)** সবচেয়ে পুরোনো ও সেবার পরিধিতে সবচেয়ে বড়; এর শক্তি হলো সেবার বৈচিত্র্য ও পরিপক্কতা। **Microsoft Azure**-এর শক্তি হলো এন্টারপ্রাইজ ও Microsoft পরিবেশের সাথে সংযোগ — যে সংস্থা ইতিমধ্যে Microsoft প্রযুক্তি ব্যবহার করে, তাদের জন্য Azure স্বাভাবিক পছন্দ হতে পারে। **Google Cloud Platform (GCP)**-এর শক্তি হলো ডেটা বিশ্লেষণ, মেশিন লার্নিং ও Kubernetes-এর জন্মস্থান হিসেবে কনটেইনার প্রযুক্তিতে গভীরতা। **IBM Cloud**-এর শক্তি হলো এন্টারপ্রাইজ, নিয়ন্ত্রিত শিল্প (যেমন ব্যাংকিং) ও হাইব্রিড ক্লাউডে দীর্ঘ অভিজ্ঞতা — অনেক প্রতিষ্ঠান IBM-এর সাথে হাইব্রিড যাত্রা শুরু করে।

বিষয়	AWS	Azure	GCP	IBM Cloud
শক্তির জায়গা	সেবার বৈচিত্র্য ও পরিপক্কতা	এন্টারপ্রাইজ ও Microsoft সংযোগ	ডেটা, AI ও Kubernetes গভীরতা	নিয়ন্ত্রিত শিল্প ও হাইব্রিড অভিজ্ঞতা
পরিচালিত Kubernetes	EKS	AKS	GKE	IBM Cloud Kubernetes Service
পরিচয় ব্যবস্থা	IAM ও Identity Center	Entra ID	Cloud IAM	IBM Cloud IAM
মনে রাখার কথা	নাম ও সেবা সবচেয়ে বেশি	Microsoft লাইসেন্স থাকলে সুবিধা	নেটওয়ার্ক ও ডেটা সেবায় শক্তিশালী	হাইব্রিড ও নিয়ন্ত্রক প্রয়োজনে বিবেচ্য

এই তুলনাটি একটি শুরুর মানচিত্র — কোনো ক্লাউডের সেবার তালিকা প্রতিনিয়ত বদলায়, তাই নির্দিষ্ট সিদ্ধান্তের আগে বর্তমান নথি (Documentation) দেখে যাচাই করুন। আর মনে রাখবেন: সেবার নাম মুখস্থ করার চেয়ে গুরুত্বপূর্ণ হলো ধারণাগুলো বোঝা — পরিচালিত Kubernetes যেকোনো ক্লাউডেই একই ধারণা, শুধু নাম ও কিছু বিবরণ আলাদা।

১৮.৫ মাল্টি-ক্লাউড আর্কিটেকচার



মাল্টি-ক্লাউড আর্কিটেকচারের তিনটি স্তর থাকে। প্রথম স্তর **নিয়ন্ত্রণ স্তর (Control Plane)**: পরিচয়, নীতি, পর্যবেক্ষণ, বিল — এগুলো যতটা সম্ভব এক জায়গায় রাখা উচিত, নইলে প্রতিটি ক্লাউডে আলাদা নিয়ম তৈরি হয়। দ্বিতীয় স্তর **ডেটা স্তর (Data Plane)**: অ্যাপ, ডেটাবেস, স্টোরেজ — এগুলো ক্লাউডভেদে ছড়িয়ে থাকে। তৃতীয় স্তর **সংযোগ স্তর (Connectivity)**: ক্লাউডগুলোর মধ্যে ও অন-প্রেমিসেসের সাথে নিরাপদ সংযোগ। এই তিন স্তরের মধ্যে সবচেয়ে অবহেলিতটি হলো প্রথমটি — দলগুলো অ্যাপ ছড়িয়ে দেয়, কিন্তু পরিচয় ও নীতি ছড়িয়ে যায়, আর তখন নিরাপত্তা-পর্যালোচনায় কেউ বলতে পারে না কার কী অনুমতি আছে।

১৮.৬ পরিচয় ফেডারেশন (Identity Federation)

মাল্টি-ক্লাউডে সবচেয়ে কঠিন সমস্যাগুলোর একটি হলো পরিচয়: প্রতিটি ক্লাউডের নিজের পরিচয় ব্যবস্থা আছে, আর প্রতিটিতে আলাদা ব্যবহারকারী ও অনুমতি রাখলে মানুষ বিভ্রান্ত হয়, আর নিরাপত্তা দুর্বল হয়। **Identity Federation** মানে একটি কেন্দ্রীয় পরিচয় প্রদানকারী (Identity Provider, IdP) — যেমন কোম্পানির কেন্দ্রীয় পরিচয় ব্যবস্থা — থেকে পরিচয় নিয়ে প্রতিটি ক্লাউডে সাময়িক অনুমতি (Short-lived Credential) পাওয়া। ব্যবহারকারী একবার সাইন ইন করেন, আর ফেডারেশন তাঁকে প্রতিটি ক্লাউডে প্রয়োজনীয় ভূমিকা (Role) দেয় — দীর্ঘমেয়াদি গোপন কী (Long-lived Key) ছড়িয়ে না রেখে।

এর জন্য তিনটি নিয়ম: প্রথমত, দীর্ঘমেয়াদি Access Key যতটা সম্ভব এড়িয়ে চলুন — সাময়িক শংসাপত্র ব্যবহার করুন। দ্বিতীয়ত, **Least Privilege** — প্রতিটি ভূমিকায় শুধু যতটুকু দরকার, ততটুকু অনুমতি। তৃতীয়ত, কেন্দ্রীয়ভাবে দেখা — কে,

কখন, কোন ক্লাউডে কী করলেন, তার লগ এক জায়গায় আসা উচিত। ফেডারেশন সেটআপ করা প্রথমে কঠিন মনে হয়, কিন্তু তিনটি ক্লাউডে তিনটি আলাদা পরিচয় ব্যবস্থা চালানোর চেয়ে অনেক সহজ ও নিরাপদ।

> **WARNING:** মাল্টি-ক্লাউডে দীর্ঘমেয়াদি Access Key ছড়িয়ে রাখা সবচেয়ে বিপজ্জনক অভ্যাসগুলোর একটি। একটি কী ফাঁস হলে আক্রমণকারী সেই ক্লাউডে প্রবেশ করতে পারে, আর কীটি কোথায় কোথায় আছে তা খুঁজে পাওয়া কঠিন হয়। সাময়িক শংসাপত্র ও ফেডারেশন ব্যবহার করুন, আর পুরোনো কীগুলো নিয়মিত পর্যালোচনা করে সরিয়ে দিন।

১৮.৭ নেটওয়ার্ক সংযোগ ও হাইব্রিড ক্লাউড

ক্লাউডগুলোর মধ্যে ও অন-প্রেমিসেসের সাথে সংযোগের দুটি মূল পথ আছে। **Site-to-Site VPN** — পাবলিক ইন্টারনেটের উপর এনক্রিপ্টেড সুড়ঙ্গ (Tunnel) — সস্তা ও দ্রুত স্থাপন করা যায়, কিন্তু গতি ও স্থিতি ইন্টারনেটের উপর নির্ভর করে। **Dedicated Connection** (যেমন AWS Direct Connect, Azure ExpressRoute, GCP Cloud Interconnect) — ক্লাউডের সাথে একটি ব্যক্তিগত সংযোগ — আরও স্থিতিশীল ও দ্রুত, কিন্তু স্থাপনে সময় লাগে ও খরচ বেশি। TechNova-র মতো দলের জন্য নিয়মটি হলো: শুরুতে VPN দিয়ে শুরু করা, আর যে সংযোগটি ব্যবসার জন্য গুরুত্বপূর্ণ হয়ে উঠবে, সেটি Dedicated সংযোগে উন্নীত করা।

Hybrid Cloud-এ অতিরিক্ত একটি বিবেচনা থাকে: অন-প্রেমিসেসের নেটওয়ার্ক ঠিকানা (IP Range) ও ক্লাউডের ঠিকানা পরিসর যেন সংঘাতে না পড়ে। ঠিকানা পরিকল্পনা আগে থেকে না করলে পরে নেটওয়ার্ক জোড়া দেওয়া অত্যন্ত কঠিন হয়। আর DNS-ও এক জায়গায় ভাবুন — প্রতিটি ক্লাউডে আলাদা নাম ব্যবস্থা থাকলে অ্যাপগুলো একে অপরকে খুঁজে পায় না।

১৮.৮ ওয়ার্কলোড স্থাপন ও Kubernetes বহনযোগ্যতা

Workload Placement মানে প্রতিটি ওয়ার্কলোড কোথায় চলবে, তা সিদ্ধান্ত নেওয়া। সিদ্ধান্তের উপাদানগুলো হলো: ডেটা কোথায় আছে (ডেটার কাছাকাছি কম্পিউট রাখলে Egress খরচ ও বিলম্ব কমে), নিয়ন্ত্রক প্রয়োজন (ডেটা কোন অঞ্চলে থাকতে হবে), সেবার প্রাপ্যতা (কোন ক্লাউডে প্রয়োজনীয় সেবা আছে), খরচ (কোথায় সস্তা), আর দলের দক্ষতা (কোন ক্লাউড দলটি ভালো চালাতে পারে)। একটি সাধারণ ভুল হলো সব ওয়ার্কলোডকে সব ক্লাউডে চালানোর মতো করে বানানোর চেষ্টা করা — এতে প্রতিটি ওয়ার্কলোড সবচেয়ে কম সাধারণ সুবিধাটুকুই ব্যবহার করতে পারে।

Kubernetes এখানে সাহায্য করে, কারণ কনটেইনার ও Kubernetes API ক্লাউড-নিরপেক্ষ: একই Deployment YAML যেকোনো ক্লাউডের পরিচালিত Kubernetes-এ চলতে পারে। কিন্তু “চলতে পারে” আর “একইভাবে চলে” এক নয়। প্রতিটি ক্লাউডের লোড ব্যালান্সার, স্টোরেজ ক্লাস, নেটওয়ার্ক নীতি ও পরিচয় সংযোগ আলাদা। তাই বহনযোগ্যতার জন্য তিনটি অভ্যাস: ক্লাউড-নির্দিষ্ট বিবরণ মডিউল বা Helm মানের (Value) মধ্যে আলাদা রাখুন; পরিচালিত ক্লাউড সেবার বদলে Kubernetes-এর ভেতরে চলা সেবা (যেমন ক্লাস্টারের ভেতরের ডেটাবেস) ব্যবহারের লোভ সামলান — পরিচালিত সেবা ছাড়লে ব্যাকআপ ও উচ্চ-প্রাপ্যতার দায় আপনার উপর আসে; আর নিয়মিত অন্য ক্লাউডে ডেপ্লয় পরীক্ষা করুন — পরীক্ষা ছাড়া বহনযোগ্যতা শুধু দাবি।

```
# বহনযোগ্যতার জন্য: ক্লাউড-নির্দিষ্ট মান আলাদা রাখুন
# values-cloud-a.yaml / values-cloud-b.yaml — শুধু মান বদলায়,
# মূল চার্ট একটাই।
# ডেপ্লয় পরীক্ষা: staging-এ অন্য ক্লাউডে নিয়মিত ডেপ্লয় করুন;
# “একদিন দরকার হলে পারব” — পরীক্ষা ছাড়া এই দাবি বিশ্বাস করবেন না।
```

১৮.৯ ক্লাউড গভর্ন্যান্স, ডেটা স্থানান্তর ও Egress খরচ

Cloud Governance মানে ক্লাউড ব্যবহারের নিয়ম: কে কী তৈরি করতে পারে, কোন অঞ্চলে তৈরি করতে পারে, কোন ট্যাগ থাকতে হবে, কোন সেবা নিষিদ্ধ। মাল্টি-ক্লাউডে গভর্ন্যান্স কঠিন হয়, কারণ প্রতিটি ক্লাউডের নীতির ভাষা আলাদা। সমাধান হলো নীতিকে ক্লাউড-নিরপেক্ষভাবে লেখা — যেমন Policy as Code দিয়ে "সব স্টোরেজ এনক্রিপ্টেড হবে" — আর প্রতিটি ক্লাউডে তা প্রয়োগ করা, সাথে কেন্দ্রীয় বিল ও খরচের দৃশ্যমানতা (২২তম অধ্যায়ে FinOps বিস্তারিত)।

Egress Cost মাল্টি-ক্লাউডের সবচেয়ে অপ্রত্যাশিত খরচ। ক্লাউডে ডেটা ঢোকানো (Ingress) সাধারণত বিনামূল্যে, কিন্তু বাইরে পাঠানো (Egress) — অন্য ক্লাউডে, অন-প্রেমিসেসে, বা ইন্টারনেটে — খরচসাপেক্ষ। দুটি ক্লাউডের মধ্যে প্রতিদিন বড় ডেটা আদান-প্রদান করলে বিলটি দ্রুত বড় হয়। তাই নকশার নিয়ম: ডেটা যেখানে তৈরি হয়, প্রক্রিয়াকরণ সেখানেই রাখুন; ক্লাউডের মধ্যে কাঁচা ডেটা বারবার পাঠাবেন না — পাঠান শুধু প্রয়োজনীয় ফলাফল; আর Dedicated সংযোগের খরচের কাঠামো আগেই বুঝে নিন। ডেটা স্থানান্তরের পরিকল্পনায় আরও আছে: স্থানান্তরের সময় ডেটা এনক্রিপ্টেড থাকবে কি না, স্থানান্তর কতক্ষণ চলবে, আর মাঝপথে ব্যর্থ হলে কীভাবে আবার শুরু হবে।

> **REAL WORLD:** TechNova একবার বিশ্লেষণের জন্য প্রতিদিন ক্লাউড A থেকে ক্লাউড B-তে পুরো ডেটাসেট কপি করত — বিল দেখে বোঝা গেল Egress খরচই কম্পিউট খরচের চেয়ে বড় হয়ে গেছে। সমাধানটি ছিল সহজ: বিশ্লেষণটি ক্লাউড A-তেই চালানো, আর শুধু সংক্ষিপ্ত ফলাফল ক্লাউড B-তে পাঠানো। আর্কিটেকচারের একটি সিদ্ধান্ত বিলটিকে অর্ধেক করে দিয়েছিল।

১৮.১০ক সরবরাহকারী পরিবর্তন: বের হওয়ার পরিকল্পনা

মাল্টি-ক্লাউড আলোচনায় একটি প্রশ্ন প্রায়ই বাদ পড়ে: একটি ক্লাউড থেকে বের হওয়া কতটা কঠিন? এই প্রশ্নের উত্তর দেওয়ার জন্য তিনটি জিনিস দেখুন। প্রথমত, ডেটা — ডেটা বের করার খরচ (Egress) ও সময় কত? বড় ডেটাসেট বের করতে সপ্তাহ লাগতে পারে, আর খরচটি আগেই হিসাব করা উচিত। দ্বিতীয়ত, সেবার নির্ভরতা — আপনার অ্যাপ কি ক্লাউডের বিশেষ সেবার (যেমন একটি বিশেষ ডেটাবেস বা AI সেবা) উপর গভীরভাবে নির্ভরশীল? নির্ভরতা যত গভীর, বের হওয়া তত কঠিন। তৃতীয়ত, দক্ষতা — দলটি কি শুধু একটি ক্লাউডের টুল জানে, নাকি বহনযোগ্য টুল (Terraform, Kubernetes, OpenTelemetry) জানে? বহনযোগ্য টুলের দক্ষতা বের হওয়ার খরচ কমায়। এই তিনটির উত্তর লিখে রাখুন — শুধু বের হওয়ার পরিকল্পনার জন্য নয়, নতুন সেবা নেওয়ার সময়ও: একটি নতুন পরিচালিত সেবা নেওয়ার আগে জিজ্ঞেস করুন, "এই সেবা ছাড়তে হলে কী লাগবে?" উত্তরটি যদি "সবকিছু আবার লিখতে হবে" হয়, তবে সেই সেবার মূল্য তার ঝুঁকির সাথে তুলনা করুন। বের হওয়ার পরিকল্পনা থাকা মানে বের হওয়ার ইচ্ছা নয় — মানে সচেতন পছন্দ।

১৮.১০ TechNova প্রোডাকশন দৃশ্যকল্প

TechNova-র মূল অ্যাপ ক্লাউড A-তে চলে, কিন্তু একটি বড় গ্রাহক চায় তাদের ডেটা একটি নির্দিষ্ট অঞ্চলে থাকুক, যেখানে ক্লাউড A-র উপস্থিতি দুর্বল — সেখানে ক্লাউড B শক্তিশালী। পরিকল্পনা: মূল অ্যাপ ক্লাউড A-তেই থাকে; সেই গ্রাহকের ডেটা ও তার জন্য একটি আলাদা অ্যাপ-ইনস্ট্যান্স ক্লাউড B-তে চলে; দুটির মধ্যে সংযোগ Dedicated সংযোগে, শুধু প্রয়োজনীয় ডেটা আদান-প্রদান হয়; পরিচয় ফেডারেশন দিয়ে একটিই সাইন-ইন; কেন্দ্রীয় পর্যবেক্ষণে দুটি ক্লাউডের মেট্রিক একসাথে দেখা যায়। এই পরিকল্পনায় মাল্টি-ক্লাউড একটি ব্যবসায়িক প্রয়োজনের উত্তর — "সবকিছু দুই ক্লাউডে" নয়, "নির্দিষ্ট ওয়ার্কলোড নির্দিষ্ট ক্লাউডে"।

১৮.১১ নিরাপত্তা ও খরচ বিবেচনা

মাল্টি-ক্লাউডে আক্রমণের পৃষ্ঠ (Attack Surface) বাড়ে — প্রতিটি ক্লাউডের কনসোল, API ও পরিচয় ব্যবস্থা একটি প্রবেশপথ। তাই কেন্দ্রীয় পরিচয়, কেন্দ্রীয় লগ, আর প্রতিটি ক্লাউডে একই নিরাপত্তা নীতি অপরিহার্য। খরচের দিক থেকে মনে রাখবেন: মাল্টি-ক্লাউড প্রায় সবসময় একক ক্লাউডের চেয়ে ব্যয়বহুল — দুটি ক্লাউডের দক্ষতা রাখতে হয়, দুটি বিল দেখতে হয়, আর Egress খরচ যোগ হয়। এই বাড়তি খরচটি তখনই যুক্তিযুক্ত যখন এর বিনিময়ে ঝুঁকি কমে, নিয়ম মানা যায়, বা এমন সেবা পাওয়া যায় যা একক ক্লাউডে নেই। খরচের হিসাবটি লিখিতভাবে করুন — অনুমানে নয়।

১৮.১২ সাধারণ ভুল

- **দুর্ঘটনাবশত মাল্টি-ক্লাউড:** পরিকল্পনা ছাড়া দুটি ক্লাউডে ছড়িয়ে পড়া — সবচেয়ে ব্যয়বহুল রূপ।
- **Egress খরচ উপেক্ষা:** ডেটা-প্রবাহের খরচ হিসাব না করে আর্কিটেকচার আঁকা।
- **প্রতিটি ক্লাউডে আলাদা পরিচয়:** দীর্ঘমেয়াদি কী ছড়িয়ে রাখা ও অনুমতির হিসাব হারানো।
- **বহনযোগ্যতার দাবি, পরীক্ষা নেই:** অন্য ক্লাউডে কখনো ডেপ্লয় না করে “বহনযোগ্য” বলা।
- **সবকিছু সব জায়গায় চালানো:** প্রতিটি ওয়ার্কলোডকে সব ক্লাউডে চালানোর চেষ্টা — ফলে কোনো ক্লাউডের সুবিধাই পুরো পাওয়া যায় না।
- **ঠিকানা পরিকল্পনা না করা:** নেটওয়ার্ক ঠিকানার সংঘাত পরে জোড়া দেওয়া কঠিন করে।

১৮.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ক্লাউডগুলোর মধ্যে বিলম্ব বেশি	পাবলিক ইন্টারনেটে ডেটা যাচ্ছে; অঞ্চল দূরে	Dedicated সংযোগ বিবেচনা করুন; ডেটার কাছে কম্পিউট রাখুন
Egress বিল হঠাৎ বেড়ে গেল	বড় ডেটা বারবার কপি হচ্ছে	ডেটা-প্রবাহের মানচিত্র আঁকুন; ফলাফল পাঠান, কাঁচা ডেটা নয়
এক ক্লাউডে চলে, অন্য ক্লাউডে ভাঙে	ক্লাউড-নির্দিষ্ট বিবরণ মূল কোডে মিশে আছে	ক্লাউড-নির্দিষ্ট মান আলাদা করুন; অন্য ক্লাউডে নিয়মিত পরীক্ষা করুন
কার কী অনুমতি আছে জানা যায় না	প্রতিটি ক্লাউডে আলাদা পরিচয়	ফেডারেশন চালু করুন; অনুমতি কেন্দ্রীয়ভাবে পর্যালোচনা করুন
অন্য ক্লাউডে ডেপ্লয় করতে দিন লাগে	দক্ষতা ও টুলিং এক ক্লাউডে আটকে আছে	সাধারণ টুলিং (Terraform, Kubernetes) ব্যবহার করুন; দলকে প্রশিক্ষণ দিন

১৮.১৪ প্রকৌশল ট্রেড-অফ: একক ক্লাউড বনাম মাল্টি-ক্লাউড

একক ক্লাউডের পক্ষে যুক্তি: সহজ, সস্তা, দল একটি দক্ষতায় গভীর হয়, একটি বিল, একটি পরিচয় ব্যবস্থা। বিপক্ষে: একটি সরবরাহকারীর উপর নির্ভরতা, তার বিভ্রাটে আপনার বিভ্রাট, আর দর-কষাকষিতে দুর্বল অবস্থান। মাল্টি-ক্লাউডের পক্ষে: ঝুঁকি ছড়ানো, সেরা সেবা বেছে নেওয়া, নিয়ন্ত্রক প্রয়োজন মেটানো। বিপক্ষে: জটিলতা, বেশি খরচ, দক্ষতা ছড়িয়ে পড়া, Egress খরচ। TechNova-র মতো ছোট ও মাঝারি দলের জন্য সাধারণত সঠিক উত্তরটি হলো: একটি মূল ক্লাউডে গভীরভাবে যাওয়া, আর মাল্টি-ক্লাউডের দরজাটি খোলা রাখা — Kubernetes ও Terraform-এর মতো বহনযোগ্য টুল

ব্যবহার করে, ক্লাউড-নির্দিষ্ট সেবায় অপ্রয়োজনীয়ভাবে আটকে না গিয়ে। যখন ব্যবসায়িক প্রয়োজন স্পষ্ট হয় — তখনই দ্বিতীয় ক্লাউডে যাওয়া, তার আগে নয়। সহজ পথটি যখন কাজ করে, তখন জটিল পথে যাওয়া প্রকৌশল নয়।

> **TROUBLESHOOTING:** মাল্টি-ক্লাউড সমস্যা নির্ণয়ের প্রথম প্রশ্নটি হলো — "ডেটাটি কোথা থেকে কোথায় যাচ্ছে, আর কেন?" ডেটা-প্রবাহের মানচিত্র ছাড়া মাল্টি-ক্লাউডের বিলম্ব, খরচ বা নিরাপত্তা সমস্যার মূল খুঁজে পাওয়া যায় না।

১৮.১৫ অধ্যায়ের সারাংশ

মাল্টি-ক্লাউড একটি কৌশলগত পছন্দ, দুর্ঘটনা নয়। AWS, Azure, GCP ও IBM Cloud-এর প্রত্যেকের নিজের শক্তির জায়গা আছে; সঠিক পছন্দটি ওয়ার্কলোড, ডেটা, নিয়ম ও দলের দক্ষতার উপর নির্ভর করে। পরিচয় ফেডারেশন, নিরাপদ নেটওয়ার্ক সংযোগ, সচেতন ওয়ার্কলোড স্থাপন, Kubernetes বহনযোগ্যতা, কেন্দ্রীয় গভর্ন্যান্স ও Egress খরচের হিসাব — এই ছয়টি স্তম্ভ ছাড়া মাল্টি-ক্লাউড টেকসই হয় না। ছোট দলের জন্য একটি মূল ক্লাউডে গভীর হওয়া ও বহনযোগ্যতার দরজা খোলা রাখাই সাধারণত সঠিক পথ।

১৮.১৬ জ্ঞান যাচাই

1. Multi-Cloud ও Hybrid Cloud-এর পার্থক্য ব্যাখ্যা করুন।
2. মাল্টি-ক্লাউডে যাওয়ার পাঁচটি কারণ লিখুন ও প্রতিটির মূল্য (খরচ/জটিলতা) বলুন।
3. AWS, Azure, GCP ও IBM Cloud-এর শক্তির জায়গাগুলো তুলনা করুন।
4. Identity Federation কী? দীর্ঘমেয়াদি Access Key কেন এড়িয়ে চলা উচিত?
5. Site-to-Site VPN ও Dedicated সংযোগের মধ্যে কখন কোনটি বেছে নেবেন?
6. Workload Placement-এর সিদ্ধান্তে কোন উপাদানগুলো বিবেচনা করবেন?
7. Kubernetes কীভাবে বহনযোগ্যতা সাহায্য করে, আর কোথায় তা থেমে যায়?
8. Egress Cost কী? একটি বাস্তব উদাহরণ দিয়ে বোঝান কীভাবে তা কমানো যায়।
9. মাল্টি-ক্লাউডে গভর্ন্যান্স কেন কঠিন? কেন্দ্রীয় নিয়ন্ত্রণ স্তর কীভাবে সাহায্য করে?
10. একটি ছোট দলের জন্য কখন একক ক্লাউড ভালো? যুক্তি দিন।

১৮.১৭ ব্যবহারিক অনুশীলন

1. TechNova-র তিনটি ওয়ার্কলোড (web অ্যাপ, বিশ্লেষণ, নিয়ন্ত্রিত গ্রাহক ডেটা) নিয়ে একটি স্থাপন-সারণি বানান: প্রতিটি কোথায় চলবে, কেন, ডেটা কোথা থেকে কোথায় যাবে, আর Egress খরচের ঝুঁকি কোথায়।
2. একটি পরিচয়-ফেডারেশন নকশা আঁকুন: একজন প্রকৌশলী কীভাবে একবার সাইন ইন করে দুটি ক্লাউডে সাময়িক অনুমতি পাবেন — ধাপগুলো লিখুন।
3. একটি ক্লাউড-তুলনা নথি লিখুন: একটি নির্দিষ্ট ওয়ার্কলোডের জন্য দুটি ক্লাউডের সেবা, শক্তি, সীমা ও ঝুঁকি তুলনা করুন — সেবার নামের সাথে বর্তমান নথি যাচাইয়ের কথা উল্লেখ করুন।
4. একটি ডেটা-প্রবাহ মানচিত্র আঁকুন: TechNova-র ডেটা কোথায় তৈরি হয়, কোথায় প্রক্রিয়া হয়, কোথায় সংরক্ষিত হয় — আর প্রতিটি প্রবাহে Egress খরচের ঝুঁকি চিহ্নিত করুন।

অধ্যায় ১৯ • PART IV — Advanced Automation & Enterprise Operations

Chapter 19 — Advanced Observability and SRE

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Observability** (পর্যবেক্ষণযোগ্যতা) ও Monitoring-এর পার্থক্য ব্যাখ্যা করতে; মেট্রিক, লগ ও ট্রেসের তিনটি স্তম্ভ দিয়ে একটি পর্যবেক্ষণ আর্কিটেকচার নকশা করতে; **OpenTelemetry**, Prometheus ও Grafana-র ভূমিকা বলতে; Distributed Tracing দিয়ে অনুরোধের পথ অনুসরণ করতে; **SLI, SLO ও Error Budget** ঠিকভাবে হিসাব করতে — যার মধ্যে ৩০ দিনের মাসে ৯৯.৯৯% প্রাপ্যতার সঠিক হিসাবও আছে; এবং Incident Response, Reliability Review ও Operational Excellence-এর চর্চা TechNova-তে প্রয়োগ করতে।

- তিনটি স্তম্ভ (Metrics, Logs, Traces) একসাথে জুড়ে সমস্যা নির্ণয় করা।
- SLI/SLO/Error Budget লেখা ও তা দিয়ে রিলিজের সিদ্ধান্ত নেওয়া।
- একটি ঘটনার (Incident) প্রতিক্রিয়া-প্রক্রিয়া ও Blameless পর্যালোচনা চালানো।
- সতর্কতা (Alert) নকশা করা যাতে Alert Fatigue না হয়।

১৯.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার Kubernetes-এ অ্যাপ চালানো, লগ দেখার অভিজ্ঞতা (`kubectl logs`), HTTP স্ট্যাটাস কোডের ধারণা, এবং CI/CD পাইপলাইনের মৌলিক ধারণা জানা থাকা দরকার। **Monitoring** মানে জানা সংকেত দেখা — যেমন CPU বেশি কি না। **Observability** মানে অজানা প্রশ্নের উত্তর দেওয়ার ক্ষমতা — সিস্টেমের ভেতরের অবস্থা বাইরে থেকে বোঝা, এমন প্রশ্নেরও উত্তর দেওয়া যা আগে থেকে ভাবা হয়নি। **SRE (Site Reliability Engineering)** মানে নির্ভরযোগ্যতাকে প্রকৌশল সমস্যা হিসেবে দেখা — সফটওয়্যার দিয়ে পরিচালনার কাজ স্বয়ংক্রিয় করা।

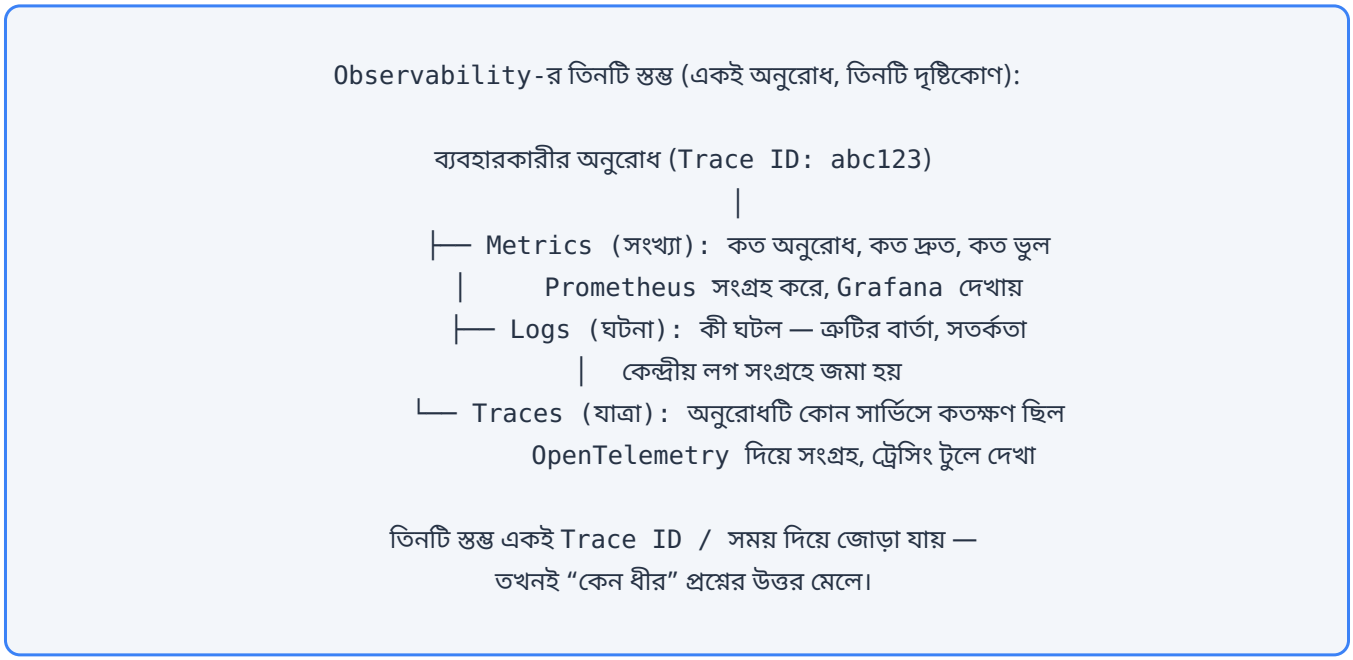
> **CONCEPT:** Monitoring জিজ্ঞেস করে “কিছু ভুল হচ্ছে কি?”; Observability জিজ্ঞেস করে “কেন ভুল হচ্ছে?” প্রথমটির জন্য ড্যাশবোর্ড যথেষ্ট; দ্বিতীয়টির জন্য মেট্রিক, লগ ও ট্রেস একসাথে জুড়তে পারা লাগে — একই অনুরোধের তিনটি দৃষ্টিকোণ।

১৯.৩ প্রথম নীতি থেকে: কেন পর্যবেক্ষণযোগ্যতা

আধুনিক সিস্টেমে একটি অনুরোধ অনেকগুলো সার্ভিস পেরিয়ে যায় — ওয়েব, API, ডেটাবেস, ক্যাশ, বাইরের সেবা। কোনো এক জায়গায় ধীর হলে ব্যবহারকারী শুধু “ধীর” দেখেন; কোথায় ধীর, তা বাইরে থেকে বোঝা যায় না। পর্যবেক্ষণযোগ্যতা এই অদৃশ্য পথটিকে দৃশ্যমান করে: প্রতিটি অনুরোধ একটি শনাক্তকারী (Trace ID) নিয়ে যাত্রা করে, প্রতিটি সার্ভিস তার সময় ও ফলাফল লিখে রাখে, আর সেই লেখাগুলো জুড়ে পুরো যাত্রাটি দেখা যায়। এই ক্ষমতা ছাড়া সমস্যা নির্ণয় হয় অনুমানে — “মনে হয় ডেটাবেস ধীর” — আর অনুমানে সমস্যা সমাধান করা যায় না, শুধু সময় নষ্ট হয়।

দ্বিতীয় নীতিটি হলো — যা মাপা যায় না, তা পরিচালনা করা যায় না। কিন্তু সবকিছু মাপা মানেই ভালো পর্যবেক্ষণ নয়। মেট্রিকের বন্যায় ডুবে যাওয়া দলগুলো প্রায়ই মূল প্রশ্নটি ভুলে যায়: ব্যবহারকারী কী অনুভব করছেন? তাই পর্যবেক্ষণের শুরুটি ব্যবহারকারীর অভিজ্ঞতা থেকে হওয়া উচিত — সফল অনুরোধের হার, প্রতিক্রিয়ার সময় — তারপর ভেতরের দিকে যাওয়া।

১৯.৪ পর্যবেক্ষণ আর্কিটেকচার: তিনটি স্তম্ভ



Metrics হলো সময়ের সাথে সংখ্যা — অনুরোধের সংখ্যা, ত্রুটির হার, প্রতিক্রিয়ার সময়ের শতকরা ভাগ (Percentile — যেমন p95 মানে ৯৫% অনুরোধ এই সময়ের মধ্যে শেষ হয়েছে)। Metrics সস্তা, দ্রুত, আর সতর্কতার জন্য উপযুক্ত। **Logs** হলো ঘটনার বিবরণ — কী ঘটল, কখন, কোন প্রসঙ্গে। Logs বিস্তারিত, কিন্তু বেশি হলে খরচ ও খোঁজার সময় বাড়ে। **Traces** হলো একটি অনুরোধের যাত্রার মানচিত্র — কোন সার্ভিসে কতক্ষণ লাগল। Traces ছাড়া Distributed সিস্টেমে ধীরতার মূল খুঁজে পাওয়া প্রায় অসম্ভব। তিনটি স্তম্ভের প্রতিটির নিজের কাজ আছে; কোনো একটিকে অন্যটির বিকল্প ভাবা ভুল।

১৯.৪.১ OpenTelemetry, Prometheus ও Grafana

OpenTelemetry হলো টেলিমেট্রি (Metrics, Logs, Traces) সংগ্রহের একটি উন্মুক্ত মান — একটি সাধারণ API ও ফরম্যাট, যাতে অ্যাপ্লিকেশন কোডে একবার যন্ত্র (Instrument) বসালে তা যেকোনো পর্যবেক্ষণ টুলে পাঠানো যায়। এর মূল্য হলো Lock-in এড়ানো: আজ যে টুলে পাঠাচ্ছেন, কাল অন্য টুলে পাঠাতে পারবেন — কোড বদলাতে হবে না। **Prometheus** হলো মেট্রিক সংগ্রহ ও সংরক্ষণের ব্যবস্থা — এটি লক্ষ্য (Target) থেকে নিয়মিত মেট্রিক টেনে নেয় (Pull), আর তার নিজের প্রশ্নের ভাষা (PromQL) দিয়ে জিজ্ঞেস করা যায়। **Grafana** হলো দৃশ্যমান করার স্তর — Prometheus ও অন্যান্য উৎস থেকে ড্যাশবোর্ড বানানো হয়। এই তিনটি একসাথে TechNova-র পর্যবেক্ষণ ভিত্তি হতে পারে: OpenTelemetry দিয়ে সংগ্রহ, Prometheus দিয়ে সংরক্ষণ, Grafana দিয়ে দেখা।

```
# Prometheus — ধারণা: কী দেখা হয়
# - http_requests_total: মোট অনুরোধ (সফল/ব্যর্থ ভাগসহ)
# - http_request_duration_seconds: প্রতিক্রিয়ার সময় (Histogram)
# - ত্রুটির হার = ব্যর্থ অনুরোধ / মোট অনুরোধ (একটি সময়-জানালায়)
# Grafana-তে ড্যাশবোর্ড: ত্রুটির হার, p95 লেটেন্সি, অনুরোধের হার —
#  এই তিনটি একসাথে দেখলে "ব্যবহারকারী কী অনুভব করছেন" বোঝা যায়।
```

১৯.৫ Distributed Tracing: অনুরোধের পথ অনুসরণ

একটি অনুরোধ যখন পাঁচটি সার্ভিস পেরিয়ে যায়, তখন প্রতিটি সার্ভিসে তার একটি অংশ (Span) তৈরি হয়, আর সব Span একই Trace ID দিয়ে জোড়া থাকে। ট্রেস দেখলে বোঝা যায়: অনুরোধটি মোট ৮০০ মিলিসেকেন্ড নিয়েছে, যার ৬০০

মিলিসেকেন্ডই গেছে একটি নির্দিষ্ট ডেটাবেস প্রশ্নে। এই দৃশ্যমানতা ছাড়া দলগুলো একে অপরকে দোষ দিত — "তোমার সার্ভিস ধীর" — আর আসল কারণটি লুকিয়ে থাকত। Tracing-এর জন্য কোডে যন্ত্র বসাতে হয় (Instrumentation) — OpenTelemetry এখানে সাহায্য করে — আর নমুনা নীতি (Sampling) ঠিক করতে হয়: সব অনুরোধের ট্রেস রাখা ব্যয়বহুল, তাই সাধারণত সব ত্রুটিপূর্ণ অনুরোধ ও ধীর অনুরোধ রাখা হয়, আর সফল অনুরোধের একটি অংশ।

> **REAL WORLD:** TechNova-র চেকআউট ধীর হয়ে গিয়েছিল — প্রতিটি দল বলছিল তাদের সার্ভিস ঠিক আছে। ট্রেস দেখে বোঝা গেল সমস্যাটি চেকআউট সার্ভিসে নয়, পেমেন্ট যাচাইয়ের একটি বাইরের কলে — যা কখনো কখনো পাঁচ সেকেন্ড নিচ্ছিল। ট্রেস ছাড়া এই অনুসন্ধানে দিন লাগত; ট্রেসসহ লেগেছিল এক ঘণ্টা।

১৯.৬ SLI, SLO ও Error Budget — নির্ভরযোগ্যতার গণিত

SLI (Service Level Indicator) মানে নির্ভরযোগ্যতার একটি মাপ — যেমন "সফল অনুরোধের হার" বা "p95 লেটেন্সি"। **SLO (Service Level Objective)** মানে সেই মাপের একটি লক্ষ্য — যেমন "৯৯.৯% অনুরোধ সফল হবে" বা "p95 লেটেন্সি ৩০০ মিলিসেকেন্ডের নিচে থাকবে"। **Error Budget** মানে SLO পূরণ না করার অনুমোদিত পরিমাণ — অর্থাৎ কতটুকু ভুল করা যাবে, তার হিসাব। Error Budget-এর ধারণাটি গুরুত্বপূর্ণ: এটি নির্ভরযোগ্যতা ও গতির মধ্যে একটি চুক্তি — বাজেট থাকলে নতুন ফিচার রিলিজ করা যায়; বাজেট শেষ হলে রিলিজ থামিয়ে নির্ভরযোগ্যতায় মনোযোগ দিতে হয়।

১৯.৬.১ ৯৯.৯৯% প্রাপ্যতার সঠিক হিসাব

Error Budget হিসাব করার সময় মাসের দৈর্ঘ্যটি পরিষ্কারভাবে বলা জরুরি, কারণ এখানেই একটি সাধারণ ভুল হয়। ৩০ দিনের মাসে মোট সময় ৩০ × ২৪ × ৬০ = ৪৩,২০০ মিনিট। ৯৯.৯৯% প্রাপ্যতা মানে ০.০১% সময় ডাউনটাইমের অনুমতি — অর্থাৎ ৪৩,২০০ × ০.০০০১ = ৪.৩২ মিনিট, অর্থাৎ **৪ মিনিট ১৯ সেকেন্ড**। এই সংখ্যাটি — ৪ মিনিট ১৯ সেকেন্ড — ৩০ দিনের মাসের জন্য সঠিক। আপনি কোথাও কোথাও ৪ মিনিট ২৩ সেকেন্ড দেখতে পারেন; সেই সংখ্যাটি একটি গড় মাস (~৩০.৪৪ দিন) ধরে হিসাব করা — অর্থাৎ বছরের গড়। মাসভিত্তিক SLO-তে ৩০ দিনের হিসাব ব্যবহার করলে ৪ মিনিট ১৯ সেকেন্ডই সঠিক, আর কোন ভিত্তিতে হিসাব করা হচ্ছে তা SLO নথিতে লিখে রাখা উচিত, নইলে দুটি দল দুটি সংখ্যা নিয়ে তর্ক করবে।

SLO (৩০ দিনের মাস)	অনুমোদিত ডাউনটাইম (Error Budget)	মানে কী
৯৯.৯% (three nines)	৪৩ মিনিট ১২ সেকেন্ড	মাসে প্রায় তিন-চতুর্থাংশ ঘণ্টা
৯৯.৯৫%	২১ মিনিট ৩৬ সেকেন্ড	অর্ধেকের মতো
৯৯.৯৯% (four nines)	৪ মিনিট ১৯ সেকেন্ড	খুবই কঠোর — প্রতি মাসে মাত্র চার মিনিট
১০০%	শূন্য	বাস্তবে অসম্ভব — এই লক্ষ্য রাখবেন না

এই সারণি থেকে একটি শিক্ষা: প্রতিটি অতিরিক্ত "৯" (Nine) খরচ অনেক বাড়িয়ে দেয়। ৯৯.৯% থেকে ৯৯.৯৯%-এ যাওয়া মানে Error Budget দশ ভাগের এক ভাগে নামিয়ে আনা — আর তা ধরে রাখতে অনেক বেশি প্রকৌশল, পর্যবেক্ষণ ও প্রস্তুতি লাগে। SLO বাছাইয়ের সময় ব্যবসায়িক প্রয়োজন জিজ্ঞেস করুন: ব্যবহারকারীর কি আসলেই ৯৯.৯৯% দরকার, নাকি ৯৯.৯% যথেষ্ট? প্রয়োজনের চেয়ে কঠোর SLO রাখা মানে অপ্রয়োজনীয় খরচ ও ধীর রিলিজ।

```
# Error Budget নীতি — ধারণা (TechNova)
# SLO: 99.9% সফল অনুরোধ (30 দিনের জানালা)
# Error Budget শেষ হলে: নতুন ফিচার রিলিজ থামবে,
#  শুধু নির্ভরযোগ্যতার কাজ চলবে (বাগ ঠিক, পারফরম্যান্স)।
```

```
# Error Budget অর্ধেকের নিচে গেলে: দলকে সতর্ক করা হবে;
#  বাজেট পুনরুদ্ধারের পরিকল্পনা লিখতে হবে।
```

১৯.৭ সতর্কতা নকশা: Alert Fatigue এড়ানো

Alert Fatigue মানে এত বেশি সতর্কতা আসা যে মানুষ আর সতর্কতাকে গুরুত্ব দেয় না — আর তখন আসল বিপদের সতর্কতাটিও উপেক্ষিত হয়। ভালো সতর্কতার নিয়ম: সতর্কতা তখনই আসবে যখন মানুষের এখনই কিছু করা দরকার। শুধু জানার জন্য হলে সেটি সতর্কতা নয় — সেটি ড্যাশবোর্ড বা টিকিট। প্রতিটি সতর্কতায় তিনটি জিনিস থাকা উচিত: কী ঘটছে, ব্যবহারকারীর উপর এর প্রভাব কী, আর প্রথম ধাপে কী করতে হবে (Runbook-এর লিংকসহ)। সতর্কতার তীব্রতা (Severity) স্তরভিত্তিক হওয়া উচিত — যেমন Page (এখনই জাগাতে হবে) বনাম Ticket (কাজের সময় দেখা যাবে)। আর নিয়মিত পর্যালোচনা করুন: কোন সতর্কতাগুলো বারবার আসে কিন্তু কিছু করা হয় না — সেগুলো হয় সীমা (Threshold) ভুল, নয়তো সতর্কতাটিই অপ্রয়োজনীয়।

১৯.৮ Incident Response: ঘটনার সময় কী করতে হয়

Incident মানে ব্যবহারকারীর উপর প্রভাব ফেলা একটি ব্যর্থতা — সাইট ডাউন, পেমেন্ট আটকে যাওয়া, ডেটা ভুল দেখানো। Incident Response-এর ধাপগুলো: (১) **সনাক্তকরণ** — সতর্কতা বা ব্যবহারকারীর অভিযোগ থেকে জানা; (২) **ঘোষণা ও সমন্বয়** — একজন **Incident Commander** দায়িত্ব নেন, যিনি সমস্যা সমাধান করেন না, সমন্বয় করেন; (৩) **প্রশমন (Mitigation)** — প্রথমে প্রভাব কমানো (যেমন ট্রাফিক সরানো, ফিচার বন্ধ করা), তারপর মূল কারণ খোঁজা; (৪) **সমাধান ও যাচাই** — ঠিক হয়েছে কি না, মেট্রিক দিয়ে যাচাই; (৫) **যোগাযোগ** — ব্যবহারকারী ও সংশ্লিষ্টদের নিয়মিত জানানো; (৬) **পর্যালোচনা (Postmortem)** — ঘটনার পর Blameless পর্যালোচনা। মনে রাখবেন: Incident-এর সময় লক্ষ্য “কে দোষী” খোঁজা নয়, “ব্যবহারকারীর ক্ষতি কীভাবে কমানো যায়” তা।

> **WARNING:** Incident-এর সময় একই সাথে অনেক মানুষ একই জিনিস ঠিক করার চেষ্টা করলে বিশৃঙ্খলা হয়। একজন Incident Commander থাকা অপরিহার্য — তিনি সিদ্ধান্ত নেন, কাজ ভাগ করেন, আর যোগাযোগের একটিই কণ্ঠস্বর রাখেন। Commander-এর ভূমিকা সবচেয়ে অভিজ্ঞ ব্যক্তির নয় — এটি একটি ভূমিকা, যে কেউ প্রশিক্ষণ নিয়ে পালন করতে পারেন।

১৯.৯ Reliability Review ও Operational Excellence

Reliability Review মানে নিয়মিত বসে দেখা — SLO কি পূরণ হচ্ছে, Error Budget কেমন চলছে, কোন ধরনের Incident বারবার হচ্ছে, আর কোন ব্যবস্থাগত (Systemic) সমস্যা আছে। এই পর্যালোচনা দোষারোপের জায়গা নয় — এটি একটি প্রকৌশল পর্যালোচনা: “সিস্টেমটি আমাদের কী শেখাচ্ছে?” **Operational Excellence** মানে এই চর্চাগুলোকে অভ্যাসে পরিণত করা: Runbook হালনাগাদ রাখা, Game Day বা Failure Simulation চালানো (২২তম অধ্যায়ে বিস্তারিত), Toil (বারবার করা হাতে-চালানো কাজ) কমানো, আর প্রতিটি Incident থেকে একটি করে স্থায়ী উন্নতি বের করা। SRE-র একটি নীতি হলো — Toil ৫০%-এর বেশি সময় নিলে নতুন ফিচারের কাজ থামিয়ে Toil কমাতে হয়, নইলে দলটি পরিচালনার কাজে ডুবে যায়।

১৯.১০ক পর্যবেক্ষণের পরিপক্বতা: ধাপে ধাপে

পর্যবেক্ষণও ধাপে ধাপে পরিপক্ব হয়। প্রথম ধাপে শুধু মৌলিক মেট্রিক ও লগ থাকে — CPU, মেমরি, ত্রুটির হার — আর সতর্কতা আসে থ্রেশহোল্ড ছাড়ালে। দ্বিতীয় ধাপে ব্যবহারকারী-কেন্দ্রিক SLI/SLO আসে, ড্যাশবোর্ডগুলো ব্যবহারকারীর

অভিজ্ঞতা দিয়ে সাজানো হয়, আর সতর্কতাগুলো কাজের সতর্কতায় পরিণত হয়। তৃতীয় ধাপে Distributed Tracing আসে — ধীরতার মূল খুঁজে পাওয়া যায় মিনিটে। চতুর্থ ধাপে Error Budget-ভিত্তিক সিদ্ধান্ত আসে — রিলিজের গতি বাজেট দিয়ে নিয়ন্ত্রিত হয়। প্রতিটি ধাপের নিজের মূল্য আছে — আর যে দল প্রথম ধাপের সমস্যা ছাড়াই চতুর্থ ধাপের টুল কিনে, তারা ব্যয়বহুল ড্যাশবোর্ড পায়, কিন্তু পর্যবেক্ষণযোগ্যতা পায় না। TechNova-র জন্য নিয়মটি: বর্তমান ধাপের সমস্যাটি সমাধান করুন, পরের ধাপের দরজা খোলা রাখুন — আর প্রতিটি ধাপে জিজ্ঞেস করুন, "এই পর্যবেক্ষণটি কি আমাদের কোনো সিদ্ধান্ত বদলেছে?" উত্তর "না" হলে পর্যবেক্ষণটি নয়, সিদ্ধান্তের প্রক্রিয়াটি দেখুন।

১৯.১০ TechNova প্রোডাকশন দৃশ্যকল্প

TechNova-র web অ্যাপের SLO ঠিক করা হলো: ৯৯.৯% সফল অনুরোধ, ৩০ দিনের জানালায় — অর্থাৎ মাসে ৪৩ মিনিট ১২ সেকেন্ড Error Budget। এক মাসে একটি খারাপ রিলিজের পর Error Budget ৮০% শেষ হয়ে গেল। SRE নীতি অনুযায়ী নতুন ফিচার রিলিজ থামানো হলো, আর দলটি দুই সপ্তাহ ধরে নির্ভরযোগ্যতার কাজ করল — ধীর ডেটাবেস প্রশ্ন ঠিক করা, সতর্কতার সীমা ঠিক করা, আর একটি Runbook লেখা। পরের মাসে Budget পুনরুদ্ধার হলো, আর রিলিজ আবার শুরু হলো। এই গল্পটি দেখায় Error Budget শুধু সংখ্যা নয় — এটি একটি সিদ্ধান্তের যন্ত্র: কখন গতি, কখন নির্ভরযোগ্যতা।

১৯.১১ নিরাপত্তা ও খরচ বিবেচনা

পর্যবেক্ষণ ডেটায় সংবেদনশীল তথ্য থাকতে পারে — লগে পাসওয়ার্ড, টোকেন বা ব্যক্তিগত ডেটা চলে আসতে পারে। তাই লগে সংবেদনশীল তথ্য না লেখার নিয়ম, লগের সংরক্ষণকাল (Retention) ঠিক করা, আর পর্যবেক্ষণ ব্যবস্থায় প্রবেশাধিকার নিয়ন্ত্রণ করা অপরিহার্য। খরচের দিক থেকে মনে রাখবেন: মেট্রিক সস্তা, লগ ও ট্রেস ব্যয়বহুল — বিশেষ করে সবকিছু দীর্ঘদিন রাখলে। সংরক্ষণকাল স্তরভিত্তিক রাখুন: সাম্প্রতিক বিস্তারিত ডেটা অল্পদিন, সংক্ষিপ্ত মেট্রিক দীর্ঘদিন। আর Sampling নীতি দিয়ে ট্রেসের পরিমাণ নিয়ন্ত্রণ করুন — সব ট্রেস রাখার দরকার নেই।

১৯.১২ সাধারণ ভুল

- **সবকিছুতে সতর্কতা:** সতর্কতা বেশি হলে মানুষ তা উপেক্ষা করে; শুধু কাজের সতর্কতা রাখুন।
- **গড় লেটেন্সি দেখা:** গড় লুকিয়ে রাখে খারাপ অভিজ্ঞতা; p95/p99 দেখুন।
- **১০০% SLO:** অসম্ভব লক্ষ্য দলকে হতাশ করে; বাস্তবসম্মত SLO রাখুন।
- **Trace ছাড়া Distributed নির্ণয়:** ট্রেস ছাড়া ধীরতার মূল খুঁজে পাওয়া যায় না।
- **Blame-ভিত্তিক Postmortem:** দোষারোপ করলে মানুষ সত্য লুকায়; Blameless পর্যালোচনা করুন।
- **লগে সিক্রেট লেখা:** লগে টোকেন বা পাসওয়ার্ড এলে তা ছড়িয়ে পড়ে।

১৯.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ড্যাশবোর্ড সবুজ, ব্যবহারকারী অভিযোগ করছেন	ভুল SLI — ভেতরের মেট্রিক, ব্যবহারকারীর অভিজ্ঞতা নয়	ব্যবহারকারী-কেন্দ্রিক SLI ঠিক করুন (সফল অনুরোধ, লেটেন্সি)
সতর্কতা আসে কিন্তু কেউ সাড়া দেয় না	Alert Fatigue; সতর্কতায় করণীয় নেই	সতর্কতা পর্যালোচনা করুন; Runbook লিংক যোগ করুন; অপ্রয়োজনীয় সরান
ধীরতা কিন্তু কোথায় জানা যায় না	Trace নেই বা Sampling খুব কম	Tracing চালু করুন; ধীর ও ব্যর্থ অনুরোধের Sampling বাড়ান

Error Budget বারবার শেষ হয়	SLO খুব কঠোর বা রিলিজ প্রক্রিয়া দুর্বল	SLO পুনর্বিবেচনা করুন; Canary ও যাচাই শক্ত করুন
Incident-এর পর একই ভুল আবার হয়	Postmortem-এর Action Item শেষ হয়নি	Action Item-এর মালিক ও সময়সীমা ঠিক করুন; পর্যালোচনায় যাচাই করুন

১৯.১৪ প্রকৌশল ট্রেড-অফ: কতটা পর্যবেক্ষণ

পর্যবেক্ষণেও ভারসাম্য দরকার। খুব কম পর্যবেক্ষণে আপনি অন্ধ; খুব বেশি পর্যবেক্ষণে আপনি ডেটার বন্যায় ডুবে যান, খরচ বাড়ে, আর মূল সংকেত হারিয়ে যায়। সঠিক জায়গাটি হলো: ব্যবহারকারীর অভিজ্ঞতার মূল মেট্রিকগুলো সবসময়, বিস্তারিত লগ ও ট্রেস প্রয়োজনে — আর সেই প্রয়োজনের সময় যেন তা হাতের কাছে থাকে। আরেকটি ট্রেড-অফ: নিজে পর্যবেক্ষণ ব্যবস্থা চালানো বনাম পরিচালিত সেবা ব্যবহার করা। নিজে চালালে নিয়ন্ত্রণ বেশি, কিন্তু রক্ষণাবেক্ষণের দায় আপনার; পরিচালিত সেবায় দায় কমে, কিন্তু খরচ ও Lock-in বাড়ে। TechNova-র মতো দলের জন্য পরিচালিত বা সহজে চালানো যায় এমন ব্যবস্থা দিয়ে শুরু করা, আর প্রয়োজনে গভীরে যাওয়া — এই ক্রমটি বাস্তবসম্মত।

> **TROUBLESHOOTING:** সমস্যা নির্ণয়ের সময় প্রথমে জিজ্ঞেস করুন — “ব্যবহারকারী কী দেখছেন?” তারপর Trace দিয়ে “কোথায় সময় যাচ্ছে?” দেখুন, তারপর Log দিয়ে “সেখানে কী ঘটছে?” দেখুন। এই ক্রমটি উল্টে দিলে আপনি ভেতরের বিবরণে ডুবে যাবেন, মূল সমস্যাটি হারিয়ে যাবে।

১৯.১৫ অধ্যায়ের সারাংশ

Observability মানে মেট্রিক, লগ ও ট্রেস — তিনটি স্তম্ভ দিয়ে সিস্টেমের ভেতর দেখতে পারা। OpenTelemetry সংগ্রহের মান, Prometheus মেট্রিকের সংরক্ষণ, Grafana দৃশ্যমানতা, আর Distributed Tracing অনুরোধের পথ দেখায়। SLI মাপে, SLO লক্ষ্য ঠিক করে, Error Budget গতি ও নির্ভরযোগ্যতার মধ্যে চুক্তি করে — ৩০ দিনের মাসে ৯৯.৯৯% মানে ৪ মিনিট ১৯ সেকেন্ড (৪ মিনিট ২৩ সেকেন্ড নয় — সেটি গড় মাসের হিসাব)। ভালো সতর্কতা, সুশৃঙ্খল Incident Response ও Blameless পর্যালোচনা — এই তিনটি ছাড়া পর্যবেক্ষণ শুধু ড্যাশবোর্ড হয়ে থাকে।

১৯.১৬ জ্ঞান যাচাই

1. Monitoring ও Observability-র পার্থক্য ব্যাখ্যা করুন।
2. Metrics, Logs ও Traces — প্রতিটির কাজ ও সীমা লিখুন।
3. OpenTelemetry কেন একটি মান হিসেবে গুরুত্বপূর্ণ?
4. Distributed Tracing কীভাবে ধীরতার মূল খুঁজে পেতে সাহায্য করে?
5. SLI, SLO ও Error Budget — তিনটির সংজ্ঞা ও সম্পর্ক ব্যাখ্যা করুন।
6. ৩০ দিনের মাসে ৯৯.৯৯% প্রাপ্যতার Error Budget হিসাব করে দেখান। ৪ মিনিট ২৩ সেকেন্ড সংখ্যাটি কেন ভিন্ন?
7. ৯৯.৯% থেকে ৯৯.৯৯%-এ যাওয়ার খরচ কেন অনেক বেশি? ব্যাখ্যা করুন।
8. Alert Fatigue কী? ভালো সতর্কতার তিনটি গুণ লিখুন।
9. Incident Response-এর ধাপগুলো লিখুন। Incident Commander-এর ভূমিকা কী?
10. Blameless Postmortem কেন গুরুত্বপূর্ণ? Toil কী?

১৯.১৭ ব্যবহারিক অনুশীলন

1. TechNova-র web অ্যাপের জন্য তিনটি SLI ও SLO লিখুন: একটি প্রাপ্যতার, একটি লেটেন্সির, একটি সফলতার হারের। প্রতিটির Error Budget ৩০ দিনের মাসে হিসাব করুন।
2. একটি সতর্কতা-পর্যালোচনা করুন (কাল্পনিক তালিকা দিয়ে): কোন সতর্কতাগুলো Page, কোনগুলো Ticket, কোনগুলো সরিয়ে দেওয়া উচিত — যুক্তিসহ লিখুন।
3. একটি Incident-এর কাল্পনিক দৃশ্য লিখুন (যেমন পেমেন্ট ধীর): Incident Commander হিসেবে প্রথম ৩০ মিনিটে কী করবেন, কাকে কী বলবেন, আর প্রশমন (Mitigation) কী হবে — তা ধাপে ধাপে লিখুন।
4. একটি Blameless Postmortem টেমপ্লেট বানান: কী ঘটেছিল, প্রভাব কী ছিল, মূল কারণ, আর তিনটি স্থায়ী Action Item (মালিক ও সময়সীমাসহ)।

অধ্যায় ২০ • PART IV — Advanced Automation & Enterprise Operations

Chapter 20 — Enterprise Security and Supply Chain Governance

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — নিরাপদ প্ল্যাটফর্ম আর্কিটেকচারের নীতিগুলো ব্যাখ্যা করতে; **IAM** (Identity and Access Management) ও **Zero Trust**-এর ধারণা প্রয়োগ করতে; সিক্রেট ব্যবস্থাপনা, **Policy as Code**, **SBOM** (Software Bill of Materials), আর্টিফ্যাক্ট স্বাক্ষর (Artifact Signing) ও সফটওয়্যার প্রমাণ (Provenance) ব্যাখ্যা করতে; Kubernetes নিরাপত্তা, কমপ্লায়েন্স, অডিট প্রমাণ ও নিরাপত্তা গভর্ন্যান্স TechNova-র প্রেক্ষাপটে নকশা করতে।

- Zero Trust-এর নীতিগুলো দিয়ে নেটওয়ার্ক ও পরিচয় নকশা করা।
- সফটওয়্যার সরবরাহ শৃঙ্খল (Supply Chain) আক্রমণের পথগুলো চিহ্নিত করা।
- SBOM, স্বাক্ষর ও Provenance দিয়ে আর্টিফ্যাক্ট যাচাইয়ের প্রক্রিয়া লেখা।
- একটি অডিট-প্রস্তুত প্রমাণ-তালিকা তৈরি করা।

২০.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার CI/CD পাইপলাইন, কনটেইনার ইমেজ, Kubernetes-এর Role-Based Access Control (RBAC)-এর প্রাথমিক ধারণা, এবং আগের অধ্যায়গুলোতে শেখা সিক্রেট ব্যবস্থাপনার নীতি জানা থাকা দরকার। **IAM** মানে কে (পরিচয়) কী (অনুমতি) করতে পারে, তার ব্যবস্থা। **Supply Chain** এখানে মানে সফটওয়্যার তৈরির শৃঙ্খল: সোর্স কোড, নির্ভরতা (Dependency), বিল্ড, আর্টিফ্যাক্ট, ডেপ্লয় — প্রতিটি ধাপে আক্রমণের সুযোগ থাকে। **Compliance** মানে নিয়ম ও মান মেনে চলা — যেমন ডেটা সুরক্ষা নিয়ম বা শিল্পের মান — আর তা প্রমাণ করতে পারা।

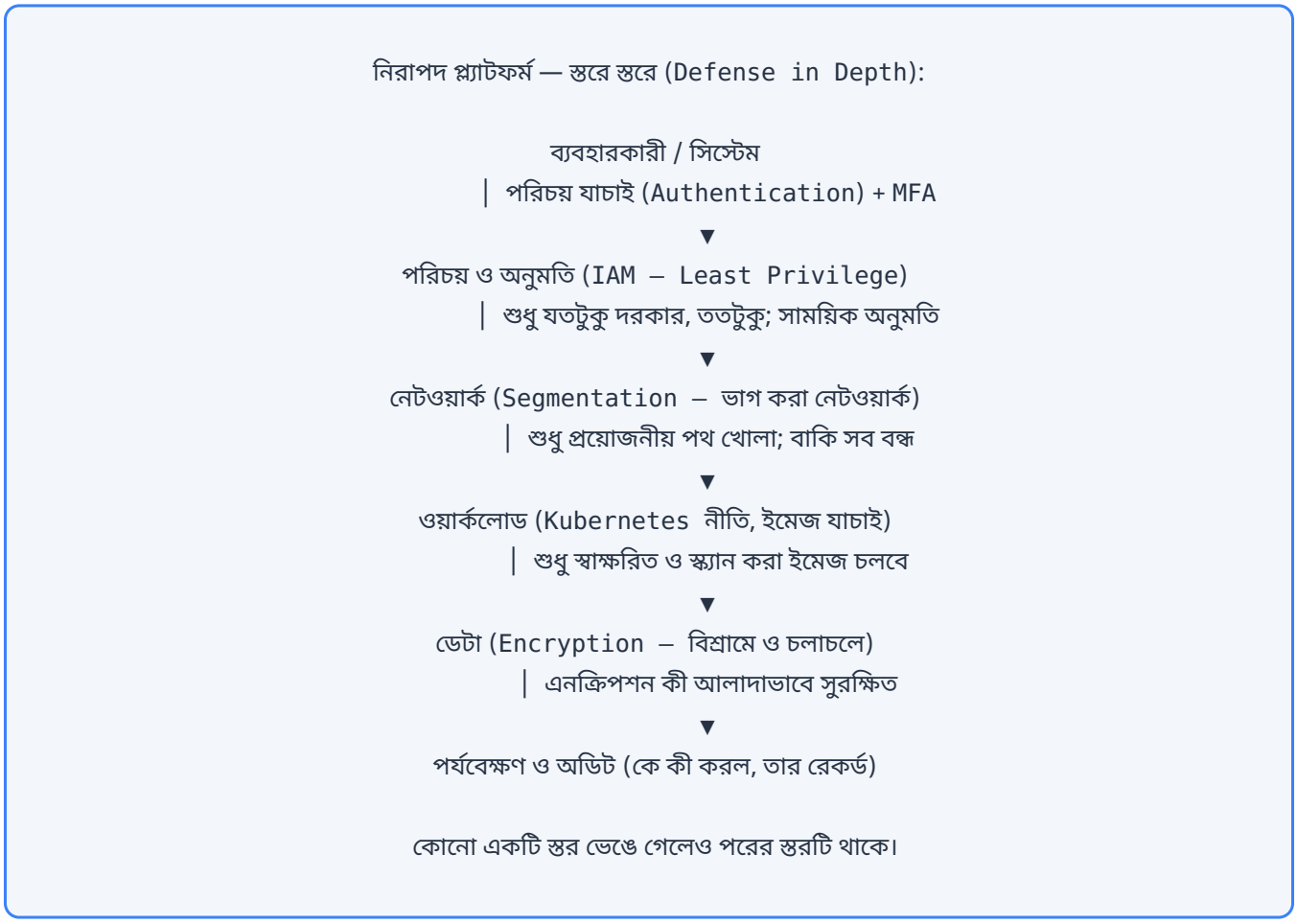
> **CONCEPT:** নিরাপত্তা একটি পণ্য নয়, একটি বৈশিষ্ট্য — আর সেটি শেষে যোগ করা যায় না। একটি পাইপলাইন যত দ্রুতই হোক, তাতে যদি অনিরাপদ আর্টিফ্যাক্ট প্রোডাকশনে যায়, তবে পাইপলাইনটি নিজেই ঝুঁকি। নিরাপদ প্ল্যাটফর্ম মানে নিরাপত্তা প্রতিটি ধাপে বোনা — কোড লেখা থেকে ডেপ্লয় পর্যন্ত।

২০.৩ প্রথম নীতি থেকে: বিশ্বাসের সীমানা কোথায়

ঐতিহ্যবাহী নিরাপত্তা ভাবত: ভেতরে নিরাপদ, বাইরে বিপজ্জনক — একটি দেয়াল (Perimeter) বানালেই হবে। কিন্তু ক্লাউড, দূরবর্তী কাজ ও তৃতীয় পক্ষের সেবার যুগে "ভেতর" বলে কিছু নেই — প্রকৌশলী বাড়ি থেকে কাজ করেন, অ্যাপ ক্লাউডে চলে, ডেটা তৃতীয় পক্ষের সেবায় যায়। তাই আধুনিক নীতিটি হলো: কোনো কিছুকে স্বয়ংক্রিয়ভাবে বিশ্বাস করা যাবে না — প্রতিটি অনুরোধ যাচাই করতে হবে: কে অনুরোধ করছে, কী চাইছে, কোন ডিভাইস থেকে, কোন প্রসঙ্গে। এই নীতিটির নাম **Zero Trust** — "কখনো বিশ্বাস করো না, সবসময় যাচাই করো" (Never Trust, Always Verify)। Zero Trust মানে বিশ্বাসের অভাব নয় — মানে বিশ্বাস অর্জন করতে হয়, প্রতিবার, প্রমাণ দিয়ে।

দ্বিতীয় নীতিটি হলো — নিরাপত্তা স্তরে স্তরে (Defense in Depth)। একটি স্তর ভেঙে গেলেও যেন পরের স্তরটি থাকে: নেটওয়ার্ক নীতি, পরিচয় যাচাই, অনুমতি, এনক্রিপশন, পর্যবেক্ষণ — কোনো একটিই যথেষ্ট নয়। আর তৃতীয় নীতি: নিরাপত্তা যত স্বয়ংক্রিয়, তত টেকসই। মানুষের স্মৃতিতে থাকা নিয়ম ভুলে যাওয়া যায়; কোডে লেখা নিয়ম প্রতিবার প্রয়োগ হয়।

২০.৪ নিরাপদ প্ল্যাটফর্ম আর্কিটেকচার



এই আর্কিটেকচারের প্রতিটি স্তর একটি প্রশ্নের উত্তর দেয়। পরিচয় স্তর জিজ্ঞেস করে “তুমি কে?” অনুমতি স্তর জিজ্ঞেস করে “তুমি কী করতে পারো?” নেটওয়ার্ক স্তর জিজ্ঞেস করে “তুমি কোথায় যেতে পারো?” ওয়ার্কলোড স্তর জিজ্ঞেস করে “এই কোডটি কি বিশ্বাসযোগ্য?” ডেটা স্তর জিজ্ঞেস করে “ডেটাটি কি সুরক্ষিত?” আর পর্যবেক্ষণ স্তর জিজ্ঞেস করে “কেউ কি অস্বাভাবিক কিছু করছে?” কোনো স্তর বাদ দিলে সেই প্রশ্নটির উত্তর দেওয়া হয় না — আর আক্রমণকারী ঠিক সেই ফাঁকটি খোঁজে।

২০.৫ IAM ও Zero Trust — বিস্তারিত

IAM-এর তিনটি অংশ: **Authentication** (আপনি কে — পাসওয়ার্ড, টোকেন, MFA), **Authorization** (আপনি কী করতে পারেন — Role ও Policy), আর **Accounting/Audit** (আপনি কী করলেন — লগ)। ভালো IAM-এর নিয়মগুলো: **Least Privilege** — শুধু যতটুকু দরকার, ততটুকু অনুমতি, আর তা নিয়মিত পর্যালোচনা; **Separation of Duties** — যে কোড লেখে, সে একা প্রোডাকশনে ডেপ্লয় করতে পারে না (অনুমোদন লাগে); **Short-lived Credentials** — সাময়িক শংসাপত্র, দীর্ঘমেয়াদি কী নয়; আর **Central Identity** — একটি পরিচয় উৎস, যাতে কেউ চলে গেলে এক জায়গায় অনুমতি বন্ধ করলেই সব জায়গায় বন্ধ হয়।

Zero Trust-এর বাস্তব প্রয়োগে তিনটি জিনিস লাগে: প্রথমত, প্রতিটি সার্ভিস একে অপরের পরিচয় যাচাই করে — শুধু “ভেতরের নেটওয়ার্কে আছে” বলে বিশ্বাস করা হয় না (এটিকে বলে **Mutual TLS** বা mTLS — দুই দিকেই শংসাপত্র দিয়ে পরিচয় যাচাই)। দ্বিতীয়ত, নেটওয়ার্ক ভাগ করা (Segmentation) — ডেটাবেস শুধু অ্যাপ সার্ভিসের সাথে কথা বলতে পারে, ইন্টারনেটের সাথে নয়। তৃতীয়ত, ক্রমাগত যাচাই — অনুমতি একবার পেলেই চিরদিনের জন্য নয়; প্রসঙ্গ বদলালে (যেমন

অস্বাভাবিক জায়গা থেকে সাইন-ইন) আবার যাচাই হয়। Zero Trust একদিনে হয় না — এটি একটি যাত্রা: প্রথমে পরিচয় শক্ত করা, তারপর নেটওয়ার্ক ভাগ করা, তারপর সার্ভিস-থেকে-সার্ভিস যাচাই।

২০.৬ সিক্রেট ব্যবস্থাপনা

Secret মানে এমন তথ্য যা ফাঁস হলে ক্ষতি হয় — পাসওয়ার্ড, API কী, শংসাপত্র, এনক্রিপশন কী। সিক্রেট ব্যবস্থাপনার নিয়মগুলো: সিক্রেট কখনো কোডে, Git-এ, বা পাইপলাইন ফাইলে লিখবেন না; সিক্রেট রাখুন একটি সিক্রেট ম্যানেজার বা ভল্টে — যেখানে প্রবেশাধিকার নিয়ন্ত্রিত, আর প্রতিটি ব্যবহারের লগ থাকে; সিক্রেট অ্যাপে পৌঁছাবে পরিবেশ চলক বা মাউন্ট করা ফাইলের মাধ্যমে, কোডের মাধ্যমে নয়; সিক্রেট নিয়মিত ঘোরানো (Rotation) উচিত — অর্থাৎ পুরোনো সিক্রেট বদলে নতুন দেওয়া; আর সিক্রেট ফাঁস হলে তা বাতিল (Revoke) করার পথ থাকতে হবে। TechNova-তে একটি সাধারণ ভুল হলো সিক্রেটকে `CHANGE-ME-...` প্লেসহোল্ডার থেকে বাস্তব মানে বদলে Git-এ কমিট করা — প্লেসহোল্ডারটি নথিতে থাকুক, বাস্তব মানটি ভল্টে থাকুক।

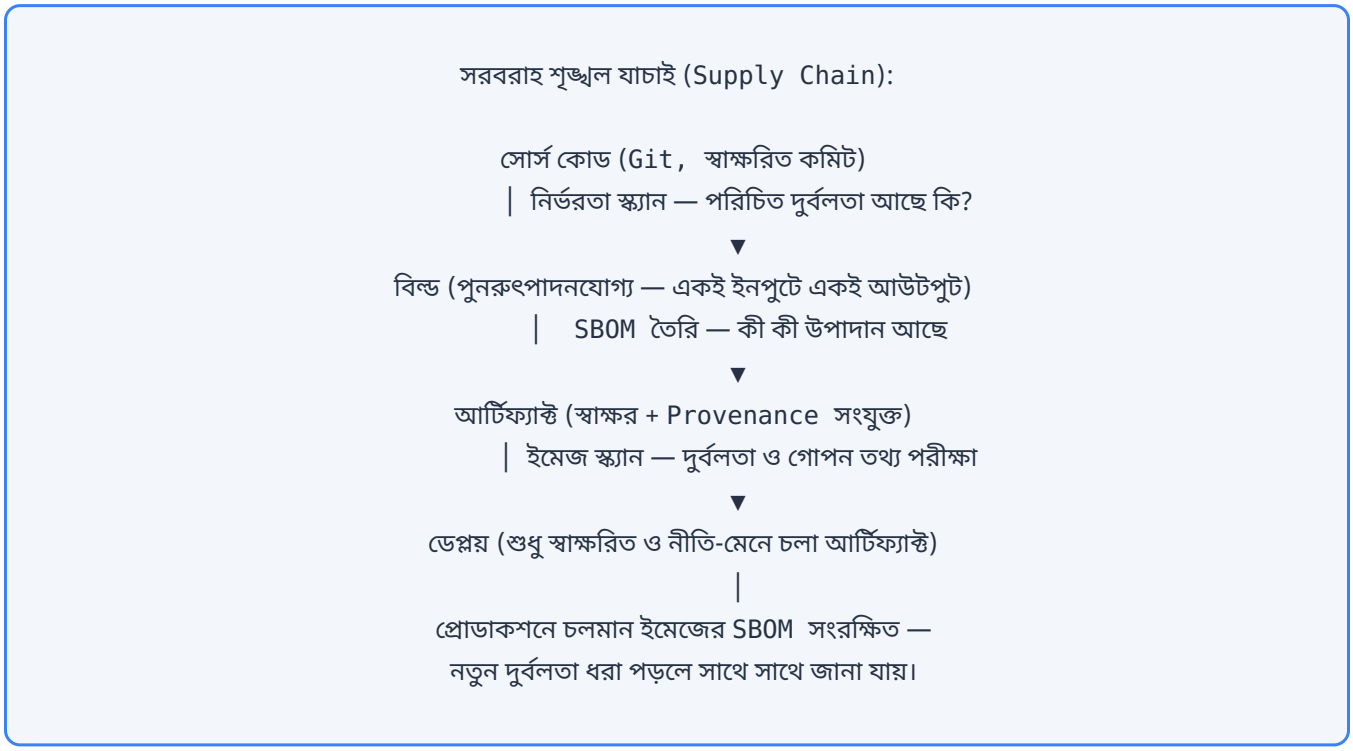
```
# সিক্রেট ব্যবস্থাপনা — নীতিগত উদাহরণ
# ভুল: DB_PASSWORD=real-password-123  (কোডে / Git-এ)
# সঠিক: DB_PASSWORD=CHANGE-ME-DB-PASSWORD  (শুধু প্লেসহোল্ডার)
# বাস্তব মান: ভল্ট থেকে পরিবেশ চলক হিসেবে আসে;
#  কে, কখন সিক্রেটটি পড়ল, তার লগ থাকে।
# kubectl-এ সিক্রেট: etcd-তে এনক্রিপ্ট করা উচিত;
#  সিক্রেটের মান কখনো লগে ছাপাবেন না।
```

২০.৭ Policy as Code: নীতি কোডে লেখা

Policy as Code মানে নিরাপত্তা ও পরিচালনার নিয়ম মানুষের নথিতে নয়, মেশিন-পাঠযোগ্য কোডে লেখা — আর সেই কোড দিয়ে স্বয়ংক্রিয়ভাবে পরীক্ষা করা। উদাহরণ: “কোনো স্টোরেজ সবার জন্য খোলা থাকবে না”, “প্রতিটি ইমেজ স্ক্যান করা থাকতে হবে”, “প্রোডাকশনে শুধু নির্দিষ্ট নেমস্পেসে ডেপ্লয় হবে” — এই নিয়মগুলো কোডে লিখলে CI পাইপলাইন বা Kubernetes Admission Controller স্বয়ংক্রিয়ভাবে তা প্রয়োগ করতে পারে। নীতি লেখার টুল হিসেবে Open Policy Agent (OPA) ও তার ভাষা Rego পরিচিত; Kubernetes-এ Gatekeeper বা Kyverno এই নীতি প্রয়োগ করে। নীতির সুবিধা: নিয়মটি সব জায়গায় একইভাবে প্রয়োগ হয়, আর নিয়ম ভাঙলে তা ধরা পড়ে — মানুষের মনে করিয়ে দেওয়ার উপর নির্ভর করে না। তবে নীতি খুব কঠোর হলে দলগুলো কাজ করতে পারে না — তাই নীতি প্রথমে সতর্কতা (Warn) হিসেবে চালু করুন, তারপর প্রয়োগ (Enforce) করুন।

২০.৮ SBOM, আর্টিফ্যাক্ট স্বাক্ষর ও Provenance

সফটওয়্যার সরবরাহ শৃঙ্খলের আক্রমণগুলো প্রায়ই নির্ভরতা বা বিল্ড প্রক্রিয়ার মধ্য দিয়ে আসে — একটি জনপ্রিয় লাইব্রেরিতে ক্ষতিকর কোড ঢোকানো, বা বিল্ড সার্ভারে হস্তক্ষেপ। এর বিরুদ্ধে তিনটি হাতিয়ার আছে। **SBOM (Software Bill of Materials)** মানে আপনার সফটওয়্যারে কী কী উপাদান আছে, তার তালিকা — কোন লাইব্রেরি, কোন সংস্করণ, কোন উৎস থেকে। SBOM থাকলে একটি লাইব্রেরিতে দুর্বলতা ধরা পড়লে আপনি সাথে সাথে জানতে পারেন আপনার কোন অ্যাপে সেই লাইব্রেরি আছে। **Artifact Signing** মানে আর্টিফ্যাক্টে (ইমেজ, প্যাকেজ) একটি ডিজিটাল স্বাক্ষর দেওয়া — যাতে ডেপ্লয়ের সময় যাচাই করা যায় আর্টিফ্যাক্টটি আসলেই আপনার বিল্ড থেকে এসেছে, মাঝপথে বদলানো হয়নি। **Provenance** মানে আর্টিফ্যাক্টটি কীভাবে তৈরি হলো, তার প্রমাণ — কোন সোর্স কোড, কোন বিল্ড, কে চালিয়েছিল, কোন ধাপ পেরিয়েছিল। এই তিনটি একসাথে বলে: “এই আর্টিফ্যাক্টটি কী দিয়ে তৈরি, কে বানিয়েছে, আর মাঝপথে কেউ বদলায়নি।”



এই শৃঙ্খলের প্রতিটি ধাপে একটি প্রশ্ন: এই ধাপে কেউ মিথ্যা কিছু ঢোকাতে পারে কি? উত্তরটি যদি “হ্যাঁ” হয়, তবে সেই ধাপে একটি যাচাই যোগ করুন — স্ক্যান, স্বাক্ষর, বা অনুমোদন।

২০.৯ Kubernetes নিরাপত্তা

Kubernetes নিরাপত্তার মূল বিষয়গুলো: **RBAC** — কে কোন রিসোর্সে কী করতে পারে, তা ভূমিকা দিয়ে নিয়ন্ত্রণ; **Network Policy** — কোন Pod কোন Pod-এর সাথে কথা বলতে পারে, তা সীমিত করা (পূর্বনির্ধারিতভাবে সবকিছু খোলা থাকে — এটাই বিপদ); **Pod Security** — Pod-গুলো Root হিসেবে চলবে না, শুধু প্রয়োজনীয় অনুমতি পাবে; **Image Policy** — শুধু বিশ্বস্ত রেজিস্ট্রির স্বাক্ষরিত ইমেজ চলবে; **Secrets** — সিক্রেট etcd-তে এনক্রিপ্ট করা ও শুধু প্রয়োজনীয় Pod-এ দেওয়া; আর **API Server**-এর প্রবেশাধিকার সীমিত করা। একটি সাধারণ ভুল হলো সবাইকে `cluster-admin` দিয়ে দেওয়া — এটি Kubernetes-এর সর্বোচ্চ অনুমতি, আর তা পেলে কেউ পুরো ক্লাস্টার নিয়ন্ত্রণ করতে পারে। অনুমতি দিন ভূমিকা অনুযায়ী, আর নিয়মিত পর্যালোচনা করুন কে কী অনুমতি পেয়েছে।

```
# Kubernetes নিরাপত্তা — পরীক্ষার ধারণা
kubectl auth can-i --list --as=developer-1 -n technova-prod
# দেখুন: একজন ডেভেলপার প্রোডাকশনে কী করতে পারেন —
#  অপ্রয়োজনীয় অনুমতি থাকলে তা সরান।
# Network Policy: পূর্বনির্ধারিতভাবে সব Pod একে অপরের সাথে
#  কথা বলতে পারে — প্রয়োজনীয় পথ ছাড়া বাকি বন্ধ করুন।
```

২০.১০ কমপ্লায়েন্স, অডিট প্রমাণ ও নিরাপত্তা গভর্ন্যান্স

Compliance মানে নিয়ম মেনে চলা — কিন্তু “মেনে চলছি” বললেই হয় না, প্রমাণ দেখাতে হয়। **Audit Evidence** মানে সেই প্রমাণ: কে, কখন, কী পরিবর্তন করলেন (Git ইতিহাস), কোন পরীক্ষা চলল (পাইপলাইন লগ), কোন নীতি প্রয়োগ হলো (Policy পরীক্ষার ফলাফল), কোন আর্টিফ্যাক্ট ডেপ্লয় হলো (SBOM ও স্বাক্ষর), আর কে প্রোডাকশনে প্রবেশ করলেন (অ্যাক্সেস লগ)। এই প্রমাণগুলো স্বয়ংক্রিয়ভাবে সংগ্রহ করা উচিত — অডিটের সময় হাতে খুঁজে বের করা যায় না। **Security**

Governance মানে নিরাপত্তার সিদ্ধান্তের কাঠামো: কে নীতি ঠিক করেন, কে ব্যতিক্রম অনুমোদন করেন, কতদিন ব্যতিক্রম চলবে, আর কখন তা পর্যালোচনা হবে। ব্যতিক্রম ছাড়া নিয়ম চলে না — কিন্তু ব্যতিক্রমের মেয়াদ ও মালিক না থাকলে ব্যতিক্রমই নিয়ম হয়ে যায়।

> **REAL WORLD:** TechNova-র একটি গ্রাহক জিজ্ঞেস করল — "আপনাদের প্রোডাকশনে কে প্রবেশ করতে পারে, আর গত মাসে কে কী বদলেছে?" উত্তরটি Git ইতিহাস, পাইপলাইন লগ, IAM পর্যালোচনা ও SBOM থেকে এক ঘণ্টায় তৈরি করা গিয়েছিল — কারণ প্রমাণগুলো স্বয়ংক্রিয়ভাবে জমা হচ্ছিল। প্রমাণ জমা না থাকলে এই প্রশ্নের উত্তর দিতে সপ্তাহ লাগত — আর গ্রাহকের বিশ্বাস হারানো যেত।

২০.১১ক নিরাপত্তা ঘটনার প্রতিক্রিয়া: প্রস্তুতি

নিরাপত্তা ঘটনা (Security Incident) — যেমন সিক্রেট ফাঁস, অননুমোদিত প্রবেশ, বা সন্দেহজনক কার্যকলাপ — ঘটলে কী করতে হবে, তা আগে থেকে লেখা থাকা উচিত। প্রস্তুতির ধাপগুলো: প্রথমত, সিক্রেট ফাঁস হলে সাথে সাথে তা বাতিল (Revoke) ও ঘোরানো (Rotate) করা — আর ফাঁসের পরিধি পরীক্ষা করা (কোথায় কোথায় সিক্রেটটি ব্যবহৃত হয়েছে, কে তা দেখেছে)। দ্বিতীয়ত, অননুমোদিত প্রবেশ সন্দেহ হলে সংশ্লিষ্ট অনুমতি সাময়িকভাবে বন্ধ করা, লগ সংরক্ষণ করা (তদন্তের জন্য), আর প্রভাবিত সিস্টেমগুলো চিহ্নিত করা। তৃতীয়ত, যোগাযোগ — কাকে জানাতে হবে (দলের নেতা, গ্রাহক, নিয়ন্ত্রক — প্রয়োজনে), আর কী জানাতে হবে (কী ঘটেছে, কী করা হচ্ছে, পরের আপডেট কখন)। চতুর্থত, পর্যালোচনা — ঘটনার পর Blameless পর্যালোচনা, আর স্থায়ী উন্নতি (যেমন সিক্রেটের স্বয়ংক্রিয় স্ক্যান, অনুমতির পর্যালোচনা)। নিরাপত্তা ঘটনার মহড়া (Tabletop Exercise) নিয়মিত চালান — বাস্তব ঘটনার সময় প্রথমবার ভাবতে হলে অনেক দেরি হয়ে যায়।

২০.১১ TechNova প্রোডাকশন দৃশ্যকল্প

TechNova-র web অ্যাপের জন্য নিরাপত্তা পরিকল্পনাটি এমন: (১) সব প্রকৌশলী কেন্দ্রীয় পরিচয় দিয়ে সাইন ইন করেন, MFA বাধ্যতামূলক; (২) প্রোডাকশন ক্লাস্টারে ডেভেলপারদের শুধু পড়ার অনুমতি, ডেপ্লয় হয় পাইপলাইনের মাধ্যমে, অনুমোদনসহ; (৩) প্রতিটি ইমেজ বিল্ডে SBOM তৈরি হয়, দুর্বলতা স্ক্যান হয়, আর স্বাক্ষর করা হয় — স্বাক্ষর ছাড়া ইমেজ প্রোডাকশনে চলবে না; (৪) Network Policy দিয়ে ডেটাবেস শুধু অ্যাপের সাথে কথা বলে; (৫) সব সিক্রেট ভল্টে, নিয়মিত ঘোরানো হয়; (৬) প্রতিটি পরিবর্তনের প্রমাণ — Git, পাইপলাইন লগ, SBOM — সংরক্ষিত থাকে। এই পরিকল্পনাটি একদিনে হয়নি — এটি ধাপে ধাপে তৈরি হয়েছে, সবচেয়ে বড় ঝুঁকি থেকে শুরু করে।

২০.১২ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তার খরচ দুই ধরনের: সরাসরি খরচ (টুল, পরিচালিত সেবা, সময়) ও পরোক্ষ খরচ (গতি কমে যাওয়া)। অতিরিক্ত কঠোর নিয়ম দলকে ধীর করে — আর তখন দল নিয়ম এড়িয়ে চলতে শুরু করে, যা আরও বিপজ্জনক। তাই নিরাপত্তা ঝুঁকি-ভিত্তিক হওয়া উচিত: সবচেয়ে বড় ঝুঁকিতে সবচেয়ে শক্ত নিয়ম, ছোট ঝুঁকিতে হালকা নিয়ম। আর মনে রাখবেন: নিরাপত্তা ভঙ্গের (Breach) খরচ — ডেটা হারানো, গ্রাহকের বিশ্বাস হারানো, আইনি জটিলতা — নিরাপত্তায় বিনিয়োগের চেয়ে অনেক বড় হয়। কিন্তু সেই যুক্তি দিয়ে অপ্রয়োজনীয় টুল কেনা যাবে না — প্রতিটি নিয়ন্ত্রণের (Control) জন্য জিজ্ঞেস করুন: "এটি কোন ঝুঁকি কমায়, আর কতটুকু?"

২০.১৩ সাধারণ ভুল

- **সবাইকে cluster-admin:** সর্বোচ্চ অনুমতি সবার হাতে দেওয়া — সবচেয়ে সাধারণ ও বিপজ্জনক ভুল।
- **সিক্রেট Git-এ:** সিক্রেট কোডে লিখলে তা ইতিহাসে থেকে যায় — মুছে ফেললেও Git ইতিহাসে থাকে।

- **স্ক্যান ছাড়া ইমেজ:** অপরীক্ষিত ইমেজ প্রোডাকশনে চালানো।
- **SBOM ছাড়া নির্ভরতা:** কোন লাইব্রেরি কোথায় আছে না জানলে দুর্বলতা ধরা পড়লে সাড়া দেওয়া যায় না।
- **নীতি শুধু নথিতে:** কোডে প্রয়োগ না হলে নীতি শুধু আশা।
- **লগে সিক্রেট:** লগে টোকেন ছাপা হলে তা ছড়িয়ে পড়ে।

২০.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ডেপ্লয় নীতি-পরীক্ষায় আটকে যায়	ইমেজ স্বাক্ষর নেই বা নীতি ভাঙছে	পরীক্ষার বার্তা পড়ুন; স্বাক্ষর ও স্ক্যানের ধাপ যাচাই করুন
Pod অন্য Pod-এর সাথে কথা বলতে পারে না	Network Policy খুব কঠোর	নীতি পর্যালোচনা করুন; প্রয়োজনীয় পথ যোগ করুন; প্রথমে Warn মোডে পরীক্ষা করুন
সিক্রেট ফাঁস হয়েছে বলে সন্দেহ	সিক্রেট লগে বা Git-এ এসেছে	সাথে সাথে সিক্রেট বাতিল ও ঘোরান; ফাঁসের পরিধি পরীক্ষা করুন
দুর্বলতা ধরা পড়ল, কোথায় আছে জানা যায় না	SBOM নেই বা হালনাগাদ নয়	SBOM তৈরি করুন; প্রতিটি আর্টিফ্যাক্টের SBOM সংরক্ষণ করুন
অডিটে প্রমাণ খুঁজে পাওয়া যায় না	প্রমাণ স্বয়ংক্রিয়ভাবে জমা হয় না	প্রমাণ সংগ্রহ স্বয়ংক্রিয় করুন; সংরক্ষণকাল ঠিক করুন

২০.১৫ প্রকৌশল ট্রেড-অফ: নিরাপত্তা বনাম গতি

নিরাপত্তা ও গতি — দুটির মধ্যে টানাপোড়েন চিরকাল থাকবে। প্রতিটি অতিরিক্ত যাচাই ডেপ্লয়কে ধীর করে; প্রতিটি বাদ দেওয়া যাচাই ঝুঁকি বাড়ায়। সঠিক ভারসাম্যটি ঝুঁকির উপর নির্ভর করে: পেমেন্ট বা ব্যক্তিগত ডেটা নিয়ে কাজ করলে কঠোর যাচাই যুক্তিযুক্ত; অভ্যন্তরীণ পরীক্ষামূলক টুলে হালকা যাচাই যথেষ্ট। আর একটি ট্রেড-অফ: নিজে নিরাপত্তা টুল চালানো বনাম পরিচালিত সেবা। নিজে চালালে নিয়ন্ত্রণ বেশি, কিন্তু রক্ষণাবেক্ষণের দায় আপনার; পরিচালিত সেবায় দায় কমে। TechNova-র মতো দলের জন্য নিয়মটি হলো — সবচেয়ে বড় ঝুঁকিগুলোতে শক্ত নিয়ন্ত্রণ, আর নিয়ন্ত্রণগুলোকে যতটা সম্ভব স্বয়ংক্রিয় করা, যাতে নিরাপত্তা গতির বাধা না হয়ে গতির অংশ হয়।

> **TROUBLESHOOTING:** নিরাপত্তা সমস্যার সময় প্রথম প্রশ্নটি হলো — “এই অনুমতি বা পথটি আসলে কার দরকার, আর কেন?” অপ্রয়োজনীয় অনুমতি সরানোই সবচেয়ে দ্রুত ও কার্যকর নিরাপত্তা উন্নতি — নতুন টুল কেনার চেয়ে।

২০.১৬ অধ্যায়ের সারাংশ

এন্টারপ্রাইজ নিরাপত্তা মানে স্তরে স্তরে প্রতিরক্ষা: কেন্দ্রীয় পরিচয়, Least Privilege, Zero Trust, নেটওয়ার্ক ভাগ, Kubernetes নীতি, এনক্রিপশন ও পর্যবেক্ষণ। সিক্রেট ভল্টে থাকে, নীতি কোডে লেখা হয়, আর সরবরাহ শৃঙ্খল SBOM, স্বাক্ষর ও Provenance দিয়ে যাচাই করা হয়। কমপ্লায়েন্স মানে নিয়ম মানা ও তার প্রমাণ স্বয়ংক্রিয়ভাবে জমা করা। নিরাপত্তা ও গতির ভারসাম্যটি ঝুঁকি দিয়ে ঠিক হয় — সবচেয়ে বড় ঝুঁকিতে সবচেয়ে শক্ত নিয়ন্ত্রণ।

২০.১৭ জ্ঞান যাচাই

1. Zero Trust বলতে কী বোঝায়? ঐতিহ্যবাহী Perimeter নিরাপত্তা থেকে এর পার্থক্য কী?
2. Defense in Depth ব্যাখ্যা করুন — TechNova-র উদাহরণ দিয়ে।
3. Least Privilege ও Separation of Duties — দুটির উদাহরণ দিন।
4. সিক্রেট ব্যবস্থাপনার পাঁচটি নিয়ম লিখুন।
5. Policy as Code কী? নীতি প্রথমে Warn মোডে চালু করা কেন ভালো?
6. SBOM কী? একটি লাইব্রেরিতে দুর্বলতা ধরা পড়লে SBOM কীভাবে সাহায্য করে?
7. Artifact Signing ও Provenance — দুটির পার্থক্য ও সম্পর্ক ব্যাখ্যা করুন।
8. Kubernetes-এ RBAC, Network Policy ও Pod Security — প্রতিটির কাজ লিখুন।
9. Audit Evidence কী? অডিটের জন্য কোন প্রমাণগুলো স্বয়ংক্রিয়ভাবে জমা করা উচিত?
10. নিরাপত্তা বনাম গতি — ভারসাম্যটি কীভাবে ঠিক করবেন?

২০.১৮ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি IAM পর্যালোচনা-তালিকা বানান: কে প্রোডাকশনে কী করতে পারে, কোন অনুমতি অপ্রয়োজনীয় মনে হয়, আর কীভাবে তা সরাবেন — তা লিখুন।
2. একটি সরবরাহ-শৃঙ্খল যাচাই-তালিকা লিখুন: সোর্স থেকে ডেপ্লয় পর্যন্ত প্রতিটি ধাপে কী যাচাই হবে (স্ক্যান, SBOM, স্বাক্ষর) — তা ধাপে ধাপে লিখুন।
3. একটি Network Policy-র পরিকল্পনা করুন: TechNova-র web, API ও ডেটাবেস — কোনটি কোনটির সাথে কথা বলতে পারবে, আর কোন পথ বন্ধ থাকবে — তা সারণিতে লিখুন।
4. একটি অডিট-প্রমাণ নথির কাঠামো বানান: একটি রিলিজের জন্য কোন প্রমাণগুলো সংরক্ষণ করবেন, কোথায় রাখবেন, আর কতদিন রাখবেন — তা লিখুন।

PART V • DEVOPS ENGINEERING MASTERY

Production Architecture & Final Capstone

Enterprise Platform Architecture, Capacity, Cost, Reliability ও DR Engineering, Engineering Leadership ও Operational Ownership, Final Enterprise DevOps Platform Capstone।

অধ্যায় ২১ • PART V — Production Architecture & Final Capstone

Chapter 21 — Enterprise Platform Architecture

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — এন্টারপ্রাইজ প্ল্যাটফর্মের রেফারেন্স আর্কিটেকচার আঁকতে; বহু-দলীয় (Multi-Team) প্রকৌশলে শেয়ার্ড অবকাঠামো, ডেভেলপার অভিজ্ঞতা (Developer Experience) ও প্ল্যাটফর্ম সীমানা (Boundary) নকশা করতে; আর্কিটেকচার মান, সিস্টেম ইন্টিগ্রেশন, নথিভুক্তকরণ, প্রযুক্তিগত গভর্ন্যান্স ও প্ল্যাটফর্ম জীবনচক্র (Lifecycle) TechNova-র প্রেক্ষাপটে প্রয়োগ করতে।

- একটি প্ল্যাটফর্ম রেফারেন্স আর্কিটেকচারের স্তরগুলো ব্যাখ্যা করা।
- প্ল্যাটফর্ম সীমানা ঠিক করা: প্ল্যাটফর্ম কী করবে, দল কী করবে।
- ডেভেলপার অভিজ্ঞতা মাপা ও উন্নত করার উপায় লেখা।
- একটি আর্কিটেকচার সিদ্ধান্তের রেকর্ড (ADR) লেখা।

২১.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার আগের অধ্যায়গুলোর ধারণাগুলো একসাথে আসে: Terraform মডিউল (অধ্যায় ১৭), প্ল্যাটফর্ম API ও Internal Developer Platform (অধ্যায় ১৭), GitOps ও CI/CD (অধ্যায় ১৬), মাল্টি-ক্লাউড (অধ্যায় ১৮), পর্যবেক্ষণ ও SRE (অধ্যায় ১৯), এবং নিরাপত্তা (অধ্যায় ২০)। **Platform Team** মানে যে দলটি অন্য দলগুলোর জন্য প্ল্যাটফর্ম বানায় ও চালায়; **Product Team** বা **Stream-Aligned Team** মানে যে দলটি ব্যবহারকারীর জন্য ফিচার বানায়। **Developer Experience (DX)** মানে একজন ডেভেলপারের প্ল্যাটফর্ম ব্যবহারের অভিজ্ঞতা — কত সহজে পরিবেশ পাওয়া যায়, কত দ্রুত ডেপ্লয় হয়, সমস্যা হলে কত দ্রুত উত্তর মেলে।

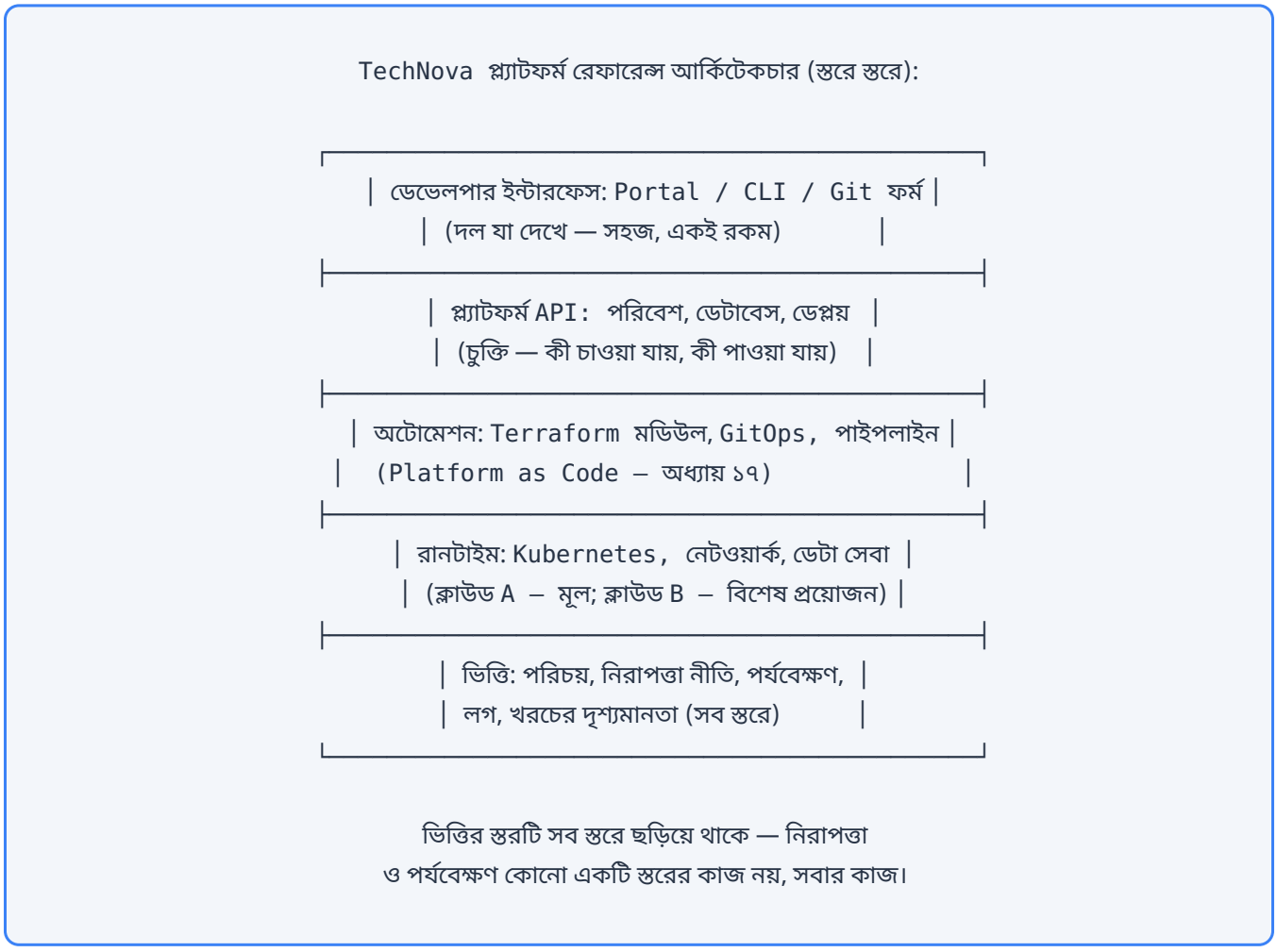
> **CONCEPT:** প্ল্যাটফর্ম একটি পণ্য, আর ডেভেলপাররা তার গ্রাহক। প্ল্যাটফর্ম দল যদি ডেভেলপারদের গ্রাহক হিসেবে না দেখে, তবে প্ল্যাটফর্মটি একটি নিয়মের বোঝা হয়ে যায় — দলগুলো তা এড়িয়ে চলতে শুরু করে, আর তখন প্ল্যাটফর্মের উদ্দেশ্যই ব্যর্থ হয়। ভালো প্ল্যাটফর্মের চিহ্ন হলো দলগুলো স্বেচ্ছায় তা ব্যবহার করে, কারণ তা তাদের কাজ সহজ করে।

২১.৩ প্রথম নীতি থেকে: প্ল্যাটফর্ম কেন দরকার

একটি দল থাকলে প্ল্যাটফর্ম দরকার হয় না — দলটি নিজেই সবকিছু সামলাতে পারে। পাঁচটি দল হলে প্রতিটি দল নিজের মতো অবকাঠামো বানাতে শুরু করে — পাঁচটি ভিন্ন পাইপলাইন, পাঁচটি ভিন্ন পর্যবেক্ষণ ব্যবস্থা, পাঁচটি ভিন্ন নিরাপত্তা নিয়ম। দশটি দল হলে এই বৈচিত্র্যই হয়ে যায় মূল খরচ: নতুন প্রকৌশলীকে পাঁচটি ভিন্ন ব্যবস্থা শিখতে হয়, নিরাপত্তা পর্যালোচনায় পাঁচটি ভিন্ন চিত্র দেখতে হয়, আর একটি ভুল পাঁচ জায়গায় পাঁচভাবে ঘটে। প্ল্যাটফর্ম এই বৈচিত্র্যকে একটি সাধারণ ভিত্তিতে আনে — কিন্তু সাবধানে: প্ল্যাটফর্মের উদ্দেশ্য সব দলকে একই ছাঁচে ঢালা নয়, উদ্দেশ্য হলো সাধারণ জিনিসগুলো একবার ভালোভাবে বানানো, যাতে দলগুলো তাদের আসল কাজে — ফিচার বানানোতে — মনোযোগ দিতে পারে।

এই ভারসাম্যটি মনে রাখা জরুরি: প্ল্যাটফর্ম যত বেশি কিছু করে, দলগুলোর স্বাধীনতা তত কমে; প্ল্যাটফর্ম যত কম করে, বৈচিত্র্য তত বাড়ে। সঠিক সীমানাটি সেখানেই, যেখানে সাধারণ প্রয়োজন শেষ হয় আর বিশেষ প্রয়োজন শুরু হয়।

২১.৪ প্ল্যাটফর্ম রেফারেন্স আর্কিটেকচার



এই আর্কিটেকচারের প্রতিটি স্তরের একটি দায়িত্ব আছে। ডেভেলপার ইন্টারফেসের দায়িত্ব সহজ হওয়া; প্ল্যাটফর্ম API-র দায়িত্ব পরিষ্কার চুক্তি রাখা; অটোমেশনের দায়িত্ব পুনরুৎপাদনযোগ্য হওয়া; রানটাইমের দায়িত্ব নির্ভরযোগ্য হওয়া; আর ভিত্তির দায়িত্ব সব জায়গায় নিরাপত্তা ও দৃশ্যমানতা দেওয়া। কোনো স্তর তার নিচের স্তরের বিবরণ উপরের স্তরে ফাঁস করলে আর্কিটেকচারটি ভেঙে যায় — যেমন ডেভেলপারকে ক্লাউডের ভেতরের বিবরণ জানতে হলে প্ল্যাটফর্ম API-র মূল্য থাকে না।

২১.৪.১ শেয়ার্ড অবকাঠামো: কী ভাগ করা হয়, কী নয়

শেয়ার্ড অবকাঠামো মানে যা সব দল ব্যবহার করে: Kubernetes ক্লাস্টার, CI/CD ব্যবস্থা, পর্যবেক্ষণ, লগ সংগ্রহ, সিক্রেট ব্যবস্থাপনা, DNS, শংসাপত্র ব্যবস্থাপনা। ভাগ করার সুবিধা: একবার বানানো, সবাই পায়; নিরাপত্তা ও খরচ এক জায়গায় দেখা যায়। ঝুঁকি: শেয়ার্ড জিনিস ভাঙলে সব দল ভাঙে — একে বলে **Blast Radius** (ক্ষতির পরিধি)। তাই শেয়ার্ড অবকাঠামোতে দুটি নিয়ম: প্রথমত, দলগুলোর মধ্যে বিচ্ছিন্নতা (Isolation) — একটি দলের ভুল যেন অন্য দলকে না ছোঁয় (নেমস্পেস, কোটা, নেটওয়ার্ক নীতি দিয়ে); দ্বিতীয়ত, পরিবর্তনের শৃঙ্খলা — শেয়ার্ড জিনিস বদলানোর আগে সব দলকে জানানো ও ধাপে ধাপে বদলানো।

২১.৫ প্ল্যাটফর্ম সীমানা (Boundary): কে কী করবে

প্ল্যাটফর্ম সীমানা ঠিক করা মানে লিখিতভাবে বলা: প্ল্যাটফর্ম দল কী দেয়, আর দলগুলো নিজেরা কী করে। উদাহরণ: প্ল্যাটফর্ম দেয় Kubernetes ক্লাস্টার, পাইপলাইন টেমপ্লেট, পর্যবেক্ষণ ও ডেটাবেস মডিউল; দল দেয় নিজের অ্যাপের কোড, পরীক্ষা, নিজের ড্যাশবোর্ড ও Runbook। সীমানা পরিষ্কার না থাকলে দুটি সমস্যা হয়: হয় প্ল্যাটফর্ম দল সবকিছু করতে গিয়ে বোতল-

ঘাড় হয়ে যায়, নয়তো দলগুলো নিজেরা শেয়ার্ড জিনিস বানাতে শুরু করে — আর বৈচিত্র্য ফিরে আসে। সীমানাটি একটি জীবন্ত নথি হওয়া উচিত — নিয়মিত পর্যালোচনা করে, দলগুলোর প্রয়োজনে বদলানো।

বিষয়	প্ল্যাটফর্ম দল দেয়	প্রোডাক্ট দল করে
পরিবেশ	পরিবেশ পাওয়ার পথ (API/ফর্ম)	নিজের পরিবেশের কনফিগারেশন চাওয়া
ডেপ্লয়	পাইপলাইন টেমপ্লেট ও GitOps	নিজের অ্যাপের ডেপ্লয় সংজ্ঞা
ডেটাবেস	ডেটাবেস মডিউল (ব্যাকআপসহ)	স্কিমা ও প্রশ্ন (Query) পরিচালনা
পর্যবেক্ষণ	মেট্রিক, লগ, ট্রেস সংগ্রহ	নিজের SLI, ড্যাশবোর্ড ও সতর্কতা
নিরাপত্তা	নীতি, স্ক্যান, সিক্রেট ব্যবস্থাপনা	নিজের কোডের নিরাপত্তা ও নির্ভরতা

২১.৬ ডেভেলপার অভিজ্ঞতা (DX): প্ল্যাটফর্মের সাফল্যের মাপ

ডেভেলপার অভিজ্ঞতা মাপা যায় — অনুমানে নয়। মাপার উপায়গুলো: **Time to First Deploy** — নতুন প্রকৌশলী কতদিনে প্রথম ডেপ্লয় করতে পারেন; **Time to Environment** — নতুন পরিবেশ পেতে কতক্ষণ লাগে; **Deploy Frequency ও Lead Time** — পরিবর্তন কমিট থেকে প্রোডাকশনে পৌঁছাতে কতক্ষণ লাগে; **Onboarding Survey** — নতুনরা কী কঠিন মনে করেন; আর **Support Load** — প্ল্যাটফর্ম দলের কাছে কত প্রশ্ন আসে, আর কোন প্রশ্ন বারবার আসে। বারবার আসা প্রশ্নগুলোই প্ল্যাটফর্মের দুর্বল জায়গা — সেগুলো নথি বা অটোমেশন দিয়ে ঠিক করুন, উত্তরের পুনরাবৃত্তি দিয়ে নয়। একটি ভালো অভ্যাস হলো নিয়মিত "ডেভেলপার যাত্রা" (Developer Journey) পর্যবেক্ষণ করা — একজন নতুন প্রকৌশলীর পাশে বসে দেখা, তিনি কোথায় আটকে যান।

> **REAL WORLD:** TechNova-তে নতুন প্রকৌশলীরা প্রথম ডেপ্লয় করতে গড়ে চার দিন নিতেন — কারণ পরিবেশ পেতে টিকিট, সিক্রেট পেতে আলাদা অনুরোধ, আর পাইপলাইন বুঝতে Wiki-র পাঁচটি পাতা লাগত। প্ল্যাটফর্ম দল একটি Portal বানাল — এক জায়গায় পরিবেশ, সিক্রেট ও পাইপলাইন — আর প্রথম ডেপ্লয়ের সময় নেমে এলো এক দিনে। মাপটি ছিল সহজ, আর উন্নতিটি ছিল দৃশ্যমান।

২১.৭ আর্কিটেকচার মান ও সিস্টেম ইন্টিগ্রেশন

Architecture Standards মানে সিস্টেম বানানোর সাধারণ নিয়ম: API কীভাবে লেখা হবে, সার্ভিসগুলো কীভাবে কথা বলবে, ডেটা কীভাবে সংরক্ষিত হবে, নামকরণ কেমন হবে। মানের উদ্দেশ্য একরূপতা নয় — উদ্দেশ্য হলো নতুন সিস্টেম যেন পুরোনো সিস্টেমের সাথে কথা বলতে পারে, আর নতুন প্রকৌশলী যেন প্রতিটি সার্ভিসে নতুন নিয়ম শিখতে না হয়। **System Integration** মানে ভিন্ন ভিন্ন সিস্টেমকে একসাথে কাজ করানো — যেমন বিলিং সিস্টেম, গ্রাহক ব্যবস্থাপনা ও অ্যাপের মধ্যে ডেটা প্রবাহ। ইন্টিগ্রেশনের জন্য তিনটি নীতি: একটি পরিষ্কার চুক্তি (API Contract) থাকা; ব্যর্থতা সামলানোর পরিকল্পনা (Retry, Timeout, Fallback) থাকা; আর ইন্টিগ্রেশনের পর্যবেক্ষণ থাকা — সিস্টেমগুলোর মধ্যে ডেটা আটকে গেলে তা যেন দেখা যায়।

২১.৭.১ Architecture Decision Record (ADR)

বড় সিদ্ধান্তগুলো লিখে রাখার উপায় হলো **ADR** — একটি ছোট নথি: প্রসঙ্গ কী ছিল, কোন বিকল্পগুলো বিবেচনা করা হয়েছিল, কোনটি বেছে নেওয়া হলো, কেন, আর কী পরিণতি হতে পারে। ADR-এর মূল্য ভবিষ্যতে: ছয় মাস পর কেউ যখন

জিজ্ঞেস করে “এই সিদ্ধান্তটি কেন নেওয়া হয়েছিল?”, তখন উত্তরটি নথিতে থাকে — মানুষের স্মৃতিতে নয়। প্রতিটি ADR-এর একটি সংখ্যা, তারিখ, অবস্থা (গৃহীত/বাতিল/প্রতিস্থাপিত) ও লেখক থাকা উচিত।

```
# ADR — কাঠামো (TechNova)
# শিরোনাম: ADR-014 — প্ল্যাটফর্ম API হিসেবে Git-ভিত্তিক ফর্ম বেছে নেওয়া
# অবস্থা: গৃহীত (2026-11-09)
# প্রসঙ্গ: দলগুলো পরিবেশ চাইতে টিকিট কাটত; সময় লাগত তিন দিন।
# বিকল্প: (ক) REST API বানানো; (খ) Git-এ YAML ফর্ম; (গ) Portal বানানো।
# সিদ্ধান্ত: (খ) — Git-ভিত্তিক ফর্ম; কারণ পর্যালোচনা Git-এ হয়,
#  আর Portal বানাতে সময় বেশি লাগত।
# পরিণতি: দলগুলোকে Git-ভিত্তিক প্রবাহ শিখতে হবে; Portal পরে বিবেচনা করা হবে।
```

২১.৮ নথিভুক্তকরণ, প্রযুক্তিগত গভর্ন্যান্স ও প্ল্যাটফর্ম জীবনচক্র

Documentation প্ল্যাটফর্মের অংশ — কোনো আলাদা কাজ নয়। ভালো নথির তিনটি স্তর: শুরু করার নির্দেশনা (Getting Started — প্রথম ডেপ্লয় পর্যন্ত), ধারণাগত নথি (কীভাবে কাজ করে — আর্কিটেকচার, সীমানা), আর রেফারেন্স (API, কনফিগারেশনের বিবরণ)। নথি হালনাগাদ না থাকলে তা ভুল তথ্য দেওয়ার চেয়ে খারাপ — মানুষ ভুল পথে যায়। তাই নথির মালিক ও পর্যালোচনার সময়সূচি থাকা উচিত।

Technical Governance মানে প্রযুক্তিগত সিদ্ধান্তের কাঠামো: কে আর্কিটেকচার মান ঠিক করেন, কে নতুন টুল অনুমোদন করেন, কে ব্যতিক্রম দেন। গভর্ন্যান্সের উদ্দেশ্য নিয়ন্ত্রণ নয় — উদ্দেশ্য হলো সিদ্ধান্তগুলো যেন সচেতনভাবে, লিখিতভাবে, আর সবার জানা থাকে। **Platform Lifecycle** মানে প্ল্যাটফর্মের জীবন: পরীক্ষা (Pilot — একটি দলের সাথে), প্রসার (Rollout — সব দলে), পরিচালনা (Operate — রক্ষণাবেক্ষণ ও উন্নতি), আর অবসান (Retire — পুরোনো অংশ সরানো)। পুরোনো অংশ সরানোও প্ল্যাটফর্মের কাজ — না সরালে প্ল্যাটফর্মটি পুরোনো জিনিসের জাদুঘর হয়ে যায়।

> **WARNING:** প্ল্যাটফর্ম দল যদি শুধু নতুন জিনিস বানায় আর পুরোনো জিনিস কখনো সরায় না, তবে প্ল্যাটফর্মটি দিন দিন ভারী হয় — রক্ষণাবেক্ষণের খরচ বাড়ে, আর নতুনরা বুঝতে পারে না কোনটি বর্তমান পথ, কোনটি পুরোনো। প্রতিটি নতুন ক্ষমতার সাথে একটি পুরোনো ক্ষমতা সরানোর পরিকল্পনাও রাখুন।

২১.৯ক প্ল্যাটফর্মের বিবর্তন: ছোট থেকে এন্টারপ্রাইজ

প্ল্যাটফর্ম একদিনে এন্টারপ্রাইজ হয় না — এটি ধাপে ধাপে বিবর্তিত হয়। প্রথম ধাপে একটি দলের জন্য একটি পাইপলাইন ও একটি পরিবেশের পথ থাকে; দ্বিতীয় ধাপে সেই পথটি কয়েকটি দলে ছড়ায়, আর তখনই সীমানা ও মানের প্রয়োজন দেখা দেয়; তৃতীয় ধাপে প্ল্যাটফর্ম দল আলাদা হয়, IDP তৈরি হয়, আর DX মাপা শুরু হয়; চতুর্থ ধাপে গভর্ন্যান্স, খরচের দৃশ্যমানতা ও নিরাপত্তা নীতি কেন্দ্রীয়ভাবে আসে। প্রতিটি ধাপে আগের ধাপের শিক্ষা কাজে লাগে — আর যে দল দ্বিতীয় ধাপের সমস্যা ছাড়াই চতুর্থ ধাপের কাঠামো বানাতে যায়, তারা একটি ভারী, অব্যবহৃত প্ল্যাটফর্ম বানায়। TechNova-র জন্য নিয়মটি তাই: বর্তমান ধাপের সমস্যাটি সমাধান করুন, পরের ধাপের জন্য দরজা খোলা রাখুন। একটি প্ল্যাটফর্মের পরিপক্কতা মাপার সহজ উপায় হলো জিজ্ঞেস করা — “একটি নতুন দল কি একা, নথি পড়ে, প্ল্যাটফর্মটি ব্যবহার করতে পারে?” উত্তর “হ্যাঁ” হলে প্ল্যাটফর্মটি পরিপক্কতার ভালো জায়গায় আছে; উত্তর “না” হলে পরের উন্নতিটি সেখানেই — নথিতে, নয়তো সহজ পথে।

বিবর্তনের সময় একটি বিপদ হলো **প্ল্যাটফর্মের বিস্তার (Scope Creep)** — প্রতিটি দলের প্রতিটি বিশেষ অনুরোধ প্ল্যাটফর্মে যোগ করা। প্রতিটি যোগ করা ক্ষমতার রক্ষণাবেক্ষণের খরচ আছে, আর প্ল্যাটফর্মটি দিন দিন ভারী হয়। এর প্রতিষেধক হলো একটি পরিষ্কার সিদ্ধান্ত-কাঠামো: একটি নতুন ক্ষমতা যোগ করার আগে জিজ্ঞেস করুন — “কতটি দল এটি ব্যবহার করবে?

এটি কি সাধারণ প্রয়োজন, নাকি একটি দলের বিশেষ প্রয়োজন? এটি না থাকলে দলটি কি নিজে করতে পারে?” শুধু সাধারণ প্রয়োজনই প্ল্যাটফর্মে যায়; বিশেষ প্রয়োজন দলের হাতে থাকে, প্ল্যাটফর্মের Escape Hatch দিয়ে। এই শৃঙ্খলাই প্ল্যাটফর্মকে হালকা ও টেকসই রাখে।

২১.৯ TechNova প্রোডাকশন দৃশ্যকল্প

TechNova-তে পাঁচটি প্রোডাক্ট দল আছে। প্ল্যাটফর্ম দল প্রথমে একটি দলের সাথে Pilot চালাল — সেই দলের জন্য পরিবেশ, পাইপলাইন ও ড্যাশবোর্ডের পথটি বানানো হলো। Pilot-এর শিক্ষা: পরিবেশ পেতে এক ঘণ্টা লাগে, প্রথম ডেপ্লয় এক দিনে হয়, আর দলটির সবচেয়ে বড় অভিযোগ — সিক্রেট পেতে আলাদা অনুরোধ লাগে। প্ল্যাটফর্ম দল সিক্রেটের পথটি Portal-এ যোগ করল, তারপর বাকি দলগুলোতে ধাপে ধাপে প্রসার করল — প্রতিটি দলের জন্য একটি করে সপ্তাহ, তাদের প্রতিক্রিয়া শুনে। ছয় মাস পর পাঁচটি দলই একই প্ল্যাটফর্মে — কিন্তু সীমানা নথিতে লেখা, আর প্রতিটি দলের নিজের SLI ও Runbook আছে।

২১.১০ক প্ল্যাটফর্মের সাফল্য ও ব্যর্থতার লক্ষণ

প্ল্যাটফর্ম সফল হচ্ছে কি না, তা বোঝার লক্ষণগুলো স্পষ্ট। সাফল্যের লক্ষণ: দলগুলো স্বেচ্ছায় প্ল্যাটফর্ম ব্যবহার করে; নতুন প্রকৌশলী দ্রুত প্রথম ডেপ্লয় করেন; প্ল্যাটফর্ম দলের কাছে একই প্রশ্ন বারবার আসে না; আর নিরাপত্তা পর্যালোচনায় প্ল্যাটফর্মের অংশগুলোতে কম সমস্যা পাওয়া যায়। ব্যর্থতার লক্ষণ: দলগুলো প্ল্যাটফর্ম এড়িয়ে নিজের মতো কাজ করে; প্ল্যাটফর্ম দল সব অনুরোধের বোতল-ঘাড় হয়ে যায়; নথি ও বাস্তবতা মেলে না; আর প্রতিটি নতুন দল প্ল্যাটফর্মে যোগ দিতে সপ্তাহ লাগায়। এই লক্ষণগুলো দেখা দিলে সমাধানটি আরও টুল কেনা নয় — সমাধানটি দলগুলোর সাথে বসে শোনা: “তোমাদের কাজে সবচেয়ে বড় বাধাটি কী?” উত্তরটি প্রায়ই একটি ছোট, নির্দিষ্ট সমস্যা — একটি কঠিন ফর্ম, একটি অস্পষ্ট ত্রুটি বার্তা, একটি ধীর ধাপ — আর সেই ছোট সমস্যাটি ঠিক করাই সবচেয়ে বড় উন্নতি। প্ল্যাটফর্মের সাফল্য মাপা হয় দলগুলোর গতিতে, প্ল্যাটফর্মের বৈশিষ্ট্যের সংখ্যায় নয়।

২১.১০ নিরাপত্তা ও খরচ বিবেচনা

শেয়ার্ড প্ল্যাটফর্মে নিরাপত্তার দায় ভাগ করা থাকে: প্ল্যাটফর্ম দল ভিত্তিটি সুরক্ষিত করে (ক্লাস্টার, নেটওয়ার্ক, নীতি), আর দলগুলো নিজের অ্যাপের নিরাপত্তা দেখে (কোড, নির্ভরতা, সিক্রেটের ব্যবহার)। এই ভাগটি পরিষ্কার না থাকলে কেউ ভাবে “প্ল্যাটফর্ম দেখবে”, আরেকজন ভাবে “দল দেখবে” — আর মাঝখানে ফাঁক থেকে যায়। খরচের দিক থেকে প্ল্যাটফর্ম দলের খরচটি সরাসরি দেখা যায়, কিন্তু তার সুবিধাটি ছড়িয়ে থাকে — প্রতিটি দলের বাঁচানো সময়। তাই প্ল্যাটফর্মের মূল্য বোঝাতে DX মেট্রিকগুলো ব্যবহার করুন — Time to First Deploy, Deploy Lead Time — শুধু প্ল্যাটফর্মের বিল নয়।

২১.১১ সাধারণ ভুল

- **প্ল্যাটফর্মকে নিয়মের বোঝা বানানো:** দলগুলোকে সাহায্য না করে শুধু নিষেধ করলে দলগুলো প্ল্যাটফর্ম এড়িয়ে যায়।
- **সীমানা না লেখা:** কে কী করবে লিখিত না থাকলে দায়িত্বের ফাঁক তৈরি হয়।
- **Pilot ছাড়া প্রসার:** একটি দলের সাথে পরীক্ষা না করে সব দলে চালু করলে ভুলটি সব জায়গায় ছড়ায়।
- **নথি হালনাগাদ না করা:** পুরোনো নথি ভুল পথে নিয়ে যায়।
- **পুরোনো জিনিস না সরানো:** প্ল্যাটফর্ম ভারী হয়ে যায়, নতুনরা বিভ্রান্ত হয়।
- **DX না মাপা:** ডেভেলপারদের অভিজ্ঞতা না মাপলে উন্নতি অনুমানে হয়।

২১.১২ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
দলগুলো প্ল্যাটফর্ম ব্যবহার করে না	প্ল্যাটফর্ম কঠিন বা তাদের প্রয়োজন মেটে না	দলগুলোর সাথে কথা বলুন; তাদের যাত্রা পর্যবেক্ষণ করুন; সহজ পথ বানান
প্ল্যাটফর্ম দল বোতল-ঘাড় হয়ে গেছে	সীমানা অস্পষ্ট; প্ল্যাটফর্ম সবকিছু করছে	সীমানা লিখুন; দলগুলোর দায়িত্ব ফিরিয়ে দিন
একটি দলের ভুল সব দলকে ছুঁয়ে যায়	বিচ্ছিন্নতা (Isolation) নেই	নেমস্পেস, কোটা ও নেটওয়ার্ক নীতি দিয়ে বিচ্ছিন্নতা বাড়ান
নতুন প্রকৌশলী প্রথম ডেপ্লয়ে দিন লাগায়	শুরুর নথি বা পথটি কঠিন	Getting Started নথি ও Portal সহজ করুন; নতুনদের পর্যবেক্ষণ করুন
কেন এই সিদ্ধান্ত নেওয়া হয়েছিল জানা যায় না	ADR লেখা হয় না	বড় সিদ্ধান্তে ADR বাধ্যতামূলক করুন

২১.১৩ প্রকৌশল ট্রেড-অফ: কেন্দ্রীকরণ বনাম স্বাধীনতা

কেন্দ্রীকরণের পক্ষে: একরূপতা, নিরাপত্তা, খরচের নিয়ন্ত্রণ, নতুনদের শেখা সহজ। বিপক্ষে: ধীর সিদ্ধান্ত, দলগুলোর বিশেষ প্রয়োজন মেটে না, প্ল্যাটফর্ম দল বোতল-ঘাড় হয়। স্বাধীনতার পক্ষে: গতি, উদ্ভাবন, দলের মালিকানা। বিপক্ষে: বৈচিত্র্য, নিরাপত্তার ফাঁক, নতুনদের শেখার বোঝা। সঠিক ভারসাম্যটি সংস্থার আকারের সাথে বদলায়: ছোট সংস্থায় স্বাধীনতা বেশি চলে; বড় সংস্থায় কেন্দ্রীকরণের মূল্য বাড়ে। TechNova-র মতো মাঝারি জায়গায় নিয়মটি হলো: সাধারণ জিনিস কেন্দ্রীভূত, বিশেষ জিনিস দলের হাতে — আর সীমানাটি নিয়মিত পর্যালোচনা করা।

> **TROUBLESHOOTING:** প্ল্যাটফর্ম সমস্যার সময় প্রথমে জিজ্ঞেস করুন — “দলগুলো আসলে কী করতে চাইছে, আর প্ল্যাটফর্ম তাদের কী করতে বাধা দিচ্ছে?” উত্তরটি প্রায়ই দেখায় সমস্যাটি টুলে নয়, সীমানা বা নথিতে।

২১.১৪ অধ্যায়ের সারাংশ

এন্টারপ্রাইজ প্ল্যাটফর্ম আর্কিটেকচার মানে একটি রেফারেন্স আর্কিটেকচার — ডেভেলপার ইন্টারফেস, প্ল্যাটফর্ম API, অটোমেশন, রানটাইম ও ভিত্তি স্তর — যেখানে নিরাপত্তা ও পর্যবেক্ষণ সব স্তরে থাকে। প্ল্যাটফর্ম সীমানা লিখিতভাবে ঠিক করা হয়, ডেভেলপার অভিজ্ঞতা মাপা হয়, আর্কিটেকচার সিদ্ধান্ত ADR-এ লেখা হয়, আর প্ল্যাটফর্মের জীবনচক্রে Pilot, প্রসার, পরিচালনা ও অবসান — সবই থাকে। সফল প্ল্যাটফর্মের চিহ্ন হলো দলগুলো স্বেচ্ছায় তা ব্যবহার করে।

২১.১৫ জ্ঞান যাচাই

1. প্ল্যাটফর্মকে পণ্য হিসেবে দেখা কেন গুরুত্বপূর্ণ? ডেভেলপাররা গ্রাহক হলে কী বদলায়?
2. রেফারেন্স আর্কিটেকচারের পাঁচটি স্তর ব্যাখ্যা করুন।
3. শেয়ার্ড অবকাঠামোতে Blast Radius কী? বিচ্ছিন্নতা (Isolation) কেন দরকার?
4. প্ল্যাটফর্ম সীমানা কী? সীমানা পরিষ্কার না থাকলে কী সমস্যা হয়?
5. Developer Experience মাপার পাঁচটি উপায় লিখুন।

6. Architecture Standard ও System Integration — দুটি ব্যাখ্যা করুন।
7. ADR কী? একটি ADR-এ কী কী থাকা উচিত?
8. Technical Governance ও Platform Lifecycle ব্যাখ্যা করুন।
9. পুরোনো প্ল্যাটফর্ম ক্ষমতা সরানো কেন জরুরি?
10. কেন্দ্রীকরণ বনাম স্বাধীনতা — ভারসাম্যটি কীভাবে ঠিক করবেন?

২১.১৬ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি প্ল্যাটফর্ম সীমানা-সারণি লিখুন: পাঁচটি বিষয়ে প্ল্যাটফর্ম দল কী দেবে, দল কী করবে — তা লিখুন।
2. একটি ADR লিখুন: TechNova-র একটি বাস্তবসম্মত সিদ্ধান্ত (যেমন প্ল্যাটফর্ম API-র মাধ্যম) নিয়ে প্রসঙ্গ, বিকল্প, সিদ্ধান্ত ও পরিণতি লিখুন।
3. একটি DX মাপের পরিকল্পনা করুন: Time to First Deploy ও Time to Environment কীভাবে মাপবেন, আর কোন মানে উন্নতি হয়েছে বলবেন — তা লিখুন।
4. একটি Pilot পরিকল্পনা লিখুন: একটি নতুন প্ল্যাটফর্ম ক্ষমতা একটি দলের সাথে পরীক্ষা করার ধাপ, সাফল্যের মাপ, আর প্রসারের শর্ত লিখুন।

অধ্যায় ২২ • PART V — Production Architecture & Final Capstone

Chapter 22 — Capacity, Cost, Reliability and DR Engineering

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Capacity Planning** (ধারণক্ষমতা পরিকল্পনা) ও সম্পদ বরাদ্দ (Resource Allocation) করতে; ক্লাউড খরচ অপ্টিমাইজেশন ও **FinOps**-এর চর্চা প্রয়োগ করতে; প্রাপ্যতা (Availability) ও নির্ভরযোগ্যতা (Reliability)-র পার্থক্য ব্যাখ্যা করতে; **RPO ও RTO**, ব্যাকআপ, **Disaster Recovery (DR)**, মাল্টি-অঞ্চল (Multi-Region) পরিকল্পনা ও ব্যর্থতা অনুকরণ (Failure Simulation) TechNova-র প্রেক্ষাপটে নকশা করতে।

- চাহিদা পূর্বাভাস দিয়ে ধারণক্ষমতা পরিকল্পনা করা।
- খরচের চালক (Cost Driver) চিহ্নিত করে অপ্টিমাইজেশনের পরিকল্পনা লেখা।
- RPO/RTO ঠিক করে DR কৌশল বেছে নেওয়া।
- একটি ব্যর্থতা অনুকরণ (Game Day) পরিকল্পনা তৈরি করা।

২২.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার Kubernetes-এ Resource Request ও Limit-এর ধারণা, ক্লাউড বিলের মৌলিক ধারণা, ১৯তম অধ্যায়ের SLI/SLO/Error Budget, এবং ব্যাকআপের প্রাথমিক ধারণা জানা থাকা দরকার। **RPO (Recovery Point Objective)** মানে কতটুকু ডেটা হারানো সহ্য করা যায় — যেমন "সর্বোচ্চ এক ঘণ্টার ডেটা হারানো যাবে"। **RTO (Recovery Time Objective)** মানে বিপর্যয়ের পর কতক্ষণের মধ্যে সেবা ফিরে আসতে হবে — যেমন "চার ঘণ্টার মধ্যে"। **FinOps** মানে ক্লাউড খরচকে প্রকৌশল, অর্থ ও ব্যবসার যৌথ দায়িত্ব হিসেবে পরিচালনা করা — শুধু অর্থ বিভাগের কাজ নয়।

> **CONCEPT:** ধারণক্ষমতা, খরচ ও নির্ভরযোগ্যতা — তিনটি একই সিদ্ধান্তের তিনটি দিক। বেশি সম্পদ মানে বেশি নির্ভরযোগ্যতা, কিন্তু বেশি খরচ; কম সম্পদ মানে কম খরচ, কিন্তু ঝুঁকি। ভালো প্রকৌশল মানে এই তিনটির ভারসাম্য সচেতনভাবে ঠিক করা — অনুমানে নয়, সংখ্যায়।

২২.৩ প্রথম নীতি থেকে: চাহিদা অনিশ্চিত, সম্পদ সীমিত

কোনো সিস্টেমের চাহিদা কখনো পুরোপুরি অনুমান করা যায় না — হঠাৎ একটি প্রচারণা, একটি সংবাদ, বা একটি ভাইরাল পোস্ট চাহিদা দশগুণ বাড়িয়ে দিতে পারে। আবার সম্পদ সীমাহীন নয় — প্রতিটি সার্ভার, প্রতিটি গিগাবাইটের খরচ আছে। ধারণক্ষমতা পরিকল্পনা এই অনিশ্চয়তার মধ্যে একটি সিদ্ধান্ত: স্বাভাবিক চাহিদার জন্য কতটুকু রাখব, হঠাৎ বৃদ্ধির জন্য কতটুকু বাড়তি (Headroom) রাখব, আর বৃদ্ধি হলে কত দ্রুত বাড়াতে পারব। এই তিনটি প্রশ্নের উত্তর না দিয়ে "যত বেশি সম্ভব" রাখা মানে খরচের অপচয়; "যত কম সম্ভব" রাখা মানে বিপর্যয়ের আমন্ত্রণ।

২২.৪ Capacity Planning ও সম্পদ বরাদ্দ

ধারণক্ষমতা পরিকল্পনার ধাপগুলো: (১) **বর্তমান ব্যবহার মাপা** — CPU, মেমরি, ডিস্ক, নেটওয়ার্ক, অনুরোধের হার — সময়ের সাথে কেমন বদলায়; (২) **বৃদ্ধি পূর্বাভাস** — ব্যবসায়িক পরিকল্পনা থেকে চাহিদা কেমন বাড়বে (যেমন নতুন গ্রাহক,

নতুন ফিচার); (৩) **Headroom ঠিক করা** — স্বাভাবিক চাহিদার উপরে কতটুকু বাড়তি রাখা হবে, যাতে হঠাৎ বৃদ্ধি সামলানো যায়; (৪) **স্কেলিং নীতি** — কখন স্বয়ংক্রিয়ভাবে বাড়বে (Horizontal Pod Autoscaler, Cluster Autoscaler), কখন মানুষের সিদ্ধান্ত লাগবে; (৫) **পরীক্ষা** — Load Test দিয়ে দেখা পরিকল্পিত ধারণক্ষমতা আসলেই কাজ করে কি না।

Kubernetes-এ সম্পদ বরাদ্দের মূল হাতিয়ার হলো **Request ও Limit**: Request মানে Pod-টি নিশ্চিতভাবে কতটুকু পাবে (সময়সূচি — Scheduling — এই মান দেখে সিদ্ধান্ত নেয়), Limit মানে Pod-টি সর্বোচ্চ কতটুকু ব্যবহার করতে পারবে। Request খুব কম দিলে Pod-গুলো এক নোডে ঠাসা হয় আর প্রতিযোগিতা হয়; Request খুব বেশি দিলে সম্পদ অপচয় হয়। Limit না দিলে একটি Pod সব সম্পদ খেয়ে ফেলতে পারে। সঠিক মান বের করার উপায় হলো বাস্তব ব্যবহার মাপা — পর্যবেক্ষণ থেকে p95 ব্যবহার দেখে Request ঠিক করা, আর ধীরে ধীরে সমন্বয় করা।

```
# সম্পদ বরাদ্দ — ধারণা (TechNova web)
# Request: বাস্তব p95 ব্যবহারের কাছাকাছি — সময়সূচির ভিত্তি
# Limit:   Request-এর চেয়ে বেশি, কিন্তু সীমাহীন নয় —
#  একটি Pod যেন সব সম্পদ খেয়ে না ফেলে।
# নিয়মিত পর্যালোচনা করুন: ব্যবহারের মেট্রিক দেখে
#   Request/Limit সমন্বয় করুন — একবার ঠিক করে ভুলে যাবেন না।
```

২২.৫ ক্লাউড খরচ অপ্টিমাইজেশন ও FinOps

ক্লাউড খরচের চালকগুলো চিহ্নিত করা প্রথম ধাপ: কম্পিউট (সার্ভার/কনটেইনার), স্টোরেজ, ডেটা স্থানান্তর (Egress — অধ্যায় ১৮), পরিচালিত সেবা, আর ভুলে যাওয়া সম্পদ (অব্যবহৃত ডিস্ক, বন্ধ না করা পরিবেশ)। অপ্টিমাইজেশনের সাধারণ উপায়গুলো: **Right-Sizing** — অতিরিক্ত বড় সম্পদ ছোট করা (ব্যবহার মেপে); **অব্যবহৃত সম্পদ সরানো** — নিয়মিত পর্যালোচনা; **সময়সূচি** — Development/Staging পরিবেশ রাতে ও ছুটির দিনে বন্ধ করা; **সংরক্ষণকাল** — লগ ও ব্যাকআপের সংরক্ষণকাল স্তরভিত্তিক করা; আর **প্রতিশ্রুতি (Commitment)** — দীর্ঘমেয়াদে ব্যবহার নিশ্চিত হলে প্রতিশ্রুতিবদ্ধ মূল্য বিবেচনা করা — তবে শুধু তখনই, যখন ব্যবহারটি সত্যিই স্থিতিশীল।

FinOps-এর মূল নীতিগুলো: খরচের দৃশ্যমানতা (কে, কীসের জন্য, কত খরচ করছে — ট্যাগ দিয়ে); দায়িত্ব (প্রতিটি খরচের একজন মালিক); আর সিদ্ধান্ত (খরচের তথ্য দিয়ে প্রকৌশল সিদ্ধান্ত নেওয়া)। FinOps মানে খরচ কমানো নয় — মানে খরচের বিনিময়ে কী মূল্য পাওয়া যাচ্ছে, তা বোঝা। একটি ব্যয়বহুল সেবা যদি ব্যবসায়িক মূল্য দেয়, তবে তা যুক্তিযুক্ত; একটি সস্তা সেবা যদি কেউ ব্যবহার না করে, তবে তাও অপচয়।

> **REAL WORLD:** TechNova-তে Staging পরিবেশগুলো সপ্তাহে সাত দিন, ২৪ ঘণ্টা চলত — যদিও কেউ রাতে তা ব্যবহার করত না। একটি সময়সূচি দিয়ে রাতে ও ছুটির দিনে Staging বন্ধ করার পর খরচটি উল্লেখযোগ্যভাবে কমেছিল — আর কেউ কোনো অসুবিধা অনুভব করেনি। সবচেয়ে সহজ অপ্টিমাইজেশনগুলো প্রায়ই সবচেয়ে বড় হয়: যা ব্যবহার হয় না, তা বন্ধ করা।

২২.৬ প্রাপ্যতা ও নির্ভরযোগ্যতা: সংখ্যার পেছনের অর্থ

Availability মানে সেবাটি যখন দরকার, তখন তা পাওয়া যায় কি না — সাধারণত শতাংশে মাপা হয়। **Reliability** মানে সেবাটি সঠিকভাবে কাজ করে কি না — শুধু চালু থাকা নয়, সঠিক উত্তর দেওয়া। একটি সেবা চালু থাকতে পারে কিন্তু ভুল উত্তর দিতে পারে — তখন তা Available, কিন্তু Reliable নয়। ১৯তম অধ্যায়ের SLO হিসাবটি এখানে আবার মনে রাখুন: ৩০ দিনের মাসে ৯৯.৯৯% প্রাপ্যতা মানে ৪ মিনিট ১৯ সেকেন্ড Error Budget (৪ মিনিট ২৩ সেকেন্ড নয় — সেটি ~৩০.৪৪ দিনের গড় মাসের হিসাব)। উচ্চ প্রাপ্যতা পেতে যা লাগে: একাধিক অঞ্চল বা জোনে চলা (একটি জোন ভাঙলে অন্যটি চালু থাকে),

স্বাস্থ্য পরীক্ষা ও স্বয়ংক্রিয় প্রতিস্থাপন, ডেটাবেসের প্রতিলিপি (Replication), আর নিয়মিত ব্যর্থতা পরীক্ষা। কিন্তু প্রতিটি অতিরিক্ত "৯"-এর খরচ অনেক বাড়ে — তাই প্রাপ্যতার লক্ষ্যটি ব্যবসায়িক প্রয়োজন দিয়ে ঠিক করুন, প্রতিযোগিতা দিয়ে নয়।

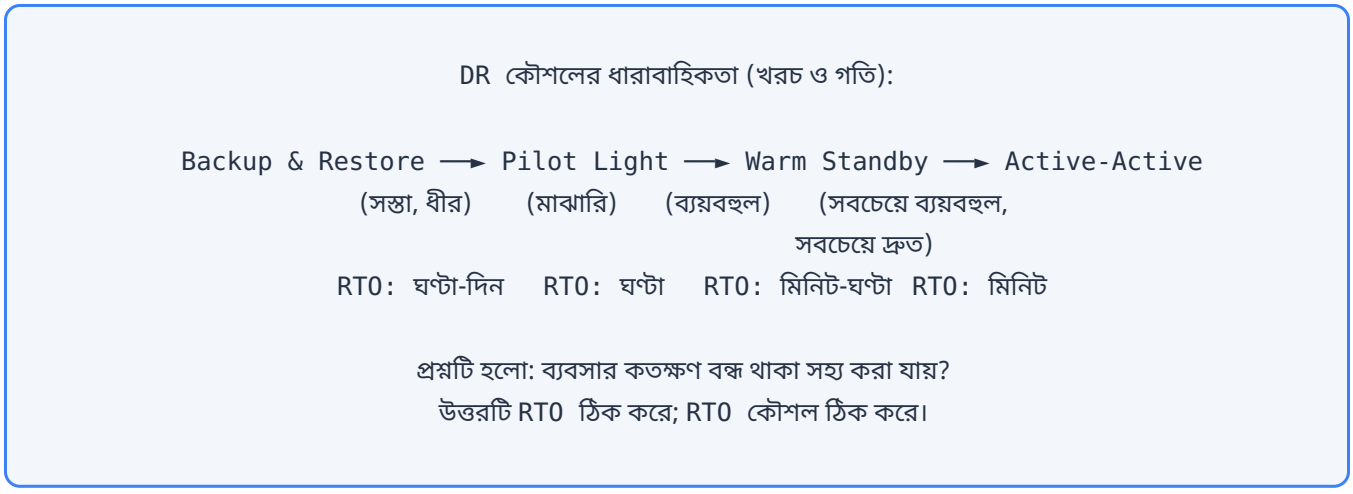
প্রাপ্যতার লক্ষ্য (৩০ দিন)	Error Budget	সাধারণত যা লাগে
৯৯.৯%	৪৩ মিনিট ১২ সেকেন্ড	একাধিক জোন, স্বয়ংক্রিয় পুনরুদ্ধার
৯৯.৯৫%	২১ মিনিট ৩৬ সেকেন্ড	উপরের সব + শক্তিশালী পর্যবেক্ষণ
৯৯.৯৯%	৪ মিনিট ১৯ সেকেন্ড	মাল্টি-অঞ্চল, নিয়মিত DR পরীক্ষা, কঠোর পরিবর্তন নিয়ন্ত্রণ

২২.৭ RPO, RTO, ব্যাকআপ ও Disaster Recovery

ব্যাকআপ ও DR এক নয়। **Backup** মানে ডেটার অনুলিপি রাখা — যাতে ভুল করে মুছে গেলে বা নষ্ট হলে ফেরানো যায়। **Disaster Recovery** মানে পুরো সেবাটি — অ্যাপ, ডেটা, নেটওয়ার্ক, DNS — একটি বড় বিপর্যয়ের পর (যেমন একটি অঞ্চলের বিভ্রাট) আবার চালু করা। RPO বলে কতটুকু ডেটা হারানো যাবে (ব্যাকআপ কত ঘন ঘন নিতে হবে), RTO বলে কতক্ষণের মধ্যে ফিরতে হবে (পুনরুদ্ধারের প্রক্রিয়া কত দ্রুত হতে হবে)। RPO ও RTO যত ছোট, DR তত ব্যয়বহুল — কারণ তার জন্য ক্রমাগত প্রতিলিপি ও প্রস্তুত পরিবেশ লাগে।

২২.৭.১ DR কৌশলের ধাপগুলো

DR কৌশলগুলো একটি ধারাবাহিকতায় (Spectrum) থাকে: **Backup & Restore** — বিপর্যয়ের পর ব্যাকআপ থেকে নতুন করে বানানো (সস্তা, কিন্তু ধীর — RTO বড়); **Pilot Light** — ছোট আকারে একটি প্রস্তুত পরিবেশ রাখা, বিপর্যয়ের সময় বড় করা (মাঝারি খরচ ও গতি); **Warm Standby** — একটি ছোট কিন্তু চলমান পরিবেশ রাখা, প্রয়োজনে বাড়ানো (দ্রুত, কিন্তু খরচ বেশি); **Multi-Region Active-Active** — দুটি অঞ্চলে একসাথে চলা (সবচেয়ে দ্রুত, সবচেয়ে ব্যয়বহুল)। TechNova-র মতো দলের জন্য সাধারণত Backup & Restore দিয়ে শুরু করা, তারপর ব্যবসায়িক প্রয়োজনে Pilot Light বা Warm Standby-তে যাওয়া — এই ক্রমটি বাস্তবসম্মত। কোন কৌশলই বেছে নিন, তা নিয়মিত পরীক্ষা না করলে তা শুধু নথি — DR পরিকল্পনা পরীক্ষা ছাড়া বিশ্বাস করবেন না।



ব্যাকআপের জন্য তিনটি নিয়ম: **3-2-1 নিয়ম** — ডেটার তিনটি অনুলিপি, দুটি ভিন্ন মাধ্যমে, একটি ভিন্ন জায়গায়; **পরীক্ষিত পুনরুদ্ধার** — ব্যাকআপ থেকে আসলেই ফেরানো যায় কি না, তা নিয়মিত পরীক্ষা করা (পরীক্ষা না করা ব্যাকআপ শুধু আশা); আর **এনক্রিপশন** — ব্যাকআপ এনক্রিপ্টেড থাকবে, আর তার চাবি আলাদাভাবে সুরক্ষিত থাকবে।

২২.৮ মাল্টি-অঞ্চল পরিকল্পনা ও ব্যর্থতা অনুকরণ

Multi-Region পরিকল্পনায় বিবেচনা করতে হয়: ডেটা কীভাবে অঞ্চলগুলোর মধ্যে প্রতিলিপি হবে (Synchronous প্রতিলিপি দ্রুত কিন্তু ধীর; Asynchronous প্রতিলিপি দ্রুত কিন্তু কিছু ডেটা হারাতে পারে — RPO-র সাথে মিলিয়ে ঠিক করুন), ট্রাফিক কীভাবে অঞ্চলগুলোর মধ্যে যাবে (DNS বা ট্রাফিক ব্যবস্থাপনা দিয়ে), আর একটি অঞ্চল ভাঙলে অন্যটি কীভাবে দায়িত্ব নেবে (Failover — স্বয়ংক্রিয় নাকি মানুষের সিদ্ধান্তে)। Failover স্বয়ংক্রিয় করলে গতি বাড়ে, কিন্তু ভুল Failover-এর ঝুঁকি থাকে — যেমন নেটওয়ার্ক বিভাজনে দুটি অঞ্চলই নিজেকে মূল ভাবা (Split-Brain)।

Failure Simulation বা **Game Day** মানে পরিকল্পিতভাবে ব্যর্থতা ঘটিয়ে দেখা সিস্টেমটি কীভাবে সাড়া দেয়: একটি Pod মুছে ফেলা, একটি জোন বন্ধ করা, ডেটাবেসের সংযোগ ছিন্ন করা — নিয়ন্ত্রিতভাবে, পর্যবেক্ষণসহ। Game Day-এর উদ্দেশ্য সিস্টেম ভাঙা নয় — উদ্দেশ্য হলো ভাঙার আগে দুর্বলতা খুঁজে পাওয়া। প্রতিটি Game Day-এর একটি পরিকল্পনা, একটি পর্যবেক্ষক, একটি থামার নিয়ম (Abort Condition), আর পরে একটি পর্যালোচনা থাকা উচিত।

```
# Game Day — পরিকল্পনার কাঠামো (TechNova)
# দৃশ্য: একটি অ্যাভেইলেবিলিটি জোনের বিভ্রাট
# অনুমান: অন্য জোনের Pod-গুলো ট্রাফিক নেবে; RTO ১ ঘণ্টার মধ্যে।
# থামার নিয়ম: ব্যবহারকারীর ত্রুটির হার একটি সীমা ছাড়ালে থামুন।
# পর্যবেক্ষণ: ত্রুটির হার, লেটেন্সি, Failover-এর সময়।
# পর্যালোচনা: কী শেখা হলো, কোন Action Item তৈরি হলো।
```

২২.৯ক মাল্টি-অঞ্চল খরচ ও ধারণক্ষমতার ভারসাম্য

মাল্টি-অঞ্চল পরিকল্পনায় খরচের হিসাবটি প্রায়ই ভুলে যাওয়া হয়: দ্বিতীয় অঞ্চলে একটি প্রস্তুত পরিবেশ রাখা মানে দ্বিগুণ খরচ নয় — Pilot Light-এ দ্বিতীয় অঞ্চলটি ছোট থাকে, Warm Standby-তে মাঝারি, Active-Active-এ প্রায় দ্বিগুণ। কিন্তু খরচ শুধু কম্পিউটের নয় — অঞ্চলগুলোর মধ্যে ডেটা প্রতিলিপির খরচ, Egress খরচ (অধ্যায় ১৮), আর দুটি অঞ্চল পরিচালনার দক্ষতার খরচও আছে। তাই মাল্টি-অঞ্চলের সিদ্ধান্তটি RTO/RPO-র সাথে খরচের সারণি দিয়ে নেওয়া উচিত: প্রতিটি কৌশলের RTO, RPO ও আনুমানিক খরচের ধরন (কম/মাঝারি/বেশি) পাশাপাশি লিখুন, আর ব্যবসার সাথে সিদ্ধান্ত নিন। একটি সাধারণ ভুল হলো সব সিস্টেমকে একই অঞ্চল-কৌশলে ফেলা — মূল পেমেন্ট সিস্টেম Warm Standby-তে, অভ্যন্তরীণ রিপোর্টিং Backup & Restore-এ — এই ভাগটি খরচ কমায়, অথচ গুরুত্বপূর্ণ সিস্টেমের সুরক্ষা কমে না। ধারণক্ষমতার দিক থেকেও মাল্টি-অঞ্চল মানে প্রতিটি অঞ্চলে Headroom রাখা — একটি অঞ্চল ভাঙলে অন্য অঞ্চলটি হঠাৎ দ্বিগুণ চাহিদা নেবে, আর সেই চাহিদা সামলানোর ধারণক্ষমতা আগে থেকে না থাকলে Failover-ই ব্যর্থ হয়।

২২.৯ TechNova প্রোডাকশন দৃশ্যকল্প

TechNova-র web অ্যাপের RPO ঠিক করা হলো এক ঘণ্টা (সর্বোচ্চ এক ঘণ্টার ডেটা হারানো যাবে) ও RTO চার ঘণ্টা (চার ঘণ্টার মধ্যে সেবা ফিরতে হবে)। এর অর্থ: ডেটাবেস ব্যাকআপ প্রতি ঘণ্টায়, ব্যাকআপ অন্য অঞ্চলে সংরক্ষিত, আর পুনরুদ্ধারের Runbook লেখা ও পরীক্ষিত। একটি Game Day-তে দেখা গেল পুনরুদ্ধারে আসলে ছয় ঘণ্টা লাগছে — Runbook-এর একটি ধাপে হাতে করা কাজ ছিল, আর ব্যাকআপ ডাউনলোডে সময় লাগছিল। দলটি Runbook স্বয়ংক্রিয় করল ও ব্যাকআপের জায়গা বদলাল — পরের পরীক্ষায় সময় নেমে এলো তিন ঘণ্টায়। এই গল্পটি দেখায় DR পরিকল্পনার মূল্য পরীক্ষায় — নথিতে নয়।

২২.১০ক খরচের সংস্কৃতি: দায়িত্ব ছড়িয়ে দেওয়া

FinOps-এর সবচেয়ে কঠিন অংশটি টুল নয় — সংস্কৃতি। খরচের দায়িত্ব যখন শুধু অর্থ বিভাগের থাকে, তখন প্রকৌশলীরা খরচ না ভেবে সম্পদ বানান — কারণ বিলটি তাঁরা দেখেন না। দায়িত্ব ছড়িয়ে দেওয়ার উপায়: প্রতিটি দলকে তাদের খরচের ড্যাশবোর্ড দেখান — মাসিক, ট্যাগ-ভিত্তিক; খরচের পরিবর্তন Pull Request-এর মতো পর্যালোচনা করুন — একটি বড় সম্পদ যোগ করার সময় তার খরচের প্রভাব লিখুন; আর খরচের উন্নতিকে স্বীকৃতি দিন — যে দল অব্যবহৃত সম্পদ সরিয়ে খরচ কমাল, তা ভাগ করে নিন। একটি দরকারি অভ্যাস হলো প্রতিটি নতুন সার্ভিসের জন্য একটি খরচের পূর্বাভাস লেখা — “এই সার্ভিসটি মাসে আনুমানিক কত খরচ করবে, আর কেন?” — তারপর বাস্তব খরচের সাথে তুলনা করা। পূর্বাভাস ভুল হলে তা শেখার সুযোগ — দোষের নয়। খরচের সংস্কৃতি মানে খরচকে ভয় পাওয়া নয় — মানে খরচের বিনিময়ে কী মূল্য পাওয়া যাচ্ছে, তা সবাই জানে।

২২.১০ নিরাপত্তা ও খরচ বিবেচনা

ব্যাকআপ ও DR-এ নিরাপত্তা অপরিহার্য: ব্যাকআপে পুরো ডেটাবেসের অনুলিপি থাকে — তা ফাঁস হলে মূল ডেটাবেস ফাঁস হওয়ার সমান। তাই ব্যাকআপ এনক্রিপ্টেড, প্রবেশাধিকার নিয়ন্ত্রিত, আর সংরক্ষণকাল ঠিক করা উচিত। খরচের দিক থেকে DR একটি বীমা — এর মূল্য তখনই বোঝা যায় যখন বিপর্যয় ঘটে। তাই DR-এর খরচটি ব্যবসায়িক ঝুঁকির সাথে মিলিয়ে ঠিক করুন: RTO যত ছোট চাইবেন, খরচ তত বেশি হবে। আর মনে রাখবেন: সব সিস্টেমের একই RTO দরকার হয় না — মূল পেমেন্ট সিস্টেমের RTO ছোট, অভ্যন্তরীণ রিপোর্টিং টুলের RTO বড় হতে পারে। প্রতিটি সিস্টেমের RPO/RTO আলাদাভাবে ঠিক করুন।

২২.১১ সাধারণ ভুল

- **পরীক্ষা না করা ব্যাকআপ:** ব্যাকআপ আছে ভাবা, কিন্তু পুনরুদ্ধার কখনো পরীক্ষা না করা।
- **সব সিস্টেমের একই RTO:** সবকিছুকে সমান গুরুত্ব দেওয়া মানে কোনোটিকেই সঠিকভাবে সুরক্ষা না দেওয়া।
- **Request/Limit অনুমানে ঠিক করা:** ব্যবহার না মেপে সম্পদ বরাদ্দ করা — অপচয় বা প্রতিযোগিতা।
- **অব্যবহৃত সম্পদ ভুলে যাওয়া:** বন্ধ না করা পরিবেশ ও ডিস্ক খরচ বাড়ায়।
- **DR নথি, পরীক্ষা নেই:** DR পরিকল্পনা লিখে তাকে বিশ্বাস করা — পরীক্ষা ছাড়া।
- **খরচ শুধু অর্থ বিভাগের কাজ ভাবা:** খরচের সিদ্ধান্ত প্রকৌশলীরাই নেন — ট্যাগ ও দৃশ্যমানতা ছাড়া তা নিয়ন্ত্রণ করা যায় না।

২২.১২ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
খরচ হঠাৎ বেড়ে গেল	অব্যবহৃত সম্পদ, বড় Egress, বা ভুল আকার	খরচের চালক দেখুন (ট্যাগ দিয়ে); অব্যবহৃত সম্পদ সরান
Pod বারবার Evict হচ্ছে	Request/Limit ভুল; নোডে সম্পদ কম	ব্যবহার মেপে Request ঠিক করুন; নোডের ধারণক্ষমতা দেখুন
পুনরুদ্ধারে RTO ছাড়িয়ে যায়	Runbook-এ হাতে করা ধাপ; ব্যাকআপ ধীর	Runbook স্বয়ংক্রিয় করুন; পুনরুদ্ধার পরীক্ষা করুন
Failover-এ ডেটা হারাল		RPO পুনর্বিবেচনা করুন; প্রতিলিপির বিলম্ব মাপুন

	Asynchronous প্রতিলিপি; RPO বোঝা হয়নি	
Load Test-এ সিস্টেম ভেঙে যায়	ধারণক্ষমতা পরিকল্পনা অনুমানে করা	Load Test-এর ফল দিয়ে ধারণক্ষমতা ঠিক করুন; স্কেলিং নীতি পরীক্ষা করুন

২২.১৩ প্রকৌশল ট্রেড-অফ: কতটা প্রস্তুতি

প্রস্তুতি একটি বীমা — বেশি প্রস্তুতি মানে বেশি খরচ, কম প্রস্তুতি মানে বেশি ঝুঁকি। সঠিক জায়গাটি ব্যবসায়িক প্রভাব দিয়ে ঠিক হয়: এক ঘণ্টা বন্ধ থাকলে কত ক্ষতি হয় — টাকা, বিশ্বাস, আইনি ঝুঁকি? সেই ক্ষতির সাথে DR-এর খরচ তুলনা করুন। আর একটি ট্রেড-অফ: Active-Active মাল্টি-অঞ্চল বনাম Pilot Light। Active-Active সবচেয়ে নিরাপদ, কিন্তু সবচেয়ে ব্যয়বহুল ও জটিল; Pilot Light সস্তা, কিন্তু Failover-এ সময় লাগে। TechNova-র মতো দলের জন্য নিয়মটি হলো: সবচেয়ে গুরুত্বপূর্ণ সিস্টেমে শক্ত DR, বাকিগুলোতে সহজ DR — আর সব DR-ই নিয়মিত পরীক্ষিত।

> **TROUBLESHOOTING:** খরচ বা ধারণক্ষমতার সমস্যায় প্রথমে জিজ্ঞেস করুন — “এই সম্পদটি কে ব্যবহার করছে, আর কেন?” ট্যাগ ছাড়া খরচের উত্তর দেওয়া যায় না — আর ট্যাগ ছাড়া অপ্টিমাইজেশন অনুমানে হয়।

২২.১৪ অধ্যায়ের সারাংশ

ধারণক্ষমতা পরিকল্পনা মানে চাহিদা মাপা, বৃদ্ধি পূর্বাভাস করা, Headroom রাখা ও স্কেলিং পরীক্ষা করা। FinOps মানে খরচের দৃশ্যমানতা, দায়িত্ব ও মূল্য-ভিত্তিক সিদ্ধান্ত। প্রাপ্যতা ও নির্ভরযোগ্যতা এক নয় — আর ৩০ দিনের মাসে ৯৯.৯৯% মানে ৪ মিনিট ১৯ সেকেন্ড। RPO বলে কতটুকু ডেটা হারানো যাবে, RTO বলে কতক্ষণে ফিরতে হবে — আর DR কৌশলটি সেই দুটি সংখ্যা থেকে আসে। ব্যাকআপ, মাল্টি-অঞ্চল পরিকল্পনা ও Game Day — তিনটিই পরীক্ষা ছাড়া শুধু নথি।

২২.১৫ জ্ঞান যাচাই

1. Capacity Planning-এর পাঁচটি ধাপ লিখুন। Headroom কী?
2. Kubernetes-এ Request ও Limit-এর পার্থক্য ও ভুল মানের পরিণতি ব্যাখ্যা করুন।
3. FinOps কী? খরচ কমানো থেকে এর পার্থক্য কী?
4. খরচের পাঁচটি চালক লিখুন ও প্রতিটির অপ্টিমাইজেশনের উপায় বলুন।
5. Availability ও Reliability-র পার্থক্য ব্যাখ্যা করুন।
6. ৩০ দিনের মাসে ৯৯.৯৯% প্রাপ্যতার Error Budget কত? ৪ মিনিট ২৩ সেকেন্ড কেন ভিন্ন হিসাব?
7. RPO ও RTO — সংজ্ঞা দিন ও উদাহরণ দিয়ে বোঝান।
8. DR কৌশলের চারটি ধাপ (Backup & Restore থেকে Active-Active) তুলনা করুন।
9. 3-2-1 ব্যাকআপ নিয়ম ব্যাখ্যা করুন। পরীক্ষিত পুনরুদ্ধার কেন জরুরি?
10. Game Day কী? একটি Game Day-এর পরিকল্পনায় কী কী থাকা উচিত?

২২.১৬ ব্যবহারিক অনুশীলন

1. TechNova-র web অ্যাপের জন্য একটি ধারণক্ষমতা পরিকল্পনা লিখুন: বর্তমান ব্যবহার, বৃদ্ধির পূর্বাভাস, Headroom, আর স্কেলিং নীতি — একটি সারণিতে।

2. একটি FinOps পর্যালোচনা করুন (কাল্পনিক বিল দিয়ে): খরচের চালকগুলো চিহ্নিত করুন, প্রতিটির মালিক ঠিক করুন, আর তিনটি অপ্টিমাইজেশন প্রস্তাব লিখুন।
3. TechNova-র তিনটি সিস্টেমের (পেমেন্ট, web অ্যাপ, রিপোর্টিং) RPO/RTO ঠিক করুন ও যুক্তি দিন কেন প্রতিটি আলাদা। তারপর প্রতিটির জন্য DR কৌশল বেছে নিন।
4. একটি Game Day পরিকল্পনা লিখুন: দৃশ্য, অনুমান, থামার নিয়ম, পর্যবেক্ষণের মেট্রিক, আর পর্যালোচনার কাঠামো — সম্পূর্ণ নথি তৈরি করুন।

অধ্যায় ২৩ • PART V — Production Architecture & Final Capstone

Chapter 23 — Engineering Leadership and Operational Ownership

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — প্রযুক্তিগত সিদ্ধান্ত নেওয়ার কাঠামো ব্যাখ্যা করতে; প্রকৌশল নথি (Documentation), Runbook, অন-কল (On-Call) পরিচালনা, Incident Command, পরিবর্তন ব্যবস্থাপনা (Change Management) ও প্রোডাকশন প্রস্তুতি (Production Readiness) প্রয়োগ করতে; জ্ঞান ভাগাভাগি (Knowledge Sharing), প্রযুক্তিগত ঋণ (Technical Debt), প্ল্যাটফর্ম পরিপক্কতা (Maturity) ও প্রকৌশল যোগাযোগ TechNova-র প্রেক্ষাপটে পরিচালনা করতে।

- একটি প্রযুক্তিগত সিদ্ধান্তের জন্য বিকল্প, মানদণ্ড ও ঝুঁকি লিখে সিদ্ধান্ত নেওয়া।
- একটি সম্পূর্ণ Runbook ও Production Readiness যাচাই-তালিকা তৈরি করা।
- টেকসই অন-কল রোটেশন ও Incident Command-এর নিয়ম লেখা।
- প্রযুক্তিগত ঋণের তালিকা করে অগ্রাধিকার ঠিক করা।

২৩.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়ে আপনার আগের অধ্যায়গুলোর প্রযুক্তিগত ধারণাগুলো — CI/CD, পর্যবেক্ষণ, SRE, Incident Response (অধ্যায় ১৯), নিরাপত্তা (অধ্যায় ২০), প্ল্যাটফর্ম আর্কিটেকচার (অধ্যায় ২১), DR (অধ্যায় ২২) — জানা থাকা দরকার। এখানে প্রশ্নটি প্রযুক্তির নয়, মানুষের ও প্রক্রিয়ার: সিদ্ধান্ত কীভাবে নেওয়া হয়, জ্ঞান কীভাবে ছড়ায়, আর প্রোডাকশনের দায়িত্ব কে নেয়। **On-Call** মানে নির্ধারিত সময়ে প্রোডাকশন সমস্যার সাড়া দেওয়ার দায়িত্ব। **Runbook** মানে একটি পরিচিত সমস্যার ধাপে ধাপে সমাধান-নির্দেশনা। **Technical Debt** মানে দ্রুত কাজ করার জন্য নেওয়া ছাড় — যা পরে সুদসহ শোধ করতে হয়।

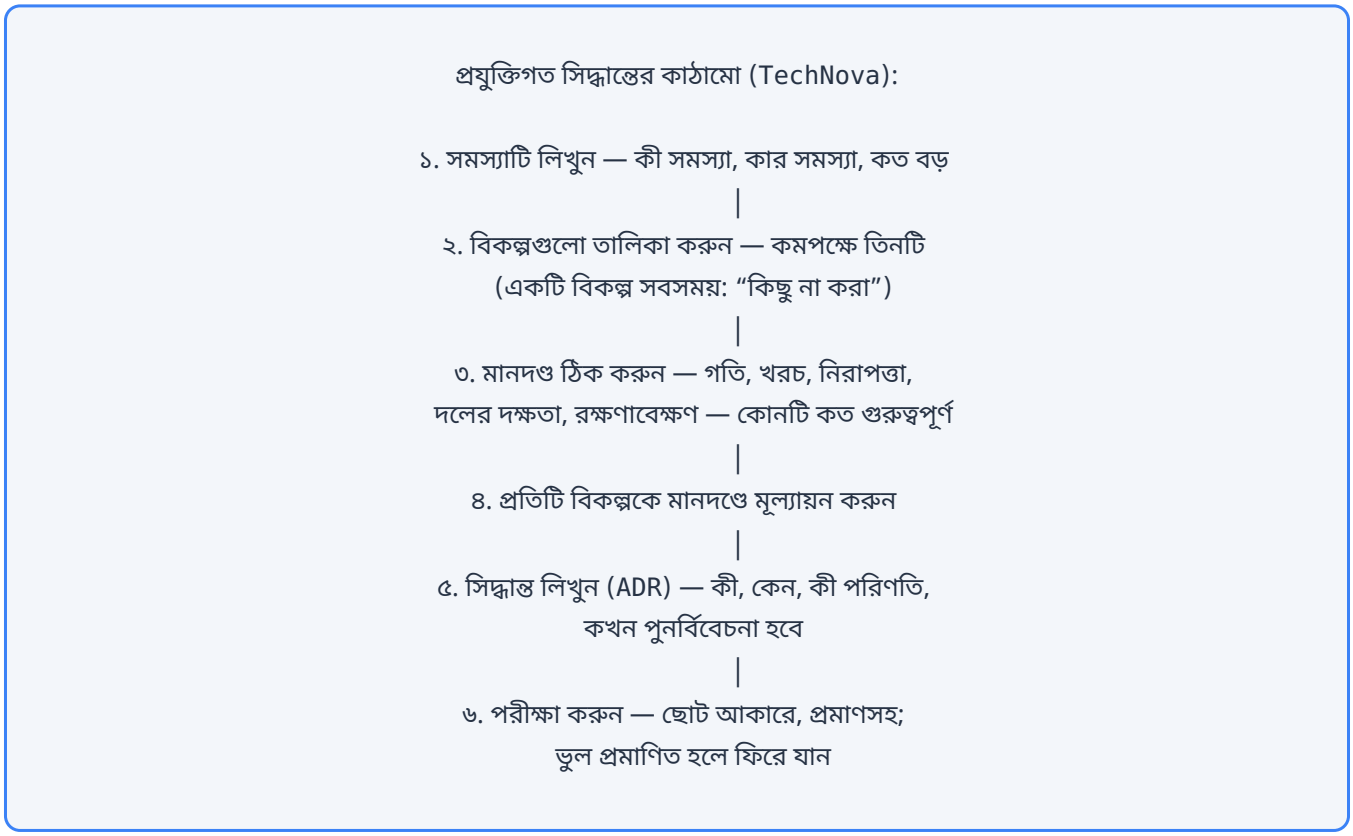
> **CONCEPT:** প্রকৌশল নেতৃত্ব মানে সবচেয়ে বেশি কোড লেখা নয় — মানে সঠিক সিদ্ধান্ত নেওয়া, সেই সিদ্ধান্ত লিখে রাখা, আর দলকে এমনভাবে সাজানো যাতে সঠিক কাজটি সহজ কাজ হয়। একজন ভালো নেতা নিজে সব সমস্যা সমাধান করেন না; তিনি এমন ব্যবস্থা বানান যাতে সমস্যাগুলো কম ঘটে, আর ঘটলে দলটি নিজেই সামলাতে পারে।

২৩.৩ প্রথম নীতি থেকে: সিদ্ধান্তের গুণগত মান

প্রযুক্তিগত সিদ্ধান্তগুলো প্রায়ই অনুভূতি বা অভ্যাস দিয়ে নেওয়া হয় — “আমরা সবসময় এভাবেই করি”, “এই টুলটি জনপ্রিয়”, “আমার আগের কোম্পানিতে এটাই ছিল”। কিন্তু ভালো সিদ্ধান্তের একটি কাঠামো থাকে: সমস্যাটি কী (প্রসঙ্গ), কোন বিকল্পগুলো আছে, প্রতিটি বিকল্পের সুবিধা-অসুবিধা কী, সিদ্ধান্তের মানদণ্ড কী (গতি, খরচ, নিরাপত্তা, দলের দক্ষতা), আর সিদ্ধান্তটি ভুল প্রমাণিত হলে কীভাবে বোঝা যাবে ও ফিরে যাওয়া যাবে। এই কাঠামোটি লিখিত হলে সিদ্ধান্তটি পর্যালোচনা করা যায়, ভবিষ্যতে বোঝা যায়, আর নতুনরা শিখতে পারে। ADR (অধ্যায় ২১) এই কাঠামোর লিখিত রূপ।

দ্বিতীয় নীতিটি হলো — সিদ্ধান্তের গতি ও সিদ্ধান্তের প্রত্যাবর্তনযোগ্যতা (Reversibility) একসাথে ভাবুন। যে সিদ্ধান্ত সহজে ফেরানো যায় (যেমন একটি ফিচার ফ্ল্যাগ চালু করা), তা দ্রুত নেওয়া যায়; যে সিদ্ধান্ত ফেরানো কঠিন (যেমন ডেটাবেস বদলানো), তা ধীরে, প্রমাণসহ নিতে হয়। সব সিদ্ধান্তকে একই গুরুত্ব দিলে দলটি ধীর হয়ে যায়; সব সিদ্ধান্তকে হালকাভাবে নিলে বড় ভুল হয়।

২৩.৪ প্রযুক্তিগত সিদ্ধান্ত নেওয়া: একটি কাঠামো



এই কাঠামোর একটি গুরুত্বপূর্ণ অংশ হলো "কিছু না করা" বিকল্পটি। অনেক সমস্যার সঠিক উত্তর হলো এখন কিছু না করা — কারণ সমস্যাটি ছোট, বা তথ্য অপর্যাপ্ত, বা সমাধানের খরচ সমস্যার চেয়ে বেশি। এই বিকল্পটি তালিকায় না থাকলে দলটি প্রতিটি সমস্যায় কিছু একটা করতে বাধ্য বোধ করে — আর অপ্রয়োজনীয় জটিলতা তৈরি হয়। আরেকটি গুরুত্বপূর্ণ অভ্যাস: সিদ্ধান্তের পুনর্বিবেচনার সময় ঠিক করা — "তিন মাস পর এই সিদ্ধান্তটি আবার দেখব" — কারণ প্রসঙ্গ বদলায়, আর পুরোনো সিদ্ধান্ত নতুন প্রসঙ্গে ভুল হতে পারে।

২৩.৫ প্রকৌশল নথি (Documentation) ও Runbook

নথি তিন ধরনের: **শুরুর নথি** (নতুনরা কীভাবে শুরু করবেন), **ধারণাগত নথি** (সিস্টেম কীভাবে কাজ করে — আর্কিটেকচার, সিদ্ধান্ত), আর **পরিচালনার নথি** (সমস্যা হলে কী করতে হবে — Runbook)। Runbook-এর জন্য নিয়ম: প্রতিটি Runbook-এ থাকবে লক্ষণ (কী দেখে বুঝবেন এই সমস্যা), প্রভাব (ব্যবহারকারীর উপর কী প্রভাব), ধাপ (কী করতে হবে, কোন ক্রমে), যাচাই (ঠিক হয়েছে কি না, কীভাবে বুঝবেন), আর ফিরে যাওয়ার পথ (ভুল হলে কী করবেন)। Runbook লেখা থাকলেই হয় না — তা পরীক্ষিত হতে হয়: একজন নতুন প্রকৌশলী শুধু Runbook পড়ে সমস্যাটি সামলাতে পারেন কি না, তা দেখুন। না পারলে Runbook-টি অসম্পূর্ণ।

```
# Runbook — কাঠামো (TechNova: ডেটাবেস সংযোগ শেষ)
# লক্ষণ: অ্যাপ লগে "connection pool exhausted"; ত্রুটির হার বাড়ছে।
# প্রভাব: নতুন অনুরোধ ব্যর্থ হচ্ছে; ব্যবহারকারী লগইন করতে পারছেন না।
# ধাপ: ১. ড্যাশবোর্ডে সংযোগের সংখ্যা দেখুন।
#      ২. দীর্ঘস্থায়ী প্রশ্ন (Long-running Query) আছে কি না দেখুন।
#      ৩. প্রয়োজনে সংযোগের সীমা সাময়িকভাবে বাড়ান (কীভাবে — কমান্ডসহ)।
#      ৪. মূল কারণ (ধীর প্রশ্ন) ঠিক করুন।
```

```
# যাচাই: ত্রুটির হার স্বাভাবিক কি না, সংযোগের সংখ্যা কমছে কি না।
# ফিরে যাওয়ার পথ: সীমা আগের মানে ফেরান; Incident ঘোষণা করুন।
```

নথির একটি সাধারণ সমস্যা হলো নথি লেখা হয়, কিন্তু হালনাগাদ করা হয় না। সমাধান: নথির মালিক ঠিক করা, আর পরিবর্তনের সাথে নথি বদলানোর নিয়ম — যেমন একটি সিস্টেম বদলানোর Pull Request-এ তার নথিও বদলাতে হয়। আর নথি যেখানে দরকার, সেখানেই রাখুন — Runbook সতর্কতার সাথে লিংক করা থাকলে Incident-এর সময় তা খুঁজতে হয় না।

২৩.৬ অন-কল (On-Call) পরিচালনা: টেকসই দায়িত্ব

অন-কল মানে প্রোডাকশনের দায়িত্ব — আর এই দায়িত্ব টেকসই না হলে মানুষ পুড়ে যায় (Burnout)। টেকসই অন-কলের নিয়মগুলো: **রোটেশন** — দায়িত্ব ঘুরে ঘুরে আসে, একজনের উপর চাপে না; **ক্ষতিপূরণ** — অন-কল সময়ের জন্য বিশ্রাম বা স্বীকৃতি থাকে; **সীমা** — একজন মানুষ একটানা কতদিন অন-কল থাকবেন, তার সীমা থাকে; **প্রস্তুতি** — অন-কলে যাওয়ার আগে প্রশিক্ষণ ও ছায়া-দায়িত্ব (Shadowing — অভিজ্ঞের সাথে থেকে শেখা) থাকে; **সতর্কতার গুণগত মান** — শুধু কাজের সতর্কতা আসে (অধ্যায় ১৯) — নইলে মানুষ রাতে বারবার জাগে; আর **পর্যালোচনা** — অন-কল সপ্তাহের পর কী ঘটল, কী কঠিন ছিল, তা নিয়ে আলোচনা হয়। একটি ভালো লক্ষণ হলো অন-কলে নতুনরা স্বেচ্ছায় আসতে চায় — কারণ তাঁরা জানেন তাঁদের সাহায্য করা হবে, আর ভুল করলে দোষ দেওয়া হবে না।

> **WARNING:** অন-কল যদি শুধু এক বা দুজন "বিশেষজ্ঞের" উপর নির্ভর করে, তবে সেই নির্ভরতাটিই সবচেয়ে বড় ঝুঁকি। তাঁরা ছুটিতে গেলে বা চলে গেলে জ্ঞানও চলে যায়। জ্ঞান ছড়িয়ে দেওয়া — Runbook, ছায়া-দায়িত্ব, জোড়ায় কাজ — অন-কলেরই অংশ, কোনো অতিরিক্ত কাজ নয়।

২৩.৭ Incident Command ও পরিবর্তন ব্যবস্থাপনা

Incident Command (অধ্যায় ১৯-এ পরিচিত) মানে Incident-এর সময় একজন সমন্বয়কারী — যিনি সিদ্ধান্ত নেন, কাজ ভাগ করেন, আর যোগাযোগের একটিই কণ্ঠস্বর রাখেন। Commander-এর ভূমিকাটি ঘূর্ণায়মান হওয়া উচিত — শুধু সবচেয়ে অভিজ্ঞ ব্যক্তি নন, প্রশিক্ষণপ্রাপ্ত যে কেউ। Incident-এর পর **Blameless Postmortem** হয় — দোষ নয়, ব্যবস্থাগত কারণ খোঁজা হয়, আর প্রতিটি Action Item-এর মালিক ও সময়সীমা থাকে।

Change Management মানে প্রোডাকশনে পরিবর্তনের নিয়ন্ত্রণ: কী বদলাচ্ছে, কেন, ঝুঁকি কী, ফিরে যাওয়ার পথ কী, কে অনুমোদন দিলেন। ভালো Change Management গতিকে থামায় না — এটি ঝুঁকি অনুযায়ী স্তরভিত্তিক হয়: ছোট, প্রত্যাবর্তনযোগ্য পরিবর্তন স্বয়ংক্রিয়ভাবে যায়; বড়, ঝুঁকিপূর্ণ পরিবর্তনে পর্যালোচনা ও অনুমোদন লাগে। পরিবর্তনের রেকর্ড (Git ইতিহাস, পাইপলাইন লগ) থাকলে Incident-এর সময় "কী বদলেছে?" প্রশ্নের উত্তর দ্রুত মেলে — আর এই প্রশ্নটিই Incident নির্ণয়ের সবচেয়ে দরকারি প্রশ্নগুলোর একটি।

২৩.৮ প্রোডাকশন প্রস্তুতি (Production Readiness)

একটি সার্ভিস প্রোডাকশনে যাওয়ার আগে যাচাই-তালিকা (Checklist) থাকা উচিত — **Production Readiness Review**। তালিকায় থাকে: পর্যবেক্ষণ আছে কি (SLI, ড্যাশবোর্ড, সতর্কতা)? Runbook আছে কি? ব্যাকআপ ও পুনরুদ্ধার পরীক্ষিত কি? নিরাপত্তা পরীক্ষা হয়েছে কি (স্ক্যান, অনুমতি)? ধারণক্ষমতা পরিকল্পনা আছে কি? রোলব্যাকের পথ পরীক্ষিত কি? অন-কলের মালিক কে? এই তালিকাটি একটি আনুষ্ঠানিকতা নয় — এটি একটি কথোপকথন: সার্ভিসের মালিক ও প্ল্যাটফর্ম দল একসাথে বসে দেখেন কী বাকি আছে। তালিকার প্রতিটি আইটেমের একটি মালিক ও সময়সীমা থাকা উচিত — নইলে তালিকাটি শুধু আশা হয়ে থাকে।

প্রস্তুতির বিষয়	যাচাই	না থাকলে কী হয়
পর্যবেক্ষণ	SLI, ড্যাশবোর্ড, সতর্কতা আছে কি?	সমস্যা ধরা পড়ে ব্যবহারকারীর অভিযোগে
Runbook	সাধারণ সমস্যার Runbook পরীক্ষিত কি?	Incident-এ সময় নষ্ট হয়, জ্ঞান একজনের মাথায়
ব্যাকআপ/DR	পুনরুদ্ধার পরীক্ষিত কি?	বিপর্যয়ে ডেটা বা সেবা ফেরে না
নিরাপত্তা	স্ক্যান, অনুমতি, সিক্রেট পরীক্ষিত কি?	দুর্বলতা প্রোডাকশনে যায়
রোলব্যাক	ফিরে যাওয়ার পথ পরীক্ষিত কি?	খারাপ রিলিজ আটকে থাকে
মালিকানা	অন-কলের মালিক ও যোগাযোগ জানা কি?	Incident-এ কেউ দায়িত্ব নেয় না

২৩.৯ জ্ঞান ভাগাভাগি, প্রযুক্তিগত ঋণ ও প্ল্যাটফর্ম পরিপক্কতা

Knowledge Sharing মানে জ্ঞানকে একজনের মাথা থেকে দলের সম্পদে পরিণত করা: জোড়ায় কাজ (Pairing), ছায়া-দায়িত্ব, অভ্যন্তরীণ আলোচনা (Tech Talk), লিখিত নথি, আর Incident পর্যালোচনায় শেখা ভাগ করা। একটি দরকারি মাপ হলো **Bus Factor** — কতজন মানুষ চলে গেলে একটি সিস্টেম অচল হয়ে যাবে? Bus Factor ১ মানে একজন চলে গেলেই বিপদ — আর তা একটি সতর্ক সংকেত, দোষ নয়।

Technical Debt মানে দ্রুত কাজ করার জন্য নেওয়া ছাড়: একটি অস্থায়ী সমাধান, একটি বাদ দেওয়া পরীক্ষা, একটি পুরোনো লাইব্রেরি। ঋণ নিজে খারাপ নয় — কখনো কখনো ঋণ নেওয়া যুক্তিযুক্ত (যেমন সময়মতো বাজারে যেতে)। খারাপ হলো ঋণের হিসাব না রাখা। ঋণের তালিকা রাখুন: কী ঋণ, কেন নেওয়া হয়েছিল, এর খরচ কী (ধীর গতি, ঝুঁকি), আর কখন শোধ করা হবে। প্রতিটি পরিকল্পনা চক্রে (Sprint/Quarter) ঋণ শোধের জন্য সময় বরাদ্দ রাখুন — নইলে ঋণ জমতে জমতে একসময় নতুন ফিচার বানানোই অসম্ভব হয়ে যায়।

Platform Maturity মানে প্ল্যাটফর্মটি কতটা পরিপক্ব: প্রাথমিক স্তরে সবকিছু হাতে করা হয়; পরের স্তরে অটোমেশন আছে কিন্তু ছড়িয়ে আছে; তারপর প্ল্যাটফর্ম একীভূত হয়; আর সর্বোচ্চ স্তরে প্ল্যাটফর্ম নিজে মাপা হয় (DX মেট্রিক) ও নিয়মিত উন্নত হয়। পরিপক্কতা একটি গন্তব্য নয় — একটি দিক: প্রতিটি স্তরে পরের স্তরের একটি করে উন্নতি বেছে নিন, সবকিছু একসাথে নয়।

> **REAL WORLD:** TechNova-তে একটি গুরুত্বপূর্ণ সিস্টেমের Bus Factor ছিল ১ — শুধু একজন প্রকৌশলী তা বুঝতেন। তিনি ছুটিতে গেলে একটি ছোট সমস্যা সমাধানে দুই দিন লেগেছিল। এরপর দলটি সিদ্ধান্ত নিল: প্রতিটি গুরুত্বপূর্ণ সিস্টেমের অন্তত দুজন মালিক থাকবে, আর Runbook লেখা হবে। ছয় মাস পর Bus Factor ১-এর সিস্টেম আর একটিও ছিল না — আর ছুটির ভয়ও কমেছিল।

২৩.১০ প্রকৌশল যোগাযোগ: লেখা, বলা ও শোনা

প্রকৌশল যোগাযোগের তিনটি রূপ: **লেখা** — নথি, ADR, Incident রিপোর্ট, রিলিজ নোট — লেখা যোগাযোগের সুবিধা হলো তা থাকে, খোঁজা যায়, আর সময়ের সাথে টিকে থাকে; **বলা** — সভা, আলোচনা, Incident-এর সময় যোগাযোগ — বলার সুবিধা হলো গতি ও প্রসঙ্গ; আর **শোনা** — অন্যের সমস্যা ও প্রতিক্রিয়া শোনা — শোনার অভাবেই অনেক প্ল্যাটফর্ম ব্যর্থ হয়, কারণ প্ল্যাটফর্ম দল দলগুলোর আসল সমস্যাটি শোনে না। ভালো যোগাযোগের নিয়ম: লিখিত সিদ্ধান্ত লিখিতভাবে জানান; Incident-এর সময় নিয়মিত, সংক্ষিপ্ত আপডেট দিন (“আমরা কী জানি, কী করছি, পরের আপডেট কখন”); আর প্রতিক্রিয়া চাওয়ার পর তা কাজে লাগান — নইলে মানুষ প্রতিক্রিয়া দেওয়া বন্ধ করে দেয়।

২৩.১১ক নেতৃত্বের দৈনন্দিন অভ্যাস

প্রকৌশল নেতৃত্ব কোনো এককালীন সিদ্ধান্ত নয় — এটি দৈনন্দিন অভ্যাসের সমষ্টি। ভালো অভ্যাসগুলো: প্রতিটি সপ্তাহে একটি করে সিদ্ধান্ত লিখে রাখা (ADR — ছোট হলেও); প্রতিটি Incident-এর পর Action Item-গুলোর অগ্রগতি দেখা — শেষ না হওয়া পর্যন্ত; প্রতিটি নতুন প্রকৌশলীর প্রথম ডেপ্লয়ের অভিজ্ঞতা শোনা — কোথায় আটকে গেলেন, তা জিজ্ঞেস করা; প্রতিটি ত্রৈমাসিকে ঋণের তালিকা পর্যালোচনা করা — কোন ঋণটি এখন সবচেয়ে বেশি খরচ করছে; আর প্রতিটি পর্যালোচনায় একটি করে প্রশ্ন করা — "আমরা কী শিখলাম, আর কী বদলাব?" এই অভ্যাসগুলো ছোট মনে হয়, কিন্তু এক বছরে এগুলোই দলের সংস্কৃতি তৈরি করে। নেতৃত্বের সবচেয়ে বড় পরীক্ষাটি হলো আপনি ছুটিতে গেলে দলটি কেমন চলে — ভালো চললে আপনি ব্যবস্থা বানিয়েছেন; খারাপ চললে আপনি নিজেই ব্যবস্থা হয়ে গেছেন — আর সেটি টেকসই নয়।

২৩.১১ TechNova প্রোডাকশন দৃশ্যকল্প

TechNova-র একটি বড় সিদ্ধান্ত আসছে: ডেটাবেস বদলানো হবে কি না। দলটি কাঠামো অনুসরণ করল: সমস্যাটি লেখা হলো (বর্তমান ডেটাবেসে একটি নির্দিষ্ট ফিচারের জন্য খরচ বেশি), তিনটি বিকল্প তালিকা করা হলো (বর্তমানটি অপ্টিমাইজ করা, নতুন ডেটাবেসে যাওয়া, কিছু না করা), মানদণ্ড ঠিক করা হলো (খরচ, ঝুঁকি, দলের দক্ষতা, স্থানান্তরের সময়), আর সিদ্ধান্তটি ADR-এ লেখা হলো — "এখন কিছু না করা, তিন মাস পর আবার দেখা" — কারণ স্থানান্তরের খরচ বর্তমান সমস্যার চেয়ে বেশি ছিল। তিন মাস পর আবার দেখা হলো — আর তখন সিদ্ধান্তটি বদলানো হলো, প্রমাণসহ। এই গল্পটি দেখায় সিদ্ধান্তের কাঠামো কীভাবে "কিছু না করা"-কেও একটি বৈধ, লিখিত সিদ্ধান্ত করে।

২৩.১২ নিরাপত্তা ও খরচ বিবেচনা

নেতৃত্বের সিদ্ধান্তগুলোতেও নিরাপত্তা ও খরচের ভারসাম্য থাকে: অন-কলের চাপ কমাতে বেশি মানুষ নিয়োগ করা খরচ বাড়ায়, কিন্তু Burnout-এর খরচ — মানুষ চলে যাওয়া, ভুল বেড়ে যাওয়া — তার চেয়ে বেশি হতে পারে। প্রযুক্তিগত ঋণ শোধ না করলে নিরাপত্তা ঝুঁকি বাড়ে — পুরোনো লাইব্রেরিতে দুর্বলতা থাকে। আর জ্ঞান না ছড়ালে নিরাপত্তা ঝুঁকি বাড়ে — একজন চলে গেলে সিক্রেট বা অনুমতির হিসাব হারিয়ে যায়। এই খরচগুলো বিলে দেখা যায় না, কিন্তু বাস্তব — তাই সিদ্ধান্তের সময় সেগুলো লিখিতভাবে বিবেচনা করুন।

২৩.১৩ সাধারণ ভুল

- **সিদ্ধান্ত না লেখা:** সিদ্ধান্ত মানুষের স্মৃতিতে থাকলে ভবিষ্যতে কেউ জানে না কেন তা নেওয়া হয়েছিল।
- **সব সিদ্ধান্তকে সমান গুরুত্ব দেওয়া:** ছোট সিদ্ধান্তে বেশি সময়, বড় সিদ্ধান্তে কম সময় — উল্টো হওয়া উচিত।
- **Runbook না পরীক্ষা করা:** অপরীক্ষিত Runbook Incident-এর সময় কাজ করে না।
- **অন-কল একজনের উপর চাপানো:** Bus Factor ১ — টেকসই নয়, ঝুঁকিপূর্ণ।
- **Blame-ভিত্তিক পর্যালোচনা:** দোষারোপ করলে মানুষ সত্য লুকায়।
- **ঋণের হিসাব না রাখা:** ঋণ জমতে থাকলে একসময় গতি থেমে যায়।

২৩.১৪ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সিদ্ধান্ত নিতে সপ্তাহ লাগে	সব সিদ্ধান্তকে সমান গুরুত্ব দেওয়া হচ্ছে	প্রত্যাবর্তনযোগ্যতা দিয়ে ভাগ করুন; ছোট সিদ্ধান্ত দ্রুত নিন

Incident-এ কেউ দায়িত্ব নেয় না	Incident Commander ও মালিকানা পরিষ্কার নয়	Commander ভূমিকা ঘূর্ণায়মান করুন; মালিকানা নথিতে লিখুন
অন-কলে মানুষ ক্লান্ত	সতর্কতা বেশি; রোটেশন নেই; বিশ্রাম নেই	সতর্কতা পর্যালোচনা করুন; রোটেশন ও বিশ্রাম ঠিক করুন
নতুন ফিচার বানাতে দিন লাগে	প্রযুক্তিগত ঋণ জমে গেছে	ঋণের তালিকা করুন; প্রতিটি চক্রে শোধের সময় রাখুন
একজন চলে গেলে জ্ঞান চলে যায়	জ্ঞান ভাগ করা হয় না; Bus Factor ১	Runbook, ছায়া-দায়িত্ব, জোড়ায় কাজ চালু করুন

২৩.১৫ প্রকৌশল ট্রেড-অফ: প্রক্রিয়া বনাম গতি

প্রক্রিয়া ও গতি — দুটির মধ্যে টানাপোড়েন চিরকাল থাকবে। বেশি প্রক্রিয়া মানে ধীর গতি, কিন্তু কম ভুল; কম প্রক্রিয়া মানে দ্রুত গতি, কিন্তু বেশি ঝুঁকি। সঠিক ভারসাম্যটি ঝুঁকি দিয়ে ঠিক হয়: প্রোডাকশনে যাওয়া পরিবর্তনে প্রক্রিয়া বেশি, পরীক্ষামূলক কাজে প্রক্রিয়া কম। আর প্রক্রিয়াটি যদি মানুষের কাজ বাড়ায়, তবে তা অটোমেশন দিয়ে সহজ করুন — যেমন অনুমোদন গেট পাইপলাইনে, নথি Pull Request-এ। ভালো প্রক্রিয়ার চিহ্ন হলো মানুষ তা মানে কারণ তা সাহায্য করে, বাধা দেয় বলে নয়।

TROUBLESHOOTING: দল ধীর হয়ে গেলে প্রথমে জিজ্ঞেস করুন — “কোন প্রক্রিয়াটি আসলে সাহায্য করছে, আর কোনটি শুধু বাধা?” উত্তরটি প্রায়ই দেখায় একটি প্রক্রিয়া সরানো বা স্বয়ংক্রিয় করাই সবচেয়ে বড় গতির উন্নতি।

২৩.১৬ অধ্যায়ের সারাংশ

প্রকৌশল নেতৃত্ব মানে সিদ্ধান্তের কাঠামো, লিখিত সিদ্ধান্ত (ADR), ভালো নথি ও পরীক্ষিত Runbook, টেকসই অন-কল, সুশৃঙ্খল Incident Command, ঝুঁকি-ভিত্তিক পরিবর্তন ব্যবস্থাপনা, আর Production Readiness যাচাই। জ্ঞান ভাগাভাগি Bus Factor বাড়ায়, প্রযুক্তিগত ঋণের হিসাব গতি ধরে রাখে, আর প্ল্যাটফর্ম পরিপক্কতা একটি দিক — গন্তব্য নয়। যোগাযোগ — লেখা, বলা ও শোনা — এই সবকিছুর ভিত্তি।

২৩.১৭ জ্ঞান যাচাই

1. ভালো প্রযুক্তিগত সিদ্ধান্তের কাঠামোটি ব্যাখ্যা করুন। “কিছু না করা” বিকল্পটি কেন গুরুত্বপূর্ণ?
2. প্রত্যাবর্তনযোগ্যতা (Reversibility) দিয়ে সিদ্ধান্তের গতি কীভাবে ঠিক করবেন?
3. Runbook-এ কী কী থাকা উচিত? অপরীক্ষিত Runbook কেন বিপজ্জনক?
4. টেকসই অন-কলের ছয়টি নিয়ম লিখুন।
5. Incident Commander-এর ভূমিকা কী? এটি ঘূর্ণায়মান হওয়া কেন উচিত?
6. Change Management কী? ঝুঁকি-ভিত্তিক স্তরবিন্যাস ব্যাখ্যা করুন।
7. Production Readiness যাচাই-তালিকায় কী কী থাকা উচিত?
8. Bus Factor কী? এটি ১ হলে কী করবেন?
9. Technical Debt কী? ঋণের হিসাব না রাখলে কী হয়?
10. প্রক্রিয়া বনাম গতি — ভারসাম্যটি কীভাবে ঠিক করবেন?

২৩.১৮ ব্যবহারিক অনুশীলন

1. TechNova-র একটি বাস্তবসম্মত সিদ্ধান্তের জন্য ADR লিখুন: সমস্যা, তিনটি বিকল্প (একটি “কিছু না করা”), মানদণ্ড, সিদ্ধান্ত ও পুনর্বিবেচনার সময় — সম্পূর্ণ নথি তৈরি করুন।
2. একটি Runbook লিখুন ও পরীক্ষা করুন: একটি সাধারণ সমস্যা (যেমন ডিস্ক পূর্ণ) নিয়ে লক্ষণ, ধাপ, যাচাই ও ফিরে যাওয়ার পথ লিখুন; তারপর একজন সহকর্মীকে শুধু Runbook পড়ে সমস্যাটি সামলাতে বলুন (বা সামলানোর ধাপগুলো লিখুন) ও কী শিখলেন তা নথিভুক্ত করুন।
3. একটি অন-কল রোটেশন পরিকল্পনা বানান: কারা থাকবেন, কতদিন করে, বিশ্রাম কীভাবে হবে, আর নতুনরা কীভাবে প্রস্তুত হবেন — তা লিখুন।
4. TechNova-র একটি প্রযুক্তিগত ঋণের তালিকা বানান (কাল্পনিক): প্রতিটি ঋণের কারণ, খরচ, আর শোধের অগ্রাধিকার — একটি সারণিতে লিখুন।

অধ্যায় ২৪ • PART V — Production Architecture & Final Capstone

Chapter 24 — Final Enterprise DevOps Platform Capstone

২৪.১ শেখার উদ্দেশ্য

এই শেষ অধ্যায়ে আপনি পুরো ভলিউমের সবকিছু একসাথে জুড়ে TechNova-র জন্য একটি সম্পূর্ণ এন্টারপ্রাইজ DevOps প্ল্যাটফর্ম নকশা করবেন — প্রয়োজনীয়তা (Requirements) থেকে আর্কিটেকচার, অবকাঠামো, Kubernetes, GitOps, Internal Developer Platform, CI/CD, নিরাপত্তা, পর্যবেক্ষণ, SRE, Disaster Recovery, FinOps, প্রোডাকশন ডেপ্লয়মেন্ট, আর শেষ প্রকৌশল পর্যালোচনা (Final Engineering Review) পর্যন্ত। এই অধ্যায় শেষে আপনি পারবেন — একটি সম্পূর্ণ প্ল্যাটফর্মের প্রয়োজনীয়তা লিখতে; তার আর্কিটেকচার আঁকতে; প্রতিটি স্তরের সিদ্ধান্ত যুক্তিসহ ব্যাখ্যা করতে; আর একটি প্রোডাকশন-প্রস্তুত পর্যালোচনা চালাতে।

- TechNova-র প্ল্যাটফর্মের প্রয়োজনীয়তা কার্যকরী ও অ-কার্যকরী ভাগে লেখা।
- সম্পূর্ণ প্ল্যাটফর্ম আর্কিটেকচার আঁকা ও প্রতিটি সিদ্ধান্তের যুক্তি দেওয়া।
- GitOps, IDP, CI/CD, নিরাপত্তা, পর্যবেক্ষণ, DR ও FinOps একসাথে জুড়ে দেওয়া।
- একটি Final Engineering Review চালানো ও উন্নতির তালিকা তৈরি করা।

২৪.২ পূর্বশর্ত জ্ঞান — ঝালিয়ে নেওয়া

এই অধ্যায়টি পুরো ভলিউমের সমাপ্তি — এখানে আগের সব অধ্যায়ের ধারণা একসাথে আসে। সংক্ষেপে মনে রাখুন: অধ্যায় ১৬ — উন্নত CI/CD, Blue-Green, Canary, Feature Flag, Progressive Delivery; অধ্যায় ১৭ — Platform as Code, Terraform মডিউল, Ansible, Crossplane, Operator, প্ল্যাটফর্ম API, Drift; অধ্যায় ১৮ — মাল্টি-ক্লাউড, পরিচয় ফেডারেশন, Egress খরচ; অধ্যায় ১৯ — Observability, SLI/SLO/Error Budget, Incident Response; অধ্যায় ২০ — Zero Trust, Policy as Code, SBOM, স্বাক্ষর, Kubernetes নিরাপত্তা; অধ্যায় ২১ — প্ল্যাটফর্ম আর্কিটেকচার, সীমানা, DX, ADR; অধ্যায় ২২ — ধারণক্ষমতা, FinOps, RPO/RTO, DR; অধ্যায় ২৩ — সিদ্ধান্ত, Runbook, অন-কল, প্রোডাকশন প্রস্তুতি। এই অধ্যায়ে আমরা এই সবগুলোকে একটি জীবন্ত প্ল্যাটফর্মে জুড়ব।

> **CONCEPT:** ক্যাপস্টোন মানে সবকিছু একসাথে বানানো নয় — মানে সবকিছু একসাথে ভাবা। একটি প্ল্যাটফর্মের প্রতিটি সিদ্ধান্ত অন্য সিদ্ধান্তকে ছোঁয়: CI/CD-র সিদ্ধান্ত নিরাপত্তাকে ছোঁয়, নিরাপত্তার সিদ্ধান্ত ডেভেলপার অভিজ্ঞতাকে ছোঁয়, খরচের সিদ্ধান্ত DR-কে ছোঁয়। ক্যাপস্টোনের কাজ হলো এই সংযোগগুলো দেখা — আর একটি সুসংগত, যুক্তিসংগত সম্পূর্ণ তৈরি করা।

২৪.৩ প্রয়োজনীয়তা (Requirements): TechNova কী চায়

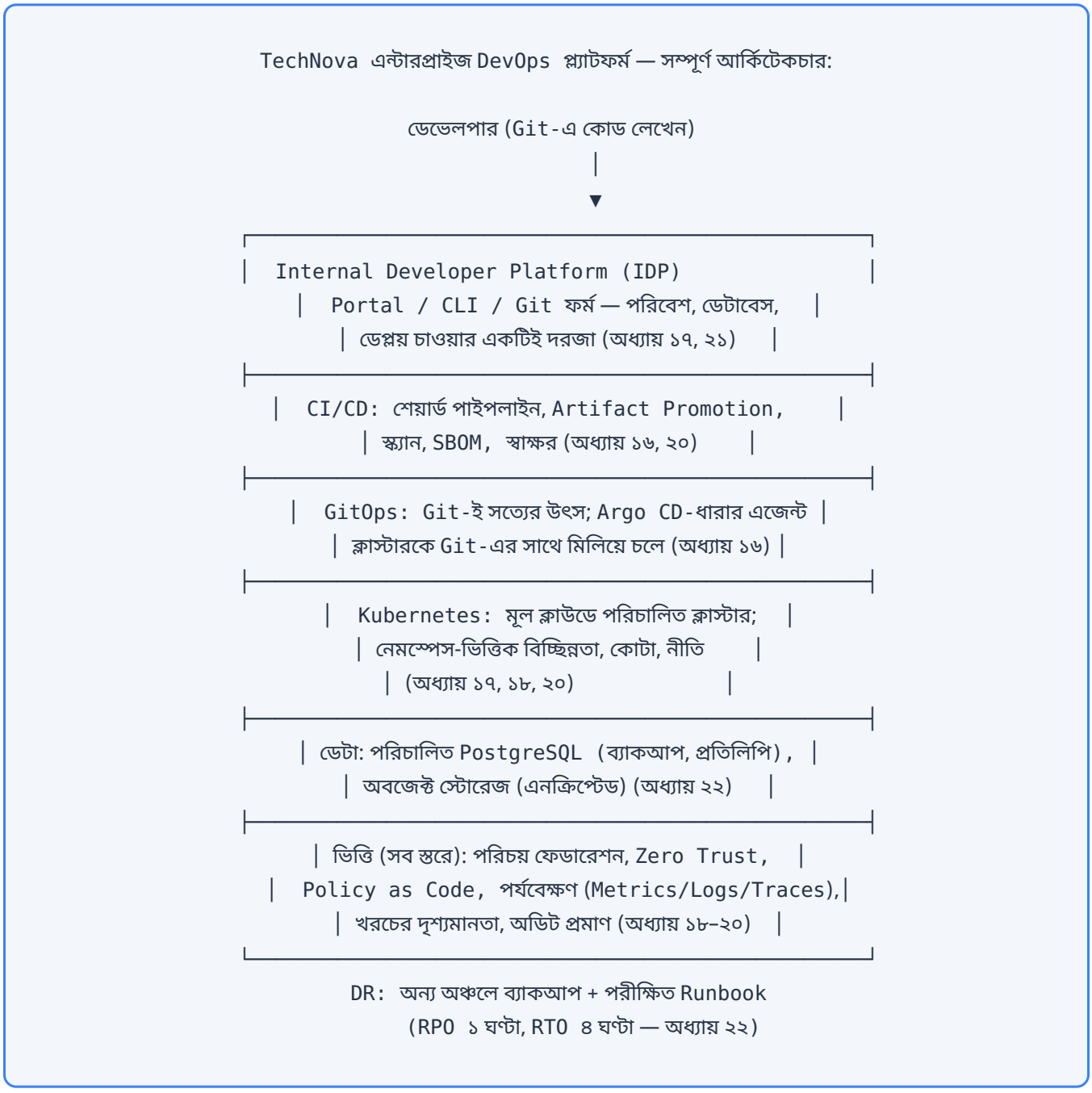
TechNova একটি কাল্পনিক সফটওয়্যার কোম্পানি — ডোমেইন `technova.example`, পাঁচটি প্রোডাক্ট দল, web অ্যাপ, API ও PostgreSQL ডেটাবেস। প্ল্যাটফর্মের প্রয়োজনীয়তাগুলো দুই ভাগে ভাগ করা যায়। **কার্যকরী প্রয়োজনীয়তা (Functional)** — প্ল্যাটফর্মটি কী করবে: দলগুলোকে পরিবেশ দেওয়া, ডেটাবেস দেওয়া, CI/CD পাইপলাইন দেওয়া, ডেপ্লয় করা, পর্যবেক্ষণ দেওয়া, সিক্রেট দেওয়া। **অ-কার্যকরী প্রয়োজনীয়তা (Non-Functional)** — প্ল্যাটফর্মটি কেমন হবে:

নিরাপদ (Zero Trust, নীতি-ভিত্তিক), নির্ভরযোগ্য (SLO-ভিত্তিক), দ্রুত (প্রথম ডেপ্লয় এক দিনে), খরচ-সচেতন (FinOps), আর ব্যবহারযোগ্য (DX মাপা হয়)।

প্রয়োজনীয়তা	লক্ষ্য (TechNova)	যেখানে বিস্তারিত আছে
পরিবেশ পাওয়া	Pull Request-এ, এক ঘণ্টার মধ্যে	অধ্যায় ১৭, ২১
প্রথম ডেপ্লয়	নতুন প্রকৌশলী এক দিনে	অধ্যায় ২১, ২৩
রিলিজ নিরাপত্তা	Canary + স্বয়ংক্রিয় রোলব্যাক	অধ্যায় ১৬
প্রাপ্যতা SLO	৯৯.৯% সফল অনুরোধ (৩০ দিন)	অধ্যায় ১৯, ২২
DR	RPO ১ ঘণ্টা, RTO ৪ ঘণ্টা	অধ্যায় ২২
নিরাপত্তা	SBOM + স্বাক্ষর, নীতি-ভিত্তিক ডেপ্লয়	অধ্যায় ২০
খরচ	ট্যাগ-ভিত্তিক দৃশ্যমানতা, মাসিক পর্যালোচনা	অধ্যায় ২২
মালিকানা	প্রতিটি সিস্টেমের দুজন মালিক (Bus Factor ≥ ২)	অধ্যায় ২৩

এই সারণিটি ক্যাপস্টোনের চুক্তি — প্রতিটি সিদ্ধান্ত এই প্রয়োজনীয়তাগুলোর বিপরীতে যাচাই করা হবে। প্রয়োজনীয়তা ছাড়া আর্কিটেকচার শুধু পছন্দের তালিকা; প্রয়োজনীয়তাসহ আর্কিটেকচার একটি যুক্তিসংগত নকশা।

২৪.৪ আর্কিটেকচার: সম্পূর্ণ ছবি



এই আর্কিটেকচারের মূল সিদ্ধান্তগুলো: একটি মূল ক্লাউড (গভীরতা ও সরলতার জন্য — অধ্যায় ১৮-এর যুক্তি), Kubernetes-কেন্দ্রিক রানটাইম (বহনযোগ্যতার দরজা খোলা রেখে), GitOps (পরিবর্তনের ইতিহাস ও রোলব্যাক Git-এ), আর IDP (দলগুলোর জন্য একটি দরজা)। প্রতিটি সিদ্ধান্তের পেছনে একটি ADR থাকা উচিত — কী, কেন, কী বিকল্প ছিল, কখন পুনর্বিবেচনা হবে।

২৪.৫ অবকাঠামো ও Kubernetes

অবকাঠামো Platform as Code দিয়ে বানানো হয় (অধ্যায় ১৭): Terraform মডিউল — নেটওয়ার্ক, ক্লাস্টার, ডেটাবেস — সংস্করণে আটকে, Remote State-এ, Drift Detection-সহ। নেটওয়ার্ক ভাগ করা: পাবলিক, অ্যাপ ও ডেটা স্তর আলাদা; ডেটাবেস শুধু অ্যাপের সাথে কথা বলে। Kubernetes ক্লাস্টারে: প্রতিটি দলের জন্য নেমস্পেস, Resource Quota (একটি দল সব সম্পদ খেয়ে না ফেলে), Network Policy (শুধু প্রয়োজনীয় পথ খোলা), Pod Security (Root নয়, শুধু প্রয়োজনীয়

অনুমতি), আর RBAC (ভূমিকা অনুযায়ী অনুমতি — কেউ `cluster-admin` নয়)। ক্লাস্টারের আপগ্রেড ধাপে ধাপে হয় — প্রথমে Staging, তারপর প্রোডাকশন — আর প্রতিটি আপগ্রেডের আগে ব্যাকআপ ও রোলব্যাক পরিকল্পনা থাকে।

```
# TechNova প্ল্যাটফর্ম — অবকাঠামোর স্তর (ধারণা)
# environments/prod/
#   network    — মডিউল: VPC, সাবনেট, রাউট (v2.3.0-তে আটকে)
#   cluster    — মডিউল: পরিচালিত Kubernetes + নোড গ্রুপ
#   database   — মডিউল: PostgreSQL + ব্যাকআপ + এনক্রিপশন
#   observability — মেট্রিক, লগ, ট্রেস সংগ্রহ
# প্রতিটি স্তরের Apply-এ অনুমোদন গেট; Drift নিয়মিত পরীক্ষা হয়।
```

২৪.৬ GitOps ও Internal Developer Platform (IDP)

GitOps-এ Git-ই কাঙ্ক্ষিত অবস্থার উৎস: অ্যাপের সংজ্ঞা, পরিবেশের কনফিগারেশন, নীতি — সব Git-এ। ক্লাস্টারের ভেতরের এজেন্ট ক্রমাগত Git-এর সাথে বাস্তব অবস্থা মিলিয়ে চলে; কেউ হাতে কিছু বদলালে তা ফিরিয়ে দেওয়া হয় (বা সতর্ক করা হয়)। রোলব্যাক মানে Git-এ আগের কমিটে ফেরা — একটি পরিচিত, পরীক্ষিত পথ। IDP সেই GitOps-এর উপরে দলগুলোর জন্য একটি সহজ দরজা: দল একটি ফর্মে (YAML) লেখে "আমার একটি পরিবেশ চাই, ডেটাবেসসহ, দুটি রেপ্লিকা" — Pull Request মার্জ হলে প্ল্যাটফর্ম তা বানিয়ে দেয়, আর ঠিকানা ও শংসাপত্রের পথ ফেরত দেয়। IDP-র সাফল্যের মাপ হলো DX মেট্রিক: Time to Environment, Time to First Deploy — আর দলগুলোর প্রতিক্রিয়া।

২৪.৭ CI/CD, নিরাপত্তা ও পর্যবেক্ষণ — একসাথে

CI/CD পাইপলাইনটি (অধ্যায় ১৬) এখন নিরাপত্তা ও পর্যবেক্ষণের সাথে জোড়া: প্রতিটি কমিটে বিল্ড, পরীক্ষা, নির্ভরতা স্ক্যান, ইমেজ স্ক্যান, SBOM তৈরি, স্বাক্ষর — তারপর Artifact Promotion (একই ইমেজ dev → staging → prod)। প্রোডাকশনে ডেপ্লয় হয় GitOps-এর মাধ্যমে, Canary-তে (Argo Rollouts/Flagger-ধারার ধাপ), মেট্রিক-ভিত্তিক বিশ্লেষণ ও স্বয়ংক্রিয় রোলব্যাকসহ। Feature Flag দিয়ে ডেপ্লয় ও রিলিজ আলাদা করা হয়। নিরাপত্তা (অধ্যায় ২০) পাইপলাইনে বোনা: নীতি-পরীক্ষা ছাড়া ডেপ্লয় হয় না, স্বাক্ষর ছাড়া ইমেজ চলে না, সিক্রেট ভল্ট থেকে আসে। পর্যবেক্ষণ (অধ্যায় ১৯) প্রতিটি স্তরে: অ্যাপের SLI/SLO, প্ল্যাটফর্মের মেট্রিক, আর Error Budget-ভিত্তিক রিলিজ সিদ্ধান্ত — বাজেট শেষ হলে রিলিজ থামে, নির্ভরযোগ্যতার কাজ চলে।

রিলিজ প্রবাহ — সবকিছু একসাথে (TechNova):

কমিট → বিল্ড + পরীক্ষা → স্ক্যান + SBOM + স্বাক্ষর
→ Artifact Promotion (dev → staging → prod)
→ GitOps ডেপ্লয় (Canary: 5% → 25% → 50% → 100%)
→ মেট্রিক বিশ্লেষণ (ত্রুটি, লেটেন্সি)
→ ভালো হলে সম্পূর্ণ; খারাপ হলে স্বয়ংক্রিয় রোলব্যাক
→ Feature Flag দিয়ে ধাপে ধাপে রিলিজ
→ Error Budget দেখে পরের রিলিজের সিদ্ধান্ত

২৪.৮ SRE, Disaster Recovery ও FinOps

SRE চর্চা (অধ্যায় ১৯) প্ল্যাটফর্মে বোনা: প্রতিটি সার্ভিসের SLI/SLO, Error Budget নীতি, সতর্কতা (শুধু কাজের সতর্কতা), Incident Command, Blameless Postmortem, আর Toil কমানোর নিয়ম। DR (অধ্যায় ২২): RPO ১ ঘণ্টা, RTO ৪ ঘণ্টা — ডেটাবেস ব্যাকআপ প্রতি ঘণ্টায়, অন্য অঞ্চলে সংরক্ষিত, পুনরুদ্ধারের Runbook পরীক্ষিত, আর নিয়মিত Game Day। FinOps (অধ্যায় ২২): প্রতিটি সম্পদে ট্যাগ (দল, পরিবেশ, মালিক), মাসিক খরচ পর্যালোচনা, অব্যবহৃত সম্পদ সরানো, Staging-এর সময়সূচি, আর খরচের তথ্য দিয়ে প্রকৌশল সিদ্ধান্ত। এই তিনটি — SRE, DR, FinOps — কোনো আলাদা দলের কাজ নয়; এগুলো প্ল্যাটফর্মের পরিচালনারই অংশ।

> **REAL WORLD:** TechNova-র ক্যাপস্টোন পরিকল্পনাটি একদিনে বাস্তবায়িত হয়নি। প্রথমে একটি দলের সাথে Pilot — পরিবেশ, পাইপলাইন ও ড্যাশবোর্ড — তারপর শিক্ষা নিয়ে বাকি দলগুলোতে প্রসার, প্রতিটি দলের জন্য একটি করে সপ্তাহ। প্রতিটি ধাপে DX মেট্রিক মাপা হয়েছিল — আর যে ধাপে মেট্রিক খারাপ ছিল, সেই ধাপটি থামিয়ে ঠিক করা হয়েছিল। প্ল্যাটফর্ম বানানো একটি প্রকল্প নয় — এটি একটি যাত্রা।

২৪.৯ প্রোডাকশন ডেপ্লয়মেন্ট: ধাপে ধাপে পরিকল্পনা

প্রোডাকশনে যাওয়ার পরিকল্পনাটি ধাপে ধাপে: **ধাপ ১ — ভিত্তি:** পরিচয়, নেটওয়ার্ক, ক্লাস্টার, পর্যবেক্ষণ — প্ল্যাটফর্মের ভিত্তি, Staging-এ পরীক্ষিত। **ধাপ ২ — Pilot:** একটি দল, একটি সার্ভিস — সম্পূর্ণ প্রবাহ (পরিবেশ থেকে রিলিজ পর্যন্ত) পরীক্ষা। **ধাপ ৩ — প্রসার:** বাকি দলগুলো, একটি করে, প্রতিক্রিয়া শুনে। **ধাপ ৪ — কঠোরতা:** নিরাপত্তা নীতি Enforce মোডে, DR পরীক্ষা, Game Day। **ধাপ ৫ — পরিচালনা:** অন-কল, পর্যালোচনা চক্র, DX মাপা, ঋণের তালিকা। প্রতিটি ধাপের একটি সাফল্যের মাপ ও একটি ফিরে যাওয়ার শর্ত থাকে — কোনো ধাপে মাপ খারাপ হলে পরের ধাপে যাওয়া হয় না। এই শৃঙ্খলাই প্রোডাকশন ডেপ্লয়মেন্টকে জুয়া থেকে প্রকৌশলে পরিণত করে।

```
# প্রোডাকশন যাওয়ার যাচাই-তালিকা (সংক্ষেপ — অধ্যায় ২৩-এর তালিকা দেখুন)
# [ ] SLI/SLO, ড্যাশবোর্ড, সতর্কতা — প্রতিটি সার্ভিসে
# [ ] Runbook পরীক্ষিত — সাধারণ সমস্যাগুলোর জন্য
# [ ] ব্যাকআপ থেকে পুনরুদ্ধার পরীক্ষিত (RTO-র মধ্যে)
# [ ] নিরাপত্তা: স্ক্যান, SBOM, স্বাক্ষর, অনুমতি পর্যালোচনা
# [ ] রোলব্যাক পরীক্ষিত (Canary + Feature Flag)
# [ ] অন-কলের মালিক ও যোগাযোগ জানা; Bus Factor >= 2
# [ ] খরচের ট্যাগ ও মাসিক পর্যালোচনার মালিক ঠিক
```

২৪.১০ Final Engineering Review: প্ল্যাটফর্মের পরীক্ষা

Final Engineering Review মানে প্ল্যাটফর্মটিকে প্রয়োজনীয়তার বিপরীতে যাচাই করা — একটি আনুষ্ঠানিক, লিখিত পর্যালোচনা। পর্যালোচনার প্রশ্নগুলো: প্রতিটি প্রয়োজনীয়তা কি পূরণ হয়েছে, প্রমাণসহ? কোন সিদ্ধান্তের যুক্তি দুর্বল? কোথায় একক ব্যর্থতার বিন্দু (Single Point of Failure) আছে? Bus Factor ১-এর জায়গা কোথায়? DR পরীক্ষার ফল কী? Error Budget কেমন চলছে? খরচ কি প্রত্যাশা অনুযায়ী? ডেভেলপাররা কী বলেন (DX মেট্রিক ও প্রতিক্রিয়া)? পর্যালোচনার ফলাফল একটি উন্নতির তালিকা — প্রতিটি আইটেমের মালিক, অগ্রাধিকার ও সময়সীমাসহ। পর্যালোচনাটি একটি এককালীন ঘটনা নয় — এটি নিয়মিত চক্রের শুরু: প্রতি ত্রৈমাসিকে প্ল্যাটফর্ম পর্যালোচনা, আর প্রতিটি পর্যালোচনায় পরের ত্রৈমাসিকের উন্নতির তালিকা।

পর্যালোচনার বিষয়	প্রমাণ	TechNova-র অবস্থা (উদাহরণ)
পরিবেশ পাওয়া	Pull Request থেকে পরিবেশ পর্যন্ত সময়ের রেকর্ড	এক ঘণ্টার মধ্যে — লক্ষ্য পূরণ
রিলিজ নিরাপত্তা	Canary রিপোর্ট, রোলব্যাক পরীক্ষার রেকর্ড	পরীক্ষিত — লক্ষ্য পূরণ
SLO	গত তিন মাসের SLI ড্যাশবোর্ড	৯৯.৯% পূরণ হচ্ছে
DR	পুনরুদ্ধার পরীক্ষার সময়	৩ ঘণ্টা — RTO ৪ ঘণ্টার মধ্যে
নিরাপত্তা	স্ক্যান, SBOM, নীতি-পরীক্ষার ফল	স্বাক্ষর ছাড়া ইমেজ বন্ধ — চালু
খরচ	ট্যাগ-ভিত্তিক বিল পর্যালোচনা	মাসিক পর্যালোচনা চলছে
মালিকানা	Bus Factor পর্যালোচনা	সব সিস্টেমে ≥ ২ — লক্ষ্য পূরণ

WARNING: Final Review-এ “সব ঠিক আছে” বলা সবচেয়ে বিপজ্জনক ফলাফল — কারণ কোনো প্ল্যাটফর্মই নিখুঁত নয়। পর্যালোচনার মূল্য তার উন্নতির তালিকায়: যে পর্যালোচনায় কোনো উন্নতির আইটেম বের হয় না, সেই পর্যালোচনাটি যথেষ্ট গভীরে যায়নি।

২৪.১১ নিরাপত্তা, খরচ ও ট্রেড-অফের চূড়ান্ত ভারসাম্য

ক্যাপস্টোনের প্রতিটি সিদ্ধান্তে তিনটি ট্রেড-অফ মনে রাখুন — যা পুরো ভলিউম জুড়ে এসেছে। **একক ক্লাউড বনাম মাল্টি-ক্লাউড:** TechNova একটি মূল ক্লাউডে গভীরভাবে গেছে, বহনযোগ্যতার দরজা খোলা রেখে — কারণ ব্যবসায়িক প্রয়োজন এখনও দ্বিতীয় ক্লাউড দাবি করে না। **সার্ভিস মেশ বনাম সরাসরি যোগাযোগ:** সার্ভিস মেশ (Service Mesh) mTLS, ট্রাফিক নিয়ন্ত্রণ ও পর্যবেক্ষণ দেয় — কিন্তু জটিলতা ও পরিচালনার খরচ বাড়ায়। TechNova-র সিদ্ধান্ত: শুরুতে সরাসরি যোগাযোগ + Network Policy, আর যখন সার্ভিসের সংখ্যা ও নিরাপত্তার প্রয়োজন বাড়বে, তখন সার্ভিস মেশ বিবেচনা — সহজ পথটি যখন কাজ করে, তখন জটিল পথে যাওয়া নয়। **GitOps বনাম প্রচলিত CI/CD:** TechNova GitOps বেছে নিয়েছে — কারণ একাধিক দল, অডিটের প্রয়োজন, আর রোলব্যাকের জন্য Git-এর ইতিহাস — এই তিনটি GitOps-এর পক্ষে। ছোট দলে প্রচলিত CI/CD যথেষ্ট হতো — কিন্তু TechNova আর ছোট নয়। এই তিনটি সিদ্ধান্তই ADR-এ লেখা — যাতে ভবিষ্যতে প্রসঙ্গ বদলালে সিদ্ধান্তগুলো পুনর্বিবেচনা করা যায়।

২৪.১২ সাধারণ ভুল (ক্যাপস্টোনে)

- **সবকিছু একসাথে বানানো:** Pilot ছাড়া পুরো প্ল্যাটফর্ম চালু করলে ভুলটি সব জায়গায় ছড়ায়।
- **প্রয়োজনীয়তা না লেখা:** প্রয়োজনীয়তা ছাড়া আর্কিটেকচার শুধু পছন্দের তালিকা।
- **DX না মাপা:** ডেভেলপারদের অভিজ্ঞতা না মাপলে প্ল্যাটফর্মের সাফল্য অনুমানে হয়।
- **DR পরীক্ষা না করা:** পরীক্ষা ছাড়া DR শুধু নথি।
- **জটিল পথ প্রথমে বেছে নেওয়া:** সার্ভিস মেশ বা মাল্টি-ক্লাউড — যখন সহজ পথ কাজ করে, তখন জটিল পথে যাওয়া।
- **Final Review-কে সমাপ্তি ভাবা:** পর্যালোচনা একটি চক্রের শুরু, শেষ নয়।

২৪.১৩ সমস্যা সমাধান নির্দেশিকা (ক্যাপস্টোনে)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
প্ল্যাটফর্ম ব্যবহার হচ্ছে না	DX খারাপ; দলগুলোর প্রয়োজন মেটে না	দলগুলোর যাত্রা পর্যবেক্ষণ করুন; সহজ পথ বানান
প্রতিটি সিদ্ধান্তে বিতর্ক	প্রয়োজনীয়তা ও ADR লেখা নেই	প্রয়োজনীয়তা লিখুন; সিদ্ধান্ত ADR-এ লিখুন
Incident-এ প্ল্যাটফর্মই ভাঙে	শেয়ার্ড ভিত্তিতে বিচ্ছিন্নতা নেই	নেমস্পেস, কোটা, নীতি দিয়ে বিচ্ছিন্নতা বাড়ান
খরচ প্রত্যাশার চেয়ে বেশি	ট্যাগ নেই; অব্যবহৃত সম্পদ	FinOps পর্যালোচনা করুন; অব্যবহৃত সম্পদ সরান
DR পরীক্ষায় RTO ছাড়িয়ে যায়	Runbook-এ হাতে করা ধাপ	Runbook স্বয়ংক্রিয় করুন; আবার পরীক্ষা করুন

২৪.১৪ ভলিউমের সমাপ্তি: আপনার পরের ধাপ

এই ভলিউমটি শেষ হলো, কিন্তু প্ল্যাটফর্ম প্রকৌশলের যাত্রা শেষ হয়নি — এটি একটি চলমান চর্চা। আপনার পরের ধাপগুলো: (১) এই অধ্যায়ের ক্যাপস্টোন নকশাটি আপনার নিজের প্রসঙ্গে (বা একটি কাল্পনিক কোম্পানিতে) প্রয়োগ করে দেখুন — প্রয়োজনীয়তা লিখুন, আর্কিটেকচার আঁকুন, ADR লিখুন। (২) একটি ছোট Pilot বানান — একটি সার্ভিস, একটি দল, সম্পূর্ণ প্রবাহ। (৩) DX মাপুন — Time to First Deploy, Time to Environment — আর উন্নতির তালিকা বানান। (৪) নিয়মিত পর্যালোচনা চালু করুন — প্রতি ত্রৈমাসিকে প্ল্যাটফর্ম পর্যালোচনা, DR পরীক্ষা, আর ঋণের তালিকা। মনে রাখবেন: ভালো প্ল্যাটফর্মের চিহ্ন হলো দলগুলো স্বেচ্ছায় তা ব্যবহার করে, রিলিজ নিরাপদ হয়, আর বিপর্যয়ের সময় সিস্টেম ফিরে আসে — কারণ এই সবকিছু আগে থেকে পরীক্ষা করা হয়েছে। এই ভলিউমের সব অধ্যায়ের শিক্ষা একটিই বাক্যে: সহজ পথটি যখন কাজ করে, তখন জটিল পথে যাবেন না; কিন্তু যখন জটিলতা সত্যিই দরকার, তখন তা সচেতনভাবে, লিখিতভাবে, আর পরীক্ষাসহ বেছে নিন।

> **TROUBLESHOOTING:** প্ল্যাটফর্মের যেকোনো সমস্যায় ফিরে যান প্রয়োজনীয়তায়: “আমরা কী সমস্যা সমাধান করতে চেয়েছিলাম, আর এই সমাধানটি কি সেই সমস্যার উত্তর দিচ্ছে?” উত্তরটি যদি “না” হয়, তবে সমাধানটি নয়, সমস্যাটি আবার দেখুন।

২৪.১৫ অধ্যায়ের সারাংশ

Final Capstone TechNova-র জন্য একটি সম্পূর্ণ এন্টারপ্রাইজ DevOps প্ল্যাটফর্ম জুড়ে দেয়: প্রয়োজনীয়তা (কার্যকরী ও অ-কার্যকরী), রেফারেন্স আর্কিটেকচার (IDP, CI/CD, GitOps, Kubernetes, ডেটা ও ভিত্তি স্তর), Platform as Code অবকাঠামো, GitOps ও IDP, নিরাপত্তা-বোনা CI/CD, SRE, DR (RPO ১ ঘণ্টা, RTO ৪ ঘণ্টা), FinOps, ধাপে ধাপে প্রোডাকশন ডেপ্লয়মেন্ট, আর Final Engineering Review। চূড়ান্ত ভারসাম্যগুলো — একক বনাম মাল্টি-ক্লাউড, সার্ভিস মেশ বনাম সরাসরি যোগাযোগ, GitOps বনাম প্রচলিত CI/CD — সচেতনভাবে, ADR-এ লিখে, সহজ পথটি প্রথমে বেছে নিয়ে ঠিক করা হয়েছে। প্ল্যাটফর্ম বানানো একটি প্রকল্প নয় — একটি চলমান যাত্রা, নিয়মিত পর্যালোচনাসহ।

২৪.১৬ জ্ঞান যাচাই

1. কার্যকরী ও অ-কার্যকরী প্রয়োজনীয়তার পার্থক্য ব্যাখ্যা করুন — TechNova-র উদাহরণ দিয়ে।
2. TechNova প্ল্যাটফর্ম আর্কিটেকচারের স্তরগুলো আঁকুন ও প্রতিটি স্তরের দায়িত্ব লিখুন।
3. GitOps কীভাবে রোলব্যাককে সহজ করে? IDP-র সাথে এর সম্পর্ক কী?

4. ক্যাপস্টোনে CI/CD, নিরাপত্তা ও পর্যবেক্ষণ কীভাবে একসাথে জোড়া হয়েছে? ব্যাখ্যা করুন।
5. RPO ১ ঘণ্টা ও RTO ৪ ঘণ্টা — এই সংখ্যাগুলো DR কৌশলকে কীভাবে ঠিক করে?
6. Final Engineering Review-এ কোন প্রশ্নগুলো করা উচিত? প্রমাণ বলতে কী বোঝায়?
7. একক ক্লাউড বনাম মাল্টি-ক্লাউড — TechNova কেন একটি মূল ক্লাউড বেছে নিয়েছে? যুক্তি দিন।
8. সার্ভিস মেশ বনাম সরাসরি যোগাযোগ — কখন কোনটি? TechNova-র সিদ্ধান্তটি ব্যাখ্যা করুন।
9. প্রোডাকশন ডেপ্লয়মেন্টের পাঁচটি ধাপ লিখুন ও প্রতিটি ধাপের সাফল্যের মাপ বলুন।
10. এই ভলিউমের সবচেয়ে গুরুত্বপূর্ণ তিনটি শিক্ষা আপনার ভাষায় লিখুন — আর কেন সেগুলো গুরুত্বপূর্ণ, তা ব্যাখ্যা করুন।

২৪.১৭ ব্যবহারিক অনুশীলন

1. আপনার নিজের প্রসঙ্গে (বা একটি কাল্পনিক কোম্পানিতে) একটি প্ল্যাটফর্ম প্রয়োজনীয়তা-সারণি লিখুন: পাঁচটি কার্যকরী ও পাঁচটি অ-কার্যকরী প্রয়োজনীয়তা, প্রতিটির লক্ষ্যসহ।
2. TechNova ক্যাপস্টোন আর্কিটেকচারটি আঁকুন (হাতে বা টুলে): প্রতিটি স্তর, প্রতিটি সংযোগ, আর প্রতিটি স্তরের দায়িত্ব লেবেল করুন। তারপর তিনটি ADR লিখুন — একটি করে একক বনাম মাল্টি-ক্লাউড, সার্ভিস মেশ, আর GitOps সিদ্ধান্তের জন্য।
3. একটি Pilot পরিকল্পনা লিখুন: একটি দল, একটি সার্ভিস, সম্পূর্ণ প্রবাহ (পরিবেশ থেকে রিলিজ পর্যন্ত), সাফল্যের মাপ, আর প্রসারের শর্ত — সম্পূর্ণ নথি তৈরি করুন।
4. একটি Final Engineering Review চালান (কাল্পনিক প্ল্যাটফর্মের উপর): প্রতিটি বিষয়ে প্রমাণ লিখুন, তিনটি উন্নতির আইটেম বের করুন (মালিক ও সময়সীমাসহ), আর পরের ত্রৈমাসিকের পর্যালোচনার তারিখ ঠিক করুন।
5. পুরো ভলিউমের একটি এক-পাতার সারাংশ লিখুন: প্রতিটি অধ্যায়ের একটি করে মূল শিক্ষা, আর কীভাবে সেগুলো একসাথে জুড়ে একটি প্ল্যাটফর্ম হয় — তা আপনার ভাষায় লিখুন।